

计算机组成原理与操作系统 第一册

从门电路、CPU、总线、DMA、特权级到内核抽象

CPU Books

本册是组成原理与操作系统合并理论卷。前半部从数字电路、CPU、内存、总线、DMA、中断和特权级讲起，再进入内核对象、调度、虚拟内存、I/O、文件系统、网络入口和恢复；完整模拟器内核和代码工程放入实践卷。

前言

操作系统不是一组系统调用名，也不是“进程、内存、文件、网络”这些名词的并列清单。它是一套把硬件资源变成可控抽象的工程协议：CPU 时间被包装成可调度实体，物理内存被包装成带权限的地址空间，设备被包装成文件和事件，崩溃风险被包装成日志、提交点和恢复规则。

因此，本版把计算机组成原理和操作系统放在同一条主线上讲。前半部不会假设读者已经理解 CPU、DMA、MMU、cache、TLB、总线、特权级和中断控制器，而是先搭一个可手算的 x86-64 教学子集：从 bit、byte、word、内存数组、RIP、寄存器、指令编码、取指译码执行、栈、中断、定时器、特权级、系统调用和分页开始，一步一步解释这些硬件为什么出现。只有看清硬件怎样执行指令、保存状态、翻译地址、投递中断和隔离权限，后面的系统调用、进程、虚拟内存、驱动和文件系统才不是空概念。

本册从最简单的硬件和单片机裸机程序开始。先看一位数据怎样被电路保存，一条指令怎样经过取指、译码、执行和访存，再看一个 `while(true)` 主循环怎样直接拥有整台机器；任务变多后为什么需要任务表和状态机；事件变多后为什么需要中断；任务可能死循环后为什么需要定时器和抢占；多个任务共享数据后为什么需要锁、条件变量和 IPC；任务互相破坏内存后为什么需要页表和地址空间；用户不能直接碰硬件后为什么需要系统调用；设备不应由 CPU 搬每个字节后为什么需要 DMA、IOMMU 和 completion；数据写一半会崩溃后为什么需要日志、`fsync`、`rename` 和 `manifest`。

本册采用 CPU 项目中的写法：每个机制都从失败场景进入，再给出原理、不变量、实践观察和测试口径。读者不应该只记住“进程是资源分配单位，线程是调度单位”这样的短句，而要能回答：状态在哪里，谁能改变状态，错误路径怎样退出，哪些证据证明系统确实按这个规则运行。

本册暂时只写理论卷，不展开实践卷工程。正文中的“实践”指纸上状态机、命令观察、实验设计和验收报告口径；完整 C++ 实现、模拟器上的内核和更大规模实验，放到后续实践卷再看。

核心思想

操作系统的每个特性都应该能回答三个问题：它依赖哪个硬件事实，它修复了前一阶段的哪个失败，又引入了哪些新的状态、成本和测试责任。

缩写与术语表

本表只收录本册反复出现、且读者读源码时会立刻遇到的缩写。缩写不是背诵目标；它们的价值是帮助你把教材概念映射到真实内核对象和代码路径。

缩写	全称或常见含义	本册语境
ABI	Application Binary Interface	系统调用参数、调用约定、用户栈、ELF 装载契约
ACPI	Advanced Configuration and Power Interface	固件向内核描述 CPU、NUMA、设备和电源信息
ALU	Arithmetic Logic Unit	CPU 数据通路中执行整数算术和逻辑运算的部件
APIC	Advanced Programmable Interrupt Controller	x86 中断投递、本地定时器、IPI 和多核启动
PCID	Process Context ID	x86-64 TLB 中区分地址空间的标签，减少切换时全局刷新
BIO	Block I/O	块层从文件系统接收的页片段读写意图
BPF	Berkeley Packet Filter	seccomp、eBPF tracing、网络和安全钩子的执行载体
CFS	Completely Fair Scheduler	Linux 公平调度类，核心变量是 <code>vruntime</code>
CFI	Control Flow Integrity	控制流保护，限制间接跳转和返回路径被劫持
CPL	Current Privilege Level	x86-64 当前特权级，决定是否可执行特权操作
COW	Copy-on-Write	<code>fork</code> 、 <code>MAP_PRIVATE</code> 和 COW 文件系统的共享后复制语义
CR/MSR	Control Register / Model-Specific Register	x86-64 控制寄存器和模型专用寄存器，保存页表根、系统调用入口和特权控制状态
DMA	Direct Memory Access	设备直接读写内存，要求映射、缓存一致性和生命周期管理
EOI	End Of Interrupt	中断处理后通知控制器可以投递后续中断
FUA	Force Unit Access	块设备写入要求到达稳定介质，而非仅进入设备缓存
GDT	Global Descriptor Table	x86-64 早期保护和段描述结构，长模式下仍参与入口设置
GFP	Get Free Page flags	内核分配上下文，决定是否可睡眠、可 I/O、可回收
IDT	Interrupt Descriptor Table	x86-64 中断、异常和陷入入口表
IOMMU	I/O Memory Management Unit	设备 DMA 地址翻译和隔离
IPI	Inter-Processor Interrupt	CPU 间重调度、TLB shutdown、多核控制消息
IRQ	Interrupt Request	外设中断请求及其内核处理路径

ISA	Instruction Set Architecture	软件可见的指令、寄存器、异常、内存和特权契约
LSM	Linux Security Module	SELinux、AppArmor、BPF LSM 等安全钩子框架
MMU	Memory Management Unit	执行虚拟地址翻译和页表权限检查的硬件单元
MMIO	Memory-Mapped I/O	CPU 通过地址空间读写设备寄存器
NAPI	New API	Linux 网络收包中断转轮询机制，控制中断风暴
NUMA	Non-Uniform Memory Access	多插槽内存局部性、调度和页分配策略
OOM	Out Of Memory	内存耗尽、回收失败和 OOM killer 决策
PCIe	Peripheral Component Interconnect Express	现代外设互联，承载配置空间、BAR、MSI/MSI-X 和 DMA
PTE/PMD/PUD/PGD	Page Table Entry 等页表层级	多级页表、权限位、缺页和 TLB 失效
RCU	Read-Copy-Update	读路径低成本、写路径延迟释放的同步机制
RSS	Resident Set Size	进程实际驻留物理页，不等于虚拟地址空间大小
skb	socket buffer	Linux 网络栈中承载包数据和协议元数据的核心对象
SMP	Symmetric Multiprocessing	多核并发、per-CPU 数据、runqueue 和 IPI
TLB	Translation Lookaside Buffer	地址翻译缓存，页表修改后需失效旧项
VFS	Virtual File System	统一 — superblock、inode、dentry、file 的文件系统抽象
VMA	Virtual Memory Area	进程虚拟地址区间策略，用于缺页判断和权限控制
WAL	Write-Ahead Log	先写日志再提交数据的崩溃恢复协议

怎样阅读本册

本册必须按顺序阅读。全书真实架构统一采用 x86-64，汇编统一采用 Intel 语法：目的操作数在前，源操作数在后，例如 `mov rax, qword ptr [rbx]`、`add rax, 1`、`syscall`、`iretq`。第一部分先补齐组成原理底座：先用一个 x86-64 教学子集手算 bit、byte、word、内存、RIP、RSP、通用寄存器、指令、栈、中断、定时器、系统调用和分页，再扩展到数字电路、寄存器、状态机、存储程序 CPU、ISA、流水线、多核、内存、总线、MMIO、DMA、IOMMU、Ring 0/3、MMU 和 APIC/MSI-X，最后进入裸机主循环、中断、定时器、启动和内核初始化，解释 OS 的入口和抢占能力怎样长出来；第二部分进入任务、线程、调度器深入、同步、内核锁和死锁；第三部分处理特权级、系统调用、`exec`、ELF 装载、虚拟内存、`mmap`、page cache 和内核分配器；第四部分把设备驱动、DMA、块层、I/O 调度、文件、page cache、文件系统、socket 入口和持久化连接起来；最后把安全隔离、panic、错误恢复、观测和验收方法收束成工程报告能力。

学习每章时都要落到四个问题：

1. 状态在哪里：进程表、runqueue、锁等待队列、页表、VMA、fd 表、socket、inode、page cache、日志还是设备队列。
2. 硬件在哪里：RIP、RSP、RFLAGS、通用寄存器、cache、TLB、页表、MMIO 寄存器、DMA 描述符、APIC/MSI-X 还是定时器。
3. 权限在哪里：CPU 特权级、页表权限、文件权限、capability、namespace 还是对象引用。
4. 等待在哪里：runnable 等 CPU、blocked 等事件、mutex 等持有者、page fault 等页面、socket 等缓冲、I/O 等 completion。
5. 提交在哪里：`exec` 切换地址空间、状态更新完成、写入 page cache、块设备 flush、目录持久化还是 manifest 生效。

本册的“实践”不是把理论卷变成代码卷，而是要求每个机制能落成纸上状态机、观察命令、实验矩阵或验收报告。若一节讲 fd，就要画出 fd table、open file description、inode 和 page cache 的关系；若一节讲调度，就要写出 runnable、running、blocked、zombie 和 reaped 的状态转换；若一节讲锁，就要画出锁顺序图和等待队列；若一节讲 `exec`，就要标出失败回滚边界和提交点；若一节讲持久化，就要写出每个崩溃点后恢复程序应该相信什么。

阅读算法小节时，不要只看伪代码能不能运行。要追问四件事：数据结构是什么，复杂度是多少，维护了哪些不变量，失败路径是否会留下半提交状态。一个教学算法可以比真实内核小，但不能把真实问题讲错。

不要把工具输出当成结论。工具只是证据来源；结论必须说明哪条 OS 契约被验证，哪条仍然只是合理假设。

目录

前言	ii
缩写与术语表	iii
怎样阅读本册	v
第 1 章 硬件从零到 CPU、裸机演进与内核入口	1
1.1 从门电路到 x86-64 执行核心	1
1.2 存储层次、地址翻译与内存系统	78
1.3 总线、设备、DMA 与中断平台	105
1.4 从裸机启动到内核契约	168
第 2 章 任务、调度与并发控制	262
2.1 任务模型与上下文切换	262
2.2 调度算法与多核工程	276
2.3 同步、IPC 与等待状态机	289
2.4 并发证据、实验与验收	303
第 3 章 特权边界、程序装载与内存管理	312
3.1 系统调用、权限与边界复制	312
3.2 exec、ELF 与用户地址空间建立	330
3.3 虚拟内存、缺页与页缓存	336
3.4 内核分配器与 C++20 内存模型	352
第 4 章 设备、I/O 与文件系统	365
4.1 驱动、DMA 与设备队列	365
4.2 块层、I/O 调度与完成语义	370
4.3 VFS、文件对象与路径解析	375
4.4 文件系统持久化与故障测试	381
第 5 章 网络入口与持久化语义	394
5.1 socket、队列与协议入口	394
5.2 网络路径与背压	400
5.3 持久化、日志与恢复	406
第 6 章 安全、错误恢复与可观测性	411
6.1 隔离边界与能力收敛	411
6.2 panic、错误路径与恢复策略	416
6.3 可观测性和调试证据	421
6.4 安全与观测深讲	427
第 7 章 实践方法与验收	434
7.1 实践路线与验收标准	434
7.2 练习、案例和错误分析	436
7.3 C++20 参考实现阅读	446
能力清单	472
Linux 源码阅读索引	474

第 1 章 硬件从零到 CPU、裸机演进与内核入口

1.1 从门电路到 x86-64 执行核心

硬件对象的统一读法：结构、时序、接口和 OS 后果

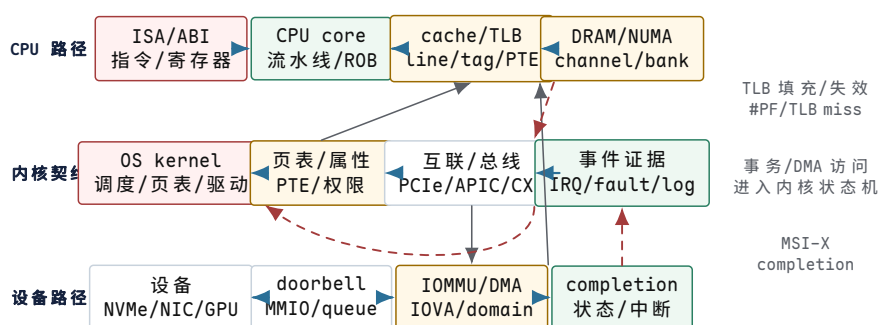
本书后面会反复出现 CPU、cache、TLB、DRAM、MMIO、DMA、IOMMU、APIC、NVMe、网卡、GPU、TPM、BMC 等硬件名词。只把它们列出来没有用。读硬件必须按一条固定路径拆：

下层物理结构 → 内部状态机 → 对上接口 → OS 契约 → 失败证据

下层物理结构回答“它由什么构成”。内部状态机回答“它什么时候改变状态”。对上接口回答“CPU、设备或内核怎样和它说话”。OS 契约回答“内核可以依赖什么、不可以假设什么”。失败证据回答“出错时硬件留下哪个寄存器、状态位、中断、错误码或日志”。

核心思想

不要问“cache 是什么”“DMA 是什么”这种孤立问题。要问：cache 的下层是 SRAM 和一致性互联，上层是 load/store 和内核数据结构；DMA 的下层是设备总线事务和内存控制器，上层是驱动描述符、IOMMU 映射和 completion。每个硬件对象都要这样贯通上下层。



图示：整机硬件不是平铺盒子，而是多条契约链。CPU load/store、页表、cache/TLB、DRAM、设备 DMA 和中断最终都要在 OS 中收敛成可验证状态。

每个硬件对象都要回答四组问题

问题	必须讲清楚的内容	没讲清会造成什么误解
结构	由哪些寄存器、阵列、队列、表项、线和状态位组成	把硬件当成黑盒函数，无法解释边界
时序	哪些路径是组合逻辑，哪些状态在时钟或事件后改变	误以为写了变量就等于全系统立刻看见
接口	上层通过指令、控制寄存器、MMIO、DMA、IRQ 还是固件表访问	混淆用户地址、物理地址、MMIO、IOVA 和内核对象
后果	正常路径、慢路径、错误路径分别怎样暴露给 OS	不知道为什么需要 shutdown、barrier、reset、fsync

例如 cache 要按这四组问题讲。结构上，它有 tag array、data array、valid/dirty/coherence state、replacement state；时序上，它要先查 index，再比较 tag，再选择 way，miss 后向下层发请求；接口上，上层看到的是 load/store 延迟和一致性，下层看到的是 line fill、writeback、invalidate；OS 后果是伪共享、锁竞争、DMA 同步、页属性和性能计数。

DMA 也一样。结构上，它有设备队列、描述符、DMA engine、IOMMU domain 和 completion ring；时序上，CPU 填 descriptor、屏障、doorbell，设备稍后 DMA 和写 completion；接口上，驱动通过 DMA API、MMIO 和 IRQ 管理它；OS 后果是 buffer 所有权、unmap 顺序、取消、超时、reset 和隔离。

硬件层次和 OS 对象的对照表

硬件对象	内部结构关键词	OS 必须建立的对象或规则
逻辑门/寄存器	门延迟、setup/hold、时钟边沿	精确状态、上下文保存、入口不可半提交
ALU/AGU	加法器、MUX、RFLAGS、地址生成	指针计算、条件分支、faulting address 证据
流水线/ROB	重命名、调度队列、按序提交	precise exception、可重试 page fault、信号语义
L1/L2/L3 cache	tag、set、way、line、MESI 状态	局部性、伪共享、锁设计、DMA cache 维护
TLB/page walker	VPN、PCID、PTE 权限、CR3	地址空间切换、TLB shutdown、COW、mprotect
DRAM/NUMA	channel、bank、row、ECC、node	页分配、迁移、回收、内存错误隔离
MMIO/PCIe BAR	地址窗口、posted write、状态寄存器	ioremap、访问顺序、驱动 probe/remove
DMA/IOMMU	descriptor、IOVA、domain、completion	pin/map/unmap、buffer 所有权、设备隔离
APIC/MSI-X	vector、priority、EOI、IPI、affinity	定时器、设备中断、跨核唤醒、shutdown
存储控制器	submission/completion queue、flush、FUA	块层、page cache、fsync、崩溃恢复

这张表也是后面阅读源码的索引。看到 `switch_mm`，要想到 CR3、PCID 和 TLB；看到 `dma_map_*`，要想到 IOVA、cache 维护和 IOMMU domain；看到 `spin_lock`，要想到 cache line ownership 和原子指令；看到 `fsync`，要想到 page cache、块层、设备缓存和稳定介质。

用公式减少口头描述

硬件结构适合用公式和不变量表达。后文会反复使用这些基础式子：

$$\text{cache line address} = \left\lfloor \frac{PA}{L} \right\rfloor, \quad \text{set} = \left(\left\lfloor \frac{PA}{L} \right\rfloor \bmod S \right)$$

$$\text{TLB reach} = N_{\text{entry}} \times \text{page size}$$

$$AMAT = T_{\text{hit}} + R_{\text{miss}} \times P_{\text{miss}}$$

$$T_{\text{clk}} \geq T_{\text{clk_to_q}} + T_{\text{logic}} + T_{\text{setup}} + T_{\text{skew}} + T_{\text{margin}}$$

DMA safe free $\Rightarrow$ completion observed $\wedge$ dma_unmap done $\wedge$ no device ownership

这些式子不是为了显得数学化，而是为了去掉含混句子。比如“cache 很快但 miss 很慢”不如写成 AMAT；“TLB 覆盖范围不够”不如写成 TLB reach；“不能提前释放 DMA buffer”不如写出安全释放不变量。

读硬件路径时先找边界

每条路径都要先找边界。边界是硬件和软件交接的地方，最容易出错，也最能解释 OS 设计。

边界	硬件交给 OS 的证据	OS 要做的事
缺页边界	faulting RIP、CR2、error code	查 VMA/PTE，分配页或发信号
系统调用边界	syscall number、ABI 参数寄存器	验证参数、复制用户数据、返回 errno/value
中断边界	vector、APIC 状态、设备 completion	ack/EOI、消费完成队列、唤醒等待者
DMA 边界	descriptor ownership、completion tag、IOMMU fault	map/unmap、回收 buffer、reset 或报错
持久化边界	block completion、flush completion、journal commit	决定 fsync 是否可返回成功
热插拔边界	bus remove event、设备错误状态	停止新请求、处理 in-flight、释放对象

后面的章节如果只给出名词，而没有说明边界、证据和不变量，就不算讲清楚。本章后续内容按这个标准展开。

硬件缺口补全：从结构到 OS 约束

前面已经给出从门电路到 CPU 的路线，但还不够。只讲“CPU、cache、DMA、中断、设备”这些名词，读者仍然不知道它们为什么必须这样接，为什么 OS 要保存某些状态，为什么驱动要按奇怪的顺序写寄存器，为什么同一条 `mov rax, qword ptr [rbx + 16]` 有时只是读内存，有时会变成缺页、MMIO 访问或设备错误。本节把这些缺口补上。

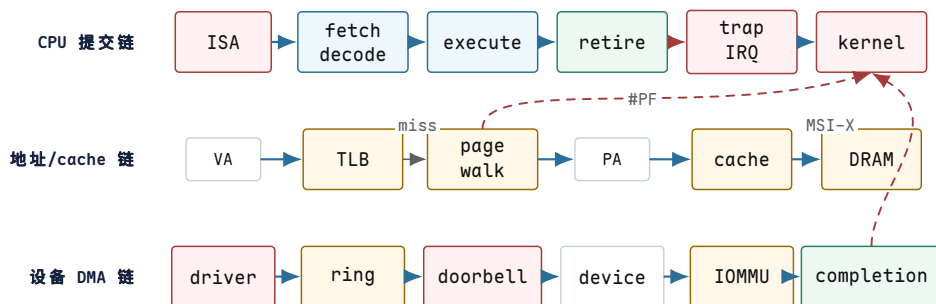
读每个硬件对象都用同一个模板：

物理/逻辑结构 → 内部状态 → 时序边界 → 对上接口 → OS 不变量 → 失败证据

如果一个解释没有说清内部状态和 OS 不变量，就还没有讲明白。例如“中断用于通知 CPU”不够，必须继续问：pending 位在哪里，mask 位怎样生效，in-service 位何时清，EOI 早了会怎样，设备状态没清会怎样，vector 如何绑定 CPU。“DMA 用于设备搬运内存”也不够，必须继续问：descriptor 谁写，doorbell 何时写，设备用什么地址，IOMMU 映射何时撤销，completion 前 buffer 能不能释放。

总图：硬件不是平面盒子，而是多条状态链

下面这张图把 OS 最常踩的硬件链路放在同一页。左侧是 CPU 内部提交链，右侧是内存翻译和 cache 链，下侧是设备 I/O 链。每条链都有自己的状态机；OS 做的事就是把这些状态机收敛成进程、页、文件、socket、request、completion 等软件对象。



图示：三条关键链：CPU 提交链、地址翻译与 cache 链、设备 DMA 链。OS 必须把它们的状态边界对齐。

时钟、复位和时钟域：状态为什么不能随便读

硬件状态由时钟推进。一个模块没有时钟，寄存器不会采样；一个模块还在复位，内部寄存器可能保持默认值；两个模块处于不同 clock domain，同一事件穿越边界时需要同步器或握手协议。OS 驱动里的“先开电源、再开时钟、再解除复位、再写寄存器”来自这些物理约束。

```

device_resume_order:
    enable parent power domain
    enable regulator
    enable functional clock
    wait clock stable or PLL locked
    deassert reset
    wait device ready bit
    program registers
    enable DMA and interrupts
  
```

如果顺序错了，错误表现常常不是立即崩溃，而是随机超时、寄存器读回全 1、DMA 不完成、中断不来。原因是 OS 以为自己在访问“寄存器变量”，实际访问的是一个还没进入 READY 状态的硬件状态机。

$$state_{device} = READY \Leftarrow power_good \wedge clock_stable \wedge reset = 0 \wedge init_done$$

对象	硬件事实	OS 约束
PLL/clock reset line	频率稳定需要 lock 时间 复位会清内部队列和寄存器	切频或启用后等待稳定证据 reset 后重建 BAR 寄存器、 队列和中断配置
clock gate	省电时关闭模块时钟	访问 MMIO 前必须 runtime resume
power domain	断电后状态可能丢失	suspend 前 停 止 DMA， resume 后重初始化
watchdog	独立时钟检测系统卡死	内核要定期喂狗，panic/死 锁时保留证据

电源管理、热管理、设备 reset 不是外围章节。它们决定“寄存器是否有意义、DMA 是否还会发生、中断是否还能投递”。这也是为什么驱动不能只写 happy path。

x86-64 指令从字节到提交：一条 add 的完整链路

以 `add rax, rbx` 为例。汇编文本先由汇编器编码成字节；CPU 用 RIP 从 I-cache 取字节；译码器识别操作、源寄存器和目的寄存器；寄存器文件读出旧 `rax/rbx`；ALU 做加法；结果先进入内部结果路径或重排序缓冲，最后在提交边界更新软件可见的 `rax` 和 `RFLAGS`。

```
add rax, rbx:
```

```

bytes = fetch(RIP)
uop = decode(bytes)
src0 = read_arch_or_renamed(rax)
src1 = read_arch_or_renamed(rbx)
tmp = integer_adder(src0, src1)
flags = compute_add_flags(src0, src1, tmp)
retire in program order:
    rax = tmp
    RFLAGS = flags
    RIP = RIP + instruction_length

```

关键点是“执行”和“提交”不同。乱序 CPU 可以提前执行后面的微操作，但 OS 能依赖的是提交后的架构状态。异常入口保存的也是提交边界上的状态，而不是 CPU 内部临时物理寄存器。

$$architectural\ state_{t+1} = retire(architectural\ state_t, committed\ instruction)$$

这条式子解释上下文切换的范围：OS 保存通用寄存器、RIP、RSP、RFLAGS、FPU/SIMD 等架构状态；不保存 ROB、调度队列、物理寄存器池、分支预测器每个内部表项。后者影响性能和侧信道，前者决定程序语义。

一条 load 为什么会牵动 MMU、cache 和异常

再看 `mov rax, qword ptr [rbx + 16]`。它表面是一次内存读，内部至少有地址生成、canonical 检查、TLB 查找、PTE 权限检查、cache 查找、可能的 page walk、可能的 page fault、最后写回寄存器。

```

mov rax, qword ptr [rbx + 16]:
    va = rbx + 16
    check canonical form
    if DTLB misses:
        pte = hardware_page_walk(CR3, va)
        if pte invalid or permission denied:
            raise #PF with CR2=va and error_code
        insert translation into DTLB
    pa = translate(va)
    line = data_cache_lookup(pa)
    if miss:
        fill line from lower cache or memory
    rax = select_qword(line, pa.offset)

```

把它写成时序链：

$$\begin{aligned}
 VA &= R_{base} + disp, \\
 VPN &= \lfloor VA/P \rfloor, \quad offset = VA \bmod P, \\
 PTE &= walk(CR3, VPN), \\
 allow &= present \wedge permission(PTE, CPL, access), \\
 PA &= PTE.PPN \times P + offset.
 \end{aligned}$$

若 $allow = false$ ，硬件不能把这条 load 当作完成。它要产生精确异常：

$$\#PF = \{CR2 = VA, RIP = RIP_{faulting}, error_code, CPL\}$$

这就是虚拟内存的根。OS 不是在任意时刻“发现内存不够”，而是在硬件给出 faulting address、错误码和 faulting RIP 后，查 VMA、分配页、装 PTE、刷新必要 TLB，然后让同一条指令重试。没有精确异常，按需分页和 COW 都不成立。

MMU 不只是翻译地址，也是在执行权限逻辑

页表项不是单纯的“虚拟页到物理页映射”。它同时编码 `present`、`writable`、`user/supervisor`、`execute disable`、`accessed`、`dirty`、`global`、`cache attribute` 等信息。硬件 `page walk` 读取 PTE 后，做的是权限判定。

$$\begin{aligned} allow(access) &= present \\ &\quad \wedge (CPL = 0 \vee user = 1) \\ &\quad \wedge (access \neq write \vee writable = 1) \\ &\quad \wedge (access \neq execute \vee NX = 0) \end{aligned}$$

PTE 位	硬件含义	OS 用途
<code>present</code>	翻译是否有效	懒分配、 <code>swap</code> 、非法地址判断
<code>writable</code>	写是否允许	<code>COW</code> 、 <code>mprotect</code> 、只读代码段
<code>user</code>	Ring 3 是否可访问	用户/内核隔离
<code>NX</code>	是否禁止取指执行	<code>W^X</code> 、栈不可执行、安全隔离
<code>accessed/dirty</code>	硬件记录访问/写入痕迹	页面回收、 <code>writeback</code> 、老化算法
<code>global/PCID</code>	控制 TLB 保留范围	降低地址空间切换成本
<code>cache attr</code>	控制内存类型	<code>RAM</code> 、 <code>MMIO</code> 、 <code>write-combining</code> 、 <code>PMem</code>

因此“特权”不是口号。Ring 3 不能访问内核页，是因为 PTE 的 `user` 位、CPL 检查和异常入口共同工作。内核不能把用户指针当可信地址，也是因为当前 CPL 和访问路径会决定 `fault` 语义。系统调用里 `copy_from_user` 的价值，就是把“可能 `fault` 的用户访问”放到可恢复边界内。

TLB shutdown 的精确条件

页表在内存里，TLB 在 CPU 里。修改 PTE 后，旧翻译可能仍在一个或多个 CPU 的 TLB 中。OS 不能只改页表就释放物理页。

$$reuse(PA) \Rightarrow \forall cpu \in users(mm) : old_translation(cpu, VA \rightarrow PA) = false$$

安全 `unmap` 顺序是：

```
safe_unmap(mm, va):
  acquire page table lock
  clear or restrict PTE
  record affected CPUs for this mm
  release page table lock
  invalidate local TLB entry
  send shutdown IPI to remote CPUs in cpumask
  wait for acknowledgements
  now old physical page may be freed or reused
```

如果省略等待，旧 CPU 可能继续用旧 TLB 写这个物理页；而该页也许已经被分配给另一个进程、内核 `slab` 或 `page cache`。结果不是普通崩溃，而是越权写和随机数据破坏。这就是 `shutdown` 贵但不能省的原因。

内存控制器、DRAM bank 和 NUMA：物理页也有位置

DRAM 不是平面数组。地址会被内存控制器拆到 `channel`、`rank`、`bank`、`row`、`column`。一次访问可能经历 `row activate`、`column access`、`precharge`、`refresh` 和队列等待。多 `socket` 或多内存控制器系统还会形成 NUMA：同一个物理页离不同 CPU 的距离不同。

$$T_{dram} = T_{queue} + T_{cmd} + T_{row} + T_{col} + T_{transfer}$$

$$BW_{max} \approx channels \times transfers/s \times bytes/transfer$$

硬件结构	带来的现象	OS 设计后果
channel bank/row buffer refresh	并行内存通路 行命中比行冲突快 DRAM 周期性恢复电荷	内存带宽和插条位置相关 顺序访问、页着色和访问模式 会影响延迟 极端延迟抖动和可靠性测试要考虑
ECC	检测/纠正位错误	MCE、EDAC、页下线、DIMM 诊断
NUMA node	CPU 到内存距离不同	first-touch、迁移、亲和性、远端访问成本

调度器和页分配器为什么要知道 NUMA？因为把线程调到远端 CPU，或者把热页放在远端内存，会让每次 cache miss 多走更长互联路径。大机器上的性能不是单靠算法复杂度决定，也由“线程在哪个 CPU、页在哪个 node、设备 DMA 到哪个 node”决定。

cache 一致性只覆盖普通内存，不自动覆盖所有设备语义

多核 CPU 的一致性协议让普通 write-back RAM 的 cache line 在 CPU 间保持一致解释。但这不等于所有观察者都按程序顺序看到所有写入，也不等于设备寄存器可当普通变量访问。

coherence: same address order *ordering*: different address visibility order

三个常见误解必须拆开：

误解	错在哪里	正确边界
cache 一致性等于屏障	一致性多约束同一 cache line	发布对象、DMA descriptor、锁释放还要内存序
store 返回等于设备看见 MMIO 是内存变量	store buffer 和互联可能延迟可见性 设备寄存器有副作用和访问宽度要求	doorbell 前用 DMA/MMIO 屏障表达顺序 用 <code>readl/writel</code> 等 API 和页属性

设备发布请求的基本不变量是：

$$descriptor\ fields\ visible \prec_{device} doorbell\ visible$$

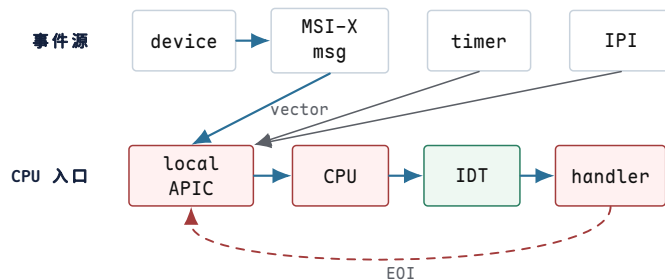
完成回收的基本不变量是：

$$free(buffer) \Rightarrow completion\ observed \wedge dma_unmap\ done \wedge owner = CPU$$

这两个式子比“加一个屏障就好了”更有解释力。屏障只是在具体架构和总线上实现这些可见性关系的工具。

APIC、vector 和中断入口：谁把事件交给哪个 CPU

x86-64 上中断不是“设备调用内核函数”。设备或控制器产生事件，APIC 体系把事件映射成 vector，CPU 在可接收边界进入 IDT 指定入口。local APIC 管本 CPU 的 timer、IPI、EOI 和 vector 接收；IOAPIC 管传统外部 IRQ 重定向；MSI/MSI-X 让 PCIe 设备写一条消息触发 vector。



图示：中断投递链。设备事件先变成 vector，再由 CPU 查 IDT 进入入口；handler 完成后还要满足设备确认和 APIC EOI。

中断控制器至少维护三类状态：

```

interrupt_state:
    pending[source]    // event has arrived
    masked[source]     // software blocks delivery
    in_service[vector] // CPU accepted but not EOied
  
```

顺序错误会产生具体故障：

错误顺序	现象	原因
未清设备状态就 EOI	中断风暴	APIC 可再投递，设备条件仍为真
清设备状态但忘 EOI	后续中断不来	vector 仍 in-service
过早开中断	handler 重入破坏共享状态	内核不变量还没恢复
所有队列同一 vector	单 CPU 成为瓶颈	多队列硬件没有分散到多核

所以中断处理不是“快点执行函数”。它是设备状态机、APIC 状态机和内核并发状态共同收敛。

定时器：抢占和超时来自可编程 deadline

定时器硬件最小模型是计数器加比较器：

$$interrupt = enable \wedge (counter \geq compare)$$

周期 tick 是每次设定固定间隔；高精度 timer 是把 compare 设成最近 deadline；tickless idle 是在没有近期事件时把下一次中断推远。

```

timer_reprogram(now):
    next = min(scheduler_deadline,
               hrtimer_deadline,
               rcu_deadline,
               device_timeout)
    local_apic_timer.compare = next
  
```

OS 里的超时不是后台线程“经常看看”。它依赖硬件在未来某个时刻把控制权交回内核。抢占也一样：若用户程序死循环，只有 timer interrupt 能强制进入内核，设置 need_resched，并在返回路径切换任务。

$$preempt(current) \Rightarrow timer\ IRQ \wedge kernel\ entry \wedge scheduler\ decision$$

这解释了为什么禁中断区必须短。禁中断期间，timer 不能及时打断本 CPU，调度延迟、RCU 延迟、设备 completion 延迟都会被拉长。

PCIe、BAR 和配置空间：设备不是凭空出现

PCIe 设备先通过配置空间被发现。配置空间提供 vendor/device ID、class code、BAR、MSI-X capability、AER capability、DMA 能力等。BAR 不是寄存器本身，而是“设备申请一段 MMIO 地址窗口”的描述。OS 分配地址后，CPU 对该物理窗口的 load/store 会被 root complex 路由到设备。

```
pci_probe_contract:
    read config space
    allocate and enable BAR resources
    set bus mastering only after DMA is prepared
    choose DMA mask and attach IOMMU domain
    allocate MSI-X vectors and set affinity
    initialize device registers and queues
    publish device to upper layer
```

关键不变量：

bus mastering = enabled $\Rightarrow$ IOMMU/domain ready $\wedge$ DMA mask valid $\wedge$ driver owns queues

如果过早启用 bus mastering，设备可能在 OS 没有准备好映射和缓冲区时发起 DMA。可靠驱动在 probe 中不是“尽快开始工作”，而是先建立设备能访问什么、不能访问什么、完成后怎样通知的边界。

NVMe 和网卡：同一种队列型设备骨架

NVMe、网卡、virtio 和很多加速器都使用相似骨架：submission ring、completion ring、doorbell、MSI-X、DMA buffer、request tag。区别在命令格式和上层语义，不在基本状态机。

```
queue_device_submit(req):
    tag = allocate_tag()
    map buffers for DMA
    fill descriptor[tag]
    store_release(descriptor.valid, true)
    dma_wmb()
    writel(new_tail, doorbell)

queue_device_complete(cqe):
    req = lookup_tag(cqe.tag)
    read status and bytes_done
    dma_unmap(req.buffers)
    complete upper object
    free tag
```

队列深度 Q 、平均服务时间 S 和吞吐近似关系：

$$throughput \leq \min(device_rate, Q/S)$$

队列太浅，设备空等；队列太深，延迟和取消成本上升。中断合并可以减少 IRQ 次数，但会增加单个请求完成可见的延迟：

$$latency_{observed} = latency_{device} + latency_{coalescing} + latency_{scheduler}$$

这就是 blk-mq、NAPI、RSS、MSI-X affinity 都围绕队列和 CPU 亲和性设计的原因。设备硬件已经多队列，OS 必须把队列、CPU、NUMA node、软中断和上层对象匹配起来。

USB 和低速总线：不是所有设备都像 PCIe

PCIe 设备能较好自描述，USB 设备通过描述符和端点自描述；I2C、SPI、GPIO、PWM、ADC 这类板级硬件通常不能完整自描述，需要 ACPI 或板级描述告诉 OS 它们存在、地址是多少、中断接在哪个 GPIO、复位脚是哪根。

总线/接口	硬件特点	OS 约束
USB	主机控制，端点和 URB 异步完成，热插拔频繁	断开后 fd 仍可能存在，操作要返回错误并释放资源
I2C/SMBus	低速共享总线，设备地址有限，事务短	访问要串行化，睡眠上下文限制，ACPI 描述很重要
SPI	片选线选择设备，控制器发起传输	片选、模式、速率、DMA/PIO 路径都要配置
GPIO	单 bit 输入输出，也可当中断线	方向、上下拉、边沿触发、去抖都影响语义
PWM/ADC	模拟世界和数字控制边界	精度、采样率、占空比、校准和权限都要处理

所以“驱动”不是一个统一写寄存器模型。USB 驱动管理 URB 生命周期；I2C 驱动管理低速寄存器事务；GPIO 驱动管理 pin 状态和中断边沿；PCIe 驱动管理 BAR、DMA 和 MSI-X。OS 统一它们的方式，是把不同硬件收敛成字符设备、输入事件、网络设备、块设备、hwmon、DRM object 等上层对象。

显示、音频和实时流：为什么 buffer 与时钟同样重要

显示和音频不是“把数据写给设备就结束”。显示控制器按刷新率扫描 framebuffer 或 plane；音频控制器按采样时钟从 ring buffer 取 PCM 数据。它们的核心约束是时间连续性。

$$audio\ underrun \Leftarrow producer_rate < consumer_rate$$

$$frame\ tear \Leftarrow scanout(buffer)\ 期间修改同一可见区域$$

```
display_flip:
    render next buffer
    submit page flip request
    hardware waits for vblank
    scanout switches plane base address
    fence signals old buffer may be reused

audio_playback:
    application fills PCM ring
    driver programs DMA period
    device consumes at sample_rate
    period interrupt wakes refill path
```

这里的 completion/fence 不只是性能工具，而是所有权边界。显示旧 buffer 只有在 scanout 不再使用后才能复用；音频 buffer 如果 refill 晚了会 underrun；视频 buffer 如果 fence 没等就写，会造成撕裂或数据竞争。OS 在这些设备上管理的是“内存 + 时间 + 所有权”三件事。

TPM、Secure Boot 和 IOMMU：安全硬件如何进入 OS

安全硬件也要按结构、状态、接口读。TPM 不是“加密芯片”四个字，它有 PCR、密钥、命令队列和访问控制。Secure Boot 不是单个开关，而是从固件到 bootloader 到内核模块的一条验证链。IOMMU 不是只为性能，它限制设备 DMA 范围，防止设备或恶意外设读写任意内存。

$$measured\ boot: \quad PCR_{new} = Hash(PCR_{old} || measurement)$$

机制	硬件状态	OS 后果
TPM PCR	启动阶段测量累积值	密钥封存、远程证明、启动篡改证据
Secure Boot	固件信任根和签名数据库	限制未签名 bootloader/内核模块
IOMMU	设备 domain 和 IOVA 页表	DMA 隔离、VFIO 直通、安全边界
CPU 特权/页表	CPL、PTE、NX、SMEP/SMAP 等	用户/内核隔离和执行保护

安全章节如果只讲策略而不讲硬件根，会变成口号。真正的隔离来自多个硬件检查点：CPU 不让 Ring 3 写 CR3，MMU 不让用户访问 supervisor 页，IOMMU 不让设备越界 DMA，TPM 让启动状态可度量。

错误报告：硬件失败必须变成可诊断证据

硬件会失败：ECC corrected error、PCIe AER、IOMMU fault、MCE、NVMe status、USB stall、GPU hang、thermal trip、watchdog reset。高质量 OS 不是假设硬件永远正确，而是把失败变成可定位证据。

失败源	证据	OS 动作
page fault	CR2、error code、RIP、CPL	修复、发信号、panic 或 oops
machine check	bank/status/address/misc	记录、页下线、进程 kill、系统下线
PCIe AER	requester、status、severity	reset 设备、降级、日志和隔离
IOMMU fault	device/requester、IOVA、权限	停止设备、撤销请求、报告驱动 bug
NVMe completion error	status code、command id、queue id	重试、fail request、reset controller
thermal/watchdog trip point	trip point、reset reason	降频、关机、保留 crash 证据

这也给测试方法定了标准。不能只测试正常读写；还要测试超时、reset、IOMMU fault、completion error、热插拔、进程取消、并发关闭、内存压力。硬件路径越异步，错误测试越重要。

端到端案例：用户态读文件到 NVMe 完成

把整节收束成一条路径。用户调用 `read(fd, buf, len)`，如果数据不在 page cache，内核可能向 NVMe 提交块 I/O。

1. syscall entry saves trap frame and validates fd
2. VFS maps file offset to page cache page
3. filesystem maps logical block to disk LBA
4. block layer builds request and assigns tag
5. DMA API maps page fragments to IOVA
6. NVMe driver fills SQE with opcode, LBA, PRP/SGL
7. dma_wmb establishes descriptor-before-doorbell
8. driver writes queue tail doorbell in BAR
9. controller DMA-reads SQE and DMA-writes data
10. controller writes CQE with command id and status
11. controller sends MSI-X to queue's CPU
12. handler/poll consumes CQE and completes request

13. driver unmaps DMA and block layer unlocks page
14. syscall copies or maps data to user buffer and returns

每一步对应一个硬件或 OS 对象：

trap frame → file/page cache → bio/request/tag
→ IOVA → SQ/CQ → MSI-X → wake up

这条路径能解释很多“为什么”：为什么系统调用要保存寄存器，为什么用户指针不能直接给设备，为什么 page cache 和 CPU cache 不是一回事，为什么 NVMe 有队列和 doorbell，为什么 completion 前不能释放 buffer，为什么中断亲和性会影响吞吐，为什么 `fsync` 又比普通 `read` 的 completion 多一层持久化语义。

本节验收：每个硬件概念都要能落到不变量

- 给出任意硬件名词，能按“结构、状态、时序、接口、OS 不变量、失败证据”六项拆开。
- 能解释时钟、复位、电源域为什么决定驱动初始化和 runtime PM 顺序。
- 能把 `add rax, rbx` 追踪到译码、寄存器读、ALU、RFLAGS、提交。
- 能把 `mov rax, qword ptr [rbx + 16]` 追踪到 AGU、TLB、PTE、cache、page fault。
- 能写出 PTE 权限判定公式，并说明 present、writable、user、NX 的 OS 用途。
- 能写出 TLB shutdown 的安全释放条件。
- 能区分 cache coherence、memory ordering、MMIO ordering 和 DMA visibility。
- 能画出 MSI-X 到 local APIC、IDT、handler、EOI 的中断路径。
- 能说明 PCIe BAR、bus mastering、DMA mask、IOMMU domain、MSI-X vector 在 probe 顺序中的位置。
- 能把 NVMe/网卡/virtio 抽象成 submission ring、completion ring、doorbell、DMA buffer 和 request tag。
- 能说明 USB、I2C、SPI、GPIO 与 PCIe 驱动模型的差别。
- 能把 TPM、Secure Boot、IOMMU、PTE 这些安全硬件落到具体状态和 OS 后果。
- 能为 page fault、MCE、AER、IOMMU fault、NVMe error 写出可观测证据。

为什么要单独讲硬件

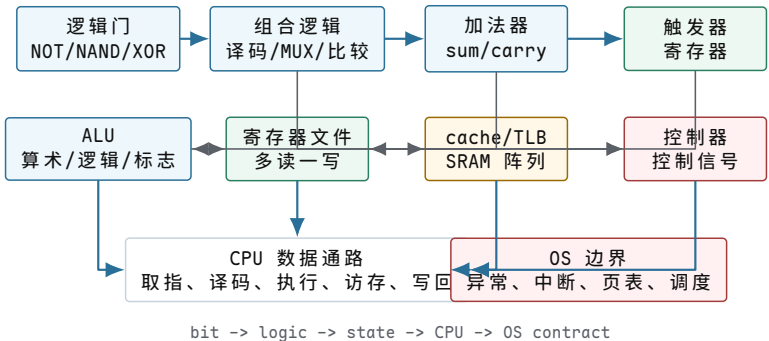
操作系统教材常把硬件压缩成几张框图：CPU、内存、总线、磁盘、网卡。这样会让后面的概念失去来源。系统调用为什么必须切栈，缺页为什么可以重试，同步为什么要原子指令，DMA 为什么要屏障，调度为什么关心 cache 和 TLB，这些问题都要回到硬件怎样由电路一步一步组成。

本章只沿 x86-64 主线解释通用计算机。目标不是把芯片设计课完整搬进来，而是把 OS 必须依赖的硬件机制讲到可推理、可画路径、可分析时延、可解释失败。每个硬件模块都按六个问题拆：

问题	需要回答什么
输入	这个模块从哪里拿 bit、地址、控制信号或时钟
输出	它把什么结果交给下一级，结果什么时候有效
组合路径	输出是否只由当前输入决定，最长逻辑路径是什么
状态	它是否保存历史，状态在什么时钟边沿更新
时延	它的关键路径如何限制频率、吞吐或响应延迟
OS 关系	它如何影响异常、特权、调度、内存、I/O 或安全

有了这张拆解表，硬件就不再是一堆名词。加法器不是“能加数”的盒子，而是一串产生 sum/carry 的门；寄存器不是变量，而是时钟边沿采样的状态；ALU 不是魔法函数，而是多个组合电路加一个选择器；CPU 不是一个大循环，而是寄存器、组合逻辑、控制信号、时钟和异常入口共同组成的状态机。

先看全章主线。下面这张图不是装饰图，而是本章后面每个小节都要回到的“硬件生长路线”。如果学习时不知道某个名词在机器里的位置，就把它放回这条链上：它是计算、选择、保存、搬运、控制，还是把错误交给 OS 的边界？



图示：硬件从门级电路生长到 OS 契约的路线图。后文每个模块都按“输入、输出、状态、时延、失败路径”解释。

读硬件图时先分清三类线

硬件图最容易被误读成“程序流程图”。程序流程图里的箭头常表示先后执行；硬件图里的线表示信号连接，很多信号在同一个周期同时变化。读图时先分清三类线：

线的类型	代表什么	读法
数据线	bit、字节、地址、cache line、页表项	数据从源头经过组合逻辑传到目的寄存器或接口
控制线	enable、select、read/write、privilege check	不搬数据，而是决定哪个 MUX 选哪路、哪个寄存器采样
时钟/握手线	clock、valid、ready、done	决定什么时候采样、什么时候一次传输完成

例如 `add rax, rbx` 不是“CPU 调用 add 函数”。真实路径更接近：寄存器编号译码产生读选择线，寄存器文件同时吐出 `rax` 和 `rbx` 的旧值，ALU 的加法网络在一个组合路径内把结果算出来，写回 MUX 选择 ALU 结果，最后在时钟边沿写入 `rax`。只要没有这个“线和采样”的模型，后面的流水线、异常回滚、页表更新和 TLB 失效都会变成空话。

晶体管到逻辑门：先得到可靠的 0 和 1

数字电路的第一层抽象是把连续电压分成两个稳定区域：低电平表示 0，高电平表示 1。晶体管可以看成受输入控制的开关。很多开关组合起来，就能实现逻辑门。

真实芯片要考虑噪声容限、功耗、扇出、电容负载和工艺波动；教材模型先抓住一件事：输入稳定后，输出经过传播延迟也会稳定。

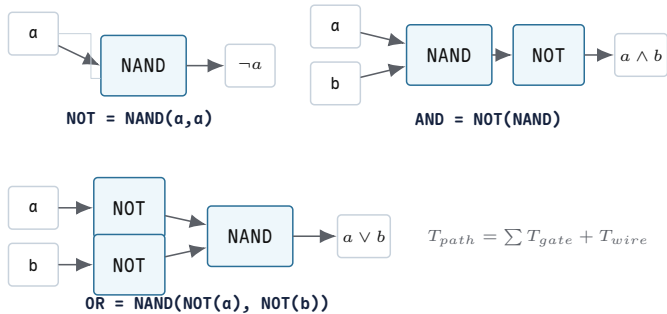
门	布尔函数	为什么重要
NOT	输出输入的反值	构造反相、使能取反、条件翻转
NAND	不是两个输入都为 1	功能完备，可构造任意组合逻辑
NOR	不是任一输入为 1	功能完备，适合某些控制逻辑
AND	两个条件都为 1	权限位、valid 位、enable 位同时满足
OR	至少一个条件为 1	多个异常源、多个 pending 位汇总
XOR	两个输入不同	加法器 sum、奇偶校验、比较辅助

“功能完备”的意思是，只用 NAND 或只用 NOR 也能搭出所有布尔函数。例如 NOT 可以用 NAND 的两个输入接同一个信号得到；AND 可以先 NAND 再 NOT；OR 可以用德摩根律得到。芯片设计不会真的每处都只手写 NAND，但理解 NAND/NOR 能说明复杂硬件最终还是由可组合的简单门构成。

```
not_a = nand(a, a)
a_and_b = not(nand(a, b))
a_or_b = nand(not(a), not(b))
```

门有传播延迟。若一个 NAND 延迟记为 t_{nand} ，三个门串联的路径至少要等三个门都稳定。组合逻辑的输出不能在输入改变后立即采样，必须给信号传播时间。这个事实会一路推导出时钟周期、关键路径、流水线和 CPU 频率上限。

下面把“功能完备”画出来。左边用 NAND 自己接自己得到反相器；中间先 NAND 再反相得到 AND；右边先把输入反相，再 NAND 得到 OR。这三个小网络说明了一件很关键的事：硬件设计不是从“CPU”这种大盒子开始，而是从可复制、可验证、可组合的小布尔网络开始。



图示：NAND 构造 NOT、AND、OR。真正芯片会使用标准单元库优化面积、功耗和延迟，但逻辑等价关系从这里开始。

门级设计的三个底层约束

第一，噪声容限。电压不是抽象的 0 和 1，而是连续物理量。数字电路必须规定低电平和高电平的安全区间，中间区域不可靠。只要噪声、串扰或电源波动把信号推到不确定区，后级门就可能读错。

第二，扇入和扇出。一个门输入太多，内部结构会变慢；一个输出驱动太多后级门，电容负载变大，也会变慢。所以“一个大 AND 一次判断 64 个条件”在逻辑式上很简洁，在电路上可能要拆成树形结构。

第三，线延迟。在现代芯片里，延迟不只来自门，也来自连线。一个信号从执行单元一侧绕到另一侧，可能比多过几级门还慢。这解释了为什么寄存器文件、调度器、旁路网络、cache tag 比较这些结构不能无限做大：不是逻辑不会写，而是时序和布线会失控。

组合逻辑：没有记忆的计算网络

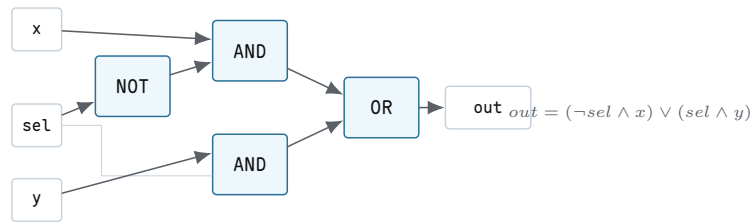
组合逻辑只由当前输入决定输出，不保存历史。加法器、比较器、译码器、选择器、优先级编码器都属于组合逻辑。组合逻辑适合表达“现在这些 bit 应该产生什么结果”，但不能表达“上一周期执行到哪里”。

从真值表到电路：先做一个 2 路 MUX

组合逻辑的设计入口通常是真值表或布尔表达式。以 2 路 MUX 为例，它有两个数据输入 `x/y`、一个选择输入 `sel`、一个输出 `out`。当 `sel = 0` 时输出 `x`，当 `sel = 1` 时输出 `y`。

$$out = (\neg sel \wedge x) \vee (sel \wedge y)$$

这条式子直接对应电路：选择信号先反相，一路和 `x` 做 AND，另一路和 `y` 做 AND，最后 OR 汇总。CPU 里几乎所有“选择下一步”的动作都可以抽象成 MUX：下一条 RIP 选顺序地址还是分支目标，ALU 第二操作数选寄存器还是立即数，写回值选 ALU 结果还是 load 数据。



图示：2 路 MUX 的门级实现。它说明控制信号本身不搬数据，却决定哪一路数据能继续向后传播。

MUX 的关键路径近似是：

$$T_{mux2} \approx T_{not} + T_{and} + T_{or}$$

更大的 MUX 可以由多级 2 路 MUX 组成。若 8 路选择直接做成树，选择路径大约经过 3 级 MUX；若把很多选择挤在一个阶段，选择网络就会成为关键路径。x86-64 的寻址和写回来源丰富，前端译码和后端选择网络都比“只有寄存器和立即数”的教学机器更重。

译码器和编码器

译码器把一个二进制编号变成多根 one-hot 信号。例如 3 位输入可以选择 8 个输出中的一个。寄存器文件要根据寄存器编号选择某个寄存器，指令译码器要根据 opcode 选择控制路径，中断控制器要根据 vector 找入口。

```
decode3(x):
    out[0] = (x == 0)
    out[1] = (x == 1)
    ...
    out[7] = (x == 7)
```

编码器反过来，把多根输入压成编号。优先级编码器还要规定多个输入同时为 1 时选谁。例如多个中断 pending 时，中断控制器要根据优先级选择一个 vector 投递给 CPU。

模块	输入输出	OS 相关影子
寄存器编号译码	register id 到读写选择线	syscall ABI 规定参数在哪些寄存器
opcode 译码	指令字节到控制信号	syscall、特权指令和非法指令检查

vector 选择	pending 中断到 vector	timer、IPI、device IRQ 分发
页表索引拆分	虚拟地址位段到 PML4/PDPT/PD/PT index	page fault handler 能定位 PTE

多路选择器 MUX

MUX 根据选择信号从多个输入中选一个输出。CPU 里到处都是 MUX：ALU 的第二个操作数来自寄存器还是立即数，写回值来自 ALU 还是 load 数据，下一 RIP 来自顺序地址、分支目标、异常入口还是返回地址。

```
writeback_value = mux(
    from_alu: alu_result,
    from_memory: load_data,
    from_immediate: sign_extended_imm,
    from_status: flags_value
)
```

MUX 的代价是选择路径也有延迟。输入很多时，MUX 会变深或变宽，时序更难。x86-64 的复杂寻址模式、复杂写回来源和丰富控制状态，都会增加前端译码和后端选择网络的复杂度。OS 不直接控制 MUX，但系统调用、异常、返回用户态都依赖“下一 RIP 只能由受控硬件路径选择”。

加法器：从一位到 64 位

半加器和全加器

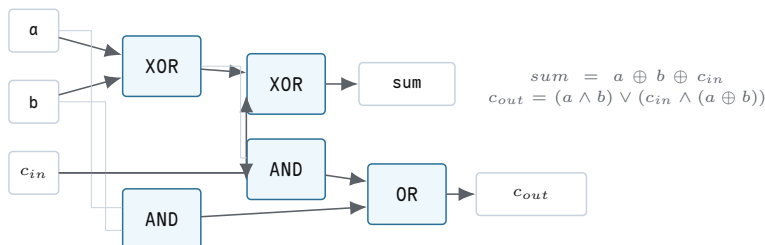
一位加法先从真值表开始。两个输入 a/b 相加，sum 表示本位结果，carry 表示进位。半加器没有低位进位：

```
sum    = a XOR b
carry  = a AND b
```

全加器增加 $carry_in$ ：

```
sum      = a XOR b XOR carry_in
carry_out = (a AND b) OR (carry_in AND (a XOR b))
```

把全加器画成门级网络更容易看出为什么有两类输出。sum 是“奇偶性”：三个输入里有奇数个 1 时为 1，所以用 XOR 串起来。carry 是“至少两位为 1”： a 和 b 同时为 1 一定产生进位；或者 a/b 恰好一个为 1，并且低位进位为 1，也会产生进位。

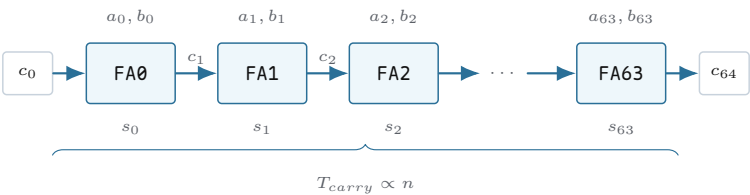


图示：全加器的门级结构。sum 路径和 carry 路径都需要等待门传播稳定；多位加法最慢的通常是 carry 链。

把 64 个全加器串起来，就是 64 位 ripple-carry adder。第 0 位产生的进位传给第 1 位，第 1 位再传给第 2 位，一直传到第 63 位。

```
carry[0] = carry_in
for i in 0..63:
    sum[i], carry[i + 1] = full_adder(a[i], b[i], carry[i])
```

这个设计直观，但关键路径很长。最高位结果必须等低位进位一路传播。若每一级全加器进位延迟是 t_{fa} ，64 位串行进位路径近似是 $64 * t_{fa}$ 。频率高的 CPU 不能只靠这种最朴素结构。



图示：ripple-carry adder。面积小、结构直观，但最高位必须等待低位进位逐级传上来。

这个图解释了“时延分析”不是附会出来的。若每个全加器的进位传播延迟为 T_{carry_fa} ，忽略布线后：

$$T_{ripple64} \approx 64 \times T_{carry_fa} + T_{sum_xor}$$

真实芯片还要加上连线、电容负载和输出选择延迟。朴素 ripple-carry 的价值是教学清楚和面积小；高频 CPU 会用更复杂的结构减少串行等待。

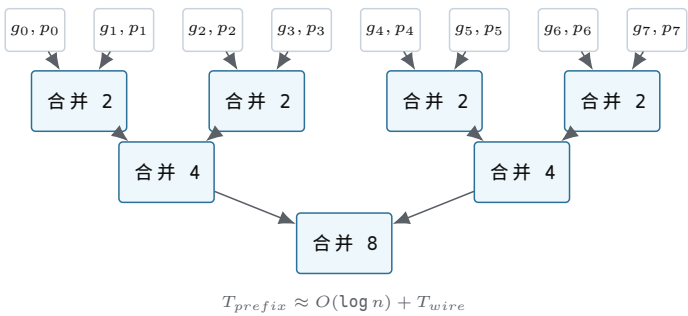
进位生成、传播和更快加法

把每一位拆成 generate 和 propagate，可以更清楚地分析进位：

```
g[i] = a[i] AND b[i]      // this bit generates carry
p[i] = a[i] XOR b[i]      // this bit propagates carry_in
carry[i + 1] = g[i] OR (p[i] AND carry[i])
```

若某一位 $g[i] = 1$ ，不管低位进位是什么，本位都会产生进位。若 $p[i] = 1$ ，本位会把低位进位传上去。carry-lookahead、carry-select、prefix adder 等结构的目标，都是减少“进位逐位等”的深度，用更多门面积换更短关键路径。

看成 generate/propagate 之后，硬件就可以把多个 bit 的进位合并成树，而不是让进位排队穿过 64 个全加器。下图画的是前缀加法器的思想，不对应某一种具体工业版图。重点是：第 0 层每一位先算本地 g/p ，第 1 层合并相邻 2 位，第 2 层合并 4 位，继续扩大覆盖范围。这样最高位不再等 63 次串行传播，而是走近似 $\log_2(64)$ 层合并网络。



图示：前缀进位的核心思想：用并行合并网络提前算高位进位，减少逐位等待。

加法器	关键路径特点	取舍
ripple-carry	进位逐位传播，深度随位宽线性增长	面积小，慢
carry-lookahead prefix adder	分组提前计算进位 树形合并 generate/propagate	面积更大，速度更快 适合高性能宽位加法，布线复杂

OS 为什么要知道这些？不是为了手画加法器版图，而是理解“硬件速度来自结构取舍”。同样的思想会出现在页表遍历、cache 查找、队列调度里：线性等待简单但慢，提前计算和并行结构更快但更复杂。

减法、比较和 RFLAGS

减法可以用补码加法实现：

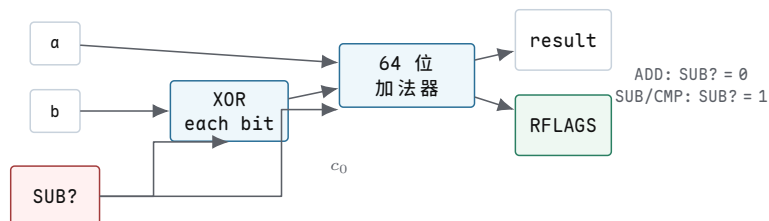
$$a - b = a + \text{bitwise_not}(b) + 1$$

比较也常复用减法路径。执行 `cmp rax, rbx` 时，硬件计算类似 `rax - rbx` 的结果，但不写回目的寄存器，只更新标志位。x86-64 常见条件分支会读取这些标志，例如 zero flag、sign flag、carry flag、overflow flag。

标志	来源	软件影响
ZF	结果是否为 0	<code>je</code> 、循环结束、比较相等
SF	结果最高位	有符号比较辅助
CF	无符号进位或借位	无符号比较、宽整数运算
OF	有符号溢出	有符号比较和溢出检测

RFLAGS 是 CPU 架构状态的一部分。中断、异常和系统调用返回时，如果错误恢复了标志位，用户程序的后续条件分支可能走错路径。这样看，RFLAGS 不是“汇编课小细节”，而是上下文保存必须精确处理的硬件状态。

减法和比较复用加法器，是硬件复用的第一个重要案例。把 `b` 每一位取反，再把初始进位设为 1，就能用同一个加法器计算 `a - b`。比较指令只是不把结果写回通用寄存器，而是把结果的零、符号、进位、溢出信息写进 RFLAGS。



图示：同一条加法路径支持 ADD、SUB 和 CMP。CMP 不写回 `result`，但会更新 RFLAGS，供后续条件跳转读取。

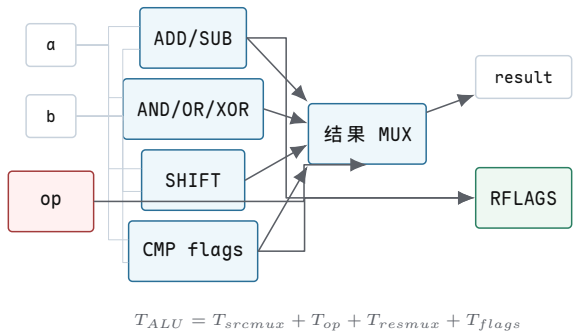
ALU：把多个组合电路收束成执行单元

ALU 可以看成一组组合电路加一个结果选择器。加法、减法、按位逻辑、移位、比较都并行或分组计算，控制信号选择最终结果和标志位。

```
alu(a, b, op):
    add_result = a + b
    sub_result = a + (~b) + 1
    and_result = a AND b
    or_result  = a OR b
    xor_result = a XOR b
    shl_result = barrel_shift_left(a, b[0..5])

    result = mux(op,
        ADD: add_result,
        SUB: sub_result,
        AND: and_result,
        OR:  or_result,
        XOR: xor_result,
        SHL: shl_result)
    flags = compute_flags(result, op)
```

把这段伪代码画成硬件，就是多条组合路径并排加一个结果 MUX。注意，MUX 选择的是已经在路径上计算或正在稳定的信号；不是软件里的 `switch` 才开始执行某个分支。为控制面积和功耗，真实 CPU 不一定每条路径都永远满速并行，但抽象上 ALU 就是“多种组合功能 + 选择 + 标志生成”。



图示：ALU 由多种组合功能和结果选择器组成。控制信号选择输出，标志路径影响后续条件分支和异常返回。

移位器和桶形移位

移位可以一位一位移，也可以用桶形移位器一次完成多位选择。64 位桶形移位器通常按 1、2、4、8、16、32 这些层级组织，每一层根据移位量的一位决定是否移动。它本质上是多层 MUX。

```
stage0 = mux(shift[0], value << 1, value)
stage1 = mux(shift[1], stage0 << 2, stage0)
stage2 = mux(shift[2], stage1 << 4, stage1)
...
```

这说明高性能硬件经常用“多层选择网络”换速度。代价是面积、功耗和布线。软件里一条 `shl rax, cl` 看起来简单，硬件里可能是一组宽 MUX 网络。

乘法和除法为什么常有更长延迟

加法是 ALU 的基础路径。乘法可以理解为部分积生成和求和，除法常需要迭代或复杂预测。高性能 CPU 会给乘法器专门硬件，但乘法延迟通常仍比简单整数加法长；除法更慢。

操作	硬件特点	性能含义
ADD/SUB	宽加法器，常见热路径	延迟低，吞吐高
SHIFT	多层选择网络	延迟中等，依赖移位器设计
MUL	部分积生成和压缩树	延迟更长，执行端口更少
DIV	迭代或复杂算法	延迟长，吞吐低

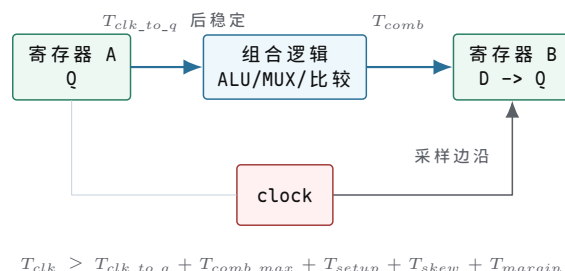
OS 热路径大量使用加减、位运算和比较，而谨慎使用除法。调度器、内存分配器、网络路经常把除法换成移位、乘法近似或预计算，不是写法偏好，而是硬件执行成本不同。

触发器和寄存器：让机器拥有状态

组合逻辑没有记忆。要让机器知道“当前 RIP 是多少”“当前任务是谁”“上一周期请求是否已经发出”，必须有时序逻辑。D 触发器在时钟边沿采样输入 `D`，把它保存为输出 `Q`。多个 D 触发器并排就是寄存器。

```
on rising_clock_edge:
    if reset:
        q = 0
    else if enable:
        q = d
```

用图看，D 触发器把“连续变化的组合逻辑输出”切成“只在时钟边沿更新的状态”。左边的组合逻辑可以在一个周期内多次抖动，只要在采样窗口前稳定，右边寄存器看到的就是一个确定值。没有这种边界，CPU 就无法定义“上一条指令的结果已经提交、下一条指令还没有提交”。



图示：寄存器到寄存器的时序路径。CPU 频率不是随意调高的，时钟周期必须容纳最长组合路径和采样要求。

寄存器引入三个重要时序参数：

参数	含义	违反后果
clock-to-Q	时钟边沿后输出稳定所需时间	后级组合逻辑起点变晚
setup time	时钟边沿前输入必须稳定多久	采样错误或亚稳态
hold time	时钟边沿后输入还要保持多久	采样到错误变化

时钟周期必须满足：

$$T_{clk} \geq T_{clk_to_q} + T_{comb_max} + T_{setup} + T_{skew} + T_{margin}$$

这就是“关键路径”概念。若一个阶段中寄存器文件读取、ALU、cache tag 比较、数据选择和写回全塞在一个周期里， T_{comb_max} 会很长，频率就上不去。流水线的本质，是在长组合路径中间插入寄存器，把一个长周期拆成多个短周期。

为什么寄存器不是“变量”

软件变量可以被编译器放在寄存器、栈、堆或被优化掉；硬件寄存器是物理状态单元。它有确定的位宽、写使能、复位策略、时钟域和读写端口。把寄存器误解成变量，会漏掉三个关键事实：

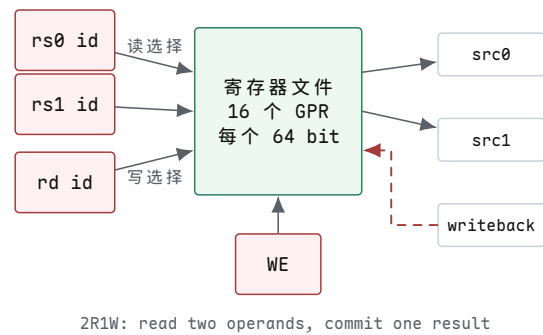
- 寄存器只在采样事件更新，不会在组合逻辑中间“逐行赋值”。
- 多个寄存器可以在同一个时钟边沿同时更新，所以硬件天然并行。
- 写使能是控制信号，错误打开会破坏架构状态，错误关闭会让指令看似执行却没有提交。

例如 `add rax, rbx` 的写回不是 ALU 一算完就“立刻改变 `rax`”。在同步模型里，ALU 输出先稳定到写回输入，等时钟边沿到来，寄存器文件的写端口才把新值写进 `rax` 对应的存储单元。这个边沿就是流水线和异常精确性的基础。

寄存器文件：CPU 的小而快状态库

通用寄存器不是内存数组的简单缩小版。寄存器文件需要同时支持多个读端口和写端口。例如一条整数二操作数指令常读两个源、写一个目的。系统调用 ABI 又规定 `rax/rdi/rsi/rdx/r10/r8/r9` 等寄存器承担系统调用号和参数。

```
read_port0 selects source register A
read_port1 selects source register B
write_port selects destination register
write_enable controls whether result is committed
```



图示：x86-64 架构寄存器文件的教学模型。真实乱序 CPU 还会有物理寄存器和重命名表，但 OS 保存的是架构可见状态。

端口越多，寄存器文件越难做快。每个读端口都要能从所有寄存器中选择一个值，每个写端口都要把数据写到指定寄存器。乱序 CPU 还会加入寄存器重命名，把 x86-64 架构寄存器映射到更多物理寄存器，减少假依赖。

层次	作用	OS 看到什么
架构寄存器	ISA 规定的软件可见寄存器	trap frame、上下文切换、ABI
物理寄存器	微架构内部重命名资源	OS 通常看不到，不逐个保存
保存区	内存中的寄存器快照	signal frame、switch context、pt_regs

上下文切换保存的是架构可见状态，而不是所有物理寄存器。中断入口保存通用寄存器，是为了让被打断代码返回后仍看到相同架构状态。理解这一点，就能区分“CPU 内部有很多临时状态”和“OS 必须保存哪些状态”。

寄存器文件还解释了 ABI 为什么重要。系统调用约定把系统调用号放在 `rax`，参数放在 `rdi/rsi/rdx/r10/r8/r9` 等寄存器，不是因为这些名字好看，而是为了让硬件入口和内核汇编入口能用确定位置取得参数。若入口保存错寄存器、恢复错寄存器，用户态看到的就是架构状态损坏。

存储阵列：从寄存器到 cache 和 DRAM

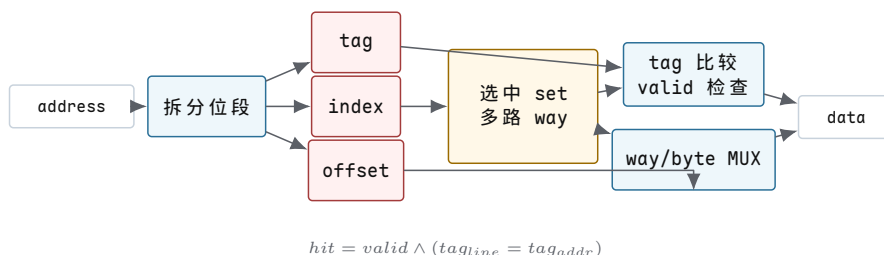
寄存器很快但面积贵，不能拿来做主内存。SRAM 用更多晶体管保存一个 bit，速度快，适合 cache、TLB 和小型片上队列。DRAM 用电容保存 bit，密度高但需要刷新，适合主内存。NAND Flash、NVMe SSD 等则提供持久存储，但访问模型完全不同。

存储	硬件特点	OS 影响
寄存器	最快，数量少，CPU 私有	上下文切换必须保存可见部分
SRAM	快，面积大，片上使用	cache、TLB、队列、预测器
DRAM	容量大，易失，要刷新	物理页、NUMA、page cache、DMA buffer
NAND/SSD	持久，块擦写，控制器复杂	文件系统、flush、FUA、磨损和错误恢复

cache 作为 SRAM 结构

cache 通常由 tag、valid、dirty、data 组成。CPU 用地址的一部分选择 set，再比较 tag，命中后读出 cache line。cache line 常见为 64 字节量级。cache 不是逐字节取一个小值，而是以 line 为单位搬运。

```
cache_lookup(address):
    offset = address bits selecting byte in line
    index = address bits selecting set
    tag = remaining high bits
    for way in set[index]:
        if way.valid and way.tag == tag:
            return way.data[offset]
    miss
```



图示：cache lookup 的硬件路径。tag 比较和 data 选择都要在时钟预算内完成，所以 cache 容量、相联度和命中延迟互相牵制。

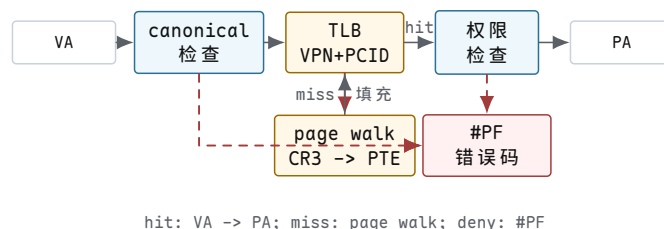
OS 设计里反复出现 cache line：自旋锁竞争会让同一 line 在 CPU 间迁移，伪共享会让不同变量互相拖慢，每 CPU 数据结构减少共享写，DMA buffer 需要考虑 CPU cache 和设备可见性。cache 是硬件，但它塑造了内核数据结构。

TLB 作为翻译缓存

TLB 缓存虚拟页到物理页和权限。它不是普通 data cache，因为它的内容来自页表遍历，并且带有权限、PCID、global 等语义。x86-64 地址翻译热路径大致是：

```
virtual address
-> canonical check
-> TLB lookup by virtual page and PCID
-> if miss, page walk from CR3
-> PTE permission check
-> physical address
```

页表项改了，内存中的 PTE 变了，但 TLB 里的旧翻译不会自动消失。于是 OS 必须执行 invalidate 和 shutdown。硬件为了快复制了状态，OS 就要负责让这些副本在关键时刻一致。



图示：地址翻译热路径。页表在内存里，TLB 是硬件缓存；OS 修改页表后必须处理 TLB 里的旧副本。

数据通路：把寄存器、ALU、内存接口连起来

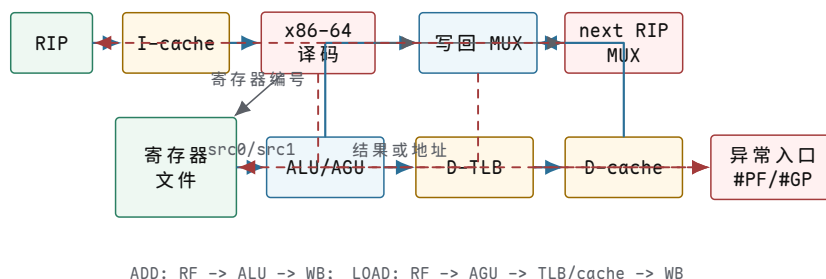
有了寄存器文件、ALU、MUX、内存接口和 RIP，就可以画一个最小数据通路。以 `add rax, rbx` 为例：

1. instruction decoder selects rax as dst/src0 and rbx as src1
2. register file reads rax and rbx
3. ALU op is set to ADD
4. ALU computes sum
5. writeback MUX selects ALU result
6. register file writes rax on clock edge
7. next RIP selects sequential address

以 `mov rax, qword ptr [rbx + 16]` 为例：

1. decoder recognizes load with base rbx and displacement 16
2. register file reads rbx
3. address generation unit computes `rbx + 16`
4. D-TLB translates virtual address
5. cache or memory returns 8 bytes
6. writeback MUX selects load data
7. register file writes rax

这两条指令共享很多硬件：寄存器文件、ALU 或地址生成器、写回 MUX、RIP 更新路径。load 额外使用 TLB、cache、内存顺序和异常路径。OS 后面讲 page fault，就是第 4 步无法完成时，硬件把控制权交给内核。



图示：最小 CPU 数据通路。图中每个 MUX 或缓存访问都可能进入关键路径；load 的失败路径直接连到 OS 的 page fault 处理。

把两条 x86-64 指令逐拍摊开

对 `add rax, rbx`，硬件关心的是两个源值、一个操作码、一个目的寄存器和标志位更新。若抽象成单周期同步模型，路径是：

$$\begin{aligned} &RIP \rightarrow ICache \rightarrow Decode \rightarrow RegRead \\ &\quad \rightarrow ALU_{add} \rightarrow WBMux \rightarrow RegWrite \end{aligned}$$

对 `mov rax, qword ptr [rbx + 16]`，路径多出地址生成和内存层次：

$$\begin{aligned} &RIP \rightarrow ICache \rightarrow Decode \rightarrow RegRead(rbx) \\ &\quad \rightarrow AGU(+16) \rightarrow DTLB \rightarrow DCache \\ &\quad \rightarrow WBMux \rightarrow RegWrite(rax) \end{aligned}$$

如果 DTLB 报告 PTE 不 present，写回路径必须关闭，不能把垃圾数据写进 `rax`；同时异常路径要记录 faulting RIP、访问类型、错误码和相关地址。OS 能重试 page fault，靠的不是内核“猜一下”，而是硬件在精确边界上禁止错误提交并留下证据。

控制器：谁来产生控制信号

数据通路回答“数据从哪里流到哪里”，控制器回答“这一拍打开哪些开关”。控制信号包括读哪个寄存器、ALU 做什么、是否访存、是否写回、下一 RIP 选哪路、当前指令是否允许在 Ring 3 执行。

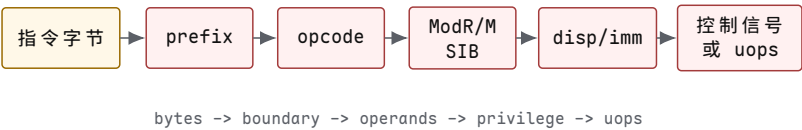
控制信号	控制对象	错误后果
reg_read_sel	寄存器文件读端口	读错参数或地址
alu_op	ALU 功能选择	算错结果或标志位
mem_read/write	cache/MMU 访问	错读、错写或破坏内存
writeback_enable	寄存器写回	状态未更新或误更新
next_rip_select	RIP 来源 MUX	跳错路径或无法进入异常入口
privilege_check	特权和权限检查	用户态执行敏感操作

硬布线控制和微码

简单指令集可以用硬布线控制器:opcode 经过译码器直接产生控制信号。x86-64 为了长期兼容和丰富指令，会把复杂指令译成内部微操作，某些复杂或少见路径由微码协助。无论内部实现如何，软件看到的 ISA 契约必须稳定。

```
x86-64 front-end sketch:
fetch instruction bytes
find instruction boundaries
decode prefixes, opcode, ModR/M, SIB, displacement, immediate
produce one or more micro-operations
send uops to scheduler and execution units
```

这解释了为什么 x86-64 解码比教学固定长度指令难：一条指令可能 1 字节，也可能十几字节；操作数可能在寄存器、内存、立即数中；寻址模式可能包含 base、index、scale、displacement。OS 不写解码器，但调试、反汇编、异常 RIP、kprobes、ftrace 都依赖“指令边界可确定”。



图示：x86-64 前端译码的教学拆分。复杂性来自变长编码和丰富寻址，不来自“汇编语法长得复杂”。

单周期 CPU：最容易理解，也最浪费

单周期 CPU 让每条指令在一个长周期内完成所有工作。对 add rax, rbx, 周期要覆盖取指、译码、读寄存器、ALU、写回。对 load, 周期还要覆盖地址生成、TLB/cache 访问和写回。为了让最慢指令也能完成，所有指令都被迫使用同样长的周期。

```
T_single >= fetch + decode + reg_read + alu + tlb + cache + writeback + setup
```

假设粗略延迟如下：

模块	示例延迟	说明
取指 I-cache	1.0 ns	命中时仍要 tag/data 路径
译码	0.8 ns	x86-64 前端更复杂
寄存器读	0.4 ns	多端口增加负担
ALU/地址生成	0.5 ns	加法或地址计算
D-TLB/cache	1.2 ns	load/store 热路径
写回和 setup	0.3 ns	写回寄存器前稳定

单周期 load 可能需要 4 ns 以上，于是频率上限不到 250 MHz；而简单 add 其实不需要等完整 load 路径。数字只是示意，结论重要：最长路径决定所有指令周期，硬件利用率差。



图示：单周期 CPU 的问题：简单 ADD 也被迫等待 load 的 TLB/cache 路径，最长路径决定所有指令的周期。

多周期 CPU：把一条指令拆成状态机

多周期 CPU 把一条指令拆成多个阶段，每个阶段用较短周期完成一部分工作。不同指令可以走不同数量的状态。例如 ALU 指令不需要 MEM 阶段，store 不需要 WB 阶段。

```

FETCH:
  IR = memory[RIP]
  RIP = RIP + instruction_length

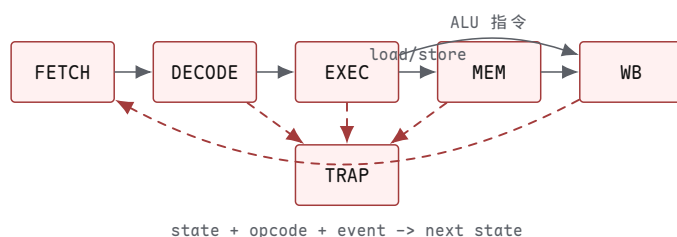
DECODE:
  decode IR
  read source registers

EXECUTE:
  ALU computes result or address

MEMORY:
  load or store if needed

WRITEBACK:
  update destination register if needed
  
```

这里第一次出现“控制器是状态机”的完整形态。控制器保存当前状态，下一状态由当前状态、opcode、异常和 ready 信号决定。内核中的驱动状态机、调度状态机、文件系统恢复状态机，形式上都和这个硬件控制器相似：状态不能跳跃，失败要有收敛路径。



图示：多周期 CPU 是一个硬件状态机。它节省长周期浪费，但单条指令仍要经历多个状态。

流水线 CPU：用寄存器切关键路径

流水线在阶段之间插入流水线寄存器，让 IF、ID、EX、MEM、WB 同时处理不同指令。它提高吞吐，不一定降低单条指令从开始到完成的延迟。

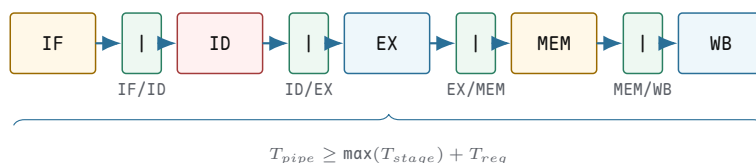
```

cycle 1: inst1 IF
cycle 2: inst1 ID  inst2 IF
cycle 3: inst1 EX  inst2 ID  inst3 IF
cycle 4: inst1 MEM inst2 EX  inst3 ID  inst4 IF
cycle 5: inst1 WB  inst2 MEM inst3 EX  inst4 ID  inst5 IF
  
```

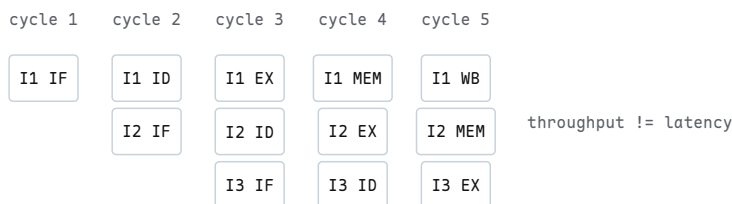
流水线周期由最慢阶段决定：

$$T_{pipe} \geq \max(T_{IF}, T_{ID}, T_{EX}, T_{MEM}, T_{WB}) + T_{pipe_reg}$$

流水线寄存器也有成本。切得太浅，频率上不去；切得太深，分支错误和异常冲刷成本增加，流水线寄存器本身耗电也增加。现代 x86-64 CPU 的前端、乱序后端、load/store、提交都比五级模型复杂，但关键思想不变：用状态边界把长组合路径拆短。



图示：五级流水线教学模型。流水线提高吞吐，但分支、异常和数据依赖都要额外硬件处理。



图示：流水线时序图。吞吐来自重叠，不是让单条指令神奇地只用一个阶段。

数据 hazard

相邻指令有依赖时，流水线会遇到数据 hazard：

```
add rax, rbx
sub rcx, rax
```

第二条需要第一条的新 `rax`。如果等第一条写回再读，流水线要停。硬件可用 forwarding 把执行结果直接送到后续执行单元，减少等待。load-use 更难，因为 load 数据要到 cache 返回后才知道。

```
if EX/MEM.rd == ID/EX.rs and EX/MEM.writes_register:
    operand = EX/MEM.forwarded_result
else if MEM/WB.rd == ID/EX.rs and MEM/WB.writes_register:
    operand = MEM/WB.forwarded_result
else:
    operand = register_file_value
```

控制 hazard 和分支预测

分支决定下一 RIP。如果等分支完全解析再取指，前端会空等。分支预测器猜方向和目标，先把预测路径送进流水线。猜错时，硬件丢弃年轻指令，从正确 RIP 重新取指。

```
if prediction_wrong:
    squash younger pipeline entries
    restore front-end RIP to correct target
    restart fetch
```

分支预测是性能优化，也带来安全问题。被冲刷的预测路径不会提交架构结果，但可能改变 cache 和预测器状态。OS 的隔离和安全补丁要考虑这种微架构痕迹。

结构 hazard

结构 hazard 是两个阶段争同一个硬件资源。例如取指和数据访问若共用同一个单端口内存，会冲突；多个执行单元不足时，后端会排队。硬件通过分离 I-cache/D-cache、多端口结构、队列和调度器缓解。

这对 OS 性能分析很重要。程序慢不一定是算法指令数多，也可能是前端取指不足、load/store 单元堵塞、执行端口竞争、cache miss 或 TLB miss。perf 看到 IPC 低时，要能把它落到这些硬件资源上。

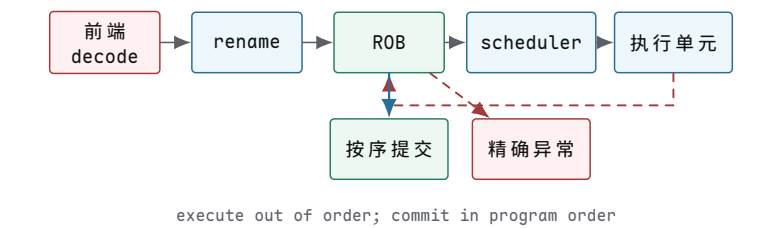
乱序执行：保持 ISA 顺序，内部尽量并行

乱序执行把“何时执行”和“何时提交”分开。指令进入 reorder buffer，源操作数准备好就可以发射到执行单元；结果先写到临时位置；最终按程序顺序提交到架构状态。

```
front-end decodes instructions into uops
rename maps architectural registers to physical registers
scheduler wakes uops whose operands are ready
execution units run ready uops out of order
ROB commits completed uops in program order
```

这样可以在 cache miss 等慢事件发生时继续执行无关工作。但 OS 依赖的是提交边界：异常必须精确，系统调用必须在受控边界进入，调试单步必须看到可解释 RIP，上下文切换保存的是已经提交或可恢复的架构状态。

内部结构	作用	OS 依赖的外部契约
rename table	消除假依赖，提高并行度	架构寄存器语义不变
scheduler	选择可执行 uop	指令结果最终按 ISA 呈现
ROB	记录顺序和异常	precise exception、可重试 fault
store buffer	缓冲写入	需要内存屏障控制可见顺序



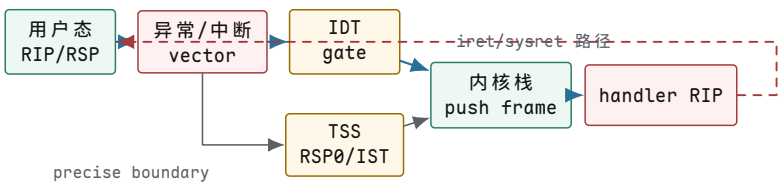
图示：乱序 CPU 的教学结构。rename 和 scheduler 提高并行度，ROB 保证 precise exception 和按序提交。

异常和中断硬件：CPU 怎样受控交还给 OS

异常和中断不是在 CPU 外面“调用内核函数”。硬件必须在可恢复边界停止当前流，保存返回所需状态，查 IDT，必要时切换到 TSS 指定的内核栈，然后跳到 handler。

```
event_entry(vector):
  finish or roll back to precise boundary
  determine whether privilege changes
  if Ring 3 -> Ring 0:
    switch RSP to TSS.RSP0 or IST stack
  push return RIP, CS, RFLAGS, RSP, SS as required
  push error code for selected exceptions
  RIP = IDT[vector].handler
```

这条路径依赖前面所有硬件：RIP 是寄存器，IDT 查找是表项读取和权限检查，TSS 提供栈，RFLAGS 记录状态，流水线和 ROB 提供精确边界。OS 的 trap frame 不是软件随便造的结构，而是硬件入口事实和汇编保存约定合成的证据。



图示：x86-64 事件入口。IDT 决定 handler，TSS 提供特权切换栈，硬件压入返回状态，OS 汇编入口再保存剩余寄存器。

这里最容易漏掉的是“切栈”的原因。用户态 `rsp` 指向用户地址空间中的栈，内核不能信任它：它可能未映射、只读、故意指向内核不该写的位置，或者在进入内核前已经接近栈底。Ring 3 到 Ring 0 时，硬件必须切到 TSS 指定的内核栈，然后再压入返回帧。这样内核入口至少拥有一块可信栈空间，才能继续保存寄存器、构造 trap frame、调用 C/C++ 层处理逻辑。

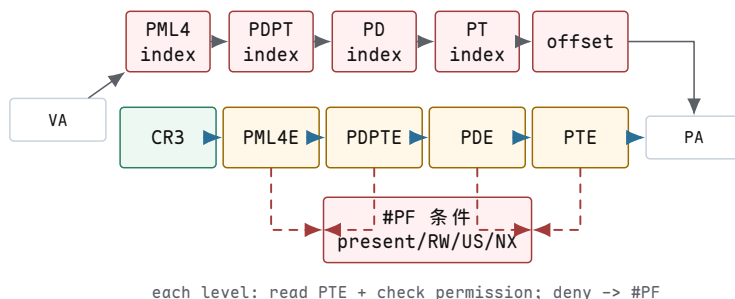
MMU 和页表硬件：地址也要通过电路决策

虚拟地址不是直接送到 DRAM。x86-64 CPU 要先检查 canonical 形式，再查 TLB，miss 时从 CR3 指向的顶级页表开始 page walk。每级页表项都要检查 present、权限、地址位和页大小等字段。

```
translate(va, access, cpl):
    require canonical(va)
    if TLB has (pcid, virtual_page):
        check cached permissions
        return physical_page + offset

    pml4e = read_phys(CR3.base + pml4_index(va) * 8)
    pdpte = read_phys(pml4e.base + pdpt_index(va) * 8)
    pde = read_phys(pdpte.base + pd_index(va) * 8)
    pte = read_phys(pde.base + pt_index(va) * 8)
    check permission bits
    fill TLB
    return pte.page + offset
```

page walk 本身也是内存访问，可能命中 cache，也可能很慢。因此 TLB 是性能关键。修改页表后必须失效旧 TLB 项，是因为硬件为了快保存了翻译副本。COW、mprotect、munmap、进程退出都要处理这个副本生命周期。



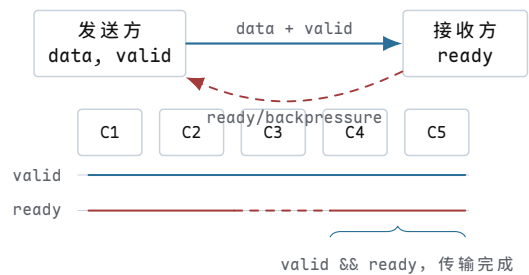
图示：x86-64 四级页表遍历的教学模型。五级分页只是多一级同类结构；本书第一阶段先按四级页表讲清楚机制。

总线 and ready/valid：模块之间怎样说话

CPU 内部和外部模块不能只靠“函数返回”。硬件常用 ready/valid 协议表达一次传输：发送方在数据有效时拉高 valid，接收方能接受时拉高 ready，二者同时为 1 的周期完成一次握手。

```
transfer happens when valid && ready
sender must keep data stable while valid && !ready
receiver may apply backpressure by lowering ready
```

这个小协议解释了很多 OS 现象。设备队列满时就是硬件或驱动层 backpressure；block layer 要限制 queue depth；网络栈要处理发送队列和接收 ring；io_uring 要区分 submission queue 和 completion queue。硬件接口天然就是异步状态机，不是同步函数调用。



图示：ready/valid 握手。valid 高但 ready 低时，发送方必须保持数据稳定；这就是硬件层面的背压。

从硬件模块到 OS 抽象

把本章的硬件模块映射回 OS，可以看到每个抽象都有电路根。

硬件层	直接能力	OS 抽象或约束
逻辑门和 MUX	条件选择、控制信号	特权入口只能走受控路径
加法器和 ALU	地址计算、比较、标志位	指针、页内偏移、RFLAGS、条件分支
触发器和寄存器	保存当前状态	trap frame、context switch、signal frame
寄存器文件	快速操作数和 ABI 位置	syscall 参数、返回值、调用约定
cache/TLB SRAM	缓存数据和地址翻译	局部性、TLB shutdown、cache line 竞争
流水线和 ROB	高吞吐和精确提交	precise fault、可重试 page fault
MMU/page walker	地址翻译和权限检查	地址空间、COW、mmap、隔离
IDT/TSS/APIC	异常、中断、栈切换	syscall、timer、IRQ、IPI、调度
ready/valid/队列	异步通信和背压	DMA ring、completion、等待队列

因此，后面讲 OS 时要持续追问：这个抽象最后落在哪个硬件状态上？谁能改它？什么时候采样？旧副本在哪里？失败时哪个入口拿到证据？能回答这些问题，才算把硬件和 OS 真正合起来学。

硬件章节验收

- 能从 NAND/NOR 说明为什么逻辑门能组合成任意布尔函数。
- 能画出半加器、全加器，并解释 64 位 ripple-carry 的关键路径。
- 能说明 carry-lookahead 为什么用更多硬件换更短时延。
- 能画出 2 路 MUX，并解释控制信号如何选择下一 RIP、写回值或异常入口。
- 能把 `cmp rax, rbx` 解释为减法路径和 RFLAGS 更新。
- 能说明寄存器的 setup/hold/clock-to-Q 如何限制时钟周期。
- 能区分组合逻辑、时序逻辑、控制信号、数据线和时钟边沿。

- 能把寄存器文件、ALU、MUX、RIP 更新连成一条 `add rax, rbx` 数据通路。
- 能把 `mov rax, qword ptr [rbx + 16]` 追踪到 AGU、D-TLB、D-cache、写回和 page fault 路径。
- 能解释单周期、多周期、流水线 CPU 的时延取舍。
- 能说明 forwarding、stall、flush、ROB 和 precise exception 分别解决什么问题。
- 能画出 x86-64 事件入口为什么要查 IDT、切 TSS 栈、压返回帧。
- 能解释 TLB 为什么不是页表本体，以及页表修改后为什么要 invalidate。
- 能把 ready/valid、队列和 backpressure 映射到 DMA ring、completion 和等待队列。

从电路到操作系统：组成原理的最小主线

很多读者第一次学操作系统时，会被一组硬件词汇挡住：CPU、寄存器、总线、MMU、DMA、APIC、IOMMU、cache、TLB、特权级。若只把这些词当作背景知识，后面的系统调用、虚拟内存、调度和驱动都会像悬空的规则。本节把计算机组成原理和操作系统接起来：先从最小数字电路开始，说明 CPU 和设备为什么这样设计，再解释 OS 为什么必须依赖这些硬件机制。

核心思想

操作系统不是硬件之外的魔法层。它是运行在受保护 CPU 模式下的一段程序，利用中断、异常、页表、定时器、总线、设备寄存器和 DMA，把原始硬件状态组织成进程、地址空间、文件和 socket。

从开关到位：硬件最初解决什么问题

最底层的电子计算机不是从“进程”开始的，而是从可控电信号开始的。晶体管可以近似看作一个由输入电压控制的开关。开关组合起来可以实现逻辑门，逻辑门组合起来可以实现加法器、比较器、选择器和译码器。把电压高低抽象成 1 和 0，就得到数字电路的基本单位：bit。

层次	解决的问题	向上一层暴露的抽象
晶体管	电信号是否导通	可控开关
逻辑门	布尔函数如何计算	AND、OR、NOT、XOR
组合逻辑	输入如何变成输出	加法器、比较器、译码器、MUX
触发器	如何记住一个 bit	时钟边沿采样的状态
寄存器	如何保存一组 bit	机器字、地址、临时值
有限状态机	如何按步骤推进	当前状态、输入、下一状态

组合逻辑没有记忆。给定输入，它经过门延迟后产生输出；输入一变，输出也会变。若只靠组合逻辑，机器不能记住“上一步执行到哪里”。触发器和寄存器引入记忆：在时钟边沿，把输入采样到内部状态，直到下一次时钟边沿才改变。时钟让复杂电路分成离散步骤，状态机让机器有“现在处于哪一步”的概念。

一个最小计数器已经包含操作系统后来要关心的三个事实：状态存在哪里，什么时候改变，谁能改变。计数器的状态在寄存器里；时钟边沿触发改变；使能信号决定是否更新。把这个思想放大，就得到 CPU 的程序计数器、寄存器文件、状态寄存器和控制逻辑。

```
on each clock edge:
    if reset:
        counter = 0
    else if enable:
        counter = counter + 1
```

这段伪代码看起来像软件，实际对应硬件状态机。操作系统学习中反复出现的“状态机”不是写作技巧，而是硬件和内核共同的基本形态：硬件在时钟和事件下推进，内核对象在系统调用、中断和锁保护下推进。

电平、采样和复位：为什么状态不是自然可靠的

把高电平叫 1、低电平叫 0，只是第一层抽象。真实电路里信号有上升沿、下降沿、传播延迟、噪声和亚稳态。数字系统要可靠工作，必须规定什么时候采样、采样前信号必须稳定多久、采样后信号要保持多久。于是出现时钟、建立时间、保持时间和复位。

概念	硬件含义	OS 为什么要知道
时钟边沿	寄存器在固定时刻采样输入	CPU 状态按离散步骤推进，异常能落在边界上
复位	把寄存器送到已知初值	上电后从固定 reset vector 或固件入口开始
ready/valid	双方确认数据是否有效	总线、DMA 队列、设备 completion 都用类似握手
超时	预期事件长期不来	驱动不能无限等设备，必须 reset 或报错
亚稳态	异步信号采样不稳定	中断、外设输入要同步，硬件和驱动都要容忍边界情况

这解释了为什么“启动”不是直接运行内核 main。CPU、DRAM、时钟、PCIe、APIC、IOMMU 和设备都需要从未知电气状态进入可编程状态。固件和早期内核初始化做的很多事，本质是让硬件寄存器有可解释的初值，让中断、DMA 和内存访问不再处于随机状态。

```
reset path sketch:
    hardware reset asserts known register defaults
    CPU starts from reset vector
    firmware trains DRAM and discovers platform
    boot code enters protected long mode
    kernel sets IDT, page tables, stacks, APIC, timers
    only then can normal traps, syscalls and scheduling work
```

如果没有复位和初始化边界，OS 无法判断某个设备寄存器里的 bit 是旧状态、随机噪声还是有效完成事件。驱动里的 probe、reset、quiesce、timeout、resume 不是附加流程，而是把硬件状态重新拉回可推理状态的工程手段。

控制信号：指令不是被“解释”，而是打开一组开关

CPU 取到指令后，译码器不会在芯片里运行一段高级语言解释器。更准确的说法是：opcode 和寻址模式被译码成一组控制信号，控制 MUX 选哪路输入、寄存器文件读写哪些端口、ALU 做什么、是否访问内存、是否更新 RIP、是否进入异常路径。

控制信号	作用	例子
reg_read	选择源寄存器编号	add rax, rbx 读 rax/rbx

alu_op	选择 ALU 操作	add、sub、and、compare、address generation
mem_read/write	是否访问数据内存或 MMIO	load/store、栈 push/pop
writeback_src	写回来自 ALU、内存还是立即数	<code>mov rax, qword ptr [rbx]</code> 来自内存
next_rip_src	下一 RIP 来自顺序、分支、异常或返回	branch、syscall、iretq
privilege_check	当前 CPL 是否允许该动作	用户态执行特权指令触发异常

一条 load 指令可以拆成更小的微操作。下面不是 x86-64 的真实微码，只是帮助读者把“指令语义”落到硬件动作：

```
micro-ops for mov rax, qword ptr [rbx + 16]:
uop0: base = read_register(rbx)
uop1: va = base + 16
uop2: pa = translate_and_check(va, READ, cpl)
uop3: data = load64(pa)
uop4: write_register(rax, data)
```

这条拆解直接通向 OS。若 uop2 失败，进入 page fault；若 uop3 命中 MMIO 地址，读操作可能有设备副作用；若 uop4 还没提交时发生异常，CPU 必须保证软件可见状态仍能恢复。指令不是一句文本，而是一组受权限、地址翻译和提交规则约束的硬件动作。

信号到状态：一条指令为什么能被中断

初学时常把 CPU 看成“正在执行某条指令”。硬件上更准确的说法是：CPU 在每个周期推进一组寄存器和流水线状态，指令只是这些状态转移的编码来源。只要状态转移能在某个边界停住，硬件就能在这个边界接收异常或中断。

```
architectural boundary:
before boundary:
    previous instructions have committed
at boundary:
    RIP/RSP/RFLAGS/general registers have defined values
after boundary:
    CPU may either fetch next instruction or enter an event vector
```

这解释了三个看似不同的 OS 机制。第一，定时器中断要等到可恢复边界交付，否则内核无法保存一个一致现场。第二，page fault 要把 faulting RIP 和错误码交给内核，否则内核不知道修复哪个访问。第三，上下文切换保存的是架构状态，不是流水线里每个临时信号。CPU 内部可以有复杂乱序执行，但对 OS 暴露出来的必须是可定义、可保存、可恢复的状态点。

硬件边界	保存或检查什么	OS 依赖它做什么
指令提交	目标寄存器、RFLAGS、内存写入是否生效	精确异常、调试、信号恢复
异常入口	RIP、错误码、访问类型、当前 CPL	page fault、非法指令、保护错误处理
中断入口	被打断 RIP/RFLAGS 和必要寄存器	抢占调度、设备 completion、IPI
系统调用入口	用户返回地址、参数寄存器、RFLAGS mask	受控进入内核并返回用户态

因此，“CPU 支持中断”不是多了一根线那么简单。它要求 CPU 能把异步事件推

迟到安全边界，能阻止更低特权级伪造入口，能在进入内核时留下足够证据，能在返回时恢复用户态可见状态。后面讲调度器时，说“时钟中断触发调度”，背后就是这套边界。

地址译码不是数组下标：从芯片选择到总线事务

内存数组模型能帮助入门，但硬件里的地址首先进入译码逻辑。译码器会根据高位地址和平台配置决定本次访问命中哪类目标：DRAM、ROM、local APIC、PCIe MMIO window、显存窗口，或者根本没有设备响应。这个过程决定了同一条 `mov rax, qword ptr [rbx]` 是普通读内存，还是读一个有副作用的设备寄存器。

```
physical_access(pa, kind):
    if pa in DRAM ranges:
        send transaction to memory controller
    else if pa in local APIC range:
        access interrupt controller register
    else if pa in PCIe BAR range:
        send MMIO transaction to device
    else:
        report machine or bus error by platform rules
```

OS 必须在启动时拿到这张物理地址地图，并把它变成自己的资源管理结构。可用 DRAM 进入页分配器；固件保留区不能分配；PCIe BAR 只能映射给驱动；APIC 寄存器只能在内核里访问。若把 MMIO 当作普通 RAM 分给用户进程，用户态就能控制设备；若把固件保留区当作普通页，内核可能覆盖 ACPI 表或设备保留内存。

地址译码结果	硬件动作	内核必须建立的规则
DRAM	可缓存普通内存事务	页分配、页表映射、NUMA 归属
ROM/固件保留 APIC/MMIO	只读或保留区域 设备寄存器事务	不进入普通页分配器 只给驱动映射，禁止用户态直接访问
PCIe BAR	发给 PCIe endpoint	probe 后由匹配驱动管理生命周期
不存在地址	平台错误或返回特殊值	不把未知范围当作可用资源

这也是为什么“物理地址”仍然需要权限和类型。页表不仅决定虚拟地址映射到哪里，还要设置 `cacheability`、可执行性、用户可访问性等属性。普通 DRAM 用 `write-back cache`；设备寄存器通常要用不可缓存或设备类型；`framebuffer` 可能用 `write-combining`。属性错了，CPU 可能把设备寄存器访问合并、缓存或重排，设备协议就会被破坏。

从轮询到中断再到 DMA：I/O 不是一个函数调用

最简单的设备访问是轮询：CPU 一直读状态寄存器，直到设备 `ready`，再读写数据寄存器。这种模型容易理解，但浪费 CPU，并且每个字节都经过 CPU。中断把“什么时候 `ready`”交给设备通知，DMA 把“大块数据搬运”交给设备或 DMA engine，CPU 只负责授权、发布描述符和处理 `completion`。

阶段	CPU 做什么	OS 新增责任
轮询 PIO 中断 I/O	循环读状态和数据寄存器 提交命令后睡眠或运行别的任务	防止长时间忙等，处理超时 保存中断现场、确认设备、唤醒等待者
DMA I/O	建立 <code>buffer</code> 映射，设备直接读写内存	<code>pin/unmap</code> 、IOMMU、 <code>cache</code> 一致性、生命周期


```
while true:
    inst = memory[rip]
    rip = rip + instruction_length
    decode inst
    execute inst
```

这就是 CPU 最小执行循环。它需要至少四类状态：

- **程序计数器**：下一条指令在哪里。
- **寄存器文件**：当前计算的临时值、地址和参数。
- **状态寄存器**：条件码、中断使能、当前模式等控制位。
- **内存接口**：从地址读取指令和数据，把结果写回内存。

存储程序模型引入了一个强大的统一视角：程序是内存中的字节，数据也是内存中的字节，CPU 根据指令格式解释这些字节。后面的 ELF 装载、地址空间、页表权限、代码段只读可执行、数据段可写不可执行，都是在管理这组字节的含义和权限。

ISA 和微架构：程序员看到的机器与芯片内部的机器

学习 CPU 时必须区分 ISA 和微架构。ISA 是指令集架构，是软件可见的契约：有哪些寄存器，指令如何编码，内存访问怎么表达，异常如何报告，系统调用如何进入内核。微架构是芯片内部怎样实现这个契约：流水线几级，是否乱序执行，cache 多大，分支预测怎么做，指令如何拆成微操作。

概念	例子	OS 关心方式
ISA	x86-64	系统调用 ABI、异常入口、页表格式、Ring 0/3 特权级
微架构	流水线、乱序、cache、预测器	性能、屏障、侧信道、上下文切换成本
ABI	参数寄存器、栈对齐、返回值	用户态和内核态、程序和库之间的二进制契约

同一个 x86-64 程序可以在不同厂商、不同代际 CPU 上运行，是因为这些 CPU 都承诺实现同一套软件可见 ISA。它们内部可能完全不同：有的能同时发射更多微操作，有的分支预测更强，有的 cache 更大。OS 既要遵守 ISA 契约，又要理解微架构成本。例如页表格式来自 ISA，TLB shutdown 成本和 cache 抖动则来自微架构和多核实现。

数据通路：一条整数加法如何变成硬件动作

把 CPU 想象成“会执行代码的盒子”太粗。更有解释力的方式是看数据通路。以 Intel 语法的 `add rax, rbx` 为例，CPU 需要从寄存器文件读 `rax/rbx`，送入 ALU，加法结果写回 `rax`。控制逻辑根据 opcode 产生读寄存器编号、ALU 操作码、写回使能等信号。

```
decode add rax, rbx:
    a = regfile.read(rax)
    b = regfile.read(rbx)
    y = alu.add(a, b)
    regfile.write(rax, y)
```

再看 `mov rax, qword ptr [rbx + imm]`。它不仅用地址生成单元计算地址，还要访问内存层次，并可能触发缺页异常。若地址合法且 cache 命中，结果很快写回寄存器；若 TLB 未命中，硬件会执行页表遍历；若页不存在或权限不对，CPU 不能继续执行这条指令，必须进入异常入口。

```

decode mov rax, qword ptr [rbx + imm]:
    va = regfile.read(rbx) + imm
    pa = translate(va, READ, current_privilege)
    if translation_fault:
        raise page_fault
    data = cache_or_memory_read(pa)
    regfile.write(rax, data)

```

这说明缺页异常不是 OS 主动“检查每次内存访问”。硬件在普通指令执行途中发现不能完成架构语义，于是转入内核。虚拟内存的速度和安全性都来自这个硬件检查路径。

为什么需要流水线

如果 CPU 完整执行完一条指令才开始下一条，许多硬件部件会空闲。流水线把取指、译码、执行、访存、写回分成阶段，不同指令同时处在不同阶段，就像工厂流水线上不同工位同时处理不同产品。

```

cycle 1: inst1 fetch
cycle 2: inst1 decode    inst2 fetch
cycle 3: inst1 execute    inst2 decode    inst3 fetch
cycle 4: inst1 memory    inst2 execute    inst3 decode
cycle 5: inst1 writeback inst2 memory    inst3 execute

```

流水线带来吞吐，也带来 hazard。数据 hazard 是后一条指令需要前一条尚未写回的结果；控制 hazard 是分支决定下一条 RIP；结构 hazard 是两个阶段争同一硬件资源。硬件用转发、暂停、预测和乱序执行缓解这些问题。

OS 为什么关心流水线？第一，中断和异常必须是精确的：CPU 要能报告“哪条指令出错，之前的指令已经提交，之后的指令没有对架构状态产生影响”。没有精确异常，内核无法可靠处理 page fault 和信号。第二，分支预测、乱序执行和缓存会留下微架构痕迹，安全隔离必须考虑侧信道。第三，内存屏障的意义来自 CPU 可能为了性能重排某些可见顺序。

从单核到多核：并行不是免费加速

单核 CPU 同一时刻只执行一个控制流。多核把多个 CPU 核放在同一系统里，让多个线程可以真正同时运行。多核增加吞吐，也把“共享内存”变成硬件协议问题：同一个物理地址的 cache line 可能同时存在于多个核心的 cache 中，硬件一致性协议要保证普通读写能以某种规则传播。

机制	解决的问题	OS 设计影响
cache coherence	多核缓存同一地址如何保持一致	锁竞争、伪共享、per-CPU 数据
atomic RMW	读改写不能被其他核插入	自旋锁、引用计数、无锁队列
IPI	CPU 之间发送控制事件	重调度、TLB shutdown、停止其他 CPU
NUMA	内存到不同 CPU 距离不同	CPU affinity、first-touch、页迁移

多核系统里，关本核中断只能阻止本核被中断处理函数打断，不能阻止另一个核访问同一数据。因此从单片机临界区演进到 SMP 内核时，必须引入自旋锁、原子操作、内存屏障和 per-CPU 数据结构。调度器也不能只维护一个全局队列，因为全局队列会变成锁和 cache line 热点。

内存芯片、地址线和最早的内存模型

CPU 要读写内存，最朴素的硬件接口是地址线、数据线和控制线。CPU 把地址放到地址线上，拉高读信号，内存芯片把对应数据放到数据线上；写入时 CPU 把地址

和数据放好，再拉高写信号。真实现代内存控制器复杂得多，但这个模型能解释地址、总线和 MMIO 的根。

```
memory_read(address):
    address_bus = address
    control.read = 1
    wait until memory_ready
    return data_bus

memory_write(address, value):
    address_bus = address
    data_bus = value
    control.write = 1
    wait until memory_ready
```

若某个地址范围接的是 RAM，读写就是普通内存。若某个地址范围接的是设备寄存器，读写就变成控制硬件。这就是 memory-mapped I/O 的根本含义：CPU 使用同一套 load/store 机制访问内存和设备，但设备寄存器有副作用，不能当普通内存优化。

总线为什么出现：多个部件如何共享通信通道

当系统里只有 CPU 和一片内存时，连线可以很直接。加入定时器、串口、磁盘控制器、网卡、显示控制器后，若每个部件都和 CPU 单独连一套线，系统会失控。总线把地址、数据和控制协议标准化，让多个主设备和从设备共享通信通道。

总线概念	含义	OS 可见影响
地址译码	某个地址范围属于哪个设备	ACPI、PCI BAR、MMIO 映射
仲裁	多个发起者谁先使用总线	DMA 与 CPU 竞争带宽
事务	一次读写请求和响应	posted write、读回确认、错误状态
中断消息	设备如何通知 CPU	IRQ、MSI/MSI-X、中断亲和性

早期系统常有前端总线、南北桥、独立 I/O 总线；现代系统把内存控制器、PCIe root complex、片上互联和 I/O MMU 集成得更深。拓扑变了，OS 面对的问题没变：发现设备、映射寄存器、分配中断、建立 DMA 地址空间、处理错误和热插拔。

MMIO 和普通内存的根本区别

MMIO 地址看起来也是地址，但读写语义不同。读一个状态寄存器可能清除某个 pending 位；写一个 doorbell 寄存器可能让设备开始读取描述符；连续两次写寄存器的顺序可能是设备协议的一部分。编译器不能删除“看似没用”的 MMIO 读，CPU 不能随意把 doorbell 写提前到描述符写之前。

```
// 正确顺序：先准备内存中的命令，再通知设备
desc.addr = dma_address
desc.len = length
desc.flags = READ
dma_wmb()
writel(queue_tail, dev.DOORBELL)
```

这里的 dma_wmb 不是装饰。它表达“设备看到 doorbell 之前，必须能看到描述符内容”。如果缺少屏障，CPU 或互联可能让设备先收到通知，再读到半初始化描述符。驱动 bug 往往不是每次复现，因为顺序竞态受 CPU、互联、cache 和负载影响。

DMA 为什么出现：CPU 不应该搬每一个字节

PIO 模型里，CPU 亲自从设备寄存器读字节再写到内存，或者反过来。少量控制数据可以这么做，大量磁盘块、网络包、音视频数据不能这么做，否则 CPU 大量

时间浪费在搬运上。DMA 让设备成为总线主设备，直接读写内存；CPU 只负责准备 buffer、描述符和地址映射。

```
pio_read_block():
    for i in 0..block_size-1:
        memory[buf + i] = read_device_data_register()

dma_read_block():
    desc.addr = dma_map(buf)
    desc.len = block_size
    desc.cmd = READ
    dma_wmb()
    writel(desc_index, dev.DOORBELL)
    sleep until completion interrupt
```

DMA 的性能来自异步和批量，复杂性也来自异步和批量。提交 DMA 后，buffer 的所有权暂时交给设备；completion 前不能释放、复用或移动。设备写入内存后，CPU cache 可能需要失效才能看到新数据。若系统有 IOMMU，内核还要为设备建立 DMA 地址到物理页的映射；若没有 IOMMU，设备 bug 或恶意设备可能写坏任意内存。

DMA 阶段	内核必须做什么	常见失败
准备 buffer	分配或 pin 页，确认方向和长度	用户页被换出或提前释放
建立映射	<code>dma_map</code> 或 IOMMU map	地址超出设备 DMA mask
发布描述符	写描述符并执行 DMA 屏障	设备读到旧描述符
等待完成	中断、轮询或 completion 队列	丢中断、超时、中断风暴
回收资源	unmap、释放 buffer、唤醒等待者	use-after-free、重复 completion

理解 DMA 后，块层、网络栈和 `io_uring` 的很多设计就不再神秘。它们都在围绕同一件事组织队列：把 CPU 上的请求变成设备能批量消费的描述符，把设备完成再变成内核和用户态能理解的 completion。

为什么需要中断控制器

设备事件不能全靠 CPU 轮询，否则空闲时浪费 CPU，高负载时又容易响应不及时。中断让设备在事件发生时通知 CPU。但设备很多、CPU 也可能很多，系统需要一个控制层决定：哪个设备的事件送到哪个 vector，哪个 CPU 接收，优先级如何，中断是否屏蔽，处理完成后如何确认。

```
device raises interrupt
-> interrupt controller records pending event
-> controller selects target CPU and vector
-> CPU finishes current instruction boundary
-> CPU enters interrupt gate
-> kernel handler reads device status
-> kernel sends EOI
```

在 x86-64 上，本地 APIC、IOAPIC、x2APIC、MSI/MSI-X 和 IPI 共同完成这些工作。OS 关心的问题包括中断投递、屏蔽、优先级、亲和性、结束通知和错误统计。

特权级不是口号：它是 CPU 执行规则的一部分

“特权”不是说内核更重要，而是 CPU 在硬件上维护的一组模式位和访问规则。当前模式决定哪些指令能执行、哪些寄存器能改、哪些地址能访问、哪些异常入口能跳转。用户态不能随便关中断、改页表、写控制寄存器、访问设备 MMIO，也不能跳到任意内核地址。

层级	x86-64 角色	说明
Ring 3	用户程序	不能执行特权指令，不能访问 supervisor-only 页 和 内核 MMIO
Ring 0	内核	可以配置页表、中断、MSR、设备资源和调度状态
VMX root	hypervisor	可运行和控制 guest，拦截敏感事件并管理二级地址翻译

特权级解决的是“不可信程序不能直接控制整台机器”。没有它，一个普通程序可以改页表读取别的进程，可以关中断让调度器失效，可以写磁盘控制器覆盖文件系统，可以改定时器破坏时间。OS 的安全边界首先依赖 CPU 拒绝这些操作。

从没有保护到有保护：一步一步推出来

最小单片机通常没有用户态/内核态之分。所有代码同权，任何函数都能访问所有寄存器和内存。这在单一固件里可接受，因为所有代码由同一开发者构建，故障域就是整台设备。一旦机器要运行多个程序，问题立即出现。

1. 多个程序共享内存：程序 A 写坏程序 B 的数据，系统无法定位和阻止。
2. 程序可能死循环：若没有定时器抢占，其他程序永远得不到 CPU。
3. 程序直接碰设备：一个程序可以重置网卡、覆盖磁盘、窃取键盘输入。
4. 程序能改控制寄存器：它可以关闭隔离机制本身。

于是硬件逐步增加四类保护能力。第一是特权级，限制敏感指令。第二是异常入口，让非法操作转入内核处理，而不是让机器随机继续。第三是 MMU 和页表，为每个程序定义可访问地址范围和权限。第四是定时器中断，让内核即使在程序不合作时也能收回 CPU。

失败	硬件机制	OS 抽象
程序互相写内存	MMU、页表、权限位	进程地址空间
程序执行特权操作	privilege mode、illegal instruction trap	系统调用和权限检查
程序长期占 CPU	timer interrupt	抢占式调度
程序直接访问设备	MMIO 只映射给内核、IOMMU	驱动、fd、设备文件

这条演进线很重要。特权级、页表、中断和系统调用不是先背定义再硬套，它们分别修复了无保护多程序系统中的具体失败。

系统调用为什么不能只是函数调用

函数调用只是在当前特权级内跳到另一个地址。若用户程序能直接调用内核内部函数，就必须允许它跳进内核地址、使用内核栈、传入任意指针并修改内核状态。这等于没有隔离。系统调用的本质是受控陷入：用户程序只能从固定入口进入，CPU 自动切换到更高特权，内核保存用户上下文，检查参数和权限，再返回用户态。

```

user code:
    put syscall number and args in ABI registers
    execute syscall/trap instruction

hardware:
    verify transition target
    
```



```

switch privilege mode
save return RIP and RFLAGS
jump to kernel entry

kernel:
switch to kernel stack
build trap frame
validate args and permissions
run syscall implementation
restore user state and return

```

这解释了为什么系统调用有成本，也解释了为什么 `vDSO`、`io_uring`、批量系统调用和共享内存队列能优化某些路径：它们不是绕开安全，而是在不破坏边界的前提下减少陷入次数或把安全检查前移。

MMU 的演进：从基址界限到多级页表

最简单的内存保护可以是基址和界限寄存器：程序访问地址时，硬件检查地址是否落在 `base <= addr < base + limit`。这能阻止越界，但无法高效表达稀疏地址空间、共享库、按页换入换出、不同页面不同权限。页表把地址空间切成固定大小的页，每页有独立映射和权限。

```

translate(virtual_address, access):
    vpn = virtual_address / PAGE_SIZE
    off = virtual_address % PAGE_SIZE
    pte = page_table[vpn]
    if !pte.present:
        raise page_fault
    if access not allowed by pte.permissions:
        raise protection_fault
    return pte.physical_page * PAGE_SIZE + off

```

多级页表是空间优化。64 位虚拟地址空间很大，若为每个虚拟页都准备一项线性页表，绝大多数项会浪费。多级页表只为实际使用的地址区间分配下级表。TLB 则是时间优化：页表遍历太慢，所以 CPU 缓存最近的地址翻译。

页表能力	直接效果	OS 用途
present 位	页面是否有物理页	懒分配、文件映射、换页
R/W/X 权限	读写执行控制	代码只读、栈不可执行、COW
user/supervisor	用户态是否可访问	用户/内核地址隔离
dirty/accessed	是否写过或访问过	回收、writeback、老化算法
global/PCID	TLB 保留和地址空间标记	降低上下文切换成本

页表不是“虚拟地址到物理地址的 map”这么简单。它是隔离、懒加载、共享、权限、安全和性能共同使用的硬件数据结构。

cache 和 TLB：为什么抽象有成本

内存比 CPU 慢得多，页表遍历也比寄存器操作慢得多。cache 缓存数据和指令，TLB 缓存地址翻译。它们让程序跑得快，也让 OS 付出维护成本。切换地址空间时，旧 TLB 项不能错误用于新进程；修改页表后，其他 CPU 的旧翻译可能仍在 TLB 中；共享数据频繁写入时，cache line 会在 CPU 间来回迁移。

```

unmap_page(process, va):
    remove PTE from page table
    for each cpu running this address space:
        send IPI to invalidate TLB entry
    wait for acknowledgements

```

only now free physical page

如果跳过 TLB shutdown, 某个 CPU 可能继续用旧翻译访问已经释放的物理页。若这页后来分配给内核对象或另一个进程, 隔离就被破坏。很多 OS 机制看起来“为什么这么麻烦”, 答案往往在 cache/TLB 和多核一致性里。

精确异常：OS 能恢复执行的前提

当一条指令触发缺页或非法操作时, 内核需要知道它是否可以修复并返回继续执行。例如用户访问尚未分配但合法的栈页, 内核可以分配物理页、更新 PTE, 然后返回让同一条 load/store 重试。要做到这一点, CPU 必须提供精确异常: 异常点之前的指令已经按顺序提交, 异常点之后的指令没有对架构状态产生影响。

```
user instruction stream:
  inst1 committed
  inst2 committed
  inst3 causes page fault <-- saved RIP points here
  inst4 not architecturally committed

kernel:
  inspect fault address and error code
  fix mapping or deliver signal
  return to inst3 if fixed
```

乱序 CPU 内部可能已经执行了 inst4 的某些微操作, 但在架构层必须让 OS 看到一个干净边界。没有这个契约, page fault、debug exception、signal、single step 和进程恢复都难以实现。

从芯片到主板：固件为什么存在

真实机器不是只有 CPU 和内存。还有 DRAM 初始化、时钟、PCIe 拓扑、ACPI 表、IOMMU、显卡、存储控制器、键盘、TPM、电源管理和多核启动。CPU 上电时, 很多硬件尚未处在 OS 可直接使用的状态。固件负责完成最早期平台初始化, 并把硬件描述交给 bootloader 或内核。

```
power on
-> reset vector
-> firmware initializes DRAM and chipset
-> firmware discovers platform tables
-> bootloader or EFI stub loads kernel
-> kernel receives memory map, initrd, command line, ACPI tables
-> kernel takes over devices and memory management
```

固件不是万能可信源。内核仍要校验内存地图、保留区、ACPI 描述和启动参数。若固件把内核占用页标为空闲, 页分配器会覆盖正在运行的内核; 若设备中断号描述错误, 驱动可能永远收不到 completion; 若 NUMA 拓扑错误, 调度和页分配会做出错误局部性判断。

把组成原理映射到 OS 章节

从这一节开始, 后面的每个 OS 机制都应能反查到硬件根。

硬件事实	推出的 OS 机制	后续章节
CPU 有 RIP、寄存器和栈	上下文保存、线程切换	调度与并发
CPU 有特权级	系统调用、内核态、用户态	特权边界
定时器可异步打断 CPU	抢占式调度、超时	中断与调度
MMU 按页检查权限	地址空间、缺页、COW	虚拟内存

cache/TLB 有维护成本	维 per-CPU、TLB shutdown、屏障	调度、内存、锁
设备通过 MMIO 暴露寄存器	驱动、设备文件、控制面	设备与 I/O
设备可 DMA 访问内存	描述符环、IOMMU、completion	块层和网络
存储设备有缓存和 flush	fsync、日志、崩溃恢复	文件系统和持久化

因此，本册合并讲组成原理和 OS 时，不追求把所有芯片细节都展开成硬件设计课，而是抓住 OS 必须依赖的组成原理主干：计算如何执行，状态如何保存，地址如何翻译，设备如何通信，中断如何进入内核，保护如何由硬件强制执行。

最小验收：概念必须能落到机制

读完本节后，下面这些问题应该能用自己的话讲清楚。

- CPU 为什么需要程序计数器、寄存器、状态寄存器和内存接口。
- ISA 和微架构有什么区别，为什么 OS 同时关心二者。
- load/store 指令如何经过地址翻译、权限检查、cache 和异常路径。
- 特权级到底限制了什么，为什么系统调用不是普通函数调用。
- 页表为什么不只是 map，而是隔离、懒加载、共享和权限的共同根。
- MMIO 为什么不能当普通内存，DMA 为什么需要生命周期、屏障和 IOMMU。
- 定时器、中断控制器和精确异常如何让内核收回控制权并恢复执行。
- cache、TLB、NUMA 和多核一致性为什么会影晌调度、锁和页表修改。

若这些问题仍然说不清，后面看到进程、虚拟内存、驱动和文件系统时，应回到本节重新把硬件路径画出来。OS 的每个抽象都不是凭空定义，而是从硬件限制、失败场景和可验证机制中长出来的。

从零搭一个 x86-64 教学子集：先能手算，再谈 OS

前面讲了从电路到 OS 的大方向，但真正理解硬件，不能只读概念。我们在这一节搭一个极小但完整的 x86-64 教学子集。它不追求真实 x86-64 的全部复杂度，而是保留所有关键骨架：bit、字节、机器字、内存地址、RIP、RSP、通用寄存器、Intel 语法指令、取指译码执行、栈、中断、Ring 0/3 和地址保护。只要这台小机器能讲清，后面再看 Linux x86-64，就只是把同一组机制放大。

核心理想

先不要问“Linux 怎么做”。先问：如果只有一块内存、几个寄存器和一组指令，程序如何运行？程序如何等待设备？程序出错如何停住？多个程序如何互不破坏？这些问题推出来的答案，就是 OS 的硬件根。

第一步：bit、byte、word 到底是什么

硬件里最小的稳定信息单位是 bit，它只有两个状态：0 或 1。8 个 bit 组成 byte。CPU 通常一次更喜欢处理固定宽度的一组 bit，例如 16 位、32 位或 64 位，这组 bit 叫 word。word 不是数学概念，而是机器一次自然处理的数据宽度。

单位	例子	含义
----	----	----

bit	0 或 1	一个开关状态、一个标志位、权限位
byte	01000001	最小寻址单位常是 1 byte，可表示字符或二进制片段
word	16/32/64 bit	CPU 算术、地址和寄存器常用宽度
cache line	常见 64 byte 量级	cache 一次搬运的数据块
page	常见 4KiB 起	MMU 和 OS 管理内存的基本单位

同一串 bit 可以被解释成不同含义。比如 01000001 可以是无符号整数 65，也可以是 ASCII 字符 A，也可以是某条指令的一部分。机器只保存 bit；含义来自解释规则。OS 做 ELF 装载、页权限、系统调用参数检查，本质上都在维护这些 bit 的合法解释方式。

第二步：数值表示为什么需要补码

无符号整数最简单：n 个 bit 表示 0 到 2^n-1 。8 位无符号数 11111111 是 255。问题是负数怎么表示。现代 CPU 通常用二进制补码：最高位有负权重。8 位补码中，11111111 表示 -1，10000000 表示 -128，01111111 表示 127。

```
8-bit unsigned:
00000000 = 0
00000001 = 1
11111111 = 255

8-bit two's complement:
00000000 = 0
00000001 = 1
01111111 = 127
10000000 = -128
11111111 = -1
```

补码的好处是加法电路统一。硬件不需要为有符号加法重做一套加法器。比如 $1 + (-1)$ ：

```
00000001
+ 11111111
=1 00000000 // 舍弃第 9 位进位，结果是 0
```

OS 为什么关心整数表示？因为长度、地址、偏移、权限位和计数器都可能溢出。系统调用里 `offset + length` 若不检查溢出，就可能绕过边界检查；页号移位若用错有符号类型，就会产生错误地址。底层表示不是数学细节，而是安全边界。

第三步：内存是一张按地址编号的字节表

先把内存想成一个数组：

```
memory[0] = byte
memory[1] = byte
memory[2] = byte
...
```

地址就是数组下标。若机器有 16 位地址，则最多能直接编号 $2^{16} = 65536$ 个 byte，也就是 64KiB。真实 64 位机器虚拟地址空间远大得多，但第一层理解就是“地址选中某个 byte”。

问题	朴素答案	后续复杂化
地址是什么	内存数组下标	虚拟地址、物理地址、DMA 地址会分开
读一个 word 怎么办	读连续多个 byte 组合	端序、对齐、cache line、缺页

写内存是什么	把 byte 放到地址位置	权限、dirty bit、写回 cache、DMA 一致性
非法地址怎么办	简单机器可能乱读乱写	MMU 触发异常，OS 决定修复或杀进程

多字节数值还涉及端序。小端机器把低有效字节放在低地址，大端机器相反。比如 32 位值 `0x12345678` 在小端内存中是：

```
address A+0: 0x78
address A+1: 0x56
address A+2: 0x34
address A+3: 0x12
```

端序影响二进制文件、网络协议、设备寄存器和调试内存转储。OS 读 ELF header、网络栈解析报文、驱动读设备描述符时，都必须知道字段的字节顺序。

第三步补充：一次内存访问要回答哪些问题

在最初模型里，`memory[address]` 像数组访问。真实机器上，即使是最普通的 load，也不是直接把数组下标交给内存芯片。CPU 必须先生成有效地址，再检查地址形式、翻译虚拟地址、检查权限、访问 cache 或内存，最后在没有异常时写回寄存器。

步骤	X64S 简化说法	真实 x86-64 会继续做什么
地址生成	<code>regs[base] + disp</code>	base/index/scale/disp 组合，检查 canonical address
权限判断	是否在允许范围	页表 U/S、R/W、NX、PKU/SMEP/SMAP 等权限
地址翻译	教学内存直接使用地址	TLB 命中或页表遍历，失败则 page fault
数据取得	读连续 byte 组合成 word	cache line、对齐、跨页、内存类型、MMIO 副作用
写回提交	更新目标寄存器	必须保证异常前后架构状态精确

对齐和跨页是理解内存访问的第一个坑。假设页大小是 `0x1000`，一条 8 字节 load 从 `0x0ffc` 开始，它会读 `0x0ffc..0x1003`，跨过页边界。前 4 字节所在页可能存在，后 4 字节所在页可能不存在。硬件不能把半个值写进 `rax` 再报错；架构上要么完整完成，要么在写回前产生异常。

```
load64 from 0x0ffc:
  bytes 0x0ffc..0x0fff are in page A
  bytes 0x1000..0x1003 are in page B
  if page B is not present:
    raise page fault before architectural writeback
```

这解释了为什么内核复制用户 buffer 不能只检查首地址。用户传入 `buf, len` 后，范围可能跨很多页；中间某页可能没有映射，某页可能只有读权限，某页可能在复制过程中被另一个线程 `munmap`。所以 `copy_from_user` 一类路径要能在每次 fault 后判断：能否修复，能否睡眠，是否应返回 `-EFAULT`。

第三步补充：地址空间不是一块连续平板

初学时把内存画成连续数组是必要简化，但进程地址空间不是一块实心平板。它是很多虚拟区间的集合：有代码段、只读数据、堆、文件映射、匿名映射、栈、guard page、vdso，也有大量没有映射的空洞。


```

process virtual address sketch:
text:      mapped, read+execute
rodata:    mapped, read only
data/heap: mapped, read+write
gap:       unmapped
mmap file: mapped, permissions by VMA
guard page: unmapped
stack:     mapped, grows down
kernel half: supervisor only

```

VMA 说明“这个范围应该是什么”，PTE 说明“这个页现在实际映射到哪里”。VMA 可以存在而 PTE 还不存在；这表示合法但尚未分配或尚未读入。PTE 可以暂时只读而 VMA 允许写；这表示写时复制。这个区分后面会反复出现：OS 不是只维护一个地址到物理页的 map，而是在维护“意图”和“当前实现状态”两层。

第四步：先定义一个 x86-64 教学 CPU

我们定义一个 x86-64 教学子集，名字叫 X64S。为了方便手算，它只开放少量指令和 64KiB 教学内存，但寄存器命名、栈语义、系统调用入口和中断返回都沿用 x86-64 主线。它有通用寄存器 `rax/rbx/rcx/rdx/rsi/rdi/rbp/rsp`，一个指令指针 `rip`，一个状态寄存器 `rflags`，一个当前特权级 `cpl`。

状态	初始解释	OS 关联
<code>rip</code>	下一条指令地址	中断、异常和上下文切换必须保存它 函数调用、系统调用、内核栈切换依赖它
<code>rsp</code>	当前栈顶地址	
<code>rax/rdi/ rsi/...</code>	通用计算寄存器	返回值、系统调用号、参数、临时状态
<code>rflags</code>	zero、carry、interrupt enable	分支、算术结果、中断开关
<code>cpl</code>	Ring 3 或 Ring 0	特权级的最小模型

这台机器是教学简化版，但它足够解释 OS。真实 x86-64 有更多寄存器、更复杂的 RFLAGS、控制寄存器、MSR、段/TSS/IST 和虚拟化状态；核心问题不变：机器状态必须能被保存、恢复、检查和保护。

第五步：定义几条指令

X64S 使用 x86-64 Intel 语法展示指令，但只选少量常用语义。真实 x86-64 编码有前缀、opcode、ModRM/SIB、位移和立即数；教学时先看语义。

指令	语义	用途
<code>mov reg, imm</code>	<code>reg = imm</code>	把常量放进寄存器
<code>mov reg, [base]</code>	<code>reg = memory[base]</code>	从内存读数据
<code>mov [base], reg</code>	<code>memory[base] = reg</code>	向内存写数据
<code>add dst, src</code>	<code>dst = dst + src</code>	算术计算
<code>sub dst, src</code>	<code>dst = dst - src</code>	比较和减法
<code>cmp a, b</code>	设置 zero/carry 等标志	条件分支前比较
<code>jmp label</code>	<code>rip = label</code>	无条件跳转
<code>jz label</code>	zero 为 1 时跳转	条件分支
<code>syscall</code>	进入 Ring 0 系统调用入口	用户请求内核服务

`iretq/sysretq` 返回较低特权级或快速 受控返回用户态
返回

真实机器会把 I/O 设计成端口 I/O 或 MMIO。本书主线使用 MMIO。教学重点是：指令不是魔法，它只是固定格式的 bit，译码器按 opcode 和寻址模式选择数据通路动作。

第六步：取指、译码、执行的完整循环

现在 X64S 能跑程序了。CPU 每次从 `rip` 指向的内存取指令字节，解码出指令长度和操作数，默认让 `rip` 前进到下一条指令；分支、异常和返回指令可以覆盖这个默认下一地址。

```
cpu_step():
    inst = decode(memory, rip)
    next_rip = rip + inst.length
    switch opcode:
        MOV_REG_IMM: regs[dst] = imm
        MOV_LOAD:    regs[dst] = read64(memory, regs[base] + disp)
        MOV_STORE:   write64(memory, regs[base] + disp, regs[src])
        ADD:         regs[dst] = regs[dst] + regs[src]
        CMP:         rflags.zero = (regs[a] == regs[b])
        JZ:          if rflags.zero: next_rip = target
    rip = next_rip
```

真实 CPU 可能因为流水线和预测更复杂，但架构语义仍要保证程序按正确顺序推进。中断和异常会保存 `rip`，所以 `fault` 保存当前指令还是 `interrupt` 保存下一条指令，是架构必须定义清楚的契约。

第七步：手跑一个程序

我们写一个程序，把内存 `0x2000` 和 `0x2008` 的两个 64 位数相加，结果写到 `0x2010`。

```
mov rbx, 0x2000
mov rax, qword ptr [rbx]      ; rax = memory[0x2000]
mov rbx, 0x2008
mov rcx, qword ptr [rbx]      ; rcx = memory[0x2008]
add rax, rcx
mov rbx, 0x2010
mov qword ptr [rbx], rax
hlt
```

假设初始内存：

```
memory[0x2000] = 7
memory[0x2008] = 5
```

执行结束后，`memory[0x2010] = 12`。这就是所有程序运行的根：指令改变寄存器，寄存器形成地址，内存保存数据，`rip` 决定下一步。OS 再复杂，也不能违反这个根模型。

为了避免“看懂代码但不会手算”，下面把每条指令执行后的关键状态列出来。假设每条教学指令长度记作 1，初始 `rip=0`，未列出的寄存器保持不变。

步	指令	执行后寄存器	执行后内存变化
0	初始	<code>rip=0</code>	<code>[0x2000]=7, [0x2008]=5</code>
1	<code>mov rbx, 0x2000</code>	<code>rbx=0x2000, rip=1</code>	无

2	mov rax, qword ptr [rbx+16]	rax=7,rip=2	无
3	mov rbx, 0x2008	rbx=0x2008,rip=3	无
4	mov rcx, qword ptr [rbx+16]	rcx=5,rip=4	无
5	add rax, rcx	rax=12,rip=5	无
6	mov rbx, 0x2010	rbx=0x2010,rip=6	无
7	mov qword ptr [rbx], rax	rip=7	[0x2010]=12

这张表训练的是 OS 读源码时的基本能力：每一步都问状态在哪里、谁改了它、失败时能否回滚。系统调用、page fault、调度切换只是把这张表中的“寄存器、内存、下一条指令”扩展成更多对象。

第七步补充：把指令编码成字节

只讲汇编文本仍然少了一层。CPU 不读取 `mov rax, qword ptr [rbx]` 这样的字符串，它读取内存里的字节。汇编器把文本翻译成机器码，链接器和装载器把这些字节放到可执行映射中，CPU 从 RIP 指向的地址取字节并译码。

为了能手算，X64S 使用固定长度教学编码。真实 x86-64 是变长编码，本书后面会讲 REX、opcode、ModRM、SIB、disp 和 imm；这里先抓住共同结构：opcode 决定操作，寄存器字段决定读写谁，位移和立即数字段提供常量。

字段	X64S 教学含义	真实 x86-64 对应
opcode	MOV_LOAD、ADD、SYSCALL	opcode byte 或 opcode map
dst/src mode	目标和源寄存器编号 寄存器、内存、立即数寻址	ModRM.reg、ModRM.r/m、REX 扩展位 ModRM.mod、SIB、rip-relative
disp	地址偏移	8/32 位 displacement
imm	指令内常量	immediate bytes, 小端编码

```
X64S teaching bytes for mov rax, qword ptr [rbx + 16]:
opcode = MOV_LOAD64
dst     = RAX
base    = RBX
disp    = 16

decode result:
operation = load64
target register = rax
effective address = rbx + 16
```

这层编码很重要，因为异常和调试都要回到字节地址。RIP 指向的不是“第几行汇编”，而是某个机器码字节。断点调试器把指令首字节临时改成断点指令；异常处理器用保存的 RIP 找到 faulting instruction；动态链接器修改 GOT/PLT 相关地址，让后续 call 走到正确函数。操作系统管理的是字节、页和权限，不是源代码行。

第七步补充：取指和数据访问有两条权限路径

同样是访问内存，取指和读写数据的权限并不完全一样。取指要求页面可执行；load 要求可读；store 要求可写；用户态还要求 U/S 位允许用户访问。于是代码页通常是 read+execute，不可写；栈和堆通常是 read+write，不可执行。

访问	检查的核心权限	OS 用途
fetch instruction	present、user、execute	NX、WX、代码段保护
load data	present、user、readable	普通读、copy_from_user、文件映射
store data	present、user、writable	写堆栈、COW fault、脏页跟踪
atomic RMW	read + write + 原子性	锁、引用计数、futex 快路径

这说明“某地址映射了”不是完整答案。一个页面可以存在但不可写，存在但不可执行，存在但只给 Ring 0 使用。后面讲 COW 时，父子进程共享同一物理页，但 PTE 暂时清 writable；第一次写触发 page fault，内核复制页面后再恢复写权限。后面讲安全时，栈不可执行和内核页不可用户访问也是同一类硬件权限检查。

```
if fetch from stack page and page.executable == false:
    raise page fault with instruction-fetch bit

if store to cow page and page.writable == false:
    raise page fault with write bit
    kernel may copy page and retry same store
```

第八步：为什么需要栈

没有栈，也能写直线程序；一旦需要函数调用、局部变量和返回地址，就需要一个按后进先出管理的区域。我们把 `rsp` 作为栈指针，约定栈向低地址增长。

```
push rax:
    rsp = rsp - 8
    memory[rsp] = rax

pop rax:
    rax = memory[rsp]
    rsp = rsp + 8

call addr:
    push next_rip
    rip = addr

ret:
    pop rip
```

栈解释了很多 OS 现象。每个线程为什么要有自己的栈？因为函数调用链、返回地址和局部变量不能混在一起。系统调用为什么要从用户栈切到内核栈？因为用户栈不可信，内核不能把自己的返回地址和局部状态放在用户可改的内存里。

第九步：内存布局从哪里来

程序不能随便把代码、数据和栈混在同一块区域。我们给 X64S 一个简单内存布局：

```
0x0000 - 0x0fff  kernel code
0x1000 - 0x1fff  user program code
0x2000 - 0x2fff  user data
0x3000 - 0x3fff  user stack
0xf000 - 0xffff  device registers
```

这张表一开始只是约定。若所有代码同权，用户程序仍然可以写 `0x0000` 覆盖

kernel code，也可以写 `0xf000` 操作设备。要把“约定”变成“保证”，需要硬件保护。OS 的复杂性就是从这里开始的。

第十步：轮询设备为什么会浪费 CPU

假设 X64S 有一个键盘设备，MMIO 状态寄存器在 `0xf000`，数据寄存器在 `0xf008`。状态 `bit0` 表示是否有字符。

```
keyboard_poll:
    mov rbx, 0xf000
loop:
    mov rax, qword ptr [rbx]          ; read status
    and rax, 1
    cmp rax, 0
    jz loop
    mov rbx, 0xf008
    mov rcx, qword ptr [rbx]          ; read character
```

这个程序功能上正确，但 CPU 在没有按键时一直空转。若系统只有一个任务，可以接受；若还要同时控制电机、刷新显示、处理网络，就不能让 CPU 卡在一个设备等待上。于是需要中断：设备有事件时打断 CPU，而不是 CPU 一直问设备。

第十一步：中断如何进入 CPU

给 X64S 增加中断机制。设备可以通过 APIC/MSI-X 路径设置 pending vector。CPU 在指令边界，如果 `rflags.IF = 1` 且有可投递中断，就保存当前返回状态，关闭同级中断，跳到 IDT 中登记的中断入口。

```
after each instruction:
    if rflags.IF and pending_interrupt:
        kernel_stack.push(rflags)
        kernel_stack.push(cs)
        kernel_stack.push(rip)
        rflags.IF = 0
        cpl = 0
        rip = idt[vector].handler
```

中断处理函数读取设备数据，清 pending，恢复 `rip/rflags`，返回被打断的程序。

```
irq_return:
    iretq restores rip, cs, rflags, rsp, ss when returning to Ring 3
```

这就是 trap frame 的雏形。真实 CPU 保存更多寄存器，也有更复杂的栈切换和优先级，但本质是一样的：硬件必须留下足够状态，让内核处理事件后能回到被打断的位置。

第十一步补充：IDT、TSS 和内核栈的关系

在 x86-64 上，异常和中断不是直接跳到任意地址。CPU 用 vector 查 IDT，IDT entry 指出 handler 地址、门类型和目标特权级。若事件从 Ring 3 进入 Ring 0，CPU 还要找到可信内核栈。这个栈信息来自 TSS 里的 `RSP0` 或 `IST` 配置。

```
ring 3 interrupt entry:
    vector = pending IRQ or exception number
    gate = IDT[vector]
    if gate target is ring 0 and current CPL is ring 3:
        new_rsp = TSS.rsp0 or selected IST stack
        switch to new_rsp
        push old SS, old RSP, RFLAGS, CS, RIP
    else:
        push frame on current privileged stack
    RIP = gate.handler
```

```
CPL = 0
```

为什么不能继续用用户栈？因为用户栈由用户程序控制。它可能未映射，可能接近 guard page，可能故意伪造内容。若内核把 trap frame 和局部变量压到用户栈，用户态就能破坏内核返回路径。因此每个线程必须有自己的内核栈：线程 A 在系统调用里睡眠时，它的内核调用链还在；线程 B 进入内核不能覆盖 A 的栈。

结构	保存什么	错误配置后果
IDT	vector 到 handler 的入口描述	page fault、timer IRQ 无法分发
TSS.RSP0	Ring 3 进入 Ring 0 时的栈	trap frame 写到错误位置或三重故障
IST	特殊异常专用栈	double fault、NMI 等无法可靠处理
per-task kernel stack	线程在内核里的调用链	睡眠线程被其他线程覆盖现场

这也是为什么“创建线程”不是只分配用户栈。内核还要准备内核栈、初始切换上下文、返回用户态所需的寄存器帧、页表归属和调度状态。没有这些，线程无法被中断、无法系统调用、无法在内核里阻塞后再恢复。

第十二步：定时器为什么能让 OS 抢回 CPU

中断不只来自键盘，也可以来自定时器。给 X64S 加一个每 1000 个周期触发一次的 local APIC timer IRQ。即使用户程序写了死循环，定时器仍会周期性把 CPU 带回内核。

```
user_bad_program:
    jmp user_bad_program

timer_irq:
    save current registers into current_task.context
    choose next runnable task
    restore next_task.context
    iretq
```

这就是抢占式调度的硬件根。没有定时器，OS 只能相信程序主动 `yield`；有了定时器，OS 可以在程序不合作时收回 CPU。调度器不是凭空切换线程，它依赖定时器中断、上下文保存和恢复。

第十三步：为什么需要用户态和内核态

现在 X64S 有中断了，但用户程序仍然可以写设备寄存器、改中断向量、覆盖内核代码。我们加入 `cpl`：Ring 3 表示用户态，Ring 0 表示内核态。再规定：

- 只有 Ring 0 能访问 `0x0000-0xffff` 的内核代码数据。
- 只有 Ring 0 能访问 `0xf000-0xffff` 的设备 MMIO。
- 只有 Ring 0 能修改 IDT、CR3、MSR 和保护配置。
- Ring 3 执行特权指令会触发异常。

```
memory_access(addr, type):
    if cpl == 3 and addr in kernel_or_device_range:
        raise protection_fault
    else:
        perform access
```

这一步把“不要乱写”变成“写不了”。操作系统最基本的安全性，不是靠应用自觉，而是靠 CPU 在每次敏感访问时强制检查。

第十四步：系统调用是受控入口

有了用户态/内核态，用户程序不能直接写设备，那它如何输出字符？答案是系统调用。X64S 使用 x86-64 的 `syscall` 指令。用户程序把系统调用号放进 `rax`，把参数放进 `rdi/rsi/rdx/r10/r8/r9`，执行 `syscall`；CPU 切到 Ring 0 入口；内核检查系统调用号和参数，再代表用户操作设备。

```
user:
    mov rax, 1          ; syscall number: write_char
    mov rdi, 'A'        ; argument
    syscall

hardware:
    save user rip/rflags into rcx/r11
    cpl = 0
    rip = syscall_entry_msr

kernel:
    if rax == write_char:
        check permission
        write device register
    return to user
```

现在可以看清：系统调用不是函数调用。函数调用不改变 `cpl`，不能绕过用户权限；系统调用由硬件提供受控入口，只有入口处的内核代码能获得特权。

第十五步：base/limit 是最小内存保护

先不引入复杂页表，给 X64S 教学模型每个用户程序两个界限寄存器：`BASE` 和 `LIMIT`。用户态访存时，硬件检查地址是否落在范围内。真实 x86-64 不用这种简单 `base/limit` 管普通用户地址空间，而是依赖页表和 U/S 权限位；这里先用它说明“硬件必须参与隔离”。

```
user_translate(addr):
    if addr >= LIMIT:
        raise protection_fault
    physical = BASE + addr
    return physical
```

这让每个程序以为自己从地址 0 开始，硬件把它平移到不同物理区域。程序 A 的 `addr=0x0100` 和程序 B 的 `addr=0x0100` 可以落到不同物理位置。它已经能实现基本隔离，但有明显局限：每个程序需要连续物理内存，不适合共享库，也不适合按需分配。

第十五步补充：base/limit 为什么会被页表取代

`base/limit` 的优点是硬件简单：一次比较和一次加法就能完成保护和重定位。但它把一个进程看成单个连续区间，这和真实程序的形态冲突。真实程序需要代码只读可执行、数据可写、栈向下增长、堆向上增长、共享库映射到任意位置、文件按需映射、`guard page` 捕获栈溢出。

需求	base/limit 的问题	分页的做法
稀疏地址空间	空洞也要占连续范围	未使用区间不分配页表下级或 PTE
不同权限	整段权限太粗	每页独立 R/W/X 和 U/S 权限

共享库	难把同一物理内容映射到多进程	多个 PTE 指向同一物理页
按需装载	必须提前准备连续物理内存	present=0, fault 时再分配或读文件
COW	整段复制成本高	PTE 只读共享, 写 fault 时复制页

这并不是说 base/limit 没有价值。它是理解“硬件参与边界检查”的第一块台阶。页表只是把同一个思想细化到页粒度, 并增加了稀疏性、共享、延迟分配和权限组合。现代 OS 的内存管理之所以强大, 是因为硬件每次访问都愿意替内核检查这一页当前是否允许这种访问。

第十六步：分页如何解决连续内存问题

分页把虚拟地址切成页号和页内偏移。假设页大小 256 byte, 虚拟地址高 8 位是虚拟页号, 低 8 位是偏移。页表记录每个虚拟页映射到哪个物理页, 以及权限。

```
translate(va, access):
    vpn = va >> 8
    off = va & 0xff
    pte = page_table[vpn]
    if !pte.present:
        raise page_fault
    if access not allowed:
        raise protection_fault
    return (pte.ppn << 8) | off
```

分页解决了三件事。第一, 虚拟连续不要求物理连续。第二, 每页可以有独立权限。第三, 不存在的页可以等访问时再分配或从文件读取。现代虚拟内存、COW、mmap、按需装载都从这里长出来。

第十六步补充：四级页表如何节省空间

真实 x86-64 常见 48 位 canonical 虚拟地址和 4KiB 页。4KiB 页内偏移需要 12 位, 剩下的高位用来索引多级页表。四级页表把虚拟地址拆成 PML4、PDPT、PD、PT、offset, 每级通常取 9 位, 因为一个 4KiB 页表页可以容纳 512 个 8 字节 PTE。

```
48-bit virtual address:
bits 47..39 PML4 index
bits 38..30 PDPT index
bits 29..21 PD index
bits 20..12 PT index
bits 11..0  page offset
```

如果用单级线性页表, 为整个低 48 位地址空间准备所有 PTE, 会消耗巨大内存; 多级页表只为实际用到的区间分配下级表。比如一个小进程只映射代码、堆、栈和少量共享库, 大量虚拟空洞没有对应页表页。这就是“稀疏地址空间”的硬件数据结构。

```
page_walk(va):
    pml4e = read(CR3 + pml4_index(va) * 8)
    if !pml4e.present: fault
    pdpte = read(pml4e.page + pdpt_index(va) * 8)
    if !pdpte.present: fault
    pde = read(pdpte.page + pd_index(va) * 8)
    if !pde.present: fault
    pte = read(pde.page + pt_index(va) * 8)
    if !pte.present or permission denied: fault
    return pte.frame + offset(va)
```

TLB 的意义也在这里变得具体。一次完整 page walk 可能读四级页表，任何一级还可能 cache miss。TLB 把最终翻译和权限缓存起来，让常见访问不必重复走表。但这也带来维护义务：PTE 改了，TLB 里的旧答案不会自动在所有 CPU 上消失。

第十六步补充：PTE 位背后的 OS 决策

PTE 不是单纯的物理页号。它是一份硬件和内核共同维护的小合同。硬件读取 present、writable、user、NX 等位来决定访问是否成立；硬件可能设置 accessed/dirty 位；内核读取这些位来做回收、写回、COW、安全策略和调试诊断。

PTE 信息	硬件视角	OS 用法
present	这一页能否翻译到物理页	demand paging、swap、file-backed lazy load
writable	store 是否允许	COW、只读代码、共享映射保护
user	Ring 3 是否能访问	用户/内核地址空间隔离
NX	是否禁止取指	W $\bar{X}$ 、栈不可执行、JIT 权限切换
accessed	页面是否被访问过	LRU 近似、老化、回收候选选择
dirty	页面是否被写过	writeback、匿名页回收、文件页落盘
global/PCID 相关	TLB 是否跨切换保留	降低上下文切换成本

这张表也解释了为什么 page fault handler 必须看 error code。一次 fault 可能是不存在页，也可能是写只读页，也可能是用户态访问 supervisor 页，也可能是取指 NX 页。处理方式完全不同：合法缺页可以分配，COW 写可以复制，权限违规要发信号或杀进程，内核访问坏用户指针可能要返回 `-EFAULT`。

第十七步：page fault 不是一定错误

如果用户访问一个 `present=0` 的页，硬件触发 page fault。内核查看这个地址是否属于合法 VMA。如果合法，例如栈增长或文件映射尚未读入，内核可以分配页、更新 PTE、返回用户态重试原指令。如果不合法，内核发送错误信号或杀死进程。

```

page_fault(va, access):
    vma = find_vma(current, va)
    if vma == none:
        kill_process(SIGSEGV)
    else if access not allowed by vma:
        kill_process(SIGSEGV)
    else:
        page = allocate_or_load_page(vma, va)
        install_pte(va, page, permissions)
        return_to_faulting_instruction()
    
```

这一步非常重要。缺页不是“内存坏了”，而是硬件把某次无法完成的访问交给 OS 决策。虚拟内存的弹性来自这个机制。

第十七步补充：缺页处理的决策树

下面把 page fault 拆成更接近真实内核的决策树。读者要特别注意：硬件只告诉内核“这次访问不能按当前页表完成”，真正的语义判断由内核结合 VMA、PTE、当前上下文和访问类型完成。

```

handle_page_fault(va, access, error_code, from_user):
    if !is_canonical(va):
        return signal_or_kernel_fault()

    vma = find_vma(mm, va)
    
```

```

if vma == none:
    return signal_or_efault()

if access violates vma.permissions:
    return signal_or_efault()

pte = lookup_pte(mm, va)
if pte.present and access == write and pte.cow:
    new_page = copy_old_page()
    install_writable_pte(va, new_page)
    flush_tlb_page(va)
    return retry

if !pte.present and vma.anonymous:
    page = allocate_zero_page()
    install_pte(va, page, vma.permissions)
    return retry

if !pte.present and vma.file_backed:
    page = page_cache_get_or_read(vma.file, va)
    install_pte(va, page, vma.permissions)
    return retry

return signal_or_kernel_fault()

```

fault 场景	内核判断依据	结果
栈增长	fault 地址贴近栈 VMA 边界	扩展 VMA 或分配新页
匿名懒分配	VMA 允许访问但 PTE 不存在	分配零页并重试
文件映射缺页	VMA 指向文件页	从 page cache 或磁盘读入
COW 写	PTE 只读且标记 COW	复制物理页，更新 PTE，重试
非法用户地址	无 VMA 或权限不符	发送 SIGSEGV 或返回 -EFAULT
内核 bug	内核态访问非法地址且不可恢复	oops 或 panic

“可恢复”不是由硬件单独决定的。用户态普通指令 fault，内核可以通过信号报告；内核在 `copy_from_user` 中 fault，可以通过异常表修正返回 `-EFAULT`；内核在任意位置解引用坏指针，通常就是内核 bug。相同的硬件 fault，在不同上下文下有不同软件语义。

第十七步补充：异常、陷阱和中断的返回地址不同

很多教材把 `interrupt`、`trap`、`fault` 都翻成“中断/异常”，读者容易混在一起。对 OS 来说，最关键的区别是保存的返回 RIP 指向哪里，以及内核返回后是否重试同一条指令。

事件	保存的 RIP	典型处理
fault	指向出错指令	page fault 修复后重试；不能修复则发信号
trap	指向下一条指令或定义好的后继位置	断点、单步调试，处理后继续

interrupt	指向下一条待执行指令边界	设备 IRQ、定时器 IRQ，处理后恢复原程序
abort	可能无法可靠恢复	double fault、机器检查等严重错误

以 page fault 为例，`mov rax, qword ptr [rdi]` 若访问的页不存在，但 VMA 允许读，内核装好 PTE 后返回，CPU 重新执行同一条 `mov`。以定时器中断为例，被打断指令已经在边界上结束，返回后执行下一条。这个差异决定了内核保存 trap frame、更新统计和投递信号的方式。

```
page fault:
    saved_rip = address of faulting load/store
    kernel may install mapping
    iretq retries saved_rip

timer interrupt:
    saved_rip = next instruction boundary
    kernel may schedule another task
    later iretq continues from saved_rip
```

这也是为什么“精确异常”如此重要。如果 CPU 不能保证 faulting RIP 和架构状态一致，内核就无法判断重试是否安全。

第十八步：把所有状态放进 task

如果要运行多个程序，OS 必须能暂停一个程序，恢复另一个程序。一个 task 至少保存这些状态：

```
struct Task {
    gpr[16]
    rip
    rsp
    rflags
    cpl
    page_table
    kernel_stack
    state: Runnable | Running | Blocked | Zombie
}
```

上下文切换就是保存当前 task 的 CPU 状态，选择下一个 runnable task，把它的状态装回 CPU。真实 OS 还要保存浮点/SIMD、调度统计、信号、fd 表、凭据、namespace 等，但最小根就是这些。

第十九步：从 X64S 教学子集映射到真实 x86-64

现在可以把 X64S 的概念映射到真实机器。

X64S 概念	真实 x86-64 对应	OS 意义
rip	RIP	异常返回和调试定位
rsp	RSP、TSS/IST 栈切换	用户栈、内核栈、调度切换
cpl	Ring 3/Ring 0	特权级和系统调用边界
syscall entry	IA32_LSTAR MSR、IDT gate	受控进入内核
page table	x86-64 四/五级页表、CR3、PCID	地址空间和权限
timer IRQ	local APIC timer、TSC deadline	抢占和超时
device range	PCIe BAR、ACPI resource	驱动访问设备寄存器

task context `task_struct/thread_info/pt_regs` 等 调度器保存恢复执行

真实机器复杂，但不是另一套逻辑。x86-64 的页表层级更多，权限位更多，系统调用入口更讲究，设备通过 PCIe 和 MSI-X，DMA 经过 IOMMU；但每一项都能在 X64S 的最小模型里找到原型。

第十九步补充：x86-64 long mode 启动后有哪些关键控制点

真实 x86-64 进入 64 位 long mode 后，OS 要配置一批“硬件查表入口”。这些入口不是普通全局变量，而是 CPU 在异常、系统调用、地址翻译和栈切换时直接读取的控制状态。

控制点	保存什么	OS 为什么必须初始化
CR3	当前顶级页表物理地址和 PCID 信息	每次地址翻译都从这里找到地址空间
IDT	vector 到 interrupt/trap gate 的表	page fault、timer IRQ、device IRQ 要有入口
GDT/TSS	段描述和特权栈信息	从 Ring 3 进 Ring 0 时选择内核栈，IST 处理特殊异常
MSR LSTAR	<code>syscall</code> 的 64 位内核入口	用户态系统调用只能跳到受控入口
MSR STAR/FMASK	<code>syscall</code> 段选择和 RFLAGS mask	进入内核时清除不安全标志位
APIC registers	本地中断、定时器、IPI 控制	抢占、跨 CPU 调度、TLB shutdown

如果 IDT 没设好，page fault 会变成无法处理的更严重异常；如果 TSS 的内核栈错误，用户态陷入内核时会把 trap frame 写到错误位置；如果 CR3 指向坏页表，取指和访存都会立即崩溃；如果 LSTAR 指向错误地址，`syscall` 就不是受控入口而是灾难。启动代码的价值，就是在允许用户程序运行前，把这些硬件入口全部变成可解释、可恢复、可审计的状态。

```
minimal x86-64 kernel entry preparation:
  build early page tables
  load CR3
  load GDT and TSS
  install IDT entries for faults and IRQs
  write syscall MSRs
  initialize local APIC timer
  create first task and kernel stack
  only then return or jump to user mode
```

这张清单也解释了为什么 OS 实践卷不能一上来写“进程”。没有页表、异常入口、内核栈和定时器，进程只是结构体；有了这些硬件控制点，进程才变成可以被隔离、打断、恢复和销毁的执行实体。

本节验收：能不能从头讲一遍机器运行

- 能解释 bit、byte、word、地址、内存数组之间的关系。
- 能手跑一个 `mov/add/mov` 小程序，说明每步 `rip` 和寄存器如何变化。
- 能解释为什么需要栈，以及 `call/ret` 如何使用栈保存返回地址。
- 能解释轮询设备为什么浪费 CPU，中断如何保存 `rip/rflags` 并跳到处理

函数。

- 能解释定时器为什么让 OS 可以抢占死循环程序。
- 能解释用户态为什么不能直接写设备寄存器，系统调用如何成为受控入口。
- 能解释 base/limit 和分页分别解决什么问题，page fault 什么时候是可修复事件。
- 能把 X64S 的 `rip/rsp/cpl/page table/timer IRQ` 映射到真实 x86-64 概念。

数值、编码与汇编：从一串 bit 到一条程序

很多组成原理教材讲到 bit 后，很快跳到 CPU。中间少掉的一层是：同一串 bit 如何被解释成整数、地址、字符、指令、权限位和文件格式。操作系统每天都在处理这些解释规则。ELF header 是字节解释规则，系统调用 ABI 是寄存器解释规则，页表项是 bit 字段解释规则，网络协议是字节序解释规则。若这一层不清楚，后面读内核结构体和二进制文件会很吃力。

同一串 bit 可以有多种含义

假设内存某个 byte 是 `0x41`，也就是二进制 `01000001`。它可以表示无符号整数 65，可以表示 ASCII 字符 `A`，可以表示机器指令的一部分，也可以表示一个权限位集合。硬件并不知道“它真正是什么”，解释权来自使用它的路径。

解释方式	0x41 的含义	谁规定
无符号整数	65	算术指令和程序类型
有符号补码	65	算术指令和类型解释
字符编码	A	ASCII/UTF-8 等编码规则
指令字节	某条指令的一部分	ISA 编码规则
权限位	bit0 和 bit6 为 1	OS 数据结构定义

这解释了为什么 C/C++ 里的类型很重要，也解释了为什么内核解析用户输入必须严格检查长度、对齐和字段含义。字节本身没有安全性，安全来自解释规则是否完整。

二进制、十六进制和位字段

二进制适合硬件，十六进制适合人读。1 个十六进制数字对应 4 个 bit。比如 `0x5a`：

```
0x5a = 0101 1010
      ^^^^ ^^^^
      0x5 0xa
```

OS 大量使用位字段。例如一个页表项低几位可能是权限和状态，高位是物理页号。

```
PTE:
bit 0      present
bit 1      writable
bit 2      user_accessible
bit 3      write_through
bit 4      cache_disable
bit 5      accessed
bit 6      dirty
bit 63     no_execute
```



```
bits 12..51 physical_page_number
```

设置、清除和测试位字段通常用 AND/OR/NOT/SHIFT。

```
set writable:
    pte = pte | PTE_WRITABLE

clear writable:
    pte = pte & ~PTE_WRITABLE

test present:
    if (pte & PTE_PRESENT) != 0
```

这不是“位运算技巧”，而是硬件契约的直接表达。CPU 做地址翻译时会读取这些 bit；OS 修改这些 bit 就是在修改硬件后续如何处理内存访问。

有符号溢出和地址溢出

8 位无符号数最大是 255。再加 1 会变成 0，这是模 2^8 的结果。硬件加法器自然产生这种环绕。软件必须决定这是正常行为还是错误。

```
uint8:
    255 + 1 = 0

16-bit address:
    0xffff + 1 = 0x0000
```

系统调用里，地址和长度经常成对出现。若用户传入 `ptr=0xffff0`，`len=0x40`，简单计算 `end = ptr + len` 在 16 位地址里会变成 `0x0030`。如果内核只检查 `end < limit`，就可能被骗过。

```
bad_check(ptr, len):
    end = ptr + len
    if end <= user_limit:
        allow

good_check(ptr, len):
    if len > user_limit:
        reject
    if ptr > user_limit - len:
        reject
    allow
```

这就是为什么整数溢出是内核漏洞高发点。组成原理里的补码和模运算，直接影响 OS 安全。

字符编码和文本不是一回事

文件里保存的是字节，不是“文字”。ASCII 用一个 byte 表示英文字符；UTF-8 用 1 到 4 个 byte 表示 Unicode 码点。中文字符通常占多个 byte。路径名、环境变量、命令行参数、终端输入，最终都是字节序列加解释规则。

```
ASCII:
    'A' -> 0x41

UTF-8:
    '中' -> e4 b8 ad
```

OS 内核通常不理解所有文本语义，它更关心字节串长度、是否包含 NUL、路径分隔符、编码是否由用户空间约定。系统调用如 `execve(path, argv, envp)` 必须逐个复制以 NUL 结尾的用户字符串，并限制总长度，避免用户传入无限长或跨坏页的参数。

指令也是字节：反汇编做了什么

机器码是字节，反汇编器按 ISA 规则把字节解释成汇编文本。比如在某个 ISA 中，前 4 bit 是 opcode，后面是寄存器编号和立即数；在 x86-64 中，编码更复杂，有前缀、opcode、ModRM、SIB、位移和立即数。

```
bytes:
    b8 2a 00 00 00

x86-64 interpretation:
    mov eax, 42
```

CPU 不看汇编文本，汇编文本只是人类可读形式。汇编器把文本转成机器码；反汇编器把机器码尽量转回文本。调试内核 oops 时看到的 RIP 和机器码，就是在定位“CPU 当时执行到哪串字节”。

x86-64 指令编码为什么看起来复杂

x86-64 为了兼容历史，指令长度可变。一条指令可以有前缀、REX 前缀、opcode、ModRM、SIB、位移和立即数。初学时不需要背完整编码表，但必须知道“汇编文本”和“机器字节”之间有明确规则，否则看 objdump、内核 oops、kprobe、ftrace 和热补丁时会断层。

字段	作用	例子
prefix	改变操作数大小、重复、锁语义等	lock、operand-size override
REX	x86-64 扩展寄存器和 64 位操作数	使用 r8-r15 或 64 位 GPR
opcode	指令种类	mov、add、call、jmp 的主编码
ModRM	寄存器和寻址模式	选择寄存器直接寻址或内存寻址
SIB	scale/index/base 地址形式	[base + index*scale + disp]
disp	内存地址位移	结构体字段偏移、栈槽偏移
imm	指令内立即数	mov eax, 42 中的 42

例如 mov eax, 42 常见机器码是：

```
b8 2a 00 00 00
| |-----|
|      imm32 = 42, little endian
opcode: mov r32, imm32 with destination eax
```

这里的 2a 00 00 00 再次体现小端：十六进制 0x0000002a 在内存里低字节在前。CPU 按 ISA 规则读这些字节，不会看到字符串 mov。

ModRM 和 SIB：内存操作数如何编码

Intel 语法中的 qword ptr [rbx + 16] 是给人的寻址表达式。机器码里，它会被拆成“基址寄存器是 rbx、位移是 16、访问宽度是 8 字节”等字段。简化看，地址生成单元执行：

```
effective_address = base + index * scale + displacement
```

常见寻址形式如下：

Intel 语法	地址生成	常见来源
[rbx]	rbx	指针解引用

<code>[rbx + 16]</code>	<code>rbx + 16</code>	结构体字段或栈槽
<code>[rbx + rcx*8]</code>	<code>rbx + rcx * 8</code>	数组元素
<code>[rbp - 32]</code>	<code>rbp - 32</code>	栈帧局部变量
<code>[rip + disp]</code>	<code>next_rip + disp</code>	位置无关代码访问全局数据

`rip-relative` 寻址是理解共享库和内核映像的重要入口。位置无关代码不把全局变量地址写死在指令里，而是用当前 `rip` 加位移找到数据。这样同一段代码可以加载到不同虚拟地址，动态链接器或内核重定位的工作量更小。

相对跳转和调用：控制流也是编码出来的

`call` 和 `jmp` 不一定把绝对地址放进机器码。很多 x86-64 直接调用使用相对位移：目标地址等于下一条指令地址加 `signed displacement`。

```
call target:
    pushed_return = next_rip
    rip = next_rip + sign_extend(rel32)
```

这解释了为什么反汇编常显示某个地址而机器码里只有一个小端位移。链接器处理 `relocation` 时，也经常是在填这些位移或 GOT/PLT 入口。OS 读 oops 和栈回溯时，看到的返回地址是 `call` 的下一条指令地址；调试器再根据符号表把它映射回函数名和源码行。

从 objdump 读证据

写作和调试时，不要只说“编译器会生成汇编”。要能拿出命令和输出。下面是典型证据路径：

```
g++ -O2 -c demo.cpp -o demo.o
objdump -dr -Mintel demo.o
readelf -Ws demo.o
readelf -r demo.o
```

`objdump -d` 给反汇编，`-r` 同时显示 `relocation`，`-Mintel` 使用 Intel 语法。若某条指令访问外部符号，目标地址在 `.o` 里可能还没确定，反汇编会看到 `relocation` 项。这个细节连接到后面的 ELF、动态链接、内核模块装载和 eBPF/JIT：机器码中的地址不是总能在编译时写死，链接和装载会继续修改或解释它。

最小实验：从 C 到对象文件的四个观察点

下面这个实验不依赖内核源码，适合作为读者的第一条证据链。写一个小文件：

```
// demo.cpp
extern int g;

int add_g(int x) {
    return x + g + 1;
}
```

编译但不链接：

```
g++ -O2 -c demo.cpp -o demo.o
objdump -dr -Mintel demo.o
readelf -Ws demo.o
readelf -r demo.o
```

应该观察四件事。第一，函数参数 `x` 进入整数参数寄存器，返回值在 `eax/rax`。第二，对外部变量 `g` 的访问不会在 `demo.o` 里得到最终地址，而是留下 `relocation`。第三，反汇编中的 `rip-relative` 访存说明代码不是把绝对地址写死。第四，优化级改变后，指令数量、寄存器选择和内存访问可能变化，但 ABI 边界不能变。

这不是为了背命令，而是训练一个习惯：凡是书里说“编译器生成”“链接器修正”“CPU 执行”，都应能用 `objdump/readelf/gdb/perf` 之类工具找到证据。没有证据的解释，容易停留在概念层。

x86-64 Intel 汇编到机器动作

考虑这段 Intel 语法汇编：

```
mov rbx, 0x2000
mov rax, qword ptr [rbx]
add rax, 1
mov qword ptr [rbx], rax
```

逐步解释：

- 1. `mov rbx, 0x2000` 把地址常量放进寄存器 `rbx`。
- 2. `mov rax, qword ptr [rbx]` 把 `rbx` 当地址，从内存读 8 字节到 `rax`。
- 3. `add rax, 1` 使用 ALU，让 `rax` 加 1。
- 4. `mov qword ptr [rbx], rax` 把 `rax` 写回 `rbx` 指向的内存。

这四条指令已经覆盖了程序的大部分本质：常量、地址、读内存、算术、写内存。C 语言里的 `x = x + 1`，编译后也会落到类似动作，只是寄存器分配和寻址模式更复杂。

地址和指针：类型给地址加意义

指针本质上是一个地址值。但 `int*`、`char*`、`struct Task*` 对同一地址的解释不同。编译器根据类型决定一次读写多少字节、按什么对齐、字段偏移是多少。

```
struct Pair {
    uint32_t a;
    uint32_t b;
};

p->b
address = p + offset_of(b)
load 4 bytes from address
```

内核大量使用结构体指针。若把用户传入的整数当内核指针使用，就是严重漏洞。用户地址和内核地址必须区分；用户地址要通过 `copy_from_user`，内核指针才可直接解引用。

对齐为什么会影响正确性和性能

很多 CPU 更喜欢在自然边界上访问数据：2 字节值地址是 2 的倍数，4 字节值地址是 4 的倍数，8 字节值地址是 8 的倍数。有些架构允许非对齐访问但慢，有些架构会触发异常。设备 DMA 描述符也常要求对齐。

对象	常见对齐	原因
16 位整数	2 字节	一次总线访问或避免跨边界
32 位整数	4 字节	ALU/内存接口自然宽度
64 位整数	8 字节	原子访问和性能
cache line 数	64 字节量级	避免伪共享、满足 DMA/cache 要求
据		
页	4KiB 或更大	页表和 MMU 管理单位

OS 里对齐随处可见：页表页必须按页对齐，内核栈按 ABI 对齐，DMA buffer 按设备要求对齐，结构体字段为了性能可能填充 padding。理解 padding 后，读二进制结构和系统调用 ABI 才不会误解字段位置。

从 C 到汇编再到 OS 证据

一个简单 C 函数：

```
int add_one(int x) {
    return x + 1;
}
```

可能变成类似汇编：

```
add_one:
    mov eax, edi
    add eax, 1
    ret
```

这里 `edi` 是参数寄存器，`eax` 是返回值寄存器。这是 ABI 规定的，不是编译器随意选择。系统调用也有 ABI：系统调用号在哪个寄存器，参数在哪些寄存器，返回值和 `errno` 如何表达。OS 不是运行“C 源码”，它运行机器码，并通过 ABI 和用户程序达成二进制契约。

本节验收

- 能把一个 byte 同时解释成整数、字符、指令字节和权限位，并说明解释规则来自哪里。
- 能用位运算说明如何设置、清除和测试页表权限位。
- 能解释整数溢出如何破坏地址范围检查。
- 能说明端序如何影响内存中的多字节值。
- 能把一段简单 C 语句拆成 load、ALU、store 动作。
- 能解释指针为什么只是地址加类型解释，为什么用户指针不能当内核指针直接用。
- 能说明对齐、padding、ABI 和系统调用参数寄存器之间的关系。

函数调用、栈和 ABI：程序如何变成可恢复的控制流

操作系统的进程、线程、系统调用和异常处理，都依赖一个更基础的事实：程序运行不是一团代码随便跳，而是有调用约定、栈帧、返回地址、寄存器保存规则和二进制接口。先把普通函数调用讲清楚，再看系统调用和上下文切换，会少很多误解。

为什么需要调用约定

函数调用要解决几个问题：参数放在哪里，返回值放在哪里，被调用函数能改哪些寄存器，返回地址如何保存，栈如何对齐，谁负责清理参数。调用者和被调用者必须遵守同一套规则，否则分别编译的代码无法互相调用。这套规则叫 calling convention，是 ABI 的一部分。

问题	调用约定回答	错误后果
参数传递	前几个参数在寄存器，更多参数在栈	函数读错输入

返回值	使用指定寄存器或内存返回区	调用者拿错结果
寄存器保存	caller-saved / callee-saved	返回后变量被破坏
栈对齐	调用前满足固定对齐	SIMD/函数序言/ABI 失败
可变参数	额外寄存器和栈布局规则	printf 一类函数解析错误

系统调用 ABI 和函数调用 ABI 相似但边界更硬。函数调用跨的是代码组织边界；系统调用跨的是用户/内核特权边界。

System V x86-64 参数分类：不只是前六个整数寄存器

常见说法是“前六个整数参数放寄存器”，这只覆盖最简单情况。System V x86-64 ABI 会按参数类别把值放进整数寄存器、SSE 寄存器或内存。整数和指针走 GPR，浮点数常走 XMM，过大或复杂聚合类型可能通过内存传递。

参数类型	常见位置	注意点
整数/指针	rdi,rsi,rdx,rcx,r8,r9	更小整数会扩展或只使用低位寄存器
浮点/double	xmm0-xmm7	和整数寄存器计数分开
小结构体	可能拆到 GPR/XMM	取决于字段分类和大小
大结构体参数	内存或指针传递	调用者要准备对象地址
大结构体返回	structure-return 隐藏指针	调用者传入返回缓冲区地址

这解释了为什么同一个 C 函数签名稍微变一下，汇编会完全不同。OS 读用户态寄存器时也要区分 ABI：普通函数 ABI 使用 rcx 作为第 4 个整数参数，而 Linux x86-64 syscall ABI 使用 r10 传第 4 个参数，因为 syscall 硬件会使用 rcx 保存返回 RIP。

```
System V function call integer args:
    arg0 rdi, arg1 rsi, arg2 rdx, arg3 rcx, arg4 r8, arg5 r9

Linux x86-64 syscall args:
    nr rax, arg0 rdi, arg1 rsi, arg2 rdx, arg3 r10, arg4 r8, arg5 r9
```

如果把这两套 ABI 混用，系统调用会读错参数。用户态 syscall wrapper 的价值之一，就是把 C 函数调用 ABI 翻译成 syscall ABI。

一个函数调用的机器级过程

假设调用 f(1, 2)。伪汇编可能是：

```
mov edi, 1      ; arg0, System V x86-64 integer arg 1
mov esi, 2      ; arg1, System V x86-64 integer arg 2
call f
; return value in eax
```

call f 至少做两件事：把返回地址压入当前 rsp 指向的栈，把 rip 改成 f 的入口。返回时 ret 从栈顶取回返回地址放回 rip。

```
call target:
    push next_rip
    rip = target

ret:
    pop rip
```


这解释了为什么栈损坏会导致程序跳飞。返回地址只是栈上的数据；若越界写覆盖返回地址，`ret` 就会把错误值装入 `rip`。现代系统用栈 canary、ASLR、NX、CFI 等机制降低这类攻击风险。

栈帧里有什么

一个典型栈帧可能包含返回地址、旧帧指针、保存的 callee-saved 寄存器、局部变量、临时 spill 槽和对齐填充。

```
high address
  caller frame
  argument spill area
  return address
  old frame pointer
  saved registers
  local variables
  temporary spill slots
low address
```

优化编译器可能省略帧指针，把局部变量放寄存器，甚至内联函数让栈帧消失。但异常展开、调试器栈回溯、perf 采样和崩溃报告都需要某种方式恢复调用链。DWARF CFI 和 frame pointer 就是为了解决“如何从当前寄存器和栈恢复上一帧”。

caller-saved 和 callee-saved

寄存器保存规则是一种成本分摊。caller-saved 表示调用者若还需要该寄存器，就在调用前自己保存；callee-saved 表示被调用者若要使用，就必须在返回前恢复。

类别	谁保存	适合用途
caller-saved	调用者	短生命周期临时值、参数寄存器
callee-saved	被调用者	跨函数调用仍要保留的局部状态
special registers	ABI 或硬件固定	RSP、RIP、RFLAGS、线程指针等

上下文切换和函数调用不同。函数调用只遵守 ABI 保存一部分寄存器；中断或线程切换必须保存足以恢复执行的架构状态。系统调用入口还要保存用户传参寄存器，因为它们是系统调用语义的一部分。

栈增长、栈溢出和 guard page

很多系统的用户栈向低地址增长。函数调用越深，`rsp` 越小。若递归过深或局部数组太大，栈会撞到未映射区域。OS 通常在栈边界放 guard page：访问它时触发 page fault，内核判断是可增长栈还是栈溢出。

```
stack layout:
  mapped stack pages
  guard page // not present
  other mappings
```

内核栈更危险。用户栈溢出通常杀进程；内核栈溢出可能破坏内核对象，导致安全漏洞或 panic。因此内核栈较小、禁止大局部数组、使用专门栈处理 NMI/double fault 等关键异常。

线程为什么需要自己的栈

线程的“执行到哪里”不只在 `rip`，还在栈里。若两个线程共用同一个栈，它们的函数调用、返回地址和局部变量会互相覆盖。每个线程必须有独立用户栈；进入内核后，也需要对应的内核栈或受控异常栈保存内核路径。

```
Thread A:
```

```

user stack A
kernel stack A

Thread B:
  user stack B
  kernel stack B

```

协程和用户态线程可以共享 OS 线程，但它们仍需要各自保存栈或保存状态机 continuation。无论实现方式如何，“下一步在哪里”和“局部状态在哪里”必须有地方保存。

系统调用栈切换与函数调用的差别

普通函数调用继续使用同一栈和同一特权级。系统调用从用户态进入内核态，不能继续信任用户栈。内核入口要切换到当前线程的内核栈，把用户寄存器保存成 `pt_regs`，再调用 C 层系统调用处理函数。

```

user:
  rax = syscall_number
  rdi = arg0
  syscall

entry:
  switch rsp from user_rsp to kernel_rsp
  push user register state into pt_regs
  call syscall_dispatch

```

这说明系统调用开销来自多层：CPU 特权切换、栈切换、寄存器保存、权限检查、对象查找、可能的调度和返回路径工作。它不是普通函数调用慢一点而已。

red zone、栈对齐和内核代码为什么要小心

System V x86-64 用户态 ABI 定义了 red zone: `rsp` 以下 128 字节在异步信号处理前不会被破坏，叶子函数可以临时使用而不调整 `rsp`。这对用户态优化有用，但内核或中断上下文不能盲目依赖用户态 red zone。内核入口会切换到内核栈，内核自身通常禁用或避免 red zone，因为中断可能在同一栈上保存状态。

```

leaf function may use red zone in user ABI:
  temporary data at [rsp - 8], [rsp - 16], ...
  no sub rsp needed

kernel entry:
  do not trust user stack
  switch to kernel stack
  save pt_regs in controlled memory

```

栈对齐同样不是格式要求。许多 SIMD 指令、函数序言和 ABI 约定假设调用边界满足 16 字节对齐。若手写汇编破坏对齐，程序可能在看似无关的库函数里崩溃。内核构造初始用户栈、信号栈和 exec 栈时，必须满足用户态 ABI，否则新程序还没进入 `main` 就可能出错。

varargs 为什么需要 ABI 细则

`printf` 这类可变参数函数不知道调用者传了多少个参数、每个参数是什么类型，只能依赖格式串和 ABI 布局去取。System V x86-64 中，可变参数函数需要知道哪些寄存器参数被使用，必要时把寄存器保存到 register save area, `va_list` 再按规则取出下一个参数。

```

va_arg conceptual path:
  if next argument class still has register slot:
    read from saved register area
  else:
    read from overflow area on stack

```

```
advance offset according to type and alignment
```

这说明 ABI 不是“编译器内部细节”。动态库、JIT、FFI、系统调用封装、eBPF helper、内核用户态边界都依赖调用约定。如果边界两侧对参数分类理解不同，错误不会表现为类型检查失败，而是直接读错寄存器或栈槽。

异常展开和 C++ 异常为什么依赖 ABI

C++ 异常抛出时，运行时要沿调用栈查找 handler，并在经过每个栈帧时执行析构。它必须知道每个函数的栈布局、返回地址、保存寄存器和清理代码位置。这些信息由 ABI、DWARF unwind table 和编译器共同维护。

```
throw:
  capture exception object
  inspect current frame unwind info
  find landing pad or caller frame
  run destructors while unwinding
  transfer control to catch handler
```

内核通常不使用 C++ 异常，但它有类似需求：oops/panic 打印栈，perf 采样回溯调用链，用户态 signal handler 返回要恢复上下文，调试器要 unwind。所有这些都要求机器状态和栈布局可解释。

signal frame：内核怎样临时改写用户控制流

信号不是在用户程序旁边开一条神秘线程。内核要让当前线程在返回用户态时先执行 signal handler，于是会在用户栈或 alternate signal stack 上构造一个 signal frame，保存原来的用户上下文，并把返回路径安排到 signal trampoline。handler 结束后，sigreturn 系统调用让内核从 signal frame 恢复原上下文。

```
deliver_signal:
  choose user stack or altstack
  write signal frame with saved registers and mask
  set user RIP = handler
  set user RSP = signal_frame
  arrange return path to sigreturn trampoline

sigreturn:
  validate frame
  restore saved user registers, RIP, RSP, RFLAGS, signal mask
```

这条路径把 ABI、栈、系统调用和安全检查连在一起。用户栈坏了，信号投递可能失败；signal frame 校验不严，用户就可能伪造上下文获得非法状态；异步信号打断普通代码，也解释了为什么可重入和 async-signal-safe 函数很重要。

exec 如何构造第一帧用户栈

execve 装载新程序后，用户程序还没有调用任何函数，但它需要一个初始栈，里面放 argc、argv、envp、auxv 和字符串数据。动态链接器和 libc 启动代码从这个栈读取参数，再调用 main。

```
initial user stack:
  argc
  argv[0] pointer
  argv[1] pointer
  ...
  NULL
  envp[0] pointer
  ...
  NULL
  auxv entries
  strings for argv/envp
```

这说明进程启动也依赖 ABI。内核不能随便把参数放某个内核对象里；用户态启动代码按 ABI 从栈读取。若栈对齐或 auxv 错，程序可能在进入 main 前就崩溃。

本节验收

- 能解释调用约定为什么必须规定参数、返回值、寄存器保存和栈对齐。
- 能画出 CALL/RET 如何用栈保存返回地址。
- 能说明栈帧中返回地址、旧帧指针、保存寄存器、局部变量的位置和作用。
- 能区分函数调用保存寄存器、系统调用保存 trap frame、线程切换保存 context。
- 能解释为什么系统调用必须切到内核栈。
- 能解释 execve 为什么要构造初始用户栈。

从数字电路到 CPU：把每一层设计掰开

上一节给出了组成原理主线，本节把最容易被教材压缩的部分展开：为什么要有逻辑门，为什么寄存器必须跟时钟绑定，为什么 CPU 不是一个巨大函数，为什么指令要编码，为什么流水线会带来异常和安全问题。读完本节，后面讲“保存上下文”“系统调用入口”“页表异常”时，读者应能把它们落回硬件状态，而不是只记术语。

布尔逻辑：从判断到电路

软件里写 if (a && b)，硬件里对应的是逻辑门。AND 门只有两个输入都为 1 时输出 1；OR 门只要有一个输入为 1 就输出 1；NOT 门把 0 变 1、1 变 0；XOR 门表示两个输入不同。所有复杂控制条件最终都能分解为这些布尔函数的组合。

门	输出规则	在 CPU 中的用途
AND	两个条件都成立	权限位同时满足、mask 检查、enable 信号
OR	任一条件成立	多个异常源合并、多个 pending 位汇总
NOT	条件取反	valid/invalid、zero/non-zero 控制
XOR	输入不同	加法器、比较器、奇偶校验
MUX	按选择信号选一路输入	选择 ALU 输入、写回来源、下一 RIP

多路选择器 MUX 非常关键。CPU 中到处都是“根据控制信号选一个值”：下一条 RIP 是顺序地址、分支目标、异常入口还是返回地址；写回寄存器的数据来自 ALU、内存、立即数或内部控制状态；load/store 的地址来自基址寄存器加偏移。MUX 把控制信号和数据通路连接起来。

```
next_pc = mux(  
    normal: next_sequential_rip,  
    branch_taken: branch_target,  
    trap: trap_vector,  
    return_from_trap: saved_pc  
)
```

这个模型会在 OS 中反复出现。系统调用入口就是一种“下一 RIP 被硬件选择到 trap vector”的情况；异常返回就是“下一 RIP 被选择回 saved RIP”的情况。区别只是硬件选择受严格权限规则约束。

半加器、全加器和 ALU

整数加法器是理解 CPU 数据通路的好入口。一位半加器接收两个 bit，输出 sum 和 carry；全加器再接收低位进位。把 64 个全加器串起来，就能做 64 位加法。真实 CPU 会用更快的进位预测结构，但语义仍是逐位加法。

```
half_adder(a, b):
    sum = a XOR b
    carry = a AND b

full_adder(a, b, carry_in):
    sum = a XOR b XOR carry_in
    carry_out = (a AND b) OR (carry_in AND (a XOR b))
```

ALU 不只做加法。它通常支持加、减、与、或、异或、移位、比较。控制单元根据指令 opcode 选择 ALU 操作。比如 `sub` 可以用加法器加上补码实现，比较可以看减法结果和符号/零标志。

ALU 操作	硬件动作	软件语义
ADD	二进制加法	指针偏移、整数运算、栈指针调整
SUB	加上补码	比较、循环计数、地址差
AND/OR/XOR	按位逻辑	位图、权限位、mask、标志位
SHIFT	左右移位	乘除 2 的幂、页号/页内偏移分解
COMPARE	产生 zero/sign/carry 等标志	分支条件和排序判断

OS 为什么关心 ALU？因为许多 OS 数据结构依赖位运算：页表权限位、CPU mask、IRQ pending 位、bitmap 分配器、slab freelist 标记、atomic bit operation。位运算不是低级技巧，而是硬件支持下的高密度状态表达。

组合逻辑的延迟和为什么需要时钟

组合逻辑不是瞬间完成。信号通过每个门都需要传播时间；电路越深，输出稳定越晚。若下一层在输出还没稳定时采样，就会得到错误值。时钟把系统切成离散周期：一个周期内，组合逻辑从寄存器输出开始传播；到下一个时钟边沿，目标寄存器采样稳定结果。

```
cycle begins:
    source registers output old values
    combinational logic computes new values
    signals settle before clock edge
clock edge:
    destination registers capture new values
```

时钟频率不能无限提高。周期必须长到足够覆盖最长组合路径、寄存器建立时间、时钟偏斜和安全余量。提高频率通常需要缩短每级逻辑，于是出现流水线。流水线不是为了让单条指令的延迟变短，而是为了让每个周期能推进不同指令的不同阶段，提高吞吐。

触发器和寄存器：状态的物理形态

触发器保存一个 bit，寄存器保存一组 bit。程序计数器是寄存器，通用寄存器文件是寄存器阵列，状态寄存器也是寄存器。内核里说“保存 CPU 状态”，最终就是把这些寄存器的软件可见值写到内存里的 trap frame 或 context 结构。

```
register64 rip
register64 gpr[16] // rax, rbx, rcx, rdx, rsi, rdi, rbp, rsp, r8..r15
register status {
```



```

privilege_mode
interrupt_enable
condition_flags
}

```

寄存器有两个重要性质。第一，它们只在特定时刻更新，所以硬件状态可以被分阶段分析。第二，它们数量有限，所以编译器和 ABI 必须决定哪些值放寄存器，哪些溢出到栈。上下文切换时，寄存器越多越宽，保存恢复成本越高。

寄存器文件为什么有读端口和写端口

一条典型整数指令可能同时读两个源寄存器，写一个目的寄存器。因此寄存器文件通常要有多个读端口和至少一个写端口。端口越多，硬件面积和时序越困难。复杂乱序 x86-64 CPU 会通过寄存器重命名、调度队列和旁路网络提高并行度，但软件可见的架构寄存器集合仍由 ISA 固定。

```

execute add rax, rbx:
    a = regfile.read_port0(rax)
    b = regfile.read_port1(rbx)
    y = alu.add(a, b)
    regfile.write_port0(rax, y)

```

这也解释了 ABI 为什么规定参数寄存器和返回值寄存器。系统调用入口不想从用户栈里慢慢解析所有参数，而是让用户态按 ABI 把系统调用号和前几个参数放进约定寄存器。内核保存 trap frame 后，就能从寄存器槽读取参数。

指令编码：为什么机器码要有格式

CPU 取到的是一串 bit。译码器必须知道哪些 bit 表示 opcode，哪些 bit 表示寄存器编号，哪些 bit 表示立即数，哪些 bit 表示访存模式。教学用固定长度指令格式通常容易画图；x86-64 指令长度可变，带有前缀、ModR/M、SIB、位移和立即数等字段，译码更复杂，但换来了长期二进制兼容和丰富寻址模式。

编码字段	作用	OS 相关例子
opcode	指令做什么	<code>syscall</code> 触发陷入
register id	读写哪些寄存器	ABI 参数和返回值位置
immediate	常量或偏移	栈调整、页内偏移、跳转距离
memory mode	如何计算地址	load/store、栈访问、结构体字段
privileged bit/encoding	是否敏感指令	用户态执行会触发异常

非法指令异常就是译码或权限检查发现当前 bit 串不能在当前模式下执行。内核可以把它变成信号，也可以在某些情况下模拟指令。虚拟化也依赖类似机制：guest 执行敏感操作时陷入 hypervisor，由 hypervisor 模拟或转发。

单周期 CPU 为什么不够

教学里常从单周期 CPU 开始：一条指令在一个长周期内完成取指、译码、执行、访存和写回。它容易理解，但周期必须长到覆盖最慢指令，例如 load 要经过指令内存、寄存器读、ALU 地址计算、数据内存、写回。于是简单加法也要等同样长的周期，浪费硬件。

```

single_cycle:
    period >= max(delay(fetch), decode, execute, memory, writeback)
    every instruction consumes one long cycle

```

多周期 CPU 把一条指令拆成多个步骤，简单指令可少走某些阶段。流水线进一步让不同指令重叠执行。每一步都是为了更好利用硬件，但每一步也增加控制复杂度：要保存中间状态，要处理 hazard，要保证异常精确。

流水线细讲：五级只是起点

经典五级流水线是 IF、ID、EX、MEM、WB。IF 取指；ID 译码和读寄存器；EX 执行 ALU 或计算地址；MEM 访问数据 cache；WB 写回寄存器。真实 CPU 可能更多级，前端和后端也更复杂，但五级模型足够解释 OS 关心的入口。

阶段	做什么	可能产生的 OS 相关事件
IF	用 RIP 取指令	指令页缺页、执行权限错误、I-cache miss
ID	译码和读寄存器	非法指令、特权指令检查
EX	ALU、分支、地址计算	除零、溢出陷阱、分支目标计算
MEM	load/store 访问内存	数据页缺页、写权限错误、对齐异常
WB	写回寄存器	架构状态提交边界

如果一条 load 在 MEM 阶段发现页不存在，CPU 必须让内核看到“前面指令已提交，这条 load 未完成，后面指令不应可见”的状态。这就是精确异常。没有精确异常，内核无法修复页表后重试同一条指令。

数据 hazard：为什么需要转发和停顿

考虑两条相邻指令：

```
add rax, rbx
sub rcx, rax
```

第二条需要第一条产生的 `rax`。但第一条可能要到 WB 阶段才写回寄存器，第二条在 ID 阶段就读寄存器，会读到旧值。硬件可用转发把 EX 结果直接送给下一条 EX；若是 load-use，数据要到 MEM 后才可用，可能必须插入 bubble。

```
if id.rs1 == ex.rd and ex.will_write:
    operand1 = forward(ex.result)
else if id.rs1 == mem.rd and mem.will_write:
    operand1 = forward(mem.result)
else:
    operand1 = regfile.read(id.rs1)
```

OS 不直接管理这些 hazard，但它受影响。上下文切换和异常入口要求流水线在可定义边界停住；性能分析里的 pipeline stall、IPC 下降、cache miss 和 branch miss，都来自这些微架构细节。

控制 hazard：分支预测和异常边界

分支指令决定下一条 RIP。若等分支结果完全计算出来再取下一条，流水线会空等。分支预测猜测方向和目标，先取预测路径。猜对提高吞吐，猜错要冲刷流水线并从正确 RIP 重新开始。

```
if branch_predictor.predict(rip) == taken:
    next_pc = predicted_target
else:
    next_rip = rip + instruction_length

later:
    if actual != predicted:
        flush younger instructions
        restart at correct_pc
```

预测路径上的指令即使最终被冲刷，也可能触碰 cache 和预测器，留下微架构痕迹。Spectre 一类漏洞正是利用这种“架构上没发生、微架构上留下痕迹”的差异。OS 的隔离设计不能只看 ISA 可见状态，还要考虑某些微架构共享结构。

乱序执行和提交：为什么要 reorder buffer

乱序 CPU 为了隐藏延迟，会让不相互依赖的指令先执行。例如一条 cache miss 的 load 等内存时，后面的独立加法可以先算。为了保持程序员看到的顺序，CPU 通常在提交阶段按程序顺序更新架构状态。reorder buffer 记录每条在途指令的结果、异常和完成状态。

```
dispatch instructions into ROB
execute ready operations out of order
write temporary results into ROB entries
commit from ROB head in program order
if head has exception:
    stop commit younger entries
    raise precise exception
```

这个设计对 OS 至关重要。异常要精确，必须在提交点报告；中断要可恢复，必须在架构状态一致的位置接收；上下文切换保存的是架构寄存器，不是 CPU 内部临时物理寄存器。OS 不需要知道 ROB 每个细节，但必须依赖“架构状态可定义”这个契约。

原子指令：多核共享状态的硬件支点

多核下，两个 CPU 同时执行 `counter = counter + 1` 会丢更新，因为它实际是 load、add、store 三步。原子读改写指令把这三步包装成硬件不可分割操作，或者通过 load-linked/store-conditional 协议保证并发检测。

```
atomic_fetch_add(addr, 1):
    lock cache line or acquire exclusive ownership
    old = memory[addr]
    memory[addr] = old + 1
    release ownership
    return old
```

自旋锁、引用计数、futex 快路径、无锁队列都依赖原子操作。没有原子操作，多核 OS 很难正确维护共享内核对象。原子操作本身也有成本：它会让 cache line 获得独占状态，并可能在 CPU 间迁移。

内存序：为什么“已经写了”不等于“别人已经看见”

单核顺序执行下，写 A 再写 B 通常很直观。多核里，写入可能先进入 store buffer，其他 CPU 看到的时间不一定等同于源代码顺序。设备通过 DMA 或 MMIO 观察内存时，还会再多一层总线顺序问题。

```
producer:
    data = 42
    ready = true

consumer:
    if ready:
        assert(data == 42) // without ordering, not guaranteed on all machines
```

release/acquire 修复这个发布模式：

```
producer:
    data = 42
    store_release(ready, true)

consumer:
    if load_acquire(ready):
        assert(data == 42)
```

OS 中这类模式无处不在：发布新 task、把 page 放入 LRU、提交 DMA 描述符、唤醒等待队列、释放锁。理解内存序不是为了写花哨无锁代码，而是为了知道哪些

顺序必须由硬件和编译器共同保证。

从 CPU 到 OS 的第一条完整路径

现在把一条用户态 load 指令串起来：

1. IF reads instruction bytes using RIP
2. I-TLB translates instruction address
3. decoder recognizes load instruction
4. register file reads base register
5. ALU computes virtual address
6. D-TLB translates data address
7. PTE permission check fails because page not present
8. pipeline records precise exception
9. CPU saves fault RIP/RFLAGS and enters kernel trap vector
10. kernel inspects fault address and VMA
11. kernel allocates or reads page, installs PTE
12. return from exception retries the same load

这条路径把组成原理和 OS 连接起来了。没有 RIP 和译码，就没有“哪条指令 fault”；没有 MMU 和 PTE，就没有权限检查；没有精确异常，就不能修复后重试；没有内核 trap 入口，就不能把硬件事件变成 OS 决策。后面的虚拟内存章节只是在这条路径上继续加细节。

本节测试：不用背名词，要能解释路径

- 给出一条 `add` 指令，能说明寄存器文件、ALU、控制信号和写回如何配合。
- 给出一条 `load` 指令，能说明虚拟地址、TLB、页表、cache 和异常如何参与。
- 给出两条有依赖的指令，能说明为什么需要转发或停顿。
- 给出一个分支，能说明预测、冲刷和微架构痕迹的区别。
- 给出一个 page fault，能说明 CPU 为什么能精确回到 faulting instruction。
- 给出一个多核计数器，能说明普通 load/add/store 为什么不够，原子操作解决什么。
- 给出一个发布对象例子，能说明 release/acquire 保护的是哪条可见性顺序。

硬件微观机制深讲：状态机、缓存一致性和 I/O 事务

上一节把整机硬件铺开，本节继续往下钻。学习操作系统最容易卡住的地方，是知道“有 cache、有中断、有 DMA”，但不知道这些机制内部怎样改变状态，为什么内核要按某种顺序写代码。本节按实现粒度讲：CPU 如何提交指令，cache line 如何在核心之间转移，TLB 如何失效，PCIe/NVMe 如何完成一次事务，IOMMU 如何把设备也放进地址空间。

同步时序：为什么硬件要靠时钟推进

组合逻辑只把输入变成输出，不能记住过去。寄存器在时钟边沿采样输入，构成下一周期的状态。CPU、内存控制器、DMA 引擎、NVMe 控制器、网卡队列，本质上都是许多状态机协同推进。

```
state machine template:
  current_state is stored in registers
  combinational logic computes next_state from inputs
  on clock edge:
```

```

if reset:
    current_state = RESET_STATE
else:
    current_state = next_state
    
```

OS 里的“状态不可跳跃”来自这里。设备 reset 后必须等 ready bit, 是因为设备内部状态机还没从 RESET 走到 READY; 写 doorbell 后要等 completion, 是因为设备只记录了“有新请求”, 数据搬运、错误检查和完成写回还要经历多个状态; 中断处理必须给 EOI, 是因为中断控制器也有 pending、in-service、masked 等状态。

流水线不是五个名词，而是一组提交规则

教科书常把流水线写成取指、译码、执行、访存、写回。真实 CPU 可能有更复杂的前端、重命名、发射、执行、回写、提交，但对 OS 最关键的是提交规则：架构状态必须按程序顺序提交，异常必须精确。

```

out-of-order core sketch:
    fetch instruction bytes
    decode into micro-ops
    rename architectural registers to physical registers
    dispatch micro-ops into queues
    execute ready micro-ops out of order
    write results to physical registers
    retire completed instructions in program order
    
```

“乱序执行”不意味着用户程序看见乱序结果。CPU 可以内部先执行后面的 load 或 ALU 操作，但提交到架构寄存器、异常点和内存可见语义时必须遵守 ISA 规定。缺页异常能返回原指令重试，依赖的就是精确异常：出错指令之前的指令已经提交，之后的指令没有提交。

精确异常的例子：load 缺页

假设指令流中第三条 load 缺页，第四条加法可能已经在乱序引擎里算过，但不能对 OS 可见。

```

inst1: add r1, r2, r3      committed
inst2: store [x], r1       committed or ordered by memory rules
inst3: load r4, [bad_va]   raises page fault
inst4: add r5, r6, r7      may have executed speculatively

architectural view at exception:
    saved RIP points to inst3
    inst1 and inst2 are visible as completed
    inst3 has not produced final architectural result
    inst4 is not architecturally visible
    
```

内核处理缺页时才能做出可靠判断：若是合法懒分配，就建立 PTE 后返回 inst3 重试；若是非法访问，就向进程发送信号。没有这个契约，page fault 不能成为虚拟内存的基础。

寄存器重命名和上下文切换

CPU 内部可能有比 ISA 更多的物理寄存器。寄存器重命名把软件可见寄存器映射到物理寄存器，消除很多假依赖。上下文切换时，OS 不需要保存所有物理寄存器，只需要保存 ISA 规定的软件可见状态。

状态	谁管理	上下文切换关系
架构通用寄存器	OS 必须保存恢复	用户程序可见，必须准确状态大，可能惰性或按需保存
浮点/SIMD 状态	OS 按 ABI 和策略保存	
物理寄存器池	CPU 微架构管理	OS 不直接保存

分支预测器状态 cache/TLB	CPU 微架构管理 CPU 和 OS 协作维护	性能和侧信道相关 不保存，但切换影响命中率
----------------------	----------------------------	--------------------------

这解释了为什么上下文切换不仅是保存几十个寄存器。它还会改变 cache/TLB 热状态、分支预测历史、内存局部性和 NUMA 访问路径。调度器选择任务时，实际在权衡“马上公平”和“保留热状态”。

cache line 状态：以 MESI 思想理解一致性

多核系统中，每个 CPU 有自己的 cache。同一个物理 cache line 可能同时存在于多个核心。硬件一致性协议维护每条 line 的状态。不同 CPU 具体协议不同，但用 MESI 思想足以理解 OS 成本。

状态	含义	典型转移
Modified	本核有最新数据，内存可能旧	其他核读时必须提供或写回
Exclusive	本核独占且干净	本核写入后变 Modified
Shared	多个核可读共享	某核写入前要让其他副本失效
Invalid	本核没有有效副本	读写时要重新获取 line

两个 CPU 写同一 cache line 上的不同变量时，line 会在两个核心间来回转移。程序逻辑上写的是两个变量，硬件上争的是同一 line 的独占权。这就是伪共享。内核的 per-CPU 数据、cacheline 对齐、分片计数器，本质都是减少 line 状态来回迁移。

锁的真实成本：cache line 所有权迁移

自旋锁通常是一个小整数。获取锁时，CPU 执行原子读改写，必须拿到锁变量所在 cache line 的独占权。很多 CPU 同时争锁时，这条 line 在核心之间移动，互联流量和延迟迅速上升。

```
test_and_set_lock(lock):
    while atomic_exchange(lock, 1) == 1:
        cpu_relax()

unlock(lock):
    store_release(lock, 0)
```

这个简单锁在低竞争时很好，高竞争时很差。排队自旋锁把“所有 CPU 抢同一变量”变成“每个 CPU 等自己的节点”，减少缓存抖动。

```
queued_spin_lock:
    node = per_cpu_node()
    pred = atomic_xchg(lock.tail, node)
    if pred != null:
        node.locked = true
        pred.next = node
        while node.locked:
            cpu_relax()
    // enter critical section
```

Linux 的 queued spinlock 和 MCS 思想相关。这里的重点不是记算法名，而是理解：高竞争锁慢的主要原因常常不是临界区代码，而是 cache line 所有权在 CPU 间转移。

内存顺序：store buffer 为什么会改变可见顺序

CPU 为了性能会把 store 先放进 store buffer，稍后再对其他核心可见。单线程看不出问题，因为本 CPU 之后的 load 可以从 store buffer 转发；多核就可能看到不同顺序。


```
CPU0:
    data = 42
    flag = 1

CPU1:
    if flag == 1:
        assert(data == 42)
```

在弱顺序模型或编译器重排下，CPU1 看到 `flag == 1` 时不一定能看到 `data = 42`，除非使用 `release/acquire` 或锁。正确发布对象要写成：

```
CPU0:
    data = 42
    store_release(flag, 1)

CPU1:
    if load_acquire(flag) == 1:
        assert(data == 42)
```

OS 内核里同样适用。唤醒等待者前要先改变等待条件；发布文件对象前要先初始化字段；设备 `doorbell` 前要先让描述符对设备可见。屏障的含义永远要和“谁观察什么”一起写。

TLB 和 page walk cache

TLB 缓存虚拟页到物理页的翻译。现代 CPU 还可能有 page walk cache，缓存页表中间层结果。修改页表后，内存里的 PTE 变了，不代表 CPU 立即不用旧翻译。OS 必须按架构要求失效相关 TLB。

```
page walk for a 4-level page table:
    read top-level entry from page table root
    read next-level entry
    read next-level entry
    read final PTE
    check present and permissions
    insert translation into TLB
```

大页减少页表层级和 TLB 项数量，但权限粒度变粗。一个 2MiB 大页内只要有 4KiB 要改权限，就可能拆成 512 个小页。拆页会造成页表分配、TLB flush 和锁竞争，所以透明大页可能提升吞吐，也可能造成延迟尖峰。

TLB shutdown 的正确性边界

释放物理页前必须保证没有 CPU 还能通过旧 TLB 访问它。否则这页可能已经分配给别的进程或内核对象，旧翻译会变成越权访问。

```
safe unmap:
    remove PTE or clear permission
    flush local TLB entry
    send shutdown IPI to CPUs that may use mm
    remote CPUs invalidate entry and acknowledge
    wait for acknowledgements
    now old physical page may be freed or reused
```

这里的“may use mm”不是随便猜。内核要跟踪哪些 CPU 正在或最近运行某个地址空间，避免给所有 CPU 发 IPI。过度 shutdown 会让大机器扩展性下降；少发一个 shutdown 则是安全 bug。

中断控制器状态：pending、masked、in-service

设备触发中断后，中断控制器通常不是直接丢给 CPU 就结束。它会记录 pending，检查 mask 和优先级，把 vector 送到目标 CPU。CPU 进入 handler 后，

中断可能处于 in-service 状态；handler 结束时需要 EOI 或等价操作。

```
interrupt controller:
  if device asserts interrupt:
    mark vector pending
  if not masked and priority allows:
    deliver vector to target CPU
    mark vector in_service
  after handler sends EOI:
    clear in_service
    deliver next pending interrupt if any
```

若驱动没有清设备状态就 EOI，中断可能立刻再次触发，形成中断风暴。若 handler 清了设备状态但忘了 EOI，后续同类中断可能被阻塞。若 mask 过久，completion 延迟会飙升。中断处理顺序必须同时满足设备状态机和中断控制器状态机。

MSI-X：中断也是一次内存写消息

传统线中断像一根线被拉高。MSI/MSI-X 更像设备向特定地址写入一条消息，中断控制器把这条消息解释成某个 vector。这样设备可以有多个 vector，适合多队列网卡和 NVMe。

```
MSI-X setup:
  kernel allocates interrupt vectors
  kernel programs device MSI-X table
  each queue gets vector and CPU affinity
  device writes MSI-X message when queue completes
  target CPU enters corresponding handler
```

多队列设备性能依赖这个映射。若 64 个 RX/TX 队列都映射到一个 CPU，硬件有多队列也发挥不出来；若队列、CPU 和 NUMA 不匹配，DMA 数据会跨节点移动。驱动初始化时的 vector 分配不是细枝末节，而是性能结构。

PCIe 事务：BAR、doorbell 和 completion

PCIe 设备通过 BAR 暴露 MMIO 寄存器。CPU 写 doorbell 时，写请求会经过 root complex、switch、链路，到设备。写可能是 posted，CPU 不等待设备真正处理完成。设备完成 DMA 后，再通过 completion queue 和 MSI-X 通知 CPU。

```
CPU submit to PCIe device:
  CPU writes descriptors in RAM
  dma_wmb()
  CPU MMIO-writes doorbell BAR
  posted write travels through PCIe fabric
  device receives doorbell
  device DMA-reads descriptors
  device performs work
  device DMA-writes completion
  device sends MSI-X message
```

这个路径解释了为什么驱动不能把 MMIO 写当函数调用。doorbell 返回不代表设备完成；甚至不一定代表设备已经处理。若驱动需要确认某个 posted write 到达，常见做法是在平台规则允许时读回一个寄存器形成排序点。

NVMe 队列内部：phase tag 和 head/tail

NVMe submission queue 由主机写、设备读；completion queue 由设备写、主机读。completion queue 常用 phase tag 区分“这一格是新一轮写入还是上一轮旧值”。队列满、队列空、环绕时都要靠 head/tail 和 phase 正确判断。

```
completion poll:
  cqe = cq[head]
```

```

if cqe.phase != expected_phase:
    return no_new_completion
process cqe
head = head + 1
if head == queue_size:
    head = 0
    expected_phase = !expected_phase
write new head doorbell

```

如果没有 phase tag，环形队列回绕后，软件无法可靠区分旧 completion 和新 completion。这个细节体现了硬件队列的共同问题：数组槽位会复用，所以必须有 generation、phase、sequence 或所有权位。

IOMMU 页表和 IOTLB

IOMMU 给设备也建立翻译缓存。设备发出 IOVA，IOMMU 查页表得到物理地址，并检查读写权限。为了速度，IOMMU 也会有 IOTLB。撤销 DMA 映射后，必要时也要失效 IOTLB，否则设备可能继续用旧映射。

```

dma_unmap:
    remove IOVA -> physical mapping from IOMMU page table
    invalidate IOTLB for this device/domain/range
    only after invalidation may page be returned to allocator

```

这和 CPU TLB shutdown 是同一类正确性问题，只是访问者从 CPU 变成设备。设备侧隔离必须考虑 IOMMU group：若多个设备在硬件上不能区分请求者 ID，就不能把它们当成完全隔离的安全主体。

非一致 DMA 平台上的 cache 维护

有些平台上，设备 DMA 与 CPU cache 不自动一致。CPU 写给设备读的 buffer 前，需要 clean cache，把 dirty line 推到设备可见层；设备写给 CPU 读的 buffer 后，需要 invalidate cache，避免 CPU 读旧 line。

```

TO_DEVICE:
    CPU writes buffer
    clean cache lines for buffer
    device DMA reads memory

FROM_DEVICE:
    device DMA writes memory
    invalidate CPU cache lines for buffer
    CPU reads fresh data

```

这就是为什么驱动必须使用 DMA API，而不是直接把物理地址写进描述符。即使本书主线是 x86-64，DMA API 仍然负责表达方向、生命周期、IOMMU 映射和必要屏障。直接绕开 API 的驱动，可能在一个测试机上偶然工作，却在启用 IOMMU、不同 cache 属性或不同设备 DMA mask 时随机损坏数据。

存储介质内部状态：FTL 的写放大

SSD 对外暴露逻辑块，对内管理 NAND 页和擦除块。覆盖写不能直接改旧页，通常写到新物理页，再把旧页标为无效。垃圾回收把仍有效的页搬到新块，然后擦除旧块。主机随机写越多，内部搬迁越多，写放大越大。

```

FTL overwrite logical block L:
    old_phys = map[L]
    new_phys = allocate_free_flash_page()
    program new_phys with data
    map[L] = new_phys
    mark old_phys invalid
    later garbage_collect blocks with many invalid pages

```

TRIM/discard 告诉 SSD 某些逻辑块不再包含有效文件数据，FTL 可以把对应物理页当作无效页，降低后续垃圾回收成本。文件系统、块层和设备固件因此共同决定长期写入性能。

可靠提交：cache flush、FUA 和日志顺序

存储设备可能有易失缓存。块层写完成不一定表示掉电后存在。可靠提交要约束顺序：先写日志或数据，再 flush 或使用 FUA，最后提交元数据。设备必须向上层报告它支持哪些 flush 语义。

```
journal transaction:
  write descriptor block
  write data or metadata blocks
  issue flush if device cache is volatile
  write commit block with FUA or followed by flush
  transaction is durable after commit reaches stable media
```

若 flush 被设备虚假完成，文件系统再正确也无法保证崩溃恢复。若文件系统漏掉目录 fsync，应用的原子替换协议也会有洞。可靠性是应用、文件系统、块层和设备共同实现的契约。

硬件算法小结

- 能解释精确异常为什么让缺页可恢复。
- 能用 MESI 思想解释伪共享和锁竞争。
- 能说明 release/acquire 约束的是跨 CPU 可见顺序。
- 能写出 TLB shutdown 和 IOMMU unmap 的安全顺序。
- 能解释 MSI-X、多队列和 IRQ affinity 为什么影响 NVMe/网卡性能。
- 能说明 NVMe completion queue 为什么需要 phase tag。
- 能解释非一致 DMA 平台为什么必须 clean/invalidate cache。
- 能把 SSD FTL、TRIM、flush 和文件系统日志顺序联系起来。

1.2 存储层次、地址翻译与内存系统

cache、TLB 与内存层次：为什么“访问内存”不是一步

很多 OS 机制都卡在内存层次上：上下文切换为什么贵，任务迁移为什么掉性能，页表修改为什么要 TLB shutdown，锁为什么在多核下慢，DMA 为什么要 cache 维护。若把内存当成一个均匀数组，这些问题都解释不清。

内存层次的基本矛盾

CPU 很快，DRAM 慢，存储设备更慢。为了让程序大多数时间不等最慢层，系统做了多层缓存：寄存器、L1/L2/L3 cache、TLB、内存、page cache、设备缓存。每一层都在用小容量换低延迟。

层次	保存什么	OS 关心点
寄存器	当前计算状态	上下文切换、ABI、系统调用参数
L1/L2/L3 cache	最近访问的内存 cache line	局部性、伪共享、锁竞争
TLB	最近虚拟页翻译	地址空间切换、页表修改、shutdown
DRAM	物理页	页分配、NUMA、回收

page cache	文件页副本	文件 I/O、writeback、mmap
设备缓存	磁盘/SSD/网卡内部状态	flush、FUA、completion 语义

OS 的工作经常是维护这些层次之间的一致解释：页表说这个虚拟页映射到哪个物理页，TLB 不能有旧答案；文件系统说数据已提交，设备缓存和稳定介质之间不能混淆；DMA 说设备已写内存，CPU cache 不能继续读旧数据。

cache line 和局部性

cache 不按字节搬运，而按 cache line 搬运。假设 cache line 是 64 byte，程序读取 `array[i]` 时，硬件可能把 `array[i]` 附近 64 byte 都搬进 cache。顺序遍历数组很快，因为空间局部性好；链表随机跳转慢，因为每个节点可能在不同 cache line。

```
good locality:
for i in 0..n:
    sum += array[i]

poor locality:
node = head
while node:
    sum += node.value
    node = node.next
```

内核热路径偏爱数组、per-CPU 数据、紧凑结构和批量处理，不是代码风格偏好，而是 cache 行为决定的。调度器、网络栈、块层都要避免把热点状态分散到随机内存。

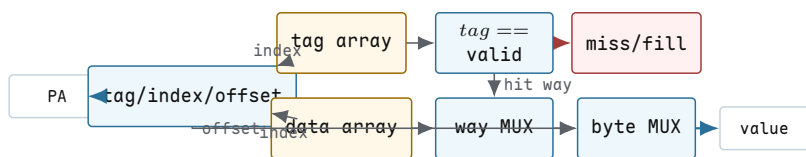
cache 的内部结构：tag、set、way、line

cache 不是“一个更快的内存数组”。它至少包含 data array、tag array、valid bit、dirty bit、一致性状态和替换状态。地址进入 cache 后，硬件按 index 选择 set，并行比较多个 way 的 tag。命中条件可写成：

$$hit = \exists way : valid[set, way] \wedge tag[set, way] = tag(PA)$$

若 line 大小为 L ，set 数为 S ，物理地址 PA 的基本拆分是：

$$line = \left\lfloor \frac{PA}{L} \right\rfloor, \quad set = line \bmod S, \quad offset = PA \bmod L, \quad tag = \left\lfloor \frac{line}{S} \right\rfloor$$



图示：cache hit 的硬件路径：index 选 set，tag 并行比较，way MUX 选中 line，offset 选出字节或字。

容量变大，tag/data array 更大，访问延迟可能增加；相联度变高，tag 比较和 way MUX 更复杂，也可能增加延迟。于是 L1 通常小而快，L2/L3 更大但更慢。它们不是“越大越好”的同一种 cache，而是在延迟、容量、功耗、面积之间取不同点。

$$AMAT = T_{L1} + MR_{L1} \times (T_{L2} + MR_{L2} \times (T_{L3} + MR_{L3} \times T_{mem}))$$

这条式子比“cache miss 很慢”更准确。优化内核热路径时，要么降低命中时间 T ，要么降低 miss rate MR ，要么降低 miss penalty。顺序预取、批处理、per-CPU 数据、减少指针追踪，本质上都在影响这三个量。

上层怎样接 cache：load/store、原子操作和页属性

CPU 上层给 cache 的请求来自取指、load、store、原子读改写和硬件 page walk。普通 load 只需要读 line；store 需要取得 line 的可写所有权；原子操作还要把读、修改、写做成对其他 CPU 可见的不可分割序列。

```
load PA:
    require line in Shared or Exclusive/Modified state
    return selected bytes

store PA:
    obtain line ownership
    modify selected bytes
    mark line dirty

locked atomic PA:
    obtain exclusive ownership
    block competing ownership changes for this operation
    read old value, compute new value, write new value
```

OS 通过页表属性影响 cache 行为。普通 RAM 映射为 write-back；MMIO 通常映射为 uncacheable 或 device 类属性；framebuffer 可能用 write-combining；持久内存还要考虑 cache line flush 和持久化屏障。页表项不是只管权限，也影响访问到达下层硬件的方式。

映射属性	适用对象	错配后果
write-back	普通 DRAM	性能最好，靠一致性协议维护 CPU 间可见性
uncacheable/device	MMIO 寄存器	若误设可缓存，设备寄存器读写可能被合并或读旧值
write-combining	framebuffer、顺序写窗口	适合批量写，不适合需要严格读回的寄存器
write-through	少见特殊场景	写延迟高，但下层更快得到更新

这就是为什么驱动不能把 MMIO 当普通指针。访问普通内存的 `*ptr = value` 可以被编译器和 CPU 优化；访问设备寄存器要用 `readl/writel` 一类 API，API 同时表达 volatile 语义、宽度、字节序和必要排序。

下层怎样接 cache: fill、evict、writeback

cache miss 不是直接“去内存拿一个变量”。L1 miss 会向 L2 发 line fill；L2 miss 向 L3；L3 miss 向内存控制器或互联。若要替换的旧 line 是 dirty，还要先 writeback 到下层。

```
cache_miss(PA):
    victim = choose_way(set(PA))
    if victim.valid and victim.dirty:
        send writeback(victim.line_address, victim.data)
    send read_for_ownership_or_shared(PA.line_address)
    wait for lower level response
    install returned line with tag, data, state
    replay waiting load/store
```

这条路径解释了为什么同一个 load 有时很快，有时慢几个数量级。若 L1 命中，路径短；若一路 miss 到 DRAM，要经过互联、内存控制器、DRAM bank/row；若还遇到 dirty victim writeback 或其他 core 的所有权竞争，延迟更高。

$$T_{miss} = T_{lookup_lower} + T_{queue} + T_{transfer} + T_{install} + T_{replay}$$

写回 cache 和 dirty line

CPU 写一个变量时，通常先写 cache line，把它标为 dirty。dirty line 何时写回内存由硬件决定。对其他 CPU，cache coherence 协议会维护普通内存一致性；对设备 DMA，则要看平台是否 DMA coherent。

```
CPU writes buffer:
  cache line becomes dirty
  DRAM may still have old bytes

device DMA reads buffer:
  if platform not coherent and driver forgot cache clean:
    device may read old bytes
```

所以驱动使用 DMA API，而不是手写物理地址。API 的一部分工作就是按平台需要做 cache clean/invalidate 和 IOMMU 映射。

伪共享：没有共享变量也会共享 cache line

两个 CPU 写不同变量，但变量落在同一 cache line，也会互相抢 line 所有权。

```
struct Counters {
  atomic<uint64_t> a; // CPU0 writes
  atomic<uint64_t> b; // CPU1 writes, same cache line
}
```

从程序语义看，a 和 b 独立；从硬件看，它们在同一 cache line。两个 CPU 都要写，line 在 CPU0 和 CPU1 之间来回迁移。解决方法是 padding 或 per-CPU 分片。

```
struct Counters {
  alignas(64) atomic<uint64_t> a;
  alignas(64) atomic<uint64_t> b;
}
```

OS 设计 runqueue、统计计数器、slab cache 时经常这么做。性能问题不是“算法复杂度”一个维度，cache line 也是复杂度的一部分。

多核一致性：写共享变量为什么会拖慢

多核 CPU 必须让普通内存看起来有一致解释。常见教学模型用 MESI 类状态描述一条 cache line 的所有权：Modified、Exclusive、Shared、Invalid。真正实现可能更复杂，但核心事实是：写某条 line 需要独占所有权，其他 CPU 的同一 line 副本要失效或降级。

状态	含义	对 OS 的影响
M	本 CPU 拥有脏副本，下层可能旧	迁移或回收前要理解 dirty writeback 不是同步点
E	本 CPU 独占干净副本	第一次写可本地升级，成本低
S	多个 CPU 可读共享副本	读共享便宜，写需要失效别人
I	本 CPU 没有有效副本	访问要重新获取 line

自旋锁慢，不只是“很多线程等一个变量”。锁变量所在 cache line 会在多个 CPU 间反复转移所有权：

$$CPU_0 \text{ write(lock)} \Rightarrow line @ CPU_0 = M, \quad CPU_1 \text{ atomic(lock)} \Rightarrow line \text{ moves to } CPU_1$$

如果同一 line 还放了其他热字段，那些字段也会被拖着迁移。per-CPU 计数器、cacheline padding、队列分片、RSS 多队列，本质上都是减少 cache line ownership 在 CPU 间来回跑。

内存屏障：cache 一致性不等于执行顺序

cache 一致性解决“同一地址最终按某种一致规则被看见”，不自动解决“不同地址按程序写的顺序被别人看见”。CPU store buffer、load buffer、乱序执行和编译器优化都会改变可见顺序。锁、原子变量、设备 doorbell、DMA descriptor 都需要明确顺序。

```
wrong DMA publish:
    desc.addr = dma_addr
    desc.len = len
    doorbell = tail           // device may see doorbell before descriptor fields

right DMA publish:
    desc.addr = dma_addr
    desc.len = len
    dma_wmb()
    writel(tail, DOORBELL)
```

这里的屏障不是为了“让 CPU 慢一点”，而是为了建立跨主体可见顺序：

descriptor writes visible to device < doorbell visible to device

普通锁也一样。unlock 前的临界区写入必须先于别的 CPU acquire lock 后读取；否则锁只保护了代码形状，没有保护内存可见性。C++20 的 acquire/release、Linux 的 `smp_load_acquire/smp_store_release` 都是在表达这个关系。

TLB 为什么存在

虚拟地址翻译需要查页表。多级页表可能访问多次内存。TLB 缓存最近翻译，让常见内存访问不必每次 page walk。

```
load [va]:
    if TLB has va page:
        pa = cached translation
    else:
        pa = page_table_walk(va)
        insert into TLB
    load cache line at pa
```

TLB 命中时，虚拟内存开销很低；TLB miss 时，硬件要走页表，可能触发更多 cache miss。程序工作集太大、随机访问太多、页太小，都可能让 TLB miss 增加。

TLB 和 cache 的先后关系

教学时可以说“先翻译 VA 到 PA，再查 cache”，但真实 CPU 为了快会并行化部分步骤。例如 VIPT L1 cache 可以用页内 offset 里的 index 位先选 set，同时 TLB 查 tag 所需的物理页号。无论微架构怎样优化，架构契约必须满足：

$$VA \xrightarrow{TLB/page\ walk} PA, \quad permission(PTE, access, CPL) = allow \Rightarrow load/store \text{ may commit}$$

若权限失败，访问不能提交。若只是 data cache miss，访问可以等待下层填充后提交。二者性质完全不同：

事件	硬件性质	OS 是否介入
----	------	---------

L1/L2/L3 cache miss	性能事件，硬件向下层取 line	通常不介入，只能通过性能计数观察
TLB miss 且 page walk 成功	性能事件，硬件填 TLB	通常不介入
PTE not present	架构异常 #PF	OS 查 VMA，分配页、读文件或发信号
权限不允许	架构异常 #PF/#GP	OS 判断 COW、mprotect 或非法访问
非 canonical 地址	架构异常	OS 通常向用户报告错误或处理内核 bug

这张表是理解虚拟内存的关键。cache miss 是“硬件还没拿到数据”；page fault 是“硬件不能合法完成这次访问”。二者都可能让程序慢，但只有后者是 OS 语义路径。

页表项是 OS 和硬件共享的数据结构

x86-64 PTE 不是普通整数。硬件 page walker 读取它，OS 修改它，TLB 缓存它的结果。PTE 里既有物理页框号，也有 present、writable、user、accessed、dirty、global、NX 等权限和状态位。

PTE 字段	硬件怎样用	OS 怎样用
present	不为 1 时访问触发 #PF	懒分配、换入、文件页加载
RW	写访问权限	COW、mprotect、只读代码页
US	用户态是否可访问	用户/内核隔离，SMAP 配合
XD/NX	是否禁止取指	W^X、栈不可执行
accessed	硬件可置位	页面回收估计热度
dirty	写入后硬件可置位	回写、COW、文件脏页判断
global	CR3 切换时可保留	内核全局映射减少 TLB 损失

因此页表锁不是形式主义。OS 修改 PTE 时，要同时考虑硬件可能正在 page walk，其他 CPU 可能缓存旧翻译，回收线程可能看 accessed/dirty，文件系统可能等脏页回写。页表是硬件和内核并发访问的共享结构。

TLB shutdown 为什么昂贵

当一个 CPU 修改某进程页表，例如取消映射某页，其他 CPU 可能还在 TLB 里缓存旧翻译。如果旧物理页被释放并重用，其他 CPU 用旧翻译访问就会破坏隔离。因此内核必须让相关 CPU 失效旧 TLB 项。

```
unmap page:
  remove PTE
  find CPUs that may run this address space
  send IPI to those CPUs
  each CPU invalidates TLB entry
  wait for acknowledgement
  now physical page can be freed
```

这涉及跨 CPU 中断和等待，所以昂贵。x86-64 内核会批量 unmap、延迟释放、使用 PCID 减少全局刷新。页表不是普通 map，TLB 让它变成多核协议。

PCID：避免每次切换都清空 TLB

若 TLB 只缓存虚拟页到物理页，不区分进程，那么切换地址空间时必须清空，否则进程 B 的虚拟地址可能命中进程 A 的翻译。x86-64 的 PCID 给 TLB 项加地址空间标签，让不同进程的同一虚拟地址可共存。PCID 不是进程 ID，也不是 pid；它是处理器用于区分 TLB 项归属的短标签，由内核分配、复用和失效。

```

TLB entry:
  virtual_page
  pcid
  physical_page
  permissions

```

这降低上下文切换成本，但没有消除页表修改成本。若某个地址空间自己的 PTE 改了，对应 PCID 的旧项仍要失效。PCID 解决的是“不同地址空间的旧翻译不要互相污染”，不解决“同一地址空间的 PTE 更新后旧翻译仍然存在”。

问题	PCID 能否解决	还需要什么
进程切换后保留部分 TLB 热度	能	正确设置 CR3/PCID 和保留策略
进程 A 的虚拟页误命中进程 B 的翻译	能	TLB key 必须包含 PCID
<code>mprotect</code> 把页从可写改为只读	不能自动解决	invalidate 当前 PCID 的目标页
<code>munmap</code> 后释放物理页	不能自动解决	shutdown 所有可能持有旧翻译的 CPU
PCID 被复用给另一个地址空间	不能自动解决	复用前清理旧 PCID 相关 TLB 项

NUMA：物理内存也有远近

多插槽机器上，每个 CPU 到本地内存控制器更近，到远端内存更远。统一的“物理内存池”只是抽象，硬件上访问延迟和带宽不同。OS 调度和页分配要尽量让线程运行在靠近其内存的 CPU 上。

```

first touch policy:
  thread on node 0 first writes page
  kernel allocates physical page from node 0

bad migration:
  thread moves to node 1
  still accesses node 0 memory
  remote latency increases

```

这解释了 CPU affinity、NUMA balancing、内存绑定策略。并不是核心越多越快，线程和内存分布不匹配会让远端访问拖慢程序。

page cache：文件 I/O 的内存层

读文件时，内核通常先把磁盘块读到 page cache，再从 page cache 拷贝给用户。再次读取同一页时，不需要访问设备。写文件时，数据可能先进入 page cache 并标记 dirty，稍后 writeback 到设备。

```

read(file, buf):
  page = page_cache_lookup(file, offset)
  if page missing:
    page = read_from_disk(file, offset)
    insert into page cache
  copy page bytes to user buffer

```

这解释了为什么 `write` 返回成功不等于落盘。它可能只是写入 page cache。可靠持久化需要 `fsync`、日志、`flush/FUA` 等机制把数据推进到稳定介质。

四种 cache 不要混在一起

很多读者看到 cache 就只想到“快一点的内存”。在 OS 里至少要区分四种名字相似但语义完全不同的缓存。

缓存	缓存的是什么	谁负责一致性
CPU data cache	物理内存中的 cache line	硬件一致性协议，DMA 时还要驱动/API 配合
I-cache	指令字节	硬件和内核在代码修改、JIT、模块加载时配合
TLB	虚拟页到物理页和权限	MMU 填充，内核在改 PTE 后失效旧项
page cache	文件内容的内存副本	内核文件系统、回写线程、fsync 和设备 flush
设备缓存	SSD/NIC/磁盘控制器内部队列或缓存	驱动命令、flush/FUA、设备完成语义

这几种缓存的错误后果不同。CPU data cache 错了，CPU 可能读到旧字节；TLB 错了，权限或物理页可能错；page cache 错了，文件语义和落盘语义错；设备缓存错了，崩溃恢复可能错。调试时必须先问“哪一层缓存给了旧答案”，不能把所有现象都叫 cache miss。

一条 load 的完整时间线

把 `mov rax, qword ptr [rdi]` 拆成时间线，可以看清 cache、TLB、页表和异常的先后关系。

1. decode instruction and read rdi
2. compute virtual address
3. check canonical form
4. lookup D-TLB by virtual page and PCID
5. if TLB miss, walk x86-64 page tables from CR3
6. check PTE present, user/supervisor, writable, NX as needed
7. combine physical frame with page offset
8. access L1 data cache by physical line
9. on cache miss, fetch line from lower cache or DRAM
10. only if all checks pass, write loaded value into rax

异常可能发生在多个位置：非 canonical 地址在 page walk 前失败；PTE 不存在或权限不符时 page fault；物理地址命中 MMIO 时变成设备访问；普通 DRAM cache miss 只是性能事件，不是异常。OS 处理 page fault 时，面对的是第 5/6 步失败，不是第 8/9 步变慢。

三条页表修改路径：COW、mprotect 和 munmap

页表修改不能只说“改 PTE”。不同路径要维护的所有权不同，因此 TLB 和物理页生命周期也不同。

路径	页表动作	必须同步的事实
COW 写 fault	分配新物理页，把当前进程 PTE 改到新页并设 writable	旧共享页引用计数减少；当前 CPU 旧只读翻译要失效或被覆盖
mprotect 收紧权限	原 PTE 保留同一物理页，但清 writable 或 executable	所有 CPU 的旧权限翻译必须失效
munmap	删除 PTE，并可能释放物理页	释放前必须等远端 TLB 不再能访问旧页

进程退出

批量拆整棵地址空间

可以批量 shutdown，但页释放仍要等旧翻译安全消失

用状态机写会更清楚：

```

unmap_and_free(mm, va):
    lock page table
    pte = clear_pte(mm, va)
    page = pte.physical_page
    record_tlb_flush(mm.pcid, va)
    unlock page table

    send shutdown to CPUs that may run mm
    wait for acknowledgements

    drop page reference
    if page.refcount == 0:
        free physical page

```

这段顺序中，最不能提前的是释放物理页。如果先把页交给分配器，远端 CPU 又用旧 TLB 写了一次，就会把新对象写坏。很多内存安全问题并不是“页表 map 写错了”，而是页表、TLB、引用计数和跨 CPU 确认之间少了一步。

PCID 的复用为什么也要有生命周期

PCID 数量有限，不可能永远给每个历史进程一个新标签。内核复用 PCID 时，要保证旧地址空间留下的 TLB 项不会被新地址空间误用。简单模型如下：

```

assign_pcid(mm):
    if mm has active pcid on this CPU:
        return mm.pcid

    pcid = choose reusable pcid
    if pcid belonged to another mm:
        invalidate all non-global TLB entries for pcid
    bind pcid to mm on this CPU
    return pcid

```

PCID 的局部性也很重要。同一个进程在 CPU 0 上的 PCID 状态，不能直接等价于 CPU 7 上的状态；每个 CPU 有自己的 TLB 和 PCID 使用历史。于是 shutdown 既要知道地址空间，也要知道哪些 CPU 可能缓存过这个地址空间。调度器、地址空间切换和 TLB 子系统因此耦合在一起。

从 page cache 到 CPU cache：文件页的两层身份

文件内容进入内存后，首先成为 page cache 中的页；CPU 读取这页时，又会把其中的 cache line 放进 CPU data cache。page cache 是内核管理的“文件页对象”，CPU cache 是硬件管理的“物理 line 副本”。它们名字都叫 cache，但层级和所有权完全不同。

file offset → page cache page → physical page → CPU cache line

一次 read 和一次 mmap 读同一个文件页，可能最终访问同一个 page cache page；区别是 read 把字节拷到用户 buffer，mmap 让用户虚拟地址直接映射到这个页。无论哪条路径，CPU 执行 load 时仍会经过 TLB 和 data cache。

层次

管理者

一致性问题

page cache	内核 VFS/文件系统/mm	文件偏移、脏页、writeback、truncate、mmap 一致性
CPU cache	硬件一致性协议和页属性	line 命中、dirty line、跨 CPU 所有权、DMA 同步
设备缓存	设备控制器和驱动协议	flush/FUA、completion、掉电恢复

`write` 返回成功通常只说明数据进入内核管理的内存状态，不说明设备稳定保存。若 page cache 页变 dirty，writeback 线程稍后把它交给块层；块设备 completion 说明设备处理了请求；`fsync` 还要保证文件系统日志和设备 flush 语义满足崩溃恢复要求。

DRAM 到 OS 页分配：不是一块均匀大数组

固件把物理内存地图交给内核。内核要排除固件保留区、MMIO hole、内核镜像、initrd、ACPI 表、崩溃内核保留区，再把可用 RAM 切成 page frame。NUMA 系统还要把页按 node 管理。

```
boot memory setup:
  read firmware memory map
  remove reserved and MMIO ranges
  mark usable DRAM ranges
  split ranges into page frames
  assign frames to NUMA nodes and zones
  initialize buddy allocator and per-cpu page lists
```

页分配器不是只维护一个 free list。它要处理 DMA zone、normal zone、high memory 历史约束、NUMA node、水位线、迁移类型、巨页、CMA、内存热插拔和内存错误下线。硬件给的是不连续、带属性、带距离和可靠性信息的物理地址空间；OS 把它整理成可分配页。

内存错误：ECC、MCE 和 page offline

DRAM 位错误可能被 ECC 纠正，也可能不可纠正。CPU 或内存控制器通过 machine check、EDAC 日志或平台错误接口报告。OS 要把硬件错误映射到物理页、进程和恢复策略。

错误类型	硬件现象	OS 策略
corrected error	ECC 修复，程序未必失败	记录地址，统计趋势，必要时预防性下线页
uncorrected recoverable	某次访问无法修复但系统可定位	杀死受影响进程或下线页
fatal machine check	CPU 无法保证继续执行安全	panic 或平台复位

这说明“物理页”也有生命周期和可靠性。内核可以把某页标记为不再分配；文件页可重新从磁盘读，匿名页可能导致进程失败；内核关键页出错更严重。硬件错误处理把 DRAM 物理事实带进了 OS 策略。

本节验收

- 能说明 cache line 为什么会让顺序数组快、随机链表慢。
- 能画出 tag/set/way/offset 的 cache lookup，并写出 set/index 公式。
- 能用 AMAT 解释 L1/L2/L3/DRAM 延迟怎样叠加。

- 能解释伪共享为什么会让两个独立计数器互相拖慢。
- 能说明 MESI 类状态为什么让写共享变量产生 line ownership 迁移。
- 能区分 cache miss、TLB miss、page fault 和权限 fault。
- 能画出虚拟地址访问经过 TLB、页表、cache、内存的路径。
- 能解释 PTE 字段如何同时服务硬件 page walk 和 OS 内存策略。
- 能说明修改页表后为什么要 TLB shutdown。
- 能解释 PCID 解决什么，不解决什么。
- 能解释 NUMA 为什么让线程迁移和页分配有关。
- 能区分 CPU cache、TLB、page cache、设备缓存和稳定介质。
- 能解释 DRAM/ECC 错误如何映射到 page offline 或进程失败。

第一阶段深讲：内存系统怎样塑造虚拟内存

操作系统的内存管理不是“把内存分给程序”这么简单。硬件内存系统由 DRAM、cache、TLB、MMU、内存控制器、NUMA 拓扑和 DMA 可见性共同组成。OS 的地址空间、页表、VMA、COW、page cache、伙伴分配器和回收算法，都是在这些硬件条件下形成的。

物理内存首先是一张不连续地图

初学者容易把物理内存想成从 0 到 N 的连续数组。真实机器启动时，固件会告诉内核哪些范围可用，哪些被固件、设备 MMIO、ACPI 表、内核镜像、initrd、显存窗口或保留区域占用。内核不能把保留区当普通页分配。

```
memory map examples:
usable RAM
reserved firmware region
kernel image and initrd
MMIO aperture for PCIe devices
ACPI tables and UEFI memory map
crash kernel reserved area
```

因此页分配器的第一步是解析内存地图，建立可用页框集合，而不是假设所有地址都能清零后使用。每个物理页还要记录状态：是否可分配，属于哪个 NUMA node，是否被保留，是否在 LRU，是否 dirty，是否被 DMA pin，是否有硬件错误。

页是硬件和 OS 的折中粒度

页太小，页表和 TLB 压力大；页太大，内部碎片和权限粒度变差。4 KiB 页是许多系统的基础粒度，但大页和巨大页用于降低 TLB miss。OS 在页粒度上做权限、映射、换入换出、COW 和 page cache，是因为硬件 MMU 也按页翻译。

粒度	优点	代价
小页	权限细，碎片小，按需分配灵活	页表项多，TLB 覆盖范围小
大页	TLB 覆盖范围大，page walk 少	分配连续内存难，拆分成本高
cache line 块设备扇区/块	硬件缓存和一致性粒度 持久存储 I/O 粒度	伪共享、写放大、对齐敏感 与页大小不总是一致

这张表说明一个重要原则：OS 很少只有一个粒度。CPU cache 按 cache line 传输，MMU 按页翻译，文件系统按块管理，设备按队列请求传输。性能问题常出现

在粒度转换处。

虚拟地址先落到 VMA，再落到 PTE

VMA 描述一段虚拟地址区间的高层语义：权限、是否匿名、是否文件映射、是否共享、增长方向、文件偏移。PTE 描述某一页当前的硬件翻译：是否 present、物理页号、读写执行权限、dirty/accessed 位等。二者不是同一个东西。

```
virtual access:
  find VMA by fault address
  if no VMA: invalid access
  if VMA permissions reject access: protection fault
  inspect or create PTE
  if needed allocate page or read file page
  install PTE and return
```

VMA 是“这段地址应该有什么语义”，PTE 是“这页现在如何被硬件翻译”。一个合法 VMA 可以暂时没有 PTE，因为懒分配或文件页尚未读入。一个 PTE 也必须受 VMA 约束，不能凭空给出 VMA 不允许的写或执行权限。

页表层级来自地址位数和稀疏性

若为整个虚拟地址空间建立一个线性页表，会巨大且浪费。多级页表把虚拟地址拆成若干索引，只为实际使用的区域分配中间页表页。地址空间越稀疏，多级页表越有意义。

```
virtual address split:
  top-level index
  middle-level index
  lower-level index
  page-table index
  page offset
```

page walk 的成本来自多次内存访问。TLB 缓存最终翻译，是为了避免每次 load/store 都走多级页表。大页减少层级和 TLB 项数量，但要求地址和物理内存按大页对齐，并牺牲一部分细粒度权限。

PTE 位是硬件和内核共同维护的协议

一个 PTE 通常不仅有物理页号，还包含权限和状态位。不同架构位名不同，但角色相似：present 决定是否有当前翻译，writable 决定写入是否允许，user/supervisor 决定用户态能否访问，execute disable 决定能否取指，accessed/dirty 帮助内核判断冷热和写回。

PTE 信息	硬件用法	内核用法
present	缺失时触发 page fault	懒分配、swap、文件缺页
writable	写保护检查	COW、只读映射、mprotect
user/supervisor	用户态访问检查	内核页隔离、用户地址空间
execute permission	取指检查	WX、代码页保护
accessed/dirty	访问和写入记录	回收、写回、冷热判断
physical frame	最终物理页	反向映射、引用计数、迁移

PTE 是典型的软硬件共享数据结构。硬件会读取甚至更新某些位，内核会修改权限、清理位、替换物理页。凡是共享，就有一致性问题：修改 PTE 后，TLB 中的旧翻译是否还有效？其他 CPU 是否也缓存了旧结果？这些问题推导出 TLB flush 和 shutdown。

缺页处理是一棵决策树

page fault handler 不能只看“地址不存在”。它要区分访问类型、VMA 类型、PTE 状态、进程状态和内存压力。

```

handle_page_fault(address, access):
    vma = find_vma(address)
    if vma missing: signal segmentation fault
    if access violates vma permissions: signal protection fault
    if anonymous and pte missing: allocate zero page
    if file-backed and pte missing: read page cache or storage
    if write to COW page: copy or reuse page
    if swapped out: read swap entry
    if allocation fails: invoke reclaim or OOM policy

```

同样是 fault，结果可能是成功分配、读文件、复制 COW 页、阻塞等待 I/O、触发回收、杀进程或向用户发送信号。把所有 fault 都理解为“程序错了”，会误解虚拟内存。

COW 的关键是同时保护父子和旧 TLB

fork 之后，父子共享物理页。为了让任何一方第一次写入都被内核截获，父子 PTE 都必须改成只读，并标记 COW。若只改子进程，父进程可以继续写共享页。若改了 PTE 但没有处理 TLB，CPU 可能继续使用旧 writable 翻译。

```

fork COW:
    for each writable private page:
        clear writable in parent PTE
        install read-only child PTE
        mark both COW
        increment physical page refcount
        invalidate stale writable TLB translations

```

COW 写 fault 的快慢取决于页是否仍共享。如果引用计数为 1，说明只剩当前进程，直接恢复 writable 即可。否则分配新页、复制内容、更新 PTE、减少旧页引用。这里的引用计数必须和页表更新同步，否则可能释放仍被映射的物理页。

TLB shutdown 是多核虚拟内存的税

一个进程可能在多个 CPU 上运行过，或者同一地址空间的多个线程正在不同 CPU 上执行。内核修改某个地址的 PTE 后，要确保所有可能使用旧翻译的 CPU 都失效对应 TLB 项。

```

shutdown timeline:
    CPU0 modifies PTE
    CPU0 records target CPUs using this mm
    CPU0 sends IPI to CPU1 and CPU2
    remote CPUs run interrupt handler and invalidate TLB entry
    remote CPUs acknowledge
    CPU0 may now free or reuse the old page

```

这就是为什么大量小范围 mprotect、munmap 或频繁映射变化可能在多核机器上很贵。成本包括 IPI、远端中断处理、等待确认和丢失 TLB 热状态。批量 unmap 和延迟释放的很多设计，都是为了减少这笔税。

cache 一致性不等于程序没有数据竞争

硬件 cache coherence 通常保证同一个物理地址的 cache line 在多个 CPU 间遵守一致性协议。但这不等于普通读写自动形成程序想要的同步。两个 CPU 同时更新共享变量，仍可能丢失更新；一个 CPU 写数据再写 flag，另一个 CPU 先看到 flag 后看到旧数据，也可能在弱内存模型上发生。

```

producer:
    data = 42
    release_store(ready, true)

consumer:

```

```
if acquire_load(ready):
    use(data)
```

锁、原子操作和内存屏障的目标是建立跨 CPU 可见顺序，不是简单地“让 cache 生效”。OS 内部的 runqueue、引用计数、页状态、设备队列，都需要这种顺序保证。

NUMA 让“内存速度”不再单一

多 socket x86-64 机器中，CPU 访问本地内存和远端内存延迟不同、带宽不同。OS 要知道 CPU 和内存 node 的拓扑，尽量让任务运行在靠近其内存的位置，或者把页迁移到任务常运行的 node。

策略	目的	代价
first touch	谁先写页就从谁本地 node 分配	初始化线程位置会影响后续性能
CPU affinity	让任务少迁移，保持缓存和内存局部性	负载不均时可能浪费 CPU
page migration	把热页迁到更近 node	迁移本身消耗带宽和 CPU
interleave	在多个 node 间分散页	降低热点但增加部分远端访问

NUMA 问题常被误判为算法慢。同一程序、同一输入，线程绑核和页放置不同，性能可能差很多。OS 调度器和内存管理必须共同考虑这种硬件拓扑。

page cache 把内存和存储连接起来

文件数据读入内存后，不应每次 read 都访问设备。page cache 用内存缓存文件页，普通 read 从缓存复制，普通 write 先写缓存再标 dirty。这样存储 I/O 被合并、重排和批量化，代价是持久化语义变复杂。

```
file read miss:
    allocate folio in page cache
    map file offset to disk block
    submit block I/O
    wait for completion
    mark folio uptodate
    copy to user

file write:
    copy from user to folio
    mark dirty
    writeback later or on fsync
```

page cache 也参与内存回收。clean 文件页可以直接丢弃，将来再从文件读；dirty 文件页要先写回；匿名页不能从文件重读，只能保留、换出到 swap 或丢弃进程。内存管理和文件系统因此不是两套独立机制。

回收和 OOM 是内存承诺失败后的策略

当空闲页不足，内核先尝试回收：丢 clean page cache，写回 dirty 页，换出匿名页，收缩 slab 缓存。如果仍无法满足分配，就进入 OOM 策略，选择牺牲某个进程或让分配失败。

```
allocation slow path:
    try per-cpu page lists
    try buddy free lists
    reclaim clean file pages
    write back dirty pages if allowed
    swap anonymous pages if configured
    compact memory for high-order allocation
    invoke OOM policy if still failing
```

不同分配上下文允许的动作不同。中断上下文不能睡眠等待磁盘 I/O；持锁路径不能随意进入会回调同一锁的回收；高阶分配可能因为碎片失败，即使总空闲内存不少。内存管理的难点在这些上下文约束。

内存系统的最低验收

第一阶段读者应该能解释：为什么 VMA 合法但 PTE 可以不存在；为什么 COW 要写保护父子双方；为什么改 PTE 后要处理 TLB；为什么 cache 一致性不等于数据竞争安全；为什么 page cache 同时属于文件系统和内存管理；为什么 OOM 不一定表示机器真的没有任何空闲字节。

这些问题回答清楚，后面的虚拟内存、文件系统和性能分析才有根基。

第一阶段源码走读：x86-64 #PF 到 VMA、PTE 与 COW

前面已经讲过虚拟地址、VMA、PTE、TLB 和 COW，但只讲概念还不够。真正读懂操作系统，必须能把一次用户态 load/store 失败拆成硬件异常、入口现场、VMA 判定、PTE 状态、缺页修复、TLB 失效和返回重试。x86-64 上这条路径的核心异常是 #PF，Linux 里它从 `arch/x86/mm/fault.c` 进入，再落到 `mm/memory.c` 的通用内存管理逻辑。

核心思想

#PF 不是“程序访问错了内存”的同义词。它只是 CPU 告诉内核：“这次地址翻译或权限检查没有按当前页表完成。”内核必须结合 CR2、error code、当前上下文、VMA 和 PTE，才能决定是分配零页、读文件页、执行 COW、返回 `-EFAULT`，还是给进程发送信号。

硬件交给内核的最小证据

x86-64 上发生 page fault 时，CPU 至少提供三类证据：faulting virtual address、错误码和被打断现场。faulting address 由 CR2 保存；错误码说明这是不是 present 页上的保护错误、是不是写访问、是不是用户态访问、是不是取指访问等；入口保存的寄存器现场说明 fault 发生在什么 RIP、什么 RSP、什么 CPL。

证据	典型来源	OS 需要回答的问题
fault address	CR2	这个地址属于哪个 VMA，是否 canonical
error code.P	page fault error code	是 PTE 不存在，还是 present 页权限拒绝
error code.W/R	page fault error code	是读、写还是取指触发
error code.U/S	page fault error code	是 Ring 3 访问，还是内核路径访问
trap frame	异常入口保存	返回后重试哪条指令，还是转成信号/错误

这张表解释了为什么“缺页处理”不能只写成 `if page missing then allocate`。同样是 #PF，缺少 PTE 可能是匿名懒分配，present 页写保护可能是 COW，用户态访问无 VMA 是 `SIGSEGV`，内核 `copy_from_user` 访问坏地址应该返回 `-EFAULT`，内核普通指针 fault 则可能是 oops。

源码地图：从 x86-64 异常到通用 mm

阅读 Linux 时，先不要在整个源码树里搜索“page fault”到处跳。应该按层次看：

路径	角色	阅读重点
----	----	------

<code>arch/x86/mm/fault.c</code>	x86-64 page fault 入口	读取 CR2、解释 error code、区分用户/内核上下文
<code>arch/x86/entry/entry_64.S</code>	异常入口汇编	trap frame、寄存器保存、进入 C 函数前的状态
<code>include/linux/mm_types.h</code>	mm 核心结构	<code>mm_struct</code> 、 <code>vm_area_struct</code> 、页表相关对象
<code>mm/mmap.c</code>	VMA 创建与查找	<code>mmap</code> 、 <code>mprotect</code> 、VMA 合并拆分
<code>mm/memory.c</code>	通用 fault 主干	<code>handle_mm_fault</code> 、匿名 fault、文件 fault、COW 页被哪些映射引用，回收和 COW 需要的关系
<code>mm/rmap.c</code>	反向映射	page cache、文件映射、读入文件页
<code>mm/filemap.c</code>	文件页 fault	
<code>arch/x86/mm/tlb.c</code>	TLB 失效	本地 flush、远端 shutdown、PCID 相关路径

这条地图的分层很重要：`arch/x86/mm/fault.c` 负责架构相关入口，`mm/memory.c` 负责大部分架构无关语义。不要把二者混成一个函数理解。x86-64 入口知道 CR2 和 error code；通用 mm 知道 VMA、folio、匿名页、文件页、COW、回收和锁。

第一步：地址形式和 VMA 查找不是同一件事

用户态传来的值先要满足 x86-64 canonical address 规则。非 canonical 地址不是“找不到 VMA”那么简单，它在页表遍历前就可以被硬件拒绝。通过这一关后，内核才用 fault address 查找当前进程的 VMA。

```
page fault classification:
address is not canonical:
    reject before treating it as a normal VMA miss
no VMA covers address:
    user instruction -> SIGSEGV
    kernel usercopy -> -EFAULT
VMA covers address but permissions reject access:
    protection failure, not demand allocation
VMA covers address and permissions allow access:
    inspect PTE and repair if possible
```

VMA 是虚拟地址区间的语义说明。它回答“这段地址是否应该存在，允许读写执行吗，来自匿名内存还是文件，私有还是共享”。PTE 是当前硬件翻译状态。一个合法 VMA 允许暂时没有 PTE；一个没有 VMA 的地址不能靠分配页修复。

第二步：error code 决定走缺失路径还是保护路径

page fault error code 的 present/protection 位把路径分成两类。若不是 present 页，常见情况是懒分配、文件页未读入或 swap 页未换入。若是 present 页上的保护 fault，常见情况是 COW、写只读映射、用户态访问 supervisor 页或取指违反 NX。

硬件现象	内核可能动作	不能误判成什么
not-present 读 匿名页	分配零页，安装 PTE，重试指令	程序错误
not-present 读 文件页	查 page cache，必要时发起 I/O	简单 malloc

present 写只读 PTE	若 VMA 允许写且 PTE 是 COW，则复制或提升	普通权限失败
present 写只读 VMA	发送信号或返回错误	COW
present 取指 NX 页	发送信号或安全违规处理	数据页缺失

这里的关键是同时看 VMA 和 PTE。VMA 允许写、PTE 不可写，可能是 COW；VMA 本身不允许写，PTE 不可写就是权限拒绝。只看 PTE 会把只读文件映射误当成 COW；只看 VMA 会看不见硬件当前状态。

匿名懒分配：合法地址为什么第一次访问才有物理页

`mmap` 或堆增长可以先建立 VMA，不立刻分配物理页。第一次读写触发 `not-present #PF`，内核分配一个清零页，安装 PTE，返回用户态重试 `faulting instruction`。用户程序感觉这条 `load/store` 成功了，但中间发生过一次异常和内核修复。

```
anonymous demand fault:
  find VMA covering fault address
  verify VMA permits the access
  allocate zero-filled physical page
  install user PTE with VMA-derived permissions
  return to the same user RIP
CPU retries the faulting load/store
```

这就是为什么 VIRT 可以很大而 RSS 很小。VMA 代表承诺，RSS 代表当前真的驻留的物理页。OS 用这个策略避免提前为大量可能永远不会访问的地址分配内存。

文件映射缺页：read 和 mmap 在 page cache 汇合

文件 `mmap` 后，用户第一次读某个页面也会触发 `not-present #PF`。内核根据 VMA 找到文件和偏移，再到 page cache 查对应页。若 page cache 命中，这是 `minor fault`；若需要从存储设备读入，这是 `major fault`。读入后安装 PTE，用户指令重试。

```
file-backed fault:
  fault address -> VMA
  VMA file + page offset -> page-cache index
  if page cache hit:
    map cached folio and return
  else:
    submit storage read, wait for completion
    mark folio uptodate
    map folio and return
```

因此 `read` 和 `mmap` 共享的不只是“文件内容”，而是内核里的 page cache 对象。前者通常把 page cache 内容复制到用户 buffer；后者把 page cache 页映射进用户地址空间。理解这一点，后面讲 page cache 回收、dirty、writeback 和文件系统日志才不会断开。

COW 的完整条件：VMA 允许写，PTE 暂时禁止写

`fork` 后父子共享匿名物理页。为了发现第一次写，父子 PTE 都要清掉 `writable`，并标记 COW。用户写入时，CPU 看到 `present` PTE 但不可写，于是产生保护型 `#PF`。内核确认 VMA 允许写、PTE 是 COW，再决定复制还是提升。

```
COW write fault:
  write to present but read-only PTE
  VMA says private mapping is writable
  PTE carries COW state
```

```

if physical page refcount > 1:
    allocate new page and copy old content
    decrement old page refcount
    install writable PTE to new page
else:
    only restore writable bit
    invalidate stale TLB translation
    return and retry the store

```

复制和提升的差别体现了 COW 的性能设计。若物理页仍被父子或其他映射共享，必须复制；若只剩当前映射，直接把 PTE 改回 writable 即可。无论哪种，都要处理 TLB，因为 CPU 可能还缓存着旧的只读或旧的可写翻译。

为什么 fork 时父进程也必须写保护

常见误解是“fork 只要把子进程页表设成只读”。这是错的。父进程如果保留 writable PTE，它可以继续写共享物理页，子进程会直接看到父进程的新内容，COW 就失效了。正确做法是父子双方都写保护，并清理父进程可能已有的 writable TLB 项。

```

fork private pages:
  for each writable private PTE:
    increment physical page refcount
    clear parent writable bit
    mark parent PTE as COW
    install child read-only COW PTE
    invalidate parent stale writable TLB entry

```

这一步把“fork 很快”变成可理解的状态机：fork 不复制所有页，只复制页表项和引用计数；第一次写才复制物理页。代价是 fork 时要批量改 PTE 权限，并承担 TLB 失效成本。

TLB shutdown: COW 的隐藏正确性条件

如果父进程在 fork 前刚写过某页，CPU TLB 可能缓存了 writable 翻译。fork 清掉父 PTE writable 后，若没有失效旧 TLB，父进程下一次写可能不触发 #PF，而是直接修改共享物理页。这不是性能问题，而是隔离正确性问题。

```

stale TLB failure:
  parent writes page, TLB caches writable PTE
  fork clears parent PTE writable but forgets TLB invalidate
  parent writes again through stale TLB
  child observes modified shared frame
  COW handler never runs

```

多核时问题更明显：同一个地址空间的线程可能在多个 CPU 上运行，远端 CPU 也可能缓存旧翻译。内核需要本地 TLB invalidate，也可能需要 IPI 触发远端 CPU 执行 shutdown。后面看到 mprotect、munmap、COW、页迁移性能问题时，要把这笔成本算进去。

用户指针 fault: 为什么有时是 -EFAULT 而不是信号

同一个坏地址，在用户指令里访问和在系统调用参数复制时访问，结果不同。用户程序自己执行 `mov rax, qword ptr [rdi]` 访问坏地址，通常应得到信号。若用户把坏指针传给 `read` 或 `write`，内核在 `copy_to_user` 或 `copy_from_user` 中 fault，则系统调用应返回 `-EFAULT`。

```

; Intel syntax, user instruction fault example
mov rax, qword ptr [rdi]

; syscall usercopy fault example, conceptual
copy_from_user(kernel_buffer, user_pointer, length)
if page fault is not repairable:

```

```
return -EFAULT
```

Linux 用异常表一类机制标记某些内核访问用户内存的指令：这里 `fault` 不代表内核坏了，而是用户参数不可访问，应跳到修复位置返回错误。没有这个区分，坏用户指针就会把内核打崩。

可观察证据：不要只看 `maps`

调试缺页路径时，`/proc/<pid>/maps` 只能告诉你 VMA，不告诉你每页是否已经有 PTE、是否驻留、是否 `dirty`。需要把多个证据合起来看：

命令或文件	能看到什么	不能单独推出什么
<code>/proc/<pid>/maps</code>	VMA 区间、权限、文件映射	物理页是否已分配
<code>/proc/<pid>/smaps</code>	RSS、dirty、shared/private 统计	每一次 <code>fault</code> 的源码分支
<code>perf stat -e page-faults</code>	page fault 总量	major/minor 和 <code>fault</code> 原因细节
<code>perf stat -e minor-faults, major-faults</code> <code>strace</code>	缺页是否涉及 I/O 系统调用返回 <code>EFAULT</code> 等错误	COW 与匿名零页的区别 用户指令自己的 <code>page fault</code>

真正的分析要能把现象落回路径。例如 VMA 存在但 RSS 没涨，说明还没实际触页；minor fault 增加但 major fault 不增，通常没有存储 I/O；fork 后父子写同一页 RSS 增加，可能是 COW 分裂；系统调用返回 `EFAULT`，说明坏用户指针被 `usercopy` 路径捕获。

配套 C++20 模型覆盖哪些失败

下面的模型不是完整 Linux，也不模拟真实指令译码。它只把第一阶段最容易讲空的断言变成测试：

- 合法匿名 VMA 第一次访问会 minor fault，分配零页后重试成功。
- 只读 VMA 的写入不能被当成 COW，必须失败且不能偷偷分配页。
- fork 后父子共享页，第一次写触发 COW，父子内容分离。
- 如果 fork 时忘记失效父进程 `writable TLB`，父进程会绕过 COW 污染子进程。
- 文件映射第一次 `fault` 通过 `page cache` 建立映射，并记录 major fault。
- `syscall usercopy` 遇到坏用户指针返回失败，而不是模拟成内核崩溃。
- 非 `canonical` 地址不能被误讲成普通 VMA miss。

第一阶段关于虚拟内存的最低验收：能解释一条用户态写指令为什么可能正常完成、minor fault 后完成、major fault 后完成、COW 后完成、返回 `-EFAULT`，或变成信号；并能指出每种结果依赖哪个 VMA/PTE/TLB 条件。

完整 C++20 模型源码

```
#include <array>
#include <cstddef>
#include <cstdint>
#include <iostream>
#include <map>
#include <optional>
#include <stdexcept>
```

```

#include <string>
#include <string_view>
#include <unordered_map>
#include <vector>

using Byte = std::uint8_t;
using Word = std::uint64_t;
using VirtualAddress = std::uint64_t;
using PageNumber = std::uint64_t;
using Pcid = std::uint16_t;
using FrameId = std::uint32_t;

constexpr std::uint64_t X86_VIRTUAL_ADDRESS_BITS = 48U;
constexpr std::uint64_t X86_CANONICAL_SIGN_BIT =
    1ULL << (X86_VIRTUAL_ADDRESS_BITS - 1U);
constexpr VirtualAddress X86_CANONICAL_LOW_MASK =
    (1ULL << X86_VIRTUAL_ADDRESS_BITS) - 1ULL;
constexpr VirtualAddress X86_CANONICAL_HIGH_MASK = ~X86_CANONICAL_LOW_MASK;
constexpr std::uint64_t X86_PAGE_SHIFT = 12U;
constexpr std::size_t X86_PAGE_SIZE = 1ULL << X86_PAGE_SHIFT;
constexpr VirtualAddress X86_PAGE_OFFSET_MASK =
    static_cast<VirtualAddress>(X86_PAGE_SIZE - 1U);
constexpr Word X86_PF_PROTECTION_BIT = 1ULL << 0U;
constexpr Word X86_PF_WRITE_BIT = 1ULL << 1U;
constexpr Word X86_PF_USER_BIT = 1ULL << 2U;
constexpr std::size_t X86_FAULT_RETRY_LIMIT = 3U;
constexpr std::size_t HASH_COMBINE_SHIFT = 1U;

constexpr Pcid TEST_PARENT_PCID = 41U;
constexpr Pcid TEST_CHILD_PCID = 42U;
constexpr Pcid TEST_FILE_PCID = 43U;
constexpr VirtualAddress TEST_ANON_PAGE = 0x0000'0000'0040'0000ULL;
constexpr VirtualAddress TEST_ANON_VA = TEST_ANON_PAGE + 0x30ULL;
constexpr VirtualAddress TEST_READ_ONLY_PAGE = 0x0000'0000'0050'0000ULL;
constexpr VirtualAddress TEST_READ_ONLY_VA = TEST_READ_ONLY_PAGE + 0x20ULL;
constexpr VirtualAddress TEST_COW_PAGE = 0x0000'0000'0060'0000ULL;
constexpr VirtualAddress TEST_COW_VA = TEST_COW_PAGE + 0x10ULL;
constexpr VirtualAddress TEST_FILE_PAGE = 0x0000'0000'0070'0000ULL;
constexpr VirtualAddress TEST_FILE_VA = TEST_FILE_PAGE + 0x80ULL;
constexpr VirtualAddress TEST_BAD_USER_VA = 0x0000'0000'0080'0000ULL;
constexpr VirtualAddress TEST_NON_CANONICAL_VA = 0x0000'8000'0000'0000ULL;
constexpr std::uint64_t TEST_FILE_PAGE_INDEX = 9U;
constexpr std::size_t TEST_FILE_BYTE_OFFSET =
    static_cast<std::size_t>(TEST_FILE_VA & X86_PAGE_OFFSET_MASK);
constexpr Byte TEST_ZERO_BYTE = 0x00U;
constexpr Byte TEST_ANON_BYTE = 0x5AU;
constexpr Byte TEST_ORIGINAL_BYTE = 0x11U;
constexpr Byte TEST_PARENT_BYTE = 0x22U;
constexpr Byte TEST_CHILD_BYTE = 0x33U;
constexpr Byte TEST_FILE_BYTE = 0x7EU;

enum class AccessKind {
    Read,
    Write,
    Execute,
};

enum class VmaKind {
    Anonymous,
    FileBacked,
};

enum class FaultContext {
    UserInstruction,
    UserCopy,
};

enum class FaultDisposition {
    Handled,
    Signal,
    BadUserPointer,
};

enum class AccessFailureKind {
    SegmentationViolation,
    BadUserPointer,
    RetryLimitExceeded,
};

struct PageTableEntry {
    FrameId frame = 0;
    bool present = false;
    bool writable = false;
    bool user = true;
    bool executable = false;
    bool cow = false;
};

struct Translation {
    FrameId frame = 0;
    std::size_t offset = 0;
};

struct Vma {
    VirtualAddress start = 0;
    VirtualAddress end = 0;
    bool readable = false;
};

```

```

    bool writable = false;
    bool executable = false;
    bool private_mapping = true;
    VmaKind kind = VmaKind::Anonymous;
    std::uint64_t file_page_index = 0;
};

struct FaultStats {
    std::size_t minor_faults = 0;
    std::size_t major_faults = 0;
    std::size_t cow_copies = 0;
    std::size_t cow_promotions = 0;
    std::size_t signals = 0;
    std::size_t bad_user_pointers = 0;
};

struct PageCacheResult {
    FrameId frame = 0;
    bool hit = false;
};

struct TlbKey {
    Pcid pcid = 0;
    PageNumber vpn = 0;

    bool operator==(const TlbKey& other) const {
        return this->pcid == other.pcid && this->vpn == other.vpn;
    }
};

struct TlbKeyHash {
    std::size_t operator()(const TlbKey& key) const {
        const std::size_t pcid_hash = std::hash<Pcid>{}(key.pcid);
        const std::size_t vpn_hash = std::hash<PageNumber>{}(key.vpn);
        return pcid_hash ^ (vpn_hash << HASH_COMBINE_SHIFT);
    }
};

bool is_canonical(VirtualAddress address) {
    const bool sign_bit_set = (address & X86_CANONICAL_SIGN_BIT) != 0;
    const VirtualAddress high_bits = address & X86_CANONICAL_HIGH_MASK;
    if (sign_bit_set) {
        return high_bits == X86_CANONICAL_HIGH_MASK;
    }
    return high_bits == 0;
}

VirtualAddress page_base(VirtualAddress address) {
    return address & ~X86_PAGE_OFFSET_MASK;
}

PageNumber virtual_page_number(VirtualAddress address) {
    return address >> X86_PAGE_SHIFT;
}

std::size_t page_offset(VirtualAddress address) {
    return static_cast<std::size_t>(address & X86_PAGE_OFFSET_MASK);
}

void require(bool condition, std::string_view message) {
    if (!condition) {
        throw std::runtime_error(std::string(message));
    }
}

class PageFault : public std::runtime_error {
public:
    PageFault(VirtualAddress fault_address, AccessKind fault_access,
              Word fault_error_code, std::string_view message)
        : std::runtime_error(std::string(message)),
          address_(fault_address),
          access_(fault_access),
          error_code_(fault_error_code) {}

    VirtualAddress address() const {
        return this->address_;
    }

    AccessKind access() const {
        return this->access_;
    }

    Word error_code() const {
        return this->error_code_;
    }

private:
    VirtualAddress address_ = 0;
    AccessKind access_ = AccessKind::Read;
    Word error_code_ = 0;
};

class AccessFailure : public std::runtime_error {
public:
    AccessFailure(AccessFailureKind failure_kind, std::string_view message)
        : std::runtime_error(std::string(message)), kind_(failure_kind) {}
};

```

```

    AccessFailureKind kind() const {
        return this->kind_;
    }

private:
    AccessFailureKind kind_ = AccessFailureKind::SegmentationViolation;
};

template <typename Callable>
void require_access_failure(AccessFailureKind expected, Callable action) {
    try {
        action();
    } catch (const AccessFailure& failure) {
        require(failure.kind() == expected, "unexpected access failure kind");
        return;
    }
    throw std::runtime_error("expected access failure was not raised");
}

class PhysicalMemory {
public:
    FrameId allocate_zero_frame() {
        const FrameId frame = static_cast<FrameId>(this->frames_.size());
        this->frames_.push_back(Frame{});
        this->refcounts_.push_back(1U);
        return frame;
    }

    FrameId allocate_copy_frame(FrameId source) {
        const FrameId frame = this->allocate_zero_frame();
        this->frames_.at(frame) = this->frames_.at(source);
        return frame;
    }

    void increment_ref(FrameId frame) {
        ++this->refcounts_.at(frame);
    }

    void decrement_ref(FrameId frame) {
        std::size_t& refcount = this->refcounts_.at(frame);
        require(refcount > 0, "physical frame refcount underflow");
        --refcount;
    }

    std::size_t refcount(FrameId frame) const {
        return this->refcounts_.at(frame);
    }

    Byte load8(FrameId frame, std::size_t offset) const {
        require(offset < X86_PAGE_SIZE, "load offset must stay inside page");
        return this->frames_.at(frame).bytes.at(offset);
    }

    void store8(FrameId frame, std::size_t offset, Byte value) {
        require(offset < X86_PAGE_SIZE, "store offset must stay inside page");
        this->frames_.at(frame).bytes.at(offset) = value;
    }

    std::size_t frame_count() const {
        return this->frames_.size();
    }

private:
    struct Frame {
        std::array<Byte, X86_PAGE_SIZE> bytes{};
    };

    std::vector<Frame> frames_;
    std::vector<std::size_t> refcounts_;
};

class PageCache {
public:
    PageCacheResult get_or_read_page(PhysicalMemory& memory,
                                     std::uint64_t file_page_index) {
        const auto iterator = this->pages_.find(file_page_index);
        if (iterator != this->pages_.end()) {
            return PageCacheResult{.frame = iterator->second, .hit = true};
        }

        const FrameId frame = memory.allocate_zero_frame();
        memory.store8(frame, TEST_FILE_BYTE_OFFSET, TEST_FILE_BYTE);
        this->pages_[file_page_index] = frame;
        return PageCacheResult{.frame = frame, .hit = false};
    }

    std::size_t size() const {
        return this->pages_.size();
    }

private:
    std::map<std::uint64_t, FrameId> pages_;
};

class Tlb {
public:
    std::optional<PageTableEntry> lookup(Pcid pcid, PageNumber vpn) const {

```



```

    const auto iterator = this->entries_.find(TlbKey{.pcid = pcid, .vpn = vpn});
    if (iterator == this->entries_.end()) {
        return std::nullopt;
    }
    return iterator->second;
}

void insert(Pcid pcid, PageNumber vpn, const PageTableEntry& entry) {
    this->entries_[TlbKey{.pcid = pcid, .vpn = vpn}] = entry;
}

void invalidate(Pcid pcid, PageNumber vpn) {
    this->entries_.erase(TlbKey{.pcid = pcid, .vpn = vpn});
}

std::size_t size() const {
    return this->entries_.size();
}

private:
    std::unordered_map<TlbKey, PageTableEntry, TlbKeyHash> entries_;
};

class AddressSpace {
public:
    explicit AddressSpace(Pcid pcid) : pcid_(pcid) {}

    Pcid pcid() const {
        return this->pcid_;
    }

    void add_vma(const Vma& vma) {
        require(is_canonical(vma.start), "VMA start must be canonical");
        require(is_canonical(vma.end - 1U), "VMA end must be canonical");
        require(vma.start < vma.end, "VMA must not be empty");
        this->vmas_.push_back(vma);
    }

    const Vma* find_vma(VirtualAddress address) const {
        for (const Vma& vma : this->vmas_) {
            if (address >= vma.start && address < vma.end) {
                return &vma;
            }
        }
        return nullptr;
    }

    void map_page(VirtualAddress virtual_page, const PageTableEntry& entry) {
        require((virtual_page & X86_PAGE_OFFSET_MASK) == 0,
            "mapped virtual address must be page aligned");
        this->entries_[virtual_page_number(virtual_page)] = entry;
    }

    const PageTableEntry* find_pte(PageNumber vpn) const {
        const auto iterator = this->entries_.find(vpn);
        if (iterator == this->entries_.end()) {
            return nullptr;
        }
        return &iterator->second;
    }

    PageTableEntry* find_pte_for_mutation(PageNumber vpn) {
        const auto iterator = this->entries_.find(vpn);
        if (iterator == this->entries_.end()) {
            return nullptr;
        }
        return &iterator->second;
    }

    const std::vector<Vma>& vmass() const {
        return this->vmas_;
    }

    const std::map<PageNumber, PageTableEntry>& entries() const {
        return this->entries_;
    }

    std::map<PageNumber, PageTableEntry>& entries_for_mutation() {
        return this->entries_;
    }

    std::size_t mapped_page_count() const {
        return this->entries_.size();
    }

private:
    Pcid pcid_ = 0;
    std::vector<Vma> vmass_;
    std::map<PageNumber, PageTableEntry> entries_;
};

class Mmu {
public:
    Translation translate(const AddressSpace& address_space, Tlb& tlb,
        VirtualAddress virtual_address, AccessKind access) const {
        if (!is_canonical(virtual_address)) {
            throw PageFault(

```

```

        virtual_address, access, this->error_code(access, false),
        "x86-64 rejected non-canonical virtual address before VMA lookup");
    }

    const PageNumber vpn = virtual_page_number(virtual_address);
    const std::optional<PageTableEntry> cached = tlb.lookup(address_space.pcid(), vpn);
    if (cached.has_value()) {
        this->check_entry(virtual_address, access, cached.value());
        return Translation{.frame = cached->frame, .offset = page_offset(virtual_address)};
    }

    const PageTableEntry* walked = address_space.find_pte(vpn);
    if (walked == nullptr || !walked->present) {
        throw PageFault(virtual_address, access, this->error_code(access, false),
            "page walk found no present PTE");
    }
    this->check_entry(virtual_address, access, *walked);
    tlb.insert(address_space.pcid(), vpn, *walked);
    return Translation{.frame = walked->frame, .offset = page_offset(virtual_address)};
}

private:
void check_entry(VirtualAddress address, AccessKind access,
    const PageTableEntry& entry) const {
    if (!entry.user) {
        throw PageFault(address, access, this->error_code(access, true),
            "ring 3 access reached supervisor PTE");
    }
    if (access == AccessKind::Write && !entry.writable) {
        throw PageFault(address, access, this->error_code(access, true),
            "write access rejected by PTE");
    }
    if (access == AccessKind::Execute && !entry.executable) {
        throw PageFault(address, access, this->error_code(access, true),
            "execute access rejected by PTE");
    }
}

Word error_code(AccessKind access, bool protection) const {
    Word code = X86_PF_USER_BIT;
    if (protection) {
        code |= X86_PF_PROTECTION_BIT;
    }
    if (access == AccessKind::Write) {
        code |= X86_PF_WRITE_BIT;
    }
    return code;
}
};

class KernelMemoryManager {
public:
    KernelMemoryManager(PhysicalMemory& memory, PageCache& page_cache, Tlb& tlb)
        : memory_(memory), page_cache_(page_cache), tlb_(tlb) {}

    const FaultStats& stats() const {
        return this->stats_;
    }

    FaultDisposition handle_page_fault(AddressSpace& address_space,
        const PageFault& fault,
        FaultContext context) {
        const Vma* vma = address_space.find_vma(fault.address());
        if (vma == nullptr || !this->vma_allows(*vma, fault.access())) {
            return this->reject_fault(context);
        }

        const PageNumber vpn = virtual_page_number(fault.address());
        PageTableEntry* pte = address_space.find_pte_for_mutation(vpn);
        const bool protection_fault =
            (fault.error_code() & X86_PF_PROTECTION_BIT) != 0;
        if (!protection_fault) {
            return this->handle_missing_pte(address_space, *vma, vpn);
        }
        if (fault.access() == AccessKind::Write && pte != nullptr && pte->cow) {
            return this->handle_cow_fault(address_space, vpn, *pte);
        }
        return this->reject_fault(context);
    }

    void fork_private(AddressSpace& parent, AddressSpace& child,
        bool invalidate_parent_tlb) {
        for (const Vma& vma : parent.vmas()) {
            child.add_vma(vma);
        }

        for (auto& [vpn, entry] : parent.entries_for_mutation()) {
            PageTableEntry child_entry = entry;
            const VirtualAddress virtual_page = vpn << X86_PAGE_SHIFT;
            const Vma* vma = parent.find_vma(virtual_page);
            if (entry.present && vma != nullptr) {
                this->memory_.increment_ref(entry.frame);
                if (vma->private_mapping && vma->writable) {
                    entry.writable = false;
                    entry.cow = true;
                    child_entry.writable = false;
                    child_entry.cow = true;
                }
            }
        }
    }
};

```

```

        if (invalidate_parent_tlb) {
            this->tlb_.invalidate(parent.pcid(), vpn);
        }
    }
}
child.map_page(virtual_page, child_entry);
}
}

private:
bool vma_allows(const Vma& vma, AccessKind access) const {
    if (access == AccessKind::Read) {
        return vma.readable;
    }
    if (access == AccessKind::Write) {
        return vma.writable;
    }
    return vma.executable;
}

FaultDisposition reject_fault(FaultContext context) {
    if (context == FaultContext::UserCopy) {
        ++this->stats_.bad_user_pointers;
        return FaultDisposition::BadUserPointer;
    }
    ++this->stats_.signals;
    return FaultDisposition::Signal;
}

FaultDisposition handle_missing_pte(AddressSpace& address_space, const Vma& vma,
                                   PageNumber vpn) {
    if (vma.kind == VmaKind::Anonymous) {
        const FrameId frame = this->memory_.allocate_zero_frame();
        address_space.map_page(
            vpn << X86_PAGE_SHIFT,
            PageTableEntry{
                .frame = frame,
                .present = true,
                .writable = vma.writable,
                .user = true,
                .executable = vma.executable,
                .cow = false,
            });
        ++this->stats_.minor_faults;
        return FaultDisposition::Handled;
    }

    PageCacheResult result =
        this->page_cache_.get_or_read_page(this->memory_, vma.file_page_index);
    this->memory_.increment_ref(result.frame);
    address_space.map_page(
        vpn << X86_PAGE_SHIFT,
        PageTableEntry{
            .frame = result.frame,
            .present = true,
            .writable = vma.writable && !vma.private_mapping,
            .user = true,
            .executable = vma.executable,
            .cow = false,
        });
    if (result.hit) {
        ++this->stats_.minor_faults;
    } else {
        ++this->stats_.major_faults;
    }
    return FaultDisposition::Handled;
}

FaultDisposition handle_cow_fault(AddressSpace& address_space, PageNumber vpn,
                                 PageTableEntry& pte) {
    if (this->memory_.refcount(pte.frame) == 1U) {
        pte.writable = true;
        pte.cow = false;
        this->tlb_.invalidate(address_space.pcid(), vpn);
        ++this->stats_.cow_promotions;
        return FaultDisposition::Handled;
    }

    const FrameId old_frame = pte.frame;
    const FrameId new_frame = this->memory_.allocate_copy_frame(old_frame);
    this->memory_.decrement_ref(old_frame);
    pte.frame = new_frame;
    pte.writable = true;
    pte.cow = false;
    this->tlb_.invalidate(address_space.pcid(), vpn);
    ++this->stats_.cow_copies;
    return FaultDisposition::Handled;
}

PhysicalMemory& memory_;
PageCache& page_cache_;
Tlb& tlb_;
FaultStats stats_;
};

class UserCpu {
public:

```

```

UserCpu(const Mmu& mmu, PhysicalMemory& memory, Tlb& tlb,
        KernelMemoryManager& kernel)
    : mmu_(mmu), memory_(memory), tlb_(tlb), kernel_(kernel) {}

Byte load8(AddressSpace& address_space, VirtualAddress virtual_address) {
    for (std::size_t attempt = 0; attempt < X86_FAULT_RETRY_LIMIT; ++attempt) {
        try {
            const Translation translation =
                this->mmu_.translate(address_space, this->tlb_, virtual_address,
                                     AccessKind::Read);
            return this->memory_.load8(translation.frame, translation.offset);
        } catch (const PageFault& fault) {
            this->handle_or_throw(address_space, fault, FaultContext::UserInstruction);
        }
    }
    throw AccessFailure(AccessFailureKind::RetryLimitExceeded,
                        "load exceeded page fault retry limit");
}

void store8(AddressSpace& address_space, VirtualAddress virtual_address,
            Byte value) {
    for (std::size_t attempt = 0; attempt < X86_FAULT_RETRY_LIMIT; ++attempt) {
        try {
            const Translation translation =
                this->mmu_.translate(address_space, this->tlb_, virtual_address,
                                     AccessKind::Write);
            this->memory_.store8(translation.frame, translation.offset, value);
            return;
        } catch (const PageFault& fault) {
            this->handle_or_throw(address_space, fault, FaultContext::UserInstruction);
        }
    }
    throw AccessFailure(AccessFailureKind::RetryLimitExceeded,
                        "store exceeded page fault retry limit");
}

bool copy_from_user(AddressSpace& address_space, VirtualAddress virtual_address,
                   Byte& output) {
    for (std::size_t attempt = 0; attempt < X86_FAULT_RETRY_LIMIT; ++attempt) {
        try {
            const Translation translation =
                this->mmu_.translate(address_space, this->tlb_, virtual_address,
                                     AccessKind::Read);
            output = this->memory_.load8(translation.frame, translation.offset);
            return true;
        } catch (const PageFault& fault) {
            const FaultDisposition disposition =
                this->kernel_.handle_page_fault(address_space, fault,
                                                FaultContext::UserCopy);
            if (disposition == FaultDisposition::Handled) {
                continue;
            }
            return false;
        }
    }
    return false;
}

private:
void handle_or_throw(AddressSpace& address_space, const PageFault& fault,
                    FaultContext context) {
    const FaultDisposition disposition =
        this->kernel_.handle_page_fault(address_space, fault, context);
    if (disposition == FaultDisposition::Signal) {
        throw AccessFailure(AccessFailureKind::SegmentationViolation,
                            "page fault became a user signal");
    }
    if (disposition == FaultDisposition::BadUserPointer) {
        throw AccessFailure(AccessFailureKind::BadUserPointer,
                            "page fault became a bad user pointer");
    }
}

const Mmu& mmu_;
PhysicalMemory& memory_;
Tlb& tlb_;
KernelMemoryManager& kernel_;
};

Vma anonymous_vma(VirtualAddress page, bool writable) {
    return Vma{
        .start = page,
        .end = page + X86_PAGE_SIZE,
        .readable = true,
        .writable = writable,
        .executable = false,
        .private_mapping = true,
        .kind = VmaKind::Anonymous,
        .file_page_index = 0,
    };
}

Vma file_vma(VirtualAddress page) {
    return Vma{
        .start = page,
        .end = page + X86_PAGE_SIZE,
        .readable = true,
    };
}

```

```

        .writable = false,
        .executable = false,
        .private_mapping = true,
        .kind = VmaKind::FileBacked,
        .file_page_index = TEST_FILE_PAGE_INDEX,
    };
}

struct TestMachine {
    PhysicalMemory memory;
    PageCache page_cache;
    Tlb tlb;
    Mmu mmu;
    KernelMemoryManager kernel;
    UserCpu cpu;

    TestMachine()
        : memory(),
          page_cache(),
          tlb(),
          mmu(),
          kernel(memory, page_cache, tlb),
          cpu(mmu, memory, tlb, kernel) {}
};

void test_anonymous_fault_allocates_and_retries() {
    TestMachine machine;
    AddressSpace process(TEST_PARENT_PCID);
    process.add_vma(anonymous_vma(TEST_ANON_PAGE, true));

    require(process.mapped_page_count() == 0U,
        "anonymous VMA should not allocate a PTE before access");
    require(machine.cpu.load8(process, TEST_ANON_VA) == TEST_ZERO_BYTE,
        "anonymous demand fault should return a zero-filled byte");
    machine.cpu.store8(process, TEST_ANON_VA, TEST_ANON_BYTE);
    require(machine.cpu.load8(process, TEST_ANON_VA) == TEST_ANON_BYTE,
        "anonymous page did not preserve user store");
    require(machine.kernel.stats().minor_faults == 1U,
        "anonymous demand paging should be a minor fault in this model");
}

void test_vma_permission_rejects_write() {
    TestMachine machine;
    AddressSpace process(TEST_PARENT_PCID);
    process.add_vma(anonymous_vma(TEST_READ_ONLY_PAGE, false));

    require_access_failure(AccessFailureKind::SegmentationViolation, [&machine,
                                                                    &process] {
        machine.cpu.store8(process, TEST_READ_ONLY_VA, TEST_ANON_BYTE);
    });
    require(machine.kernel.stats().signals == 1U,
        "write to read-only VMA should become a user-visible failure");
    require(process.mapped_page_count() == 0U,
        "permission failure must not allocate a page");
}

void test_cow_split_preserves_parent_child_isolation() {
    TestMachine machine;
    AddressSpace parent(TEST_PARENT_PCID);
    parent.add_vma(anonymous_vma(TEST_COW_PAGE, true));

    machine.cpu.store8(parent, TEST_COW_VA, TEST_ORIGINAL_BYTE);
    const FrameId shared_frame =
        parent.find_pte(virtual_page_number(TEST_COW_VA))>frame;
    AddressSpace child(TEST_CHILD_PCID);
    machine.kernel.fork_private(parent, child, true);

    require(machine.memory.refcount(shared_frame) == 2U,
        "fork should share the original frame before either process writes");
    machine.cpu.store8(parent, TEST_COW_VA, TEST_PARENT_BYTE);
    require(machine.cpu.load8(parent, TEST_COW_VA) == TEST_PARENT_BYTE,
        "parent COW write did not install private data");
    require(machine.cpu.load8(child, TEST_COW_VA) == TEST_ORIGINAL_BYTE,
        "child should still see the pre-COW byte");
    require(machine.kernel.stats().cow_copies == 1U,
        "shared COW page should require one copy");

    machine.cpu.store8(child, TEST_COW_VA, TEST_CHILD_BYTE);
    require(machine.cpu.load8(child, TEST_COW_VA) == TEST_CHILD_BYTE,
        "child COW promotion did not make the child page writable");
    require(machine.kernel.stats().cow_promotions == 1U,
        "last private COW reference should be promoted without copying");
}

void test_missing_tlb_shootdown_breaks_cow() {
    TestMachine machine;
    AddressSpace parent(TEST_PARENT_PCID);
    parent.add_vma(anonymous_vma(TEST_COW_PAGE, true));

    machine.cpu.store8(parent, TEST_COW_VA, TEST_ORIGINAL_BYTE);
    AddressSpace child(TEST_CHILD_PCID);
    machine.kernel.fork_private(parent, child, false);

    machine.cpu.store8(parent, TEST_COW_VA, TEST_PARENT_BYTE);
    require(machine.cpu.load8(child, TEST_COW_VA) == TEST_PARENT_BYTE,
        "without shootdown, stale writable TLB should corrupt the shared page");
    require(machine.kernel.stats().cow_copies == 0U,

```

```

    "stale TLB write should bypass the COW handler entirely");
}

void test_file_fault_and_bad_user_pointer() {
    TestMachine machine;
    AddressSpace process(TEST_FILE_PCID);
    process.add_vma(file_vma(TEST_FILE_PAGE));

    require(machine.cpu.load8(process, TEST_FILE_VA) == TEST_FILE_BYTE,
        "file-backed fault did not load page-cache content");
    require(machine.kernel.stats().major_faults == 1U,
        "first file-backed page should model a major fault");
    require(machine.page_cache.size() == 1U,
        "file fault should populate exactly one page-cache entry");

    Byte copied = TEST_ZERO_BYTE;
    require(!machine.cpu.copy_from_user(process, TEST_BAD_USER_VA, copied),
        "bad user pointer should be reported to the syscall layer");
    require(machine.kernel.stats().bad_user_pointers == 1U,
        "bad user pointer statistic was not updated");
}

void test_non_canonical_address_is_not_a_vma_miss() {
    TestMachine machine;
    AddressSpace process(TEST_PARENT_PCID);
    process.add_vma(anonymous_vma(TEST_ANON_PAGE, true));

    require(!is_canonical(TEST_NON_CANONICAL_VA),
        "test address must be non-canonical");
    require_access_failure(AccessFailureKind::SegmentationViolation, [&machine,
                                                                    &process] {
        static_cast<void*>(machine.cpu.load8(process, TEST_NON_CANONICAL_VA));
    });
}

int main() {
    test_anonymous_fault_allocates_and_retries();
    test_vma_permission_rejects_write();
    test_cow_split_preserves_parent_child_isolation();
    test_missing_tlb_shutdown_breaks_cow();
    test_file_fault_and_bad_user_pointer();
    test_non_canonical_address_is_not_a_vma_miss();
    std::cout << "x86-64 page fault and COW model checks passed\n";
}

```

1.3 总线、设备、DMA 与中断平台

内存、总线、设备和 DMA：从芯片连线到 OS I/O

CPU 再强，也必须和内存、设备、时钟、中断控制器一起工作。操作系统里最难的 bug，常常出在这些边界：地址翻译和 TLB 不一致，MMIO 顺序错，DMA buffer 生命周期错，中断确认顺序错，IOMMU 映射漏撤销。本节把这些路径拆开讲。

内存芯片的最小读写协议

最朴素的 SRAM 可以抽象为地址输入、数据输入、数据输出、读写控制和片选信号。CPU 或内存控制器给出地址，选择读或写，等待信号稳定。DRAM 更复杂，需要行列地址、刷新、bank、burst，但对 OS 的第一层抽象仍是：某个物理地址可以读写某些字节。

```

SRAM read:
    chip_select = 1
    write_enable = 0
    address = A
    wait access_time
    data = data_out

SRAM write:
    chip_select = 1
    write_enable = 1
    address = A
    data_in = V
    wait write_time

```

物理地址不是自然存在的数字，而是系统设计给内存控制器和设备译码的编号。哪些地址连到 DRAM，哪些地址连到固件 ROM，哪些地址连到 PCIe MMIO 或 APIC 寄

存器，由芯片组、PCIe root complex、固件表和硬件设计共同决定。OS 通过内存地图知道哪些物理页可用，哪些必须保留。

五种地址：名字相似，边界完全不同

I/O 章节里最容易混的是地址。用户指针、内核指针、物理地址、MMIO 地址、DMA 地址不是同一种东西换个名字。它们由不同主体使用，走不同翻译路径，错误也由不同机制报告。

地址类型	谁使用	如何翻译或解释	常见错误
用户虚拟地址	Ring 3 程序和内核拷贝路径	当前进程页表、VMA/PTE 权限	直接交给设备，设备看不懂
内核虚拟地址	内核代码	内核页表映射到物理页或 MMIO	当成用户指针检查，或泄漏给用户
物理地址	内存控制器和平台地址译码	地址范围决定 DRAM、APIC、PCIe MMIO 等目标	误以为用户态可直接访问
MMIO 地址	驱动经 <code>ioremap</code> 后访问	物理地址窗口连到设备寄存器	当普通 RAM cache/合并/重排
DMA/IOVA	设备发出的地址	IOMMU domain 翻译到物理页	unmap 后设备迟到写、方向标错

一次用户 `read(fd, buf, len)` 可能同时出现这五种地址：`buf` 是用户虚拟地址；内核临时缓冲区是内核虚拟地址；页框有物理地址；设备寄存器在 MMIO BAR 中；描述符里放的是 DMA 地址或 IOVA。讲清地址类型，I/O 路径的一半混乱就消失了。

地址空间地图：同一个 load 可以读内存也可以读设备

CPU 执行 load/store 时只发出地址和访问类型。地址译码决定这个请求去 DRAM、ROM、PCIe MMIO 窗口，还是本地 APIC 寄存器。于是同一条 store 指令，写 DRAM 是保存数据，写设备寄存器可能是启动 DMA，写中断控制器可能是发送 IPI。

地址区域	背后硬件	软件语义
RAM	内存控制器和 DRAM	普通可缓存内存、页分配来源
ROM/Flash	固件或只读存储	启动代码、固件表、只读镜像
MMIO window interrupt controller framebuffer	设备寄存器或队列门铃 local APIC、IOAPIC、 MSI/MSI-X 显存或显示控制器窗口	驱动控制设备状态 mask、priority、EOI、IPI 图形输出或 DMA buffer

这就是为什么内核页表要给不同区域设置不同属性。普通 RAM 可以 cache；MMIO 通常要设为 device/uncacheable/strongly ordered 类属性；framebuffer 可能使用 write-combining。属性错了，轻则性能差，重则设备协议被 CPU 合并或重排破坏。

DRAM 为什么慢，cache 为什么必要

CPU 周期很短，DRAM 访问要经过内存控制器、行激活、列访问、总线传输和排队。若每条 load 都等 DRAM，CPU 大部分时间会空转。cache 利用局部性：刚访问过的数据很可能很快再访问，相邻数据也可能很快访问。

局部性	例子	cache 如何利用
时间局部性	循环反复访问同一变量	保留最近 cache line
空间局部性	顺序遍历数组	一次搬入整条 cache line
指令局部性	顺序执行函数体	I-cache 预取连续指令
共享局部性	多线程读同一只读表	多核 cache 可共享只读 line

cache line 是最小搬运单位。一个 8 字节变量被修改，硬件可能搬运和独占整个 64 字节 line。OS 数据结构若把多个 CPU 频繁写的变量放在同一 line，就会产生伪共享。许多内核结构会显式对齐到 cache line，就是为了避免这种硬件级共享。

cache 写策略和脏数据

写穿透 cache 每次写都同步写到下层，简单但慢。写回 cache 先只改 cache line，把 line 标为 dirty，稍后再写回下层。现代 CPU 多用写回策略。于是“CPU 执行了 store”不等于“DRAM 立刻有新值”，更不等于“设备通过 DMA 已经能看到新值”。

```
CPU store to cached memory:
line = load line into cache if absent
modify bytes in cache line
mark line dirty
write back later on eviction or flush
```

对普通内存，硬件一致性协议让其他 CPU 最终能看到正确结果。对非一致 DMA 设备，驱动必须在设备读内存前 clean cache，把 dirty 数据推到设备可见层；在设备写内存后 invalidate cache，避免 CPU 继续读旧 line。是否需要这些操作取决于平台 DMA 一致性模型，但驱动必须按 DMA API 写，不能凭感觉。

TLB：页表太慢后的缓存

多级页表翻译一次虚拟地址可能要读多次内存。如果每条 load/store 都完整 page walk，开销巨大。TLB 缓存虚拟页到物理页和权限。命中时，地址翻译接近几个周期；未命中时，硬件 page walker 或内核参与填充。

```
translate(va):
    vpn = va >> PAGE_SHIFT
    entry = tlb.lookup(vpn, asid)
    if entry.hit and entry.permissions allow access:
        return entry.ppn | page_offset(va)
    else:
        pte = page_table_walk(vpn)
        if invalid: raise page_fault
        tlb.insert(vpn, pte)
```

TLB 让虚拟内存可用，也让页表修改变复杂。修改 PTE 后，旧翻译可能仍在本地 CPU 或其他 CPU 的 TLB 中。内核必须在释放物理页、收紧权限或取消映射前完成必要失效。多核下这通常需要 IPI，也就是 TLB shootdown。

页大小和大页：为什么不是越大越好

常见基础页大小是 4KiB，也有 2MiB、1GiB 这类大页。大页减少页表层级和 TLB 压力，适合大连续内存区域；小页减少内部碎片，权限粒度更细，适合普通进程堆栈和文件映射。

页大小	优点	代价
小页	细粒度权限、少内部碎片、回收灵活	页表多、TLB 覆盖范围小

大页	TLB 覆盖范围大、page walk 少	内部碎片、拆分成本、权限粗
透明大页	用户无感提高部分负载性能	折叠/拆分带来延迟尖峰

OS 内存管理的很多复杂性来自这些折中。数据库、虚拟机和大模型推理可能受益于大页；延迟敏感系统可能不希望后台折叠大页造成抖动。页不是抽象单位而已，它直接影响 TLB、cache、回收和 I/O。

总线事务：读、写、posted write 和错误

总线或互联把请求从发起者送到目标。读事务通常需要请求和响应；写事务可能是 posted write，即发起者把写请求发出后不等待设备真正处理完成。posted write 提高吞吐，但让“写了寄存器”与“设备已经执行命令”之间产生距离。

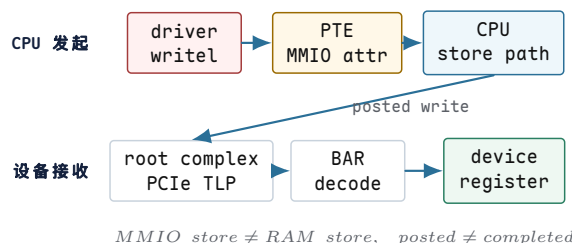
```
MMIO posted write:
CPU writes DOORBELL
interconnect queues write
CPU continues
device receives write later

MMIO read after write:
CPU writes DOORBELL
CPU reads STATUS
read often forces prior posted writes to reach device ordering point
```

驱动有时会在关键 MMIO 写后读回某个寄存器，以确保写到达设备或满足排序要求。具体规则依平台和 API 而定，但核心思想是：设备不是 CPU 内部函数，MMIO 访问经过互联、缓冲和设备状态机。

MMIO 访问的完整路径

一次 `writel(tail, doorbell)` 从内核代码到设备，至少跨过页表属性、CPU store 路径、uncacheable/write-combining 规则、root complex、PCIe transaction layer、设备 BAR 译码和设备内部寄存器。它不是普通内存写。



图示：MMIO 写从内核 API 到设备寄存器的路径。页属性和访问 API 共同约束排序、合并、宽度和可见性。

可写成两个顺序不变量：

normal memory descriptor writes $\prec_{dma}$ MMIO doorbell

MMIO command write $\prec_{device}$ status/completion observed

第一条由 `dma_wmb` 和正确的 doorbell 写建立；第二条通常要靠设备协议、读回、completion 或中断确认建立。若只看到 CPU 执行完一条 store，就宣布设备完成，是把 CPU 本地顺序误当成设备全局顺序。

PCIe 设备是怎样被发现的

PCIe 设备有配置空间，里面包含 vendor ID、device ID、class code、BAR、MSI/MSI-X capability 等信息。固件或内核枚举总线，给 BAR 分配地址窗口，给设备分配 bus/device/function 编号。驱动通过 ID 表匹配设备。

```
pci enumeration:
  for each bus/device/function:
    read vendor_id
    if present:
      read class/device ids
      size BARs and allocate MMIO ranges
      discover MSI/MSI-X capability
      create pci_dev object
      bind matching driver
```

BAR 是 Base Address Register。设备通过 BAR 告诉系统“我需要一段 MMIO 地址空间”。内核分配物理地址范围后，驱动再 ioremap 到内核虚拟地址。用户进程不能直接访问这个 MMIO，除非内核通过受控接口映射并检查权限。

PCIe 的上下层契约

PCIe 往下是链路、lane、TLP、重试和错误报告；往上是内核看到的 pci_dev、BAR 资源、MSI-X vector、DMA mask 和 AER 状态。驱动不直接管理电信号，但必须尊重这些上层对象。

PCIe 层面	硬件事实	OS 对象或策略
link/lane	速率、宽度、训练、降速	性能上限、链路错误、热插拔恢复
configuration space	vendor/device/class/capability	设备发现、驱动匹配、能力启用
BAR	设备 MMIO 窗口需求	resource 分配、ioremap、权限控制
MSI-X table	多个中断 vector	队列到 CPU 亲和性、NAPI/blk-mq 并行
bus mastering	设备能发起 DMA	启用前必须准备 DMA mask/IOMMU/domain
AER	可纠正/不可纠正错误	错误日志、reset、设备下线

因此驱动 probe 的真实顺序不是“发现设备就开始收发”。它要先确认资源、启用设备、配置 DMA、分配中断、初始化队列，最后才允许上层提交请求：

```
pci_driver_probe(dev):
  enable PCI device and MMIO resources
  set DMA mask and attach IOMMU domain
  ioremap BAR registers
  allocate submission/completion rings
  request MSI-X vectors and set affinity
  initialize hardware registers
  publish block/net/char device to upper layer
```

BAR、寄存器、doorbell 和 descriptor 不是一回事

PCIe 设备常被一句“通过 BAR 做 MMIO”带过，但里面有几层对象。BAR 是设备申请的地址窗口；寄存器是窗口内有定义的偏移；doorbell 是某些寄存器的用途，写它表示“有新工作”；descriptor 是内存中的请求记录，通常不在 BAR 里，而在普通 RAM 中，设备通过 DMA 读取。

概念	位于哪里	作用
BAR	PCIe 配置空间中的资源描述	告诉 OS 设备需要多大的 MMIO 地址窗口

MMIO window	物理地址空间中的一段范围	CPU load/store 会被路由到设备
register	MMIO window 内的具体偏移	控制设备、读取状态、配置队列
doorbell	一类寄存器用途	通知设备队列 tail 或新命令已发布
descriptor	普通 RAM 或设备可 DMA 的内存	描述 buffer 地址、长度、命令、标志
completion entry	普通 RAM 中的完成记录或设备队列	设备写回状态、tag、字节数、错误码

错误顺序通常发生在 descriptor 和 doorbell 之间。CPU 必须先把 descriptor 写完整，并用 DMA 写屏障保证设备可见，然后再写 doorbell。若先敲 doorbell，设备可能立刻 DMA 读取半初始化 descriptor；若 completion 读完前就释放 buffer，设备或上层可能仍在使旧对象。

描述符环：CPU 和设备共享队列

高性能设备不希望 CPU 每个请求都写一堆寄存器。更常见的做法是：内存存在 RAM 中建立 descriptor ring，CPU 作为生产者填写描述符，设备作为消费者读取描述符；完成后设备写回 status 或 completion ring，再中断或让 CPU 轮询。

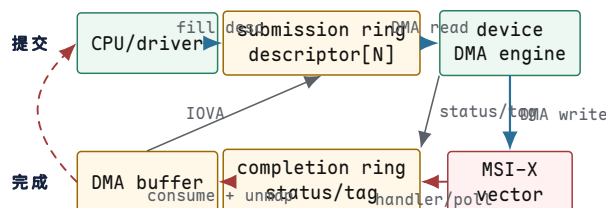
```

TX ring:
desc[0] -> buffer A, len, flags
desc[1] -> buffer B, len, flags
...
driver_tail: next slot driver may fill
device_head: next slot device may consume

submit:
fill desc[tail]
dma_wmb()
tail = tail + 1
writel(tail, DOORBELL)

```

环形队列有几个不变量：不能覆盖设备尚未消费的描述符；不能回收设备尚未完成的 buffer；发布 tail 前描述符必须完整可见；处理 completion 后必须 unmap DMA 并释放上层资源。网络、NVMe、virtio、io_uring 都可以用这个模型理解。



图示：队列型设备的通用结构：descriptor 在普通 RAM，doorbell 在 MMIO，数据通过 DMA，完成通过 completion ring 和 MSI-X 返回。

安全释放不变量可以直接写成：

$$free(buffer) \Leftarrow completion(tag) \wedge dma_unmap(buffer) \wedge owner(buffer) = CPU$$

取消请求也必须维护这个不变量。用户关闭 fd 或超时返回，不代表设备已经不再 DMA。只有驱动确认 request 不会再完成，或者已经把迟到 completion 收敛掉，buffer 才能释放。

descriptor ring 的所有权状态机

描述符环不是数组循环使用这么简单。每个 slot 都在 CPU 和设备之间转移所有权。只有拥有者可以改 slot 内容；不拥有的一方只能等待状态变化或读取协议允许的字段。

```
Free:
  CPU owns slot, no in-flight request

Prepared:
  CPU filled descriptor but has not published tail

Submitted:
  CPU published tail and rang doorbell
  device may read descriptor by DMA
  CPU must not rewrite descriptor or free buffer

Completed:
  device wrote completion entry
  CPU may consume status, unmap DMA, wake waiter

Free again:
  CPU has released request and buffer ownership
```

状态	谁拥有	不能做什么
Free	CPU	设备不能读取无效 descriptor
Prepared	CPU	不能让设备看到半初始化字段
Submitted	设备/硬件路径	CPU 不能复用 buffer 或覆盖 slot
Completed	CPU 正在回收	不能重复完成同一请求
Resetting	驱动收敛故障	不能接受新提交，必须处理 in-flight

这个状态机解释了为什么驱动要有 request tag、队列深度和超时处理。completion 到来时，驱动通过 tag 找回 request；超时时，驱动必须判断请求是否还可能完成；reset 时，驱动要把所有 Submitted 状态收敛到失败、重试或已取消。

DMA 地址不等于用户虚拟地址

用户程序传给 read/write 的是用户虚拟地址。CPU 访问它要通过当前进程页表。设备不能直接理解这个地址。驱动要么复制数据到内核 buffer，要么 pin 用户页，拿到物理页，再通过 DMA API 得到设备可用的 DMA 地址。若有 IOMMU，DMA 地址通常是设备虚拟地址 IOVA。

```
user pointer
-> validate VMA and permissions
-> pin user pages or copy into kernel pages
-> physical pages
-> dma_map creates DMA address / IOVA
-> descriptor contains DMA address
-> device DMA reads/writes memory
```

这条路径很容易出 bug。用户 buffer 长度可能溢出；页可能跨多个物理页，需要 scatter-gather list；设备 DMA mask 可能只能访问低 32 位地址；IOMMU 映射可能失败；用户取消请求时，in-flight DMA 不能简单释放。

pin、copy、map 三个动作的区别

用户 buffer 进入 I/O 路径时，内核有三种常见处理方式。copy 是把用户数据

复制到内核 buffer；pin 是暂时阻止用户页被回收或迁移；map 是把物理页授权给设备 DMA。三者经常连续出现，但语义不同。

动作	改变了什么	没有改变什么
copy	产生一份内核拥有的数据副本	用户原页仍可变化，设备不一定可见
pin	增加页的长期或短期固定约束	设备仍不能自动访问该页
dma map	建立设备可用的 DMA 地址和方向权限	不代表请求已经完成或 buffer 可释放
unmap	撤销设备访问授权并做必要同步	不自动释放用户页或 request 对象
unpin	解除页固定，允许回收/迁移	不说明设备是否成功处理数据

因此“零拷贝”不是零管理。它少了一次 CPU 复制，却增加了 pin 生命周期、IOMMU 映射、scatter-gather、取消和 completion 回收。高性能 I/O 的难点不在少写几行拷贝代码，而在把这些生命周期全部收敛正确。

一次块设备 read 中地址如何换手

把 `read(fd, buf, len)` 放到 NVMe 这类块设备路径里，可以看到五种地址如何逐步换手。用户 API 只有 `buf`，但硬件完成一次 I/O 需要用户虚拟地址、内核虚拟地址、物理页、DMA/IOVA 和 MMIO doorbell 共同参与。

```
read(fd, user_buf, len):
    kernel validates user virtual range
    page cache chooses or allocates kernel pages
    block layer builds request from page fragments
    DMA API maps physical pages to device IOVA
    driver writes descriptor with IOVA and length
    dma_wmb()
    driver writes MMIO doorbell
    device DMA writes data into mapped pages
    device writes completion entry
    interrupt handler completes request
    kernel copies bytes to user_buf if needed
```

阶段	地址或对象	错误后果
用户参数检查	用户虚拟地址和长度	越界、溢出、坏指针导致错误访问
page cache/bio	内核页和物理页片段	页生命周期不清导致 use-after-free
DMA map	IOVA、方向、长度、设备 domain	IOMMU fault 或设备读写旧数据
descriptor doorbell	RAM 中的队列项 MMIO 寄存器偏移	设备读到半初始化命令 顺序错会让设备提前消费队列
completion	completion queue entry 和 request tag	找不回请求，重复完成或永久等待

这里最容易被误解的是“设备完成”。设备 completion 只说明设备按协议处理了某个 descriptor，并把结果写回 completion queue；它不自动说明用户 buffer 已经可见，也不自动说明文件系统状态已经持久化。内核还要撤销 DMA 映射、处理

错误码、更新 page cache 或 bio 状态、唤醒等待者，并把结果翻译成系统调用返回值。

scatter-gather：物理内存不连续怎么办

用户看到的虚拟 buffer 可以连续，但背后的物理页通常不连续。大 I/O 不应该强迫内核分配一大段连续物理内存。scatter-gather 描述符让一个请求由多个片段组成，每个片段有 DMA 地址和长度。

```
sglist for 10KB user buffer:
segment0: dma=0x1000a000 len=4096
segment1: dma=0x20345000 len=4096
segment2: dma=0x0ff12000 len=2048
```

块层 bio、网络 skb fragment、DMA engine 都会遇到类似结构。算法上要处理片段合并、最大段数、对齐、设备边界和 IOMMU 映射成本。所谓“零拷贝”不是没有成本，而是把复制成本换成页 pin、映射、片段管理和完成回收成本。

IOMMU 和设备隔离

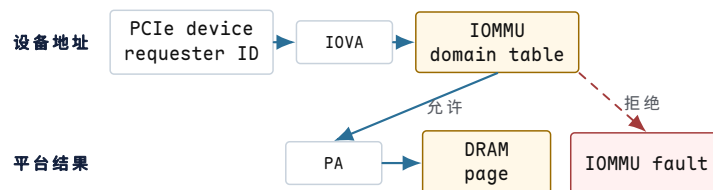
IOMMU 给设备访问内存也加了一层翻译。设备发出 DMA 地址，IOMMU 查设备所属 domain 的页表，得到物理地址并检查权限。这样内核可以只把某个 buffer 映射给设备，completion 后撤销映射。

```
device DMA:
device emits IOVA
IOMMU checks device domain
if mapping exists and permission ok:
    physical = translate_iova(IOVA)
    perform memory access
else:
    report IOMMU fault
```

IOMMU 还支撑虚拟化设备直通。把一个 PCIe 设备交给虚拟机时，hypervisor 不能让它 DMA 到宿主任意内存；IOMMU domain 必须限制在该虚拟机拥有的页。IOMMU 不是性能功能，而是设备侧隔离的硬件根。

IOMMU 的上下游：设备看 IOVA，内存控制器看 PA

IOMMU 位于设备 DMA 请求和内存系统之间。设备 descriptor 里放的是 DMA address/IOVA；IOMMU 根据 requester id 找到 domain，再查 IOMMU 页表，把 IOVA 翻译成物理地址。内存控制器最后只看到物理地址请求。



图示：IOMMU 把设备发出的 IOVA 翻译成物理地址，并按设备 domain 检查权限。没有映射或方向错误会形成 IOMMU fault。

映射生命周期可以写成：

```
dma_map(page, direction):
    pin or otherwise own page for I/O lifetime
    create IOVA -> PA mapping in device domain
    perform cache maintenance if platform requires it
    return IOVA

dma_unmap(IOVA, direction):
    ensure device no longer owns the buffer
    synchronize cache if platform requires it
```

```
remove IOVA mapping
allow page ownership to return upward
```

方向也有语义。DMA_TO_DEVICE 表示设备读内存，CPU 写入 buffer 后要让设备看到新数据；DMA_FROM_DEVICE 表示设备写内存，CPU 读取前要避免旧 cache line；DMA_BIDIRECTIONAL 成本更高，不能为了省心乱用。

中断和 DMA completion 的完整路径

现在串起一次 NVMe 读取或网卡接收的典型路径：

1. kernel allocates/pins pages for data
2. kernel maps pages for DMA and builds sglist
3. driver fills descriptors in submission ring
4. dma_wmb publishes descriptors to device
5. driver writes doorbell MMIO
6. device reads descriptors via DMA
7. device transfers data to/from memory
8. device writes completion entry
9. device raises MSI-X interrupt
10. interrupt handler schedules poll or completion work
11. driver reads completion ring
12. driver unmaps DMA and releases/publishes buffer
13. waiting task or upper stack is woken

每一步都有状态和失败。DMA map 可能失败；设备可能超时；中断可能被合并；completion 可能表示错误；用户进程可能已经取消；reset 可能正在进行。驱动代码之所以长，是因为它必须覆盖这些状态，而不是因为工程师喜欢复杂。

设备 reset：异步系统的故障收敛

设备可能挂起、固件崩溃、PCIe 链路错误、DMA fault 或长期无 completion。驱动 reset 不是简单写一个 reset bit，它要停止新请求，冻结队列，等待或取消 in-flight，通知上层失败，重建硬件状态，再恢复服务。

```
reset_device(dev):
    mark dev as resetting
    stop accepting new submissions
    quiesce queues
    abort or wait in-flight requests
    mask interrupts
    issue hardware reset
    reinitialize registers and rings
    re-enable interrupts
    fail or retry old requests by policy
    mark dev as running
```

如果 reset 时还有旧 DMA 写入已释放 buffer，就会造成内存破坏；如果 reset 后忘记重建 IOMMU 映射，设备会 DMA fault；如果上层不知道请求失败，用户程序会永久等待。I/O 子系统的可靠性，核心就是让异步失败最终收敛到可解释状态。

实践观察：用命令把硬件路径看见

在真实 Linux 系统上，可以用下面命令观察硬件和 OS 的连接点。这里不是要求背命令，而是把“硬件描述、驱动绑定、中断、IOMMU、内存属性”变成可验证对象。

```
lspci -nn
lspci -vv -s <device>
cat /proc/interrupts
find /sys/bus/pci/devices -maxdepth 2 -type l
dmesg | grep -i -E 'iommu|dma|msi|nvme|eth'
cat /proc/iomem
```

```
cat /proc/meminfo
```

观察报告至少要回答：设备如何被发现，BAR 映射在哪里，使用哪个 IRQ/MSI-X vector，驱动名称是什么，是否启用 IOMMU，DMA mask 是多少，错误统计在哪里看。没有这些证据，谈驱动和 DMA 只是概念。

从上层系统调用到底层硬件的 I/O 契约

把一次 read 或 send 贯穿到底，完整链条不是“应用调用内核，内核调用设备”。更准确的是：

```
syscall → VFS/socket/block/net object → request/skb/bio
→ DMA mapping → descriptor → doorbell
→ device state machine → completion → wake up
```

每一层只向上承诺自己能承诺的事实。系统调用返回值承诺用户可见语义；块层 completion 承诺块请求完成；设备 completion 承诺硬件处理了 descriptor；DMA unmap 承诺设备访问授权撤销；文件系统 fsync 才承诺持久化语义。把这些承诺混在一起，就会写出“completion 到了就可以向用户承诺落盘”的 bug。

本节测试：画出数据所有权

- 能区分用户虚拟地址、内核虚拟地址、物理地址、DMA 地址和 IOVA。
- 能说明为什么 MMIO 访问需要专用 API 和顺序约束。
- 能画出 descriptor ring 的 head/tail、owned by CPU/device、completion 状态。
- 能说明 DMA buffer 在提交、设备访问、completion、unmap 各阶段归谁所有。
- 能解释 IOMMU 如何限制设备 DMA 范围，以及映射失败或 fault 如何处理。
- 能写出 $descriptor\ writes \prec doorbell$ 和 $completion \wedge dma_unmap \Rightarrow buffer$ 可释放这两个不变量。
- 能说明 PCIe BAR、MSI-X、DMA mask、AER 分别在驱动 probe 中解决什么问题。
- 能解释中断合并、poll budget 和 completion queue 如何影响延迟与吞吐。
- 能为设备 reset 写出停止新请求、处理 in-flight、重建硬件状态的顺序。

硬件平台机制深讲：不止 cache

前面已经讲了 cache、TLB、DMA、MMIO 和 descriptor ring，但一台机器不只靠这些对象运行。很多操作系统 bug 来自更外围的硬件事实：寄存器位有清除语义，中断有 mask 和 in-service 状态，定时器有 deadline 和漂移，USB 有主机控制器调度，网卡有 RX refill，SSD 有 FTL 和 volatile write cache，显示有 scanout deadline，音频有周期中断，固件表决定资源边界，TPM/IOMMU/BMC 决定安全边界。

本节把这些机制按“硬件状态机 → OS 对象 → 不变量 → 故障证据”补齐。读者遇到任何新硬件，都应该先问四件事：

$$\begin{aligned}
state &= \{registers, queues, buffers, clocks, power, errors\}, \\
event &= interrupt \vee poll \vee timeout \vee fault, \\
ownership &= CPU \vee device \vee firmware \vee user, \\
commit &= visible \text{ to caller } \wedge recoverable \text{ after failure.}
\end{aligned}$$

硬件接口的三种形态

OS 看到硬件，通常不是直接看到电路，而是看到三类接口：寄存器、共享内存队列和异步事件。寄存器用于控制和状态；共享内存队列用于批量提交工作；异步事件用于告诉 CPU “某个状态变了”。三者组合起来，才是一台现代设备的完整接口。

接口	硬件含义	OS 必须维护的事实
control reg	配置模式、启动/停止、mask、reset	写入顺序、位语义、访问宽度和电源状态
status reg	busy、ready、error、cause、pending	读取是否有副作用，清除是否需要特定写法
shared ring	CPU 和设备共享 descriptor/completion	head/tail、所有权、屏障、DMA 映射生命周期
IRQ/event	设备、定时器或 CPU 间消息	mask、ack、EOI、亲和性、丢失和合并
firmware	描述硬件拓扑和资源	内存地图、IRQ、NUMA、ACPI 方法、保留区

许多错误来自把一种接口当成另一种接口。MMIO status 不是普通变量，因为读它可能清除 pending 位；descriptor 不是寄存器，因为设备通过 DMA 异步读取它；中断不是函数调用，因为它可以被 mask、合并、重定向，也可能在状态尚未处理完时再次到来。

寄存器位语义：读写不是普通赋值

设备寄存器的每一位都可能具有专门语义。普通内存里 $x = 1$ 表示写入值 1；MMIO 寄存器里写 1 可能表示置位、清除、触发、门铃、解锁或无效。读寄存器也可能改变设备状态。

位语义	例子	驱动错误
RW	普通可读写配置位	read-modify-write 未加锁，覆盖别的位
W1C	write one to clear 错误/中断原因	写回整寄存器时误清其他 pending 位
R0 sticky	错误发生后保持到 reset 或清除	只看瞬时状态，漏掉历史故障
RC	read to clear	调试打印读一次就把事件清掉
W0 trigger	写入触发动作，如 doorbell/reset	重复写导致重复提交或误 reset
self clearing	写 1 后硬件完成时自动清 0	轮询必须有超时，不能无限等

正确驱动不会用“把结构体映射到寄存器地址然后随便赋值”的方式控制设备。它要知道每个寄存器的访问宽度、对齐、端序、副作用和顺序要求。尤其是 W1C 位，常见错误是：

```

wrong:
    v = readl(STATUS)
    v &= ~ERROR_BIT
    writel(v, STATUS)    // may clear unrelated W1C bits

right:
    writel(ERROR_BIT, STATUS_W1C)

```

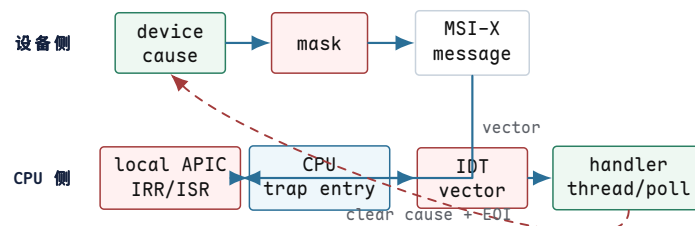
寄存器访问的不变量可以写成：

$$\begin{aligned}
 \text{valid_mmio_write} \Leftarrow & \text{device} = \text{powered} \wedge \text{clock} = \text{enabled} \\
 & \wedge \text{reset} = \text{deasserted} \wedge \text{width} = \text{specified} \\
 & \wedge \text{side_effect} = \text{understood}.
 \end{aligned}$$

这条式子解释了为什么驱动 probe、resume、reset path 都要重复配置寄存器。设备断电或 reset 后，寄存器不是“仍然保存上次值”的 C++ 对象，而是回到硬件定义的初始状态。

中断路径：从设备 cause 到内核 handler

中断不是“设备调用内核函数”。更准确地说，设备先把内部 cause 位置位；若没有被 mask，就通过引脚、MSI 或 MSI-X 把事件投递到中断控制器；APIC 记录 pending；CPU 在可中断边界接受 vector；硬件切到内核入口；内核保存现场、查 IDT、运行 handler；最后确认设备状态和 APIC in-service 状态。



图示：中断的完整路径。设备 cause、MSI-X、APIC IRR/ISR、IDT 和 handler 各有独立状态，不能只用“中断来了”概括。

中断确认顺序必须和设备协议匹配。若设备采用 level-triggered 或“状态未清就会再次触发”的模型，handler 需要先处理或清除设备 cause，再向 APIC 发送 EOI：

$$\text{clear}(\text{device_cause}) \prec \text{EOI}(\text{APIC})$$

如果先 EOI，APIC 可能立即接受同一个仍然 pending 的事件，造成中断风暴；如果只 EOI 不清设备 cause，CPU 会反复进入 handler；如果只清设备 cause 不读 completion ring，可能丢掉真实完成。正确 handler 一般是状态驱动的：

```

irq_handler(vector):
    reason = read_device_status()
    if reason has completions:
        disable_or_mask_device_irq_if_polling()
        schedule_poll()
    if reason has errors:
        record error and start recovery
    clear_device_interrupt_cause(reason)
    send_EOI_to_local_APIC()

```

APIC、IPI 和多核控制

x86-64 多核系统里，每个 CPU 有 local APIC。设备中断可以通过 IOAPIC 或 MSI/MSI-X 路由到某个 CPU；CPU 也可以向其他 CPU 发送 IPI。IPI 不是普通消息队列，它会打断目标 CPU 的执行，用于 TLB shutdown、reschedule、stop-machine、多核启动等强控制路径。

APIC 相关状态	作用	OS 后果
IRR	已到达但尚未服务的 vector	中断可能排队，不是实时函数调用
ISR	正在服务的 vector	EOI 过早或过晚都会影响再次投递
TPR/priority	屏蔽低优先级中断	过度屏蔽会增加延迟
MSI-X affinity	队列 vector 绑定到 CPU	网络/块设备队列和 NUMA 局部性
IPI target	目标 CPU 集合	TLB shutdown 和 reschedule 成本随 CPU 数增加

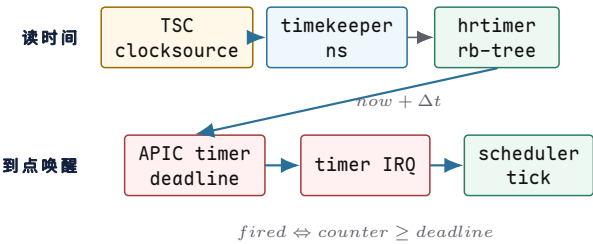
TLB shutdown 可以写成 IPI 协议：

```
sender : clear PTE, build cpu_mask, send IPI,
receiver : enter vector, invalidate TLB, ack,
sender : wait( $\forall \text{cpu} \in \text{mask} : \text{ack}$ ), then free page.
```

这说明 shutdown 的成本不是一条 `invlpg`，而是跨 CPU 中断、远端停顿、确认等待和 TLB 热状态损失。进程越大、CPU 越多、页表修改越频繁，这条路径越容易成为瓶颈。

定时器：调度不是靠 while 循环抢时间

抢占式 OS 必须有硬件定时器。定时器的本质是比较“当前计数”和“目标 deadline”，到期后产生中断。x86-64 上常见时间相关硬件包括 TSC、local APIC timer、HPET、PIT、RTC 和设备自己的硬件 timestamp。OS 通常把它们分成两类：clocksource 提供“现在几点”，clockevent 提供“到点叫醒我”。



图示：时间系统分成“读当前时间”和“安排下一次中断”。调度 tick、超时、睡眠、网络重传和 I/O timeout 都依赖这条链。

系统里至少有三种时间语义：

时间	语义	常见用途
monotonic time	单调递增，不受手动改时影响	超时、调度、性能测量
realtime wall clock	人类日期时间，可被校准或修改	文件时间戳、日志、证书有效期
cycle counter	CPU 或平台计数器	精细性能测量、clocksource 原料

定时器 bug 常表现为“任务睡不醒、超时过早、时钟跳变、CPU 空转、延迟尖峰”。调试时要分清：是 `clocksource` 不稳定，`clockevent` 没设置成功，定时器中断被屏蔽，还是 `handler` 运行太久。调度器的时间片不是抽象数字，它最终要落到硬件 `deadline`：

$$\text{preempt}(\text{current}) \Leftarrow \text{runtime}(\text{current}) \geq \text{slice}(\text{current}) \wedge \text{timer_irq delivered}.$$

轮询、中断合并和背压

高频 I/O 不能每个包或每个 `completion` 都触发一次中断。网卡、NVMe 和高速 USB 控制器会使用中断合并、批量 `completion` 和轮询。中断降低空闲延迟，轮询降低高负载开销，背压防止上层无限制提交。

机制	解决什么	代价
每事件中断 <code>interrupt moderation</code>	空闲时低延迟 合并多个完成再中断	高负载下中断风暴 增加单个请求尾延迟
<code>poll budget</code>	一次 <code>poll</code> 处理有限数量事件	防止长期占用 CPU，但可能留下积压
<code>queue depth</code>	限制 <code>in-flight</code> 请求数	过小浪费设备并行，过大增加排队延迟
<code>backpressure</code>	告诉上层暂缓提交	需要把拥塞传播到 <code>socket</code> 、 <code>bio</code> 或应用

用公式看更清楚：

$$\text{latency} = \text{queueing} + \text{service} + \text{interrupt_delay} + \text{software_dispatch}.$$

吞吐优化经常增加 `queueing` 或 `interrupt_delay`，低延迟优化经常降低队列深度和合并窗口。OS 不是追求单一指标，而是根据负载选择中断、轮询、批处理和背压的平衡点。

USB：设备不是直接把字符交给进程

USB 是主机控制模型。键盘、鼠标、摄像头、U 盘和声卡都不是主动“调用进程”；主机控制器按调度读取或传输 `endpoint`，DMA 到内存，完成 URB，再由类驱动转换成 `input`、`block`、`audio` 或 `video` 对象。

```

USB keyboard input:
xHCI schedules interrupt endpoint transfer
device returns HID report
host controller DMA writes transfer buffer
xHCI writes transfer event
MSI-X interrupt wakes USB core
HID driver parses key code
input subsystem publishes key event
tty or compositor consumes event
    
```

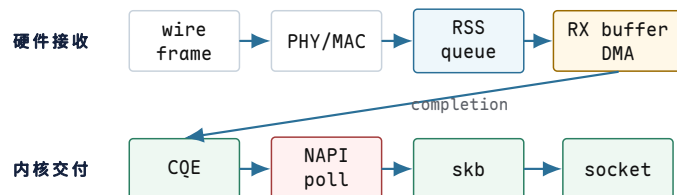
USB 插拔还引入枚举状态机：

$$\text{attached} \rightarrow \text{powered} \rightarrow \text{reset} \rightarrow \text{addressed} \rightarrow \text{configured} \rightarrow \text{driver bound}.$$

如果设备在 `configured` 后突然断开，所有 `in-flight` URB 都要失败并回收；如果用户进程还持有设备 `fd`，后续 `read/write` 要得到明确错误；如果断开时 DMA `buffer` 未撤销，驱动会留下内存生命周期 bug。热插拔能力越强，驱动状态机越不能只写初始化路径。

网卡收包：硬件队列、NAPI 和协议栈

网卡收包路径把多个硬件细节连接在一起：PHY/MAC 接收比特流，硬件校验帧，选择 RSS 队列，DMA 写入 RX buffer，写 completion，触发 MSI-X；内核用 NAPI poll 批量处理 packet，构造 skb，进入协议栈，最后唤醒 socket。



图示：网卡收包不是“中断后 copy 到 socket”。它先是 DMA 和 completion，再由 NAPI、skb 和协议栈逐层提升语义。

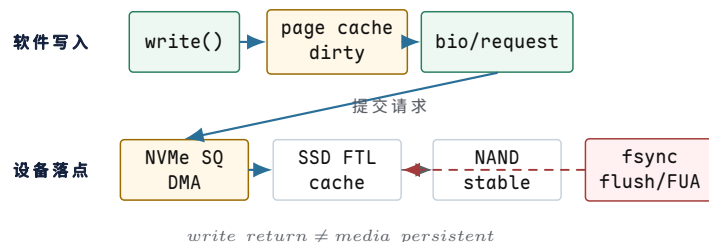
这条路径里有几个关键不变量：

$$\begin{aligned}
 RX\ buffer\ reusable &\Leftarrow packet\ consumed \wedge DMA\ unmapped\ or\ resynced, \\
 queue\ progress &\Leftarrow refill\ buffers \wedge update\ tail, \\
 socket\ delivery &\Leftarrow protocol\ accepts\ packet \wedge receive\ queue\ has\ space.
 \end{aligned}$$

若 RX buffer refill 不及时，网卡会丢包；若 IRQ affinity 和应用 CPU 不匹配，cache line 在 CPU 间迁移；若 GRO 合并太激进，吞吐提高但尾延迟可能增加；若 socket receive queue 满了，背压或丢包会向上暴露。网络性能不是协议栈单独决定的，也由 DMA buffer、队列、亲和性和 cache line ownership 决定。

NVMe 和 SSD: completion 不等于持久化

NVMe 控制器用 submission queue 和 completion queue 工作。SSD 内部还有 NAND page、erase block、FTL 映射表、垃圾回收、磨损均衡、DRAM/SRAM 缓存和掉电保护。块 I/O completion 表示控制器处理了命令；文件系统的持久化还要看 flush、FUA、日志提交和设备是否真的把 volatile cache 推到稳定介质。



图示：文件写入经历 page cache、块层、NVMe 队列、SSD 控制器和 NAND。只有具备 flush/FUA 与文件系统提交语义时，才能向应用承诺崩溃后存在。

持久化顺序可以写成：

$$data\ blocks \prec journal\ commit \prec directory\ entry\ durable$$

实际应用常用模式是：

```

write temp file data
fsync(temp file)
rename temp file to final name
fsync(parent directory)

```

这不是“保守写法”，而是在多层缓存和文件系统元数据之间建立 crash-consistency。SSD 有写缓存、FTL 重映射和垃圾回收；文件系统有 page cache、日志和目录项；应用如果只相信 write 返回，就把内存可见性误当成掉电持久性。

显示和 GPU：像素也有 deadline

显示系统不是把 framebuffer 写完就结束。显示控制器按刷新率持续 scanout，逐行从 framebuffer 或显存读像素，经 plane、scaler、composer、encoder 输出到 HDMI/DP/eDP 面板。GPU 渲染和显示 scanout 是两个异步时间线，需要 fence 和 vblank 同步。

```
frame presentation:
application renders commands
GPU writes render target
fence signals rendering complete
compositor chooses buffers
atomic modeset updates plane at vblank
display engine scans out next frame
```

显示不变量：

$scanout(buffer) \Rightarrow render_fence(buffer) = signaled \wedge buffer \text{ not freed until } scanout \text{ done.}$

如果违反它，用户看到的不是内核崩溃，而是撕裂、花屏、上一帧残影或随机闪烁。GPU hang recovery 也和块设备 reset 类似：停止提交，判断哪些 fence 永远不会 signal，重置硬件，把错误传播给进程或图形栈。

音频：环形缓冲区和实时约束

音频设备通常用 DMA 环形缓冲区。CPU 周期性填充播放 buffer，设备按采样率消耗；录音方向相反，设备写 buffer，CPU 消耗。这里的核心是 deadline：播放方向如果 CPU 填得太慢，就 underflow；录音方向如果 CPU 读得太慢，就 overflow。

$$available_playback_time = \frac{frames_queued}{sample_rate}.$$

如果队列里只剩 240 帧，采样率 48kHz，那么只剩 5ms。调度延迟、关中断时间、page fault、I/O 抖动都可能导致爆音。音频系统因此关心 pinned buffer、period interrupt、实时调度、功耗状态退出延迟和时钟同步。它不是“普通文件写入声卡”。

固件表：内核不是凭空知道硬件

x86-64 内核启动后，需要从 UEFI、ACPI、SMBIOS、MP table、PCIe 配置空间等来源获得硬件信息。固件告诉内核：哪些物理内存可用，哪些保留；CPU 和 APIC 如何编号；NUMA 距离如何；PCIe root complex 在哪里；设备中断和电源方法是什么。

固件/表	提供什么	错误后果
UEFI memory map	RAM、保留区、ACPI、MMIO	页分配器覆盖保留区或漏用内存
ACPI MADT	APIC、CPU、中断控制器	CPU 启动失败、中断路由错误
ACPI SRAT/SLIT	NUMA node 和距离	页放置、调度亲和性错误
ACPI DSDT/SSDT	设备资源和电源方法	设备 probe、suspend/resume 失败
SMBIOS	主板、DIMM、固件版本	运维诊断、ECC 错误定位

固件是 OS 的输入，不是绝对真理。内核要处理固件 bug、保留区重叠、资源冲突、错误 ACPI 方法和设备 quirks。很多“驱动为什么有一堆特殊判断”的答案，是某些机器固件给出的硬件描述不完全符合规范。

TPM、Secure Boot、IOMMU 和 BMC：安全边界也是硬件

安全不是只靠内核权限检查。Secure Boot 让固件验证下一级启动组件；TPM 用 PCR 记录测量值并可封存密钥；IOMMU 限制设备 DMA；CPU 提供页表、NX、SMEP/SMAP、用户/内核特权；服务器 BMC 则是 OS 之外的独立管理计算机。

硬件机制	保护对象	不能解决什么
Secure Boot	启动链组件完整性	已启动内核里的逻辑漏洞
TPM	测量启动状态、保护密钥释放条件	不判断软件是否无 bug
PCR/sealing	设备 DMA 地址范围	CPU 内核代码越权访问
IOMMU	CPU 执行和访问权限	数据泄露侧信道和逻辑授权错误
SMEP/SMAP/NX		
BMC	远程管理、传感器、复位控制	本身也是高权限攻击面

一条安全不变量可以写成：

secret release $\Leftarrow$ PCRs = expected $\wedge$ boot chain verified $\wedge$ runtime policy allows.

这解释了为什么磁盘加密、远程证明、内核 lockdown、设备直通和云主机隔离都要看硬件根。没有 IOMMU，恶意或失控设备可能 DMA 到任意内存；没有启动测量，密钥可能在未知内核上被释放；BMC 若被攻破，攻击者可能在 OS 之外重启、刷固件或读传感器。

硬件错误：超时、AER、MCE、NMI 和 watchdog

硬件失败不只是一句“设备坏了”。OS 要把不同错误归类为可重试、可隔离、可降级或必须 panic。PCIe AER 报告链路和事务错误；MCE 报告 CPU/内存机器检查；NMI 可用于不可屏蔽诊断；watchdog 负责发现系统长期不响应；thermal/throttling 表示硬件为了保护自己降低频率或关机。

错误源	典型证据	OS 策略
MMIO timeout	轮询 ready 超时、读回无效	reset 设备、下线驱动、返回 I/O error
PCIe AER	corrected/non-fatal/fatal 计数和日志	记录、链路恢复、函数级 reset
IOMMU fault	requester id、IOVA、权限错误	停止设备、定位坏 descriptor 或越权 DMA
MCE/ECC	物理地址、bank、syndrome	page offline、杀进程、panic
watchdog	CPU 长时间不调度或不响应 IPI	dump stacks、panic、重启
thermal throttle	频率下降、温度事件	降频、迁移负载、风扇策略

错误处理也有提交边界。比如块设备写失败时，文件系统必须知道哪些 bio 失败，哪些 page 仍 dirty，哪些事务不能提交；网卡 reset 时，协议栈必须知道哪些 skb 已经发送、哪些要重试或丢弃；内存不可纠正错误如果落在匿名页，可杀进程，若落在内核关键结构，可能只能 panic。

把硬件讲清楚的检查表

以后书中出现任何硬件概念，都可以用这张表自检。如果某一行答不上来，说明概念还没讲清楚。

问题	必须能回答
状态在哪里	寄存器、队列、buffer、PTE、TLB、APIC、firmware table 还是设备内部 SRAM

谁拥有它	CPU、设备、固件、用户进程、内核子系统还是远端 CPU
怎么推进	时钟边沿、MMIO 写、DMA、doorbell、中断、轮询、timeout 还是 reset
如何观察	status register、completion、tracepoint、perf、dmesg、AER/MCE/IOMMU 日志
如何同步	lock、屏障、DMA map/unmap、EOI、flush、fence、fsync、IPI
失败怎么办	重试、回滚、杀进程、下线页、reset 设备、降级、panic 或报告给用户

这张表比背术语重要。cache 只是其中一类状态；硬件平台还有时钟、复位、电源、固件、中断、设备队列、存储介质、安全根和错误通道。OS 之所以复杂，是因为它必须把所有这些异步状态收敛成用户能理解的文件、进程、socket、时间、权限和错误码。

- 能解释寄存器 W1C、read-to-clear、doorbell 和 status sticky 位的区别。
- 能画出 device cause、MSI-X、APIC、IDT、handler、EOI 的中断路径。
- 能区分 clocksource 与 clockevent，并说明 timer tick 怎样导致抢占。
- 能说明轮询、中断合并、队列深度和背压如何共同影响延迟与吞吐。
- 能把 USB 键盘输入、网卡收包、NVMe 写入、显示 scanout、音频 DMA 各自画成硬件到 OS 对象的路径。
- 能解释为什么 write 返回、块设备 completion、fsync 和掉电持久化不是同一个承诺。
- 能说明 UEFI/ACPI/SMBIOS 给内核提供哪些硬件事实，以及固件错误会造成什么后果。
- 能说明 TPM、Secure Boot、IOMMU、SMEP/SMAP/NX 和 BMC 各自保护或暴露的边界。
- 能根据 AER、IOMMU fault、MCE、watchdog 和 thermal 证据判断错误大致来自哪层硬件。

整机硬件图谱：一台计算机不只有 CPU

只讲 CPU、DMA、MMU 仍然不够。真实计算机是一组硬件子系统的组合：CPU、内存、固件、主板互联、存储、网络、显示、USB、音频、传感器、电源、散热、安全芯片、管理控制器。操作系统要把这些异构硬件纳入统一生命周期：发现、初始化、命名、授权、驱动、错误处理、休眠唤醒、热插拔和观测。

整机从哪里开始：x86-64 主板和平台

本书主线采用 x86-64 桌面和服务平台。常见主板形态包括：CPU 插槽、内存槽、PCIe 插槽、芯片组、固件 Flash、网卡、存储接口、USB 控制器、风扇、电源管理、TPM 和 BMC。OS 首先要把这些部件从固件表和总线配置空间中识别出来，再建立内核对象和驱动生命周期。

平台形态	典型硬件组织	OS 重点
PC/服务器	CPU + DRAM + PCIe + 芯片组 + UEFI/ACPI	枚举、NUMA、PCIe、热插拔、电源状态

x86-64 虚拟机	虚拟 CPU、虚拟设备、virtio、emulated firmware	hypervisor 边界、虚拟中断、虚拟 IOMMU
服务器管理面	BMC、IPMI/Redfish、传感器、风扇	带外管理、故障日志、电源控制

OS 首先要回答“这台机器到底有什么”。x86-64 平台主要从 ACPI、PCI 配置空间和 UEFI 内存地图获得信息；虚拟机从虚拟固件、虚拟 PCI/virtio 总线和 hypervisor 暴露的 paravirtual 接口获得信息。

固件和启动芯片

固件存放在 Flash 或 ROM 中，负责上电后的最早期初始化。x86-64 PC/服务器上常见 UEFI；服务器还可能有 BMC；安全启动、TPM 测量和 UEFI 变量共同构成启动可信链的一部分。

固件部件	作用	OS 关系
UEFI/BIOS	平台初始化和启动服务	提供内存地图、ACPI、启动协议
BMC	服务器带外管理控制器	远程电源、传感器、风扇、故障日志
TPM/安全芯片	测量启动、密钥保护	secure boot、磁盘解锁、远程证明

固件不是 OS 的替代品。固件把机器带到可启动状态，OS 接管后要建立自己的内存管理、调度、驱动和安全策略。固件表也可能有 bug，所以内核必须校验地址、长度、重叠和设备描述。

DRAM、ECC 和内存控制器

DRAM 保存运行时数据，内存控制器负责把 CPU 的内存请求转换成 DRAM 命令。服务器常使用 ECC 内存，可以检测并纠正单 bit 错误，报告不可纠正错误。NUMA 系统中，每个 CPU socket 可能有本地内存控制器。

机制	解决的问题	OS 可见现象
ECC	检测/纠正内存位翻转	EDAC/MCE 日志、页下线
NUMA	多控制器和远近不同	节点、CPU affinity、内存策略
memory hotplug	运行时增减内存	online/offline page blocks
row hammer 缓解	DRAM 行干扰风险	刷新策略、隔离、错误报告

内存不是“永远正确的数组”。硬件错误会发生，OS 要记录、隔离和尽量恢复。服务器内核可能把频繁出错的物理页下线，避免再次分配。

PCIe：现代高速外设的主干

PCIe 连接 NVMe SSD、网卡、GPU、采集卡、USB 控制器等设备。它是点对点高速互联，有 root complex、switch、endpoint。设备通过配置空间被枚举，通过 BAR 暴露 MMIO，通过 MSI/MSI-X 投递中断，通过 DMA 访问内存。

```
PCIe device lifecycle:
  enumerate config space
  allocate BAR address ranges
  enable bus mastering
  configure DMA mask and IOMMU domain
  allocate MSI-X vectors
  bind driver and run probe
```

PCIe 错误也要处理。链路可能降速，设备可能 surprise removal，AER 会报告可纠正/不可纠正错误。驱动不能假设设备永远在线。

存储设备：HDD、SATA SSD、NVMe、eMMC/UFS

存储设备差异很大。机械硬盘有寻道和旋转延迟；SATA SSD 使用 AHCI/SATA 协议；NVMe 面向 PCIe，多队列、低延迟、高并行；手机常见 eMMC/UFS。OS 不直接暴露这些差异给普通应用，而是通过块层、文件系统和 page cache 抽象。

设备	特点	OS 设计点
HDD	寻道慢，顺序吞吐好	I/O 调度、合并、预读
SATA SSD	无机械寻道，队列较有限	TRIM、flush、写放大
NVMe	PCIe、多队列、MSI-X	blk-mq、CPU/queue 亲和、低延迟
eMMC/UFS	移动设备常见	电源管理、寿命、队列深度

write 成功、设备 completion、设备缓存 flush、数据到稳定介质是不同阶段。文件系统的日志和 fsync 就是为了约束这些阶段的顺序。

网络设备和 PHY

网卡不是只把字节交给 CPU。它包含 MAC、PHY、DMA 队列、checksum offload、TSO/GSO、RSS、多队列、中断合并和硬件过滤。PHY 负责物理层信号，MAC 处理链路层帧，驱动管理 RX/TX ring。

网卡能力	作用	OS 影响
RX/TX ring	批量收发描述符	NAPI、队列背压、DMA 生命周期
RSS	按流分散到多个队列	多核并行和缓存局部性
checksum offload	硬件算校验和	skb 标志和协议栈路径
TSO/GSO	大包分段卸载	降低 CPU 开销，影响抓包理解
interrupt moderation	合并中断	吞吐和延迟折中

网络性能问题常常不是 TCP 算法一个因素，还涉及 IRQ 亲和性、队列数量、NUMA、驱动 NAPI budget、socket buffer 和用户进程调度。

GPU、显示控制器和 framebuffer

显示输出不是 printf。最简单模型是 framebuffer：内存里一块区域表示屏幕像素，显示控制器按刷新率读取这块内存并输出到屏幕。现代 GPU 复杂得多，有命令队列、显存、shader、DMA、显示管线、同步对象和电源管理。

```
simple framebuffer:
    pixel_address = fb_base + (y * stride + x) * bytes_per_pixel
    write color to pixel_address
    display controller scans out framebuffer
```

OS 图形栈要管理显存、buffer sharing、page flip、vblank、GPU 命令提交、进程隔离和 hang recovery。即使本书不展开完整图形系统，也必须知道显示设备同样是“内存 + 队列 + 中断 + DMA + 权限”的组合。

USB、输入设备和 HID

USB 是主机控制的总线。键盘、鼠标、摄像头、存储设备、声卡都可能挂在 USB 上。USB 设备有描述符，说明设备类型、接口、端点和传输类型。HID 键盘通过 interrupt endpoint 周期性报告按键状态。

USB 概念	含义	OS 关系
descriptor	设备自描述信息	枚举和驱动绑定
endpoint	设备通信端点	控制/批量/中断/等时传输
URB	USB request block	异步提交和 completion
HID report	输入设备状态包	键盘鼠标事件解析

USB 热插拔意味着驱动必须随时处理 remove。用户还拿着旧 fd 时，设备可能已经拔掉；内核要让后续 read/write 返回错误，而不是访问已释放硬件状态。

音频、摄像头和实时流

音频和摄像头强调连续流。音频设备需要固定采样率、环形 buffer、低延迟和 underrun/overrun 处理。摄像头涉及传感器、ISP、DMA buffer、像素格式和帧同步。

```
audio playback:
user fills ring buffer
device DMA reads samples
period interrupt fires
kernel wakes audio server
user fills next period
```

这类设备说明“吞吐够”不等于“体验好”。实时流关心抖动、周期、buffer 深度、调度延迟和电源状态切换。

电源、散热和频率

现代计算机不断调整频率、电压和电源状态。CPU 有 P-state/C-state，设备有 runtime PM，整机有 suspend/resume，服务器有风扇和温度传感器，手机有热限制。OS 要在性能、功耗、温度和响应延迟之间权衡。

机制	含义	影响
P-state/DVFS	调整频率电压	性能、功耗、温度
C-state	CPU 空闲深度	唤醒延迟、节能
runtime PM	单设备空闲省电	设备访问前需 resume
suspend/resume	整机睡眠唤醒	驱动状态保存恢复
thermal throttling	温度过高降频	延迟尖峰、吞吐下降

性能测试必须记录频率和温度，否则同一程序两次结果可能只是因为第一次冷机、第二次热降频。

安全硬件：TPM、IOMMU、内存加密和安全启动

安全不是纯软件。TPM 可保存密钥和测量启动链；Secure Boot 验证启动组件；IOMMU 限制设备 DMA；内存加密保护 DRAM 内容；x86-64 CPU 提供 NX、SMEP、SMAP、CET、PKU/PKS 等硬件防护。

硬件安全机制	防什么	OS 使用方式
TPM	密钥被直接导出、启动链篡改	measured boot、磁盘解锁
Secure Boot	未签名启动组件	固件验证 bootloader/kernel
IOMMU	设备任意 DMA	DMA 隔离、虚拟机直通
NX	数据页执行代码	<code>W\TU\textasciicircum X</code> 、 栈不可执行

SMEP/SMAP CET	内核误执行或误访问用户页 返回地址和间接跳转被劫持	内核漏洞利用缓解 shadow stack、indirect branch tracking
PKU/PKS	同一地址空间内按 key 限 制访问	用户页或内核页的细粒度 保护

硬件安全机制都有边界。TPM 不能修复运行中内核漏洞；Secure Boot 不能保证用户空间没漏洞；IOMMU 不能防 CPU 自己写错内存。OS 必须把它们组合成分层防御。

虚拟化硬件

虚拟机让 guest OS 以为自己有 CPU、内存和设备。硬件虚拟化提供 VM entry/exit、二级页表、虚拟中断和设备直通支持。没有硬件辅助，很多特权指令和页表操作只能慢速模拟。

机制	作用	OS/Hypervisor 关系
VMX/SVM	运行 guest 并拦截敏感事件	hypervisor 控制 guest
EPT/NPT	guest physical 到 host physical 翻译	内存隔离和 overcommit
virtio SR-IOV	半虚拟化设备接口 设备虚拟功能	高效虚拟 I/O 直通性能和隔离

虚拟化再次说明硬件和 OS 是契约关系。guest 的“物理地址”可能只是 hypervisor 管理的第二层虚拟地址。

硬件观测入口

真实系统里可以通过多种入口观察硬件：

```
lscpu
lsmem
lspci -nn
lsusb
cat /proc/cpuinfo
cat /proc/iomem
cat /proc/interrupts
find /sys/devices -maxdepth 3 -type d
dmesg | grep -i -E 'acpi|pci|nvme|usb|iommu|tpm|thermal'
```

学习硬件不能只看框图。要能把命令输出映射到本节硬件：CPU 拓扑、NUMA node、PCIe 设备、USB 设备、中断 vector、MMIO 区间、驱动名称、电源状态和错误日志。

本节验收

- 能画出一台 x86-64 PC/服务器的主要硬件块和互联关系。
- 能说明 UEFI、ACPI、PCIe 枚举分别提供什么信息。
- 能区分 DRAM/ECC/NUMA、NVMe/网络/GPU/USB/音频等设备的 OS 关注点。
- 能解释电源、散热和频率为什么影响性能测试。
- 能说明 TPM、Secure Boot、IOMMU、NX 等安全硬件分别解决哪类问题。
- 能用系统命令找到设备、驱动、中断、MMIO 和 IOMMU 相关证据。

整机硬件详解：从芯片、板卡到操作系统对象

前面的章节已经解释了位、寄存器、指令、页表、cache、TLB、总线、MMIO、DMA 和中断。本节继续把范围扩大到一台真实计算机的全部重要硬件。目标不是把硬件工程每个公式都讲完，而是让读者知道：每个硬件部件为什么出现，给 OS 暴露了什么接口，失败时能留下什么证据，内核为什么必须为它建立对象、策略和测试。

核心思想

学习硬件不能停在“CPU、内存、磁盘、网卡”四个名词。真实机器还包括供电、时钟、复位、固件、芯片组、互联、存储介质、控制器、传感器、安全芯片、管理控制器、显示链路、音频链路、无线模块和调试设施。OS 的职责是把这些不同速度、不同可靠性、不同权限边界的硬件收敛为统一的软件契约。

先画整机，而不是先背术语

一台通用计算机可以先按“能源、时钟、计算、存储、互联、外设、管理、安全、观测”九类画图。这样做的好处是：一个新名词出现时，可以先判断它属于哪一类，再问它和 OS 的关系。

类别	典型硬件	OS 关系
能源	PSU、VRM、PMIC、电池、充电芯片	电源状态、功耗限制、掉电恢复
时钟复位	晶振、PLL、clock controller、reset controller	时间源、定时器、设备初始化顺序
计算	CPU、GPU、NPU、DSP、协处理器	调度、驱动、内存共享、队列提交
存储	SRAM、DRAM、Flash、NAND、磁盘、NVRAM	页分配、块层、文件系统、持久化语义
互联	片上总线、内存总线、PCIe、USB、CXL、I2C、SPI	枚举、带宽、延迟、热插拔、错误处理
外设	网卡、显示、音频、摄像头、键鼠、传感器	驱动、输入输出对象、权限和命名
管理	UEFI、BMC、EC、ACPI	启动、硬件描述、电源热管理
安全	TPM、Secure Boot、IOMMU、CPU 内存保护	启动测量、密钥保护、DMA 隔离
观测	PMU、JTAG、串口、trace buffer、错误寄存器	性能分析、故障定位、测试验收

这张表还可以帮助区分“硬件能力”和“OS 抽象”。例如 NVMe 是存储控制器和协议，OS 把它变成块设备和文件；TPM 是安全芯片，OS 把它用于测量启动和密钥封存；GPU 是计算与显示硬件，OS 把它包装成显存对象、命令队列、fence 和显示平面。

供电：没有稳定电源就没有正确执行

CPU 执行指令之前，系统必须先有稳定供电。台式机和服务从 PSU 获得多路电压，主板上的 VRM 把电压转换成 CPU、内存、芯片组和外设需要的电压。手机和嵌入式设备常用 PMIC 管理电池、充电、升降压、低功耗状态和电源域。

硬件	作用	OS 可见影响
PSU	提供整机电源	掉电、功率不足、服务器冗余电源事件
VRM	给 CPU/GPU/内存提供稳定电压	频率电压调节、功耗墙、热限制

PMIC	管理移动设备电源轨和电池	runtime PM、充电状态、低电量策略
power domain	允许局部断电	驱动 resume 前必须重新打开电源域
brown-out detector	检测电压过低	嵌入式复位、错误日志、数据保护

电源不稳定会造成随机重启、设备掉线、存储写入失败、PCIe 链路错误和内存错误。OS 不能假设所有失败都是软件 bug。可靠系统会把电源事件纳入日志，例如服务器 BMC 记录电源冗余状态，移动系统记录电池温度和充电限制，嵌入式系统在掉电前触发保存或保护动作。

电源域和驱动生命周期

x86-64 PC 和服务器的也会把 GPU、网卡、NVMe 控制器、USB 控制器、音频、BMC、风扇和传感器放入不同电源域或电源管理层级。设备空闲时关闭电源可以省电，但关闭电源也意味着寄存器状态、队列状态和时钟状态可能丢失。驱动的 `suspend/resume`、`runtime_suspend/runtime_resume` 本质上就是在管理这个硬件状态机。

```
runtime resume of a device:
  enable parent power domain
  enable regulator
  enable clocks
  deassert reset
  restore registers
  rebuild DMA rings if needed
  allow new I/O

runtime suspend:
  stop accepting new I/O
  wait or cancel in-flight requests
  save device state
  assert reset if required
  disable clocks
  disable regulator
  mark device suspended
```

如果驱动在设备断电后仍访问 MMIO，轻则读到无效值，重则总线错误。若 resume 后忘记恢复中断使能，设备完成了 I/O 也没人通知 CPU。若关闭电源域时还有 DMA 进行，设备可能写到已回收的内存。电源管理不是附加功能，而是设备生命周期的一部分。

时钟、PLL 和复位：硬件动作为什么要按顺序

芯片内部不是所有部件都用同一个频率。晶振提供基础时钟，PLL 产生 CPU、总线、内存、显示、音频、USB 等不同频率。reset controller 控制各模块复位线。许多设备初始化必须按顺序执行：先供电，再开时钟，再解除复位，再配置寄存器。

部件	作用	常见失败
crystal	稳定基础频率	时间漂移、外设协议频率不准
PLL	倍频和分频	lock 失败、切频瞬间不稳定
clock gate	关闭空闲模块时钟	驱动访问无时钟设备导致超时
reset line	把模块拉回初始状态	reset 顺序错，设备状态不可预测

watchdog clock	独立计时	主时钟挂死后仍能复位系统
----------------	------	--------------

音频设备尤其依赖准确时钟。采样率偏差会导致声音速度和同步问题；USB 和 PCIe 也要求稳定链路时钟；CPU 调频需要内核知道频率变化，否则调度统计和性能计数会失真。OS 的 clock framework、reset framework 和 regulator framework 就是为了把这些硬件依赖显式化。

复位向量和第一段代码

上电或复位后，CPU 从架构规定的位置取第一条指令，这个位置叫复位向量。第一段代码通常在 ROM 或固件 Flash 中。它不能假设 DRAM 已经可用，因为 DRAM 训练和控制器初始化还没做；也不能假设栈、堆、中断、cache 都可用。启动链必须一步一步把硬件带到可运行状态。

```
reset vector:
  run immutable ROM code
  choose boot source
  initialize minimal SRAM stack
  verify and load next firmware stage
  initialize DRAM controller
  load bootloader or kernel image
  pass hardware description to kernel
```

从这里可以看出，启动安全和启动可靠性天然交织。若 BootROM 验证失败，应停止启动或进入恢复模式；若 DRAM 初始化失败，内核根本没有稳定内存可用；若 bootloader 给错内存地图，内核页分配器可能覆盖固件保留区或内核镜像本身。

主板、封装和插槽：硬件拓扑不是软件想象

PC 主板把 CPU、内存、PCIe 插槽、芯片组、固件 Flash、USB、音频、网卡、电源管理和风扇连接起来。服务器主板还会有多个 CPU socket、多组内存通道、多条 PCIe switch、BMC、存储背板、传感器和远程管理网口。拓扑决定中断路由、NUMA 距离、DMA 可达性、热插拔路径和故障隔离边界。

拓扑事实	为什么重要	OS 影响
内存通道数量	决定内存带宽上限	NUMA、带宽测试、页放置
PCIe 插槽连接位置	决定设备离哪个 CPU 更近	IRQ affinity、DMA locality
PCIe switch	多设备共享上游链路	带宽竞争、错误隔离
BMC 独立管理网口	OS 之外还有管理平面	安全边界、运维日志
天线和射频前端	无线质量受物理设计影响	驱动只能看到链路质量结果

同样一块 NVMe SSD 插在不同插槽，性能可能不同；同一个网卡绑定到不同 CPU 的中断亲和性，延迟也可能不同。OS 性能分析不能只看进程代码，也要看硬件拓扑。

CPU 核内部：不只是 ALU

CPU 核通常包含取指前端、分支预测器、译码器、寄存器重命名、调度队列、整数执行单元、浮点和 SIMD 单元、load/store 单元、提交逻辑、L1 cache、TLB 和异常控制。ISA 只定义软件可见结果，微架构决定这些结果如何高效实现。

内部部件	作用	OS 相关性
branch predictor	猜测下一条执行路径	侧信道、上下文污染、性能抖动
rename/ROB	支持乱序执行和精确提交	精确异常、投机执行边界

load/store unit	处理内存访问	page fault、memory ordering、MMIO 限制
FPU/SIMD	浮点和向量计算	上下文保存、信号帧、内核禁用或受限使用
PMU	统计周期、cache miss、分支等事件	perf、性能诊断、回归测试

OS 不需要在每次调度时管理分支预测器细节，但安全补丁可能需要清理某些预测状态；OS 不需要理解每条 SIMD 指令的数学意义，但必须保存用户可见的向量寄存器；OS 不需要直接安排乱序执行，但必须使用屏障建立跨 CPU 和设备的可见顺序。

微码、MSR 和控制寄存器

许多 CPU 有微码，用更低层的内部操作实现复杂指令或修补硬件问题。x86 还提供大量 MSR，用于控制系统调用入口、性能计数、时间戳、缓存策略、调试和安全缓解。控制寄存器则管理页表、保护模式、cache、特权状态等核心能力。

```
examples of privileged CPU state:
page table root register
interrupt descriptor table pointer
syscall entry address
control bits for paging and protection
model specific registers for timer and PMU
```

这些寄存器只能由内核或更高特权层修改。普通进程若能修改系统调用入口，就可以劫持整个内核入口；若能修改页表根，就可以读取任意物理内存；若能禁用中断，就可以阻止调度。特权保护不是抽象原则，而是硬件寄存器权限的直接结果。

异构计算：GPU、NPU、DSP 和加速器

现代计算机的“计算”不只在 CPU 上。GPU 擅长大规模并行图形和通用计算；NPU 擅长神经网络算子；DSP 常用于音频、基带和信号处理；压缩、加密、视频编解码也常有专用加速器。OS 必须管理这些设备的内存、队列、权限和错误恢复。

加速器	硬件特点	OS/驱动任务
GPU	命令队列、显存、shader、display engine	显存分配、命令提交、fence、hang recovery
NPU	固定算子或矩阵加速	模型 buffer、DMA、调度、功耗控制
DSP	实时信号处理	firmware 加载、共享内存、低延迟消息
crypto engine	AES、SHA、公钥或随机数	密钥边界、异步请求、熵质量
video codec	编解码帧流	buffer 队列、格式协商、同步

加速器驱动的核心不是“调用硬件算一下”，而是管理异步命令。应用提交命令后，设备稍后完成；期间 buffer 不能释放，权限不能越界，错误要能回报，进程退出要能回收。GPU hang recovery 和块设备 reset 在结构上很像：停止新请求、处理旧请求、重建硬件状态、把结果传播给等待者。

DRAM 细节：bank、row、refresh 和 ECC

DRAM 不是一个简单数组。它由 channel、rank、bank、row、column 组织。访问前可能要激活某一行，读写后需要保持和预充电；电荷会泄漏，所以必须刷新。内存控制器负责把 CPU 请求调度到 DRAM 命令序列，并尽量利用行命中和并行 bank。

DRAM 概念	含义	OS/系统影响
---------	----	---------

channel	独立内存通路	多通道提升带宽，插条位置影响性能
bank	可并行访问的内部区域	访问模式影响延迟和带宽
row buffer	已激活行的缓存	顺序和局部访问更快
refresh	周期性恢复电荷	极端实时系统要考虑刷新抖动
ECC	检测或纠正位错误	错误日志、页下线、可靠性报告

ECC 错误是硬件可靠性进入 OS 的典型入口。可纠正错误不一定立即造成程序失败，但频繁出现在同一页或同一 DIMM 上就可能预示硬件退化。内核可以记录物理地址、页、DIMM 和错误类型，并在必要时 soft offline 某页。

内存类型：SRAM、DRAM、NAND、NOR、NVRAM、HBM

不同存储硬件的速度、寿命、随机访问能力和持久性完全不同。OS 之所以有页缓存、块层、文件系统、wear leveling、DAX、swap 和日志，是因为硬件不是一个统一的“保存字节”的盒子。

存储类型	特点	OS 设计点
SRAM	快、贵、容量小	cache、片上 SRAM、启动早期栈
DRAM	易失、容量大、随机访问	物理页、NUMA、page cache、DMA buffer
NOR Flash	可随机读取、擦写慢	固件存储、嵌入式启动镜像
NAND Flash	块擦除、页写入、有限寿命	FTL、TRIM、写放大、坏块管理
NVRAM/PMem	持久且接近内存访问模型	DAX、cache flush、持久化顺序
HBM	高带宽、封装近计算单元	GPU/AI 加速器、带宽敏感任务

持久内存最能说明“内存”和“存储”边界会变化。CPU 可能像访问内存一样访问持久介质，但 CPU cache 仍然是易失的。程序要保证崩溃后一致，仍然需要 flush cache line、写持久化屏障和设计日志协议。

CXL 和内存扩展

CXL 让 CPU 和设备之间可以进行更紧密的缓存一致性和内存语义互联。它可以用于内存扩展、内存池化和加速器共享内存。对 OS 来说，问题不是“多了一根更快的线”，而是出现了新的内存层级、距离、可靠性和热插拔模型。

CXL 场景	带来的能力	OS 问题
内存扩展	增加容量	NUMA 距离、页分配策略、回收优先级
内存池化	多主机共享资源池	隔离、热插拔、故障域
加速器一致内存	CPU 和设备共享数据结构	cache 一致性、权限、同步模型

当内存速度和可靠性不再统一时，OS 需要把“哪类页放在哪类内存”变成策略：热页放近处，冷页放远处；内核关键对象放可靠内存；大缓存可放扩展内存；故障设备上的页要迁移或下线。

系统互联和外设总线

x86-64 PC 和服务服务器常见 PCIe、USB、SATA、SMBus、LPC、CXL 等互联。不同总线的差别在于速度、主从关系、枚举能力、热插拔能力、事务模型和错误报告。

互联	常见用途	OS 关注点
PCIe	GPU、NVMe、网卡、加速器	枚举、BAR、MSI-X、AER、IOMMU
USB	外接设备、HID、存储、音频	热插拔、描述符、URB、权限
SMBus/I2C/SPI	传感器、PMIC、触摸、Flash	ACPI 描述、低速事务、同步访问
UART	串口控制台、调试、嵌入式通信	early console、日志、恢复入口
GPIO/PWM/ADC	板级控制、LED、风扇、按键、采样	pinctrl、sysfs 或字符设备、权限

总线决定驱动模型。PCIe 设备通常能自描述并可枚举；I2C/SMBus 设备常不能完整自描述，在 x86-64 平台上通常由 ACPI 告诉内核地址、硬件 ID 和中断资源；USB 设备热插拔频繁，驱动必须随时处理断开；UART 简单可靠，所以常作为最早日志通道。

PCIe 深讲：配置空间、BAR、MSI-X 和 AER

PCIe 设备被发现时，内核读取配置空间，识别 vendor ID、device ID、class code 和 capability。BAR 告诉内核设备需要的 MMIO 或 I/O 地址窗口。MSI/MSI-X 让设备通过内存写消息触发中断。AER 报告链路和事务错误。

```
PCIe probe evidence:
config space says what device is
BAR says where registers live
DMA mask says what addresses device can reach
MSI-X table says how many interrupt vectors it can use
AER status says whether link or transaction errors occurred
```

NVMe 和高端网卡会使用多个 MSI-X vector，让不同队列绑定不同 CPU，减少锁竞争并提高缓存局部性。若所有中断都落到一个 CPU，吞吐可能上不去；若 IRQ affinity 和 NUMA 不匹配，数据会跨节点移动。硬件队列、CPU 拓扑和调度策略必须一起看。

USB 深讲：描述符、端点和传输类型

USB 是主机控制的层级总线。设备插入后，主机读取设备描述符、配置描述符、接口描述符和端点描述符，然后选择配置并绑定驱动。端点支持控制、批量、中断和等时传输。键盘鼠标常用 HID interrupt endpoint，U 盘使用 bulk，摄像头和音频会用等时或批量流。

传输类型	特点	典型设备
control	用于枚举和控制命令	所有 USB 设备
bulk	保证可靠性，延迟不固定	存储、打印、部分网卡
interrupt	周期轮询，小数据、低延迟	键盘、鼠标、手柄
isochronous	保证带宽，不重传	音频、视频采集

USB 的关键是异步和热插拔。一个 URB 提交后，设备可能完成、失败、超时，也可能被拔掉。驱动必须处理用户还持有文件描述符但设备已经消失的情况。正确行为不是崩溃，而是让后续操作返回明确错误，并释放内核资源。

低速板级总线：I2C、SPI、GPIO、PWM、ADC

很多“计算机硬件”不是高速设备，而是低速控制和传感器。I2C 连接温度传感器、PMIC、触摸控制器、摄像头配置芯片；SPI 连接 Flash、显示屏、传感器；GPIO 读取按键、控制 LED、复位设备；PWM 控制风扇、背光和电机；ADC 采样电压和传感器信号。

硬件	常见用途	OS 抽象
I2C	低速寄存器访问，多从设备共享两根线	adapter、client、regmap
SPI	全双工同步串行，常用于 Flash 和屏幕	controller、device、transfer
GPIO	单个数字输入输出引脚	pinctrl、gpio、interrupt line
PWM	占空比控制	背光、风扇、电机、蜂鸣器
ADC/DAC	模拟和数字转换	传感器、电池电压、音频或工业控制

这些硬件让 OS 必须理解“板级设计”。同一类 x86-64 主板也可能接不同触摸芯片、传感器、EC、风扇控制器或背光控制器。内核镜像相同，但 ACPI 描述不同。驱动不能假设某个 GPIO 或 IRQ 一定存在，必须从固件描述中获取资源。

存储硬件一：机械硬盘

机械硬盘由盘片、磁头、主轴、电机、缓存和控制器组成。随机 I/O 慢，因为磁头要寻道，盘片要旋转到目标扇区；顺序 I/O 快，因为磁头移动少、预读有效。OS 早期的 I/O 调度大量考虑合并和排序，就是为了减少寻道。

HDD 事实	软件含义	OS 对策
寻道和旋转延迟高	随机小 I/O 代价大	合并、排序、预读
顺序吞吐较好	连续文件和大块读写有优势	extent、readahead、writeback
设备缓存易失 坏扇区可能出现	写完成不等于落盘 读写错误需要上报和重试	flush、FUA、fsync SMART、错误日志、文件系统校验

理解 HDD 后才能理解为什么传统文件系统强调布局、块组、预分配和顺序写。即使 SSD 普及，这些概念仍有价值，因为“把很多小随机操作变成批量顺序操作”也是现代存储和网络系统的通用优化。

存储硬件二：SSD、NAND 和 FTL

SSD 内部是 NAND Flash、控制器、DRAM 缓存、固件和 FTL。NAND 不能像 DRAM 一样随意覆盖单个字节，通常按页写、按块擦除。FTL 把主机看到的逻辑块地址映射到内部物理页，并处理垃圾回收、磨损均衡和坏块。

NAND/SSD 事实	影响	OS 接口
页写块擦 擦写寿命有限	覆盖写会变成内部搬迁 热点写会磨损介质	写放大、TRIM/discard wear leveling、SMART 寿命指标
内部并行通道 掉电保护差异	队列深度影响吞吐 缓存中的数据可能丢失	blk-mq、多队列、异步 I/O flush、FUA、文件系统日志

所以 fsync 的成本不是内核故意慢，而是它要求存储控制器把易失缓存和内部映射状态推进到崩溃后可恢复点。不同 SSD 的掉电保护能力不同，数据库和文件系统必须把硬件承诺纳入设计。

NVMe：队列化存储协议

NVMe 面向 PCIe 和多核系统设计。它使用 submission queue 和 completion queue，每个队列可以绑定一个或一组 CPU。驱动把命令写入提交队列，敲 doorbell；设备完成后写完成队列并触发 MSI-X。


```

NVMe read path:
filesystem maps file offset to block
block layer builds request
nvme driver fills command in submission queue
driver rings doorbell MMIO
controller DMA reads/writes data
controller writes completion entry
MSI-X interrupt wakes completion path

```

NVMe 让存储从“一个设备一把大锁”变成“多队列、多 CPU、批量 completion”。OS 的块层因此发展出 blk-mq。性能测试时要看队列深度、CPU 亲和性、NUMA、IOMMU、文件系统、page cache 和设备固件，而不能只看应用发了多少 read。

移动存储：eMMC、UFS、SD 卡

手机、平板和嵌入式设备常用 eMMC、UFS 或 SD 卡。它们有不同的命令集、队列能力、电源状态和可靠性。UFS 支持更高并行和性能，eMMC 更简单，SD 卡质量差异极大。移动系统还必须考虑功耗、热限制和突然断电。

设备	特点	OS 关注点
eMMC	集成控制器和 NAND，移动常见	boot partition、RPMB、寿命、flush
UFS	高性能移动存储，多队列能力更强	command queue、电源模式、热管理
SD 卡	可插拔、质量差异大	热插拔、写保护、错误恢复、超时

嵌入式产品的文件系统选择必须考虑存储特性。日志策略、写放大、断电保护、只读分区、A/B 更新、校验和坏块管理都不是纯软件偏好，而是对 Flash 介质和用户拔电行为的响应。

网络硬件：PHY、MAC、队列和 offload

有线网卡通常分为 PHY 和 MAC。PHY 处理物理信号，MAC 处理以太帧。现代 NIC 有 RX/TX ring、多队列、RSS、checksum offload、TSO/GSO、GRO、VLAN、时间戳和过滤规则。无线网卡还要处理射频、信道、加密、漫游、省电和固件。

网卡能力	目的	OS 影响
RSS	把不同流分到不同 RX 队列	多核并行、IRQ affinity
interrupt moderation	合并中断	吞吐提升但延迟增加
checksum offload	硬件计算校验和	抓包显示和 skb 标志要理解
TSO/GSO	大段交给硬件分片	减少 CPU 开销，改变包可见粒度
hardware timestamp	精确打时间戳	PTP、金融交易、工业控制

网络性能诊断必须同时看协议栈和硬件。TCP 拥塞控制、socket buffer、NAPI budget、队列深度、IRQ affinity、NUMA、本机防火墙、网卡 offload 和交换机链路都可能是瓶颈。

无线硬件：Wi-Fi、蓝牙、基带和射频

无线设备不是“没有线的网卡”这么简单。Wi-Fi 驱动常与设备固件协作，固件负责扫描、连接、加密、速率控制或省电的一部分。蓝牙涉及控制器、HCI、配对、低功耗

和音频 profile。手机基带通常是独立处理器，和应用处理器通过共享内存、PCIe、USB 或专用链路通信。

无线部件	特点	OS/安全关注
Wi-Fi firmware	处理射频和协议细节	固件加载、崩溃恢复、监管域
Bluetooth controller	HCI 命令和事件	配对权限、音频延迟、共存
cellular modem	独立基带系统	隔离、共享内存、SIM、紧急呼叫
RF front-end	功放、滤波、天线切换	信号质量、功耗、SAR 限制

无线问题经常无法只靠应用日志解释。链路质量、射频干扰、固件状态、省电模式、监管域限制、天线设计和驱动队列都可能参与。OS 要提供足够的统计和错误事件，让问题能从“网慢”拆成可验证路径。

显示硬件：从 framebuffer 到现代 DRM/KMS

最简单的显示模型是 framebuffer：内存中一块像素数组被显示控制器按刷新率扫描输出。现代图形系统把渲染和显示分开：GPU 渲染到 buffer，display engine 负责 scanout，compositor 协调窗口合成，KMS 设置模式、plane、CRTC、connector 和 page flip。

显示概念	含义	OS 关系
framebuffer	像素所在内存区域	early console、简单显示
plane	可叠加的显示层	cursor、overlay、视频平面
CRTC	扫描输出时序单元	分辨率、刷新率、vblank
connector	HDMI、DP、eDP、DSI 等输出	热插拔、EDID、链路训练
fence	buffer 使用完成同步点	防止显示未完成 buffer 被复用

显示错误可以来自像素格式、stride、cache flush、EDID 错误、vblank 时序、GPU 命令超时、显示链路训练失败或电源域没恢复。OS 图形栈看似复杂，是因为它要同时管理内存、异步命令、用户权限、实时刷新和热插拔。

GPU 内存和命令提交

GPU 驱动要管理显存或共享内存、页表、命令 buffer、同步对象和错误恢复。用户进程不能把任意命令直接丢给硬件，驱动必须校验命令、限制访问范围、管理地址空间，并在 GPU 挂起时回收或重置。

```
GPU command lifecycle:
userspace allocates buffer object
userspace records command buffer
driver validates referenced buffers
driver maps buffers into GPU virtual address space
driver submits command to hardware queue
GPU executes asynchronously
fence signals completion
buffer can be reused or displayed
```

这条路径和 NVMe、网卡类似：都是队列、DMA、completion、权限和 reset。区别是 GPU 命令更复杂，可能引用大量 buffer，且错误可能来自用户提交的 shader 或命令序列。因此 GPU 驱动既是设备驱动，也是用户态图形 API 与硬件之间的安全边界。

音频硬件：codec、DMA ring 和低延迟

音频链路包括应用、音频服务、内核音频框架、DMA buffer、音频控制器、codec、DAC/ADC、功放和扬声器或麦克风。播放时设备周期性从 ring buffer 取 PCM 样本；录音时设备周期性写入 ring buffer。每个 period 完成时触发中断或通知。

```
audio playback timing:
user fills period N
DMA engine reads period N
hardware plays samples
period interrupt wakes refill
user must fill period N+K before underrun
```

音频的核心指标不是最大吞吐，而是稳定低延迟。buffer 太小容易 underrun，产生爆音；buffer 太大延迟高，影响实时通话和乐器输入。调度延迟、CPU 省电状态、锁竞争、中断屏蔽和驱动 period 设置都会影响音频体验。

摄像头和图像链路

x86-64 PC 上的摄像头路径通常是 USB UVC 摄像头或 PCIe 采集卡。USB 摄像头把传感器读出、压缩或格式转换封装在设备内部，通过 xHCI endpoint 把帧传给主机；PCIe 采集卡通过 BAR、MSI-X 和 DMA ring 交付帧。内核侧的核心问题仍然是 buffer 生命周期、像素格式、时间戳、同步和权限。

摄像头部件	作用	OS/驱动问题
USB UVC device	采集、编码或格式转换	endpoint、带宽、帧率、断连恢复
PCIe capture card	高带宽图像输入	BAR、MSI-X、DMA ring、IOMMU
xHCI host controller	USB 传输调度	completion、带宽预算、错误恢复
V4L2 buffer	内核和用户共享帧	mmap、DMA-BUF、同步和权限

摄像头说明外设常常不是单个设备，而是一条流水线。任何一个环节的时钟、复位、电源、buffer 或格式不匹配都会导致无图、花屏、丢帧或延迟。驱动模型必须能表达 pipeline 中多个子设备之间的依赖。

输入设备：键盘、鼠标、触摸屏和传感器

输入设备把物理世界事件转换成内核事件。键盘可能走 USB HID 或矩阵扫描；触摸屏可能通过 I2C 报告多点坐标；加速度计、陀螺仪、光线传感器、温度传感器通过 I2C/SPI/ADC 周期采样。OS 要把这些不同硬件统一成输入事件、传感器数据和权限控制。

设备	硬件事件	OS 抽象
键盘	key matrix 或 HID report	key code、tty、快捷键
鼠标	相对位移、按键、滚轮	pointer event、桌面输入
触摸屏	多点坐标、压力、手势原始数据	multi-touch slot、输入子系统
IMU	加速度、角速度	sensor hub、时间戳、校准
温度传感器	芯片或环境温度	thermal zone、风扇和降频策略

输入是安全边界。键盘事件可能包含密码；传感器数据可能泄露用户行为；触摸注入可能影响授权操作。桌面和移动 OS 会对输入设备、后台读取和辅助功能接口施加权限，不只是因为用户体验，而是因为输入数据本身敏感。

EC、BMC 和管理平面

很多机器有 OS 之外的控制器。笔记本有 EC，管理键盘热键、电池、风扇和电源状态。服务器有 BMC，可在主 CPU 关机时远程开关机、读传感器、控制风扇、记录硬件错误、提供远程控制台。它们有自己的固件、网络接口和权限边界。

控制器	能做什么	OS 关注点
EC	电池、键盘热键、风扇、盖合状态	ACPI 事件、热管理、休眠唤醒
BMC	带外管理、电源、传感器、KVM	安全隔离、硬件日志、远程恢复
sensor hub	聚合低功耗传感器	时间戳、批量上报、省电
secure controller	密钥、指纹、安全启动辅助	权限、固件更新、攻击面

BMC 尤其值得警惕。它常有独立网络栈和高权限硬件访问能力，即使主 OS 很安全，BMC 漏洞也可能影响整机。生产系统的安全审计不能只看 Linux 用户态和内核，也要看固件、BMC、管理网和供应链更新。

安全硬件：TPM、TEE、熔丝和随机数

安全硬件的目标是提供软件难以伪造或窃取的根。TPM 可以保存密钥、记录启动测量并做远程证明；TEE 提供隔离执行环境；芯片熔丝保存不可逆配置；硬件随机数发生器提供熵；IOMMU 限制设备 DMA；CPU 权限位限制代码执行和内存访问。

安全硬件	提供的根	OS 使用方式
TPM	密钥和测量寄存器	measured boot、磁盘解锁、证明
Secure Boot	启动组件签名验证	防止未授权内核或 bootloader
TEE	隔离执行环境	DRM、支付、密钥操作、远程证明
fuse/OTP	不可变配置或唯一 ID	防回滚、设备身份、启动策略
TRNG	硬件熵源	内核随机池、密钥生成

安全硬件不是万能。Secure Boot 只能约束启动链，不保证运行中的内核无漏洞；TPM 能保护密钥不直接导出，但不能阻止已经授权的恶意软件请求解密；TEE 可能被侧信道或固件漏洞影响。OS 要把安全硬件放进整体威胁模型，而不是把它当作开关。

虚拟化硬件和嵌套地址翻译

虚拟化硬件让 hypervisor 能运行 guest OS，同时拦截敏感操作。CPU 提供 VM entry/exit、虚拟中断、二级页表和设备直通支持。二级页表把 guest physical address 再翻译到 host physical address；IOMMU 把直通设备限制在 guest 拥有的内存范围。

```

guest load:
  guest virtual -> guest physical by guest page table
  guest physical -> host physical by second-stage page table
  cache/TLB may cache combined translation
  violation causes VM exit or injected page fault

```

虚拟化让“物理地址”这个词变得分层。guest 看到的物理地址并不一定是机器真实物理地址。性能问题也分层：guest page fault、EPT violation、host reclaim、vCPU 调度、虚拟中断和设备后端都可能参与。

时间硬件：TSC、HPET、RTC 和 PTP

OS 需要多种时间。RTC 保存墙上时间，断电后靠电池维持；TSC 提供高频单调计数；HPET、local APIC timer 和 TSC deadline timer 提供定时事件；网卡硬件时间戳支持 PTP 精确同步。

时间硬件	用途	风险
RTC	启动时读取日期时间	电池耗尽、时间错误
TSC/counter	高速单调时间源	频率不稳、跨 CPU 不同步的旧系统
timer	调度 tick、hrtimer、超时	中断延迟、tickless 复杂性
interrupt		
PTP clock	网络硬件时间戳	驱动和交换机配置错误
watchdog	检测无进展	误触发或未覆盖死锁路径

时间错误会破坏很多系统：证书验证、日志排序、数据库事务、分布式锁、TCP 重传、调度统计和性能分析。OS 要区分 wall time 与 monotonic time，避免时间被 NTP 调整后让超时计算倒退。

热管理：传感器、风扇、降频和关机

芯片功耗变成热。温度过高会造成错误、老化甚至损坏。硬件提供温度传感器、功耗计数器、风扇控制、频率电压调节和紧急关机。OS 的 thermal framework 把传感器、冷却设备和策略连接起来。

```
thermal control loop:
  read temperature sensor
  compare with trip points
  choose cooling action
  reduce CPU/GPU frequency or increase fan
  if critical threshold reached, shutdown
```

热管理会直接改变性能。基准测试若不记录温度、频率和功耗限制，结果不可解释。移动设备上，持续 GPU/NPU 负载可能让 CPU 被迫降频；服务器上，风道堵塞可能让所有任务变慢。OS 性能诊断必须把热状态作为一等证据。

硬件错误：从静默错误到可恢复错误

硬件会失败。内存可能 bit flip，PCIe 可能链路错误，NVMe 可能超时，网卡可能 firmware crash，GPU 可能 hang，风扇可能停转，电池可能过热。OS 的目标不是假装硬件完美，而是尽量把错误变成可记录、可隔离、可恢复或可明确失败的状态。

错误类型	硬件证据	OS 行为
ECC correctable	syndrome、物理地址、DIMM	记录、计数、页下线候选
PCIe AER	requester、status、severity	reset 设备、降级或上报
NVMe timeout	command id、queue id、status	abort、reset controller、失败请求
GPU hang	fence timeout、engine state	reset engine、杀死 guilty context
thermal critical	sensor trip	降频、关机、记录事件

可恢复错误也要严肃对待。一次 ECC 可纠正错误可能只是偶发，重复错误可能是内存条损坏；一次 PCIe correctable error 可能无害，持续出现可能是线缆、插槽或信号完整性问题。OS 要提供统计和时间序列，而不仅是打印一句日志。

硬件观测：从命令输出反推机器结构

学习硬件要能落到证据。在 Linux 上，下面命令能把硬件描述、驱动绑定、中断、内存、块设备、网络 and 电源热状态暴露出来。

```
lscpu
lsmem
numactl --hardware
lspci -nn
lspci -vv -s 0000:xx:yy.z
lsusb -t
cat /proc/iomem
cat /proc/interrupts
cat /proc/timer_list
find /sys/bus/pci/devices -maxdepth 2 -type l
find /sys/class/thermal -maxdepth 2 -type f
find /sys/class/power_supply -maxdepth 2 -type f
dmesg | grep -i -E 'acpi|efi|pci|aer|nvme|iommuitpm|thermal|edac'
```

观察报告不要只贴输出，要回答问题：CPU 有几个 NUMA node；内存是否有可见错误报告；NVMe 挂在哪个 PCIe 路径；网卡用了几个 MSI-X；IOMMU 是否启用；USB 设备挂在哪个 root hub；温度传感器如何命名；启动日志里固件给了哪些表。

案例：一次开机失败怎么分层排查

假设机器按下电源后无法进入系统。不要直接说“系统坏了”。按硬件启动链分层：

1. 无任何灯和风扇：先查电源、主板供电、PMIC、保险丝或电池。
2. 有电但无画面：查复位、DRAM 初始化、固件、显示链路或串口日志。
3. 固件界面可见但无启动项：查存储枚举、UEFI NVRAM、启动顺序。
4. bootloader 可见但内核不启动：查内核镜像、initrd、命令行、内存地图。
5. 内核启动后 panic：查 early console、异常寄存器、页表、驱动 probe。
6. 进入用户态失败：查 rootfs、init、动态链接器、权限和挂载。

这就是“从硬件到 OS”的排查路径。每一层都要找证据：电源灯、串口、BMC 事件、固件日志、内核 early printk、dmesg、设备枚举和文件系统检查。

案例：设备明明插上了，为什么 OS 看不到

设备不可见也要分层。PCIe 设备看不到，可能是插槽没供电、reset 没释放、链路训练失败、BIOS/UEFI 禁用、资源分配失败、热插拔事件丢失或设备损坏。USB 设备看不到，可能是线缆、供电、hub、枚举描述符错误或驱动拒绝。I2C/SMBus 设备看不到，可能是 ACPI 没描述、地址错、上拉电阻问题或设备还在复位。

```
device invisible checklist:
  is power enabled?
  is clock enabled?
  is reset deasserted?
  does bus enumerate it?
  does firmware describe it?
  did driver probe run?
  did probe fail and unwind?
  are permissions hiding it from user space?
```

这个清单比“重装驱动”更可靠，因为它沿着硬件事实走。驱动只是链条最后几环之一。若设备没有供电或总线根本枚举不到，再好的驱动也不会被调用。

案例：一次 NVMe 超时

NVMe 超时通常表现为请求长时间没有 completion。排查时要同时看块层、驱动、PCIe、IOMMU、电源管理和设备固件。


```
NVMe timeout path:
block request issued
command placed in submission queue
doorbell written
no completion arrives before timeout
driver tries abort
if abort fails, reset controller
failed requests complete with error
filesystem or application observes EIO or delay
```

可能原因包括：设备固件挂起，PCIe 链路错误，APST 省电状态恢复失败，IOMMU fault，MSI-X 中断丢失，队列内存损坏，电源不足或温度过高。一次超时不是单纯“磁盘慢”，而是一条异步硬件队列没有收敛。

案例：网络延迟突然升高

网络延迟升高可能来自应用、协议栈、网卡、PCIe、CPU 调度、电源和交换机。硬件方向的典型检查包括：网卡是否降速；是否发生 RX/TX error；IRQ 是否集中在一个 CPU；NAPI poll 是否积压；CPU 是否降频；网卡 interrupt moderation 是否过大；是否跨 NUMA 访问。

```
network latency evidence:
ethtool <dev>
ethtool -S <dev>
cat /proc/interrupts
mpstat -P ALL 1
perf top or perf record
tc -s qdisc show
ss -tin
```

如果只看应用日志，可能只看到“请求慢”。把硬件路径展开后，才能判断是 TCP 重传、队列积压、中断亲和性、CPU 频率、网卡固件还是交换机链路造成的。

案例：屏幕花屏或黑屏

显示故障可以来自 GPU 渲染、buffer 格式、显示控制器、连接器、线缆、显示器、背光或电源域。黑屏不等于系统死机，系统可能仍在运行，只是 scanout 失败。串口、SSH、键盘灯和日志能帮助区分。

```
display failure path:
is GPU driver loaded?
did modeset succeed?
is connector detected?
did EDID parse?
is framebuffer allocated?
did page flip complete?
is backlight enabled?
did GPU hang or reset?
```

花屏常见于像素格式、stride、cache 同步或显存损坏；黑屏可能是 EDID、链路训练、背光、电源域或 GPU hang。图形系统复杂，是因为它跨 CPU、GPU、显示控制器、内存、同步和用户态合成器。

案例：音频爆音和摄像头丢帧

音频爆音通常来自 buffer underrun：设备按固定速率取样本，但用户态或内核没有及时填充。原因可能是调度延迟、CPU 深睡眠、period 太小、锁竞争、中断屏蔽或音频服务负载过高。摄像头丢帧类似：传感器按帧率输出，ISP 和 DMA 必须及时提供 buffer。

现象	硬件本质	排查证据
----	------	------

音频爆音	DMA ring 没及时补数据	underrun 计数、调度延迟、period 设置
摄像头丢帧	frame buffer 不足 或 pipeline 堵塞	V4L2 drop、CSI error、ISP 日志
视频撕裂	page flip 不在 vblank 边界	compositor 日志、vblank 计数
输入延迟	中断、调度或合成延迟	evtest 时间戳、trace、CPU 频率

这些设备说明实时系统的难点不是平均吞吐，而是 deadline。只要偶尔超过周期，就会产生用户可见问题。OS 必须用调度、优先级、buffer 深度和电源策略共同控制抖动。

案例：温度上升后的性能坍塌

程序开始运行很快，几分钟后变慢，常见原因是热限制。CPU 或 GPU 温度达到阈值后，thermal governor 要求降低频率，PMIC 或固件也可能施加功耗上限。风扇策略、散热条件和环境温度都会改变结果。

```
thermal performance chain:
workload increases switching activity
power rises
temperature sensor crosses trip point
governor reduces frequency or voltage
scheduler sees lower capacity
application throughput drops
```

因此性能报告必须记录温度、频率、governor、功耗限制和持续时间。只跑一次短 benchmark 得到的峰值，不能代表长期稳定性能。操作系统要把硬件热状态纳入调度和能耗模型，尤其在移动设备和异构核心系统中。

源码级入口：从 Linux 看硬件对象

下面列出读 Linux 内核时常见的硬件对象入口。这里不要求一次读完，而是给读者建立“概念到源码路径”的索引。

硬件/机制	典型源码路径	先看什么
PCI core	<code>drivers/pci/</code>	枚举、BAR、MSI、AER
USB core	<code>drivers/usb/core/</code>	描述符、URB、hub、热插拔
DMA API	<code>kernel/dma/</code>	map/unmap、coherent、IOMMU
IOMMU	<code>drivers/iommu/</code>	domain、IOVA、fault
clock framework	<code>drivers/clk/</code>	prepare、enable、rate
regulator	<code>drivers/regulator/</code>	电源轨、约束、enable
thermal	<code>drivers/thermal/</code>	thermal zone、cooling device
DRM/KMS	<code>drivers/gpu/drm/</code>	plane、CRTC、connector、fence
ALSA HD-audio	<code>sound/pci/hda/</code>	codec、stream、DMA buffer
V4L2/media	<code>drivers/media/</code>	subdev、pipeline、buffer queue
EDAC/MCE	<code>drivers/edac/</code> 、 <code>arch/x86/kernel/cpu/mce/</code>	内存错误和机器检查

阅读源码时要抓四件事：设备对象在哪里创建，资源在哪里申请，失败路径如何回滚，异步 completion 如何回到上层。只看 happy path 会低估驱动复杂度。

最小实验：把本机硬件做成报告

读者可以用一台 Linux 机器做硬件报告。报告不追求设备越贵越好，而追求证据完整。

1. 画出 CPU、内存、PCIe、USB、存储、网络、显示、电源热管理的整机图。
2. 用 `lscpu` 和 `numactl` 说明 CPU 拓扑、cache 层级和 NUMA。
3. 用 `lspci -vv` 找一个 NVMe 或网卡，标出 BAR、MSI-X、driver、IOMMU group。
4. 用 `cat /proc/interrupts` 找到该设备中断落在哪些 CPU。
5. 用 `lsusb -t` 找到 USB 拓扑和速率。
6. 用 `find /sys/class/thermal` 找到温度传感器和 cooling device。
7. 用 `dmesg` 找启动中的 ACPI/EFI/PCI/IOMMU/TPM/NVMe 证据。
8. 任选一个故障假设，写出从硬件到 OS 的排查路径。

这个实验的评分重点不是命令数量，而是能否把命令输出解释成硬件结构。例如看到 MSI-X vector，不只写“有中断”，还要说明多队列设备如何用这些 vector 分散 completion；看到 IOMMU group，不只写“启用了 IOMMU”，还要说明设备隔离边界。

硬件概念的学习顺序

如果读者觉得硬件太多，可以按下面顺序学习。每一步都要能画图、能解释 OS 用途、能说出一个失败案例。

1. 位、门电路、寄存器、时钟、状态机。
2. CPU 取指执行、寄存器、栈、异常和特权级。
3. 物理地址、虚拟地址、页表、TLB、cache。
4. MMIO、总线、中断、DMA、IOMMU。
5. 固件、启动链、UEFI 内存地图、ACPI。
6. PCIe、USB、I2C/SPI/GPIO 等设备发现方式。
7. 存储、网络、显示、音频、摄像头等端到端 I/O。
8. 电源、时钟、热管理、休眠唤醒。
9. 安全硬件、管理控制器、虚拟化硬件。
10. 硬件错误、观测工具和故障恢复。

这个顺序从“机器如何一步一步执行”走到“整机如何长期可靠运行”。操作系统的抽象就在这条路上逐层长出来：线程来自寄存器和栈，进程来自页表和特权级，文件来自存储控制器和块层，socket 来自网卡和协议栈，权限来自硬件边界和内核对象。

本节硬验收

- 能画出一台 x86-64 PC 或服务器的供电、时钟、CPU、内存、互联、外设、固件和安全硬件图。

- 能解释 power domain、clock、reset 的顺序为什么会影响驱动 probe 和 resume。
- 能区分 PCIe、USB、I2C、SPI、UART、GPIO 这几类互联的发现方式和驱动模型。
- 能说明 HDD、SSD、NVMe、eMMC/UFS 的介质差异如何影响块层和文件系统。
- 能从 PHY/MAC/RX ring/MSI-X/NAPI/socket queue 讲出网卡收包路径。
- 能从 framebuffer/GPU command/fence/CRTC/connector 讲出显示路径。
- 能说明音频、摄像头和传感器为什么更关心周期、buffer 和抖动。
- 能说明 EC、BMC、TPM、TEE、IOMMU 这些“CPU 之外的控制面”分别带来什么边界。
- 能用 lspci、lsusb、dmesg、/proc/interrupts、/sys 为硬件结论找证据。
- 能把一次设备不可见、NVMe 超时、网络延迟、黑屏、音频爆音或热降频拆成硬件到 OS 的分层路径。

硬件案例深拆：从事件、对象到证据

本节把硬件知识压到具体路径里。每个案例都回答五个问题：事件从哪里来，经过哪些硬件，内核维护哪些对象，用户看见什么，失败时从哪里取证。这样写的目的不是多堆页数，而是把“计算机相关硬件”变成能解释、能调试、能测试的系统知识。

案例零：一条 C 语句如何落到硬件

先看最普通的一句代码：

```
array[i] = array[i] + 1;
```

编译器可能把它变成地址计算、load、加法、store。CPU 执行时并不是直接“改数组”，而是一步步经过寄存器、地址生成、TLB、页表权限、cache、store buffer 和一致性协议。

```
1. compute virtual address: base + i * element_size
2. look up TLB for virtual page
3. if TLB misses, walk page table
4. check user/supervisor and write permission
5. load cache line into L1 if absent
6. read old value from cache line
7. add one in ALU
8. place store into store buffer
9. make modified cache line visible by coherence rules
```

这条路径解释了几个常见现象。若虚拟地址没有合法 VMA，内核给进程发送 SIGSEGV；若 VMA 合法但 PTE 不 present，内核可能分配页、读取文件页或执行 COW；若两个 CPU 更新同一 cache line 上的不同计数器，性能会因为伪共享下降；若 store 还在 store buffer 中，另一个 CPU 看到的顺序可能需要原子操作和内存屏障来约束。

硬件步骤	出错或变慢的原因	OS 证据
地址计算	指针越界、整数溢出	sanitizer、异常地址、core dump

TLB/page walk	页表不存在、权限不够	page fault 统计、 <code>perf stat TLB miss</code>
cache 访问	cache miss、伪共享	<code>perf stat cache miss</code> 、 LLC miss
store 可见性	数据竞争、缺少屏障	KCSAN、TSAN、lockdep、代码审查

所以“内存访问”不是一个单点动作。操作系统中的缺页、COW、mmap、锁、原子变量、NUMA 和性能分析，都在为这条路径服务。

案例一：复位到第一个用户进程

从按下电源到第一个用户进程运行，机器经历了多级控制权转移。每一级都只拥有下一步所需的最小能力。

```
power good
-> reset vector
-> immutable BootROM or firmware entry
-> DRAM training and platform init
-> bootloader or EFI stub
-> kernel decompression and early entry
-> early page tables and console
-> memory allocator and scheduler
-> driver probing and root filesystem
-> exec first user process
```

这条链路里，硬件和软件界限不断变化。UEFI 固件初始化 DRAM、PCIe、ACPI 和基础平台状态；bootloader 或 EFI stub 把内核镜像、initrd、命令行、UEFI 内存地图传给内核；内核接管页表、中断、调度和驱动；最后 `execve` 把第一个用户态程序装入地址空间。

阶段	最关键的硬件事实	失败证据
复位向量	CPU 从固定地址取指	无串口输出、BMC 事件、逻辑分析仪
DRAM 初始化 固件表	内存控制器和内存条必须稳定 内存地图、设备、中断、NUMA 信息	内存训练失败码、固件日志 ACPI 错误、 <code>dmesg</code> 启动日志
内核早期 驱动 probe	栈、BSS、页表、early console 资源、IRQ、DMA、clock、reset	early panic、异常寄存器 probe fail 日志、sysfs 设备和 ACPI 节点
用户态启动	rootfs、init、ELF、动态链接器	kernel panic、 <code>No init found</code>

Linux 源码阅读可以从 `arch/x86/boot/`、`arch/x86/kernel/head64.c`、`init/main.c`、`drivers/firmware/efi/`、`drivers/acpi/`、`drivers/pci/` 开始。读的时候不要只看函数名，要问：这一行代码依赖哪些硬件已经可用，哪些还不可用。

案例二：一次定时器抢占

抢占式调度不是调度器单独完成的。它依赖硬件定时器在未来某个时间点打断 CPU。假设用户程序正在死循环，定时器中断到来后，CPU 切入内核入口，内核记录当前任务运行时间，设置重新调度标志，在返回路径切换到另一个任务。

```
user loop running
-> local APIC timer or TSC deadline timer fires
-> CPU enters interrupt vector
-> kernel saves interrupt frame
-> tick handler updates scheduler clock
```

```

-> current task exhausts slice
-> return path calls scheduler
-> switch registers and stack to next task
-> return to user mode of next task

```

这里至少有四种硬件状态参与：定时器、当前 RIP 和寄存器、内核栈、页表或地址空间标识。若中断被长期关闭，抢占会延迟；若时钟源不可靠，调度统计会错；若保存寄存器不完整，任务恢复后会计算错误。

内核对象	对应硬件状态	典型源码入口
<code>task_struct</code>	任务身份、调度状态、地址空间	<code>include/linux/sched.h</code>
内核栈	中断和系统调用时的可信栈	<code>arch/x86/entry/entry_64.S</code>
调度类	runnable 任务的选择策略	<code>kernel/sched/</code>
<code>clockevent</code>	产生下一次定时事件	<code>kernel/time/</code>
上下文切换代码	保存恢复寄存器和栈	<code>arch/x86/kernel/process_64.c</code>

可观测证据包括 `cat /proc/interrupts` 中 timer 相关计数、`perf sched-trace-cmd record -e sched:*/ /proc/schedstat`。若系统卡顿，要区分是没有定时器中断、调度器没选到任务、任务不可运行，还是 CPU 被更高优先级中断或关中断区域占用。

案例三：TLB shutdown 为什么会拖慢多核

进程取消映射一段内存时，内核修改页表。但其他 CPU 可能仍然缓存旧 TLB 项。若内核立即释放物理页，另一个 CPU 可能继续用旧翻译访问已释放页，破坏隔离。因此内核必须让所有相关 CPU 失效旧翻译。

```

unmap virtual page:
  take mm locks
  remove or modify PTE
  collect CPUs that may run this address space
  send IPI to those CPUs
  each CPU invalidates TLB entry or range
  wait for acknowledgement
  free physical page after old translations are gone

```

这就是为什么频繁 `munmap`、频繁改变页权限、频繁释放大块映射，在多核系统上可能很贵。成本不只是改内存里的页表项，还包括跨 CPU IPI、等待远端 CPU 响应、打断远端工作流和刷新 TLB 热状态。

设计选择	好处	代价
立即 shutdown	隔离清晰，回收及时	IPI 和等待成本高
批量 unmap	分摊 shutdown 成本	回收延迟、实现复杂
PCID	降低地址空间切换刷新	标签管理和一致性仍复杂
大页	减少 TLB 项数量	拆分和权限粒度成本

Linux 相关路径可读 `mm/mmu_gather.c`、`arch/x86/mm/tlb.c`、`include/linux/mmu_gather.h`。观察可以用 `perf stat -e tlb:tlb_flush` 或架构支持的 TLB 事件，再配合 `perf record` 看是否大量时间落在 TLB flush 和 IPI 路径。

案例四：一次 `malloc` 到物理页

`malloc` 通常先在用户态分配器里找空闲块。若分配器没有合适空间，它可能通过 `brk` 扩展堆，或通过 `mmap` 建立新映射。即使系统调用成功，物理页也不一定立刻分配。第一次写入时触发缺页，内核才分配零页或物理页。

```
malloc large object:
userspace allocator decides mmap
kernel creates VMA
syscall returns virtual address
user first writes address
CPU raises page fault because PTE absent
kernel checks VMA permissions
allocator takes physical page from buddy/slab path
kernel zero-fills page
kernel installs writable PTE
faulting instruction retries
```

这个案例把用户分配器、VMA、缺页、物理页分配、cache 和安全连接起来。零填充不是浪费，而是隔离要求：新进程不能读到旧进程释放页里的数据。NUMA first-touch 也在这里生效：哪个 CPU 第一次写页，内核通常倾向从本地 NUMA node 分配页。

现象	硬件/内核原因	证据
<code>malloc</code> 快，首次访问慢	懒分配和缺页	<code>perf stat page-faults</code>
跨 NUMA 变慢	页在远端 node	<code>numastat</code> 、 <code>numactl</code>
RSS 小于虚拟内存	VMA 已建但页未实际分配	<code>/proc/<pid>/smaps</code>
释放后内存不立刻回系统	分配器缓存和 VMA 策略	<code>mallinfo</code> 、 <code>pmap</code>

源码入口包括 `mm/mmap.c`、`mm/memory.c`、`mm/page_alloc.c`、`mm/huge_memory.c`。学习时要把“虚拟地址可用”和“物理页已经分配”分开。

案例五：一次 DMA 描述符顺序错误

假设驱动要让设备读取一段 `buffer`。正确顺序是：准备 `buffer`，映射 DMA，填写描述符，执行 DMA 写屏障，再写 `doorbell`。若屏障缺失，设备可能先看到 `doorbell`，再读到旧描述符或半初始化描述符。

```
correct submit:
desc.addr = dma_addr
desc.len = len
desc.flags = READ
dma_wmb()
writel(new_tail, DOORBELL)

buggy submit:
desc.addr = dma_addr
desc.len = len
writel(new_tail, DOORBELL)
desc.flags = READ
```

这个 bug 难复现，因为它依赖 CPU store `buffer`、cache、互联排序和设备读取时机。低负载、单核或某种 CPU 上可能一直正常；换平台、换优化级别、换设备后就出现随机超时或 DMA 错误。

错误	后果	证据
----	----	----

doorbell 早于描述符可见	设备读到旧命令	command timeout、设备错误状态
completion 前释放 buffer	迟到 DMA 写坏内存	KASAN、IOMMU fault、随机崩溃
方向标错	cache clean/invalidate 错	数据旧、数据损坏
unmap 漏掉	IOVA 泄漏或权限残留	DMA debug、IOMMU 统计

Linux 提供 DMA API 和 DMA debug 是为了让驱动不要直接拼物理地址。相关入口包括 [Documentation/core-api/dma-api.rst](#)、[kernel/dma/](#)、[lib/dma-debug.c](#)、[drivers/iommu/](#)。驱动评审时必须追踪 buffer 所有权：CPU 写、设备读、设备写、CPU 读，每次转换都要有对应屏障和 map/unmap。

案例六：冷缓存文件读取

用户调用 `read(fd, buf, len)`。若 page cache 没有对应页，路径会从 VFS 进入文件系统，再到块层、I/O 调度、NVMe 或其他存储驱动，最后设备 DMA 把数据放进页缓存。

```
cold read:
user read()
fd -> file -> inode
page cache lookup misses
filesystem maps file offset to disk blocks
block layer builds bio/request
driver submits command to device queue
device DMA fills page cache page
completion interrupt wakes waiting task
kernel copies data to user buffer
```

关键是区分三种“完成”。设备 completion 表示硬件完成了 DMA；page cache uptodate 表示内核页里有了数据；`read` 返回表示数据已经复制到用户 buffer。这三个时间点不同。

层次	对象	源码入口
系统调用	fd、file、iov iterator	fs/read_write.c
页缓存	page/folio、xarray	mm/filemap.c
文件系统	inode、extent、block mapping	fs/ext4/ 、 fs/xfs/
块层	bio、request、blk-mq queue	block/
设备驱动	submission/completion queue	drivers/nvme/host/

证据可以用 `strace` 看系统调用，用 `vmtouch` 或 `mincore` 判断缓存，用 `blktrace` 或 `bpftrace` 观察块 I/O，用 `iostat -x` 看设备队列。若第二次读取明显更快，通常是 page cache 命中，不是硬盘突然变快。

案例七：写文件、掉电和 fsync

写文件最容易误解。应用调用 `write` 成功，只表示内核接收了数据，通常还在 page cache 中。后台 writeback 会稍后把 dirty page 写到设备。若应用需要崩溃后数据存在，必须使用 `fsync` 或文件系统提供的事务语义。

```
safe replace pattern:
write new contents to temp file
fsync(temp file)
rename temp file to target name
fsync(parent directory)
```

这段模式看起来繁琐，是因为硬件和文件系统有多个持久化对象：文件数据、inode 元数据、目录项、日志提交、设备 volatile cache。只同步文件不一定同步目录项；设备返回写完成不一定意味着掉电后仍存在，除非 flush/FUA 语义把数据推到稳定介质。

阶段	崩溃后风险	约束方式
write 返回	数据可能只在内存	fsync、同步写策略
writeback 完成	设备缓存可能易失	flush、FUA、掉电保护
日志提交	元数据恢复有序	journaling、barrier
rename 完成	目录项可能未持久化	fsync parent directory

源码入口包括 mm/page-writeback.c、fs/sync.c、fs/ext4/fsync.c、block/blk-flush.c。测试应使用故障注入或虚拟块设备模拟掉电，而不是只在正常关机后看文件是否存在。

案例八：USB 键盘按键到 shell

USB 键盘不是把字符直接交给 shell。设备周期性上报 HID report，host controller 用 DMA 把报告放入内存，USB core 完成 URB，HID 驱动解析按键，input 子系统产生事件，tty line discipline 把按键转成终端输入，shell 的 read 被唤醒。

```
key press:
key matrix changes
keyboard firmware builds HID report
USB host polls interrupt endpoint
DMA writes report to memory
HID driver parses usage code
input subsystem emits key event
tty handles canonical mode and echo
shell read() returns after line discipline policy
```

同一个按键可能因为键盘布局、修饰键、终端模式、输入法、tty 设置而得到不同字符。硬件只报告按键状态，字符语义在更高层解释。

失败点	表现	证据
USB 枚举失败	设备不存在	lsusb、dmesg
HID report 异常	按键错乱或无响应	hid-recorder、内核日志
input 层未创建设备	桌面无输入	/proc/bus/input/devices
tty canonical 模式	输入到回车才返回	stty -a
shell 被信号打断	read 返回错误或重启	strace

相关源码入口有 drivers/usb/core/、drivers/hid/、drivers/input/、drivers/tty/。这个案例说明“输入”跨越硬件、固件、总线、驱动、内核子系统和用户态协议。

案例九：网卡收包到 recv

网卡收包路径是硬件队列和协议栈协作的经典例子。驱动预先把 RX buffer 映射给设备，设备收到包后 DMA 写入 buffer，写 completion，触发中断。内核通常用 NAPI 关闭一段时间中断并轮询多个包，减少中断风暴。

```
packet receive:
NIC receives bits from PHY
```

```

MAC validates frame and writes DMA buffer
NIC writes descriptor status
MSI-X interrupt targets selected CPU
NAPI poll harvests packets
driver builds skb
IP/TCP stack processes headers
socket receive queue gets data
epoll or recv wakes user task

```

网络路径里的硬件细节会直接影响上层延迟。RSS 决定流分到哪个队列；IRQ affinity 决定中断在哪个 CPU；NUMA 决定 DMA buffer 离 CPU 远近；interrupt moderation 决定中断合并延迟；GRO 决定包在协议栈里的聚合粒度。

调优项	改变什么	风险
RX ring size	可暂存包数量	太小丢包, 太大延迟和内存占用
interrupt moderation	中断频率	吞吐和延迟折中
RSS indirection	流到队列映射	负载不均或破坏局部性
IRQ affinity	completion 运行 CPU	热点 CPU 或跨 NUMA
GRO/TSO offload	包处理粒度	抓包和延迟理解更复杂

源码入口包括 `net/core/dev.c`、`net/ipv4/tcp_input.c`、`drivers/net/ethernet/`。证据命令包括 `ethtool -S`、`ethtool -l`、`cat /proc/interrupts`、`nstat`、`ss -tin`、`dropwatch`。

案例十：Wi-Fi 连接突然不稳定

Wi-Fi 不稳定时，应用只能看到请求慢或断线。底层可能是射频信号差、漫游、信道拥塞、省电策略、固件崩溃、监管域限制或驱动队列堵塞。Wi-Fi 设备通常有固件，很多 MAC 层策略在固件中执行，OS 通过命令和事件与它协作。

```

Wi-Fi path:
supplicant requests connection
driver sends commands to firmware
firmware scans and authenticates
radio transmits frames over selected channel
firmware reports events and received frames
kernel network stack exposes normal netdev

```

有线网络里 PHY/MAC 已经复杂，无线还多了信道、速率控制、重传、加密、漫游和省电。故障排查要看 `iw dev wlan0 link`、`iw dev wlan0 station dump`、`dmesg` 固件日志、`wpa_supplicant` 日志、信号强度、重传计数和省电状态。

案例十一：GPU 渲染一帧到屏幕

用户界面的一帧通常由应用渲染、合成器合成、GPU 执行命令、显示控制器扫描输出。应用可能使用 OpenGL、Vulkan 或软件渲染；合成器把多个窗口合成到目标 buffer；KMS 在 vblank 边界 page flip。

```

frame lifecycle:
application renders into buffer
compositor receives buffer and fence
GPU executes composition commands
fence signals render completion
KMS schedules page flip at vblank
display controller scans buffer
monitor shows pixels

```

显示路径里有两个异步边界：GPU 命令完成和显示扫描完成。若 buffer 在 GPU 完成前被显示，会出现未完成内容；若 buffer 在显示扫描时被修改，会撕裂；若 page flip 错过 vblank，会多等一个刷新周期。

对象	作用	Linux 入口
buffer object	显存或共享内存对象	drivers/gpu/drm/
fence	异步完成同步	drivers/dma-buf/
plane/CRTC	显示合成和扫描输出	DRM/KMS helper
connector	HDMI、DP、eDP、DSI	EDID、hotplug、modeset

黑屏或卡顿不能只怪应用。证据包括 `dmesg GPU hang`、`/sys/class/drm/` 连接器状态、compositor 日志、`modetest`、`intel_gpu_top` 或对应厂商工具。

案例十二：音频爆音

音频设备以固定采样率消耗数据。假设采样率 48kHz，period 只有几毫秒。用户态音频服务必须在设备读完当前 period 前填好后续数据。任何调度延迟、锁竞争、CPU 深睡眠、中断延迟都可能造成 underrun。

```
audio underrun:
device DMA consumes playback ring
period interrupt should wake audio thread
audio thread wakes too late
next period contains no valid samples
hardware outputs silence or repeats stale data
user hears pop or gap
```

这个案例说明实时不是“CPU 总体很空”就够。即使平均 CPU 使用率只有 20%，一次几十毫秒的调度停顿也能毁掉音频。证据包括 ALSA underrun 日志、音频服务日志、`cyclicttest`、调度 trace、CPU C-state 和频率状态。

案例十三：摄像头采集一帧

摄像头路径从光开始。x86-64 PC 上常见的 USB UVC 摄像头会在设备内部完成传感器读出和图像处理，再通过 USB 主机控制器把帧数据传到内存；PCIe 采集卡则通过 BAR、MSI-X 和 DMA 队列交付帧。V4L2 或 media framework 把 buffer 交给用户态。每一帧都有时间戳和 buffer 生命周期。

```
camera frame:
sensor exposure
USB xHCI or PCIe capture card transfers pixels
host controller or device DMA writes output buffer
MSI-X or completion event marks buffer done
V4L2 dequeues buffer to userspace
application encodes or displays frame
```

摄像头比普通字符设备复杂，因为它是 pipeline。sensor、CSI receiver、ISP、scaler、codec 都可能是不同子设备。格式、分辨率、帧率、lane 数、时钟和电源顺序必须一致。证据包括 `media-ctl -p`、`v4l2-ctl --all`、帧 drop 计数、CSI error 和 ISP 日志。

案例十四：温度传感器触发降频

CPU 或 GPU 高负载运行后温度升高。传感器超过 trip point，thermal framework 调用 cooling device，可能降低 CPU 频率、限制 GPU、提高风扇转速或最终关机。用户看到的是吞吐下降或延迟升高。

```
thermal throttling:
workload raises power
temperature sensor crosses trip point
thermal governor selects cooling state
```

```
cpufreq or devfreq reduces frequency
scheduler capacity model changes
application observes lower throughput
```

硬件热状态是性能报告必须包含的上下文。证据包括 `/sys/class/thermal/thermal_zone*/temp`、`cpupower frequency-info`、`turbostat`、`perf stat`、风扇转速和环境温度。若不记录这些，两个 benchmark 结果没有可比性。

案例十五：Secure Boot 到磁盘解锁

安全启动和磁盘加密常组合使用。固件验证 bootloader，bootloader 验证内核或测量内核，TPM PCR 记录启动链状态。磁盘密钥可以封存到特定 PCR 状态，只有启动链未被篡改时才自动解锁。

```
measured boot:
firmware measures bootloader into PCR
bootloader measures kernel and initrd
kernel or initramfs asks TPM to unseal key
TPM checks PCR values
if values match policy, releases secret
disk mapper decrypts root filesystem
```

这条路径解决的是“离线篡改启动组件后窃取磁盘数据”的一部分风险。它不解决运行中 root 权限滥用，不解决内核漏洞，不解决用户已解锁后的数据泄露。安全硬件必须放在威胁模型中解释。

机制	防护对象	边界
Secure Boot	未授权启动组件	运行中漏洞仍可能攻破系统
TPM sealing	密钥只在匹配测量状态释放	授权状态下恶意软件仍可请求使用
IOMMU	设备 DMA 越权	CPU 内核漏洞不由 IOMMU 防住
内存加密	物理 DRAM 窃听或冷启动部分风险	运行中软件访问仍受权限控制

源码和工具入口包括 `drivers/char/tpm/.security/integrity/ima/.mokutil`、`tpm2_pcrread`、`systemd-cryptenroll`。学习安全硬件时不要只问“开没开”，要问它保护哪条路径、失败时如何恢复。

案例十六：虚拟机直通一块网卡

设备直通把真实 PCIe 设备交给虚拟机使用。hypervisor 要从 host driver 手中解绑设备，绑定到 VFIO，建立 IOMMU domain，把设备 DMA 限制在 guest 内存范围内，并把中断重定向给 guest。

```
device passthrough:
host checks IOMMU group isolation
host binds device to vfio-pci
VM maps guest memory into IOMMU domain
device DMA is translated by IOMMU
MSI/MSI-X interrupts are remapped to guest
guest driver thinks it owns a real PCIe device
```

直通性能好，但隔离依赖 IOMMU group、ACS、interrupt remapping 和设备本身可信度。若同一 group 中有多个不可隔离设备，把其中一个交给 guest 可能让 guest 影响 host 或其他设备。

风险	原因	证据或约束
DMA 越界 中断注入错误 group 不隔离	IOMMU 配置错误或关闭 interrupt remapping 问题 PCIe 拓扑共享请求者 ID	<code>dmesg</code> IOMMU、VFIO 日志 VM 日志、host IRQ 统计 <code>/sys/kernel/iommu_groups</code>
设备 reset 不完整	设备状态泄漏到下个 owner	FLR 支持、驱动 reset 路径

相关入口包括 `drivers/vfio/`、`drivers/iommu/`、`drivers/pci/quirks.c`。这个案例把 PCIe、IOMMU、中断、安全和虚拟化连接在一起。

案例十七：BMC 远程重启一台服务器

服务器 BMC 可以在主机 OS 崩溃时仍然工作。管理员通过管理网络登录 BMC，查看传感器、日志和远程控制台，执行 power cycle。主 CPU 被断电重启，但 BMC 的固件和网络连接可能一直在线。

```
BMC recovery path:
administrator connects over management network
BMC reads platform event log
BMC controls power rails through board logic
host CPU resets and firmware starts again
OS later reads hardware error records
```

BMC 是运维救命工具，也是安全风险。它有独立凭据、独立网络栈、固件更新路径和高权限硬件控制能力。生产环境应隔离管理网，审计 BMC 账户，更新固件，记录远程操作。

案例十八：JTAG、串口和早期调试

当内核还没启动到日志系统，普通调试器和文件系统都不可用。x86-64 硬件调试依赖更低层入口：串口 early console、Intel DCI/JTAG、逻辑分析仪、BMC SOL、panic blink code、firmware debug port。

调试入口	适用阶段	价值
UART early console	bootloader 和内核早期	最简单稳定的文本证据
Intel DCI/JTAG	裸机、固件和早期内核	停 CPU、看寄存器和内存
BMC SOL	服务器远程串口	无需现场接线
logic analyzer	信号级时序问题	查 I2C/SPI/GPIO/复位线
crash dump	内核运行后崩溃	保存内存和寄存器证据

这解释了为什么底层工程重视串口。屏幕和网络依赖太多驱动，串口依赖少，最适合定位启动早期、panic 和设备 probe 问题。

硬件相关 Linux 源码阅读路线

为了避免“工业对标只是概述”，这里给出一个实际阅读顺序。每个路径都建议先读数据结构，再读 probe，再读失败回滚，再读中断或 completion。

主题	源码路径	先抓的对象
启动主线	<code>init/main.c</code> 、 <code>arch/x86/kernel/head64.c</code>	early init、initcall
ACPI	<code>drivers/acpi/</code>	device 描述、电源状态和资源
PCIe	<code>drivers/pci/</code>	<code>pci_dev</code> 、BAR、MSI

USB	<code>drivers/usb/core/</code>	<code>urb</code> 、 <code>endpoint</code> 、 <code>hub event</code>
DMA/IOMMU	<code>kernel/dma/</code> 、 <code>drivers/iommu/</code>	<code>mapping</code> 、 <code>domain</code> 、 <code>fault</code>
块层	<code>block/</code>	<code>bio</code> 、 <code>request</code> 、 <code>blk-mq</code>
NVMe	<code>drivers/nvme/host/</code>	<code>queue</code> 、 <code>command</code> 、 <code>completion</code>
网络	<code>net/core/dev.c</code> 、 <code>drivers/net/</code>	<code>sk_buff</code> 、 <code>NAPI</code> 、 <code>netdev</code>
DRM/KMS	<code>drivers/gpu/drm/</code>	<code>GEM</code> 、 <code>plane</code> 、 <code>CRTC</code> 、 <code>fence</code>
音频	<code>sound/core/</code> 、 <code>sound/pci/hda/</code>	<code>PCM ring</code> 、 <code>codec</code> 、 <code>stream</code>
摄像头	<code>drivers/media/</code>	<code>media graph</code> 、 <code>V4L2 buffer</code>
热管理	<code>drivers/thermal/</code>	<code>thermal zone</code> 、 <code>cooling device</code>
电源管理	<code>drivers/base/power/</code>	<code>runtime PM</code> 、 <code>system sleep</code>
安全芯片	<code>drivers/char/tpm/</code>	<code>command</code> 、 <code>PCR</code> 、 <code>session</code>

读源码要带着硬件问题读。例如读 `pci_driver.probe` 时，问：资源哪里来，MMIO 如何映射，IRQ 如何分配，DMA mask 如何设置，失败时如何释放。读 `NAPI` 时，问：为什么中断处理不直接把所有包处理完，poll budget 如何平衡延迟和吞吐。

硬件测试报告模板

下面这个模板适合本章学习后的最小测试。它不要求写代码，但要求把硬件结论落到证据。

```
Hardware report:
Machine:
  CPU model, core/thread count, cache levels, NUMA nodes
  memory size, memory type if visible, ECC status if visible
Firmware:
  BIOS/UEFI version, ACPI evidence, Secure Boot status
PCIe:
  list NVMe/GPU/NIC path, BAR, driver, IRQ/MSI-X, IOMMU group
Storage:
  device model, queue count, scheduler, flush support, SMART summary
Network:
  link speed, queues, offload, IRQ affinity, error counters
USB/Input:
  topology, HID devices, hotplug evidence
Display/Audio:
  connectors, modes, audio devices, underrun or error counters
Power/Thermal:
  temperature sensors, frequency governor, cooling devices
Security:
  TPM, Secure Boot, IOMMU, kernel lockdown if enabled
Failure case:
  choose one failure and trace hardware -> kernel -> user symptom
```

报告的判断标准：每个结论都要有命令输出或日志证据；每个设备都要能说出驱动对象；每个性能结论都要说明硬件瓶颈可能在哪里；每个安全结论都要说明防护边界。

常见误区集中纠正

- “CPU 访问内存就是读数组”：实际还经过虚拟地址、TLB、页表权限、cache 和异常路径。
- “write 成功就是落盘”：通常只是内核接收，持久化还需要 writeback、flush 和文件系统顺序。
- “中断越多越实时”：过多中断会造成 CPU 浪费和抖动，很多高性能路径会合并中断或轮询。
- “DMA 就是更快复制”：DMA 是设备直接访问内存，关键是所有权、映射、cache 和完成回收。
- “设备插上驱动就该工作”：还需要供电、时钟、复位、总线枚举、固件描述、IRQ、DMA 和权限。
- “虚拟机里的物理地址是真的物理地址”：guest physical 还要经过二级页表到 host physical。
- “安全硬件打开就安全”：安全硬件只保护特定边界，不能替代内核和用户态安全。
- “性能只看算法”：cache、TLB、NUMA、频率、温度、I/O 队列和中断亲和性都可能主导结果。

本节能力验收

读完本节后，不要求记住所有芯片型号，但要能做到以下事情：

- 能从一条 C 内存访问讲到 TLB、页表、cache、store buffer 和缺页。
- 能从电源复位讲到固件、bootloader、内核 early init 和第一个用户进程。
- 能解释定时器中断如何让内核抢回 CPU。
- 能解释 TLB shutdown 为什么是多核虚拟内存的昂贵操作。
- 能解释 malloc 成功和物理页真正分配之间的差别。
- 能给 DMA 提交流程写出正确的所有权和屏障顺序。
- 能完整讲出冷缓存文件读取、可靠文件写入、USB 键盘、网卡收包、GPU 显示、音频播放、摄像头采集的路径。
- 能把 Wi-Fi 不稳、NVMe 超时、黑屏、音频爆音、热降频、设备不可见拆成硬件层、驱动层、内核对象层和用户可见层。
- 能根据 Linux 源码路径找到至少一个硬件子系统的 probe、资源申请、completion 和错误恢复代码。
- 能写出一份本机硬件测试报告，并给每个结论附上命令或日志证据。

第一阶段深讲：设备、总线、DMA 与中断的内核契约

设备是操作系统最容易讲浅的部分。把驱动说成“读写寄存器”，就会漏掉总线枚举、资源分配、DMA 地址、队列所有权、中断亲和性、电源状态、错误恢复和安全隔离。本节把设备看作一个和 CPU 并行运行的状态机：内核通过寄存器和共享内存给它任务，它通过中断和 completion 把结果交回来。

设备不是函数调用对象

函数调用有明确进入和返回。设备请求不是这样。内核提交请求后，设备可能立即完成，也可能稍后完成，也可能超时，也可能在复位后返回错误。请求对象必须在这段时间内保持身份稳定。

```
device request lifecycle:
allocated
initialized
mapped for DMA
visible in submission queue
owned by device
completed or timed out
unmapped
reported to upper layer
released
```

这个生命周期解释了为什么驱动代码到处有引用计数、状态位、超时定时器和完成回调。没有它们，内核无法判断一个 buffer 现在归 CPU 还是设备，无法处理迟到 completion，也无法在进程退出时安全取消请求。

总线枚举决定设备怎样被发现

不同总线的发现方式差别很大。PCIe 设备有配置空间，可以枚举 vendor、device、class 和 BAR；USB 设备有描述符和接口；I2C/SPI 这类板载设备常不能完整自描述，在本书 x86-64 主线里通常需要 ACPI 提供地址、IRQ 和电源资源。OS 的 driver model 就是为这些差异建立统一框架。

总线	发现方式	驱动关心的资源
PCIe	配置空间扫描	BAR、MSI-X、DMA mask、AER
USB	设备和接口描述符	endpoint、URB、电源、热插拔
I2C	固件描述地址和类型	设备地址、中断 GPIO、时钟
SPI	固件描述片选和模式	片选、频率、mode、DMA 能力
platform bus	ACPI 描述	MMIO、IRQ、clock、reset、regulator

一个驱动的 probe 函数不应假设资源永远存在。它要检查 MMIO 映射是否成功，中断是否申请成功，DMA mask 是否能覆盖需要地址，设备固件是否加载成功，电源域是否已经打开。probe 失败要逆序释放已经申请的资源。

MMIO 寄存器是设备协议，不是普通变量

MMIO 读写会产生副作用。读状态寄存器可能清 pending 位；写控制寄存器可能启动设备；写 doorbell 可能让设备读取共享队列。编译器和 CPU 对普通内存的优化不能随便套到 MMIO 上，所以内核使用专门的读写接口。

```
submit queue entry:
sq[tail].addr = dma_addr
sq[tail].len = len
sq[tail].cmd = READ
dma_wmb()
writel(new_tail, device_doorbell)
```

这个顺序有协议含义。设备看到 doorbell 时，必须已经能读到完整描述符。若把 writel 当普通 store，或者缺少 DMA 写屏障，设备可能读到旧内容。驱动 bug 的随机性，很多来自这种硬件可见顺序。

DMA 地址不是用户虚拟地址

设备不能拿用户虚拟地址直接 DMA。用户虚拟地址只对某个进程和 CPU MMU 有意义。设备看到的是总线地址或 IOVA。内核必须把参与 DMA 的页 pin 住或使用内核 buffer，再建立设备可用地址映射。

```
prepare DMA:
  choose buffer pages
  ensure pages remain resident
  map pages for device direction
  get dma_addr visible to device
  program descriptor with dma_addr
```

DMA 方向也很重要。ToDevice 表示 CPU 写好数据，设备读；FromDevice 表示设备写，CPU 之后读；Bidirectional 两者都有。方向决定 cache clean/invalidate 和 IOMMU 权限。方向标错可能表现为旧数据、数据损坏或 IOMMU fault。

IOMMU 给设备建立地址空间

没有 IOMMU，设备拿到总线地址后可能访问很大物理范围。IOMMU 让内核为设备建立受限映射：某个 IOVA 只对应被授权的物理页，方向和权限受控。这样设备或驱动出错时，不至于随意覆盖内核内存。

机制	作用	代价
IOVA	给设备看的地址	需要映射管理和失效
IOTLB	缓存设备侧翻译	unmap 后要处理失效
domain	一组设备的隔离空间	设备分组和直通复杂
DMA remapping	限制设备可访问页	增加映射开销

虚拟化和设备直通尤其依赖 IOMMU。把设备交给虚拟机时，宿主必须确保设备只能访问该虚拟机的内存，而不能 DMA 到宿主或其他虚拟机。

中断处理要分快慢两层

硬件中断入口运行在受限上下文里，不能做太多慢操作。典型设计是 top half 尽快确认设备、取走必要状态、屏蔽或重新使能中断，然后把耗时工作推到 softirq、tasklet、workqueue、threaded IRQ 或 NAPI poll 中。

```
interrupt top half:
  read device interrupt status
  acknowledge or mask interrupt
  record that work is pending
  schedule bottom half
  return from interrupt

bottom half:
  process completion queue
  unmap DMA buffers
  wake waiters
  submit follow-up work if needed
```

分层的原因是延迟和可重入。中断关闭太久会影响定时器和其他设备；在硬中断里睡眠会破坏调度语义；做大量包处理会造成中断风暴。网络 NAPI 就是把高频中断转换为预算受控的轮询，减少每包中断成本。

MSI-X 和中断亲和性连接到多核性能

现代高速设备常有多个队列，每个队列配一个或多个 MSI-X vector。OS 可以把不同队列的中断分配到不同 CPU，让网络包或存储完成更接近处理它们的线程和内存。

```
multi-queue device:
  queue 0 interrupt -> CPU 0
```

```
queue 1 interrupt -> CPU 1
queue 2 interrupt -> CPU 2
queue 3 interrupt -> CPU 3
```

若所有中断都落到 CPU0, CPU0 会成为瓶颈, 其他 CPU 等待数据。若中断在 CPU 和内存 node 之间乱跑, 会增加 cache 和 NUMA 成本。设备队列、IRQ affinity、RSS/RPS、块层 tag 和用户线程绑核, 是同一个性能问题的不同层次。

设备队列把硬件并行度暴露给内核

NVMe、网卡、GPU 等设备都通过队列表达并行能力。队列深度越大, 设备能同时处理的请求越多, 但排队过深也会增加尾延迟。内核要在吞吐和延迟之间控制 in-flight 请求数量。

队列状态	含义	OS 策略
submission queue	CPU 写入设备要做的命令	合并、排序、限流、doorbell
completion queue	设备写入已完成命令	phase tag、轮询、中断
queue depth	同时在飞请求数量	吞吐和尾延迟折中
tag/request id	请求身份	乱序完成、取消、超时处理

设备完成可以乱序返回, 所以 request id 必须稳定。若复用 tag 过早, 旧 completion 可能错误匹配新请求。NVMe completion 的 phase tag 正是为了解决环形队列槽位复用问题。

电源管理是设备状态机的一部分

设备空闲时关闭时钟、电源域或进入低功耗状态可以省电, 但 resume 后寄存器和队列状态可能丢失。驱动必须知道设备从 suspended 到 active 需要哪些步骤。

```
runtime suspend:
  stop new requests
  drain or cancel in-flight requests
  save registers if required
  disable interrupts
  gate clocks and power

runtime resume:
  enable power and clocks
  restore registers and queues
  reenale interrupts
  accept requests again
```

若 suspend 时仍有 DMA, 设备可能写入已经被释放的 buffer。若 resume 后忘记恢复 doorbell 或队列地址, 提交请求会超时。电源管理 bug 常表现为“第一次正常, 睡眠唤醒后失败”。

错误恢复比正常路径更能检验驱动质量

设备可能超时、返回错误 completion、发生 PCIe AER、被用户拔出、固件崩溃或电源丢失。驱动必须把错误传播到上层, 而不是让任务永久睡眠。

```
timeout handler:
  mark request timed out
  stop affected queue
  attempt device reset
  complete request with error or retry
  wake waiters
  unfreeze queue if recovery succeeded
```

错误恢复要特别防止双完成。一个请求超时后，驱动可能已经向上层返回错误；如果设备随后给出迟到 completion，驱动必须识别它属于旧请求，不能再次完成或释放同一个对象。引用计数和 generation id 常用于解决这类问题。

设备安全边界

设备驱动运行在内核里，攻击面很大。用户态通过 ioctl、mmap、网络包、USB 描述符或 GPU 命令间接影响驱动。所有来自设备和用户的数据都必须验证。IOMMU 限制 DMA，权限系统限制谁能打开设备文件，seccomp/LSM 可以继续收紧接口。

攻击面	风险	防线
用户 ioctl 参数	越界、类型混淆、UAF	参数复制、长度检查、对象引用
设备 DMA	覆盖任意内存	IOMMU、DMA API、方向权限
恶意 USB 设备	描述符畸形、热插拔竞态	描述符验证、授权策略
网络包	协议解析漏洞	长度检查、分层解析、fuzz
GPU 命令	访问其他进程显存	IOMMU、命令验证、上下文隔离

这就是为什么“驱动只是硬件适配层”是不准确的。驱动同时是性能路径、权限边界、错误恢复层和硬件状态机管理者。

设备契约的最低验收

第一阶段应能解释：为什么 MMIO 访问需要专门接口；为什么 DMA buffer 在 completion 前不能释放；为什么 IOMMU 不是可有可无的性能组件；为什么中断要分 top half 和 bottom half；为什么多队列设备要考虑中断亲和性；为什么驱动 reset 必须处理所有 in-flight 请求。

能回答这些问题，后面学习块层、网络栈、文件系统和 io_uring 才不会停留在 API 表面。

中断、定时器与内核入口

原理

轮询模型的问题是 CPU 必须不断主动检查外设。中断把方向反过来：外设的事件发生时打断 CPU，CPU 保存当前执行现场，跳到中断向量表中登记的处理函数，处理最少必要工作，然后恢复被打断的代码。中断是操作系统从“一个主循环”走向“多个异步事件”的关键。

中断处理函数必须短。原因很具体：中断期间可能屏蔽同级或低级中断；处理太久会增加其他事件延迟；处理函数如果访问复杂共享状态，会和主循环或其他中断产生竞态。因此常见设计是 top half 只读取硬件状态、清中断、把事件放入队列；bottom half 或普通任务稍后完成复杂处理。

```
interrupt uart_rx_isr() {
    byte b = UART_DATA;
    ring_push(&rx_queue, b);
    UART_CLEAR_INTERRUPT();
}

void uart_task_step() {
    while (ring_pop(&rx_queue, &b)) {
        parse_byte(b);
    }
}
```


中断引入了共享状态竞争。主循环和中断处理函数可能同时访问 `rx_queue`。在单核单片机上，最简单的临界区是短暂关中断，修改队列指针，再开中断。这个方法有效，但必须短；如果关中断期间做慢操作，就会丢事件或增加控制延迟。

```
disable_interrupts();
bool ok = ring_push(&queue, value);
enable_interrupts();
```

这个临界区就是锁的祖先。以后在多核机器上，关本核中断不能阻止另一个核心同时访问同一数据结构，于是需要自旋锁、互斥锁和原子操作。但基本问题没有变：某段代码需要在状态不变量被临时打破时不被并发打断。

中断控制器为什么不是可选部件

若只有一个外设和一个 CPU，外设可以直接把中断线连到 CPU。真实系统有多个外设、多个中断源、多个优先级，甚至多个 CPU。中断控制器负责把混乱的硬件事件整理成 CPU 能处理的 vector：记录 pending 位，应用 mask，比较优先级，选择目标 CPU，并在处理完成后接收 EOI。

```
interrupt_controller:
    pending[source] = device_irq_line
    enabled[source] = software_mask
    priority[source] = configured_priority
    target[source] = cpu_id

deliver():
    candidates = pending & enabled
    source = highest_priority(candidates)
    cpu = target[source]
    cpu.raise_interrupt(vector[source])
```

没有中断控制器，OS 就不能可靠地屏蔽某类中断，不能区分中断来源，不能把高吞吐设备分散到多个 CPU，也不能在多核系统中发送 IPI。中断控制器是“外设事件”和“CPU 异常入口”之间的路由层。

x86-64 中断控制器：local APIC、IOAPIC、MSI/MSI-X

x86-64 主线里，中断控制器不是一个单独芯片名，而是一组协作机制。local APIC 属于每个 CPU，用于本地 timer、IPI 和接收 vector；IOAPIC 把传统外部 IRQ 重定向到目标 CPU 和 vector；MSI/MSI-X 让 PCIe 设备通过内存写投递中断；x2APIC 扩展了 APIC 编址和访问方式，适合大规模多核系统。

机制	主要用途	OS 关心点
local APIC	每 CPU 中断、本地 timer、IPI	多核调度、TLB shutdown、timer tick
IOAPIC	把外部 IRQ 重定向到 CPU/vector	IRQ 路由、触发方式、亲和性
MSI/MSI-X	PCIe 设备通过内存写投递中断	多队列设备、vector 分配、隔离
x2APIC	扩展 APIC 编址和访问方式	大规模 CPU、虚拟化、IPI 成本

MSI/MSI-X 值得单独强调。传统中断像一根线；MSI 是设备向一个特殊地址写入消息来触发中断。这样 PCIe 设备可以为多个队列分配多个 vector，让网卡 RX queue 0 中断送到 CPU0，RX queue 1 中断送到 CPU1。多队列网络和 NVMe 的伸缩性很大程度依赖这种硬件能力。

定时器和时间片

定时器中断是操作系统调度的起点。没有定时器，只有任务主动让出 CPU 时，系统才能切换任务；一旦任务死循环，整个系统就失控。定时器按固定频率打断 CPU，内核在中断中增加 tick、检查超时、唤醒等待任务，并在必要时设置“需要调度”的标志。

```
interrupt timer_isr() {
    ticks++;
    for each timer in timer_wheel[current_slot]:
        if (timer.expired(ticks)) {
            make_task_runnable(timer.task);
        }
    if (current_task.time_slice_used++ >= current_task.time_slice) {
        need_reschedule = true;
    }
}
```

直接扫描所有定时器的复杂度是 $O(n)$ 。如果系统只有几个任务，这很简单；如果有几十万个超时请求，每个 tick 都扫描全表就不可接受。常见算法包括最小堆、时间轮和分层时间轮。最小堆插入和删除是 $O(\log n)$ ，查看最近超时是 $O(1)$ ；时间轮把时间划分成槽，插入通常接近 $O(1)$ ，但精度和最大超时范围受槽设计影响。

定时器从硬件计数器到 OS 超时

硬件定时器最小形态是一个递增或递减计数器，加一个比较寄存器。计数器来自固定频率时钟；当计数器达到比较值时，硬件产生中断。OS 在中断里更新软件时间，并设置下一次比较值。

```
program_next_timer(deadline):
    timer.compare = deadline
    timer.enable_interrupt = true

timer_interrupt():
    now = timer.counter
    run_expired_timers(now)
    next = earliest_deadline()
    program_next_timer(next)
```

周期 tick 的做法是每隔固定时间中断一次，例如 1ms 或 10ms。高精度和 tickless 的做法是按最近事件编程下一次中断。前者实现简单，后者减少无意义中断，但要求内核精确维护最近 deadline。无论哪种方式，抢占和超时都依赖硬件在未来把控制权交回内核。

异常和系统调用

中断来自外设，异常来自当前指令执行，例如除零、非法指令、缺页、权限错误。系统调用也是一种受控入口：用户程序主动执行特定指令，请求进入内核。三者共同构成内核入口。

入口	来源	典型处理
中断	外设或定时器	读取设备状态，唤醒任务，记录 tick
异常	当前指令失败	发送信号、修复缺页、终止进程
系统调用	用户主动请求	检查参数和权限，操作内核对象

从这里开始，OS 的边界清楚了：用户代码不能直接操作所有资源，而是通过受控入口进入内核；内核根据当前上下文、权限和对象状态决定是否允许请求。

异常入口保存的是诊断证据

异常不是“出错就跳到内核”这么简单。硬件必须保存足够证据，让内核判断能否修复。以 page fault 为例，内核需要知道 faulting address、访问类型、当前特

权级、出错指令地址和错误码。以非法指令为例，内核需要知道是哪条指令，以便发送信号或模拟指令。

证据	用途	缺失后果
fault RIP	返回或报告出错指令	无法重试或定位错误
fault address	判断哪个虚拟地址出错	无法处理缺页或 SIGSEGV
access type	读、写、执行、用户/内核	无法区分 COW、NX、权限错误
RFLAGS/CPL	中断状态、特权级、条件位	返回路径不安全
error code/cause	架构提供的分类原因	只能粗暴终止，无法精细恢复

这也是内核崩溃日志要打印寄存器、RIP、栈和 fault address 的原因。它们不是噪声，而是硬件给出的最低层证据。没有这些证据，调试只能猜。

上下文保存

中断、异常和调度都需要保存上下文。x86-64 的最小上下文包括 RIP、RFLAGS、通用寄存器和 RSP。若要切换任务，还要保存当前任务的内核栈、地址空间标识和调度账本。

```
struct Context {
    uint64_t rip;
    uint64_t rsp;
    uint64_t rflags;
    uint64_t regs[16];
};

void switch_to(Task* prev, Task* next) {
    save_context(&prev->ctx);
    current = next;
    restore_context(&next->ctx);
}
```

这段伪代码隐藏了真实汇编细节，但说明了不变量：一个任务被切走时，它的下一条指令位置和栈必须能恢复；一个任务被恢复时，寄存器必须像它从未离开 CPU 一样。上下文保存不正确，任何高级调度策略都没有意义。

上下文分两类：异常上下文和调度上下文

异常上下文由硬件入口和低级入口共同保存，描述“被打断的执行现场”。调度上下文描述“线程被切走后下次如何继续”。二者有关但不完全相同。系统调用和中断进入内核时，需要 trap frame；线程主动睡眠或被调度出去时，需要保存 callee-saved 寄存器、内核栈位置和调度状态。

上下文	保存内容	典型时机
trap frame	用户 RIP、RSP、RFLAGS、系统调用参数、错误码	中断、异常、系统调用入口
switch context	内核栈指针、callee-saved 寄存器、返回地址	scheduler 切换内核线程
地址空间上下文	页表根、PCID、TLB 状态	切换进程地址空间
浮点/SIMD 上下文	FPU、向量寄存器、控制字	用户使用浮点/SIMD 或信号处理

把这几类上下文混在一起会导致很多误解。例如中断打断用户程序时，内核需要 trap frame 才能返回用户态；调度器在两个都已进入内核的线程之间切换时，通

常只需要保存内核执行所需的最小寄存器集合。地址空间切换还要改页表根和处理 TLB。

trap frame、switch context 和 stack frame 的区别

“保存现场”这个说法太粗，会掩盖三种完全不同的结构。trap frame 是硬件入口和汇编入口为异常/中断/系统调用保存的用户或被打断现场；switch context 是调度器在内核线程之间切换时保存的最小恢复信息；stack frame 是普通函数调用在栈上的返回地址、旧帧指针和局部变量布局。

结构	谁创建	用来回答什么问题
trap frame	CPU 硬件入口 + 低级汇编入口	从哪个 RIP/RSP/RFLAGS 返回用户态或被打断点
switch context	调度器和架构切换代码	下次调度到该线程时，从哪个内核位置继续
stack frame	编译器按 ABI 生成的函数调用代码	当前函数返回到哪里，局部变量在哪里
signal frame	内核在用户栈上构造	用户态 signal handler 结束后如何恢复原现场

一个线程在系统调用中睡眠时，通常同时拥有这些结构：用户态曾经有 stack frame；进入内核后有 trap frame；内核 C 函数有自己的 stack frame；调度出去时还要保存 switch context。调度器切换的是“当前内核执行点”，不是直接切换到用户程序入口；返回用户态时才消费 trap frame。

```
user calls read()
-> user stack frame exists
-> syscall creates trap frame
-> kernel functions create kernel stack frames
-> task sleeps, scheduler saves switch context
-> later scheduler restores switch context
-> syscall completes and return path restores trap frame
```

这也解释了为什么每个线程需要独立内核栈。线程睡在内核里时，它的内核 stack frames 和 trap frame 都还活着；另一个线程进入内核不能覆盖这段栈。

算法与实现分析

中断入口的最小状态机

中断入口要做的第一件事不是调用 C 函数，而是保存足够现场，让被打断代码将来能继续。可以把入口写成四段：硬件自动保存、汇编入口补充保存、C 层分发、返回路径恢复。

```
interrupt_entry(vector):
    frame = hardware_saved_frame()
    save_scratch_registers(frame)
    switch_to_kernel_stack_if_needed(frame)
    dispatch_interrupt(vector, frame)
    maybe_preempt_on_return(frame)
    restore_registers(frame)
    iret(frame)
```

这个状态机维护两个不变量。第一，任何会被中断处理函数破坏的寄存器，要么已经保存，要么按 ABI 规定不需要保存。第二，返回用户态之前，内核必须处理挂起信号、抢占标志和可能的调度请求。若在中断中直接切换任务，必须保证旧任务的内核栈仍保存完整 trap frame。

精确中断和延迟中断

外部中断通常在指令边界被接收，CPU 完成当前指令的架构效果后进入中断入口。这样内核返回时可以从下一条指令继续。异常则和当前指令绑定：缺页异常通常要返回到同一条指令重试，除零或非法指令可能发送信号。精确性让 OS 能用同一套保存/恢复机制处理多种入口。

```
external interrupt:
    instruction N committed
    interrupt taken before N+1
    saved RIP = address of N+1

page fault:
    instruction N cannot complete
    saved RIP = address of N
    kernel may fix page and retry N
```

中断延迟来自多个来源：CPU 临时关中断、硬件优先级屏蔽、长时间持有自旋锁、中断控制器路由、cache miss、下半部积压。实时系统关心最大延迟；通用系统也要避免长时间关中断，因为它会让定时器、网络、块 I/O 和 RCU 都延后。

中断控制器与优先级

真实硬件通常通过中断控制器管理多路中断。控制器至少要完成屏蔽、优先级、确认和结束通知。教学模型可以抽象成 pending bitmap：

```
interrupt_controller_tick():
    pending = pending_bits & enabled_bits
    if pending == 0:
        return
    vector = highest_priority(pending)
    pending_bits.clear(vector)
    cpu_raise_interrupt(vector)

end_of_interrupt(vector):
    controller.eoi(vector)
```

若忘记 EOI，控制器可能不再投递后续同类中断；若过早 EOI，同一个设备可能在 handler 尚未处理完共享状态时再次打断。驱动通常先读取设备状态并清设备侧中断源，再向控制器确认完成。顺序错误会导致中断风暴或丢事件。

IRQ、vector、handler 和 completion 的区别

设备 I/O 里还有一组容易混的词。IRQ 是“有中断事件需要处理”的硬件/内核资源；vector 是 CPU 入口编号；handler 是内核注册的处理函数；completion 是某个具体 I/O 请求完成的事实。一个 IRQ 可以对应一个队列，一次 handler 可以处理多个 completion，一个 completion 也可能在轮询模式下被发现而不是靠中断发现。

概念	表示什么	常见误解
IRQ	CPU 被通知有事件	误以为它等于某个请求完成
line/vector		
handler	处理该类事件的函数	误以为 handler 只能处理一个事件
completion entry	设备写回的请求结果	误以为一定伴随一次中断
wait queue	等待某个条件的任务集合	误以为中断直接唤醒某个用户函数
poll budget	一次下半部最多处理多少完成	误以为只要有中断就会处理完所有包/请求

高吞吐设备通常把中断当成“提醒你去看队列”，而不是“这个请求完成了”的一对一消息。网卡可能一次中断后处理几十个 RX completion; NVMe 可能一个 MSI-X vector 对应一个 completion queue; NAPI 甚至会暂时关闭中断，改用轮询批量处理。这样吞吐更好，但延迟、budget、队列深度和背压都必须进入设计。

电平触发、边沿触发和 MSI

中断触发方式影响 handler 的确认顺序。边沿触发在信号变化瞬间产生事件；若软件没及时处理，可能错过后续边沿。电平触发只要设备中断线保持有效，控制器就认为中断仍 pending；若 handler 没清设备状态，EOI 后会立刻再次进入。MSI/MSI-X 则是设备写一个中断消息，通常不依赖共享物理线。

触发方式	特点	handler 重点
边沿触发	变化瞬间产生一次事件	不能丢边沿，事件队列要足够
电平触发	条件保持则持续 pending	必须清设备侧原因再 EOI
MSI/MSI-X	设备写消息触发 vector	vector 分配、队列亲和性、屏蔽同步

很多“中断风暴”来自清除顺序错误或设备状态未消费。OS 不能只从 CPU 侧看中断，还要看设备侧 pending 条件是否被真正清掉。

top half 与 bottom half 的分工

中断上半部只做必须立即完成的工作：读取硬件状态、清中断源、把事件入队、唤醒下半部。下半部可以在软中断、工作队列或普通内核线程中运行。

```

irq_top_half(dev):
    status = mmio_read(dev.status)
    mmio_write(dev.ack, status)
    push_event(dev.event_queue, status)
    schedule_bottom_half(dev)

bottom_half(dev):
    while pop_event(dev.event_queue, &event):
        process_event(event)
        wake_waiters(dev.waitq)

```

分工的理由是延迟控制。上半部越长，其他中断越晚被处理；下半部越慢，队列越容易积压。因此测试不只看功能，还要看最大关中断时间、队列峰值和事件丢失计数。

定时器数据结构选择

若每个 tick 扫描所有定时器，复杂度是 $O(n)$ 。最小堆适合定时器数量中等、过期时间分散的系统；时间轮适合大量短超时；分层时间轮适合同时刻覆盖短超时和长超时。

结构	插入	取最近超时	适用场景
线性表	$O(1)$	$O(n)$	少量定时器，教学 baseline
最小堆	$O(\log n)$	$O(1)$	通用超时，精度清楚
单级时间轮	接近 $O(1)$	接近 $O(1)$	大量短超时，槽宽可接受
分层时间轮	接近 $O(1)$	接近 $O(1)$	长短超时混合

选择定时器结构时要写清精度、最大超时、插入成本和 tick 成本。一个只在 10 个定时器上正确的实现，不能直接推广到 100000 个 socket 超时。

实践

本章实践建议写一个事件日志。每次进入中断、退出中断、唤醒任务、设置 `need_reschedule`，都记录 `tick`、事件类型和当前任务。然后用日志回答：哪个事件打断了谁，哪个任务被唤醒，什么时候真正运行。

```
tick=100 enter timer_isr current=control
tick=100 wake task=telemetry reason=period
tick=100 set need_reschedule
tick=100 exit timer_isr current=control
tick=100 switch control -> telemetry
```

同时实现两个定时器队列版本：全表扫描和最小堆。比较 `n` 从 10 到 100000 时，每个 `tick` 的处理成本。这个实验能说明为什么 OS 算法不能只看功能正确，还要看事件规模。

测试

最低测试清单如下：

- 中断处理函数只做短路径工作，不调用可能阻塞的函数。
- 环形队列在主循环和 ISR 同时访问时不破坏 `head/tail/size`。
- 定时器 `tick` 单调增加，超时任务只被唤醒一次。
- 时间片用尽后设置 `reschedule` 标志，不能在任意位置破坏当前上下文。
- 上下文切换后，任务能从被切走的位置继续执行。
- 异常路径能区分可修复缺页、权限错误和不可恢复错误。

这一章的验收结论是：中断和定时器让系统获得外部事件和抢占能力，但也迫使我们认真处理共享状态、上下文保存和入口权限。

源码级补充：x86 IDT、APIC 与 Linux 中断路径

x86 异常入口保存了什么

x86-64 的中断和异常入口由 IDT 描述。CPU 根据 `vector` 找到 `gate`，切换特权级时切到内核栈，硬件压入返回所需状态。某些异常还会压入 `error code`。汇编入口随后保存通用寄存器，形成内核能理解的寄存器帧。

```
struct TrapFrame {
    // software-saved general registers
    u64 r15, r14, r13, r12, r11, r10, r9, r8;
    u64 rdi, rsi, rbp, rbx, rdx, rcx, rax;

    // hardware-saved frame
    u64 vector;
    u64 error_code;
    u64 rip;
    u64 cs;
    u64 rflags;
    u64 rsp;
    u64 ss;
};
```

真实布局会因入口类型和内核版本变化，但读源码时要抓住语义：`rip/cs/rflags/rsp/ss` 决定如何返回；`error code` 帮助 `page fault` 和保护异常分类；通用寄存器保存系统调用参数和被打断计算状态。

IDT、APIC、IOAPIC 和 MSI

IDT 是 CPU 本地表；APIC/IOAPIC/MSI 解决“设备事件如何送到哪个 CPU 的哪个 vector”。传统外设中断可经 IOAPIC 重定向到本地 APIC；PCIe 设备常用 MSI/MSI-X，通过一次内存写消息触发中断。多核重调度和 TLB shutdown 使用 IPI，由一个 CPU 通过本地 APIC 向另一个 CPU 投递。

```
device event
-> IOAPIC redirection entry or MSI message
-> local APIC on target CPU
-> CPU vector
-> IDT gate
-> low-level entry
-> generic IRQ dispatch
-> device handler
-> EOI
```

中断亲和性影响性能。网卡 RX 队列中断若全部打到 CPU0，CPU0 会成为瓶颈；把队列中断分散到处理对应网络栈的 CPU，可以减少跨核 cache line 迁移。但过度分散也会让连接状态在 CPU 间跳动。真实系统因此有 irq affinity、RSS/RPS/XPS 等策略。

Linux 中断路径阅读点

读 Linux 中断路径可以从这些位置开始：

- arch/x86/kernel/idt.c: IDT gate 初始化。
- arch/x86/entry/entry_64.S: 低级入口、寄存器保存、返回路径。
- kernel/irq/handle.c: 通用 IRQ 分发。
- kernel/irq/manage.c: request_irq、request_threaded_irq。
- kernel/time/timer.c、kernel/time/hrtimer.c: 低精度 timer wheel 与 hrtimer。

阅读时不要只看 handler 函数。要同时看谁注册 handler、handler 运行在何种上下文、是否允许睡眠、谁发送 EOI、错误返回如何统计、设备移除时如何注销并等待 in-flight handler 完成。

四种下半部机制

机制	特点	适用场景
softirq	固定类型，常用于网络和块层，高并发	极热路径，要求不能随意睡眠
tasklet	基于 softirq 的延后执行，同一 tasklet 串行	旧驱动常见，新代码逐步少用
workqueue	在内核线程中执行，可睡眠	需要分配内存、拿 mutex、访问慢设备
threaded IRQ	把 IRQ handler 线程化	降低硬中断时间，适合较复杂设备处理

选择机制的核心问题是上下文：是否需要睡眠，是否要高吞吐，是否允许同一事件并行，是否要限制 CPU 占用。把会睡眠的代码放进 hard IRQ 或 softirq，是典型内核 bug。

hrtimer、timer wheel 与 tickless

低精度超时适合 timer wheel，例如 TCP 重传、普通超时、周期任务；高精度计时适合 hrtimer，例如纳秒级睡眠、音视频定时、精确 profiler。tickless 则在 CPU 空闲或单任务运行时减少周期 tick，降低功耗和噪声。

```

timer wheel:
    cheap insertion, coarse granularity, many ordinary timeouts

hrtimer:
    ordered by expiry in high-resolution time, higher overhead, precise wakeup

tickless:
    program next hardware timer to nearest event instead of fixed periodic tick

```

tickless 不表示没有时间。它表示内核不再无条件固定频率打断 CPU，而是计算下一个必要事件并设置硬件定时器。调度统计、RCU、负载均衡和超时都要适配这种模式。

1.4 从裸机启动到内核契约

端到端案例：把硬件和 OS 串成一条路径

硬件最难学的地方，是每个部件单独看都能理解，合起来就乱。本节用几个端到端案例串起来。每个案例都按同一模板：事件从哪里来，经过哪些硬件，内核维护哪些对象，用户程序看到什么，哪里可能失败。

案例一：按下键盘到终端显示字符

按键不是直接变成 shell 输入。以 USB 键盘为例，键盘把按键状态放进 HID report，USB host controller 按轮询周期取回 report，通过 DMA 写入内存，触发中断或 completion。内核 USB/HID 驱动解析 report，输入子系统产生 key event，终端行规程处理字符，等待 read 的 shell 被唤醒。

```

key press
-> USB HID report
-> host controller DMA
-> USB interrupt/completion
-> HID driver
-> input subsystem
-> tty line discipline
-> wake shell blocked in read()

```

可能失败的地方很多：USB 设备拔出，HID report 格式异常，键盘布局映射不同，tty canonical 模式等待回车，shell 被信号打断，终端缓冲区满。一个简单按键，跨了设备、DMA、中断、驱动、输入子系统、tty 和进程调度。

案例二：读取 NVMe 上的一个文件

用户程序调用 read(fd, buf, len)。如果数据在 page cache 中，内核直接复制到用户 buffer；如果不在，文件系统把文件 offset 转成逻辑块，块层构造 bio/request，NVMe 驱动填 submission queue，设备 DMA 读取数据，completion queue 写回结果并中断，内核把页放入 page cache，再复制给用户。

```

read()
-> fd table
-> file/inode
-> page cache miss
-> filesystem block mapping
-> bio/request
-> NVMe submission queue
-> PCIe DMA
-> completion interrupt
-> page cache filled
-> copy_to_user

```

关键误区：`read` 返回不是设备每次都读盘；`page cache` 命中时不会访问 NVMe。另一个误区：NVMe completion 表示设备完成了这次读请求，不等于后续用户处理一定完成。

案例三：写文件并保证崩溃后存在

用户调用 `write`，数据通常先进 `page cache`，页被标记 `dirty`。后台 `writeback` 或 `fsync` 把数据提交给块层。文件系统可能先写日志，再写数据和元数据。设备内部可能有 `volatile cache`，所以还需要 `flush` 或 `FUA` 约束持久化。

```
write()
-> copy_from_user
-> page cache dirty
-> filesystem transaction
-> block layer request
-> device cache
-> flush/FUA
-> stable media
-> fsync returns
```

若程序只调用 `write` 后崩溃，数据可能还在内存。若调用 `fsync(file)` 但目录项没同步，`rename/create` 的目录更新可能不持久。可靠写入要区分数据、元数据、目录项和 `manifest`。

案例四：网卡收到一个 TCP 包

网卡 RX ring 中预先放了 DMA buffer。包从网线进入 PHY/MAC，网卡把数据 DMA 到 buffer，写 `completion/status`，通过 MSI-X 中断通知 CPU。驱动 NAPI poll 取包，构造 `skb`，经过协议栈 IP/TCP，放入 `socket receive queue`，唤醒阻塞在 `recv` 或 `epoll` 的进程。

```
packet on wire
-> PHY/MAC
-> NIC RX DMA into buffer
-> MSI-X interrupt
-> NAPI poll
-> skb
-> IP/TCP stack
-> socket receive queue
-> wake user process
```

性能问题可能出在 IRQ 全打到一个 CPU、NAPI budget 太小、socket buffer 太小、用户进程调度不及时、NUMA 跨节点、GRO/TSO 配置、包过滤规则。网络不是单纯“协议栈算法”，它强依赖硬件队列和 CPU 拓扑。

案例五：在屏幕上显示一个像素

最简单 framebuffer 模型中，用户或 compositor 写入一块图像 buffer。显示控制器按刷新率 scanout 这块 buffer。为了避免撕裂，系统使用 `double buffering` 或 `page flip`，在 `vblank` 边界切换显示 buffer。现代 GPU 路径还包括渲染命令队列、显存分配、同步 `fence` 和合成器。

```
draw pixel
-> write into render buffer
-> GPU or CPU rendering completes
-> compositor submits page flip
-> display controller switches buffer at vblank
-> monitor receives scanout
```

显示问题可能不是“画图代码错”，而是 `buffer stride`、像素格式、`cache flush`、GPU `fence` 未完成、`vblank` 同步、显示模式或电源状态。OS 图形栈是硬件队列和用户态协议的组合。

案例六：播放一段声音

音频程序把 PCM 数据写入 ring buffer。x86-64 PC 上的 HD-audio 控制器或 USB audio 设备通过 DMA 或主机控制器传输周期性读取 buffer，把样本送到 codec 和 DAC。每播放一个 period，设备产生中断或 completion，内核唤醒音频服务继续填充。若用户进程调度太晚，buffer underrun，就会爆音。

```
audio app
-> userspace audio server
-> kernel ALSA buffer
-> DMA to audio controller
-> DAC
-> speaker
-> period interrupt wakes refill
```

音频强调延迟和稳定周期。更大的 buffer 抗抖动但延迟高；更小 buffer 延迟低但更容易 underrun。调度器、电源管理和中断延迟都会影响音频体验。

案例七：系统睡眠再唤醒

suspend 不是简单暂停 CPU。内核要冻结用户进程，停止设备新 I/O，等待或取消 in-flight 请求，保存设备寄存器，关闭中断源，进入低功耗状态。唤醒时，固件或硬件事件恢复 CPU，内核重新初始化时钟、中断和设备，恢复队列，再解冻进程。

```
suspend:
  freeze userspace
  suspend filesystems/devices
  disable non-wakeup IRQs
  save device state
  enter low power state

resume:
  restore clocks and interrupts
  resume devices
  restart queues
  thaw userspace
```

常见 bug：设备 resume 后寄存器没恢复，DMA 队列丢失，中断未重新打开，时钟源跳变，网络连接超时，USB 设备重新枚举。电源管理是整机状态机，不是单个设备函数。

案例八：CPU 过热导致性能下降

程序运行一段时间后变慢，可能不是算法变了，而是温度升高触发降频。温度传感器报告超过阈值，thermal framework 要求 cpufreq 降低频率，甚至限制 GPU/NPU。风扇、散热片、功耗限制都会影响结果。

```
high workload
-> CPU power rises
-> temperature sensor crosses threshold
-> thermal governor requests lower frequency
-> cpufreq changes P-state
-> benchmark throughput drops
```

性能报告必须记录频率、温度、功耗状态和 governor。否则“优化后变慢”可能只是热状态不同。操作系统性能分析离不开硬件传感器。

案例九：虚拟机里一次磁盘读

guest 里调用 read，guest 文件系统走到虚拟块设备。若使用 virtio-blk，guest driver 把请求放入 virtqueue，触发 VM exit 或通知 host。host vhost/virtio 后端把请求转成宿主文件或块设备 I/O。completion 再反向通知

guest。

```
guest read()
-> guest page cache/block layer
-> virtio queue
-> VM exit or ioeventfd
-> host backend
-> host storage stack
-> completion
-> virtual interrupt to guest
```

这里有两层 OS、两层 page cache、两层调度和可能的二级页表翻译。虚拟化性能问题常来自 VM exit、队列深度、host page cache、NUMA 放置和虚拟中断延迟。

案例十：一次内存错误被 OS 处理

服务器 ECC 检测到可纠正错误，硬件通过机器检查或 EDAC 报告给 OS。内核记录 DIMM、地址、错误计数。若某页错误频繁，内核可能把页下线，迁移数据，避免继续使用。

```
DRAM bit flip
-> ECC corrects
-> memory controller logs syndrome
-> machine check/EDAC reports
-> kernel logs physical address
-> page may be soft-offlined
```

这说明硬件错误不是玄学。高可靠系统要能定位 DIMM、页、进程影响范围，并给运维留下证据。

案例方法：每条路径都写五列

以后分析任何硬件到 OS 的路径，都写五列：

列	要写什么
事件源	用户调用、设备事件、定时器、故障、温度、电源状态
硬件路径	CPU、总线、DMA、中断、cache、IOMMU、设备队列
内核对象	fd、inode、page、skb、request、task、timer、workqueue
用户可见结果	返回值、errno、唤醒、信号、性能变化、日志
失败证据	计数器、trace、dmesg、sysfs、perf、错误码

这样写，硬件概念就不再散。每个设备、每个中断、每个 DMA 都能落到用户可见行为和内核对象上。

本节验收

- 能完整讲出按键到 shell 读到字符的路径。
- 能区分文件读的 page cache 命中和 NVMe 真实 I/O。
- 能说明 write、writeback、flush、fsync 的阶段差异。
- 能讲出网卡 RX ring 到 socket receive queue 的路径。
- 能讲出 framebuffer/GPU page flip 到屏幕显示的路径。
- 能解释 suspend/resume 为什么是整机状态机。
- 能把虚拟机 I/O 拆成 guest、hypervisor、host 三层。
- 能用“五列表”分析一个新硬件案例。

特权机器完整走读：从用户指令到内核返回

现在把前面的硬件拼起来，走读一台带 OS 的机器如何处理用户程序。目标不是背 x86-64 细节，而是建立完整路径：用户指令、异常、中断、系统调用、页表、设备、调度如何互相连接。

机器启动后的两类代码

系统启动后，内核代码运行在特权模式，用户程序运行在非特权模式。它们不是简单“谁更重要”，而是 CPU 对它们执行不同规则。

能力	用户态	内核态
普通算术和分支	可以	可以
访问用户页	可以，受页表限制	可以，但必须小心用户指针
访问内核页	不可以	可以
修改页表根	不可以	可以
开关中断	不可以	可以
访问设备 MMIO	通常不可以	驱动可以
返回任意模式	不可以	通过受控返回路径

用户态的限制不是库函数实现的，而是 CPU 和页表强制的。用户程序绕过 libc 直接写设备地址，也会 fault。

路径一：普通用户指令

用户程序执行普通加法：

```
add rax, rbx
```

硬件动作是取指、译码、读寄存器、ALU 加法、写回。没有进入内核。OS 不参与每一条普通指令，否则程序会慢到不可用。OS 只通过页表、特权级、调度器和中断在边界处介入。

路径二：用户 load 命中

用户程序读取已映射内存：

```
mov rax, qword ptr [rbx]
```

硬件查询 TLB，确认页表权限允许用户读，cache 命中后返回数据。整个过程不进内核。虚拟内存能高效运行，是因为权限检查由硬件快速完成，而不是内核逐条检查。

路径三：用户 load 缺页

若 TLB/page table 发现 PTE 不 present，CPU 进入 page fault。

```
hardware:
    save faulting RIP
    save fault address and error code
    switch to kernel entry

kernel:
    find VMA
    if legal:
        allocate/load page
        install PTE
        return to same instruction
    else:
        deliver SIGSEGV or kill process
```

这条路径说明 page fault 是硬件和 OS 的合作。硬件只知道访问无法完成；OS 判断这是合法懒分配、文件映射、COW，还是非法地址。

路径四：用户写只读 COW 页

`fork` 后，父子进程可共享同一物理页，但 PTE 标为只读。任何一方写入时触发保护异常。

```
write to COW page:
  hardware sees PTE writable = 0
  page fault with write error
  kernel sees VMA allows write and page is COW
  allocate new page
  copy old content
  update PTE writable to new page
  return and retry store
```

没有页表权限位和精确异常，COW 就无法透明实现。用户程序以为自己只是普通写内存，OS 在异常路径完成复制。

路径五：系统调用

用户程序要写文件：

```
write(fd, buf, len)
```

`libc wrapper` 把系统调用号和参数放进寄存器，执行 `syscall` 指令。CPU 切换到内核入口。内核保存寄存器，检查 `fd`、用户 `buffer`、权限和对象状态，然后执行文件路径。

```
sys_write:
  file = current.fd_table[fd]
  check file is writable
  copy bytes from user buffer
  write into page cache or device path
  return bytes_written or -errno
```

这里有两个边界。系统调用返回成功表示内核接受并完成了该调用语义，不一定表示数据持久化。用户 `buffer` 复制成功表示当时那段用户地址可读，不表示用户以后不会修改原 `buffer`。

路径六：定时器中断打断用户程序

用户程序正在运行，定时器到了。

```
hardware:
  finish current instruction boundary
  save user RIP/RFLAGS
  enter timer vector

kernel timer handler:
  account time to current task
  run expired timers
  if time slice exhausted:
    set need_resched

return path:
  if need_resched:
    schedule()
  return to chosen task
```

若调度器选择另一个任务，返回用户态时就不回到原任务，而是恢复新任务的 RIP/RSP/寄存器/页表。用户程序感受到的是“自己某段时间没运行”，不是显式函数调用。

路径七：设备完成中断

磁盘或网卡完成 DMA 后发中断。

```

device:
    writes data/status to memory
    raises MSI-X interrupt

kernel:
    enter IRQ handler
    acknowledge interrupt
    read completion queue
    unmap DMA buffer
    mark request done
    wake waiting task

```

这条路径说明设备和 CPU 是并发的。CPU 提交请求后可能去运行别的任务；设备完成时异步打断某个 CPU；内核把 completion 转换成等待队列唤醒或上层回调。

路径八：返回用户态前的检查

内核不能处理完就随便返回。返回用户态前，要检查信号、抢占、调试、审计、单步、恢复寄存器和返回模式。

```

prepare_return_to_user:
    if signal_pending:
        build signal frame on user stack
    if need_resched:
        schedule
    run syscall exit tracing/audit
    restore user registers
    return with user mode bit

```

返回路径是安全边界。若返回模式错、栈错、寄存器泄露、信号帧构造错，都可能造成崩溃或漏洞。很多内核 entry 代码复杂，是因为它同时处理性能快路径和安全慢路径。

一张总图：OS 介入点

事件	谁先发现	内核做什么
普通加法	不进入内核	不介入
合法内存命中	MMU/TLB/cache	不介入
缺页	MMU	分配/装载页面或报错
权限错误	MMU	COW、信号或杀进程
系统调用	用户执行陷入指令	检查对象和权限，执行请求
定时器	timer hardware	统计时间、调度、超时
设备完成	device/interrupt controller	completion、唤醒、错误处理
非法指令	decoder/privilege check	信号、模拟或杀进程

这张表是后续全书的入口图。OS 不控制每个周期，但它控制关键边界；硬件不懂进程和文件，但它能在边界处把控制权交给内核。

本节验收

- 能说明普通指令为什么不进入内核，系统仍然能被 OS 管理。
- 能画出合法 load、缺页 load、COW store 三条路径的差异。
- 能从 `write(fd, buf, len)` 说到寄存器 ABI、系统调用入口、fd 表、用户复制、page cache。

- 能解释定时器中断如何让当前任务被换出。
- 能解释设备 completion 如何唤醒等待 I/O 的任务。
- 能说明返回用户态前为什么要处理信号、抢占和 tracing。

第一阶段深讲：硬件机制怎样逼出操作系统

前面已经介绍了许多硬件部件。本节不再按“CPU、内存、总线、设备”逐项罗列，而是换一个角度：每增加一种硬件能力，系统会遇到什么新问题，操作系统为什么必须出现相应的对象、策略和算法。这样读者不会把组成原理看成操作系统前面的附录，而能看到二者是一条连续的工程演进线。

核心思想

操作系统的大部分核心对象，都可以从硬件限制中推出来：寄存器和 RIP 推出线程上下文，特权级推出系统调用和内核入口，MMU 推出地址空间，定时器推出抢占调度，DMA 和中断推出驱动队列，cache 和多核推出锁、屏障和 per-CPU 数据。

第一步：只有一段固件时，还没有操作系统

从最小机器开始想象：一颗微控制器，一段烧在 Flash 里的固件，一小片 SRAM，几个 MMIO 寄存器控制 GPIO、串口和定时器。复位后 CPU 从固定地址取指，固件设置栈指针，初始化外设，然后进入主循环。这个系统没有进程、没有文件、没有用户态，也没有“内核”这个边界。

```
reset:
    set stack pointer
    initialize clock and UART
    initialize timer
    while true:
        poll input
        update state
        write output
```

这种系统能工作，是因为它的信任模型极简单：所有代码由同一个固件作者写入，所有代码拥有同样权限，系统只运行一个应用。若固件写错地址，整机崩溃；若循环太久，外设响应变慢；若中断处理函数和主循环同时改同一个变量，就可能出现竞态。但这些问题还没有被抽象成“操作系统问题”，因为没有多个互不信任程序需要隔离。

此时最重要的硬件概念是复位、时钟、栈、MMIO 和中断。它们已经预告了后来的 OS 机制：复位和启动链会演进成 bootloader 与内核初始化；栈会演进成每线程内核栈；MMIO 会演进成驱动资源；中断会演进成异常入口、调度时钟和设备完成路径。

第二步：中断让控制流不再只有一条

轮询主循环的问题是响应时间受循环长度限制。若主循环正在处理一段耗时计算，串口输入可能溢出，定时事件可能延迟。中断的出现解决了这个问题：硬件事件可以打断当前执行，把 CPU 引到预先登记的入口函数。

```
main loop running:
    instruction N
    timer interrupt becomes pending
    CPU saves minimal interrupted state
    CPU jumps to interrupt vector
    handler runs
    CPU restores state and returns
```

这里已经出现了“上下文”的雏形。中断入口必须保存被打断代码需要继续执行的状态，例如 RIP、RFLAGS 和通用寄存器。保存不完整，主程序返回后就会计算错误。保存太多，延迟增加。内核后来的上下文切换，本质上是把这种“保存和恢复执行现场”从中断扩展到多个线程之间。

中断还引入了并发的第一个版本。即使只有一个 CPU，主循环和中断处理函数也会交错访问共享变量。若主循环正在更新一个多字节计数器，中断在中间读取，就可能看到半更新状态。最早的临界区可以简单到关中断：

```
disable_interrupts()
shared_counter = shared_counter + 1
enable_interrupts()
```

但这个方法只能保护单核上的中断交错。进入多核以后，另一个 CPU 不受本核关中断影响，必须使用锁和原子操作。理解这一点，才能明白为什么“关中断”和“自旋锁”是不同层次的互斥手段。

第三步：定时器把协作式程序推向抢占式系统

如果所有代码都愿意主动让出 CPU，系统可以用协作式调度：任务运行一会儿，调用 `yield`，再让别人运行。这在早期小系统或事件循环里可行，但一旦某个任务死循环，整个系统就失去响应。硬件定时器把 OS 从“相信任务会合作”推进到“内核可以强制收回 CPU”。

```
timer interrupt:
    save current register frame
    account current runtime
    if current consumed its time slice:
        mark current runnable
        choose next runnable task
        switch stack and registers
    return to selected task
```

这段路径逼出了三个对象。第一是 `task`：内核必须有地方保存每个执行流的寄存器、栈、状态和调度账本。第二是 `runqueue`：内核必须知道哪些 `task` 现在可运行。第三是 `scheduler`：当定时器或阻塞事件改变状态时，内核必须选择下一个 `task`。

定时器同时也定义了时间的可信来源。没有稳定时钟，睡眠超时、网络重传、调度统计、文件时间戳和性能测量都会失真。硬件定时器不是小外设，它是多任务 OS 的节拍器。

第四步：特权级把“库函数”变成“内核入口”

若所有程序都同权，任何程序都可以关中断、改页表、访问磁盘控制器、读写别人的内存。为了运行多个互不信任程序，CPU 必须提供至少两种执行模式：受限的用户态和完全控制硬件的内核态。用户态代码遇到需要特权的操作时，不能直接做，只能通过受控入口请求内核代做。

```
user process wants to read file:
    place syscall number and arguments in registers
    execute syscall/trap instruction
    CPU switches to privileged mode
    CPU jumps to kernel entry
    kernel validates fd, buffer and permission
    kernel performs operation
    CPU returns to user mode with result
```

系统调用的本质是特权边界切换，不是普通函数调用。普通函数调用信任调用者传来的指针，系统调用不能信任。用户给内核一个地址，内核必须确认它属于用户地址空间、权限合适、长度不会溢出，并且复制过程中要能处理缺页。若内核直接解引用恶意用户指针，用户程序就可能诱导内核读写任意地址。

这一步还解释了为什么 ABI 很重要。系统调用入口要约定号码放在哪个寄存器，

参数放在哪些寄存器，返回值和错误码如何表达，哪些寄存器会被破坏。特权边界不是只靠“进入内核”四个字，它是一套硬件状态、寄存器约定、栈切换、权限检查和返回路径共同组成的协议。

第五步：MMU 把地址从数字变成权限对象

没有 MMU 时，地址就是物理地址。程序写地址 0x1000，就真的写那块物理存储。多个程序共享机器时，这显然不够：一个程序的指针错误可以覆盖另一个程序或内核。MMU 引入虚拟地址到物理地址的翻译，并在翻译时检查权限。

```
CPU executes load/store:
    virtual_address = base + offset
    lookup TLB
    if miss, walk page table
    check present, user/supervisor, read/write/execute
    produce physical address or raise page fault
```

这一步把地址空间推出来了。进程不再只是“正在运行的代码”，而拥有一张自己的地址翻译表。相同虚拟地址在不同进程里可以指向不同物理页；内核页可以映射为 supervisor only；代码页可以 executable but not writable；栈页可以 writable but not executable。

MMU 也把异常变成正常控制流。访问不存在的页不一定是错误，可能是懒分配；写只读 COW 页不一定是非法，可能是 fork 后第一次写；访问文件映射缺页可能需要从磁盘读入。page fault handler 是 OS 和硬件合作的典型路径：硬件发现指令无法按当前翻译完成，内核检查 VMA 和 PTE，修复状态后让同一条指令重试。

第六步：cache 和 TLB 让“正确顺序”变得昂贵

cache 和 TLB 是为了速度出现的。没有 cache，CPU 大量时间等待内存；没有 TLB，每次访存都要走多级页表。它们的副作用是状态被复制到了多个地方：内存里有页表，TLB 里有翻译缓存；DRAM 里有数据，多个 CPU cache 里也可能有同一 cache line。

当内核修改页表权限时，只改内存中的 PTE 不够，旧 TLB 项可能还在 CPU 里。当驱动准备 DMA 描述符时，只把字段写到 cache line 不够，设备必须在 doorbell 后能看到正确内容。当两个 CPU 修改同一共享变量时，只看源代码顺序不够，硬件可能通过 store buffer、失效队列和乱序执行改变可见时间。

```
page permission downgrade:
    update PTE in memory
    flush local TLB entry
    send shutdown IPI to other CPUs using this address space
    wait until remote CPUs acknowledge
    only then free or reuse the physical page
```

这一步逼出了 TLB shutdown、内存屏障、cache flush、DMA 同步、原子操作和锁。很多 OS 难点并不在“怎样表达一个状态”，而在“怎样让所有硬件观察者在正确时刻看到同一个状态”。

第七步：总线和设备把内核推向驱动模型

只要 CPU、内存和一个串口，系统还能写死设备地址。但真实机器有许多设备，设备可以热插拔，地址窗口由固件或总线枚举分配，中断可以有多个 vector，DMA 地址受设备能力和 IOMMU 限制。内核必须把硬件资源抽象成驱动模型：发现设备、匹配驱动、申请资源、初始化设备、处理中断、提交请求、回收资源。

```
driver probe:
    identify device
    map MMIO BAR or register range
    allocate IRQ/MSI-X vectors
    set DMA mask and create DMA mappings
    initialize submission and completion queues
```



```
register block/net/input/char device object
```

驱动模型的关键是生命周期管理，远不止“会写寄存器”。probe 成功后，设备对象对上层可见；remove 时必须阻止新请求、等待或取消旧请求、释放 IRQ、unmap DMA、销毁队列、关闭电源和时钟。若错误路径少释放一步，就泄漏资源；多释放一步，就可能 use-after-free。

第八步：DMA 和 completion 把同步调用变成异步协议

PIO 时代，CPU 可以在一个函数里读完设备数据。DMA 时代，CPU 只提交描述符，设备稍后完成。于是一次 I/O 被拆成两个时间点：提交和完成。中间这段时间里，请求对象、buffer、DMA 映射和等待者都必须保持有效。

```
submit read request:
    pin or allocate buffer pages
    map pages for DMA_FROM_DEVICE
    fill device descriptor
    ring doorbell
    put task to sleep or return EAGAIN

completion interrupt:
    read completion queue
    unmap DMA
    mark buffer uptodate
    wake task or post event
```

这个异步协议解释了为什么内核里到处都有 request、bio、skb、urb、kiocb、fence、completion、wait queue。它们都是在保存“已经提交但还没完成”的状态。同步系统调用只是表面：read 可以在内部提交异步 I/O，然后让当前线程睡眠；设备完成时，中断路径唤醒它，系统调用才返回。

第九步：多核把一个内核变成分布式状态机

单核内核可以用“当前 CPU 正在做什么”理解很多事情。多核内核不行。多个 CPU 同时进入内核，同时修改不同对象，甚至同时修改同一对象。每 CPU runqueue、per-CPU 计数器、本地中断、远程 IPI、RCU grace period、跨 CPU TLB shutdown，都是多核带来的结构。

多核问题	简单做法为什么不够	OS 机制
全局 runqueue 锁竞争	所有 CPU 抢同一把锁	每 CPU runqueue + 负载均衡
全局引用计数抖动	cache line 在 CPU 间来回迁移	per-CPU 计数、批量汇总
读多写少数据 页表失效传播	普通锁让读者互相阻塞 远端 CPU 仍有旧 TLB	RCU、seqlock、读写锁 IPI shutdown、PCID
NUMA 远端访问	内存延迟不均匀	CPU affinity、页迁移、 first touch

多核系统里，性能 bug 常常是共享状态设计错误。一个看似无害的全局计数器，在 128 核机器上可能变成所有 CPU 争用的 cache line。一个在单核上正确的“关中断保护”，在多核上不能保护另一个 CPU。OS 设计必须从一开始就问：这个状态属于全局、每 CPU、每进程、每设备，还是每请求？

第十步：文件和 socket 是硬件差异的收敛结果

从硬件看，磁盘、网卡、管道、终端和定时器完全不同。从用户态看，它们可以都变成 fd。这个统一来自 OS 在中间建立的对象层，而不是硬件本身相似：fd 表、open file description、inode/socket/device file、ops 函数表、等待队列和 readiness 事件共同支撑了统一接口。

```

user read(fd):
  fd table maps integer to struct file
  struct file points to object and operations
  operation may hit page cache, pipe buffer, socket receive queue, or device
  if data unavailable:
    task sleeps on object's wait queue
  completion path wakes waiters

```

统一接口有巨大价值：shell 重定向、管道、epoll、权限检查、引用计数、close-on-exec 都可以围绕 fd 建立。但统一接口不能抹掉语义差异。普通文件 read 返回 EOF，socket read 受协议影响，pipe 受容量和对端关闭影响，timerfd 返回计数，eventfd 返回计数器。教材若只说“文件是一切”，就会掩盖这些关键差异。

从硬件机制到 OS 对象的映射

下面这张表是第一阶段的压缩地图。后续章节每讲一个 OS 机制，都应能回到这里找到硬件来源。

硬件机制	直接问题	OS 对象或算法
RIP、寄存器、栈	执行现场如何保存恢复	thread context、kernel stack、context switch
中断向量	异步事件如何进入可信代码	IDT/vector table、IRQ handler、softirq
定时器	不合作任务如何被抢占	scheduler tick、hrtimer、time accounting
特权级	不可信程序如何请求特权操作	syscall、trap frame、copy_from_user
MMU/PTE	地址如何隔离和授权	address space、VMA、page fault、COW
TLB	翻译缓存如何保持一致	shutdown、PCID、lazy TLB
cache/coherence	多核共享数据如何可见	locks、atomics、barriers、RCU
MMIO	CPU 如何控制设备	ioremap、readl/writel、regmap
DMA	设备如何直接访问内存	DMA mapping、IOMMU、buffer lifetime
MSI/MSI-X	设备如何报告完成	IRQ affinity、completion queue、NAPI
PCIe/USB/I2C	设备如何被发现和匹配	bus model、driver probe、resources
DRAM/NUMA	内存距离和可靠性不同	page allocator、NUMA policy、memory error handling
持久存储	写入何时算持久	page cache、writeback、journal、fsync

第一阶段读者应该形成的判断力

到这里，不要求读者已经能写完整内核，但应该能做出一组判断。看到系统调用，要想到特权级、寄存器 ABI、用户指针检查和返回路径。看到 page fault，要想到 MMU、VMA、PTE、TLB、COW 和缺页重试。看到设备驱动，要想到 MMIO、DMA、IRQ、队列、屏障和错误路径。看到多核性能问题，要想到 cache line、锁粒度、per-CPU、IPI 和 NUMA。

这就是第一阶段的目标：不把硬件当背景，不把 OS 当概念，而是把二者合成一条可追踪的执行链。

第一阶段全链路案例：从硬件事件走到内核对象

上一节说明硬件机制如何逼出操作系统对象。本节继续用长案例把这些对象串起来。每个案例都尽量避免只给结论，而是按事件推进：硬件先发生什么，CPU 看到什么，内核保存什么状态，用户态最后观察到什么。这样写比背诵名词慢，但更接近真实调试。

案例一：一条系统调用从用户态走到设备

考虑用户程序执行：

```
read(fd, buffer, 4096)
```

若 `fd` 指向一个冷缓存普通文件，这一行会跨过几乎所有 OS 层次。用户态先把系统调用号、`fd`、`buffer` 地址、长度放进约定寄存器，执行 `syscall` 指令。CPU 切换到内核态，进入系统调用入口，内核保存用户寄存器形成 `trap frame`。

```
user registers:
  syscall number = read
  arg0 = fd
  arg1 = user buffer pointer
  arg2 = length
```

```
CPU transition:
  switch privilege to kernel
  jump to syscall entry
  use kernel stack
  save user register frame
```

内核不能相信任何参数。`fd` 可能无效；`buffer` 可能指向未映射地址；长度可能过大；文件可能不可读；调用可能被信号打断。检查通过后，`fd` 表把小整数映射到 `struct file`，`struct file` 再指向 `inode`、文件偏移和操作函数。若 `page cache` 命中，内核只需从缓存页复制到用户 `buffer`。冷缓存时，路径继续下行。

```
fd table -> struct file
struct file -> inode and file operations
page cache lookup(file, index)
if miss:
  filesystem maps logical file block to disk block
  block layer creates request
  driver submits DMA command to device
```

设备请求提交后，当前线程没有数据可返回。同步 `read` 的表现是“阻塞”，但内核内部已经变成异步：`request` 在设备队列里，`task` 在等待队列里，`buffer` 或 `page cache folio` 等待 DMA 完成。CPU 可以调度别的任务运行。

```
current task:
  state = sleeping
  wait_reason = I/O
  wait_queue = folio waiters or request waiters

device:
  command descriptor visible through DMA
  completion will arrive by interrupt or polling
```

设备完成 DMA 后，写 `completion queue` 或状态寄存器，触发 MSI-X 中断。CPU 进入中断入口，驱动读取 `completion`，解除 DMA 映射，标记 `folio uptodate`，唤

醒等待任务。任务重新变成 `runnable`，调度器稍后选中它，系统调用继续执行，把页缓存中的字节复制到用户 `buffer`，更新文件 `offset`，返回读取字节数。

层次	关键对象	失败或延迟来源
用户态 ABI	<code>syscall number</code> 、参数寄存器	参数错误、信号打断
fd 层 VFS/page cache	<code>fd table</code> 、 <code>struct file</code> <code>inode</code> 、 <code>folio</code> 、 <code>xarray</code>	<code>EBADF</code> 、权限不匹配 <code>cache miss</code> 、 <code>folio lock</code> 等待
文件系统 块层	<code>extent</code> 、 <code>block mapping</code> <code>bio</code> 、 <code>request</code> 、 <code>queue</code>	元数据 I/O、碎片 队列拥塞、调度延迟
驱动/设备	<code>DMA descriptor</code> 、 <code>completion</code>	超时、IRQ 丢失、设备错误
返回用户态	<code>copy_to_user</code> 、 <code>offset</code>	用户页缺页、短读

这个案例最重要的结论是：一次 `read` 不是“内核读磁盘”。它是用户态 ABI、权限、页缓存、文件系统映射、块层队列、DMA、中断、调度和用户内存复制的组合。定位慢读时也必须逐层排查：是系统调用太多，`page cache miss`，文件系统碎片，块队列拥塞，设备慢，还是用户 `buffer` 缺页。

案例二：一次写入为什么不能保证掉电后存在

再看：

```
write(fd, data, len)
```

普通 `buffered write` 往往只把用户数据复制进 `page cache`，把 `folio` 标记为 `dirty`，然后返回。它没有承诺设备已经写入，更没有承诺掉电后还能读到。这个语义来自硬件速度差异和批量 I/O：如果每次小写都同步落盘，吞吐会极低，存储寿命也会受影响。

```
buffered write:
  validate fd and user buffer
  find or allocate page cache folio
  copy bytes from user to folio
  mark folio dirty
  update inode size if needed
  return len
```

`dirty folio` 稍后由后台 `writeback` 或显式 `fsync` 推进到块层。若文件系统有日志，还要先记录元数据意图，再写 `home location`。若设备有 `volatile cache`，还要 `flush` 或使用 `FUA`。若目录项也改变，例如安全替换文件时使用 `rename`，父目录也要 `fsync`。

```
durable replace:
  write temp file data
  fsync temp file
  rename temp to target
  fsync parent directory
```

这个模式看似重复，实际保护的是不同对象。第一个 `fsync` 保护文件数据和 `inode` 必要元数据；`rename` 改变目录项；第二个 `fsync` 保护目录项持久化。若省略父目录同步，崩溃后可能出现文件数据已经在盘上，但目录项没有指向它。

硬件层也有顺序问题。存储设备可能为了性能重排写入；文件系统日志必须用 `barrier/flush` 保证“日志提交记录”不会早于日志内容持久化。否则恢复时看见 `commit`，却读不到完整日志，崩溃一致性就被破坏。

案例三：一次 page fault 如何从错误变成分配

用户程序访问一个刚 mmap 出来的匿名页：

```
char* p = mmap(..., PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_ANONYMOUS, ...);
p[0] = 1;
```

mmap 成功时，内核通常只建立 VMA，未必立刻分配物理页。第一次写入时，CPU 查 TLB 失败，走页表发现 PTE 不 present，于是产生 page fault。硬件把 fault address、访问类型、当前 RIP 和错误码交给内核入口。

```
page fault handler:
  read fault address and error code
  find VMA covering address
  verify write permission
  allocate zeroed physical page
  install writable PTE
  flush or update TLB as needed
  return to faulting instruction
```

这不是“程序崩溃”。只要 VMA 合法且权限允许，内核可以修复页表，然后让同一条指令重试。重试时 TLB 能找到新 PTE，store 成功。懒分配让大虚拟地址空间变得便宜，也让实际物理内存按需使用。

COW fault 类似但更微妙。fork 后父子共享同一物理页，PTE 都是 read-only + COW。任一方写入时，page fault handler 发现这是 COW 写 fault：若页仍共享，就分配新页复制内容，再把写入方 PTE 改成 writable；若页已经独占，可以直接恢复 writable。

fault 类型	内核动作	用户态结果
匿名懒分配	分配零页并安装 PTE	指令重试成功
COW 写入	复制共享页或恢复写权限	父子内容分离
文件映射缺页	从 page cache 或存储读入	访问到文件内容
权限错误	生成信号或异常	SIGSEGV 等失败
内核地址访问	拒绝用户访问	进程被终止或返回错误

理解 page fault 后，许多现象会变得自然：虚拟内存大于物理内存不奇怪，malloc 后首次访问慢不奇怪，fork 很快也不奇怪。真正需要警惕的是 fault 路径持锁、I/O、OOM 和 TLB shutdown 造成的长尾延迟。

案例四：一次网络包如何变成 socket 可读

网卡收到以太网帧时，通常通过 DMA 把数据写入预先准备好的 receive ring buffer。设备更新 completion 或 descriptor 状态，触发中断或等待 NAPI 轮询。驱动把 buffer 包装成 skb，交给网络协议栈。

```
NIC receive:
  packet arrives on wire
  NIC chooses RX descriptor
  DMA writes packet into memory
  NIC reports completion
  driver builds skb
  protocol stack parses Ethernet/IP/TCP
  TCP queues payload to socket receive queue
  wake tasks or epoll waiters
```

这条路径有两个队列层次。硬件有 RX ring，决定设备把包放到哪里；内核有 socket receive queue，决定应用何时能读到数据。中间还有 GRO、校验和卸载、路由、Netfilter、TCP 重排、流控和拥塞控制。应用调用 recv 看到的是字节流，但内核背后处理的是包、段、序列号和状态机。

为什么高性能网络程序常用 epoll? 因为 readiness 和阻塞状态要被集中管理。若 socket receive queue 为空, `recv` 在阻塞模式下会睡眠, 在非阻塞模式下返回 `EAGAIN`。epoll 把“哪些 fd 可能可读”组织成事件集合, 避免一个线程为大量连接逐个阻塞。

阶段	关键状态	常见故障
RX ring	descriptor、DMA address、buffer ownership	ring 满、buffer 泄漏、DMA 错误
中断/NAPI	budget、poll 状态、IRQ affinity	中断风暴、软中断延迟
协议栈	skb、路由、TCP 状态	丢包、乱序、重传
socket	receive queue、wait queue	backlog 满、应用读取慢
用户态	epoll interest、非阻塞循环	忙等、短读、背压缺失

这个案例说明网络不是独立于 OS 的应用层话题。它和 DMA、cache、NUMA、中断亲和性、调度延迟、内存分配都有关系。一个网络服务 p99 延迟升高, 可能是 TCP 重传, 也可能是 softirq 被抢占, 也可能是 RX buffer 分配慢, 还可能是用户态没有及时 drain socket。

案例五：一次进程切换到底切了哪些硬件状态

调度器选择下一个任务只是开始。真正的上下文切换要处理寄存器、栈、地址空间、FPU/SIMD 状态、调试寄存器、性能计数、CPU 本地状态和调度统计。不同架构细节不同, 但核心问题相同: 哪些状态是任务私有的, 哪些状态是 CPU 私有的, 哪些状态可以延迟保存。

```
context switch:
  save callee-saved registers of prev
  save prev kernel stack pointer
  choose next kernel stack
  switch address space if mm differs
  handle lazy or eager FPU state
  update current task pointer
  restore next registers
  return into next task's previous kernel path
```

地址空间切换可能影响 TLB。若 x86-64 启用 PCID, 旧地址空间的 TLB 项可以带标签保留; 若没有标签或必须失效, 切换成本更高。FPU/SIMD 状态很大, 早期系统常用 lazy save, 但安全性和实现复杂性让许多系统转向 eager 策略。调度器的“公平”算法只是上层, 底层切换成本会反过来影响线程数量和时间片选择。

状态	为什么要处理	优化或风险
通用寄存器	任务恢复后继续计算	保存集合由 ABI 和入口路径决定
内核栈	每个任务有自己的内核执行现场	栈溢出会破坏内核对象
页表根/PCID	不同进程地址空间不同	PCID 降低 TLB 刷新
FPU/SIMD	用户可见寄存器很多	lazy 策略复杂, eager 成本固定
分支/cache 状态	微架构状态影响性能和安全	侧信道缓解可能增加成本

因此“线程很轻量”只在相对进程创建、地址空间复制而言成立。大量 runnable 线程仍会带来调度排队、cache 污染、TLB 压力和栈内存占用。OS 教材必须把调度算法和硬件切换成本一起讲, 否则容易误导读者以为线程只是一个软件列表节点。

案例六：一次设备复位为什么会影响所有等待者

设备可能超时、固件崩溃、PCIe 链路错误或队列失去响应。驱动不能只打印错误然后继续提交请求。可靠复位要把设备状态机和内核请求状态机一起处理。

```
device reset:
  stop accepting new requests
  mark queues frozen
  collect in-flight requests
  disable interrupts or mask vectors
  reset hardware
  rebuild rings and registers
  decide old requests: retry, fail, or complete
  unfreeze queues
  wake all affected waiters
```

这个过程解释了为什么驱动需要 request id、引用计数、超时计时器、错误码和 completion 路径。若复位时漏掉某个 in-flight 请求，等待它的进程会永久睡眠；若复位后旧 completion 迟到，可能错误完成新请求；若直接释放 DMA buffer，设备可能在复位前后继续写内存。

设备复位也是“硬件不可靠”进入 OS 设计的例子。操作系统不能假设设备总按协议完成。它必须有超时、取消、错误传播和恢复策略。高质量驱动的难点往往不在正常提交，而在这些异常路径。

案例七：一次安全边界检查跨越了哪些硬件

用户程序打开文件，看起来是路径权限问题，但硬件也参与了安全边界。用户态无法直接读磁盘，是因为没有设备 MMIO 权限；无法修改内核文件缓存，是因为页表隔离；无法伪造系统调用结果，是因为返回路径由内核控制；无法直接访问别的进程地址，是因为 MMU 和特权级共同限制。

```
open(path, flags):
  syscall enters privileged mode
  kernel copies path from user memory safely
  VFS resolves path under namespace and mount rules
  DAC/capability/LSM checks decide permission
  file object is installed into fd table
  user receives only an integer handle
```

fd 是安全能力的一种形式：用户态不能直接拿 inode 指针，只能拿内核发放的整数句柄。后续 read/write/ioctl 都要通过这个句柄回到内核检查对象和操作。若 fd 泄漏到子进程，权限也随之泄漏，所以 close-on-exec 是安全边界的一部分。

边界	硬件基础	OS 策略
用户/内核	特权级、页表 U/S 位	syscall、copy_from_user、KASLR
进程/进程	地址空间、PCID/TLB	VMA、PTE、COW、信号权限
进程/文件	不能直接访问设备和缓存	fd、inode 权限、LSM
设备/内存	IOMMU、DMA 权限	dma_map/unmap、隔离域
容器/宿主	namespace 仍共享内核	capability、seccomp、cgroup、LSM

这说明安全不是一个章节里的附加功能，而是硬件和内核对象从一开始就共同维护的边界。后续学习 namespace、seccomp、LSM 时，应把它们看成在基础硬件边界上继续收紧权限；它们不能凭空创造隔离。

第一阶段验收：能沿路径说清楚，而不是背定义

完成第一阶段后，读者应该能把下面几条路径讲完整：

- 从复位向量到内核第一个 C 函数：哪些硬件已可用，哪些还不可用。
- 从系统调用入口到返回用户态：寄存器、栈、特权级和用户指针如何处理。
- 从虚拟地址访问到物理页：VMA、PTE、TLB、page fault 如何合作。
- 从设备 DMA 到用户 read 返回：描述符、completion、中断、等待队列如何合作。
- 从定时器中断到任务切换：调度账本、runqueue、上下文和地址空间如何合作。
- 从 write 返回到 fsync 持久：page cache、writeback、日志、flush 如何合作。

如果只能背出 CPU、DMA、TLB 的定义，却不能沿这些路径说明状态变化，那还是浅层理解。第一阶段的目标就是把定义变成可追踪的状态机。

第一阶段深讲：CPU 对操作系统暴露的契约

CPU 不是操作系统可以随意想象的执行器。它给软件暴露的是一组精确契约：指令如何改变寄存器，异常在什么边界发生，特权模式如何切换，页表根在哪里，系统调用从哪里进入，返回用户态前哪些状态必须恢复。理解这些契约，比背一张 CPU 结构图更重要。

寄存器不是临时变量，而是架构状态

软件能直接观察和依赖的 CPU 状态称为架构状态。通用寄存器、程序计数器、栈指针、状态寄存器、控制寄存器、浮点和向量寄存器，都属于这个范围。微架构内部的乱序窗口、重命名寄存器、分支预测表通常不可直接观察，但它们会影响性能和安全。

状态	对普通程序的意义	对 OS 的意义
RIP	下一条或当前指令位置	异常返回、信号、调试、上下文切换
RSP	当前栈顶	用户栈、内核栈、ABI 对齐、栈溢出
通用寄存器	参数、返回值、临时计算	syscall ABI、trap frame、调度保存恢复
RFLAGS	条件码、中断使能、模式位	中断开关、返回用户态、单步调试
页表根寄存器	当前地址空间	进程切换、内核映射、TLB 语义
向量寄存器	SIMD/浮点计算	信号帧、延迟保存、上下文切换成本

一条用户指令被中断打断后，程序将来还要像没发生过中断一样继续运行。为此，内核不能只保存 RIP，还要保存足够恢复用户可见状态的寄存器。x86-64 ABI 和入口代码共同定义保存集合；凡是用户态能依赖的状态，内核要么保存恢复，要么在 ABI 中明确它会被破坏。

指令执行不是一句“取指执行”

最小模型可以写成取指、译码、执行、访存、写回。真实 CPU 可能乱序、预测和并行，但最终必须提交出与 ISA 一致的结果。OS 关心的是每条指令在异常和中断面前的边界。

```
instruction retirement view:
  fetch bytes from instruction address
  decode according to ISA
  read architectural operands
```

```
execute operation or memory access
if fault, report precise exception
otherwise commit architectural result
```

“提交”这个概念很关键。乱序 CPU 可以提前执行后面的指令，但不能让用户态看到不符合顺序语义的架构状态。若一条 load 触发 page fault，内核必须知道 faulting instruction 是哪一条，之前的指令已经生效，之后的指令不能对架构状态产生不可回滚影响。否则 page fault handler 无法修复状态并重试。

精确异常是虚拟内存的地基

精确异常的意思是，异常发生时 CPU 能给出一个像顺序执行一样的状态点：异常指令之前的指令已经完成，异常指令和之后的指令没有提交最终架构效果。缺页、除零、非法指令、断点、系统调用，都依赖这个性质。

```
if instruction at RIP faults:
    saved_rip = RIP of faulting instruction
    saved_error = reason and access type
    architectural state corresponds to before this instruction commits
    kernel may fix the cause and return to saved_rip
```

没有精确异常，COW 就很难实现。写 COW 页的 store 必须在真正写入共享物理页之前停下来，让内核复制新页并更新 PTE，再回到同一条 store。若 store 已经部分写入旧页，父子隔离就被破坏。硬件异常语义直接决定 OS 能实现哪些内存抽象。

fault、trap、interrupt 的差别

中文里常把异常、中断、陷入混着说，但写内核时必须分清事件来源和返回语义。

类型	来源	典型例子	返回位置
fault	当前指令无法完成	page fault、保护错误	常返回同一条指令重试
trap	当前指令主动进入内核或调试	syscall、breakpoint	常返回下一条指令
interrupt	外部异步事件	timer、device IRQ、IPI	返回被打断位置
abort	严重硬件或不可恢复错误	machine check、总线错误	可能无法安全返回

这个分类影响内核入口保存什么状态。fault 要保留足够信息让指令重试；syscall 要按 ABI 把返回值写回指定寄存器；interrupt 要避免破坏被打断上下文；严重硬件错误可能只能记录日志、隔离页或 panic。

用户栈和内核栈为什么要分开

用户栈由用户程序控制，可能被破坏、未映射、接近栈底，甚至故意指向恶意地址。内核进入特权态后不能继续信任用户栈，否则中断帧、返回地址和内核局部变量都会落在不可信内存里。因此系统调用、中断和异常入口通常切换到内核栈。

```
on privilege transition:
    CPU or entry code chooses kernel stack for current task
    save user RIP, RSP, RFLAGS and selected registers
    run kernel handler on trusted stack
    restore or validate state before returning to user mode
```

每个线程需要自己的内核栈，因为线程可能在系统调用中阻塞。若所有线程共享一个内核栈，线程 A 阻塞在内核里后，线程 B 进入内核就会覆盖 A 的调用链。内核栈大小有限，所以内核代码不能随意放巨大局部数组，也不能无限递归。

系统调用入口是一段极谨慎的汇编

系统调用入口处，C 运行环境还不完整。入口代码要处理用户寄存器、切换栈、建立内核可用的调用约定、屏蔽或允许中断、检查系统调用号、进入 C 函数分发。返回路径同样危险：返回用户态前要处理信号、调度、审计、单步调试和可能的抢占。

```
syscall entry outline:
  save user register frame
  switch to trusted kernel stack if needed
  sanitize user-provided state
  dispatch syscall table entry
  store return value in ABI register
  check pending signal or reschedule flag
  restore user state
  return to user mode
```

这里不能有“差不多”。若入口漏保存一个用户可见寄存器，程序会随机坏；若没有清理某些特权状态，用户态可能继承内核状态；若返回路径错误恢复 RFLAGS，用户程序可能以错误中断状态或错误单步状态运行。

上下文切换不是只保存寄存器

最浅的说法是“上下文切换保存现场”。更准确地说，它要切换当前 CPU 正在代表哪个任务执行。这个切换至少涉及调度实体、内核栈、寄存器保存区、地址空间、抢占计数、FPU 状态和 CPU 本地 current 指针。

```
switch from prev to next:
  prev.thread.rsp = current RSP
  save callee-saved registers into prev thread context
  current = next
  load next address-space root if mm changes
  switch RSP to next.thread.rsp
  restore next registers
  return as if next's old schedule call just finished
```

注意最后一句：上下文切换不是“跳到 next 的 main 函数”。next 之前可能正在系统调用中睡眠，也可能刚被定时器抢占。恢复后，它从自己上次离开 CPU 的内核位置继续运行，最终回到用户态或继续内核逻辑。

FPU 和 SIMD 为什么让切换变贵

现代向量寄存器很大。若每次切换都保存所有向量状态，成本高；若延迟保存，又要处理任务第一次使用 FPU/SIMD 时的异常、跨任务泄漏和安全补丁。许多系统在性能和安全之间选择 eager 策略，即每次切换都明确保存恢复用户可见向量状态。

策略	好处	风险或成本
lazy save	不使用 FPU 的任务不付成本	状态管理复杂，历史上暴露过安全问题
eager save	语义清晰，隔离更直接	每次切换固定成本增加
按需扩展状态	只为使用的扩展保存空间	需要精确追踪任务使用的 CPU 特性

这解释了为什么 CPU 新增向量扩展不只是编译器问题。OS 要知道如何启用、保存、恢复、暴露给用户、在信号帧中表示，并在不支持时给出正确非法指令异常。

控制寄存器把硬件策略交给内核

页表根、向量表基址、系统调用入口、定时器配置、中断屏蔽、cache 策略、性能计数器控制，这些状态通常只能在内核或更高特权层修改。它们让 OS 能建立策略，也让错误配置变得危险。

```
privileged configuration examples:
```

```

set page table root for current address space
install exception vector table
configure syscall entry point
program local timer deadline
enable or disable selected interrupts
configure performance monitoring counters

```

普通用户程序不能直接改这些寄存器。若能改页表根，它可以访问任意内存；若能改异常向量，它可以劫持系统调用；若能关闭定时器中断，它可以阻止抢占。特权级的价值，就体现在这些寄存器的写权限被硬件强制限制。

x86-64 特权状态逐项拆开

很多读者卡在这里，是因为书上只说“控制寄存器”“中断表”“系统调用入口”，却没有说这些名字各自保存什么。下面这张表把本册反复出现的 x86-64 特权状态拆开。它们不是一堆需要背的缩写，而是内核启动、系统调用、缺页、中断和调度必须配置的硬件根。

硬件状态	保存什么	OS 依赖它做什么
CR0/CR4/EFER	分页、保护、长模式和扩展控制位	打开 long mode、启用分页、启用 PCID 等能力
CR2	最近一次 page fault 的线性地址	page fault handler 查 VMA/PTE 的起点
CR3	顶级页表物理地址和 PCID 相关位	切换地址空间，建立当前 CPU 的翻译根
RFLAGS.IF	当前 CPU 是否接受可屏蔽中断	临界入口、抢占、中断返回语义
IDTR/IDT	vector 到 gate/handler 的表	page fault、timer IRQ、IPI、device IRQ 分发
GDTR/GDT/TSS	段描述、TSS、RSP0、IST	从 Ring 3 进 Ring 0 时选择可信栈
IA32_LSTAR	syscall 的 64 位入口 RIP	用户态只能跳到内核指定入口
IA32_STAR/IA32_FMASK	syscall 段选择和 RFLAGS mask	进入内核时约束返回状态和标志位
IA32_KERNEL_GS_BASE	内核 GS base 备用值	swaps 后找到 per-CPU/current 数据
local APIC registers	本地定时器、IPI、EOI、pending 状态	抢占 tick、跨 CPU 调度、TLB shutdown

把这些状态放在一起看，系统调用就不再神秘。用户执行 `syscall` 时，CPU 用 MSR 找入口；入口代码用 `swaps` 找到当前 CPU 的内核数据；用 TSS/per-CPU 数据切到可信内核栈；用 IDT 处理异步中断和异常；用 CR3/PCID 决定当前地址空间；用 RFLAGS 控制返回时的中断和调试状态。少配置任意一项，内核入口都会从“受控边界”变成不可诊断崩溃。

CR3、PCID 和地址空间对象不是一回事

源码里常见 `mm_struct`、页表页、CR3、PCID、TLB 这些词。它们在同一条路径上，但不是同一个对象。`mm_struct` 是内核管理的地址空间对象；页表页是内存里的多级表；CR3 是 CPU 当前使用的顶级页表根；PCID 是 TLB 项的短标签；TLB 是 CPU 内部缓存的翻译结果。

名字	所在位置	不能误解成什么
<code>mm_struct</code>	内核内存	不是硬件直接读取的寄存器

PML4/页表页	物理内存	不是进程的全部内存，只是翻译结构
CR3	CPU 控制寄存器	不是普通指针，写它会改变当前翻译根
PCID	TLB 标签/CR3 相关位	不是 pid，也不是永久身份
TLB entry	CPU 内部缓存	不是页表本体，PTE 改了它不会自动消失

切换地址空间时，内核的目标不是“复制进程内存”，而是让当前 CPU 后续的取指和 load/store 使用 next 的翻译根。简化算法如下：

```
switch_address_space(prev_mm, next_mm):
    if prev_mm == next_mm:
        keep CR3 and TLB state
        return

    if next_mm has valid PCID that was not recycled unsafely:
        load CR3 with next_mm.pml4_phys and next_mm.pcid
    else:
        pcid = allocate_or_recycle_pcid()
        invalidate stale TLB entries for pcid
        load CR3 with next_mm.pml4_phys and pcid

    ensure kernel global mappings remain valid
```

这里三个容易出错的点。第一，PCID 只是有限标签，复用前必须处理旧翻译，否则新地址空间可能命中旧地址空间的 TLB 项。第二，PTE 权限收紧后，即使 CR3 没变，也要失效对应 TLB 项。第三，释放物理页前必须确认所有可能持有旧翻译的 CPU 都失效完成，否则旧 TLB 可能写到已经分配给别人的页。

IDT、TSS 和 IST：入口为什么需要可信栈

异常和中断不是普通 `call handler`。CPU 收到 vector 后查 IDT，确认 gate 类型、目标特权级和 handler RIP。若事件从 Ring 3 进入 Ring 0，CPU 需要把栈切到内核可信栈；x86-64 用 TSS 中的 RSP0 或 IST 项提供这些栈。IST 专门给 NMI、double fault 等高风险入口使用，避免当前内核栈已经损坏时继续往坏栈上写。

```
event delivery:
    vector = fault_vector or irq_vector
    gate = IDT[vector]
    if CPL changes from Ring 3 to Ring 0:
        RSP = TSS.RSP0 for current CPU
    if gate selects IST:
        RSP = TSS.IST[gate.ist_index]
    push return state and optional error code
    RIP = gate.handler
```

入口	关键硬件证据	内核第一步要做什么
page fault	vector、error code、CR2、RIP	保存寄存器，查 VMA/PTE，判断能否修复
general protection	vector、error code、RIP	判断是用户违规还是内核 bug
invalid opcode	RIP 和指令字节	发送信号、模拟指令或报告内核错误
timer IRQ	APIC vector、被打断 RIP	记账，设置 need_resched，必要时调度

IPI	APIC vector、发送方协议	执行 reschedule、TLB invalidate 或 stop CPU
device IRQ	MSI-X vector、设备状态寄存器	读取 completion，确认中断，唤醒等待者

这解释了为什么入口汇编通常先保存通用寄存器，再把 `rsp` 作为参数传给 C 函数。C 代码不能假设普通调用约定已经完整建立；它接手的是一份由硬件和汇编共同构造的 `trap frame`。

```
page_fault_entry sketch:
  swapgs if entering from user mode
  push general purpose registers
  mov rdi, rsp
  call do_page_fault
  restore general purpose registers
  iretq
```

syscall 入口到底做了哪些事

x86-64 的 `syscall` 快，但它并不替内核完成所有入口工作。它把用户返回 RIP 放到 `rcx`，把用户 `RFLAGS` 放到 `r11`，把 RIP 改为 `IA32_LSTAR` 指定的内核入口，并按 `IA32_FMASK` 清理部分标志位。它不会自动保存所有通用寄存器，也不会自动复制用户参数，更不会自动验证用户指针。

```
; user side, Intel syntax
mov rax, SYS_read
mov rdi, fd
mov rsi, buf
mov rdx, len
syscall
```

进入内核后，低级入口必须把硬件给出的最小状态扩展成内核能处理的状态：

```
syscall entry sketch:
  swapgs
  save user RSP into per-CPU or task state
  load trusted kernel RSP
  save rcx as user RIP
  save r11 as user RFLAGS
  save syscall arguments from registers
  call syscall dispatcher
  prepare return value in rax
  handle signal, tracing, audit or reschedule
  return with sysretq or iretq
```

`sysretq` 和 `iretq` 的区别也来自硬件契约。`sysretq` 快，但要求返回状态满足更严格条件；`iretq` 通用，能恢复更完整的状态，适合信号、调试、异常返回或复杂标志位场景。内核返回路径不是简单 `return`，而是在判断“当前保存的用户状态能不能走快速出口”。

典型故障：硬件名词错配会怎样崩

下面这些故障比定义更能说明名词边界。

错误	直接后果	可观察证据
CR3 指向未映射或损坏页表	取指或访存立即 fault，可能递归 fault	早期 boot 崩溃、page fault 日志异常
TSS.RSP0 错误	Ring 3 进 Ring 0 时 trap frame 写到坏栈	double fault、triple fault、无日志重启

IDT gate 未安装	对应异常或 IRQ 无入口	unhandled vector、系统挂死
IA32_LSTAR 错误	<code>syscall</code> 跳到错误 RIP	用户一调用系统调用就崩溃
忘记 TLB invalidate	权限收紧或 unmap 后旧访问仍成功	随机内存破坏、安全绕过
中断返回恢复 RFLAGS 错误	中断状态或调试状态错乱	丢中断、单步调试异常、抢占异常
DMA completion 前释放页	设备迟到写破坏新对象	IOMMU fault、KASAN、随机数据损坏

读源码时不要问“这个字段叫什么”就停住，而要继续问：谁写它，硬件什么时候读它，错误时进入哪个 vector，能否恢复，日志里有什么证据。这样才算把组成原理学成 OS 能用的判断力。

源码阅读锚点

为了避免只停在概念，本节对应的 Linux 源码可从下面入口读。路径会随版本变化，但职责长期稳定。

源码入口	重点看什么
<code>arch/x86/entry/entry_64.S</code>	<code>syscall</code> 、 <code>interrupt</code> 、 <code>exception</code> 入口如何保存和恢复状态
<code>arch/x86/kernel/cpu/common.c</code>	GDT、TSS、CPU 特性和 per-CPU 初始化
<code>arch/x86/mm/tlb.c</code>	PCID、TLB flush、shutdown 和 lazy TLB
<code>arch/x86/kernel/apic/arch/x86/mm/fault.c</code>	local APIC、IPI、timer、EOI 和中断投递 page fault 如何读取 CR2、解析 error code、进入内存管理
<code>kernel/sched/core.c</code>	调度器如何把 <code>need_resched</code> 变成上下文切换

x86-64 名字与共同原则

本册不再把多个 ISA 混在一起讲。x86-64 的名字就是本书后续源码和汇编的默认语言：RIP/RSP/RFLAGS 表示执行现场，CR3 表示当前页表根，IDT 表示中断和异常入口，`syscall` 表示用户进入内核，Ring 3/Ring 0 表示用户态和内核态。重要的是把这些名字背后的角色讲清：有用户态和特权态，有异常入口，有地址翻译，有中断，有返回用户态的受控路径，有保存恢复上下文的要求。

问题	x86-64 名字	OS 角色
用户进入内核	<code>syscall</code>	受控系统调用入口
特权层级	Ring 3/Ring 0	用户态和内核态隔离
页表根	CR3	当前地址空间翻译根
异常入口	IDT	中断、异常和 trap 的分发入口
返回用户态	<code>sysretq/iretq</code>	恢复用户 RIP/RSP/RFLAGS 和 CPL

学习 OS 时要把“名字”和“角色”分开。CR3 不是普通变量，它是地址空间切换的硬件根；IDT 不是函数表，它是异常和中断的受控入口；`syscall` 不是函数调用，它是跨特权级入口。

CPU 契约的最低验收

读完本节，应该能回答以下问题：为什么 `page fault` 可以返回同一条指令重试；为什么系统调用不能使用用户栈保存内核状态；为什么上下文切换要考虑地址空间和向量寄存器；为什么 `syscall`、IDT、TSS、CR3、PCID 和 APIC 是同一条内核入口链上的不同硬件支点；为什么特权寄存器必须由硬件限制写入。

如果这些问题能回答清楚，进程、线程、系统调用和虚拟内存就不再是孤立概念，而是 CPU 契约自然推出的软件结构。

第一阶段源码走读：x86-64 系统调用入口

第一阶段不能只说“系统调用进入内核”。真正要过关，必须能把一条 `syscall` 指令映射到 x86-64 寄存器约定、低级入口汇编、`pt_regs`、C 层分发、用户指针复制、错误码和返回路径。这样后面讲 `open`、`read`、`mmap`、`exec`、信号和 `seccomp` 时，读者知道每个对象为什么能被内核检查，而不是把系统调用当成神秘函数表。

核心理念

x86-64 `syscall` 入口是 OS 最核心的安全边界之一。它不是“跳到一个 C 函数”，而是先由 CPU 和入口汇编建立可信执行环境，再由 C 层根据系统调用号、寄存器参数、当前任务、`seccomp`、`tracing`、权限和用户指针决定能否推进。

先固定源码入口和读法

读 Linux 时先看这些路径。不同内核版本函数名会有细微变化，但对象和边界基本稳定。

源码路径	先看什么	读法
<code>arch/x86/entry/entry_64.S</code>	<code>entry_SYSCALL_64</code> 、返回用户态路径	看寄存器怎样保存、栈怎样切换、何时走慢路径
<code>arch/x86/entry/common.c</code>	<code>do_syscall_64</code> 、 <code>exit work</code>	看系统调用号、 <code>seccomp</code> 、 <code>tracing</code> 、信号、调度检查
<code>arch/x86/entry/syscalls/syscall_64.tbl</code>	系统调用号表	看 ABI 如何把数字绑定到实现名
<code>include/linux/syscalls.h</code>	<code>SYSCALL_DEFINE*</code> 声明	看 C 实现怎样接入统一表项
<code>arch/x86/include/asm/ptrace.h</code>	<code>struct pt_regs</code>	看 <code>trap frame</code> 在 C 层怎样表示
<code>lib/usercopy.c</code> 、 <code>arch/x86/lib/usercopy_64.c</code>	<code>usercopy</code>	看用户指针复制怎样和异常修复配合

读入口代码时不要从第一行汇编硬啃。先带着四个问题读：用户态把什么放进寄存器，CPU 自动改了什么，入口汇编补保存了什么，C 层分发前还有哪些安全工作。

用户态 ABI：参数不是压到栈上

x86-64 Linux 系统调用 ABI 把系统调用号放在 `rax`，最多六个参数放在 `rdi/rsi/rdx/r10/r8/r9`。注意第四个参数用 `r10`，不是 C 函数调用 ABI 的 `rcx`。原因是 `syscall` 指令会用 `rcx` 保存用户返回 RIP，用 `r11` 保存用户 RFLAGS。

寄存器	<code>syscall</code> ABI 含义	为什么重要
<code>rax</code>	系统调用号，返回时放返回值	调度到哪个 <code>sys_*</code> 实现
<code>rdi</code>	第 1 参数	例如 <code>write(fd,...)</code> 的 <code>fd</code>

rsi	第 2 参数	例如用户 buffer 指针
rdx	第 3 参数	例如长度
r10	第 4 参数	rcx 被硬件占用，不能放参数
r8/r9	第 5/6 参数	六参数系统调用使用
rcx	用户返回 RIP	sysretq 快速返回需要
r11	用户 RFLAGS	返回时恢复用户可见标志

这张表解释了为什么系统调用 wrapper 不是普通 C 调用。libc 或内联汇编必须把参数移动到这些寄存器，再执行 `syscall`。如果你在调试器里看系统调用入口，先看寄存器，而不是看用户栈。

CPU 做了什么，入口汇编又补了什么

`syscall` 指令本身只做一部分特权转移：保存返回 RIP 到 `rcx`，保存 RFLAGS 到 `r11`，把 RIP 改成内核配置的入口地址，按 MSR 配置更新特权相关状态。它不会自动保存所有通用寄存器，也不会替你验证用户参数。

```
before syscall:
    rax = syscall number
    rdi/rsi/rdx/r10/r8/r9 = arguments
    rip = address of syscall instruction
    rsp = user stack

after CPU syscall transition:
    rcx = user return rip
    r11 = user rflags
    rip = IA32_LSTAR entry address
    execution enters ring 0 entry path
```

随后 `entry_64.S` 的入口代码补齐内核需要的上下文：处理 `swapgs`，找到当前 CPU 的内核数据，切换到当前线程的内核栈，按约定把寄存器保存成 `pt_regs` 形状，再调用 C 层分发函数。

```
entry_SYSCALL_64 outline:
    swapgs or equivalent per-cpu base transition
    save user rsp
    load current task kernel stack
    push user ss, user rsp, rflags, user cs, user rip style frame
    save general-purpose registers into pt_regs
    call do_syscall_64(regs, nr)
```

这里的关键不是背汇编每一行，而是理解入口汇编在构造一个 C 层可读的事实包：这个任务从哪里来，用户 RIP/RSP/RFLAGS 是什么，参数寄存器是什么，返回时要恢复到哪里。

pt_regs：低级入口交给 C 代码的证据包

`pt_regs` 可以理解为“这次陷入的证据包”。系统调用、page fault、中断和调试异常都会需要类似对象，只是字段含义和错误码不同。系统调用路径尤其关心参数寄存器、系统调用号、用户返回 RIP、用户栈和返回值位置。

pt_regs 类字段	来源	用途
通用寄存器	用户态执行 <code>syscall</code> 前的寄存器	提取参数、保存用户可见状态
orig_ax	原始系统调用号	tracing/seccomp/restart <code>syscall</code> 需要原号
ip	用户返回 RIP	信号、ptrace、返回路径验证
sp	用户 RSP	信号帧构造和调试输出

flags	用户 RFLAGS	单步、方向标志、中断相关状态
ax	返回值位置	C 层结果写回给用户态

入口代码漏保存字段，后面不是“少打印一个值”，而是可能破坏调试、信号、系统调用重启、seccomp、ptrace 和返回用户态。第一阶段读源码时要形成这个判断：低级结构体字段是安全边界的一部分。

C 层分发：不只是查表调用

C 层分发通常可以抽象成下面的流程，但真实路径还要处理 tracing、audit、seccomp、ptrace、系统调用重启、信号和返回用户态前的 work。

```
do_syscall_64(regs, nr):
    if nr is outside syscall table:
        regs.ax = -ENOSYS
        return
    if syscall_enter_work is pending:
        run tracing/audit/seccomp hooks
        nr may be changed or rejected
    result = syscall_table[nr](regs arguments)
    regs.ax = result
    if syscall_exit_work is pending:
        run tracing/audit/signal/resched checks
```

这说明系统调用号表只是中间一步。比如 seccomp 可能在真正执行前拒绝某个系统调用；ptrace 可能观察或改写参数；audit 可能记录事件；信号可能导致阻塞系统调用返回 `-EINTR` 或被重启。把系统调用理解成“数组下标调函数”会漏掉安全和可观测性。

以 `write(fd, buf, len)` 追一条完整路径

用户态调用 `write(1, buf, len)` 时，硬件入口只知道三个整数：fd、用户地址、长度。它不知道这是终端、文件、pipe 还是 socket。内核必须逐层把整数还原成受保护对象。

```
write(fd, user_buf, len):
    fd table lookup -> struct file
    check file was opened for writing
    copy or pin user buffer under user access rules
    dispatch file->f_op->write_iter
    update file offset or stream queue
    return bytes written or negative errno
```

如果 fd 无效，返回 `-EBADF`。如果用户 buffer 不可读，返回 `-EFAULT`。如果对象暂时不能写且是非阻塞 fd，返回 `-EAGAIN`。如果信号打断等待，可能返回 `-EINTR`。这些错误不是装饰，它们分别来自 fd 表、用户地址空间、对象等待语义和信号状态。

用户指针复制：fault 可以是正常错误路径

用户指针的特殊性在于：它来自不可信地址空间，但内核必须按请求访问它。直接解引用用户指针会把用户错误变成内核错误。正确路径是 `usercopy`：访问前后有范围、权限、异常修复和对象生命周期约束。

```
copy_from_user kernel idea:
    while bytes remain:
        try load from user address
        if page fault happens at a usercopy instruction:
            jump to exception-table fixup
            return bytes not copied or -EFAULT
        store into kernel-owned buffer
```


异常表的价值在这里体现出来：同样是 page fault，如果 faulting RIP 在 usercopy 的异常表范围内，它是用户输入错误；如果发生在普通内核指针解引用上，它更可能是内核 bug。第一阶段讲 fault 时必须把这两类区分开。

返回路径：快返回和慢返回

系统调用返回不是简单 `return result`。内核要决定能否走快速 `sysretq` 风格路径，还是必须走更通用的 `iretq` 风格路径。若用户返回 RIP/RFLAGS、tracing、信号、单步调试、抢占或兼容状态需要特殊处理，就可能走慢路径。

返回条件	快路径倾向	慢路径原因
普通系统调用，无 pending work	可快速恢复寄存器并返回	无
有信号待投递	不适合直接快返	需要构造用户信号帧
需要调度	不适合直接快返	返回前要切换任务
ptrace/seccomp/audit work	不适合直接快返	需要通知观察者或改写结果
返回状态不满足快速指令限制	走通用返回	<code>iretq</code> 能表达更完整状态恢复

这也是为什么返回路径经常比入口更容易出安全问题。入口时内核取得控制权；返回时内核把控制权交回给不可信程序，必须保证所有用户可见状态都被正确恢复，内核私密状态没有泄漏。

观察证据：用真实系统校准理解

原理卷不要求读者修改内核，但要求能把系统调用入口的解释和系统证据对上。

命令或文件	能观察什么	如何判读
<code>strace -e trace=write ./a.out</code>	用户态发起的系统调用和 <code>errno</code>	只能看到边界结果，看不到内核内部等待
<code>perf trace -e syscalls sys_enter_*</code>	<code>syscall enter/exit</code> tracepoint	能看到系统调用号、参数和返回时序
<code>grep syscall /proc/kallsyms</code>	内核符号是否可见	受权限和 <code>kallsyms</code> 设置影响
<code>cat /proc/<pid>/syscall</code>	线程当前 <code>syscall</code> 和参数	只在任务正在系统调用中时有意义
<code>bpftrace tracepoint syscall tracepoint</code>	<code>syscall tracepoint</code> 采样	可按 <code>pid</code> 、 <code>comm</code> 、 <code>errno</code> 过滤

用这些证据时要保持边界感。`strace` 看到 `write(1,...)=5`，只能证明用户/内核边界返回了 5；它不能证明数据已经落盘，也不能说明底层是否发生 page fault、锁等待、设备 I/O 或信号处理。要解释更深路径，需要结合 `perf`、`ftrace`、`eBPF`、`/proc/<pid>/stack` 和具体子系统日志。

第一阶段验收题

读完本节，应能不看书回答这些问题。

1. 为什么 x86-64 Linux 第四个系统调用参数放在 `r10` 而不是 `rcx`？
2. `rcx` 和 `r11` 在 `syscall` 入口中分别保存什么？
3. 为什么入口汇编要构造 `pt_regs`，而不是直接调用 C 函数？
4. `do_syscall_64` 查表前后为什么还有 `seccomp`、`tracing`、`audit` 和 `signal work`？

5. `copy_from_user` 遇到 page fault 为什么通常返回 `EFAULT` 而不是让内核崩溃？
6. `sysretq` 和 `iretq` 都能回用户态，为什么还要区分快路径和慢路径？
7. `strace` 能证明什么，不能证明什么？

这些问题能答清，系统调用就不再是“用户调用内核函数”，而是一条从 x86-64 硬件入口、低级汇编、C 层分发、对象权限、用户内存和返回路径共同组成的受控边界。

第一阶段补课：为什么还不能算完成

前面的第一章已经放进了大量硬件知识，但“知识多”不等于“第一阶段完成”。第一阶段的完成标准不是能背出 CPU、DMA、IOMMU、TLB、PCIe、APIC 这些词，而是能把一个硬件事实推导成一个 OS 设计，再把这个设计落到源码对象、可运行模型、失败案例和观测证据上。若读者看完仍然不知道“为什么必须有这个机制”，第一阶段就没有完成。

第一阶段真正要回答的六个问题

问题	必须能解释到什么程度	不合格表现
程序为什么会跑	RIP、取指、译码、寄存器、内存读写、异常	只说 CPU 执行指令
OS 为什么能抢回 CPU	定时器中断、trap frame、内核栈、调度入口	只说调度器切换进程
用户程序为什么不能乱改机器地址	特权级、控制寄存器、页权限、MMIO 保护	只说内核权限更高
地址为什么会 fault	VMA/PTE/TLB、权限、present、COW、缺页上下文	只说内存不够或空指针
设备为什么不是普通变量	MMIO 副作用、DMA 所有权、中断 completion、屏障	只说读写设备寄存器
启动为什么有严格顺序	复位向量、固件、DRAM、页表、异常入口、驱动 probe	只说 bootloader 加载内核

这六个问题是第一阶段的硬标准。后面的进程、文件系统、网络、安全，都要回到这些硬件事实。如果这里没有过关，后面章节会变成一堆互不相连的规则。

从单片机到通用 OS 的因果链

很多教材直接从“进程、线程、虚拟内存”开始，读者就会觉得 OS 是人为规定出来的。更好的方式是从最小单片机一步一步推到通用 OS。

阶段	当前能力	下一个机制为什么出现
裸循环	一个 <code>while(1)</code> 控制所有设备	等待设备会浪费 CPU，需要中断
中断驱动裸机	设备能异步通知 CPU	多个任务仍要合作让出 CPU，需要定时器抢占
抢占式内核	定时器可打断死循环	任务互不信任，需要特权级和内存保护
多程序系统	用户态不能直接控制硬件	程序要请求资源，需要系统调用 ABI
虚拟内存系统	地址空间隔离和懒分配	频繁拷贝太贵，需要 COW、mmap、page cache

多核系统	多任务真正并行	共享状态会竞争，需要锁、原子、IPI、per-CPU
高速 I/O 系统	设备通过 DMA 批量搬运数据	生命周期复杂，需要队列、completion、IOMMU

注意这个链条里每一步都来自失败场景：轮询浪费 CPU，协作任务可能不让出 CPU，单一地址空间会互相破坏，直接设备访问会越权，PIO 搬数据太慢，多核共享变量会乱序和竞争。OS 不是先验地复杂，而是每个硬件失败场景都迫使它增加一层结构。

一条完整路径：键盘字符到用户进程

用一个小事件把第一阶段串起来：用户按下键盘，终端程序读到一个字符。

1. 键盘控制器或 USB 主机控制器收到物理事件。
2. 设备把事件放入自己的 FIFO、端点或 DMA buffer。
3. 设备通过中断控制器投递 IRQ 或 MSI。
4. CPU 在指令边界响应中断，硬件保存返回状态，进入内核入口。
5. 内核中断处理读取设备状态，确认事件，必要时把重活交给线程化中断或 workqueue。
6. 驱动把 scancode 或 HID report 转成输入事件，写入内核缓冲区。
7. 等待在 `read`、`poll` 或 `epoll` 上的进程被唤醒。
8. 调度器在某次调度点选择该进程运行。
9. 系统调用返回用户态，用户 buffer 中出现字符或事件结构。

这条路径至少涉及中断、内核栈、设备寄存器、等待队列、调度器、用户指针复制和系统调用返回。若读者只能说“键盘产生中断，程序读到字符”，说明第一阶段仍然没有过关。

另一条完整路径：一次文件读到 NVMe

再看 `read(fd, buf, len)` 最终落到 NVMe 的路径。

```

user read(fd, buf, len)
syscall entry saves user state and checks fd
VFS resolves file and offset
page cache lookup misses
filesystem maps file offset to block
block layer builds request
NVMe driver fills submission queue entry
dma_map user or kernel buffer pages
driver rings doorbell MMIO register
device DMA-reads/writes memory
device posts completion and raises MSI-X
interrupt handler completes request
sleeping task wakes and copies data to user buffer

```

第一阶段需要读者能指出每个硬件点：doorbell 是 MMIO，数据移动靠 DMA，completion 靠中断或轮询，DMA 地址可能经过 IOMMU，用户 buffer 不能被设备无限期持有，缺页和 page cache 会改变路径。这样进入文件系统章节时，读者看到 page cache、bio、request、completion 才不会断线。

为什么必须加可运行模型

文字解释容易让人以为自己懂了。第一阶段必须至少有一个可运行模型，把下面这些断言跑出来：

- 用户态不能访问内核页。
- 用户态不能访问 MMIO。
- 内核可以通过 MMIO 通知设备。
- 设备事件会产生中断，CPU 保存 trap frame 并进入内核模式。
- DMA 映射完成前后有明确所有权，unmap 后设备不能继续写。

这些不是实践卷才需要的内容。原理卷也需要最小代码，因为它证明书里的概念能落成状态机。下一节给出一个 C++20 模型，覆盖第一阶段最小硬件契约。

第一阶段映射表：硬件对象怎样变成内核对象

学硬件最容易卡在“知道名词但不知道用在哪里”。本节把第一阶段核心硬件逐项映射到 OS 对象、源码入口、观测证据和失败案例。读者以后遇到任何新硬件，都应该按这张表追问：它暴露什么寄存器或协议，内核为它建什么对象，失败时谁等待，谁唤醒，谁回收资源。

CPU 状态到任务上下文

硬件事实	OS 对象	源码阅读入口	证据
RIP/通用寄存器	trap frame、thread context	arch/x86/entry/、switch_to	perf sched 、内核栈
状态寄存器	中断位、模式位、返回状态	异常返回路径	信号、ptrace、崩溃现场
内核栈	每线程内核执行上下文	task/thread struct	/proc/<pid>/stack
FPU/SIMD 寄存器	扩展上下文保存区	FPU lazy/eager save 代码	信号帧、上下文切换成本

失败案例：上下文切换漏保存寄存器，用户程序会随机算错；返回用户态前模式位错误，可能把用户代码带回高特权；中断栈损坏，异常嵌套会让系统无法可靠 panic。

MMU 到地址空间

硬件事实	OS 对象	源码阅读入口	证据
页表根	mm_struct /地址空间	mm/ 、架构页表代码	/proc/<pid>/maps
PTE 权限	VMA、PTE、fault handler	mm/memory.c	SIGSEGV、page fault 计数
TLB	shutdown、PCID	mm/tlb.c	IPI、TLB miss、mprotect 成本
COW 只读页	页引用计数和写 fault	fork 和 fault 路径	fork 后 RSS、minor fault

失败案例：修改 PTE 后不失效 TLB，旧权限继续生效；COW 页忘记设只读，父子进程互相污染；释放页表页早于远端 CPU shutdown，旧翻译可能写坏新对象。

中断控制器到事件分发

硬件事实	OS 对象	源码阅读入口	证据
IRQ line/vector	irq descriptor、handler	kernel/irq/	/proc/interrupts

MSI/MSI-X IPI	每队列中断向量 CPU 间控制消息	PCI/MSI 子系统 smp call、resched、 TLB shutdown	<code>lspci -vv</code> perf/trace 中 IPI
定时器事件	clockevent、hrtimer、 tick	<code>kernel/time/</code>	<code>/proc/timer_ list</code>

失败案例：中断未确认会形成中断风暴；中断亲和性不合理会让 CPU0 成瓶颈；定时器未初始化就把 CPU 标记 online，任务迁移过去后无法抢占。

总线 和 MMIO 到 驱动资源

硬件事实	OS 对象	源码阅读入口	证据
PCI BAR	resource、ioremap 区间	PCI core、driver probe	<code>lspci -vv</code> 、 <code>/proc/iomem</code>
ACPI resource	platform/PCI resource	ACPI parser、PCI core	dmesg probe 日志
MMIO 寄存器	<code>readl/writel</code> 协议	各设备驱动	超时、状态寄存 器
posted write	需要读回或屏障确认	arch I/O barrier	随机设备状态 错误

失败案例：把 MMIO 当普通内存缓存，读写会被重排或省略；驱动 probe 失败不逆序释放资源，后续驱动无法绑定；BAR 解析错误会访问不属于该设备的地址窗口。

DMA 和 IOMMU 到 buffer 生命周期

硬件事实	OS 对象	源码阅读入口	证据
DMA descriptor	请求队列、ring、SQE	NVMe/NIC/GPU 驱动	队 列 深 度、 completion
DMA 地址/IOVA	DMA mapping、IOMMU PTE	DMA API、IOMMU 子系 统	IOMMU fault 日志
cache 一致性	clean/invalidate、 方向标志	arch DMA ops	旧数据、数据损 坏
completion	中断、轮询、fence	驱 动 completion 路 径	timeout/reset 日志

失败案例：completion 前释放 buffer，会出现迟到 DMA 写；方向标错会导致设备看到旧数据或 CPU 看到旧数据；IOMMU unmap 后设备继续 DMA，应触发 fault，而不是写到任意内存。

存储硬件到文件系统语义

硬件事实	OS 对象	源码阅读入口	证据
块设备队列	request queue、tag set	block layer、 NVMe/SCSI	<code>iostat -x</code> .blk trace
设备 cache	flush/FUA 请求	filesystem fsync 路径	fsync 延 迟、 flush 计数
NAND/FTL	discard/TRIM、写放 大	block discard、FS 策略	SMART、设备日 志

介质错误	bad block、I/O error	error handler、retry	dmesg、EIO
------	---------------------	---------------------	-----------

失败案例：`write` 返回成功后断电，数据可能只在 page cache 或设备 cache 中；没有 `flush/FUA`，日志提交顺序可能不满足崩溃恢复；设备 reset 时 in-flight 请求不处理，会让等待者永久睡眠。

第一阶段读源码的最低路线

不需要一开始读完整 Linux。第一阶段源码阅读只抓硬件边界：

1. `arch/x86/entry/`：系统调用、中断和异常如何保存状态。
2. `arch/x86/mm/` 与 `mm/memory.c`：页表、fault、TLB 失效。
3. `kernel/irq/` 与 `kernel/time/`：IRQ 和定时器如何变成内核事件。
4. `drivers/pci/`、`drivers/base/`：设备如何被发现和绑定驱动。
5. `kernel/dma/`、`drivers/iommu/`：DMA 地址空间如何建立和回收。
6. 一个具体驱动，例如 NVMe 或 `e1000/e1000e`：队列、doorbell、DMA、completion 如何闭环。

读每个入口都只问四个问题：谁拥有对象，什么时候发布给硬件，失败时怎样回滚，完成时怎样通知等待者。第一阶段能用这四问读通一个驱动的提交和完成路径，才算真正过关。

第一阶段可运行模型：把 CPU、MMIO、中断和 DMA 变成状态机

前面几节把硬件事实映射到内核对象，本节再补最后一块：用一个可编译的 C++20 小模型，把“用户态不能访问内核页”“用户态不能访问 MMIO”“设备通过中断通知 CPU”“DMA 映射有生命周期”这些说法变成能失败、能断言、能复现的状态机。

这个模型不是为了模拟 x86-64 的全部细节。第一阶段需要的是最小硬件契约：只要这几个契约讲不清，后面的系统调用、调度、文件系统和网络 I/O 都会变成背概念。

核心思想

一个适合原理卷的硬件模型不追求外设数量，而追求边界清楚：CPU 以某个模式发起访问，总线按地址把访问分发到 RAM 或 MMIO，内存权限决定访问是否成立，设备完成后投递中断，DMA engine 只能在映射有效期内写入被授权的 buffer。

模型覆盖的硬件契约

模块	模型里的结构	对应 OS 原理
CPU 特权级	<code>PrivilegeLevel::Ring3</code> 与 <code>PrivilegeLevel::Ring0</code>	x86-64 CPL、系统调用入口、中断返回
页权限	<code>PagePermission</code> 与 <code>PhysicalMemory::check_access</code>	用户页、内核页、PTE 权限、保护异常
总线解码	<code>Bus::load8/store8</code> 按地址选择 RAM 或 MMIO	物理地址空间、设备寄存器窗口、 <code>ioremap</code>
MMIO 设备	<code>SimpleDevice</code> 的 <code>status/data/doorbell</code>	驱动寄存器协议、doorbell、副作用写
中断控制器	<code>InterruptController::raise/take_next</code>	IRQ descriptor、pending 事件、handler 分发

trap frame	Cpu::handle_one_interrupt	内核入口、异常帧、返回用
	保存 RIP/RFLAGS/CPL	户态所需状态
DMA 映射	DmaEngine::map/unmap/ device_write	DMA API、IOMMU 授 权、 buffer 所有权

这张表里的每一行都刻意对应一个 OS 对象。CPU 模式对应“谁有资格执行特权操作”；页权限对应“用户指针为什么要检查”；MMIO 对应“设备为什么不能被当作普通变量”；中断对应“设备完成为什么能唤醒等待者”；DMA 对应“completion 前为什么不能释放 buffer”。

状态机比名词更重要

第一阶段最容易写成硬件名词表。名词表的问题是没有失败边界：读者知道 DMA、IOMMU、MMIO，却不知道什么时候该拒绝访问，什么时候该保存现场，什么时候资源已经不能再被设备使用。这个模型用几个状态转移强制表达边界。

1. 用户态写普通用户页：总线走 RAM 路径，页权限允许，写入成功。
2. 用户态写内核页：地址仍是 RAM，但页权限拒绝，抛出保护 fault。
3. 用户态写 MMIO doorbell：总线发现地址属于设备窗口，但模式不是 kernel，抛出保护 fault。
4. 内核写 MMIO doorbell：设备接收命令，更新内部状态，并向中断控制器投递 device interrupt。
5. CPU 处理 pending interrupt：保存 trap frame，切换到 Ring 0，把 RIP 改成设备中断入口。
6. 内核为 buffer 建立 DMA mapping：设备拿到 handle 后才能写入对应 RAM。
7. DMA unmap 后设备继续写：mapping 已失效，模型必须拒绝这次迟到写。

这些状态转移对应真实内核里的不变量：权限检查必须发生在访问之前；中断入口必须保存足够恢复用户态的状态；设备拥有 buffer 的时间必须被内核精确管理；unmap 不是注释，而是权限撤销。

从模型读硬件：每个类为什么这样分

这个 C++20 listing 不是把所有逻辑塞进一个 Machine 类，而是按硬件责任拆开。这样做不是为了形式好看，而是让每条 OS 契约有清楚的所有者。

C++20 类型	持有的状态	不该越界负责的事
PhysicalMemory	字节数组和页权限	不知道 MMIO doorbell 语义
Bus	地址范围分发和 Ring 0 检查	不保存 trap frame
SimpleDevice	设备寄存器状态和中断投递	不直接修改 CPU 寄存器
InterruptController	pending interrupt 队列	不决定 handler 地址
Cpu	CPL、RIP、RFLAGS、trap frames	不绕过 bus 直接碰设备
DmaEngine	DMA mapping 生命周期	不接受已撤销 handle

如果把这些责任混在一起，模型会更短，但教学价值会下降。真实内核里也一样：页表代码不应该偷偷完成设备请求，驱动不应该随意改调度器状态，调度器不应该直接解释 PCIe doorbell。边界清楚，错误才能定位；边界混乱，所有 bug 都像“系统偶现卡死”。

权限检查算法：先分类地址，再检查主体

模型里的 `Bus::store8` 先按地址判断访问目标，再按 CPL 判断是否允许访问 MMIO。普通 RAM 则交给 `PhysicalMemory::check_access` 检查页权限。这个顺序反映一个关键原则：地址类型和访问主体都参与判定。

```
store8(address, value, cpl):
    if address belongs to MMIO:
        require cpl == Ring0
        device.write(address, value)
        return

    memory.check_access(address, cpl, Write)
    memory[address] = value
```

真实 x86-64 路径更复杂：虚拟地址先做 canonical 检查和页表翻译，PTE 里还有 U/S、R/W、NX 和 memory type；翻译后的物理地址再进入平台地址译码，可能命中 DRAM 或 MMIO。模型把它压缩成两层，是为了让读者先抓住不变量：用户态不能凭地址文本绕过权限，设备寄存器不能被当成普通数组元素。

若删掉哪条检查	测试如何失败	真实风险
Ring 3 访问内核页检查	用户写 <code>TEST_KERNEL_ADDRESS</code> 成功	用户破坏内核对象
Ring 3 访问 MMIO 检查	用户写 <code>doorbell</code> 成功	普通进程控制设备
DMA active handle 检查	unmap 后迟到写成功	use-after-free DMA
中断 trap frame 保存	trap 计数不对或 RIP 丢失	无法恢复被打断任务

DMA 生命周期算法：handle 不是地址本身

模型刻意让设备拿到 `DmaHandle`，而不是直接拿普通内存地址。这是为了强调 DMA mapping 是授权对象。handle 有创建、使用、撤销三个阶段；撤销后再使用必须失败。真实系统中 handle 可能表现为 IOVA、DMA address 或 descriptor 中的地址字段，但语义一样：设备只能在映射有效期内按方向访问 buffer。

```
dma.map(start, length):
    reject zero length
    reject MMIO or non-RAM range
    allocate fresh handle
    record start, length, active=true

dma.device_write(handle, offset, value):
    mapping = find active mapping by handle
    reject offset outside mapping
    memory.dma_store8(mapping.start + offset, value)

dma.unmap(handle):
    mapping = find active mapping by handle
    mapping.active = false
```

这段算法比“DMA 可以访问内存”更接近驱动真实难点。提交请求后，buffer 不能被用户释放后立即重用，不能被内核提前归还给 slab，不能在 completion 前改权限，不能在 unmap 后接受迟到设备写。后续讲网络 RX ring 和 NVMe completion 时，仍然是在维护这个生命周期。

如何扩展这个模型而不把它写乱

若继续扩展第一阶段模型，优先增加能破坏 OS 不变量的机制，而不是增加外设数量。推荐顺序如下。

1. 把 16 位教学地址换成 x86-64 canonical 虚拟地址和页表索引。
2. 给 PTE 增加 user、writable、executable、accessed、dirty。
3. 给 TLB 增加 PCID，并测试 stale TLB 权限。
4. 给中断入口增加 vector、error code、RSP 和 RFLAGS.IF。
5. 给 DMA 增加方向、IOMMU domain 和 scatter-gather 片段。
6. 给设备 reset 增加 in-flight 请求取消和迟到 completion。

这正是下一节 x86-64 契约模型做的事。它不是推翻本节，而是把同一组状态机换成更贴近真实 x86-64 的名字和断言。

编译和运行

在书根目录运行下面两条命令，可以独立编译并执行本节模型。

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -Werror \
    source/latex/listings/stage_one_hardware_model.cpp \
    -o /tmp/stage_one_hardware_model \
    /tmp/stage_one_hardware_model
```

期望输出如下：

```
stage one hardware model checks passed
```

如果读者把模型里的任一条边界删掉，例如允许用户态写 MMIO、允许用户态写内核页、或者允许 unmap 后继续 DMA 写，程序都应该失败。这个失败比“书上说不可以”更有教学价值，因为它把抽象规则变成了可观察的断言。

测试断言和失败含义

断言	失败说明	对应真实系统风险
用户页读写成功	基础 RAM 路径或页权限初始化错误	用户程序无法正常执行
用户态写内核页失败	缺少特权隔离	用户进程可破坏内核对象
用户态写 MMIO 失败	设备寄存器暴露给用户态	用户程序可直接控制硬件
内核写 doorbell 后产生中断	设备完成路径没有闭环	等待者可能永久睡眠
中断保存 trap frame	返回路径缺少现场	用户态恢复后状态错乱
DMA 写入映射 buffer 成功	buffer 授权或地址翻译错误	I/O 数据无法进入内存
unmap 后 DMA 写失败	生命周期撤销无效	迟到 DMA 可能写坏新对象

这组断言不是完整 OS 测试，但它覆盖第一阶段必须过关的最小边界。后面讲系统调用时，可以把“用户指针检查”接回页权限；讲调度时，可以把“中断入口保存现场”接回 trap frame；讲文件系统和网络时，可以把“请求完成”接回中断与 DMA 生命周期。

完整 C++20 模型源码

```

#include <array>
#include <cstdint>
#include <cstdlib>
#include <deque>
#include <iostream>
#include <optional>
#include <stdexcept>
#include <string>
#include <string_view>
#include <utility>
#include <vector>

using Address = std::uint16_t;
using Byte = std::uint8_t;
using Word = std::uint16_t;
using DmaHandle = std::uint32_t;

constexpr std::size_t HARDWARE_MEMORY_SIZE = 64U * 1024U;
constexpr std::size_t HARDWARE_PAGE_SIZE = 256U;
constexpr std::size_t HARDWARE_PAGE_COUNT =
    HARDWARE_MEMORY_SIZE / HARDWARE_PAGE_SIZE;
constexpr Address HARDWARE_KERNEL_START = 0x0000U;
constexpr Address HARDWARE_KERNEL_END = 0x0FFFU;
constexpr Address HARDWARE_USER_START = 0x1000U;
constexpr Address HARDWARE_USER_END = 0xDFFFU;
constexpr Address HARDWARE_MMIO_START = 0xF000U;
constexpr Address HARDWARE_MMIO_STATUS = 0xF000U;
constexpr Address HARDWARE_MMIO_DATA = 0xF002U;
constexpr Address HARDWARE_MMIO_DOORBELL = 0xF004U;
constexpr Address HARDWARE_TIMER_HANDLER = 0x0020U;
constexpr Address HARDWARE_DEVICE_HANDLER = 0x0040U;
constexpr Address TEST_USER_BUFFER = 0x2000U;
constexpr Address TEST_KERNEL_ADDRESS = 0x0100U;
constexpr Byte TEST_DEVICE_BYTE = 0x5AU;
constexpr Byte TEST_USER_BYTE = 0x33U;
constexpr std::size_t TEST_DMA_LENGTH = 1U;
constexpr DmaHandle DMA_INVALID_HANDLE = 0U;

enum class PrivilegeLevel : std::uint8_t {
    Ring0 = 0U,
    Ring3 = 3U,
};

enum class AccessKind {
    Read,
    Write,
    Execute,
};

enum class FaultKind {
    None,
    Protection,
    InvalidMmio,
    InvalidDma,
};

enum class InterruptVector {
    Timer,
    Device,
};

struct PagePermission {
    bool user = false;
    bool readable = false;
    bool writable = false;
    bool executable = false;
};

struct TrapFrame {
    Address rip = 0;
    Word rflags = 0;
    PrivilegeLevel previous_cpl = PrivilegeLevel::Ring3;
};

class Fault : public std::runtime_error {
public:
    Fault(FaultKind kind, std::string_view message)
        : std::runtime_error(std::string(message)), kind_(kind) {}

    [[nodiscard]] FaultKind kind() const {
        return this->kind_;
    }

private:
    FaultKind kind_ = FaultKind::None;
};

class PhysicalMemory {
public:
    PhysicalMemory() {
        this->set_range_permission(
            HARDWARE_KERNEL_START,
            HARDWARE_KERNEL_END,
            PagePermission{
                .user = false,
                .readable = true,
                .writable = true,
            }
        );
    }
};

```

```

        .executable = true,
    });
    this->set_range_permission(
        HARDWARE_USER_START,
        HARDWARE_USER_END,
        PagePermission{
            .user = true,
            .readable = true,
            .writable = true,
            .executable = true,
        }
    );
}

[[nodiscard]] Byte load8(Address address, PrivilegeLevel cpl) const {
    this->check_access(address, cpl, AccessKind::Read);
    return this->bytes_.at(address);
}

void store8(Address address, Byte value, PrivilegeLevel cpl) {
    this->check_access(address, cpl, AccessKind::Write);
    this->bytes_.at(address) = value;
}

void dma_store8(Address address, Byte value) {
    this->check_address_in_ram(address);
    this->bytes_.at(address) = value;
}

[[nodiscard]] bool can_execute(Address address, PrivilegeLevel cpl) const {
    try {
        this->check_access(address, cpl, AccessKind::Execute);
        return true;
    } catch (const Fault&) {
        return false;
    }
}

private:
void set_range_permission(Address start, Address end, PagePermission permission) {
    const std::size_t first_page = page_index(start);
    const std::size_t last_page = page_index(end);
    for (std::size_t page = first_page; page <= last_page; ++page) {
        this->permissions_.at(page) = permission;
    }
}

static std::size_t page_index(Address address) {
    return static_cast<std::size_t>(address) / HARDWARE_PAGE_SIZE;
}

void check_address_in_ram(Address address) const {
    if (address >= HARDWARE_MMIO_START) {
        throw Fault(FaultKind::Protection, "address is not normal RAM");
    }
}

void check_access(Address address, PrivilegeLevel cpl, AccessKind access) const {
    this->check_address_in_ram(address);
    const PagePermission& permission = this->permissions_.at(page_index(address));
    if (cpl == PrivilegeLevel::Ring3 && !permission.user) {
        throw Fault(FaultKind::Protection, "ring 3 cannot access kernel page");
    }
    const bool allowed =
        (access == AccessKind::Read && permission.readable) ||
        (access == AccessKind::Write && permission.writable) ||
        (access == AccessKind::Execute && permission.executable);
    if (!allowed) {
        throw Fault(FaultKind::Protection, "page permission rejected access");
    }
}

std::array<Byte, HARDWARE_MEMORY_SIZE> bytes_{};
std::array<PagePermission, HARDWARE_PAGE_COUNT> permissions_{};
};

class InterruptController {
public:
    void raise(InterruptVector vector) {
        this->pending_.push_back(vector);
    }

    [[nodiscard]] std::optional<InterruptVector> take_next() {
        if (this->pending_.empty()) {
            return std::nullopt;
        }
        const InterruptVector vector = this->pending_.front();
        this->pending_.pop_front();
        return vector;
    }

private:
    std::deque<InterruptVector> pending_;
};

class SimpleDevice {
public:
    explicit SimpleDevice(InterruptController& interrupts) : interrupts_(interrupts) {}

```

```

[[nodiscard]] Byte read(Address address) const {
    if (address == HARDWARE_MMIO_STATUS) {
        return this->ready_ ? 1U : 0U;
    }
    if (address == HARDWARE_MMIO_DATA) {
        return this->data_;
    }
    throw Fault(FaultKind::InvalidMmio, "unknown MMIO read");
}

void write(Address address, Byte value) {
    if (address != HARDWARE_MMIO_DOORBELL) {
        throw Fault(FaultKind::InvalidMmio, "unknown MMIO write");
    }
    this->ready_ = true;
    this->data_ = value;
    this->interrupts_.raise(InterruptVector::Device);
}

private:
    InterruptController& interrupts_;
    bool ready_ = false;
    Byte data_ = 0;
};

class Bus {
public:
    Bus(PhysicalMemory& memory, SimpleDevice& device)
        : memory_(memory), device_(device) {}

    [[nodiscard]] Byte load8(Address address, PrivilegeLevel cpl) const {
        if (address >= HARDWARE_MMIO_START) {
            this->require_ring0(cpl);
            return this->device_.read(address);
        }
        return this->memory_.load8(address, cpl);
    }

    void store8(Address address, Byte value, PrivilegeLevel cpl) {
        if (address >= HARDWARE_MMIO_START) {
            this->require_ring0(cpl);
            this->device_.write(address, value);
            return;
        }
        this->memory_.store8(address, value, cpl);
    }

private:
    static void require_ring0(PrivilegeLevel cpl) {
        if (cpl != PrivilegeLevel::Ring0) {
            throw Fault(FaultKind::Protection, "ring 3 cannot access MMIO");
        }
    }

    PhysicalMemory& memory_;
    SimpleDevice& device_;
};

struct DmaMapping {
    DmaHandle handle = DMA_INVALID_HANDLE;
    Address start = 0;
    std::size_t length = 0;
    bool active = false;
};

class DmaEngine {
public:
    explicit DmaEngine(PhysicalMemory& memory) : memory_(memory) {}

    [[nodiscard]] DmaHandle map(Address start, std::size_t length) {
        if (length == 0 || start >= HARDWARE_MMIO_START) {
            throw Fault(FaultKind::InvalidDma, "invalid DMA mapping");
        }
        const std::size_t end =
            static_cast<std::size_t>(start) + length;
        if (end > HARDWARE_MMIO_START) {
            throw Fault(FaultKind::InvalidDma, "invalid DMA mapping");
        }
        const DmaHandle handle = this->next_handle_++;
        this->mappings_.push_back(DmaMapping{
            .handle = handle,
            .start = start,
            .length = length,
            .active = true,
        });
        return handle;
    }

    void device_write(DmaHandle handle, std::size_t offset, Byte value) {
        DmaMapping& mapping = this->find_active_mapping(handle);
        if (offset >= mapping.length) {
            throw Fault(FaultKind::InvalidDma, "DMA write outside mapped buffer");
        }
        const Address target =
            static_cast<Address>(mapping.start + static_cast<Address>(offset));
        this->memory_.dma_store8(target, value);
    }
};

```

```

}

void unmap(DmaHandle handle) {
    DmaMapping& mapping = this->find_active_mapping(handle);
    mapping.active = false;
}

private:
DmaMapping& find_active_mapping(DmaHandle handle) {
    for (auto& mapping : this->mappings_) {
        if (mapping.handle == handle && mapping.active) {
            return mapping;
        }
    }
    throw Fault(FaultKind::InvalidDma, "DMA handle is not active");
}

PhysicalMemory& memory_;
DmaHandle next_handle_ = 1U;
std::vector<DmaMapping> mappings_;
};

class Cpu {
public:
    Cpu(Bus& bus, InterruptController& interrupts)
        : bus_(bus), interrupts_(interrupts) {}

    void user_store(Address address, Byte value) {
        this->bus_.store8(address, value, PrivilegeLevel::Ring3);
    }

    [[nodiscard]] Byte user_load(Address address) const {
        return this->bus_.load8(address, PrivilegeLevel::Ring3);
    }

    void kernel_store(Address address, Byte value) {
        this->bus_.store8(address, value, PrivilegeLevel::Ring0);
    }

    void handle_one_interrupt() {
        const std::optional<InterruptVector> vector = this->interrupts_.take_next();
        if (!vector.has_value()) {
            return;
        }
        this->trap_frames_.push_back(TrapFrame{
            .rip = this->rip_,
            .rflags = this->rflags_,
            .previous_cpl = this->cpl_,
        });
        this->cpl_ = PrivilegeLevel::Ring0;
        this->rip_ = this->handler_address(*vector);
    }

    [[nodiscard]] PrivilegeLevel cpl() const {
        return this->cpl_;
    }

    [[nodiscard]] Address rip() const {
        return this->rip_;
    }

    [[nodiscard]] std::size_t trap_count() const {
        return this->trap_frames_.size();
    }

private:
    static Address handler_address(InterruptVector vector) {
        switch (vector) {
            case InterruptVector::Timer:
                return HARDWARE_TIMER_HANDLER;
            case InterruptVector::Device:
                return HARDWARE_DEVICE_HANDLER;
        }
        throw std::logic_error("unknown interrupt vector");
    }

    Bus& bus_;
    InterruptController& interrupts_;
    PrivilegeLevel cpl_ = PrivilegeLevel::Ring3;
    Address rip_ = HARDWARE_USER_START;
    Word rflags_ = 0;
    std::vector<TrapFrame> trap_frames_;
};

void require(bool condition, std::string_view message) {
    if (!condition) {
        throw std::runtime_error(std::string(message));
    }
}

template <typename Fn>
void require_fault(FaultKind expected, Fn&& fn) {
    try {
        fn();
    } catch (const Fault& fault) {
        require(fault.kind() == expected, "unexpected fault kind");
        return;
    }
}

```



```

    }
    throw std::runtime_error("expected a fault");
}

int main() {
    PhysicalMemory memory;
    InterruptController interrupts;
    SimpleDevice device(interrupts);
    Bus bus(memory, device);
    DmaEngine dma(memory);
    Cpu cpu(bus, interrupts);

    cpu.user_store(TEST_USER_BUFFER, TEST_USER_BYTE);
    require(cpu.user_load(TEST_USER_BUFFER) == TEST_USER_BYTE, "user RAM failed");
    require_fault(FaultKind::Protection, [&] {
        cpu.user_store(TEST_KERNEL_ADDRESS, TEST_USER_BYTE);
    });
    require_fault(FaultKind::Protection, [&] {
        cpu.user_store(HARDWARE_MMIO_DOORBELL, TEST_USER_BYTE);
    });

    cpu.kernel_store(HARDWARE_MMIO_DOORBELL, TEST_DEVICE_BYTE);
    cpu.handle_one_interrupt();
    require(cpu.cpl() == PrivilegeLevel::Ring0, "interrupt did not enter ring 0");
    require(cpu.rip() == HARDWARE_DEVICE_HANDLER, "device interrupt vector mismatch");
    require(cpu.trap_count() == 10, "trap frame was not saved");

    const DmaHandle handle = dma.map(TEST_USER_BUFFER, TEST_DMA_LENGTH);
    dma.device_write(handle, 0U, TEST_DEVICE_BYTE);
    dma.unmap(handle);
    require(cpu.user_load(TEST_USER_BUFFER) == TEST_DEVICE_BYTE, "DMA write failed");
    require_fault(FaultKind::InvalidDma, [&] {
        dma.device_write(handle, 0U, TEST_USER_BYTE);
    });

    std::cout << "stage one hardware model checks passed\n";
    return EXIT_SUCCESS;
}

```

第一阶段 x86-64 契约模型：把架构名字落到断言

上一节的小模型故意很小，用来证明“模式、权限、MMIO、中断、DMA 生命周期”这些边界必须存在。但它还没有把 x86-64 的关键名字直接放进状态机：canonical address、四级页表索引、PTE 权限、TLB/PCID、CPL、IDT、RFLAGS、APIC pending、MSI-X 风格中断、IOMMU 映射和 `iretq` 返回。第一阶段若要真正服务后面的 OS 章节，就必须再前进一步：不是完整模拟器，而是把 x86-64 最小契约做成可运行断言。

核心思想

原理卷里的 x86-64 模型不追求跑真实机器码，而追求把“硬件为什么强迫 OS 这样设计”固定下来。只要 PTE 权限、TLB 失效、CPL、IDT、trap frame、APIC、IOMMU 和 DMA 生命周期能被测试击穿，读者就能把后续的系统调用、缺页、调度和驱动路径接回硬件事实。

这个模型比上一节多了什么

机制	模型里的对象	要证明的不变量
canonical address 四级页表索引	<code>is_canonical</code> 与 <code>split_virtual_address</code> <code>PML4/PDPT/PD/PT/offset</code> 拆分	非 canonical 虚拟地址不能继续页表遍历 虚拟地址不是一个平面整数，而是页表路径加页内偏移
PTE 权限	<code>PageTableEntry</code> 的 <code>present/writable/user/execute</code>	用户态、写、执行分别有独立权限位
TLB/PCID	<code>TlbKey\{pcid, vpn\}</code>	改 PTE 后旧 TLB 仍可能保存旧权限，必须失效
CPL	<code>PrivilegeLevel::Ring3/Ring0</code>	Ring 3 不能访问 supervisor 页和 MMIO

trap frame	RIP/RSP/RFLAGS/CPL/vector/error code	异常和中断返回必须有足够证据
IDT/APIC	InterruptDescriptorTable 与 LocalApic	设备完成不是函数调用，而是 pending vector 分发
IOMMU/DMA	IommuDomain 与 DmaEngine	设备只能写被映射、方向允许、生命周期有效的 buffer

注意这里的“x86-64”不是把所有 Intel 手册字段搬进书里，而是保留后续 OS 必须依赖的最小集合。比如模型没有实现真实机器码译码，但它明确保存 RIP；没有实现完整 CR3 位布局，但用 PCID 区分 TLB 项；没有实现真实 MSI-X table，但用 APIC pending vector 表达设备完成的异步交付。

地址翻译算法：从 canonical 检查到 TLB 填充

模型里的 `Mmu::translate` 故意把地址翻译写成短路径，但它覆盖了真实 OS 最关心的顺序：先拒绝非 canonical 地址，再查 TLB，再做页表遍历，再检查 PTE 权限，最后把翻译放入 TLB。

```
translate(address_space, tlb, va, access, cpl):
    indexes = split_virtual_address(va)
    vpn = va >> 12

    if tlb has (pcid, vpn):
        entry = tlb entry
        check access against entry and cpl
        return entry.page_base + indexes.offset

    entry = page_table_walk(vpn)
    check access against entry and cpl
    tlb.insert(pcid, vpn, entry)
    return entry.page_base + indexes.offset
```

这个顺序解释了两个常见误解。第一，TLB 不是只缓存物理地址，它还缓存权限相关信息；所以 PTE 权限收紧后，旧 TLB 项也可能让访问继续通过。第二，非 canonical 地址甚至不应该进入正常页表语义；它不是“某页不存在”，而是地址形式本身不满足 x86-64 规则。

失败点	模型抛出的 fault	OS 后续判断
非 canonical PTE 不存在	<code>FaultKind::NonCanonical</code> <code>FaultKind::PageFault</code>	用户信号或内核 bug demand paging、文件页、非法地址
用户访问 supervisor 页	<code>FaultKind::PageFault</code>	安全违规或坏用户指针
写只读页	<code>FaultKind::PageFault</code>	COW、mprotect、权限错误
执行 NX 页	<code>FaultKind::PageFault</code>	W^X 或代码注入防护

TLB 失效算法：为什么要指定 PCID 和 VPN

模型用 `TlbKey\{pcid,vpn\}` 标识一条缓存翻译。PCID 表示地址空间标签，VPN 表示虚拟页号。这样同一个虚拟地址在不同进程中可以同时存在于 TLB 中，而不会互相误用。

```
invalidate one page:
    key = (current_pcid, va >> 12)
    tlb.erase(key)
```

```

invalidate address space:
    erase all entries whose key.pcid == target_pcid

global shutdown on SMP:
    update page table in memory
    send IPI to CPUs that may hold target PCID
    each CPU invalidates matching entries
    wait until acknowledgements arrive

```

模型只实现单 CPU 的 `invalidate_page`，但测试三故意证明“不失效就会错”。真实多核系统里，错误更隐蔽：当前 CPU 可能已经失效了，另一个 CPU 仍持有旧权限。物理页被释放复用后，旧翻译就可能写到新对象上。这就是为什么 `unmap`、`COW`、`mprotect` 和进程退出都要认真处理 TLB shutdown。

trap frame 字段：每个字段都服务一个返回问题

模型的 trap frame 保存 RIP、RSP、RFLAGS、CPL、vector、error code 和 fault address。它看起来像一组字段，实际每个字段都回答一个返回路径问题。

字段	保存原因	若缺失会怎样
RIP	知道从哪里重试或继续	page fault 无法回到 faulting instruction
RSP	恢复用户栈或被打断栈	返回后函数调用链损坏
RFLAGS	恢复条件码、中断位、单步等状态	用户程序观察到错误标志或调试失效
CPL	知道返回 Ring 3 还是留在 Ring 0	特权级恢复错误
vector	知道事件来源	page fault、timer、device 处理混淆
error code	知道访问类型和权限失败原因	COW 与非法访问无法区分
fault address	知道哪个地址触发异常	VMA 查找和用户指针诊断失败

真实 x86-64 的入口代码还会保存通用寄存器，某些异常由硬件压入 error code，系统调用路径还涉及 `rcx/r11` 和 MSR 约定。本模型保留最小集合，是为了让读者看清：trap frame 不是“现场”这个抽象词，而是一份能支持后续决策和返回的证据。

系统调用路径：和函数调用逐项对比

系统调用经常被粗略说成“用户调用内核函数”。这个说法会误导。函数调用只改变 RIP/RSP；系统调用改变特权级、入口地址、RFLAGS 部分位，并把用户返回点放到架构约定位置或 trap frame 中。模型把它简化为 `Cpu::syscall` 保存 frame、进入 Ring 0、跳到固定 handler，并清除 IF。

动作	普通 <code>call f</code>	<code>syscall</code> 入口
入口目标	由当前代码地址空间中的 <code>call target</code> 决定	由内核配置的受控入口决定
特权级	不改变 CPL	Ring 3 进入 Ring 0
栈信任	使用当前用户栈或当前栈	内核必须切到可信内核栈
参数来源	ABI 寄存器和栈	<code>syscall</code> ABI 寄存器，用户指针需验证
返回规则	<code>ret</code> 弹出返回地址	<code>sysretq</code> 或 <code>iretq</code> 恢复用户状态

失败表达	函数约定	errno、信号、进程终止、部分完成
------	------	--------------------

所以系统调用的成本不仅是“跳转慢”。它要跨权限边界，建立可信栈，复制或验证用户数据，更新审计和追踪状态，处理信号和抢占，再安全返回。后续所有内核入口都应按这个标准检查：是否有受控入口，是否保存足够状态，是否验证用户提供的地址和长度。

IOMMU 映射算法：设备也要有地址空间

CPU 用页表限制进程，设备用 IOMMU domain 限制 DMA。模型里的 `IommuDomain::map` 分配 IOVA，记录物理页、长度和方向；`translate_for_write` 在设备写入前检查映射存在、方向允许、offset 不越界。

```
iommu.map(pa, len, FromDevice):
    require page aligned physical address
    require non-zero length
    iova = allocate device virtual address
    record iova -> {pa, len, direction}
    return iova

device writes:
    mapping = lookup(iova)
    require direction allows device write
    require offset < mapping.length
    physical = mapping.pa + offset
    memory.store8(physical, value)
```

注意这里方向是权限的一部分。磁盘读取到内存时，设备需要 `FromDevice` 写权限；网卡发送内存中的包时，设备需要 `ToDevice` 读权限。方向标错可能造成 cache 维护错误或 IOMMU 拒绝。unmap 后映射被删除，迟到 DMA 必须变成 `fault`，而不是继续写旧物理页。

把测试映射到后续章节

这个模型的测试不是独立小玩具，它们会在后续章节反复出现。

模型测试	后续机制	复用的问题
canonical 和页表索引	exec、mmap、page fault	用户地址是否合法，页表如何定位 PTE
PTE 权限 fault	COW、WX、用户/内核隔离	fault 是可修复还是违规
stale TLB	unmap、mprotect、fork COW	改 PTE 后如何让 CPU 停止用旧答案
syscall/interrupt frame	系统调用、信号、调度	返回用户态需要恢复哪些证据
IOMMU/DMA 生命周期	块层、网卡、文件 I/O	buffer 什么时候属于设备，什么时候归还内核

读者如果能改坏其中一个检查并理解哪个测试失败，就已经具备读 OS 源码的基本姿势：不是背“内核会保护我们”，而是能指出保护在哪个入口、哪张表、哪个状态位和哪个失效动作上发生。

测试一：地址不是任意 64 位数

x86-64 的虚拟地址虽然放在 64 位寄存器里，但常见实现只使用低 48 位或更宽的 canonical 形式。高位必须是符号扩展。模型里的第一组测试要求：

```
low canonical user address is accepted
high canonical kernel address is accepted
non-canonical address is rejected before page walk
virtual address splits into PML4/PDPT/PD/PT/offset
```

这个测试对应真实 OS 的两个边界。第一，坏地址可能在页表查询前就被硬件拒绝；第二，页表不是“map 里查一个地址”这么简单，而是由虚拟地址位段逐级索引。后面讲 page fault 时，读者要区分“地址形式非法”“页表项不存在”“页表项存在但权限不允许”。

测试二：PTE 权限必须产生可诊断 fault

模型把用户页、只读页、不可执行页、内核页和 MMIO 页分别映射出来。Ring 3 访问普通用户页应该成功；Ring 3 访问 supervisor 页、写只读页、执行 NX 页都必须触发 page fault，并在 trap frame 里保存 vector、fault address 和 error code。

访问	期望结果	OS 解释
Ring 3 读写用户页	成功	普通用户内存
Ring 3 读 supervisor 页	page fault, user bit 置位	用户越权访问内核映射
Ring 3 写只读页	page fault, write bit 置位	COW 或权限错误入口
Ring 3 执行 NX 页	page fault, instruction bit 置位	WX、栈不可执行和代码页保护

这些断言让“缺页不是一种错误”变得具体。内核必须看 fault address、error code、当前 CPL 和 VMA/PTE，才能判断是合法懒分配、COW、非法地址还是安全违规。

测试三：改 PTE 不等于旧 TLB 自动消失

模型故意做了一个危险过程：先让写访问填充 TLB，再把 PTE 改成只读，但不做 TLB invalidate。此时第二次写仍然通过，因为 TLB 里保存着旧 writable 权限。只有执行 `invalidate_page(pcid, vpn)` 后，下一次写才按新 PTE 触发 page fault。

```
write user page -> TLB caches writable PTE
kernel clears writable bit in page table
write still succeeds while stale TLB exists
invalidate PCID+VPN
write now faults
```

这就是 TLB shutdown 的根。多核机器上，别的 CPU 可能还缓存着旧翻译。OS 修改页表后，不仅要改内存里的 PTE，还要让相关 CPU 失效旧 TLB；否则权限收紧、unmap、COW 分离都可能被旧缓存绕过。

测试四：系统调用和中断都要保存返回证据

模型把 `syscall` 简化为：从 Ring 3 保存 trap frame，进入 Ring 0，把 RIP 改为 `X86_SYSCALL_HANDLER`，并清掉 `RFLAGS.IF`。返回用 `iretq` 恢复 RIP/RSP/RFLAGS/CPL。设备完成路径则是内核写 MMIO doorbell，设备向 local APIC 投递 pending vector，CPU 从 IDT 查 handler。

入口	共同点	差别
系统调用	保存用户现场，进入 Ring 0	入口由约定 MSR/ABI 决定，来自同步指令

设备中断	保存被打断现场, 进入 Ring 0	入口由 APIC pending vector 和 IDT 决定, 来自异步设备事件
page fault	保存 fault 现场, 进入 Ring 0	还要保存 fault address 和 error code

这说明“进入内核”不是一句话。不同入口的来源、错误码、返回规则、可重启性不同。后面讲系统调用、定时器抢占、设备 completion、信号和调度时, 都要回到这张表。

测试五: DMA 受 IOMMU 和生命周期约束

模型的 IOMMU domain 给设备发 IOVA。设备写入时, 必须满足三件事: IOVA 存在, 方向允许设备写, offset 在映射长度内。unmap 后再写必须失败。

```
iova = iommu.map(user_page, len, FromDevice)
device_write(iova, offset, value) succeeds
iommu.unmap(iova)
device_write(iova, offset, value) faults
```

这条测试对应真实驱动里最容易出问题的生命周期: completion 前释放 buffer、方向标错、unmap 太早或太晚、设备 reset 后迟到 DMA。IOMMU 不是性能装饰, 它把设备访问也变成可授权、可撤销、可诊断的地址空间。

编译和运行

在书根目录运行:

```
make test
```

期望看到三个 listing 都被编译并运行, 其中 x86-64 契约模型输出:

```
x86-64 stage one contract model checks passed
```

make check 现在也会依赖这些 C++ listing 测试。以后若书稿里的模型和解释脱节, 或者 C++ 代码被改坏, 构建会直接失败。这比把代码贴在书里但不执行更可靠。

完整 C++20 模型源码

```
#include <cstdint>
#include <cstddef>
#include <deque>
#include <functional>
#include <iostream>
#include <map>
#include <optional>
#include <stdexcept>
#include <string>
#include <string_view>
#include <unordered_map>
#include <vector>

using Byte = std::uint8_t;
using Word = std::uint64_t;
using VirtualAddress = std::uint64_t;
using PhysicalAddress = std::uint64_t;
using PageNumber = std::uint64_t;
using Pcid = std::uint16_t;
using InterruptVector = std::uint8_t;
using Iova = std::uint64_t;

constexpr std::uint64_t X86_VIRTUAL_ADDRESS_BITS = 48U;
constexpr std::uint64_t X86_PAGE_SHIFT = 12U;
constexpr std::uint64_t X86_PAGE_TABLE_INDEX_BITS = 9U;
constexpr std::uint64_t X86_PML4_SHIFT = 39U;
constexpr std::uint64_t X86_PDPT_SHIFT = 30U;
constexpr std::uint64_t X86_PD_SHIFT = 21U;
constexpr std::uint64_t X86_PT_SHIFT = 12U;
constexpr std::uint64_t X86_PAGE_SIZE = 1ULL << X86_PAGE_SHIFT;
constexpr std::uint64_t X86_PAGE_OFFSET_MASK = X86_PAGE_SIZE - 1ULL;
constexpr std::uint64_t X86_PAGE_TABLE_INDEX_MASK =
    (1ULL << X86_PAGE_TABLE_INDEX_BITS) - 1ULL;
constexpr VirtualAddress X86_CANONICAL_LOW_MASK =
    (1ULL << X86_VIRTUAL_ADDRESS_BITS) - 1ULL;
constexpr VirtualAddress X86_CANONICAL_HIGH_MASK = ~X86_CANONICAL_LOW_MASK;
```



```

constexpr VirtualAddress X86_CANONICAL_SIGN_BIT =
    1ULL << (X86_VIRTUAL_ADDRESS_BITS - 1U);
constexpr Word X86_RFLAGS_INTERRUPT_ENABLE = 1ULL << 9U;
constexpr Word X86_PAGE_FAULT_WRITE_BIT = 1ULL << 1U;
constexpr Word X86_PAGE_FAULT_USER_BIT = 1ULL << 2U;
constexpr Word X86_PAGE_FAULT_INSTRUCTION_BIT = 1ULL << 4U;
constexpr std::size_t HASH_COMBINE_SHIFT = 1U;

constexpr InterruptVector X86_VECTOR_PAGE_FAULT = 14U;
constexpr InterruptVector X86_VECTOR_SYSCALL = 0x80U;
constexpr InterruptVector X86_VECTOR_TIMER = 0x20U;
constexpr InterruptVector X86_VECTOR_DEVICE = 0x31U;

constexpr Pcid TEST_USER_PCID = 7U;
constexpr VirtualAddress TEST_USER_RIP = 0x0000'0000'0040'1000ULL;
constexpr VirtualAddress TEST_USER_RSP = 0x0000'0000'0070'0000ULL;
constexpr VirtualAddress TEST_USER_VA = 0x0000'0000'0040'2000ULL;
constexpr VirtualAddress TEST_USER_PAGE = 0x0000'0000'0040'2000ULL;
constexpr VirtualAddress TEST_READ_ONLY_VA = 0x0000'0000'0040'3000ULL;
constexpr VirtualAddress TEST_NX_VA = 0x0000'0000'0040'4000ULL;
constexpr VirtualAddress TEST_KERNEL_VA = 0xFFFF'8000'0000'1000ULL;
constexpr VirtualAddress TEST_MMIO_VA = 0xFFFF'8000'000F'0000ULL;
constexpr VirtualAddress TEST_NON_CANONICAL_VA = 0x0000'8000'0000'0000ULL;
constexpr VirtualAddress X86_SYSCALL_HANDLER = 0xFFFF'8000'0000'8000ULL;
constexpr VirtualAddress X86_TIMER_HANDLER = 0xFFFF'8000'0000'9000ULL;
constexpr VirtualAddress X86_DEVICE_HANDLER = 0xFFFF'8000'0000'A000ULL;
constexpr PhysicalAddress TEST_USER_PA = 0x0000'0000'0010'0000ULL;
constexpr PhysicalAddress TEST_READ_ONLY_PA = 0x0000'0000'0011'0000ULL;
constexpr PhysicalAddress TEST_NX_PA = 0x0000'0000'0012'0000ULL;
constexpr PhysicalAddress TEST_KERNEL_PA = 0x0000'0000'0020'0000ULL;
constexpr PhysicalAddress X86_MMIO_BASE = 0x0000'0000'00F0'0000ULL;
constexpr PhysicalAddress X86_MMIO_DOORBELL = X86_MMIO_BASE;
constexpr PhysicalAddress X86_MMIO_END = X86_MMIO_BASE + X86_PAGE_SIZE;
constexpr Byte TEST_USER_BYTE = 0x5AU;
constexpr Byte TEST_DEVICE_BYTE = 0xC3U;
constexpr std::size_t TEST_DMA_LENGTH = 64U;
constexpr std::size_t TEST_DMA_OFFSET = 8U;

enum class PrivilegeLevel : std::uint8_t {
    Ring0 = 0U,
    Ring3 = 3U,
};

enum class AccessKind {
    Read,
    Write,
    Execute,
};

enum class FaultKind {
    NonCanonical,
    PageFault,
    Protection,
    MissingInterruptHandler,
    DmaFault,
};

enum class DmaDirection {
    ToDevice,
    FromDevice,
    Bidirectional,
};

struct Fault : public std::runtime_error {
    Fault(FaultKind fault_kind, std::string_view message)
        : std::runtime_error(std::string(message)), kind(fault_kind) {}

    FaultKind kind;
};

struct PageTableIndexes {
    std::uint16_t pml4 = 0;
    std::uint16_t pdpt = 0;
    std::uint16_t pd = 0;
    std::uint16_t pt = 0;
    std::uint16_t offset = 0;
};

struct PageAttributes {
    bool writable = false;
    bool user = false;
    bool executable = false;
};

struct PageTableEntry {
    PhysicalAddress page_base = 0;
    bool present = false;
    bool writable = false;
    bool user = false;
    bool executable = false;
};

struct Translation {
    PhysicalAddress physical = 0;
    PageTableEntry entry;
};

```

```

struct TlbKey {
    Pcid pcid = 0;
    PageNumber vpn = 0;

    bool operator==(const TlbKey& other) const {
        return this->pcid == other.pcid && this->vpn == other.vpn;
    }
};

struct TlbKeyHash {
    std::size_t operator()(const TlbKey& key) const {
        const std::size_t pcid_hash = std::hash<Pcid>{}(key.pcid);
        const std::size_t vpn_hash = std::hash<PageNumber>{}(key.vpn);
        return pcid_hash ^ (vpn_hash << HASH_COMBINE_SHIFT);
    }
};

struct TrapFrame {
    VirtualAddress rip = 0;
    VirtualAddress rsp = 0;
    Word rflags = 0;
    PrivilegeLevel cpl = PrivilegeLevel::Ring3;
    InterruptVector vector = 0;
    Word error_code = 0;
    std::optional<VirtualAddress> fault_address;
};

struct DmaMapping {
    PhysicalAddress physical_base = 0;
    std::size_t length = 0;
    DmaDirection direction = DmaDirection::Bidirectional;
};

bool is_canonical(VirtualAddress address) {
    const bool sign_bit_set = (address & X86_CANONICAL_SIGN_BIT) != 0;
    const VirtualAddress high_bits = address & X86_CANONICAL_HIGH_MASK;
    if (sign_bit_set) {
        return high_bits == X86_CANONICAL_HIGH_MASK;
    }
    return high_bits == 0;
}

PageNumber virtual_page_number(VirtualAddress address) {
    return address >> X86_PAGE_SHIFT;
}

PhysicalAddress page_align(PhysicalAddress address) {
    return address & ~X86_PAGE_OFFSET_MASK;
}

PageTableIndexes split_virtual_address(VirtualAddress address) {
    if (!is_canonical(address)) {
        throw Fault(FaultKind::NonCanonical, "virtual address is not canonical");
    }
    return PageTableIndexes{
        .pml4 = static_cast<std::uint16_t>(
            (address >> X86_PML4_SHIFT) & X86_PAGE_TABLE_INDEX_MASK),
        .pdpt = static_cast<std::uint16_t>(
            (address >> X86_PDPT_SHIFT) & X86_PAGE_TABLE_INDEX_MASK),
        .pd = static_cast<std::uint16_t>(
            (address >> X86_PD_SHIFT) & X86_PAGE_TABLE_INDEX_MASK),
        .pt = static_cast<std::uint16_t>(
            (address >> X86_PT_SHIFT) & X86_PAGE_TABLE_INDEX_MASK),
        .offset = static_cast<std::uint16_t>(address & X86_PAGE_OFFSET_MASK),
    };
}

Word page_fault_error_code(AccessKind access, PrivilegeLevel cpl) {
    Word code = 0;
    if (access == AccessKind::Write) {
        code |= X86_PAGE_FAULT_WRITE_BIT;
    }
    if (access == AccessKind::Execute) {
        code |= X86_PAGE_FAULT_INSTRUCTION_BIT;
    }
    if (cpl == PrivilegeLevel::Ring3) {
        code |= X86_PAGE_FAULT_USER_BIT;
    }
    return code;
}

void require(bool condition, std::string_view message) {
    if (!condition) {
        throw std::runtime_error(std::string(message));
    }
}

template <typename Callable>
void require_fault(FaultKind expected, Callable action) {
    try {
        action();
    } catch (const Fault& fault) {
        require(fault.kind == expected, "unexpected fault kind");
        return;
    }
    throw std::runtime_error("expected fault was not raised");
}

```

```

class PhysicalMemory {
public:
    void store8(PhysicalAddress address, Byte value) {
        this->bytes_[address] = value;
    }

    Byte load8(PhysicalAddress address) const {
        const auto iterator = this->bytes_.find(address);
        if (iterator == this->bytes_.end()) {
            return 0;
        }
        return iterator->second;
    }

private:
    std::unordered_map<PhysicalAddress, Byte> bytes_;
};

class Tlb {
public:
    std::optional<PageTableEntry> lookup(Pcid pcid, PageNumber vpn) const {
        const auto iterator = this->entries_.find(TlbKey{.pcid = pcid, .vpn = vpn});
        if (iterator == this->entries_.end()) {
            return std::nullopt;
        }
        return iterator->second;
    }

    void insert(Pcid pcid, PageNumber vpn, const PageTableEntry& entry) {
        this->entries_[TlbKey{.pcid = pcid, .vpn = vpn}] = entry;
    }

    void invalidate_page(Pcid pcid, PageNumber vpn) {
        this->entries_.erase(TlbKey{.pcid = pcid, .vpn = vpn});
    }

    std::size_t size() const {
        return this->entries_.size();
    }

private:
    std::unordered_map<TlbKey, PageTableEntry, TlbKeyHash> entries_;
};

class AddressSpace {
public:
    explicit AddressSpace(Pcid pcid) : pcid_(pcid) {}

    Pcid pcid() const {
        return this->pcid_;
    }

    void map_page(VirtualAddress virtual_page, PhysicalAddress physical_page,
        PageAttributes attributes) {
        require(is_canonical(virtual_page), "mapped virtual address must be canonical");
        require((virtual_page & X86_PAGE_OFFSET_MASK) == 0,
            "virtual page must be page aligned");
        require((physical_page & X86_PAGE_OFFSET_MASK) == 0,
            "physical page must be page aligned");
        this->entries_[virtual_page_number(virtual_page)] = PageTableEntry{
            .page_base = physical_page,
            .present = true,
            .writable = attributes.writable,
            .user = attributes.user,
            .executable = attributes.executable,
        };
    }

    void set_writable(VirtualAddress virtual_page, bool writable) {
        PageTableEntry& entry = this->entry_for_mutation(virtual_page);
        entry.writable = writable;
    }

    const PageTableEntry& entry_for(PageNumber vpn) const {
        const auto iterator = this->entries_.find(vpn);
        if (iterator == this->entries_.end() || !iterator->second.present) {
            throw Fault(FaultKind::PageFault, "page walk found no present PTE");
        }
        return iterator->second;
    }

private:
    PageTableEntry& entry_for_mutation(VirtualAddress virtual_page) {
        require(is_canonical(virtual_page), "updated virtual address must be canonical");
        const auto iterator = this->entries_.find(virtual_page_number(virtual_page));
        require(iterator != this->entries_.end(), "updated PTE must exist");
        return iterator->second;
    }

    Pcid pcid_ = 0;
    std::map<PageNumber, PageTableEntry> entries_;
};

class Mmu {
public:
    Translation translate(const AddressSpace& address_space, Tlb& tlb,

```

```

        VirtualAddress virtual_address, AccessKind access,
        PrivilegeLevel cpl) const {
    const PageTableIndexes indexes = split_virtual_address(virtual_address);
    const PageNumber vpn = virtual_page_number(virtual_address);
    const std::optional<PageTableEntry> cached = tlb.lookup(address_space.pcid(), vpn);
    if (cached.has_value()) {
        this->check_access(cached.value(), access, cpl);
        return Translation{
            .physical = cached->page_base + indexes.offset,
            .entry = cached.value(),
        };
    }

    const PageTableEntry& walked = address_space.entry_for(vpn);
    this->check_access(walked, access, cpl);
    tlb.insert(address_space.pcid(), vpn, walked);
    return Translation{
        .physical = walked.page_base + indexes.offset,
        .entry = walked,
    };
}

private:
void check_access(const PageTableEntry& entry, AccessKind access,
    PrivilegeLevel cpl) const {
    if (!entry.present) {
        throw Fault(FaultKind::PageFault, "PTE is not present");
    }
    if (cpl == PrivilegeLevel::Ring3 && !entry.user) {
        throw Fault(FaultKind::PageFault, "ring 3 tried to access supervisor page");
    }
    if (access == AccessKind::Write && !entry.writable) {
        throw Fault(FaultKind::PageFault, "write access denied by PTE");
    }
    if (access == AccessKind::Execute && !entry.executable) {
        throw Fault(FaultKind::PageFault, "execute access denied by PTE");
    }
}

};

class LocalApic {
public:
    void raise(InterruptVector vector) {
        this->pending_.push_back(vector);
    }

    std::optional<InterruptVector> take_next() {
        if (this->pending_.empty()) {
            return std::nullopt;
        }
        const InterruptVector vector = this->pending_.front();
        this->pending_.pop_front();
        return vector;
    }

    std::size_t pending_count() const {
        return this->pending_.size();
    }

private:
    std::deque<InterruptVector> pending_;
};

class InterruptDescriptorTable {
public:
    void set_handler(InterruptVector vector, VirtualAddress handler) {
        require(is_canonical(handler), "interrupt handler must be canonical");
        this->handlers_[vector] = handler;
    }

    VirtualAddress handler_for(InterruptVector vector) const {
        const auto iterator = this->handlers_.find(vector);
        if (iterator == this->handlers_.end()) {
            throw Fault(FaultKind::MissingInterruptHandler, "missing IDT handler");
        }
        return iterator->second;
    }

private:
    std::map<InterruptVector, VirtualAddress> handlers_;
};

class MmioDevice {
public:
    explicit MmioDevice(LocalApic& apic) : apic_(apic) {}

    void store8(PhysicalAddress address, Byte value) {
        require(address == X86_MMIO_DOORBELL, "only doorbell MMIO is modeled");
        this->last_value_ = value;
        this->apic_.raise(X86_VECTOR_DEVICE);
    }

    Byte last_value() const {
        return this->last_value_;
    }

private:

```

```

    LocalApic& apic_;
    Byte last_value_ = 0;
};

class Bus {
public:
    Bus(PhysicalMemory& memory, MmioDevice& device)
        : memory_(memory), device_(device) {}

    void store8(PhysicalAddress physical, Byte value, PrivilegeLevel cpl) {
        if (this->is_mmio(physical)) {
            this->require_ring0(cpl);
            this->device_.store8(physical, value);
            return;
        }
        this->memory_.store8(physical, value);
    }

    Byte load8(PhysicalAddress physical, PrivilegeLevel cpl) const {
        if (this->is_mmio(physical)) {
            this->require_ring0(cpl);
            return 0;
        }
        return this->memory_.load8(physical);
    }

private:
    bool is_mmio(PhysicalAddress physical) const {
        return physical >= X86_MMIO_BASE && physical < X86_MMIO_END;
    }

    void require_ring0(PrivilegeLevel cpl) const {
        if (cpl != PrivilegeLevel::Ring0) {
            throw Fault(FaultKind::Protection, "ring 3 cannot access MMIO");
        }
    }

    PhysicalMemory& memory_;
    MmioDevice& device_;
};

class IommuDomain {
public:
    Iova map(PhysicalAddress physical_base, std::size_t length,
            DmaDirection direction) {
        require((physical_base & X86_PAGE_OFFSET_MASK) == 0,
            "DMA mapping must start on a page boundary");
        require(length > 0 && length <= X86_PAGE_SIZE,
            "DMA mapping length must fit one modeled page");
        const Iova iova = this->next_iova_;
        this->next_iova_ += X86_PAGE_SIZE;
        this->mappings_[iova] = DmaMapping{
            .physical_base = physical_base,
            .length = length,
            .direction = direction,
        };
        return iova;
    }

    void unmap(Iova iova) {
        const std::size_t erased = this->mappings_.erase(iova);
        require(erased == 1, "DMA unmap must remove an existing mapping");
    }

    PhysicalAddress translate_for_write(Iova iova, std::size_t offset) const {
        const DmaMapping& mapping = this->mapping_for(iova);
        require(this->allows_device_write(mapping.direction),
            "IOMMU rejected device write direction");
        require(offset < mapping.length, "DMA write offset outside mapping");
        return mapping.physical_base + offset;
    }

private:
    const DmaMapping& mapping_for(Iova iova) const {
        const auto iterator = this->mappings_.find(iova);
        if (iterator == this->mappings_.end()) {
            throw Fault(FaultKind::DmaFault, "IOMMU has no mapping for IOVA");
        }
        return iterator->second;
    }

    bool allows_device_write(DmaDirection direction) const {
        return direction == DmaDirection::FromDevice ||
            direction == DmaDirection::Bidirectional;
    }

    Iova next_iova_ = X86_PAGE_SIZE;
    std::map<Iova, DmaMapping> mappings_;
};

class DmaEngine {
public:
    DmaEngine(IommuDomain& domain, PhysicalMemory& memory)
        : domain_(domain), memory_(memory) {}

    void device_write(Iova iova, std::size_t offset, Byte value) {
        const PhysicalAddress physical = this->domain_.translate_for_write(iova, offset);

```

```

    this->memory_.store8(physical, value);
}

private:
    IommuDomain& domain_;
    PhysicalMemory& memory_;
};

class Cpu {
public:
    Cpu(AddressSpace& address_space, Tlb& tlb, const Mmu& mmu, Bus& bus,
        LocalApic& apic, const InterruptDescriptorTable& idt)
        : address_space_(address_space),
          tlb_(tlb),
          mmu_(mmu),
          bus_(bus),
          apic_(apic),
          idt_(idt) {}

    void enter_user(VirtualAddress rip, VirtualAddress rsp) {
        require(is_canonical(rip), "user RIP must be canonical");
        require(is_canonical(rsp), "user RSP must be canonical");
        this->cpl_ = PrivilegeLevel::Ring3;
        this->rip_ = rip;
        this->rsp_ = rsp;
        this->rflags_ = X86_RFLAGS_INTERRUPT_ENABLE;
    }

    void enter_kernel(VirtualAddress rip) {
        require(is_canonical(rip), "kernel RIP must be canonical");
        this->cpl_ = PrivilegeLevel::Ring0;
        this->rip_ = rip;
    }

    Byte load8(VirtualAddress virtual_address) {
        const Translation translation =
            this->translate_or_trap(virtual_address, AccessKind::Read);
        return this->bus_.load8(translation.physical, this->cpl_);
    }

    void store8(VirtualAddress virtual_address, Byte value) {
        const Translation translation =
            this->translate_or_trap(virtual_address, AccessKind::Write);
        this->bus_.store8(translation.physical, value, this->cpl_);
    }

    void execute_at(VirtualAddress virtual_address) {
        const Translation translation =
            this->translate_or_trap(virtual_address, AccessKind::Execute);
        this->rip_ = virtual_address;
        this->last_fetch_physical_ = translation.physical;
    }

    void syscall() {
        require(this->cpl_ == PrivilegeLevel::Ring3,
            "syscall entry starts in ring 3 for this model");
        this->push_trap_frame(X86_VECTOR_SYSCALL, 0, std::nullopt);
        this->cpl_ = PrivilegeLevel::Ring0;
        this->rip_ = X86_SYSCALL_HANDLER;
        this->rflags_ &= ~X86_RFLAGS_INTERRUPT_ENABLE;
    }

    void handle_one_interrupt() {
        const std::optional<InterruptVector> vector = this->apic_.take_next();
        if (!vector.has_value()) {
            return;
        }
        this->push_trap_frame(*vector, 0, std::nullopt);
        this->cpl_ = PrivilegeLevel::Ring0;
        this->rip_ = this->idt_.handler_for(*vector);
        this->rflags_ &= ~X86_RFLAGS_INTERRUPT_ENABLE;
    }

    void iretq() {
        require(!this->trap_frames_.empty(), "iretq requires a saved frame");
        const TrapFrame frame = this->trap_frames_.back();
        this->trap_frames_.pop_back();
        this->rip_ = frame.rip;
        this->rsp_ = frame.rsp;
        this->rflags_ = frame.rflags;
        this->cpl_ = frame.cpl;
    }

    PrivilegeLevel cpl() const {
        return this->cpl_;
    }

    VirtualAddress rip() const {
        return this->rip_;
    }

    std::size_t trap_count() const {
        return this->trap_frames_.size();
    }

    const TrapFrame& last_trap() const {
        require(!this->trap_frames_.empty(), "trap frame stack must not be empty");
    }

```



```

    return this->trap_frames_.back();
}

PhysicalAddress last_fetch_physical() const {
    return this->last_fetch_physical_;
}

private:
Translation translate_or_trap(VirtualAddress virtual_address, AccessKind access) {
    try {
        return this->mmu_.translate(
            this->address_space_, this->tlb_, virtual_address, access, this->cpl_);
    } catch (const Fault& fault) {
        if (fault.kind == FaultKind::PageFault || fault.kind == FaultKind::NonCanonical) {
            this->push_trap_frame(
                X86_VECTOR_PAGE_FAULT,
                page_fault_error_code(access, this->cpl_),
                virtual_address);
        }
        throw;
    }
}

void push_trap_frame(InterruptVector vector, Word error_code,
    std::optional<VirtualAddress> fault_address) {
    this->trap_frames_.push_back(TrapFrame{
        .rip = this->rip_,
        .rsp = this->rsp_,
        .rflags = this->rflags_,
        .cpl = this->cpl_,
        .vector = vector,
        .error_code = error_code,
        .fault_address = fault_address,
    });
}

AddressSpace& address_space_;
Tlb& tlb_;
const Mmu& mmu_;
Bus& bus_;
LocalApic& apic_;
const InterruptDescriptorTable& idt_;
PrivilegeLevel cpl_ = PrivilegeLevel::Ring3;
VirtualAddress rip_ = TEST_USER_RIP;
VirtualAddress rsp_ = TEST_USER_RSP;
Word rflags_ = X86_RFLAGS_INTERRUPT_ENABLE;
PhysicalAddress last_fetch_physical_ = 0;
std::vector<TrapFrame> trap_frames_;
};

class Machine {
public:
    Machine()
        : address_space_(TEST_USER_PCID),
          device_(apic_),
          bus_(memory_, device_),
          dma_(iommu_, memory_),
          cpu_(address_space_, tlb_, mmu_, bus_, apic_, idt_) {
        this->install_idt();
        this->install_mappings();
        this->cpu_.enter_user(TEST_USER_RIP, TEST_USER_RSP);
    }

    AddressSpace& address_space() {
        return this->address_space_;
    }

    Tlb& tlb() {
        return this->tlb_;
    }

    PhysicalMemory& memory() {
        return this->memory_;
    }

    IommuDomain& iommu() {
        return this->iommu_;
    }

    DmaEngine& dma() {
        return this->dma_;
    }

    Cpu& cpu() {
        return this->cpu_;
    }

    MmioDevice& device() {
        return this->device_;
    }

    LocalApic& apic() {
        return this->apic_;
    }

private:
    void install_idt() {

```

```

    this->idt_.set_handler(X86_VECTOR_TIMER, X86_TIMER_HANDLER);
    this->idt_.set_handler(X86_VECTOR_DEVICE, X86_DEVICE_HANDLER);
}

void install_mappings() {
    this->address_space_.map_page(
        TEST_USER_PAGE,
        page_align(TEST_USER_PA),
        PageAttributes{.writable = true, .user = true, .executable = true});
    this->address_space_.map_page(
        TEST_READ_ONLY_VA,
        page_align(TEST_READ_ONLY_PA),
        PageAttributes{.writable = false, .user = true, .executable = true});
    this->address_space_.map_page(
        TEST_NX_VA,
        page_align(TEST_NX_PA),
        PageAttributes{.writable = true, .user = true, .executable = false});
    this->address_space_.map_page(
        TEST_KERNEL_VA,
        page_align(TEST_KERNEL_PA),
        PageAttributes{.writable = true, .user = false, .executable = true});
    this->address_space_.map_page(
        TEST_MMIO_VA,
        X86_MMIO_BASE,
        PageAttributes{.writable = true, .user = false, .executable = false});
}

PhysicalMemory memory_;
AddressSpace address_space_;
Tlb tlb_;
Mmu mmu_;
LocalApic apic_;
InterruptDescriptorTable idt_;
MmioDevice device_;
Bus bus_;
IommuDomain iommu_;
DmaEngine dma_;
Cpu cpu_;
};

void test_canonical_address_and_indexes() {
    require(is_canonical(TEST_USER_VA), "low canonical user address rejected");
    require(is_canonical(TEST_KERNEL_VA), "high canonical kernel address rejected");
    require(!is_canonical(TEST_NON_CANONICAL_VA),
        "non-canonical address was accepted");
    const PageTableIndexes indexes = split_virtual_address(TEST_USER_VA);
    require(indexes.pml4 == 0, "unexpected user PML4 index");
    require(indexes.offset == (TEST_USER_VA & X86_PAGE_OFFSET_MASK),
        "page offset was not preserved");
    require_fault(FaultKind::NonCanonical, [] {
        static_cast<void*>(split_virtual_address(TEST_NON_CANONICAL_VA));
    });
}

void test_page_permissions_and_fault_frame() {
    Machine machine;
    machine.cpu().store8(TEST_USER_VA, TEST_USER_BYTE);
    require(machine.cpu().load8(TEST_USER_VA) == TEST_USER_BYTE,
        "user page read/write failed");

    require_fault(FaultKind::PageFault, [&machine] {
        machine.cpu().load8(TEST_KERNEL_VA);
    });
    require(machine.cpu().last_trap().vector == X86_VECTOR_PAGE_FAULT,
        "page fault did not save page-fault vector");
    require(machine.cpu().last_trap().fault_address == TEST_KERNEL_VA,
        "page fault did not save faulting virtual address");
    require((machine.cpu().last_trap().error_code & X86_PAGE_FAULT_USER_BIT) != 0,
        "ring 3 page fault did not set user bit");

    require_fault(FaultKind::PageFault, [&machine] {
        machine.cpu().store8(TEST_READ_ONLY_VA, TEST_USER_BYTE);
    });
    require((machine.cpu().last_trap().error_code & X86_PAGE_FAULT_WRITE_BIT) != 0,
        "write fault did not set write bit");

    require_fault(FaultKind::PageFault, [&machine] {
        machine.cpu().execute_at(TEST_NX_VA);
    });
    require((machine.cpu().last_trap().error_code &
        X86_PAGE_FAULT_INSTRUCTION_BIT) != 0,
        "execute fault did not set instruction bit");
}

void test_tlb_invalidation_requirement() {
    Machine machine;
    machine.cpu().store8(TEST_USER_VA, TEST_USER_BYTE);
    require(machine.tlb().size() == 1U, "first store should populate TLB");

    machine.address_space().set_writable(TEST_USER_PAGE, false);
    machine.cpu().store8(TEST_USER_VA, TEST_DEVICE_BYTE);
    require(machine.cpu().load8(TEST_USER_VA) == TEST_DEVICE_BYTE,
        "stale TLB did not preserve old writable permission");

    machine.tlb().invalidate_page(TEST_USER_PCID, virtual_page_number(TEST_USER_VA));
    require_fault(FaultKind::PageFault, [&machine] {
        machine.cpu().store8(TEST_USER_VA, TEST_USER_BYTE);
    });
}

```

```

    });
}

void test_syscall_and_interrupt_entry() {
    Machine machine;
    machine.cpu().syscall();
    require(machine.cpu().cpl() == PrivilegeLevel::Ring0,
        "syscall did not enter ring 0");
    require(machine.cpu().rip() == X86_SYSCALL_HANDLER,
        "syscall did not load IA32_LSTAR-style handler");
    require(machine.cpu().last_trap().vector == X86_VECTOR_SYSCALL,
        "syscall did not save syscall frame");
    machine.cpu().iretq();
    require(machine.cpu().cpl() == PrivilegeLevel::Ring3,
        "iretq did not restore ring 3 after syscall");

    machine.cpu().enter_kernel(X86_SYSCALL_HANDLER);
    machine.cpu().store8(TEST_MMIO_VA, TEST_DEVICE_BYTE);
    require(machine.device().last_value() == TEST_DEVICE_BYTE,
        "MMIO doorbell did not reach device");
    require(machine.apic().pending_count() == 1U,
        "device did not raise MSI-X style interrupt");
    machine.cpu().handle_one_interrupt();
    require(machine.cpu().rip() == X86_DEVICE_HANDLER,
        "device interrupt did not dispatch through IDT");
}

void test_mmio_privilege_and_dma_lifetime() {
    Machine machine;
    require_fault(FaultKind::PageFault, [&machine] {
        machine.cpu().store8(TEST_MMIO_VA, TEST_DEVICE_BYTE);
    });

    const Iova iova = machine.iommu().map(
        page_align(TEST_USER_PA), TEST_DMA_LENGTH, DmaDirection::FromDevice);
    machine.dma().device_write(iova, TEST_DMA_OFFSET, TEST_DEVICE_BYTE);
    require(machine.memory().load8(page_align(TEST_USER_PA) + TEST_DMA_OFFSET) ==
        TEST_DEVICE_BYTE,
        "DMA write did not reach mapped physical page");
    machine.iommu().unmap(iova);
    require_fault(FaultKind::DmaFault, [&machine, iova] {
        machine.dma().device_write(iova, TEST_DMA_OFFSET, TEST_USER_BYTE);
    });
}

int main() {
    test_canonical_address_and_indexes();
    test_page_permissions_and_fault_frame();
    test_tlb_invalidation_requirement();
    test_syscall_and_interrupt_entry();
    test_mmio_privilege_and_dma_lifetime();
    std::cout << "x86-64 stage one contract model checks passed\n";
}

```

第一阶段证据与验收：用可观察现象校准理解

第一阶段的目标是让硬件名词落到可观察现象，而不是让读者背出更长清单。真正理解 CPU、MMU、DMA、中断、cache 和设备队列，应该能在系统上找到证据：日志、计数器、地址映射、页错误、块 I/O、网络队列、调度事件和错误报告。

观察启动链：哪些硬件事实被交给内核

启动日志是硬件到 OS 的第一份证据。它会显示内存地图、CPU 特性、NUMA 拓扑、时钟源、中断控制器、PCIe 设备、IOMMU、存储设备和根文件系统。读启动日志时，不要只找错误，要把它当成硬件契约清单。

```

evidence to collect on a Linux system:
dmesg
cat /proc/cpuinfo
cat /proc/iomem
cat /proc/interrupts
lspci -vv
lsblk
numactl --hardware

```

证据	说明什么	关联章节
/proc/iomem	RAM、保留区、MMIO 窗口	物理内存地图、设备地址

<code>/proc/interrupts</code>	IRQ 分配和计数	中断控制器、设备完成
<code>lspci -vv</code>	BAR、MSI-X、AER、IOMMU group	PCIe、驱动资源
<code>dmesg</code>	probe、错误、时钟、IOMMU、NUMA	启动链、设备模型
<code>numactl --hardware</code>	CPU 与内存 node 距离	NUMA、页放置

这些命令是原理卷的校准工具，不替代实践卷实验。书里说“MMIO 是地址空间的一部分”，`/proc/iomem` 应该能看到设备窗口；书里说“中断有亲和性和计数”，`/proc/interrupts` 应该能看到每个 CPU 上的计数分布。

观察系统调用和特权边界

`strace` 能看到用户态请求了哪些系统调用，但看不到内核内部所有路径。它适合验证“库函数不是系统调用本身”和“系统调用有返回值与错误码”。例如一个 `read` 可能返回短读，一个 `open` 可能因权限失败，一个 `mmap` 可能成功但物理页尚未分配。

```
questions while reading strace:
Which calls cross the user/kernel boundary?
Which file descriptors are created, duplicated, or closed?
Which calls return EAGAIN, EINTR, EFAULT, or EPERM?
Does mmap only create virtual ranges, or also force page allocation?
```

特权边界的负面证据同样重要。用户态传入坏指针时，内核应返回 `EFAULT` 或终止进程，而不是崩溃；无权限打开文件应失败且不产生副作用；被 `seccomp` 拒绝的系统调用不应进入危险路径。

观察虚拟内存：VMA、RSS 和 page fault

进程的虚拟地址空间可以从 `/proc/<pid>/maps` 和 `/proc/<pid>/smaps` 观察。`maps` 显示 VMA，`smaps` 显示更细的 RSS、dirty、shared/private 等统计。`page fault` 可以用 `perf stat` 或进程资源统计观察。

```
memory evidence:
cat /proc/<pid>/maps
cat /proc/<pid>/smaps
perf stat -e page-faults,minor-faults,major-faults <command>
pmap -x <pid>
```

现象	可能原因	需要区分
VIRT 很大 RSS 很小	VMA 已建，页未实际分配	懒分配、文件映射、保留地址空间
minor fault 多	分配零页或建立 PTE	不一定访问磁盘
major fault 多	需要存储 I/O	文件页或 swap 读入
shared 增加	共享库或共享映射	不等于进程独占内存增加
dirty 增加	写过但未回写	page cache 或匿名页

这组证据对应的核心理解是：虚拟地址、VMA、PTE、物理页、RSS 不是同一层概念。一个地址可见，不代表物理页已分配；一个页在 RSS 中，不代表它独占；一个 `page fault` 发生，不代表程序错误。

观察调度：on-CPU 与 off-CPU 分开

调度问题最容易被误判。请求慢可能是 CPU 算得慢，也可能是 `runnable` 但排队，也可能是 `sleeping` 等 I/O 或锁。证据上要分 `on-CPU`、`runnable wait` 和 `blocked wait`。

```
scheduler evidence:
perf sched record/report
trace-cmd record -e sched:*
cat /proc/<pid>/sched
cat /proc/schedstat
pidstat -w
```

若上下文切换次数非常高，可能是线程过多、锁竞争、I/O 小请求过多或时间片太短。若 runnable wait 高，说明任务想运行但拿不到 CPU。若 CPU 使用率低但延迟高，可能大量时间在 I/O、锁或内存回收中睡眠。

观察设备和 I/O：队列、completion、错误

块设备和网络设备都有队列。吞吐不只由设备峰值决定，还由队列深度、请求大小、合并、完成延迟、中断亲和性和 CPU 处理能力共同决定。

```
I/O evidence:
iostat -x
lsblk -o NAME,MODEL,ROTA,SIZE
cat /sys/block/<dev>/queue/*
ethtool -S <iface>
cat /proc/interrupts
dmesg | grep -i -E "nvme|pcie|iommu|dma|reset|timeout"
```

现象	可能原因	硬件/OS 联系
设备利用率高且 await 高	队列拥塞或设备慢	块层请求和设备服务时间
中断集中在一个 CPU	IRQ affinity 不均	多队列和 CPU 拓扑
IOMMU fault reset/timeout 日志	DMA 地址或权限错误 设备或驱动失去响应	DMA mapping 生命周期 in-flight 请求恢复
网络 drop 增加	ring 满或协议栈处理慢	NAPI、socket backlog、应用读取

这些现象应该能回到前文的状态机：请求是否已经提交，设备是否拥有 buffer，completion 是否回来，等待者是否被唤醒，错误是否传播到用户态。

第一阶段负面测试思维

高质量理解必须包含负面路径。只验证正常读写成功，无法证明边界正确。第一阶段可以用纸上或小实验设计下面这些测试：

机制	应注入的失败	期望结果
用户指针检查	传入无效用户地址	返回错误或信号，内核不崩溃
page fault	访问无 VMA 地址	进程失败，不影响其他进程
COW	fork 后父子分别写同一地址	内容分离，互不污染
DMA 生命周期	completion 前释放 buffer 的模拟	测试应能发现所有权错误
中断处理	completion 丢失或迟到	超时路径唤醒等待者并报告错误

设备复位	in-flight 请求未完成时 reset	请求被 retry 或 fail, 不永久睡眠
持久化	write 后崩溃模拟	无 fsync 不承诺持久, 有 fsync 按语义恢复

负面测试的价值是逼迫我们说清楚“不变量”。例如“completion 前不能释放 buffer”来自设备仍可能 DMA 的事实；“PTE 改权限后要失效 TLB”关系到隔离条件；“write 不等于持久”来自 page cache、设备 cache 和崩溃恢复共同决定的语义。

第一阶段最终验收清单

下面的清单可以作为第一阶段完成标准。每一项都要求能讲出硬件来源、内核对象、正常路径、失败路径和可观察证据。

- 能从复位向量讲到内核初始化：固件、内存地图、页表、异常入口、驱动 probe。
- 能从系统调用讲到特权边界：寄存器 ABI、内核栈、用户指针、返回路径。
- 能从一次 load/store 讲到 MMU：VMA、PTE、TLB、fault、COW、权限。
- 能从定时器中断讲到调度：tick、runtime accounting、runqueue、context switch。
- 能从 DMA 提交讲到 completion：描述符、屏障、IOMMU、中断、等待队列。
- 能从 file read 讲到硬件设备：fd、page cache、filesystem、block layer、NVMe。
- 能从网络包讲到 socket：RX ring、NAPI、skb、TCP receive queue、epoll。
- 能从 write 讲到崩溃恢复：dirty folio、writeback、journal、flush、fsync。
- 能解释多核带来的成本：cache line、锁竞争、IPI、TLB shutdown、NUMA。
- 能用至少一种系统证据验证自己的解释，而不是只停留在定义。

这份清单完成后，第一阶段才算过关。后续进入进程、内存、文件系统和网络时，读者已经有一条硬件到 OS 的主线，不会把每章当成孤立概念。

硬件基本信息：操作系统依赖的机器事实

操作系统不是在抽象空间里运行的。它依赖一组非常具体的硬件事实：CPU 怎样取指和切换特权级，内存怎样寻址和缓存，设备怎样通过寄存器、中断和 DMA 与 CPU 通信，定时器怎样提供时间基准，启动固件怎样把第一段代码交给内核。若这些事实不清楚，后面的进程、虚拟内存、驱动、文件系统和安全机制都会变成悬空概念。

核心思想

硬件给 OS 的不是“进程、文件、socket”，而是指令、寄存器、地址、中断、页表、缓存、总线和设备寄存器。OS 的工作是把这些硬件事实组织成可验证的契约。

CPU 执行模型

CPU 的基本循环是取指、译码、执行、提交结果。程序计数器指向下一条指令；通用寄存器保存整数、地址和临时值；状态寄存器保存中断使能、条件码、当前特权级等控制位；栈指针指向当前调用栈。操作系统进行上下文切换时，最少要保存足以恢复执行的这组状态。

硬件状态	作用	OS 关心点
程序计数器	指向下一条指令	中断、异常、调度后从哪里继续
栈指针	指向当前栈顶	用户栈和内核栈必须分开管理
通用寄存器	保存参数和临时值	系统调用参数、返回值、上下文保存
状态寄存器	保存特权级、中断位、标志位	返回用户态前必须恢复正确权限
控制寄存器	指向页表、控制 MMU 和中断	只有内核能修改关键控制状态

现代 CPU 还会乱序执行、预测分支、合并内存访问。多数时候程序看见的是架构保证的顺序，但内核和驱动不能忽略内存屏障。比如驱动必须先写完 DMA 描述符，再敲 doorbell 通知设备；锁实现必须用 acquire/release 语义，否则另一个 CPU 可能看到锁已释放却看不到临界区内的数据更新。

CPU 设计为什么会分成取指、译码和执行

取指、译码和执行不是教材为了好记而拆出的词，而是硬件实现上的自然分工。取指单元只关心 RIP、指令 cache 和下一条地址；译码单元把二进制指令解释成内部控制信号；执行单元使用 ALU、分支单元、load/store 单元或浮点/SIMD 单元完成动作。把这些职责分开，CPU 才能流水化、预测和并行发射。

```
front end:
  fetch bytes at RIP
  predict next RIP
  decode bytes into operations

back end:
  read registers
  execute ALU/branch/load/store
  access cache or memory
  commit architectural state
```

OS 关心这种划分有三个原因。第一，异常通常在执行或访存阶段才发现，但必须回报给架构层一个精确 RIP。第二，指令 cache、数据 cache 和 TLB 的冷热会影响系统调用、上下文切换和缺页性能。第三，分支预测和乱序执行虽然不改变正常架构语义，却会影响安全隔离和侧信道防护。

寄存器为什么不只是变量

寄存器是 CPU 中软件可见的最快状态。普通变量可以在内存里，也可以被编译器临时放进寄存器；但寄存器本身属于架构状态，上下文切换、中断返回、系统调用 ABI 都要明确处理。若内核切换线程时漏保存一个被用户程序依赖的寄存器，用户程序会在看似随机的位置计算错误。

寄存器类别	例子	OS 影响
通用寄存器	参数、返回值、临时整数	trap frame、系统调用参数、上下文切换
栈指针	用户栈和内核栈位置	特权切换必须换到可信内核栈
程序计数器	下一条或异常指令地址	信号、调试、异常恢复
状态寄存器	中断位、条件码、模式位	返回用户态前必须恢复受控状态

控制寄存器	页表基址、MMU 开关、系统控制	只能由内核或更高特权层修改
向量/浮点寄存器	SIMD、浮点状态	惰性保存、信号帧、ABI 兼容

浮点和 SIMD 寄存器尤其说明“保存上下文”并不免费。它们数量多、宽度大，频繁保存会拖慢切换；惰性保存又要处理异常、信号和内核使用 SIMD 的边界。现代内核通常谨慎限制内核态浮点/SIMD 使用，原因不是风格，而是上下文状态太大。

特权级、异常和系统调用

用户程序不能直接修改页表、关中断或访问任意设备寄存器。CPU 用特权级区分普通代码和内核代码。用户态执行特权指令会触发异常；系统调用是用户态主动进入内核态的受控入口；中断是设备或定时器异步打断 CPU 的入口；异常是当前指令执行失败产生的入口。

```
user mode:
  execute normal instructions
  syscall -> controlled entry to kernel
  page fault -> exception entry to kernel
  timer interrupt -> asynchronous entry to kernel

kernel mode:
  validate context
  inspect object and permission
  update kernel state
  return to user or schedule another task
```

特权级解释了为什么系统调用不是普通函数调用。普通函数调用仍然在同一信任边界里；系统调用跨越用户/内核边界，必须切换栈、保存 trap frame、检查用户指针、处理信号和抢占，再安全返回。

特权机器的最小硬件清单

要让特权级真正有用，CPU 不能只提供一个“当前模式”标志。它至少需要提供五类硬件支持：受保护的 control 寄存器、受控的陷入入口、返回用户态的指令、异常原因记录、以及对内存访问的权限检查。

硬件能力	解决的问题	缺失后果
模式位/特权级	区分用户代码和内核代码	用户可执行所有敏感指令
陷入向量表	限制用户只能从固定入口进入内核	用户可跳到内核任意位置
异常原因寄存器	告诉内核为何进入	无法区分缺页、非法指令和系统调用
内核栈切换	避免使用不可信用户栈处理内核逻辑	用户栈损坏可破坏内核入口
页表权限检查	限制用户访问地址范围和权限	进程可读写内核和其他进程

这也解释了为什么“把库函数写得安全一点”不能代替内核边界。库函数仍然运行在用户态，用户可以绕过库直接发指令。只有 CPU 在每条敏感路径上强制检查，OS 才能把不可信程序约束在用户态。

异常、陷入和中断的共同结构

异常、陷入和中断来源不同，但入口结构相似：CPU 停下当前控制流，保存返回所需状态，切换到受信任入口，把原因交给内核，内核处理后决定返回、杀死任务、切换任务或 panic。

```

hardware_entry(reason):
    save return RIP and status
    if crossing privilege boundary:
        switch to kernel stack
    set current privilege to kernel
    jump to vector[reason]

kernel_entry:
    save general registers
    build trap frame
    dispatch by reason
    decide return or schedule

```

入口路径必须极小心，因为它在系统最脆弱的位置运行。页表可能刚刚出错，用户指针不可信，中断状态敏感，栈也可能需要切换。许多架构把入口的一部分固定在汇编里，就是为了精确控制寄存器保存和返回协议。

内存、地址总线与页表

CPU 发出的地址通常先是虚拟地址。MMU 根据页表把虚拟页翻译成物理页，并检查 R/W/X、用户/内核、present 等权限。页表基址保存在控制寄存器中；TLB 缓存最近翻译，减少页表遍历成本。操作系统通过切换页表和设置权限，让每个进程看到自己的地址空间。

概念	含义	常见错误理解
虚拟地址	程序使用的地址	误以为一定已经占用物理内存
物理地址	内存控制器看到的地址	误以为用户程序能直接使用
页表项	虚拟页到物理页和权限的记录	只当作普通 map，忽略权限
TLB	地址翻译缓存	修改页表后忘记失效旧翻译
VMA	合法虚拟区间的内核记录	误以为没有 PTE 就一定非法

页表是隔离的硬件根。没有页表权限，进程可以互相读写；没有用户/内核页区分，用户程序可以访问内核数据；没有 NX，数据页可能被当作代码执行。OS 的内存管理不是“分配内存”这么简单，而是在性能、隔离、共享、懒加载和回收之间维护一组不变量。

从物理地址到虚拟地址的演进

最早的简单机器可以让程序直接使用物理地址。这样最容易理解：地址 0x1000 就是某个内存芯片上的位置。但多程序系统里，直接物理地址有三个硬伤。第一，程序彼此知道并可能覆盖对方地址。第二，程序必须按实际物理位置链接和装载。第三，内存碎片会让大程序难以找到连续空间。

虚拟地址把程序看到的地址和内存条上的物理地址分开。每个进程都可以以为自己从固定地址开始，有连续的栈、堆和映射区；内核用页表决定这些虚拟页实际落到哪些物理页，或者暂时不落到任何物理页。

阶段	能力	局限
直接物理地址 基址/界限 分段	实现最简单，裸机常见 一个程序一段连续保护区 不同段不同基址和权限	无隔离，装载位置僵硬 不适合稀疏地址和共享库 外部碎片和模型复杂

分页	固定大小页、独立权限和懒映射	页表/TLB 成本、内部碎片
多级分页	稀疏地址空间节省页表内存	page walk 更复杂，需 TLB 加速

分页之所以成为现代通用 OS 的核心，是因为它把“地址连续幻觉”和“物理页离散管理”解耦。进程堆可以在虚拟地址上连续增长，物理页却可以来自任意空闲页框；共享库可以映射到多个进程；fork 可以先共享物理页，等写入时再 COW。

页表项为什么要有这么多位

初学者容易把页表项看成 vpn -> ppn 的字典项。真实页表项还包含权限、状态和性能位，因为硬件必须在每次地址翻译时直接做决定。

位或字段	含义	OS 用法
present/valid	翻译是否有效	缺页、懒分配、换出
physical page number	指向物理页框	页分配和共享的实际对象
read/write/execute	允许哪些访问	代码保护、COW、NX 防护
user/supervisor	用户态是否可访问	内核页和用户页隔离
accessed	最近是否访问	页面回收近似 LRU
dirty	是否写过	文件页 writeback、匿名页换出
global/PCID	TLB 保留或地址空间标识	降低切换刷新成本

这些位让 OS 能把策略推迟到事件发生时。例如 mmap 文件时可以只建 VMA，不立刻读文件；第一次访问触发缺页，内核再读取对应页并安装 PTE。COW 也靠权限位实现：父子进程先共享只读页，写入触发保护异常，内核复制物理页并恢复可写权限。

缓存层次与一致性

CPU 访问寄存器最快，其次是 L1/L2/L3 cache，再往后是内存和存储设备。cache 以 cache line 为单位搬运数据。多核系统中，同一个物理地址可能被多个 CPU cache 缓存，硬件一致性协议负责让普通内存读写最终保持一致。但设备 DMA、MMIO 和某些弱内存模型仍要求内核显式使用屏障和 cache 维护操作。

层次	特点	OS 影响
寄存器	每个 CPU 私有，最快	上下文切换必须保存可见寄存器
L1/L2/L3 cache	以 cache line 缓存内存	锁竞争、伪共享、迁移成本来自这里
主内存	所有 CPU 共享	页分配、NUMA、DMA buffer 来自这里
存储设备	持久但慢	page cache、writeback、fsync 用来隐藏和约束它

缓存解释了许多 OS 设计选择。每 CPU runqueue 减少共享 cache line 抖动；per-CPU slab cache 减少全局锁；任务迁移会损失 cache/TLB 热状态；DMA 写入内存后，CPU 可能需要 invalidate cache 才能看到设备写入的新数据。

cache line、伪共享和锁为什么会慢

cache 不是按单个字节搬运，而是按 cache line 搬运，常见大小是 64 字节量级。若两个 CPU 反复写同一 cache line 上的不同变量，硬件仍必须在两个 CPU 之间转移这条 line 的独占所有权。这叫伪共享：程序逻辑上没有共享同一个变量，硬件上却共享了同一个 cache line。

```
struct BadCounter {
    atomic<uint64_t> cpu0_count;
    atomic<uint64_t> cpu1_count; // likely same cache line
};

struct BetterCounter {
    alignas(64) atomic<uint64_t> cpu0_count;
    alignas(64) atomic<uint64_t> cpu1_count;
};
```

OS 大量使用 per-CPU 数据，就是为了避免所有 CPU 争同一条 cache line。调度器每 CPU runqueue、网络每队列统计、slab 每 CPU 缓存，本质上都是把热写路径分散到局部状态，必要时再汇总。锁慢也常不是临界区代码慢，而是锁变量所在 cache line 在 CPU 间高速迁移。

内存屏障到底在约束谁

内存屏障约束的不是“C 语言语句顺序”这么简单。它同时约束编译器优化、CPU store buffer、cache 一致性可见顺序以及设备观察顺序。普通用户程序通常用高级原子语义间接表达；内核和驱动经常必须直接选择 acquire、release、full barrier、DMA barrier 或 MMIO barrier。

场景	需要的顺序	典型工具
发布对象指针	对象字段先可见，指针后可见	store-release/load-acquire
释放锁	临界区写入先可见，锁状态后释放	unlock release 语义
提交 DMA 描述符	描述符内存先对设备可见，再 doorbell	dma_wmb/MMIO write barrier
读取设备完成	先看到 completion，再读取设备写入数据	dma_rmb/cache invalidate

屏障不是越多越好。多余屏障会拖慢热路径；缺失屏障会造成难复现的并发错误。正确做法是先写出“谁发布什么，谁观察什么，观察到标志后必须看到哪些数据”，再选择最弱但足够的顺序约束。

总线、MMIO 和设备寄存器

设备通过总线连接到 CPU 和内存。很多设备把控制寄存器映射到物理地址空间，CPU 通过 MMIO 读写它们。MMIO 不是普通内存：读取可能有副作用，写入顺序可能被设备解释为命令序列，编译器和 CPU 不能随意优化这些访问。

```
driver_init(dev):
    writel(dev.mmio + RESET, 1)
    wait_until(readl(dev.mmio + STATUS) & READY)
    writel(dev.mmio + IRQ_ENABLE, RX_DONE | TX_DONE)
```

驱动必须给轮询加超时，因为设备可能坏掉。驱动也必须理解寄存器语义：有些状态位写 1 清除，有些寄存器读一次就弹出队列，有些寄存器要求先写高位再写低位。OS 不能把设备当作可靠函数调用，它面对的是异步硬件状态机。

芯片如何被 OS 发现

单片机工程里，外设地址通常写在芯片手册和头文件中；通用 OS 不能为每块主板硬编码所有设备。在本书的 x86-64 主线里，它需要从 UEFI/ACPI 和总线协议中发现硬件：ACPI 描述板级设备和电源关系，PCI/PCIe 配置空间描述可枚举设备，设备的 BAR 告诉内核 MMIO 区间，中断属性告诉内核如何接收事件，DMA mask 告诉内核设备能访问多少位地址。


```
discover device:
  read firmware table or bus config space
  identify vendor/device/compatible string
  reserve MMIO region
  map MMIO into kernel virtual address
  allocate interrupt vector
  set DMA mask and IOMMU domain
  call matching driver probe()
```

这条路径解释了驱动 probe 为什么复杂。驱动不是拿到一个“设备对象”就能工作，它要把硬件描述转换成受内核管理的资源，并在任一步失败时回滚。MMIO、IRQ、DMA、clock、reset、power domain 和上层注册对象都属于生命周期的一部分。

中断、定时器和时钟源

中断让设备主动通知 CPU。定时器中断给 OS 提供抢占和超时能力；高精度时钟源提供时间戳；watchdog 检测系统是否长时间没有前进。没有定时器，任务死循环时系统无法抢回 CPU；没有中断，驱动只能轮询设备，浪费 CPU 并增加延迟。

硬件机制	OS 用途	失败表现
外设中断	I/O completion、网络包、设备错误	中断风暴、丢中断、延迟尖峰
定时器中断	时间片、超时、周期任务	任务霸占 CPU，超时不触发
时钟源	时间戳、性能统计、调度账本	时间倒退或漂移导致判断错误
watchdog	检测软锁死和硬锁死	无响应时缺少故障证据

中断处理要短，因为它打断了当前执行流。复杂工作通常放到下半部、工作队列或内核线程。这个原则不是风格问题，而是延迟和可抢占性的硬边界。

定时器为什么是抢占的硬件根

协作式系统依赖任务主动让出 CPU。只要当前任务不调用 `yield`，系统就无法切换。定时器打破了这个假设：硬件在未来某个时间点强制产生中断，CPU 无论当前运行哪个用户程序，都必须进入内核入口。内核因此获得重新选择任务的机会。

```
timer interrupt:
  save current user/kernel context
  account elapsed time to current task
  run expired timers
  if current task exhausted slice:
    mark need_resched
  return path invokes scheduler if needed
```

定时器也支撑超时语义。睡眠、socket 超时、锁等待超时、TCP 重传、块设备请求超时、watchdog 都需要“未来某时刻再回来检查”。没有可靠时间源，OS 只能处理已经发生的事件，无法给等待设置边界。

时钟源和时钟事件设备

OS 通常区分 clock source 和 clock event device。clock source 用来读取当前时间或单调计数；clock event device 用来安排下一次中断。前者回答“现在是多少”，后者回答“什么时候叫醒我”。

类别	用途	错误影响
clock source	时间戳、调度统计、超时比较	时间倒退、漂移、统计错误
clock event	周期 tick、hrtimer、抢占	超时不触发、任务不被抢占
watchdog	检测长期无进展	死锁或关中断过久无证据

tickless 内核不是没有定时器，而是不再固定频率无条件打断 CPU。内核计算最近的必要事件，把硬件定时器设置到那个时间点。这能降低空闲功耗和噪声，但要求调度、RCU、负载统计和 hrtimer 更精确地维护时间状态。

DMA 与 IOMMU

PIO 由 CPU 搬运数据，简单但慢。DMA 让设备直接读写内存，CPU 只准备描述符、buffer 和地址映射。DMA 的难点是：设备访问的是 DMA 地址，不一定等同于 CPU 物理地址；buffer 在设备完成前不能释放；cache 可能需要 clean/invalidate；IOMMU 需要建立和拆除设备页表映射。

```
submit_dma(req):
    req.page = pin_user_or_kernel_page(req.buffer)
    req.dma = dma_map(dev, req.page, req.len, TO_DEVICE)
    fill_descriptor(req.dma, req.len)
    memory_barrier()
    ring_doorbell(dev)
```

DMA 是高性能 I/O 的基础，也是内核 bug 的高风险区域。一个错误的 DMA 地址可能让设备覆盖内核对象；一个过早释放的 buffer 可能被设备迟到写入；一个缺失的 cache invalidate 可能让 CPU 读到旧数据。

IOMMU：给设备也加上页表

MMU 约束 CPU 访问内存，IOMMU 约束设备访问内存。没有 IOMMU 时，设备拿到的总线地址可能直接对应物理内存，驱动错误或恶意设备可以 DMA 到任意位置。有 IOMMU 后，内核为每个设备或设备组建立 DMA 地址空间，只映射允许访问的 buffer。

```
dma_map(dev, page, len, direction):
    iova = allocate_device_virtual_address(dev.domain, len)
    iommu_map(dev.domain, iova, page.phys, len, permissions(direction))
    return iova

dma_unmap(dev, iova, len):
    iommu_unmap(dev.domain, iova, len)
    free_device_virtual_address(iova)
```

IOMMU 的代价是映射维护和 IOTLB 成本。高性能驱动会批量映射、复用 DMA buffer 或使用页池，减少每包 map/unmap。虚拟化场景中，IOMMU 还用于设备直通隔离：把设备 DMA 限制在某个虚拟机拥有的内存范围内。

DMA 和用户页：pin 不是复制

当用户态提交异步 I/O 时，内核可以复制用户数据到内核 buffer，也可以 pin 用户页让设备直接 DMA。pin 的意思是这些物理页在 I/O 完成前不能被回收、迁移或换出。它避免复制，但把用户内存生命周期和设备生命周期绑在一起。

策略	优点	代价
复制到内核 buffer	生命周期简单，用户页可立即变化	多一次内存复制
pin 用户页 DMA	少复制，适合大 I/O	页不能迁移/回收，completion 前必须保持有效
共享 ring buffer	系统调用少，批量提交完成	需要严格所有权和内存顺序协议

这解释了为什么高性能 I/O 接口往往要显式注册 buffer、固定队列大小、区分 submit 和 completion。性能不是免费来的，它把简单的同步复制变成了异步所有权协议。

启动固件和第一条内核指令

机器上电后，CPU 不会直接运行 C 语言内核。固件先初始化基本硬件，bootloader 把内核镜像和启动参数放入内存，然后跳到内核入口。内核早期代码要设置栈、清 BSS、复制 data、建立早期页表、解析内存地图、初始化 console 和分配器。

```
PowerOn
-> Firmware
-> Bootloader
-> KernelEntry
-> StackReady
-> BssCleared
-> EarlyConsole
-> PageTablesReady
-> SchedulerReady
-> FirstUserProcess
```

启动阶段的约束最严格：还没有完整内存分配器，不能随意分配；还没有调度器，不能睡眠；还没有完整日志，错误证据有限；还没有完整设备模型，很多硬件不能访问。理解这些限制，后面读内核初始化代码才不会把“为什么这么麻烦”误解成历史包袱。

硬件细节深讲：从一条指令到一次设备完成

很多操作系统书厚，不是因为它们列了更多名词，而是因为它们会反复把同一个机制放到不同层次里解释。本节从硬件层展开，说明 OS 为什么必须关心指令执行、内存层次、设备协议和启动约束。

一条 load 指令背后发生了什么

用户程序写下一句 `x = *p`，看起来只是读取内存。硬件实际要经过多个检查和缓存层次。CPU 先得到虚拟地址，TLB 查询是否有缓存的翻译；若 TLB 未命中，硬件页表遍历器读取多级页表；页表项必须 present，并允许当前特权级读取；然后物理地址进入 cache 层次，L1 未命中查 L2，L2 未命中查 L3，最后才可能访问内存控制器。

```
load virtual_address:
    translation = tlb_lookup(virtual_address)
    if translation.miss:
        pte = page_table_walk(current.cr3, virtual_address)
        if not pte.present or not pte.readable:
            raise_page_fault()
        tlb_insert(virtual_address, pte)
    physical = pte.physical_page + page_offset(virtual_address)
    line = cache_lookup(physical)
    if line.miss:
        line = memory_read(cache_line_address(physical))
    return extract_word(line, physical)
```

这条路径解释了几个 OS 事实。第一，页表权限是硬件执行的，不是内核每次手工检查。第二，TLB 让地址翻译变快，但也让页表修改必须配合失效。第三，cache line 让相邻数据访问更快，也让伪共享和锁竞争变得昂贵。第四，缺页异常不是函数调用，而是 CPU 在执行普通指令时发现无法继续，转入内核处理。

一次 store 为什么需要内存屏障

普通程序通常假设代码顺序就是别人看到的顺序。多核和设备环境中，这个假设不总成立。CPU 可以把 store 暂存在 store buffer，编译器可以重排不相关内存访问，设备可能通过总线异步观察内存。内核因此使用 memory barrier 表达顺序约束。

```

publish_object(obj):
    obj.field1 = value1
    obj.field2 = value2
    store_release(global_ptr, obj)

read_object():
    obj = load_acquire(global_ptr)
    if obj != null:
        use(obj.field1, obj.field2)

```

release/acquire 的意思是：发布者在指针可见前完成对象初始化；读取者看到指针后，也能看到发布者之前写入的字段。锁、RCU、无锁队列和驱动描述符都依赖类似规则。没有屏障，bug 可能只在特定 CPU、特定优化级别、特定负载下出现。

NUMA：内存不是均匀的

多路服务器上，CPU 插槽和内存控制器形成 NUMA node。本地内存访问比远端内存低延迟、高带宽。调度器、页分配器和内存回收因此不能只把所有 CPU 和内存看成一个平面池。

现象	原因	OS 对策
线程迁移后变慢	cache 和 TLB 热状态丢失	CPU affinity、迁移成本估计
访问远端内存慢	内存挂在另一个 NUMA node	first-touch、NUMA-aware allocation
局部内存耗尽	某 node 压力更高	本地回收、跨 node fallback
锁竞争放大	cache line 在插槽间移动	per-CPU 数据、分片锁、RCU

这也是为什么“线程数等于核心数”不是万能规则。若线程频繁共享同一批 cache line，即使核心很多，也可能被 cache coherence 拖慢。

MMIO、PIO 和 DMA 的差别

设备 I/O 有三种典型方式。PIO 由 CPU 直接搬运数据，适合简单设备或早期启动。MMIO 让 CPU 通过地址读写设备寄存器，适合控制设备状态。DMA 让设备直接访问内存，适合大吞吐数据传输。

方式	数据路径	OS 难点
PIO	CPU 读写数据端口或寄存器	CPU 被搬运占用，吞吐低
MMIO	CPU 读写设备寄存器	访问顺序、副作用、超时、错误状态
DMA	设备直接读写内存	buffer pin、IOMMU、cache 一致性、completion

一个块设备写请求可能同时使用三者：驱动用 MMIO 配置队列，用 DMA 传输数据，用中断报告 completion。OS 教材如果只讲“设备读写”，就会漏掉驱动里最容易出错的生命周期和顺序问题。

启动时为什么不能使用完整内核能力

启动早期没有堆、没有调度、没有完整页表、没有普通文件系统，也不一定有中断。很多初学者会问：为什么不一开始就调用统一日志、统一分配器、统一驱动框架？原因是这些框架本身依赖更基础的能力。

```
early code can use:
```

```
fixed stack
linker symbols
bootloader memory map
early console
bump allocator

early code cannot assume:
  scheduler exists
  page allocator is ready
  interrupts are enabled
  file system is mounted
  all CPUs are online
```

启动代码的复杂性不是历史负担，而是依赖关系决定的。一个成熟内核会把初始
化分层：先建立能输出错误的 console，再建立能描述内存的 boot memory，再建
立页表和分配器，最后建立调度器、设备和用户态。

硬件基础检查题

- 1. 为什么修改页表后还要处理 TLB，而不是只改内存中的页表项？
- 2. 为什么驱动写完描述符后要有 memory barrier 再通知设备？
- 3. 为什么中断处理函数不能随意睡眠？
- 4. 为什么 DMA buffer 在 completion 前不能释放？
- 5. 为什么多核调度不能只用一个全局 runqueue？
- 6. 为什么启动早期不能调用普通 malloc 或普通文件 I/O？

能回答这些问题，后面的虚拟内存、调度、驱动和文件系统才有硬件根基。

时间尺度：为什么 OS 总在权衡

操作系统的大量设计都是在不同时间尺度之间权衡。寄存器访问可以看作 1 个
CPU 周期量级，L1 cache 是几个周期，内存可能是上百周期，NVMe I/O 可能是微秒
到毫秒，网络和机械磁盘可能更慢。调度器一次上下文切换、一次 TLB shutdown、
一次 fsync 都不是“抽象操作”，它们背后有硬件成本。

操作	典型尺度	OS 设计影响
寄存器访问	CPU 周期	上下文切换要保存恢复寄存器
L1/L2 cache 命中	几到几十周期	数据局部性、per-CPU 数据结构重要
主内存访问	上百周期量级	page fault 和 cache miss 成本明显
TLB shutdown	跨 CPU 中断和等待	频繁改页表会拖慢多核程序
NVMe I/O	微秒到毫秒	需要队列、异步、批量和 completion
fsync/flush	可能毫秒级或更高	可靠提交不能放在热路径随意调用

这些数字不是让读者背精确值，而是建立数量级直觉。一个算法少一次内存访问
可能值得优化；一个功能如果每个请求都 fsync，就可能把吞吐限制在存储设备
flush 延迟上；一个锁如果让 cache line 在 CPU 间来回迁移，会比临界区本身更
贵。

一个完整硬件路径：网卡收到一个包

以网卡接收数据包为例，可以把硬件和 OS 串起来。

1. driver allocates RX buffers
2. driver maps buffers for DMA
3. driver fills RX descriptor ring
4. NIC receives packet from wire
5. NIC DMA-writes packet into memory
6. NIC writes completion/status
7. NIC raises interrupt or waits for interrupt moderation
8. interrupt handler schedules poll
9. poll reads descriptors, invalidates cache if needed
10. network stack consumes packet
11. socket receive queue wakes user process

这条路径里任何一步都可能出错：buffer 太少会丢包；DMA 映射错误会写坏内存；中断合并过大增加延迟；poll budget 太大影响调度；用户进程不读 socket 会导致接收队列积压并产生背压。教材要讲厚，就必须把这样的路径完整展开，而不是只说“网卡通过中断通知内核”。

从单片机循环到通用操作系统

理解操作系统最好不要从“Linux 有哪些子系统”开始，而要从最小机器开始。一个简单单片机程序通常只有一个主循环和若干中断处理函数。它没有用户态和内核态之分，没有页表，没有进程，也没有文件系统。程序直接读写外设寄存器，所有代码处在同一信任边界内。

```
bare_metal_main():
    init_clock()
    init_gpio()
    init_uart()
    while true:
        poll_button()
        update_led()
        handle_uart_bytes()
```

这种模型能工作，是因为问题规模小：任务数量少，开发者知道所有代码，内存布局固定，失败影响有限。一旦系统要同时运行多个互不信任程序，或要让一个程序卡住时不拖死全部功能，就需要更强的机制。

阶段	新需求	产生的 OS 机制
单循环裸机	周期性处理少量设备	主循环、轮询、少量中断
中断驱动裸机	外设事件不能丢	中断向量、临界区、环形缓冲区
协作式多任务	多个任务共享 CPU	任务控制块、显式 yield、简单栈切换
抢占式内核	任务不能长期霸占 CPU	定时器中断、调度器、保存恢复上下文
多程序系统	程序互不信任	特权级、系统调用、页表、权限检查
通用系统	设备和文件很多	驱动模型、VFS、块层、网络栈、日志恢复

这条演进线解释了为什么 OS 特性不是凭空增加的。调度器来自“不能信任任务主动让出 CPU”；系统调来自“用户程序不能直接改内核状态”；虚拟内存来自“程序不能互相读写”；文件系统来自“存储设备不能只暴露原始扇区”；网络栈来自“设备字节流必须变成可复用通信接口”。

协作式任务为什么不够

协作式任务把每个任务写成能主动让出的函数。它比单循环清晰，但它依赖每个任务都守规矩。一个任务忘记调用 `yield`，或者在循环中等待硬件状态，就会让系统停止响应。

```
task_a():
    while true:
        do_a_small_step()
        yield()

task_b_bug():
    while device_not_ready():
        ;    // forgot to yield, all other tasks starve
```

抢占式调度用定时器中断打破这个假设。即使当前任务不合作，硬件也会在时间片到达时进入内核。内核保存当前寄存器，选择另一个 `runnable` 任务，再恢复其寄存器。这就是“硬件定时器 + trap frame + scheduler”共同形成的控制权回收机制。

trap frame：异常入口的最小证据

CPU 进入内核时，必须留下足够信息，让内核知道从哪里来、为什么来、如何返回。这个保存下来的状态通常称为 `trap frame` 或 `interrupt frame`。不同架构字段不同，但语义类似。

字段	含义	用途
RIP	被打断或出错的指令位置	返回、信号、调试栈
RSP	进入异常前的栈指针	恢复用户栈或诊断坏栈
RFLAGS	中断位、条件码、特权级信息	判断返回权限和中断状态
通用寄存器	参数和临时值	系统调用参数、恢复用户执行
错误码	page fault、保护异常等附加原因	区分缺页、权限和非法访问

`trap frame` 解释了系统调用和异常为何能返回到原程序。它也解释了为什么内核 `panic` 时要打印寄存器和指令地址：这些字段是定位错误路径的最低成本证据。

```
trap_entry(vector, hardware_frame):
    switch_to_kernel_stack_if_from_user()
    save_general_registers()
    tf = current.trap_frame
    dispatch_trap(vector, tf)
    restore_general_registers()
    return_from_trap(tf)
```

入口代码必须非常保守。它运行在栈切换、页表边界和中断状态都敏感的位置，不能调用可能睡眠的复杂函数。很多内核把低级入口代码写成汇编或极薄的 C 包装，就是为了精确控制保存顺序和 ABI。

中断控制器、优先级和 IPI

设备不是直接“喊 CPU”。多数机器有中断控制器负责接收设备中断、屏蔽中断、设置优先级并把中断送到某个 CPU。多核系统还需要 CPU 之间互相发送 IPI，用于重调度、TLB shutdown、停止其他 CPU 或调试。

机制	用途	OS 风险
interrupt mask	临时屏蔽某类中断	屏蔽过久造成延迟或丢事件

interrupt priority	高优先级事件先处理	优先级反转或低优先级饥饿
interrupt affinity	把设备中断送到指定 CPU	热点 CPU 过载或缓存局部性变差
MSI/MSI-X	设备用内存写触发中断消息	向量分配、隔离和设备错误处理
IPI	CPU 间发送控制事件	频繁 IPI 会破坏多核伸缩性

TLB shutdown 是 IPI 的典型用途。一个 CPU 修改了某进程页表，其他正在运行该进程的 CPU 可能仍有旧 TLB 项。修改方必须通知这些 CPU 失效旧翻译，并等待确认后才能释放旧物理页。这个过程昂贵，所以现代内核会尽量批量处理 unmap 和权限变化。

内存模型：编译器顺序、CPU 顺序和设备顺序

很多并发错误来自把三种顺序混在一起。源代码顺序是程序员写下的顺序；编译器可能在不改变单线程语义的前提下重排；CPU 可能因为 store buffer、乱序执行和缓存协议让其他 CPU 以不同时间观察到写入；设备通过 DMA 和 MMIO 观察到的顺序还要额外受总线和设备协议影响。

顺序层次	例子	常用约束
编译器顺序	普通变量访问被优化或重排	volatile 语义、编译器屏障、原子操作
CPU 顺序	store 先进入缓冲区，稍后对其他核可见	acquire/release、full fence
设备顺序	描述符写入和 doorbell MMIO 顺序	DMA barrier、MMIO write barrier

驱动发布 DMA 描述符时，必须先让内存中的描述符对设备可见，再写 MMIO doorbell。锁释放时，必须先让临界区写入对其他 CPU 可见，再让锁状态变成未持有。RCU 发布指针时，必须先初始化对象，再发布地址。这些看似不同的场景，本质都是“先写数据，再发布可见性”的顺序要求。

硬件抽象的边界

OS 会抽象硬件，但不能把硬件成本抹掉。页表让每个进程看到独立地址空间，但 TLB shutdown 仍然昂贵；block layer 把设备包装成请求队列，但 flush 仍然慢；socket 把网卡包装成字节流，但拥塞和丢包仍然存在；调度器把 CPU 时间切片给线程，但 cache/TLB 热状态仍然影响性能。

学习硬件基础的验收标准：看到一个 OS 机制时，能回答它依赖哪个硬件入口、改变哪些硬件状态、失败时硬件给出什么证据、性能瓶颈来自哪个硬件层次。

本章小结

- CPU 特权级和页表权限是 OS 隔离的硬件根。
- cache、TLB、NUMA 和内存屏障解释了内核为何重视局部性和顺序。
- MMIO、DMA、中断和 IOMMU 构成设备驱动的基本硬件接口。
- 启动阶段受依赖限制，不能使用尚未初始化的内核服务。
- 所有高级 OS 机制都能追溯到硬件状态、硬件事件和硬件成本。

操作系统地图与契约

原理

操作系统的第一性问题不是“提供哪些 API”，而是“怎样让多个不可信、会失败、会竞争的程序共享同一台机器”。硬件只提供有限的 CPU 核、寄存器、物理内存、cache、TLB、页表、中断、总线、设备寄存器、DMA 和特权级。应用希望看到的是进程、线程、虚拟地址、文件、socket、权限、时间和错误码。操作系统处在中间，把硬件事实翻译成可编程契约。

一个契约至少包含四层含义。第一，状态由谁持有，例如进程表、页表、文件描述符表、inode、设备队列。第二，谁可以修改状态，例如系统调用、异常处理、中断处理、内核线程或回收器。第三，失败怎样表达，例如返回错误码、发送信号、杀死进程、回滚事务或留下可恢复日志。第四，外部观察者能看到什么，例如命令输出、trace、统计计数、日志和测试结果。

操作系统同时解决六类问题。抽象让程序不必直接操作物理设备；隔离让一个程序不能随便破坏另一个程序；并发让多个任务共享 CPU 和 I/O；保护让用户态无法绕过页表、特权级和设备边界；持久化让数据在进程退出或机器崩溃后仍有解释；可观测性让系统在出错时有证据，而不是只留下“偶现问题”。

这五类问题不能分开学习。进程调度要理解线程睡眠和唤醒，睡眠和唤醒又依赖同步原语；虚拟内存隔离要理解页表权限，权限错误又通过异常进入内核；文件写入要理解 page cache，page cache 又属于内存管理；可靠提交要理解 fsync、目录项和崩溃恢复。

本册把操作系统看成一组状态机的组合。进程有生命周期，线程有运行状态，页表有映射状态，fd 有引用状态，文件页有 clean、dirty、writeback 状态，设备请求有 submitted、completed 状态，持久化输出有 prepared、committed 状态。能画出状态机，就能开始解释 OS；只会背 API 名字，无法判断边界。

核心理想

本册所有机制都用同一个问题模板分析：谁拥有状态，谁能改变状态，改变时如何同步，失败时如何回滚，外部如何证明它真的发生。

从工程角度看，操作系统的每个功能都同时有“用户可见语义”和“内核维护成本”。例如进程给用户一个独立执行环境，但内核要维护地址空间、fd 表、信号、父子关系和退出状态；虚拟内存给用户一个连续地址幻觉，但内核要维护 VMA、页表、TLB 失效和回收策略；文件给用户一个字节流或目录树，但内核要维护 inode、page cache、块分配、权限和崩溃恢复。只讲语义会太抽象，只讲实现会迷失在细节里，二者必须一起讲。

OS 对象不是名词，而是硬件状态的管理账

进程、线程、地址空间、文件描述符、inode、socket、设备请求这些词容易被背成名词。更准确地说，它们是内核为了管理硬件状态而建立的账本。CPU 只能执行当前寄存器里的上下文，OS 于是建立 task；MMU 只能按当前页表翻译地址，OS 于是建立 mm 和 VMA；设备只认识描述符和 doorbell，OS 于是建立 request、bio、skb、completion queue；存储设备只完成块写，OS 于是建立 inode、journal 和 fsync 语义。

OS 对象	管理的硬件事实	为什么不能只交给应用
task/thread	RIP、RSP、寄存器、内核栈、调度状态	应用不能自己抢占死循环，也不能保存别人的上下文
address space	CR3、PTE、TLB、VMA 意图	应用不能自己修改页表来隔离自己和别人

fd table	进程内整数句柄到内核对象引用	应用需要受控引用资源，而不是直接拿内核指针
file/inode	page cache、块映射、权限、引用计数	应用不能直接维护全局目录树和崩溃恢复
socket	NIC 队列、协议状态、收发缓冲区	应用不能直接管理网卡 DMA 和 TCP 状态机
request/bio/skb	DMA buffer、队列位置、completion	设备异步完成，需要内核追踪所有权和失败

这张表背后的规则是：只要某个硬件状态会被多个程序共享，或者失败后需要恢复，就不能让应用直接操作原始硬件。内核必须建立一个对象，给它身份、权限、生命周期、等待队列和可观测证据。对象不是为了“面向对象”而存在，而是为了让硬件状态可命名、可授权、可回收、可诊断。

容易混淆的三层语义

OS 教材里很多概念看起来相同，是因为它们在不同层次上都叫“完成”“可读”“地址”“上下文”。读源码时必须把用户 API 语义、内核对象语义和硬件事件语义分开。

说法	用户 API 层	内核对象层	硬件层
地址	指针值	VMA/PTE 和权限	TLB、物理地址、MMIO/IOVA
可读	read 不会立刻阻塞	buffer/队列中有数据或状态满足	中断、completion、DMA 写入
写完成	系统调用返回字节数	page cache/socket buffer/request 状态更新	设备队列接受、介质 flush 可能还没发生
上下文	用户调用链和线程身份	task、trap frame、switch context	RIP/RSP/RFLAGS、CR3、PCID
错误	errno 或信号	对象状态拒绝、权限失败、生命周期结束	fault、IRQ error、DMA/IOMMU fault

例如“可读”在 pipe、socket、文件和设备上含义不同。普通文件几乎总是可读，因为内核可以从 page cache 或磁盘推进；socket 可读表示接收队列有数据或连接关闭/错误；设备 fd 可读可能表示驱动队列里有 completion。用户只看到 poll 返回，但内核必须把对象状态和硬件事件翻译成统一接口。

再看“写完成”。写入 pipe 只表示数据进入内核 pipe buffer；写入 TCP socket 只表示数据进入发送路径，不等于对端应用收到；写入普通文件只表示 page cache 被修改，不等于稳定落盘；写入带 O_DIRECT 的块设备也可能只表示设备接受了请求，仍要理解 flush/FUA 和错误返回。把这些层混在一起，是很多系统设计错误的源头。

四条完整路径：从硬件事件到内核决策

下面用四条路径把本章的地图落到具体步骤。它们不是后续章节的替代品，而是读完整本书时不断回看的骨架。

路径一：write(fd, buf, len) 为什么不是一次普通函数调用

应用看见的是一个 C 库函数，硬件看见的是一次受控特权级转换，内核看见的是多个对象状态更新。

```
user:
    rax = SYS_write
    rdi = fd
    rsi = user_buf
    rdx = len
    syscall

kernel entry:
    save user RIP/RSP/RFLAGS and registers
    switch to trusted kernel stack
    locate current task and fd table
    validate fd refers to writable file/socket/pipe
    validate user buffer range and copy or pin pages
    update target object state
    return byte count or -errno in rax
```

阶段	关键检查	失败表达
系统调用入口	syscall number 是否存在，当前上下文是否允许	-ENOSYS、审计拒绝、seccomp 信号
fd 查找	fd 是否打开，是否有写权限	-EBADF、-EPERM
用户 buffer	地址是否 canonical、VMA 是否允许读、复制中是否 fault	-EFAULT 或部分写入
目标对象	pipe/socket/file 是否可写、是否阻塞	-EAGAIN、睡眠、信号中断
提交语义	数据进入哪里：page cache、socket buffer、设备队列	返回值不一定表示落盘或发出网络包

这条路径说明了 OS 契约的层次。系统调用返回成功，只说明内核接受并完成了该接口定义下的一部分工作。写普通文件可能只是写入 page cache；写 socket 可能只是进入发送缓冲；写终端可能经过驱动队列和中断。若业务需要“崩溃后仍存在”，还要继续追 fsync 和存储 flush；若业务需要“对端已收到”，还要追协议确认。

路径二：一次 page fault 如何在硬件和内核之间分工

用户态一条 `mov rax, qword ptr [rdi]` 可能触发 page fault。硬件做的是检查和转交，内核做的是解释和修复。

```
CPU:
    compute effective address from rdi
    check canonical form
    lookup TLB or walk page table
    if missing or permission denied:
        save fault RIP and error code
        record fault address
        enter page fault handler

kernel:
    find VMA covering fault address
    compare access type with VMA permissions
    inspect PTE state
    allocate zero page, read file page, perform COW, or reject
    install PTE and invalidate stale TLB when needed
    return to faulting RIP or deliver signal
```

硬件看到	内核进一步区分	结果
present=0	地址属于匿名 VMA	分配零页, minor fault
present=0	地址属于文件映射	page cache 命中或发起磁盘 I/O
write denied	PTE 标记 COW	复制页并恢复写权限
execute denied	NX 或 VMA 不可执行	发送信号, 安全拒绝
user bit denied	用户访问 supervisor 页	发送信号或记录攻击/bug

缺页机制的关键不是“慢”，而是“可恢复边界”。如果 CPU 不能保存 faulting RIP, COW store 就无法重试；如果内核不能判断 VMA 意图, present=0 就无法区分合法懒分配和非法访问；如果 TLB 失效不正确, PTE 更新后旧权限仍可能生效。

路径三：定时器中断如何变成抢占

用户程序可能永远不主动让出 CPU。定时器中断给内核一个周期性入口，让它能统计时间、处理超时并决定是否切换任务。

```
local APIC timer fires:
CPU takes interrupt at instruction boundary
entry saves interrupted RIP/RFLAGS/RSP
timer handler increments scheduler clock
expired timers wake sleeping tasks
current task runtime is charged
if time slice or policy says preempt:
    set need_resched
return path calls scheduler before going back to user
scheduler saves current kernel context and restores next
```

这里有两个容易混淆的点。第一，中断 handler 通常不在任意深处直接把用户栈改成另一个任务；它设置调度标志，在安全返回点进入调度。第二，切换任务不是只改一个 current 指针，还可能涉及内核栈、callee-saved 寄存器、地址空间、FPU/SIMD 状态、CPU 本地数据和 TLB 语义。

动作	需要保存什么	保存错误的现象
中断进入	用户 RIP/RSP/RFLAGS 和 scratch 状态	返回后用户程序寄存器随机坏
调度统计	当前任务运行时间、vruntime、状态	CPU 时间不公平或饥饿
切换内核上下文	内核栈指针、callee-saved 寄存器	任务从错误调用链恢复
切换地址空间	CR3/PCID、必要 TLB 处理	任务访问到别人的内存

路径四：设备完成如何唤醒一个睡眠任务

设备 I/O 是异步的。提交请求后，任务可能睡眠；设备通过 DMA 和中断报告完成；内核再把 completion 变成任务可见结果。

```
submit I/O:
    allocate request object
    map buffer for DMA
    fill descriptor ring
    publish descriptor with barrier
    write MMIO doorbell
    if synchronous API:
        put task on wait queue and schedule away

device:
```



```
reads descriptor by DMA
transfers data
writes completion entry
raises MSI-X interrupt

interrupt/bottom half:
read completion queue
unmap DMA and mark request done
store status and byte count
wake tasks waiting on request
```

这条路径解释了为什么 I/O 子系统一定有 request identity。中断到来时，CPU 不知道“哪个用户函数在等它”，只看到某个设备队列有 completion。内核必须通过队列位置、tag、request 指针或 completion entry 找回原请求，撤销 DMA 授权，更新上层对象，再唤醒等待者。没有这层身份映射，异步 I/O 无法可靠对应到发起者。

从组成原理到 OS 抽象

组成原理解释机器能做什么，操作系统解释这些能力如何被组织成可共享、可隔离、可恢复的服务。把二者分开学时，读者很容易知道“CPU 有特权级”，却不知道系统调用为什么必须切栈；知道“页表做地址翻译”，却不知道 fork 为什么能 COW；知道“DMA 很快”，却不知道驱动为什么要维护 descriptor ring 和 completion。

组成原理事实	OS 抽象	关键问题
RIP、寄存器和栈可保存	线程上下文	切走后如何从同一位置恢复
特权级限制敏感指令	用户态/内核态	用户如何安全请求内核服务
异常入口保存 trap frame	page fault、signal、debug	错误能否修复并返回原指令
定时器可强制中断	抢占式调度	不合作任务如何被收回 CPU
MMU 检查页表权限	地址空间	进程如何互相隔离又能共享
cache/TLB 需要维护	性能和一致性策略	为什么迁移、锁和 unmap 有成本
MMIO 控制设备	驱动模型	寄存器副作用和顺序如何管理
DMA 异步访问内存	高性能 I/O	buffer 何时归 CPU，何时归设备

本册后文所有章节都按这张映射阅读。看到“进程”，不要只想 pid，要想到一组保存的 CPU 上下文、地址空间和内核对象。看到“文件”，不要只想路径，要想到 fd 表、file、inode、page cache、块层和设备队列。看到“权限”，不要只想用户 ID，要想到 CPU 特权级、页表 U/S 位、capability、namespace、LSM 和对象引用。

同一个词在 POSIX、内核和硬件中的落点

读手册时常见的是 POSIX 词，例如 process、file descriptor、signal、pipe、socket。读内核时看到的是 task、file、inode、skb、wait queue。读硬件时又变成 RIP、CR3、IRQ、DMA、MMIO。三套词不是互相替代，而是同一条路径的不同层。

用户语义	内核对象	硬件支点	关键边界
process	task、mm、files、cred	RIP/RSP、CR3、PCID	身份、地址空间、权限分离
thread	task、kernel stack、thread info	寄存器、RSP、FPU state	可调度上下文，不复制全部资源
fd	fd table、struct file	设备队列、page cache、DMA	整数句柄不是文件本体
signal	pending signal、signal frame	trap frame、用户栈、 <code>iretq</code>	异步交付但必须恢复用户现场
socket	socket、skbuff、wait queue	NIC ring、MSI-X、DMA	协议状态和设备完成分离
timer	hrtimer、timer wheel	local APIC timer、TSC deadline	未来事件需要硬件唤醒入口

例如 fd 是一个用户态整数，但内核要通过当前进程的 fd table 找到 struct file，再通过 file operations 调到 pipe/socket/inode/device 的具体实现。若底层是网卡，最终还会走 skb、驱动队列、DMA 和 MSI-X completion。把 fd 当成“文件本体”，就解释不了 dup、fork 后共享 offset、socket readiness、设备拔出后旧 fd 返回错误这些现象。

为什么从单片机开始

单片机不是因为简单才低级，而是因为它暴露了 OS 的原始需求。一个裸机程序拥有全部内存和设备，所有代码同权，主循环靠自觉返回，中断直接修改全局状态。它让我们能看见每个高级机制出现前的失败。

裸机阶段的问题	直接补丁	通用 OS 机制
函数忙等设备	改成状态机或中断	阻塞、等待队列、completion
任务靠自觉返回	任务表和显式 yield	线程和协作式调度
任务可能不合作	加硬件定时器	抢占和时间片
全局变量被 ISR 打断	关中断临界区	锁、原子操作、内存屏障
所有代码同权	约定模块边界	CPU 特权级和系统调用
所有内存同权	手工内存布局	MMU、地址空间、页表权限
CPU 搬每个字节	DMA 控制器	异步 I/O 和驱动队列

这也是本册合并组成原理与 OS 的原因：如果不知道硬件能否强制中断、能否区分用户/内核、能否检查页表权限，就无法判断一个 OS 设计到底是“规范约定”还是“硬件保证”。

实践

阅读任何 OS 现象时，先写一张“状态和边界表”。

问题	要填写的内容
状态对象	进程、线程、页、fd、inode、socket、设备请求、日志记录
所有者	用户态对象、内核对象、硬件寄存器、设备控制器
入口	系统调用、异常、中断、内核线程、后台回收

权限	特权级、页表权限、文件权限、capability、namespace、引用计数
等待	runqueue、futex、page fault、I/O completion、锁、条件变量
提交	状态更新、引用释放、写入 page cache、落盘、manifest 生效
证据	返回值、errno、trace、计数器、日志、测试断言

例如一次 `write(fd, buf, len)` 可以这样拆：用户态只有一个整数 `fd` 和一段用户 `buffer`；内核要先从进程 `fd` 表找到 `open file description`，再检查写权限，再复制用户 `buffer`，再把页标记为 `dirty`，最后返回写入字节数或错误。这个返回值通常不等于数据已经安全落盘。要证明持久化，还需要 `fsync`、目录同步或更高层 `manifest`。

再看一次 `page fault`。用户态执行 `load/store`，硬件发现页表项不存在或权限不符，CPU 进入异常入口，内核根据 `VMA` 判断这是合法缺页、权限错误还是非法地址。合法缺页可能分配物理页或读取文件页；权限错误可能发送信号；非法访问可能终止进程。这里的核心不是“缺页很慢”，而是虚拟地址访问被拆成硬件检查和内核决策。

契约维度

一个 OS 机制要讲完整，至少要覆盖五个维度。

维度	追问	例子
身份	对象如何被引用	<code>pid</code> 、 <code>tid</code> 、 <code>fd</code> 、 <code>inode number</code> 、 <code>socket</code> 四元组、 <code>DMA tag</code>
权限	谁能做什么	页表 <code>R/W/X</code> 、 <code>fd flags</code> 、 <code>capability</code> 、 <code>seccomp</code> 、 <code>namespace</code>
生命周期	何时创建和释放	<code>fork/exit/wait</code> 、 <code>open/dup/close</code> 、 <code>page refcount</code> 、 <code>request completion</code>
等待	无法立即推进时停在哪里	<code>runqueue</code> 、 <code>wait queue</code> 、 <code>page fault</code> 、 <code>I/O queue</code> 、 <code>timer</code>
提交	何时对外可见或持久	系统调用返回、 <code>PTE</code> 安装、 <code>DMA</code> 完成、 <code>fsync</code> 、 <code>manifest</code>

这五个维度能帮助读者判断一个解释是否完整。比如“`socket` 可读”只覆盖了等待维度，还要问：哪个 `fd` 引用它，谁有权限读，缓冲区里数据生命周期如何，读取后 `offset` 或队列怎样变化，关闭时如何唤醒等待者。

层次与反向依赖

操作系统经常呈现层次结构：硬件在底层，内核抽象在中间，用户 API 在上层。但真实问题会反向穿透层次。一个 `write` 系统调用看似属于文件 API，却可能等待内存回收、块层队列、驱动中断、设备 `flush` 和调度器唤醒。一个 `page fault` 看似属于内存管理，却可能读取文件页，触发块设备 I/O。一个安全拒绝看似属于权限系统，却依赖路径解析、`namespace`、凭据和 `LSM`。

```
user write()
-> fd table
-> file object
```

```

-> page cache
-> writeback
-> block layer
-> driver DMA
-> interrupt completion
-> wakeup

```

因此阅读本册时，不要把章节理解成互不相干的模块。章节只是学习顺序，真实系统是交叉依赖的状态机网络。

演进路线

本册后续章节按一条明确路线展开。

1. 数字电路和 CPU：bit、寄存器、时钟、状态机、取指译码执行。
2. 总线和设备：地址译码、MMIO、设备寄存器、中断和 DMA。
3. 保护硬件：特权级、异常入口、MMU、页表和定时器。
4. 单片机裸机程序：一个主循环直接拥有整台机器。
5. 事件和状态机：阻塞函数被拆成可返回的 step。
6. 中断：外设主动打断 CPU，事件不再只靠轮询。
7. 定时器：系统获得时间基准和抢占入口。
8. 任务上下文：程序计数器、寄存器和栈成为可保存对象。
9. 调度器：多个 runnable 任务按策略共享 CPU。
10. 同步：共享状态需要互斥、等待谓词和唤醒协议。
11. 内存保护：地址空间、页表和权限阻止任务互相破坏。
12. 系统调用：用户程序通过受控入口请求内核操作资源。
13. 文件和 I/O：设备被包装成对象、句柄、队列和 completion。
14. 持久化：数据从“写过”推进到“崩溃后仍可解释”。

这条路线的意义是把 OS 特性从原因中长出来。不是先背“进程、线程、虚拟内存、文件系统”，而是从每个阶段遇到的失败推出下一层机制。

失败	需要的机制	新增代价
固定电路不通用	存储程序 CPU	指令编码、取指译码、异常语义
单核状态无法被安全共享	多核一致性和原子操作	cache line 迁移、内存序、锁竞争
程序直接用物理地址	MMU 和虚拟地址	页表、TLB、缺页成本
CPU 搬运所有 I/O 数据	DMA 和描述符队列	buffer 生命周期、IOMMU、completion
轮询浪费 CPU	中断	异步共享状态
任务死循环霸占 CPU	定时器抢占	上下文保存和调度开销
任务互相覆盖内存	地址空间和权限	页表、缺页、TLB 成本
用户能随便改设备 I/O 等待拖住计算	系统调用和特权级阻塞队列、异步和 completion	入口检查和复制成本 请求身份和 buffer 生命周期

测试

OS 理论卷的测试不是只跑程序，而是验证契约。一个合格测试要覆盖正常路径、错误路径和边界路径，并且能指出是哪条不变量断了。

- 能把一次系统调用拆成用户态入口、内核对象、权限检查、状态更新和返回值。
- 能区分“函数返回成功”“内核接受请求”“设备完成”“业务提交生效”。
- 能解释至少一个等待现象：等 CPU、等锁、等页面、等 I/O、等子进程。
- 能说明至少一个隔离边界：进程地址空间、文件权限、用户/内核态、namespace。
- 能为任意章节机制写出状态、入口、失败和证据四列。

反例同样重要。若读者把 fd 当成文件本体，把 write 成功当成落盘，把线程 ready 当成 running，把 malloc 成功当成物理页已分配，把 notify_one 当成消息投递，就说明 OS 契约还没有建立。

从单片机裸机程序开始

原理

理解操作系统，最稳的起点不是 Linux，也不是进程和虚拟内存，而是一块最小单片机。假设机器只有一个 CPU 核、一段 Flash、一段 SRAM、几个内存映射寄存器、一个 GPIO 灯、一个串口和一个定时器。上电后 CPU 从固定地址取第一条指令，初始化栈指针，清零 BSS，设置外设寄存器，然后进入 while (true) 主循环。

这时还没有“操作系统”。程序直接拥有整台机器：所有内存都能读写，所有外设寄存器都能访问，任何函数都可以关中断、改时钟、写串口、改 GPIO。裸机程序的优点是简单、可预测、启动快；缺点也同样清楚：只要任务变多，主循环就开始混乱；只要某个操作阻塞，整个系统就停住；只要某段代码写坏内存，没有隔离层能救它。

最小裸机程序通常是轮询模型。代码不断检查输入设备是否有数据、检查定时器是否到点、更新输出设备。轮询的本质是 CPU 主动问外设“你准备好了吗”。如果事件很频繁，轮询很直接；如果事件很稀疏，CPU 大量时间都浪费在重复检查；如果轮询顺序写错，高优先级事件可能被低优先级慢操作延迟。

一个 MCU 芯片里面通常有什么

单片机不是“缩小的 PC”，而是一颗芯片上集成了 CPU 核、Flash、SRAM、时钟、复位控制器、GPIO、UART、SPI/I2C、定时器、ADC、DMA 控制器和中断控制器。很多外设不是通过复杂总线枚举出来，而是在芯片手册里固定了基地址和寄存器布局。

部件	作用	与 OS 的关系
CPU core	执行指令和异常入口	上下文、特权模式、中断响应
Flash	保存程序镜像和只读数据	类似启动介质和只读代码段
SRAM	保存栈、堆、全局变量	物理内存管理的最小形态
中断控制器	管理外设中断	IRQ 分发和优先级的原型
SysTick/Timer	周期时间基准	tick、超时和抢占的原型
GPIO/UART/SPI/I2C	连接外部设备	字符设备和总线驱动的原型

这些部件通常共享同一个地址空间。CPU 访问 0x20000000 可能是 SRAM，访问 0x40000000 可能是 UART 寄存器。区别不在指令，而在地址译码后连到哪个硬件。这个事实会一路延伸到大型 OS：驱动通过 MMIO 地址访问 PCIe 设备寄存器，本质仍是 load/store 触发硬件状态机。

芯片设计者怎样选择寄存器接口

外设寄存器是硬件和软件的协议。设计一个 UART 外设时，芯片设计者通常会暴露控制寄存器、状态寄存器、数据寄存器、波特率寄存器和中断使能寄存器。软件写控制寄存器启用发送接收；读状态寄存器判断 TX ready 或 RX available；读写数据寄存器收发字节。

UART register model:

CTRL	bit0 enable_rx, bit1 enable_tx
STATUS	bit0 rx_ready, bit1 tx_empty, bit2 overrun
DATA	read pops received byte, write submits transmit byte
BAUD	divisor for baud rate generator
IRQ_EN	enable rx/tx/error interrupt

注意 DATA 寄存器不是普通内存。读它可能弹出接收 FIFO 的一个字节，写它可能启动发送状态机。状态位也可能有特殊清除规则，例如“写 1 清除”。驱动必须严格按手册顺序访问这些寄存器，否则问题不是编译错误，而是硬件进入不可预期状态。

从外设状态机看驱动

一个 UART 发送器可以看成硬件状态机：空闲时等待软件写 DATA；收到字节后按波特率移位输出；完成后设置 tx_empty；若开启中断，则向中断控制器发请求。软件驱动的任务不是“调用发送函数”，而是和这个状态机同步。

UART_TX_IDLE:

```
if software writes DATA:
    shift_reg = DATA
    state = UART_TX_BUSY
```

UART_TX_BUSY:

```
shift one bit per baud tick
if all bits sent:
    STATUS.tx_empty = 1
    if IRQ_EN.tx_empty:
        raise_irq()
    state = UART_TX_IDLE
```

这就是为什么驱动要处理 ready 位、超时、FIFO 满、错误位和中断确认。硬件不会执行 C 函数返回值协议，它只会按寄存器位和状态机推进。操作系统把这种硬件状态机包装成 read/write/poll/ioctl，但底层仍然必须尊重硬件协议。

```
init_clock();
init_gpio();
init_uart();

while (true) {
    if (uart_has_byte()) {
        byte b = uart_read();
        handle_command(b);
    }
    if (timer_expired()) {
        update_control_loop();
    }
}
```



```

}
flush_logs_if_needed();
}

```

这段代码看起来不像 OS，但它已经暴露了 OS 的胚胎问题。第一，多个活动共享一个 CPU，谁先执行？第二，事件到达时间不可控，怎样避免漏事件？第三，某个处理函数太慢，怎样不拖死其他工作？第四，共享状态由谁保护？第五，故障后怎样恢复到可解释状态？操作系统只是把这些问题系统化。

从超级循环到任务表

裸机主循环通常被称为超级循环。它的结构是一个大循环里按固定顺序调用多个任务函数。第一版可以直接写：

```

while (true) {
    task_read_sensor();
    task_control_motor();
    task_send_telemetry();
    task_blink_led();
}

```

这种写法的问题是隐含了调度策略：每个任务的优先级就是它在代码里的顺序；每个任务的时间预算完全靠自觉；任何任务卡住，后面的任务都不会运行。要让它可分析，第一步是把任务放进表里，并给每个任务明确的周期、deadline 和上次运行时间。

```

struct Task {
    const char* name;
    uint32_t period_ms;
    uint32_t next_release_ms;
    void (*run)();
};

while (true) {
    uint32_t now = ticks_ms();
    for each task in tasks:
        if (now >= task.next_release_ms) {
            task.run();
            task.next_release_ms += task.period_ms;
        }
}

```

这个算法是最小合作式调度器。它没有抢占，任务只有在自己返回后才把 CPU 交出来。它的复杂度是每轮扫描 $O(n)$ ， n 是任务数量；它的关键不变量是 `next_release_ms` 单调前进，每个任务函数必须短小、不能无限循环、不能执行不可控阻塞。

合作式调度器已经比手写超级循环好，因为任务的释放条件变成数据。但它仍然无法处理一个任务超时的问题。如果 `task_send_telemetry` 等串口缓冲区空等了 50ms，那么 1ms 周期的控制任务就会迟到。于是我们需要区分“能立刻做的计算”和“需要等待的 I/O”，这会把系统推向事件队列和中断。

最小调度算法的复杂度和失败边界

周期任务表的扫描算法很容易写，但它把每轮成本变成 $O(n)$ 。如果任务数量很少，成本可忽略；如果任务很多或周期很短，调度器本身会吃掉可观 CPU 时间。更重要的是，它没有强制时间预算，只能在任务返回后继续扫描。

策略

算法成本

失败边界

固定顺序超级循环	每轮 $O(n)$	任务顺序即隐式优先级，慢任务拖累后续任务
周期任务表	每轮扫描 $O(n)$	任务不返回则全局停顿，deadline 只能事后发现
优先级就绪队列	取最高优先级可为 $O(1)$ 或 $O(\log n)$	低优先级可能饥饿，需要抢占和时间统计
时间片轮转	每次切换接近 $O(1)$	交互延迟和吞吐受时间片影响

这里已经能看见调度器设计的核心：策略不是口号，必须落到数据结构和复杂度。任务少时，简单扫描最可靠；任务多、优先级复杂、需要抢占时，就要引入 runqueue、时间统计和上下文切换。

状态机替代阻塞函数

裸机程序最常见的灾难是阻塞等待：

```
void send_packet(Packet p) {
    for each byte in p:
        while (!uart_tx_ready()) {
            // busy wait
        }
        uart_write(byte);
}
```

这段代码在功能上正确，但会占住整个 CPU。更好的写法是把发送过程改成状态机：如果硬件暂时不能发送，就保存进度并返回，让其他任务继续运行。

```
enum TxState { Idle, Sending };

void uart_tx_step() {
    if (tx_state == Idle || !uart_tx_ready()) {
        return;
    }
    uart_write(tx_buffer[tx_pos++]);
    if (tx_pos == tx_len) {
        tx_state = Idle;
    }
}
```

状态机的核心代价是复杂性。阻塞函数把“下一步在哪里”藏在调用栈里；状态机把下一步显式放在 `tx_state` 和 `tx_pos` 里。操作系统中的线程，就是另一种保存“下一步在哪里”的方法：线程把寄存器、栈和程序计数器保存为上下文，使阻塞代码看起来仍像顺序代码。但线程要付出调度和栈内存成本。

资源所有权和最小内核对象

没有 OS 时，所有模块都能直接操作硬件寄存器。小程序可以这样写；程序变大后，它会造成所有权混乱。串口驱动、日志模块、命令行模块如果都直接写 UART 寄存器，就无法保证输出顺序，也无法处理并发写入。

于是我们引入最小内核对象：一个对象拥有硬件资源，其他模块只能通过它的接口请求操作。比如串口对象维护发送队列、接收队列、错误计数和当前硬件状态。它不一定需要特权级，但已经有了内核思想：资源集中管理，状态对外封装，错误通过接口返回。

```
struct UartDevice {
    RingBuffer rx;
    RingBuffer tx;
    uint32_t overrun_count;
    bool tx_busy;
```

```
};

bool uart_submit(UartDevice* dev, Span bytes);
bool uart_read(UartDevice* dev, byte* out);
```

从这里可以看到后续 fd、socket、设备文件的影子。用户不应该直接改设备寄存器，而应该持有一个句柄，请求拥有者执行操作。差别只是大型 OS 里这个拥有者运行在内核态，并且有硬件权限保护。

从裸机模块边界到用户/内核边界

裸机程序里也可以规定“只有 UART 模块能写 UART 寄存器”，但这只是团队约定。任何 C 文件只要拿到寄存器地址，仍然能直接写硬件。通用 OS 不能依赖所有程序守规矩，它必须让硬件禁止用户程序访问设备寄存器。

```
bare-metal convention:
  app calls uart_submit()
  but app could still write UART_BASE directly

protected OS:
  user VA does not map UART MMIO
  user write to UART physical address triggers fault
  app must call write(fd, ...)
  kernel driver performs MMIO after permission checks
```

这就是模块化和隔离的区别。模块化让代码更清楚，但不能阻止恶意或错误代码；隔离由 CPU 特权级、页表权限和异常入口强制执行。OS 的很多复杂性，就是把裸机里的“请大家遵守约定”升级成“硬件保证不能绕过”。

实践

本章实践不需要复杂环境。拿任意一个单片机或模拟器，写三版程序：

1. 超级循环：固定顺序调用四个任务。
2. 周期任务表：每个任务有 period 和 next release。
3. 非阻塞状态机：把串口发送或传感器读取改成 step 函数。

报告要记录每个任务最长运行时间、周期、deadline、是否可能阻塞、是否访问共享状态。对于每个任务，写出下面的表：

字段	内容
输入	读取哪些寄存器、队列或全局状态
输出	写哪些寄存器、队列或全局状态
时间预算	最坏执行时间和周期
等待点	是否忙等硬件、锁或缓冲区
失败	超时、溢出、校验失败、设备错误

测试

最低测试不是“灯会闪”，而是证明调度和状态机边界：

- 任一任务执行一次后必须返回主循环。
- 周期任务的 `next_release_ms` 单调前进，不因时间溢出立刻失控。
- 串口发送缓冲区满时，提交失败或施加背压，而不是覆盖旧数据。
- 非阻塞状态机在硬件未 ready 时不改变发送进度。

- 任一任务超时应被计数或记录，不能静默拖慢全系统。

这一章的验收结论是：操作系统不是凭空出现的。只要裸机程序里出现多个活动、共享资源、等待硬件、错误恢复和时间预算，就已经需要 OS 式的状态管理。

源码级补充：真实 MCU 启动、UART 与链接脚本

x86-64 早期入口、_start 与最小启动代码

早期启动的第一份“内核源码”通常不是 C 函数，而是入口汇编和链接脚本。x86-64 上，UEFI/bootloader 或 EFI stub 把内核镜像放入内存，传入启动参数和内存地图；早期入口建立最小栈、清 BSS、准备早期页表和 IDT，然后才调用 `kernel_main` 一类 C 入口。

```
early_x86_64_entry:
    lea rsp, [rip + boot_stack_top]
    copy .data from flash LMA to sram VMA
    zero .bss
    build early page tables
    load provisional IDT
    call early_console_init
    call kernel_main
halt_loop:
    hlt
    jmp halt_loop
```

这里已经出现了 OS 启动的三个根概念：入口地址、内存布局和异常表。后面 IDT、内核入口和多核启动更复杂，但本质仍是“硬件和 bootloader 按固定规则跳到一段受信任代码，代码建立可运行环境”。

VMA 与 LMA：为什么 data 要复制

链接脚本里常见 VMA 和 LMA。VMA 是代码运行时使用的地址，LMA 是镜像中加载的位置。已初始化全局变量的初值保存在 Flash 中，但运行时变量要位于 SRAM，所以启动代码要从 LMA 复制到 VMA。

```
MEMORY {
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 512K
    SRAM (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}

SECTIONS {
    .text : { *(.text*) *(.rodata*) } > FLASH
    .data : AT(ADDR(.text) + SIZEOF(.text)) {
        __data_start = .;
        *(.data*)
        __data_end = .;
    } > SRAM
    __data_load = LOADADDR(.data);
    .bss : {
        __bss_start = .;
        *(.bss*) *(COMMON)
        __bss_end = .;
    } > SRAM
}
```

若 data 未复制，带初值的全局对象会从错误内容开始；若 BSS 未清零，锁、队列和指针的初始值不可预测。这个失败和大型 OS 的早期页表、per-CPU 区域未初始化一样，通常不会在第一条错误处崩溃，而是在后续随机位置表现出来。

GPIO 和 UART 寄存器

裸机驱动的核心是 MMIO。GPIO 通常有模式寄存器、输出数据寄存器、输入数据寄存器；UART 通常有状态寄存器、数据寄存器、波特率 divisor、控制寄存器。真实芯片位名不同，但结构类似。

```
#define UART_SR    (UART_BASE + 0x00)
#define UART_DR    (UART_BASE + 0x04)
#define UART_BRR   (UART_BASE + 0x08)
#define UART_CR    (UART_BASE + 0x0c)
#define UART_TXE   (1u << 7)
#define UART_RXNE  (1u << 5)

void uart_putc(char c) {
    uint32_t deadline = ticks + 10;
    while (!(read32(UART_SR) & UART_TXE)) {
        if (time_after(ticks, deadline)) {
            uart_errors.timeout++;
            return;
        }
    }
    write32(UART_DR, c);
}
```

这里的超时是工程边界。没有超时的驱动把硬件故障扩大成系统停顿。大型内核中的块设备超时、网络设备 reset、watchdog 都是同一思想：设备是异步状态机，不是可靠函数。

轮询、中断和 CPU 占用率

假设 UART 每秒收到 100 个字节，CPU 频率 100MHz。若主循环每 10 微秒轮询一次状态寄存器，一秒轮询 100000 次，其中只有 100 次有用；若每次轮询花 30 个周期，纯轮询检查就耗掉 300 万周期，约 3% CPU。若轮询间隔变大，CPU 占用下降，但输入延迟和溢出风险上升。

模式	优点	失败边界
忙轮询	实现最简单，延迟可控	浪费 CPU，慢操作会漏事件
周期轮询	降低 CPU 消耗	延迟取决于周期，burst 易溢出
中断驱动	空闲时不耗 CPU，事件到达即处理	ISR 竞态、队列满、中断风暴
DMA + 中断	大吞吐低 CPU	buffer 生命周期、cache、一致性复杂

这张表也适用于大 OS：epoll 减少无效轮询，NAPI 在高负载下把中断转为轮询，io_uring 用共享队列减少系统调用往返。机制不同，目标都是在 CPU 占用、延迟和复杂度之间折中。

MCU 上的 DMA：从串口接收到环形缓冲区

即使在单片机上，DMA 也不是高级服务器才有的东西。典型场景是 UART 持续接收数据，CPU 不想每个字节都进中断。驱动可以准备一个环形缓冲区，让 DMA 控制器把 UART 接收数据直接写入 SRAM，半满或满时再中断 CPU。

```
uart_rx_dma_setup():
    dma.src = UART_DATA_REGISTER
    dma.dst = rx_ring_buffer
    dma.len = RING_SIZE
    dma.mode = circular
    dma.enable_half_full_irq = true
    dma.enable_full_irq = true
```

```
start_dma()

dma_irq():
    produced = compute_dma_write_position()
    publish_new_bytes_to_parser(produced)
```

这个例子把 DMA 的三条规则讲清楚。第一，buffer 在 DMA 运行期间不能随意改用途。第二，CPU 读取 DMA 写入的数据前，必须理解缓存一致性和写入位置。第三，completion 不一定表示全部 I/O 结束，也可能只是半缓冲区可消费。大型 OS 的网卡 RX ring 和块设备 completion，本质上是这个模型的扩展。

真实代码阅读入口

读真实 MCU 代码时，建议按下面顺序找：

- 1. 启动汇编或 startup 文件：向量表、reset handler、栈顶符号。
- 2. 链接脚本：Flash/SRAM 区域、.text/.data/.bss、堆栈布局。
- 3. HAL 或裸驱动：UART/GPIO/TIMER 寄存器定义和初始化序列。
- 4. 中断文件：IRQ handler 如何清中断、入队、唤醒主循环。
- 5. 示例 main：是否有阻塞等待、是否有超时、是否有错误计数。

x86-64 教学内核常见文件可以包括 boot/entry_64.S、kernel/head64.c、linker.ld、drivers/early_uart.c。不要先读完整驱动框架，先把 UEFI/bootloader 交接到第一个字符输出的路径追通。

启动、链接脚本与内核初始化

原理

从裸机程序走向内核，第一道门是启动。机器上电后不会自动理解 C 语言，也不会自动准备堆、栈、全局变量和设备。CPU 只知道从复位向量或固件指定入口取指。操作系统必须把“二进制文件如何进入内存、第一条指令在哪里、栈在哪里、全局变量如何初始化、设备何时可用”这些问题全部讲清。

在单片机上，启动路径通常是 reset handler：设置栈指针，把 .data 从 Flash 复制到 SRAM，把 .bss 清零，初始化时钟和最小串口，然后调用 main 或 kernel_main。在 PC 或服务器上，路径更长：固件初始化硬件，bootloader 加载内核镜像和启动参数，切换 CPU 模式，建立最小页表，最后跳到内核入口。

启动阶段的特点是“没有服务可用”。还没有调度器，不能睡眠；还没有内存分配器，不能随便 malloc；还没有完整中断，不能依赖异步事件；还没有文件系统，不能读普通文件；还没有日志系统，调试输出只能靠串口、早期 console 或内存缓冲区。这些约束解释了为什么内核初始化代码通常按严格阶段组织。

上电后 CPU 为什么不能直接运行内核 C 函数

C 函数依赖一组运行环境：栈指针有效，全局变量已初始化，BSS 为零，代码和数据位于链接器预期地址，调用约定可用，必要的地址映射存在。上电瞬间这些条件都不自动成立。CPU 只处在架构定义的复位状态，从固定入口取指；剩下的环境必须由启动代码一步步建立。

C 代码假设	谁建立	若缺失会怎样
栈可用	reset handler 或 bootloader	函数调用和局部变量破坏未知内存
全局变量已初始化	启动代码复制 .data、清零 .bss	锁、队列、指针从随机值开始

代码地址正确	链接脚本和装载器	跳转、全局地址、异常表错误
内存可访问	页表或物理映射	取指或访存 fault
最小输出可用	early console 初始化	出错时没有证据

所以内核入口往往先是汇编或极小 C 代码。它不是为了炫技，而是因为在运行时环境建立前，普通 C 抽象还没有可靠落脚点。

链接脚本决定内存布局

链接脚本把代码段、只读数据、已初始化数据、未初始化数据和栈安排到具体地址。没有链接脚本，编译器和链接器不知道内核应该放在物理地址哪里，也不知道哪些符号表示段边界。

```
SECTIONS {
    . = 0x80200000;
    __kernel_start = .;
    .text : { *(.text*) }
    .rodata : { *(.rodata*) }
    .data : { *(.data*) }
    __bss_start = .;
    .bss : { *(.bss*) *(COMMON) }
    __bss_end = .;
    __kernel_end = .;
}
```

这些符号不是装饰。启动代码需要 `__bss_start` 和 `__bss_end` 来清零 BSS；早期物理页分配器需要 `__kernel_end` 知道哪些内存已经被内核占用；页表初始化要知道哪些区域可执行、只读或可写。

链接地址、加载地址和重定位

链接地址是代码认为自己运行的位置，加载地址是 bootloader 把镜像放到内存的位置。二者可以相同，也可以不同。若不同，早期代码要么以位置无关方式运行，要么完成重定位，要么建立页表把链接虚拟地址映射到实际物理地址。

```
kernel linked at virtual address: 0xffffffff81000000
kernel loaded at physical address: 0x00100000

early page table maps:
0xffffffff81000000 -> 0x00100000
```

这解释了为什么 64 位内核常有“高半区内核”概念。用户进程使用低地址范围，内核映射在高虚拟地址范围；每个进程切换页表时仍保留内核映射，系统调用和中断进入内核后能访问同一套内核代码和数据。这个设计依赖页表，而不是物理内存真的在高地址。

启动阶段的状态机

内核启动可以写成状态机：

```
Reset
-> MinimalStack
-> DataCopied
-> BssCleared
-> EarlyConsole
-> BootMemoryDiscovered
-> PageTablesReady
-> InterruptsReady
-> SchedulerReady
-> InitProcessStarted
```


每个阶段只能使用之前已经建立的能力。比如 `EarlyConsole` 之前不能输出复杂日志；`BootMemoryDiscovered` 之前不能建立通用页分配；`SchedulerReady` 之前不能阻塞等待；`InterruptsReady` 之前不能依赖设备中断。

x86-64 模式演进：实模式、保护模式、长模式

x86-64 的历史兼容层让启动路径明显分阶段。复位后 CPU 从兼容模式开始，早期代码或固件/bootloader 需要逐步建立 GDT、打开保护机制、准备页表、启用 PAE 和长模式，最后跳到 64 位代码。现代 UEFI 和 bootloader 会屏蔽许多细节，但内核仍要理解当前进入时的模式和页表状态。

```
legacy shape:
  reset vector
  real mode firmware/bootloader
  load kernel setup code
  build provisional GDT
  enter protected mode
  build early page tables
  enable long mode
  jump to 64-bit kernel entry
```

GDT 在长模式下不再像 32 位分段那样用于普通地址计算，但仍参与代码段、数据段、TSS、特权切换和系统调用入口设置。TSS 也不只是历史名词，x86-64 可用 IST 为 NMI、double fault 等关键异常提供专用栈，避免在当前栈损坏时继续崩溃。

BIOS、UEFI 和 bootloader 的职责边界

BIOS 风格启动和 UEFI 风格启动不同，但目标相同：把内核镜像和硬件描述交给内核。bootloader 不应替内核做所有工作，它只建立足够条件让内核接管。内核接管后，必须停止依赖大部分固件 Boot Services，改用自己的内存管理、驱动和中断。

阶段	可依赖对象	退出后的边界
固件早期	DRAM 初始化、平台基础设置	内核通常看不到细节，只接收结果
bootloader	读取磁盘/网络、加载镜像、构造参数	不再参与普通运行时服务
UEFI Boot Services	分配内存、访问协议、获取内存地图	<code>ExitBootServices</code> 后不可再调用
内核 early boot	启动参数、内存地图、早期 console	必须校验并建立自己的对象

`ExitBootServices` 是一个硬边界。退出前，固件仍可能拥有内存和设备；退出后，内核承担管理责任。若内核拿到内存地图后，固件又分配了内存，而内核没有重新获取地图，就可能把固件仍使用的页当成空闲页。

实践

实践时先写一个启动依赖表：

阶段	可用能力	禁止事项
Reset	寄存器、固定入口	使用全局变量假设已初始化
MinimalStack	栈、少量汇编	调用复杂 C 运行库
BssCleared	零初始化全局变量	动态分配内存
EarlyConsole	最小输出	格式化复杂日志、大缓冲
BootMemory	物理内存范围	使用完整虚拟内存假设

PageTables
Scheduler

基础地址翻译
任务切换

访问未映射设备
在早期 init 中遗留永久锁

然后画出内核镜像布局：text、rodata、data、bss、boot stack、page tables、free memory。只要能解释“内核自己占了哪些页，哪些页可以交给分配器”，后续物理页管理才有根。

算法与实现分析

BSS 清零和 data 复制

```
boot_zero_bss():
    p = __bss_start
    while p < __bss_end:
        *p = 0
        p += word_size

boot_copy_data():
    src = __data_load_start
    dst = __data_start
    while dst < __data_end:
        *dst++ = *src++
```

这两个循环是 $O(n)$ ， n 是段大小。它们必须在使用对应全局变量之前完成。若 BSS 没清零，全局队列、锁和指针会从随机值开始，后续 bug 很难定位。

早期 bump allocator

完整页分配器建立前，内核常用 bump allocator 分配早期对象。它只维护一个 next 指针，每次按对齐向上推进，不支持释放。

```
early_alloc(size, align):
    next = align_up(early_next, align)
    end = next + size
    if end > early_limit: panic()
    early_next = end
    return next
```

分配是 $O(1)$ ，实现极简单。代价是不能释放，且必须在切换到正式分配器后停止使用。它适合页表、启动参数副本、早期设备表等生命周期贯穿启动阶段的对象。

初始化依赖图

内核初始化不能只靠函数排列顺序。一个模块可能依赖 console，一个模块依赖内存分配器，一个模块依赖中断，一个模块必须在调度器启动前完成。把这些关系写成依赖图，才能解释为什么初始化常被分成 early、core、subsys、device、late 等阶段。

```
init_node {
    name: "page_allocator"
    requires: ["boot_memory_map", "early_console"]
    provides: ["alloc_pages"]
}

run_init_graph(nodes):
    ready = nodes with all requirements satisfied
    while ready not empty:
        n = pop(ready)
        run(n.init)
        mark_provided(n.provides)
```

```

add newly unblocked nodes to ready
if unfinished nodes remain:
    panic("init dependency cycle or missing provider")

```

这个算法是拓扑排序，复杂度是 $O(V + E)$ 。它的工程价值是把“某函数必须先跑”变成可检查约束。若出现环，例如文件系统初始化依赖块设备，而块设备初始化又要从文件系统读取配置，就必须拆出更早的配置来源。

启动参数与可信边界

bootloader 会把 UEFI 内存地图、命令行、initrd 和 ACPI 表交给内核。内核不能盲信这些数据，因为它们可能来自固件 bug、配置错误或攻击面。早期解析必须检查地址范围、长度、对齐和重叠。

```

validate_memory_region(region):
    require region.start aligned to PAGE_SIZE
    require region.length > 0
    require checked_add(region.start, region.length, &end)
    require end <= physical_address_limit
    require not overlaps(kernel_image)
    require not overlaps(previous_accepted_regions)

```

若把内核镜像所在物理页误标为空闲，后续页分配器会把正在运行的内核代码分配给别人。启动阶段的错误通常不是立刻崩溃，而是在很久以后变成随机损坏，因此必须在早期就把内存地图打印和校验做好。

ACPI 和 PCI 配置空间：硬件不是凭空出现的

内核要知道“有哪些 CPU、哪些内存、哪些设备、设备寄存器在哪、用哪个 IRQ、挂在哪个 NUMA node、有哪些电源关系”。在本书的 x86-64 主线里，这部分信息主要来自 UEFI/ACPI、PCI/PCIe 配置空间和固件内存地图。它们都不是设备驱动本身，而是硬件描述入口。

描述来源	x86-64 常见用途	描述内容
ACPI	PC、服务器	CPU 拓 扑、NUMA、PCI routing、电 源、热 管 理、平台设备资源
PCI config	可枚举 PCIe 设备	vendor/device、BAR、MSI、能力链
UEFI memory map	启动阶段内存交接	RAM、保 留 区、runtime services、ACPI reclaim 区域

驱动 probe 的输入很多来自这些描述。若 ACPI resource 或 PCI BAR 错，MMIO 映射到错误地址；若 IRQ routing 或 MSI-X 配置错，设备完成不会唤醒驱动；若 DMA mask 或 IOMMU domain 配置错，设备可能访问不到高地址内存。硬件描述错误会表现成软件 bug，所以启动阶段要尽早暴露和校验资源表。

从单核启动到多核启动

多核系统通常只有 bootstrap processor 先运行内核入口，其他 CPU 处于等待状态。主 CPU 建立全局页表、per-CPU 区域和调度数据后，再唤醒 application processors。每个 CPU 都需要自己的内核栈、当前任务指针、runqueue 和中断状态。

```

boot_cpu_main():
    setup_global_memory()
    setup_percpu_area(boot_cpu)
    for cpu in secondary_cpus:
        allocate_cpu_stack(cpu)

```

```

    send_startup_ipi(cpu, secondary_entry)
    wait_until_all_online()
    start_scheduler()

secondary_entry(cpu):
    load_kernel_page_table()
    set_percpu_base(cpu)
    init_local_interrupts()
    mark_cpu_online(cpu)
    idle_loop()

```

这里的不变量是：CPU online 之前不能被调度器选作迁移目标；per-CPU 数据初始化之前不能接收普通设备中断；所有 CPU 使用的内核文本映射必须一致。多核启动把“初始化顺序”从单线流程变成了并发协议。

per-CPU 数据为什么必须很早准备

多核 OS 不能让所有 CPU 频繁写同一批全局变量。每个 CPU 需要自己的当前任务指针、内核栈、runqueue、计数器、本地 timer 状态、抢占计数和中断统计。这些对象通常放在 per-CPU 区域，通过特殊基址寄存器或固定偏移快速访问。

```

per_cpu_area[cpu]:
    current_task
    kernel_stack_top
    runqueue
    irq_count
    preempt_count
    local_timer_state

```

secondary CPU 进入内核后，必须先设置自己的 per-CPU base，才能安全访问 `current` 这类隐式变量。否则 CPU1 可能误用 CPU0 的当前任务或 runqueue，造成调度和中断统计混乱。per-CPU 是多核性能优化，也是正确性边界。

测试

- 链接脚本导出的段边界单调递增且按页对齐。
- BSS 清零后，全局计数器、指针和队列初值可预测。
- 早期分配器返回地址满足对齐，且不会覆盖内核镜像。
- 未初始化 console 前，日志路径不会访问空设备。
- 调度器 ready 前，任何初始化函数都不能睡眠。
- 页表 ready 后，text 只读可执行，data/bss 可写不可执行。

启动章节的验收标准是：能从第一条指令解释到第一个用户任务启动，中间每一阶段可用能力和禁止事项都清楚。

源码级补充：x86-64、UEFI 与 start_kernel

RESET 到长模式

x86 CPU 复位后不是直接处于 64 位长模式。传统路径会经历实模式、保护模式和长模式切换；现代 UEFI 固件和 bootloader 会替内核完成大量工作，但内核仍必须理解启动时 CPU 模式、页表、内存地图和调用约定。

```

RESET
-> firmware initializes platform
-> bootloader loads kernel image and initrd
-> build boot parameters and memory map

```

```

-> enter protected/long mode as required by protocol
-> jump to decompressor or kernel entry
-> early page tables
-> start_kernel()

```

实模式只能直接访问有限地址空间，保护模式引入段和保护，长模式引入 64 位寄存器和分页模型。操作系统教材不需要把每条切换指令都背下来，但必须知道：64 位内核能运行，是因为早期代码已经设置好 GDT、控制寄存器、EFER.LME、页表和跳转序列。

BIOS 与 UEFI 的差异

BIOS 风格启动常从磁盘引导扇区开始，bootloader 逐步加载更多代码。UEFI 则提供标准化的 Boot Services、Runtime Services、内存地图和 PE/COFF 应用加载环境。UEFI 下 bootloader 或 EFI stub 能以更结构化方式获得 GOP framebuffer、ACPI 表、initrd 和命令行。

项目	BIOS 风格	UEFI 风格
入口形态	引导扇区和 bootloader 阶段	EFI 应用或 EFI stub
固件服务	中断调用，模式受限	Boot Services/Runtime Services
内存地图	e820 等接口	GetMemoryMap
图形输出	VGA/vesa 等路径	GOP framebuffer
退出固件	bootloader 控制移交	ExitBootServices 后服务受限

`ExitBootServices` 是重要边界。退出前可以调用 UEFI Boot Services 分配内存、打开协议、读取设备；退出后内核必须依靠自己的驱动和内存管理。若退出前后内存地图不同步，内核可能误用固件保留页。

Linux start_kernel 调用序列

Linux 的 `start_kernel` 是阅读大型内核初始化的主干。它不是所有细节的开始，但它把架构初始化、内存、调度、RCU、中断、定时器、VFS、proc、driver model、init 进程串起来。

```

start_kernel()
  setup_arch()
  setup_command_line()
  mm_init()
  sched_init()
  radix_tree_init()
  trap_init()
  init_IRQ()
  tick_init()
  rcu_init()
  vfs_caches_init()
  rest_init()

```

不同版本顺序会变化，读源码时应抓依赖：`setup_arch` 解析固件和内存；`mm_init` 建立内存分配基础；`sched_init` 建立 runqueue；`trap_init/init_IRQ` 建立入口和中断；`rest_init` 创建 kernel thread 和第一个用户态 init 的路径。

initcall 机制

驱动和子系统不能全都手写进一个巨大初始化函数。Linux 用 `initcall` 等级把初始化函数放入特殊段，启动时按等级依次调用。常见等级包括 `early`、`core`、`postcore`、`arch`、`subsys`、`fs`、`device`、`late`。

```
early_initcall(foo_early);
core_initcall(mm_subsystem);
subsys_initcall(bus_subsystem);
fs_initcall(vfs_feature);
device_initcall(driver_init);
late_initcall(debug_late);
```

initcall 的实质是链接脚本和函数指针数组。它让模块能声明“我属于哪个初始化阶段”，但不能自动解决所有依赖。若一个 device initcall 假设某总线已注册，而总线初始化等级写错，启动会出现只在特定配置下复现的失败。

多核启动和 CPU hotplug

bootstrap processor 先运行全局初始化，然后通过 IPI 或固件接口唤醒 application processors。每个 CPU 上线前要设置 per-CPU base、内核栈、本地 APIC、idle task 和 runqueue。CPU hotplug 则要求运行中 CPU 可下线 and 上线，所以初始化/清理路径必须可重复、可回滚。

```
bringup_cpu(cpu):
    allocate_idle_task(cpu)
    allocate_percpu_area(cpu)
    send_startup_ipi(cpu)
    wait cpu reports online
    add cpu to scheduler domains
    enable interrupts on cpu
```

失败案例：CPU 已经标记 online 但本地 timer 未初始化，调度器可能把任务迁过去后无法抢占；CPU 下线时未迁走 timer 或 workqueue，后续事件投递到离线 CPU。多核启动不是“让其他核心开始跑”，而是把每个 CPU 纳入调度、中断和时间子系统的一致状态。

第 2 章 任务、调度与并发控制

2.1 任务模型与上下文切换

进程、线程与调度

原理

操作系统的第一个核心职责是分配 CPU 时间。硬件只有有限核心，用户却希望多个程序同时推进。进程提供资源边界，线程提供执行边界，调度器在可运行实体之间选择下一个执行者。这里的关键不是“轮流执行”四个字，而是每个执行实体必须处在可解释的状态里。

进程不是一个正在运行的函数。它至少包含地址空间、文件描述符表、信号处理状态、凭据、资源限制、子进程关系和一个或多个线程。线程是内核可调度实体，拥有寄存器上下文、栈、调度状态和等待原因；同一进程内的线程共享地址空间和许多进程资源，所以它们能高效共享数据，也更容易互相破坏。

进程生命周期可以简化为 created、runnable、running、blocked、zombie、reaped。线程状态可以简化为 runnable、running、sleeping、stopped、terminated。真实内核状态更多，但第一版必须先守住这些边界：blocked 线程不能被调度执行，terminated 线程不能重新运行，zombie 进程已经退出但仍等待父进程回收退出状态。

调度策略必须回答：谁处于 runnable，谁在 blocked，时间片用完后怎么办，优先级是否影响选择，任务退出后资源如何回收，唤醒是否会造成饥饿。一个可解释的调度器必须把这些状态外化，而不是只返回一个线程编号。

上下文切换是调度的成本边界。切换时内核要保存当前线程的寄存器、栈指针和调度账本，选择下一个线程，恢复它的上下文，并可能改变地址空间和 TLB 状态。线程切换不一定改变地址空间，进程切换通常会带来更重的内存管理成本。高并发程序如果制造过多 runnable 线程，就会把时间花在 runqueue 等待、抢占和上下文切换上。

等待是调度的另一半。一个线程不运行，可能是被抢占，也可能是主动睡眠，可能等锁、等 I/O、等 page fault、等子进程、等信号、等定时器。分析性能和死锁时，必须区分 on-CPU 时间和 off-CPU 时间。只看 CPU 使用率会误导：CPU 低可能是线程都阻塞了，CPU 高可能是大量线程在抢锁或忙等。

实践

纸上实践先画三张表。第一张是进程表，列出 PID、父 PID、状态、退出码、地址空间和 fd 表引用。第二张是线程表，列出 TID、所属 PID、状态、等待原因、优先级、最近运行 CPU 和已消耗 tick。第三张是 runqueue，列出当前可运行线程的顺序和调度账本。

tick	事件	runqueue	运行线程	状态变化
0	创建 A/B	A,B	A	A running
1	时间片结束	B,A	B	A runnable, B running
2	B 发起 I/O	A	A	B blocked(io)
3	I/O 完成	A,B	A	B runnable
4	A exit	B	B	A zombie

这张表比一句“轮转调度”更有价值，因为它同时呈现了状态、队列和事件。后续任何复杂策略都可以在这张表上增加字段，例如虚拟运行时间、deadline、优先级、CPU affinity、NUMA node 或 cgroup quota。

实践报告必须区分三个时间: wall time、runnable wait 和 on-CPU time。wall time 是请求从开始到结束的总时间; runnable wait 是线程已经可运行但还没拿到 CPU 的时间; on-CPU time 是真正执行指令的时间。把三者混在一起, 会把调度问题误判成算法问题。

算法与实现分析

FCFS: 最简单也最容易产生长等待

先来最简单的调度算法: first-come, first-served。任务按进入 runnable 队列的顺序执行, 一个任务运行到阻塞、退出或主动让出 CPU 后, 才轮到下一个任务。

```
while true:
    task = runqueue.pop_front()
    run_until_block_or_exit(task)
    if task.state == RUNNABLE:
        runqueue.push_back(task)
```

FCFS 的实现成本很低: 入队 $O(1)$, 出队 $O(1)$ 。它的问题是 convoy effect。一个长 CPU 任务排在前面, 后面的短交互任务会长时间等待。若系统只跑批处理, FCFS 可以作为 reference; 若系统需要交互响应, 它通常不合格。

Round-robin: 用时间片限制独占

Round-robin 在 FCFS 上增加时间片。每个 runnable 任务最多运行一个 quantum, 时间片到期后被放回队尾。它解决了“一个任务长期霸占 CPU”的问题, 但引入了时间片选择: 时间片太大, 交互延迟高; 时间片太小, 上下文切换成本高。

```
on timer_tick:
    current.used_ticks += 1
    if current.used_ticks == current.quantum:
        current.used_ticks = 0
        current.state = RUNNABLE
        runqueue.push_back(current)
        current = runqueue.pop_front()
```

假设上下文切换成本是 S , 时间片是 Q , CPU 时间被切换消耗的比例大约是 $S / (S + Q)$ 。 Q 越小, 响应越好, 但浪费越多。这个公式不是精确模型, 却能解释为什么时间片不能无限小。

优先级调度和饥饿

优先级调度把 runqueue 拆成多个队列, 总是选择最高优先级的 runnable 任务。它适合表达“控制任务比日志任务重要”, 但如果高优先级任务持续可运行, 低优先级任务可能永远得不到 CPU。

```
for priority from HIGH to LOW:
    if !queue[priority].empty():
        return queue[priority].pop_front()
return idle_task
```

解决饥饿的常见方法是 aging: 等待越久, 优先级逐步提高。aging 的实现要小心, 不能每个 tick 扫描所有任务, 否则任务多时成本过高。可以在任务入队时记录等待起点, 在选择或周期性维护时计算有效优先级。

MLFQ: 用反馈区分交互和批处理

Multilevel feedback queue 使用多个优先级队列, 新任务先进入高优先级; 如果任务用完整个时间片, 说明它偏 CPU 密集, 下次降级; 如果任务很快阻塞或让出 CPU, 说明它可能是交互任务, 保留较高优先级。MLFQ 不需要提前知道任务类型, 而是从行为反馈中调整。

```
on_task_created(task):
```

```

task.level = 0
queues[0].push_back(task)

on_quantum_expired(task):
    task.level = min(task.level + 1, MAX_LEVEL)
    queues[task.level].push_back(task)

on_task_blocked_before_quantum(task):
    queues[task.level].push_back_when_woken(task)

```

MLFQ 的关键参数包括层数、每层时间片、降级规则、周期性 boost。boost 用来防止低层任务永久饥饿：每隔一段时间，把所有 runnable 任务提升回高层。实现上要记录任务当前层级、已用时间、最近 boost epoch。

公平调度：虚拟运行时间

公平调度不只问“谁先来”，而是问每个 runnable 任务已经得到多少 CPU 服务。可以给每个任务维护虚拟运行时间 `vruntime`：任务实际运行越久，`vruntime` 越大；权重越高，虚拟时间增长越慢。每次选择 `vruntime` 最小的任务。

```

on_task_run(task, delta_exec):
    task.vruntime += delta_exec * NICE_0_WEIGHT / task.weight

pick_fair(rbtree):
    return leftmost_node_by_vruntime(rbtree)

```

若用红黑树保存 runnable 任务，插入、删除和选择下一个任务大约是 $O(\log n)$ ，取最左节点可缓存到接近 $O(1)$ 。这个模型解释了为什么 nice 值不是简单固定优先级：它影响获得 CPU 的比例，而不是让低优先级任务永远不运行。

deadline 调度与准入控制

实时或准实时任务还会声明运行时间和截止时间。例如任务说“每 20ms 需要 3ms CPU”。调度器若盲目接受所有请求，总利用率超过 100 后必然 miss deadline。因此 deadline 调度需要准入控制。

```

admit(task):
    new_util = total_runtime_over_period + task.runtime / task.period
    if new_util > CPU_CAPACITY:
        return -EBUSY
    add_to_deadline_class(task)
    return OK

pick_deadline():
    return task with earliest absolute_deadline

```

EDF 在单 CPU 理想模型中有很强性质，但真实系统还有抢占成本、临界区、缓存失效、中断关闭和共享资源阻塞。理论可调度不等于工程一定按时，测试必须记录 deadline miss、最大关抢占时间和锁等待。

调度指标

调度器优化必须有指标。常用指标包括平均响应时间、p95/p99 延迟、吞吐、上下文切换次数、公平误差、迁移次数、cache miss、deadline miss 和能耗。只报告“更快”没有意义。

指标	说明	常见误判
on-CPU time	真正执行指令的时间	把等待 I/O 误认为算法慢
runnable wait	可运行但没拿到 CPU 的时间	把调度拥塞误认为锁死

context switches	切换次数和成本	只看次数，不看每次成本
migration count	跨 CPU 迁移	均衡改善但缓存局部性变差
fairness error	实得 CPU 与权重目标差距	平均公平掩盖尾延迟

真实系统为什么更复杂

真实通用 OS 还要处理多核、NUMA、CPU affinity、实时任务、cgroup 配额、负载均衡和能耗。多核调度不是简单地把一个队列给所有 CPU，因为全局队列会产生锁竞争；每 CPU runqueue 能降低竞争，但需要负载均衡。迁移任务可以平衡负载，却会丢失 cache/TLB 热状态。调度器必须在公平、吞吐、延迟、局部性和能耗之间折中。

task 不是一个正在运行的函数

硬件正在执行的是寄存器、RIP、RSP、RFLAGS 和页表上下文。内核把这组执行状态包装成 task，再让 task 指向地址空间、文件表、信号状态、凭据、namespace 和 cgroup。task 是可调度实体，不是进程所有资源的简单大盒子。

字段类别	保存什么	为什么重要
执行现场	内核栈、保存寄存器、返回路径	被切走后还能从同一点继续
调度状态	state、policy、prio、vruntime、on_rq	判断能否运行、怎样排序、是否已入队
地址空间	mm、active_mm	进程切换可能要切页表和 TLB
资源指针	files、fs、signal、sighand	fork、exec、exit 要复制、共享或释放
安全身份	cred、namespace、cgroup	权限、限额和可见性都依赖它
亲和性	allowed CPUs、last CPU、NUMA 统计	负载均衡不能只看队列长度

state 和 on_rq 要分开理解。state 说明任务是 runnable、sleeping、stopped 还是 exiting；on_rq 说明它是否已经挂到某个 runqueue。一个 runnable 任务没有入队，会永远等不到 CPU；一个 sleeping 任务留在 runqueue，会被错误调度。调度器的大量锁和状态检查，就是为了让这两本账一致。

阻塞系统调用怎样离开 CPU

阻塞不是“函数停住”。内核要把当前任务从可运行集合移走，把它挂到等待队列，并保证事件到达时能找回它。以等待 pipe 或 socket 可读为例：

```
blocking_read(obj):
    lock(obj.lock)
    while !data_available(obj):
        prepare_to_wait(obj.read_waitq, current)
        current.state = TASK_INTERRUPTIBLE
        unlock(obj.lock)
        schedule()
        lock(obj.lock)
        finish_wait(obj.read_waitq, current)
    if signal_pending(current):
        unlock(obj.lock)
        return -EINTR
    copy_data_to_user()
    unlock(obj.lock)
```

```
return bytes
```

检查谓词和入等待队列必须受同一把锁保护。若先检查发现没有数据，还没入队时生产者写入并唤醒，唤醒会丢失；随后当前任务睡下，就形成 lost wakeup。谓词循环也不是模板写法：任务可能被信号唤醒，可能被虚假唤醒，也可能多个等待者竞争同一份数据。

唤醒怎样进入 runqueue

唤醒不等于立即运行。唤醒方通常只把等待任务改回 runnable，放到目标 CPU 的 runqueue；是否抢占当前任务，要看调度类、优先级、抢占状态和 CPU 位置。

```
wake_up(waitq):
    lock(waitq.lock)
    for task in waitq:
        if task.wait_condition_is_satisfied:
            remove_from_waitq(task)
            task.state = TASK_RUNNING
            enqueue_task(select_runqueue(task), task)
            if task_should_preempt_current(task):
                set_need_resched(target_cpu)
    unlock(waitq.lock)
```

若目标 CPU 正在用户态运行，`need_resched` 常在返回用户态、定时器中断返回或显式调度点处理。若目标 CPU 是另一个核心，内核可能发 IPI 让它尽快重新调度。这样看，wakeup 的成本包括等待队列锁、runqueue 锁、跨 CPU 通知和 cache line 迁移。

抢占点和不可抢占区

定时器中断可以打断用户态程序，但内核不能在任意位置无条件切走当前任务。持有自旋锁、关闭中断、正在修改 runqueue、正在操作每 CPU 数据时，随意抢占会破坏不变量。内核用 `preempt_count`、中断状态和锁规则标记这些区域。

```
timer_interrupt:
    account_runtime(current)
    if current.timeslice_expired:
        set_need_resched(current)
    if returning_to_user and need_resched:
        schedule()
    else if kernel_preemptible and preempt_count == 0:
        schedule()
```

实时系统关心的不是平均时间片，而是最长不可抢占区、最长关中断区和最高优先级任务被低优先级持锁路径阻塞的时间。一个系统平均响应很好，也可能因为某条持锁路径过长而出现不可接受的尾延迟。

fork、exec、wait 和调度器的连接

`fork` 创建的新任务不能在半初始化状态进入 runqueue。内核必须先准备内核栈、保存的用户寄存器、调度实体、地址空间引用、文件表引用、信号状态和 PID，再让调度器看到它。子进程第一次运行时不是从程序开头执行，而是从系统调用返回路径回到用户态，只是返回值寄存器被设置为 0。

```
fork scheduling boundary:
    allocate child task
    prepare child saved registers
    copy or share mm/files/signal/cred
    allocate pid and link parent-child relation
    enqueue child as runnable
    parent returns child pid
    child later returns 0 from copied frame
```


`exec` 成功后仍是同一个 `task` 被调度，但地址空间、用户栈、入口 `RIP`、部分信号处理和 `close-on-exec fd` 改变。`wait` 要求已退出子进程保留 `zombie` 壳，直到父进程读取退出码。调度器只负责不再运行 `zombie`；进程管理还要保证退出状态只被回收一次，最后一个引用释放 `task`。

源码阅读入口

进程创建看 `kernel/fork.c` 的 `copy_process`；退出和回收看 `kernel/exit.c` 的 `do_exit`、`release_task`；调度主干看 `kernel/sched/core.c` 的 `__schedule`、`try_to_wake_up`；公平类看 `kernel/sched/fair.c` 的 `enqueue/dequeue`、`vruntime` 和负载均衡。阅读时把每一步标成四类动作：改 `task` 状态、改 `runqueue`、改资源引用、通知等待者。

测试

调度测试至少覆盖三类情况：多个 `runnable` 进程应轮转推进；`blocked` 进程不应获得 `tick`；`terminated` 进程不应重新出现。还要检查没有 `runnable` 进程时，调度器返回 `idle`，而不是访问空队列。

- `fork` 后父子进程关系可解释，父进程能回收子进程退出状态。
- `exec` 后地址空间替换，但 `PID`、部分 `fd` 和进程身份按规则保留。
- 线程阻塞后从 `runqueue` 消失，唤醒后回到 `runnable` 集合。
- 时间片结束只改变调度状态，不应破坏进程资源。
- 没有 `runnable` 线程时进入 `idle`，而不是忙等消耗 `CPU`。
- 优先级或 `aging` 不得让低优先级任务永久饥饿，除非策略明确允许。

后续可以加入优先级、`deadline`、`affinity`、负载均衡和 `cgroup` 配额，但必须保留基础回归。任何复杂策略都不能让 `blocked` 或 `terminated` 任务被错误调度。

进程与调度实现深讲：从 `task` 到 `switch`

前面的章节已经讲了调度算法。本节把进程和调度器按内核实现拆开：进程对象有哪些字段，`fork` 如何复制资源，`exec` 如何替换地址空间，`wait` 如何回收子进程，调度器如何维护 `runqueue`，睡眠唤醒如何避免丢事件，上下文切换到底切了什么。

进程不是一个结构体，而是一组引用

Linux 中常用 `task_struct` 表示可调度实体。它不把所有资源都内嵌进去，而是引用地址空间、文件表、信号处理、凭据、命名空间、`cgroup`、调度实体等对象。这样线程可以共享一部分资源，`fork` 可以复制或共享不同对象，`exec` 可以替换地址空间但保留 `PID` 和部分文件描述符。

对象	回答的问题	生命周期事件
<code>task_struct</code>	这个可调度实体是谁、处于什么状态	<code>fork</code> 创建、 <code>exit</code> 释放
<code>mm_struct</code>	用户地址空间是什么	<code>fork</code> <code>COW</code> 、 <code>exec</code> 替换、 <code>exit</code> 释放
<code>files_struct</code>	<code>fd</code> 表指向哪些打开文件	<code>fork</code> 共享或复制、 <code>exec</code> <code>close-on-exec</code>
<code>fs_struct</code>	<code>cwd</code> 、 <code>root</code> 、 <code>umask</code> 等进程文件系统上下文	<code>chdir</code> / <code>chroot</code> / <code>fork</code>
<code>sighand</code> <code>cred</code>	信号处理函数和 <code>mask</code> <code>uid</code> 、 <code>gid</code> 、 <code>capability</code> 、安全标签	<code>fork</code> / <code>exec</code> / <code>signal</code> <code>setuid</code> 、 <code>exec</code> 、 <code>LSM</code> 检查

sched_entity

调度账本

enqueue、dequeue、
account runtime

源码入口可以从 `include/linux/sched.h`、`kernel/fork.c`、`kernel/exit.c`、`fs/exec.c` 看。阅读时要抓引用计数和共享边界：线程共享 `mm` 和 `files` 时，一个线程关闭 `fd` 会影响同进程其他线程。

fork 的实现顺序

`fork` 不是复制一整个进程镜像。内核先分配新的 `task`，复制或共享资源，建立父子关系，把子任务放入 `runnable` 集合，最后返回父子两个不同返回值。若中间失败，必须按已经成功的步骤逆序回滚。

```
fork sketch:
  allocate task_struct and kernel stack
  copy scheduler attributes
  copy or share mm_struct with COW PTEs
  copy or share files_struct
  copy signal and credential state
  assign pid
  link into parent children list
  make child runnable
  return child pid to parent, 0 to child
```

最容易错的是“什么时候可见”。子任务在所有必要状态初始化完成前不能进入 `runqueue`，否则另一个 CPU 可能立刻调度它，看到半初始化对象。父子关系也要在锁保护下建立，否则 `wait` 路径可能漏掉子进程。

fork 的 COW 页表细节

`fork` 复制页表时，匿名可写页通常不复制物理页，而是父子 PTE 都改成只读并标记 COW。物理页引用计数增加。任一方写入时，触发写保护缺页，内核复制新页，再把写入方 PTE 改为可写。

```
fork COW:
  for each writable anonymous PTE:
    clear write bit in parent PTE
    install read-only PTE in child
    mark both as COW
    increment page refcount
  flush parent TLB entries if permissions were tightened

write fault:
  if page refcount == 1:
    make PTE writable
  else:
    allocate new page
    copy old page contents
    decrement old page refcount
    map new page writable
```

注意父进程 PTE 也要收紧权限，否则父进程写入不会 `fault`，就会直接修改共享页，破坏 `fork` 语义。权限收紧后还要处理 TLB，否则 CPU 可能继续使用旧的 `writable` 翻译。

exec 的关键承诺

`exec` 成功后，当前进程身份大体保留，但用户地址空间被新程序替换。PID 不变，部分 `fd` 保留，设置了 `close-on-exec` 的 `fd` 关闭，信号处理按规则重置，凭据可能因 `setuid` 或文件 `capability` 改变。

```
execve sketch:
```

```

copy argv/envp from user memory
open executable and check permissions
identify binary format
create new mm_struct
map program segments
map interpreter if dynamically linked
create user stack with argc/argv/envp/auxv
apply credential transitions
commit new mm atomically
start at entry point

```

exec 失败要小心。理想情况下，在 commit 新地址空间前失败，旧程序仍能收到错误返回并继续运行；一旦已经不可逆地销毁旧地址空间，就只能终止进程。成熟内核会尽量把可能失败的检查和分配放在 commit 之前。

用户栈的构造

新程序启动时不是 C 的 main 被内核直接调用。内核把初始用户栈构造成 ABI 规定的布局，包括 argc、argv 指针数组、envp 指针数组、auxiliary vector 和字符串数据。入口点通常是动态链接器或 _start。

```

initial user stack high to low:
  strings for argv and envp
  alignment padding
  auxiliary vector entries
  NULL
  envp pointers
  NULL
  argv pointers
  argc

```

auxv 给用户态传递页大小、随机数地址、程序头信息、vDSO 入口等。动态链接器用这些信息装载共享库、处理重定位和准备 libc。OS 的 exec 不是“把文件读到内存跳过去”这么简单，它要建立完整 ABI 环境。

wait 和 zombie 的意义

子进程退出后不能立刻完全消失，因为父进程还可能调用 wait 获取退出状态和资源使用信息。于是退出进程进入 zombie 状态，保留 PID、退出码和少量统计；父进程 wait 后，内核释放最后对象。

```

exit:
  close files
  release mm
  record exit_code
  notify parent
  become ZOMBIE
  schedule away forever

wait:
  find child in ZOMBIE state
  copy exit status to parent
  unlink child from parent list
  free final task reference

```

若父进程不 wait，zombie 会留下。若父进程先退出，子进程会被 reparent 给 init 或 subreaper。这里的设计保证退出状态不丢，同时避免已退出任务再次运行。

runqueue 的最小不变量

每 CPU runqueue 至少要维护当前任务、runnable 任务集合、锁、调度时钟和统计。一个任务在同一时刻只能处于一个位置：某个 CPU 的 current，某个 runqueue，某个等待队列，或者退出路径。

- RUNNING 任务必须等于某个 CPU 的 current。
- RUNNABLE 任务必须在且只在一个 runqueue 中。
- SLEEPING 任务不能留在 runqueue 中。
- ZOMBIE 任务不能被调度器选中。
- 迁移任务时，源 runqueue 和目标 runqueue 的锁顺序必须固定。
- 改变任务状态和队列位置必须在同一临界区内完成。

调度 bug 常常不是 pick 算法错，而是不变量破坏：任务重复入队，睡眠任务未出队，唤醒时状态已经变化，迁移时两个 CPU 同时看到同一任务。

CFS 的核心数据结构

公平调度类用 `sched_entity` 记录虚拟运行时间、权重、统计和树节点。runqueue 中的 CFS 队列按 `vruntime` 排序，最左边是当前最“亏”的任务，优先运行。

```
CFS account:
    delta_exec = now - curr.exec_start
    weighted = delta_exec * NICE_0_LOAD / curr.weight
    curr.vruntime += weighted
    update rq.min_vruntime
```

```
CFS pick:
    next = leftmost entity in rb_tree
    return task_of(next)
```

`min_vruntime` 是队列的基准。新唤醒任务不能简单设置为 0，否则会长期占便宜；也不能设置得太大，否则交互响应差。实际实现会把新实体放在相对 `min_vruntime` 合理的位置，并配合唤醒抢占规则。

红黑树为什么合适

公平调度需要频繁插入、删除和取最小 `vruntime`。普通链表插入简单但查找最小慢；堆取最小快，但删除任意 sleeping 任务不方便；红黑树能在 $O(\log n)$ 完成插入、删除和按 key 排序。

结构	优点	缺点
FIFO 队列	$O(1)$ ，简单	不能表达公平权重和 <code>vruntime</code> 排序
优先级数组	固定优先级快	动态公平排序不自然
堆	取最小快	删除任意实体复杂
红黑树	排序、插入、删除平衡	常数成本高于队列

算法选择来自操作需求。调度器不是为了展示数据结构，而是因为任务会频繁睡眠、唤醒、迁移、改变权重，必须支持这些操作。

睡眠唤醒的无丢失顺序

等待队列正确性的核心是同一把锁保护“条件检查”和“入队睡眠”。否则唤醒可能发生在检查之后、入队之前，等待者永远睡下去。

```
correct wait:
    lock(obj.lock)
    while !condition:
        prepare_to_wait(waitq, current, SLEEPING)
        unlock(obj.lock)
        schedule()
```

```

    lock(obj.lock)
    finish_wait(waitq, current)
    unlock(obj.lock)

waker:
    lock(obj.lock)
    change condition
    wake_up(waitq)
    unlock(obj.lock)

```

很多真实内核代码会用更复杂的 helper，但它们都在维护这个原则。条件变量、pipe、socket、futex、completion、wait event 都逃不开同一条规则。

上下文切换切的是什么

上下文切换分成调度器层和架构层。调度器选择 next，更新 current、统计、状态；架构层保存 callee-saved 寄存器、栈指针和必要控制状态，切到 next 的内核栈，再返回到 next 之前停下的位置。

```

schedule:
    prev = current
    next = pick_next_task(rq)
    if prev != next:
        rq.current = next
        context_switch(prev, next)

context_switch:
    switch address space if needed
    switch kernel stack
    save prev callee-saved registers
    restore next callee-saved registers
    return as next task

```

这里有一个反直觉点：context_switch 返回时，执行流已经变成 next 任务。prev 将来被调度回来时，会从它当初离开的地方继续返回。这就是线程切换能伪装成普通函数返回的原因。

地址空间切换和 lazy TLB

若两个线程共享同一个 mm_struct，切换时不需要换页表根。若切到不同进程，通常要切换页表根或地址空间标识。切换页表会影响 TLB；x86-64 PCID 可以减少全量刷新。

```

switch_mm(prev_mm, next_mm):
    if prev_mm == next_mm:
        keep current page table root
    else:
        load next page table root with PCID when enabled
        ensure stale translations cannot be used

```

内核线程可能没有自己的用户地址空间，常借用上一个进程的 mm 或使用特殊内核地址空间。lazy TLB 这类优化的目标，是减少无意义的页表切换和 TLB 刷新。

负载均衡和调度域

多核系统不是一个平面。SMT sibling、同一核心、同一 LLC、同一 NUMA node、跨 socket 的迁移成本不同。调度器通常建立调度域，从近到远做负载均衡。

```

load balance levels:
    SMT siblings
    cores sharing LLC
    CPUs in same NUMA node
    CPUs across NUMA nodes

```

迁移一个任务可能减少 runnable wait，却损失 cache 热状态并增加远端内存访问。x86-64 服务器和工作站上，调度器要估计 SMT sibling、共享 LLC、NUMA node、跨 socket 迁移、turbo 预算和功耗封顶。任务放错位置会让锁竞争、cache miss、远端内存访问和尾延迟一起变差。

cgroup CPU 配额

容器和服务隔离需要限制一组任务的 CPU 使用。cgroup CPU controller 可以按周期给 group 分配 runtime budget。预算耗尽后，该组任务被 throttled，直到下个周期补充预算。

```
cgroup quota:
  period = 100ms
  quota = 20ms
  tasks in group may run total 20ms per period
  after budget exhausted:
    throttle group
  at next period:
    refill runtime and unthrottle
```

这会产生典型现象：容器内 CPU 使用看似没满，但请求出现周期性延迟尖峰，因为 group 被配额限流。性能分析必须看 cgroup throttling 统计，而不是只看宿主主机总体 CPU。

调度实现测试

- fork 后子任务不能在初始化完成前进入 runqueue。
- COW fork 后父子任一方写入都不能修改另一方可见内容。
- exec 失败路径不得破坏旧程序继续运行的必要状态。
- wait 必须能回收 zombie，并且不能回收非子进程。
- 睡眠和唤醒并发时不能 lost wakeup。
- 迁移任务时不能同时出现在两个 CPU 的 runqueue。
- cgroup quota 耗尽时 group 被 throttle，周期补充后能恢复。
- 上下文切换后 callee-saved 寄存器、栈和地址空间必须正确。

第二阶段深讲：进程生命周期不是 fork/exec/wait 三个词

进程章节容易被讲成系统调用列表：fork 创建，exec 装载，wait 回收，exit 退出。这样的讲法太薄。真实内核维护的是一组互相引用的对象：任务、地址空间、文件表、信号状态、凭据、命名空间、调度实体、父子关系和资源统计。生命周期问题的核心是：什么时候对象对其他 CPU 可见，什么时候能失败，失败后怎样回滚，最后一个引用在哪里释放。

从硬件执行流到 task

硬件只知道当前 CPU 正在执行一组寄存器、RIP、栈和页表根。操作系统把这组状态包装成 task。task 不等于进程的全部资源；它是可调度实体，指向更大的资源集合。

资源	为什么不直接内嵌进 task	生命周期问题
地址空间	线程可共享，exec 可替换	引用计数、COW、TLB shutdown
文件表	fork 可共享，dup 共享 open file	close-on-exec、并发 close/read

信号处理	进程级和线程级状态交织	handler、mask、pending queue
凭据	安全检查频繁读取，变更较少	RCU 发布、setuid/exec 转换
命名空间	容器内多个进程共享视图	引用计数、权限边界
调度实体	每个线程都要被调度	runqueue、vruntime、CPU affinity

这种“task 指向对象”的结构让共享和替换成为可能。同一进程内多个线程共享 mm 和 files; exec 替换 mm，但保留 PID 和部分 fd; fork 复制 task，同时复制或共享不同对象。若把所有资源平铺在一个大结构里，这些语义会变得混乱。

创建进程的真正顺序

创建进程不能先把半成品放进 runqueue。只要任务进入 runnable 集合，另一个 CPU 就可能立刻运行它。因此 fork 的顺序必须从“不可见半成品”逐步推进到“对调度器可见”。

```
copy_process outline:
  allocate task and kernel stack
  initialize minimal scheduler state
  copy credentials or install new references
  copy/share files, fs, signal state
  copy mm with COW page tables if needed
  allocate pid
  link child into parent relationships
  publish task to pid tables
  make task runnable only after invariants hold
```

每一步都有失败可能：内存不足、pid 不足、文件表复制失败、页表复制失败、权限检查失败。错误路径必须按逆序释放已经拿到的引用。这里的质量不在正常路径跑通，而在第 5 步失败时前 4 步是否完整回滚。

阶段	成功后新增什么	失败时必须回滚什么
分配 task	task 内存和内核栈	释放栈和 task 对象
复制 files	fd 表引用增加	put 每个 open file 引用
复制 mm	页表页、PTE 引用、COW 标记	释放页表页、撤销引用
分配 pid	pid 映射占用	释放 pid,不能留下僵尸映射
链接父子	父进程 child list 可见	从列表摘除,避免 wait 看到半成品
入队运行	调度器可选择任务	之后失败通常不能简单回滚，要走退出路径

这张表解释了为什么内核代码里有大量 goto bad_*。这些标签表达的是阶段化回滚：从哪个阶段失败，就只回滚已经成功取得的资源。高质量内核代码宁愿显式写出阶段，也不能让半初始化对象泄漏到全局结构。

fork 后父子为什么返回不同值

fork 的一个经典语义是父进程返回子 PID，子进程返回 0。硬件没有“两个返回值”的指令，内核通过修改子任务保存的寄存器帧实现这个效果。父进程系统调用正常返回时，返回值寄存器写入 child pid；子任务第一次被调度时，从复制出的内核返回路径继续，返回值寄存器已经被设置为 0。

```
parent path:
  ret = child.pid
```



```

return_to_user(parent_regs with ret)

child prepared frame:
child_regs = copy(parent_regs)
child_regs.return_value = 0
when scheduled, return_to_user(child_regs)

```

这个细节能帮助读者理解上下文切换：子进程不是从 `main` 开头重新执行，也不是内核调用了两次用户函数。它从和父进程相同的用户 PC 返回，只是寄存器状态略有不同。

exec 是不可逆提交点

`execve` 成功后，旧程序映像消失，新程序从入口点运行。难点在于失败语义：在提交点之前失败，应返回错误给旧程序；提交点之后失败，旧程序已经没有完整地址空间可恢复，只能终止任务。

```

exec phases:
  copy argv/envp from old user memory
  open executable and check permissions
  parse binary format
  allocate new mm
  map text/data/interpreter/stack
  prepare credentials and auxv
  commit new mm and credentials
  jump to new entry

```

因此 `exec` 要尽量把可能失败的分配、权限检查和格式解析放在 `commit` 前。`commit` 动作要原子地切换 `mm`、设置新用户栈和入口、处理 `close-on-exec` fd、重置信号处理规则、应用 `setuid/capability` 变化。这个点之后，内核不能再假装旧程序还在。

线程创建和资源边界

线程和进程的差别不是“更轻量”四个字。线程共享地址空间、文件表和许多进程级状态，但拥有独立的可调度上下文、用户栈、内核栈、TLS、信号 `mask` 和调度状态。共享使通信便宜，也让错误传播更广。

资源	进程间 fork 后	同进程线程间
地址空间	初始 COW，之后可分离	共享同一 <code>mm</code>
fd 表	可复制或共享，取决于语义	通常共享， <code>close</code> 影响其他线程
用户栈	子进程有复制视图	每线程独立栈
信号 mask	每线程独立 mask	进程 pending 与线程 pending 并存
PID/TID	父子不同 PID	同 TGID，不同 TID

线程库还要处理 TLS 和启动 `trampoline`。内核创建新线程时准备一个初始寄存器状态，使它第一次返回用户态时进入用户态线程入口，参数指向线程函数和参数对象。线程退出时要清理用户栈、TLS、`robust futex`、`join` 状态和内核 `task` 引用。

exit 不是立刻消失

任务调用 `exit` 后，不能简单释放自己。它可能还在别的表里有引用，父进程还要读取退出码，`ptrace` 或 `wait` 还可能观察它。于是退出分成多个阶段：停止用户执行、释放大资源、记录退出状态、通知父进程、进入 `zombie`，最后由 `wait` 或 `reaper` 释放 `task`。

```
do_exit:
```

```
mark task exiting
close files and release mm
release futexes and timers
record exit_code and resource usage
notify parent and reparent children
become zombie
schedule away forever

wait:
    find zombie child
    copy status to parent
    unlink child
    put final task reference
```

zombie 是一个必要状态：它保留父进程需要读取的结果，同时保证已退出任务不会再被调度。如果父进程不 wait，系统要通过 init 或 subreaper 接管，防止 zombie 永久积累。

父子关系和 reparent

进程树不是装饰。父进程负责 wait 子进程；信号传播、作业控制、会话、进程组都依赖关系结构。若父进程先退出，子进程不能失去回收者，内核会把它们 reparent 给 init 或最近的 subreaper。

```
parent exits:
    for each child:
        choose new_parent = nearest subreaper or init
        move child to new_parent.children
        if child is zombie:
            notify new_parent
```

这里的并发点是父子列表、退出状态和 wait 扫描必须在锁保护下保持一致。否则 wait 可能漏掉刚退出的子进程，或者子进程被两个父对象同时看到。

进程生命周期的测试矩阵

进程生命周期测试应该覆盖正常路径、失败路径和并发路径。

测试	关键断言	风险
fork 基本路径	父返回 child pid，子返回 0	寄存器帧设置错误
fork 失败注入	每个阶段失败后引用数归零	资源泄漏或半成品可见
fork COW	父子写入互不污染	PTE/TLB 权限错误
exec 成功	PID 保留，地址空间替换	fd/signal/cred 规则错误
exec 失败	commit 前失败旧程序继续	旧 mm 被提前破坏
exit/wait	zombie 被 wait 回收一次	重复回收或泄漏
多线程 close/read	共享 fd 表语义一致	use-after-close 或引用泄漏
reparent	父退出后子由 reaper 回收	zombie 泄漏

如果一本书只讲 fork/exec/wait 的定义，不讲这些断言，读者就无法写出可靠内核，也无法读懂真实源码里的复杂错误路径。

2.2 调度算法与多核工程

调度器深入：时间、等待与多核

原理

上一章讲了基本调度算法，本章把调度器当成内核子系统看。调度器不是一个简单的 `pick_next()` 函数，它维护可运行队列、睡眠队列、时间片账本、优先级、CPU 亲和性、负载均衡、抢占标志和统计数据。它还必须和锁、定时器、中断、系统调用返回路径配合。

调度器的核心循环是：当前任务运行；某个事件改变任务状态；如果当前任务不能继续运行或应该让出 CPU，内核选择下一个 `runnable` 任务；保存当前上下文，恢复下一个上下文。事件可能来自时间片耗尽、任务阻塞、任务退出、任务被唤醒、高优先级任务到达或 CPU 负载均衡。

调度器维护两个不同问题。第一是选择：在 `runnable` 集合里选谁。第二是状态转换：任务如何进入 `runnable`、`running`、`sleeping`、`stopped`、`zombie`。很多 bug 不在选择算法，而在状态转换，例如任务已经 `sleeping` 却仍留在 `runqueue`，或者唤醒丢失导致任务永远睡眠。

实践

实践报告要把调度器拆成四张表：

表	内容
任务表	pid/tid、状态、优先级、时间片、等待原因
<code>runqueue</code>	每个 CPU 的 <code>runnable</code> 任务和顺序
<code>sleep queue</code>	等待对象、超时时间、唤醒条件
统计表	on-CPU 时间、 <code>runnable wait</code> 、 <code>context switch</code> 、迁移次数

当报告说“调度不公平”时，必须给出 `runnable wait` 分布；当报告说“线程没运行”时，必须说明它是 `runnable` 但没拿到 CPU，还是 `sleeping` 等事件。

算法与实现分析

睡眠和唤醒

阻塞系统调用和条件变量最终都要把任务放入某个等待队列。正确顺序是：持有保护等待条件的锁，检查条件，不满足则把当前任务加入等待队列并把状态改为 `sleeping`，然后原子地释放锁并调度。唤醒方改变条件后，从等待队列取任务，改为 `runnable`，放入 `runqueue`。

```
sleep(wait_queue, lock):
    current.state = SLEEPING
    wait_queue.push(current)
    unlock(lock)
    schedule()
    lock(lock)

wake_one(wait_queue):
    task = wait_queue.pop()
    if task != null:
        task.state = RUNNABLE
        enqueue(runqueue_for(task), task)
```

这里最危险的是 `lost wakeup`。如果等待者检查条件后、真正入队前，唤醒者改变条件并发出唤醒，那么等待者可能永远睡眠。解决方法是用同一把锁保护“检查条件”和“加入等待队列”。

抢占点

内核不能在任意机器指令之间随便切换。抢占点通常出现在中断返回、系统调用返回、显式阻塞、时间片耗尽或内核允许抢占的位置。如果当前代码持有自旋锁或处于中断上下文，就不能睡眠调度。

```
maybe_preempt():
    if need_reschedule
        and preempt_count == 0
        and !in_interrupt():
        schedule()
```

`preempt_count` 是实现边界的典型例子。它不是业务概念，而是内核用来防止在不可抢占区域切走任务的计数。进入自旋锁或中断上下文时增加，退出时减少。若计数错误，可能在持锁时调度，导致死锁。

每 CPU runqueue

多核系统若使用一个全局 runqueue，所有 CPU 都要争同一把锁。每 CPU runqueue 降低锁竞争：每个 CPU 主要从自己的队列取任务；周期性负载均衡把任务从繁忙 CPU 迁到空闲 CPU。

```
schedule_on_cpu(cpu):
    rq = cpu.runqueue
    lock(rq.lock)
    next = pick_next(rq)
    unlock(rq.lock)
    switch_to(next)

load_balance():
    busiest = find_busiest_cpu()
    idle = find_idle_cpu()
    migrate_some_tasks(busiest.rq, idle.rq)
```

多核调度的取舍是局部性和均衡。任务留在原 CPU，cache/TLB 更热；迁移到空闲 CPU，等待时间更低。调度器不能只看队列长度，还要看任务亲和性、NUMA 节点、迁移成本和负载类型。

实时任务

实时调度关注 `deadline`，而不只是平均公平。Rate Monotonic 适合周期任务，周期越短优先级越高；Earliest Deadline First 总是选择绝对 `deadline` 最近的任务。单处理器上，在理想条件下 EDF 可以达到更高利用率，但实现要维护按 `deadline` 排序的队列。

```
pick_edf(runqueue):
    return min_by(runqueue, task.absolute_deadline)
```

实时系统必须做可调度性分析。若任务集合的最坏执行时间总和超过 CPU 能力，任何调度算法都无法保证 `deadline`。操作系统只能执行策略，不能创造时间。

唤醒抢占

一个高优先级或交互任务被 I/O 唤醒时，如果只是放到 runqueue 尾部，可能仍要等当前 CPU 密集任务用完整个时间片。唤醒抢占的思想是：唤醒路径比较新任务 and 当前任务，若新任务更应该运行，就设置 `need_reschedule`，让返回路径尽快切换。

```
try_wake_task(task):
    rq = select_runqueue(task)
```

```

lock(rq.lock)
task.state = RUNNABLE
enqueue(rq, task)
if should_preempt(task, rq.current):
    rq.current.need_reschedule = true
    send_reschedule_ipi_if_remote(rq.cpu)
unlock(rq.lock)

```

这里有两个风险。第一，过度抢占会增加上下文切换，降低吞吐。第二，跨 CPU 唤醒需要 IPI，若频率过高，会把多核机器变成互相打断的系统。调度器必须在响应延迟和扰动成本之间折中。

负载均衡的粒度

负载均衡不是简单平均任务数量。一个 CPU 上有 1 个 CPU 密集任务，另一个 CPU 上有 20 个大部分睡眠的任务，按数量看不平衡，按实际负载可能合理。更好的指标是 runnable load、最近 CPU 使用、任务权重和亲和性。

```

load_balance_domain(domain):
    busiest = cpu with highest runnable_load
    idle = cpu with lowest runnable_load
    if busiest.load <= idle.load + imbalance_threshold:
        return
    task = choose_migratable_task(busiest, idle)
    if task != null:
        migrate(task, busiest, idle)

```

迁移必须持有源和目标 runqueue 锁，并规定锁顺序，例如按 CPU id 从小到大加锁。否则两个 CPU 同时互相迁移任务，会在 runqueue 锁上形成死锁。

调度器不变量

调度器测试要检查不变量，而不是只看任务有没有输出。

- 每个 RUNNABLE 线程恰好在一个 runqueue 中。
- RUNNING 线程只能是某个 CPU 的 current。
- SLEEPING 线程必须在一个等待队列中，或带有可解释超时。
- ZOMBIE 进程不能再获得 CPU，但退出状态必须可被父进程读取。
- 同一线程不能同时出现在迁移源队列和目标队列。
- 持有自旋锁、关中断或 `preempt_count > 0` 时不能调度到会睡眠的路径。

测试

- lost wakeup 测试：唤醒和睡眠并发发生时，等待者不会永久睡眠。
- 持有自旋锁时不能进入会睡眠的路径。
- 每 CPU runqueue 中，同一任务不能同时出现在两个队列。
- 任务迁移后，旧 CPU 不再能调度该任务。
- 时间片耗尽、阻塞、退出和唤醒都能触发正确重调度。
- 实时任务 deadline miss 必须被记录，而不是静默忽略。

调度与并发深讲：从“能跑”到“可解释地推进”

调度器和同步原语经常被分开讲，但真实系统中二者紧密绑定。线程等待锁会离开 CPU；I/O completion 会唤醒线程；定时器会打断当前线程；优先级和锁持有时间会相互影响；关闭路径要唤醒所有等待者。若只学调度算法，不学等待语义，就无法解释“线程为什么不运行”。

线程不运行的六种原因

用户看到“程序卡住”，内核可能处在完全不同的状态。

原因	内核状态	诊断方向
等 CPU	RUNNABLE 但不在 CPU 上	runnable wait、runqueue 长度、优先级
等锁	SLEEPING 在 mutex/futex 等待队列	持锁者、锁顺序、临界区长度
等 I/O	SLEEPING 在 completion 等待队列	块层队列、设备延迟、writeback
等页面	page fault 或内存回收	major fault、swap、page cache miss
等事件	poll/epoll/timer/condition variable	等待谓词、关闭路径、超时
已退出	ZOMBIE 或 TERMINATED	父进程是否 wait、资源是否释放

这张表比“调度器选择下一个任务”更重要。调度器只能从 runnable 集合里选任务；一个 sleeping 任务不运行，通常不是调度策略问题，而是等待条件没有满足或唤醒丢失。

完整的阻塞读路径

以 pipe 的阻塞读为例。读者发现缓冲区为空，不能忙等，否则浪费 CPU。它必须把自己放入等待队列，改变状态为 sleeping，然后调度走。写者写入数据后，改变条件并唤醒读者。

```

pipe_read(pipe, buf, len):
    lock(pipe.lock)
    while pipe.empty and not pipe.write_closed:
        current.state = SLEEPING
        enqueue(pipe.read_waitq, current)
        schedule_unlocking(pipe.lock)
        lock(pipe.lock)
    if pipe.empty and pipe.write_closed:
        unlock(pipe.lock)
        return 0
    n = copy_from_pipe(pipe, buf, len)
    wake_waiters(pipe.write_waitq)
    unlock(pipe.lock)
    return n

pipe_write(pipe, buf, len):
    lock(pipe.lock)
    while pipe.full and not pipe.read_closed:
        sleep_on(pipe.write_waitq, pipe.lock)
    if pipe.read_closed:
        unlock(pipe.lock)
        return -EPIPE
    n = copy_to_pipe(pipe, buf, len)
    wake_waiters(pipe.read_waitq)
    unlock(pipe.lock)
    return n

```


这里有多重边界：读端关闭要唤醒写者，写端关闭要唤醒读者；读者被信号打断时要返回 `EINTR` 或部分结果；非阻塞 `fd` 应返回 `EAGAIN` 而不是睡眠。一个合格 OS 模型必须把这些边界都写出来。

优先级反转

优先级反转是并发与调度相互影响的典型问题。低优先级线程 `L` 持有锁，高优先级线程 `H` 等这个锁，中优先级线程 `M` 持续运行，导致 `L` 得不到 CPU 释放锁，`H` 虽然优先级高却无法推进。

```
L: lock(A), do work slowly
H: tries lock(A), blocks
M: CPU-bound, runnable

without inheritance:
  scheduler runs M before L
  H waits for L indirectly

with priority inheritance:
  L temporarily inherits H priority
  scheduler runs L so it can release A
```

priority inheritance 不是让锁更快，而是让等待关系反馈给调度器。实现要维护“谁在等谁”的依赖图，释放锁时撤销继承。若多把锁嵌套，继承链可能跨多个线程，测试必须覆盖链式等待。

死锁调试的步骤

死锁不是“程序没输出”。它是等待图出现环。调试时先列出每个线程持有的资源和等待的资源。

```
Thread A: holds inode.lock, waits page.lock
Thread B: holds page.lock, waits inode.lock

wait-for graph:
  A -> page.lock -> B
  B -> inode.lock -> A
```

解决死锁的常用手段是全局锁顺序、trylock 回退、缩短临界区、拆分锁或使用 RCU。不要用“加更多锁”解决死锁，更多锁通常只会增加等待图边数。正确方法是减少共享状态、明确所有权和规定顺序。

调度和同步练习

1. 给三个任务 `A/B/C`，分别是 CPU 密集、周期 I/O、交互输入，手算 FCFS、RR、MLFQ 的等待时间。
2. 构造一个 lost wakeup 的时序，并说明哪一把锁应同时保护等待谓词和等待队列。
3. 画出 priority inversion 的等待图，写出 priority inheritance 后每个线程的临时优先级。
4. 为一个有界队列写关闭协议：生产者关闭、消费者关闭、双端同时关闭分别怎样唤醒。
5. 设计一个测试，证明 blocked 线程永远不会被调度器选中。

调度算法对比：同一负载下的不同结论

考虑三个任务：`A` 是长 CPU 任务，需要 30 个 tick；`B` 是交互任务，每运行 1 tick 就等待 I/O 5 tick；`C` 是短任务，只需要 3 tick。不同调度策略会给出完全不同的用户体验。

策略	表现	问题
FCFS	若 A 先到，C 和 B 长时间等待	convoy effect，交互体验差
RR	A 不能长期霸占 CPU，B/C 能穿插运行	时间片过小会增加切换成本
优先级	B 可以更快响应输入	低优先级任务可能饥饿
MLFQ	通过行为反馈照顾交互任务	参数复杂，可能被任务行为利用
公平调度	长期 CPU 份额接近权重	响应延迟需要额外 wakeup/preempt 规则
EDF	deadline 最近者先运行	需要准入控制，过载时仍会 miss

这说明没有“唯一正确”的调度算法。调度器服务于目标：批处理吞吐、桌面交互、实时 deadline、容器公平、能耗控制或尾延迟。教材中算法比较的目的，不是背哪个更先进，而是理解每个算法优化了什么、牺牲了什么。

关闭路径是并发程序的必修课

很多同步教材只讲生产者消费者的正常路径，却不讲关闭。真实 OS 对象一定会关闭：pipe 端点关闭、socket 断开、进程退出、设备拔出、文件释放。关闭路径的核心规则是：改变关闭状态后，必须唤醒所有可能等待这个状态的线程。

```

queue_close(q):
    lock(q.lock)
    q.closed = true
    wake_all(q.not_empty)
    wake_all(q.not_full)
    unlock(q.lock)

queue_pop(q):
    lock(q.lock)
    while q.empty and not q.closed:
        sleep(q.not_empty, q.lock)
    if q.empty and q.closed:
        unlock(q.lock)
        return CLOSED
    item = pop(q)
    wake_one(q.not_full)
    unlock(q.lock)
    return item

```

若关闭只唤醒消费者，不唤醒生产者，正在等待空间的生产者可能永远睡眠。若关闭状态不受同一把锁保护，等待者可能错过关闭通知。关闭路径是检验并发模型是否完整的最好办法。

常见误区

- “线程没运行就是调度器不公平”：它可能根本不在 runnable 集合里。
- “加锁就安全”：锁必须保护正确对象，并且所有路径都遵守同一锁规则。
- “notify 等于发送消息”：notify 只唤醒等待者，条件仍要重新检查。
- “自旋锁比 mutex 快”：临界区短且不能睡眠时自旋才合适，长等待会浪费 CPU。

- “测试跑过一次没有死锁”：死锁是时序问题，需要锁顺序和等待图分析。

从时间片轮转到反馈队列

Round Robin 的优点是简单：每个 runnable 任务轮流运行一个时间片。它解决了 FCFS 中长任务霸占 CPU 的问题，但它无法区分交互任务和批处理任务。交互任务经常运行很短时间就阻塞等待 I/O，如果它每次被唤醒后还要排在长 CPU 任务后面，用户会感觉系统迟钝。

多级反馈队列 (MLFQ) 的核心是用任务行为估计任务类型：刚创建或刚被唤醒的任务放在较高优先级；任务用完整个时间片，说明偏 CPU 密集，降低优先级；任务很快阻塞，说明偏交互或 I/O 密集，保持或提升优先级；周期性提升所有任务，防止饥饿。

```
on_tick(current):
    current.slice_used += 1
    if current.slice_used == quantum[current.level]:
        demote(current)
        enqueue(current)
        schedule()

on_sleep(current):
    current.sleep_start = now

on_wakeup(task):
    if now - task.sleep_start >= interactive_threshold:
        promote(task)
    enqueue(task)
```

MLFQ 的难点不是队列，而是参数。时间片太短，上下文切换成本高；时间片太长，交互延迟差；提升太频繁，CPU 任务占便宜；提升太少，低优先级任务饥饿。工业调度器通常不会直接照搬教科书 MLFQ，而是把类似思想融入更复杂的权重、虚拟运行时间和唤醒抢占规则。

公平调度的虚拟运行时间

公平调度不承诺每次都平均，而承诺长期 CPU 份额接近权重。一个常见思路是维护每个任务的虚拟运行时间 `vruntime`。任务实际运行越久，`vruntime` 越大；权重越高，`vruntime` 增长越慢。调度器选择 `vruntime` 最小的任务，让落后的任务追上来。

```
account_runtime(task, delta_exec):
    task.vruntime += delta_exec * NICE_0_WEIGHT / task.weight

pick_next(rq):
    return leftmost_task_by_vruntime(rq.tree)
```

若用平衡树保存 runnable 任务，选择最小 `vruntime` 是 $O(\log n)$ 插入删除加 $O(1)$ 或 $O(\log n)$ 取最小，具体取决于实现是否缓存最左节点。这个算法牺牲了极简实现，换来权重公平和较平滑的延迟。它仍然需要补充唤醒抢占、最小时间粒度、CPU affinity 和负载均衡，否则单靠 `vruntime` 无法覆盖完整系统需求。

实时调度：可调度性比算法名字重要

实时系统关注 deadline。固定优先级调度中，Rate Monotonic 把周期短的任务设为更高优先级；动态优先级调度中，EDF 选择绝对 deadline 最近的任务。它们都不能解决过载：如果任务最坏执行时间总和超过 CPU 能力，deadline miss 只是时间问题。

算法	需要的输入	失败模式
----	-------	------

RM	周期、最坏执行时间、固定优先级	高优先级任务过多时低优先级 miss
EDF	绝对 deadline、剩余执行时间	过载时大量任务连续 miss
CBS/预算调度	runtime budget、period、deadline	预算耗尽后任务被限速

实时任务必须接受准入控制。准入控制回答“把这个任务加入后，已有保证是否还能成立”。没有准入控制，所谓实时优先级只是让某些任务更早失败。测试也必须记录 deadline miss、最大响应时间和抢占延迟，而不是只证明任务最终运行过。

futex：用户态快路径和内核等待队列

互斥锁若每次加锁解锁都进入内核，成本很高。futex 的思想是：无竞争时完全在用户态用原子操作完成；只有竞争时才进入内核，把线程挂到与某个用户地址关联的等待队列上。

```
mutex_lock(m):
    if atomic_compare_exchange(m.state, 0, 1):
        return
    while true:
        old = atomic_exchange(m.state, 2)
        if old == 0:
            return
        futex_wait(&m.state, expected=2)

mutex_unlock(m):
    old = atomic_exchange(m.state, 0)
    if old == 2:
        futex_wake(&m.state, 1)
```

关键是 `futex_wait` 必须原子地检查用户地址仍等于 `expected`，再入队睡眠。否则 `unlock` 可能在检查和入队之间发生，造成 `lost wakeup`。futex 展示了一个重要原则：OS 不需要接管所有同步操作，它只需要在用户态快路径失败时提供可靠睡眠和唤醒。

信号、取消和可中断睡眠

阻塞调用不能只考虑“等到事件发生”。进程可能收到信号，线程可能被取消，文件描述符可能被关闭，设备可能拔出。内核等待点要定义是否可中断，以及中断后返回什么。

```
wait_event_interruptible(q, condition):
    lock(q.lock)
    while !condition:
        if signal_pending(current):
            unlock(q.lock)
            return -EINTR
        sleep(q, q.lock)
    unlock(q.lock)
    return 0
```

可中断等待提高响应性，但让调用者必须处理部分完成。例如 `read` 已复制 100 字节后收到信号，通常应返回 100，而不是丢弃已完成工作；若 0 字节完成才被信号打断，才返回 `EINTR`。这种细节决定系统调用是否可组合。

并发对象的状态机写法

讲同步不能只写“加锁”。每个共享对象都应该有状态机。以有限队列为例，状态包括 `open`、`closing`、`closed`；计数包括 `item` 数和等待者；事件包括 `push`、`pop`、`close producer`、`close consumer`、`timeout`、`signal`。

当前状态	事件	动作
open, empty	pop blocking	消费者入 not_empty 等待队列
open, full	push blocking	生产者入 not_full 等待队列
open	close producer	标记 no_more_push, 唤醒消费者
open	close consumer	标记 no_more_pop, 唤醒生产者
closed, empty	pop	返回 EOF/CLOSED
closed/full	push	返回 EPIPE/CLOSED

状态机有两个好处。第一，它迫使设计者写出关闭路径和错误路径。第二，它直接变成测试矩阵：每个状态和事件组合至少有一个测试或证明。

同步算法复杂度和适用边界

不同同步原语服务不同场景。自旋锁避免睡眠切换，适合极短临界区和中断上下文；mutex 允许睡眠，适合可能等待较久的临界区；读写锁适合读多写少，但写者饥饿要处理；RCU 让读者几乎无锁，代价是更新路径复杂且对象释放要等 grace period。

原语	适合	不适合
spinlock	短临界区、不能睡眠上下文	I/O、缺页、可能阻塞路径
mutex	普通内核对象互斥	中断上下文、自旋锁内层
rwlock	读多写少且临界区短	写频繁或读者长期持有
semaphore	资源计数与限流	表达复杂对象不变量
RCU	读多、遍历多、更新少	需要立即释放或强一致更新

原语选择错误会直接变成性能或正确性问题。把长 I/O 放在自旋锁内，会浪费 CPU 甚至死锁；用读写锁保护写多对象，会让写者互相等待且读者无收益；用 RCU 保护需要立即撤销权限的对象，可能留下过期访问窗口。

并发测试：从随机到可复现

并发测试不能只靠压力跑。压力测试能暴露问题，但失败时如果没有事件日志和可控调度，很难复现。更可靠的方法是把关键同步点做成可插桩事件，枚举或随机探索不同交错。

```
test_lost_wakeup():
    pause_after(reader, "checked_empty")
    writer.write(byte)
    writer.wake()
    resume(reader)
    assert(reader.eventually_returns(byte))
```

高质量并发测试至少记录：线程 id、对象 id、锁 acquire/release、状态改变、入队/出队、睡眠/唤醒、错误返回和超时。测试失败时要能画出等待图或事件序列。否则“偶现卡住”会变成无法定位的黑盒。

源码级补充：task_struct 与 Linux 调度类

Linux 的调度对象不是一个抽象“线程”词条，而是 task_struct 加上调度类私有实体。读 include/linux/sched.h、kernel/sched/core.c、kernel/sched/fair.c 时，先找这些字段：pid/tgid、state、flags、mm、active_mm、files、signal、sighand、ptrace、cgroups、cpus_mask、se、rt、dl。

字段或对象	作用
<code>task_struct</code> .	<code>runnable</code> 、 <code>interruptible sleep</code> 、 <code>uninterruptible sleep</code> 等状态
<code>mm/active_mm</code>	用户地址空间和内核线程借用地址空间
<code>files</code>	fd table, fork/clone 时按标志共享或复制
<code>signal/sighand</code>	进程级信号状态和 handler 表
<code>sched_entity se</code>	CFS 的 <code>vruntime</code> 、权重、红黑树节点
<code>cpus_mask</code>	CPU affinity, 限制可运行 CPU 集合
<code>cgroups</code>	CPU、memory、io 等资源控制归属

进程创建读 `kernel/fork.c` 的 `copy_process`：它根据 `CLONE_*` 标志决定共享还是复制 `mm`、`files`、`sighand`、`thread group`。进程退出读 `kernel/exit.c` 的 `do_exit`、`exit_notify` 和 `wait` 路径：退出不是立即删除 `task`，而是留下 `zombie` 供父进程读取退出状态。

fork、clone、vfork 和线程

`fork` 复制进程抽象，地址空间用 COW；`clone` 用标志精确控制共享哪些对象；`vfork` 暂时共享地址空间并挂起父进程直到子进程 `exec/exit`；NPTL 线程通常是共享 `mm`、`files`、`sighand` 的 `clone` 任务。`TLS`、`set_tid_address`、`robust futex list` 都和线程退出清理有关。

```
clone flags for pthread-like thread:
CLONE_VM      share address space
CLONE_FS      share fs context
CLONE_FILES   share fd table
CLONE_SIGHAND share signal handlers
CLONE_THREAD  same thread group
CLONE_SETTLS  set thread-local storage base
```

这解释了为什么“进程”和“线程”在内核里不是两种完全不同对象，而是同一种 `task` 在资源共享标志上的不同组合。

上下文切换：switch_mm、FPU 和 TLB

`__schedule` 选择 `next` 后，底层切换要保存/恢复 CPU 上下文、切换内核栈、切换地址空间、处理 lazy FPU 状态。若 `prev` 和 `next` 属于同一 `mm`，可能避免完整地址空间切换；若不同 `mm`，需要更新页表基址并处理 TLB/PCID。

```
context_switch(prev, next):
    prepare_task_switch(prev, next)
    if prev.mm != next.mm:
        switch_mm(prev.mm, next.mm)
    switch_to(prev, next) // registers and stack
    finish_task_switch(prev)
```

上下文切换成本不只是保存寄存器。`cache/TLB` 热状态丢失、跨 CPU 迁移、FPU/向量寄存器保存和安全缓解措施都会影响成本。因此调度器要在公平、延迟和局部性之间折中。

调度类顺序

Linux 调度类有优先顺序：`stop`、`deadline`、`realtime`、`fair`、`idle`。核心调度路径从高到低询问各类是否有可运行任务。

```
pick_next_task(rq):
    for class in [stop, dl, rt, fair, idle]:
        p = class.pick_next_task(rq)
        if p != null:
```



```
return p
```

stop class 用于极高优先级的 CPU 控制任务；deadline 处理 `SCHED_DEADLINE`；rt 处理 `SCHED_FIFO/RR`；fair 处理普通任务；idle 在无事可做时运行。若实时任务配置错误，普通 CFS 任务会长期得不到 CPU，所以实时调度必须配合权限和 runtime 限制。

CFS、红黑树和唤醒抢占

CFS 用 `vruntime` 表示任务已获得的加权 CPU 服务，runqueue 用红黑树按 `vruntime` 排序，选择最左任务。关键参数包括 `sched_latency`、`min_granularity`、`wakeup_preempt` 规则和 `task weight`。

```
enqueue_task_fair(rq, p):
    update_curr(rq)
    place_entity(p.se)
    rb_insert_by_vruntime(rq.cfs_tasks, p.se)

pick_next_task_fair(rq):
    return rb_leftmost(rq.cfs_tasks)
```

唤醒抢占不是简单“新任务来了就抢”。它比较新任务的 `vruntime`、交互延迟、当前任务运行时间和最小粒度。过度抢占会提高响应但增加上下文切换；抢占太少会让交互任务尾延迟变差。

cgroup、NUMA balancing 与能源调度

cgroup CPU controller 用权重、配额和周期限制任务组 CPU 时间。NUMA balancing 通过采样页访问和迁移任务/页面改善内存局部性。能源感知调度 EAS 和 schedutil 把任务放置、CPU capacity、频率调节和能耗模型联系起来。

机制	调度影响
CPU quota	组用完周期预算后 throttled，即使有 runnable 任务也不能运行
CPU weight	多组竞争时按权重分配长期份额
CPU affinity	限制任务可运行 CPU，可能改善局部性也可能造成热点
NUMA balancing	在任务迁移和页面迁移之间寻找更低远端访问成本
EAS/schedutil	结合 CPU capacity 和频率，平衡性能与能耗

诊断“调度不公平”时必须看 cgroup throttling、affinity、rt/deadline 任务、NUMA 迁移和 CPU 频率。只看 runqueue 长度会漏掉资源控制和硬件拓扑。

第二阶段深讲：调度器工程化，从算法到多核策略

调度算法课常用 FCFS、RR、优先级、MLFQ、CFS、EDF 做比较，但内核调度器不是算法博物馆。真实调度器要在中断、锁、cache、NUMA、cgroup、实时任务、能耗和安全边界之间做决策。本节关注算法落地后的工程问题：运行队列怎么组织，唤醒如何抢占，负载如何迁移，配额如何限流，证据如何解释。

调度器只从 runnable 集合里选

很多性能分析误把“线程没运行”归因于调度器。调度器只负责选择 runnable 任务；如果任务 sleeping，它等待的是锁、I/O、page fault、timer、信号或条件变量。正确诊断必须先问任务在哪个集合中。

任务位置	含义	调度器能否直接运行
CPU current	正在运行或在内核入口中	已经在运行
runqueue	runnable, 等待 CPU	可以被选择
wait queue	条件不满足, 睡眠中	不能, 必须被唤醒
completion/request	等设备或异步结果	不能, completion 路径唤醒
zombie list	已退出, 等父进程 wait	不能

这个区分决定了工具选择。runnable wait 高, 查 runqueue 和 CPU 配额; sleep wait 高, 查锁、I/O 或等待对象; zombie 多, 查父进程是否 wait。没有这个分类, 调度分析会变成猜测。

每 CPU runqueue 为什么必要

单一全局 runqueue 实现简单, 但多核上所有 CPU 都要竞争一把锁, 且每次调度都会移动同一组 cache line。每 CPU runqueue 把局部调度变成局部锁, 降低争用; 代价是负载不均时需要跨 CPU 迁移。

```
per-cpu scheduling:
    local timer tick updates current CPU's rq
    local wakeup usually enqueues on target CPU rq
    idle CPU tries to pull tasks from busy CPU
    periodic balance compares load across domains
```

这就产生了调度域: SMT sibling、同一 LLC、同一 NUMA node、跨 node。近距离迁移成本低, 远距离迁移成本高。调度器不能只看任务数量, 还要看任务负载、cache 热度、内存位置和 CPU 能力。

唤醒路径比定时器路径更影响交互延迟

交互任务多数时间在睡眠, 事件到来后被唤醒。如果唤醒后还要等很久才运行, 用户会感到卡顿。因此调度器不仅要在时间片耗尽时选择任务, 还要在 wakeup 时判断是否应抢占当前任务。

```
wakeup(task):
    task.state = RUNNABLE
    place task on target runqueue
    if task should preempt current:
        set reschedule flag
    if remote CPU:
        send reschedule IPI
```

这里有两个难点。第一, 唤醒任务放到哪个 CPU: 放回上次 CPU 有 cache 局部性, 放到空闲 CPU 有低延迟, 放到数据所在 NUMA node 有内存局部性。第二, 是否抢占当前任务: 过度抢占会增加切换成本, 抢占不足会增加交互延迟。

CFS 的 min_vruntime 是队列坐标系

只讲 vruntime 还不够。每个 CFS runqueue 还维护 min_vruntime, 表示队列已经推进到的公平时间基准。新任务和唤醒任务不能简单设为 0, 否则它们会在树最左边占便宜; 也不能放得太靠后, 否则唤醒延迟差。

```
enqueue_fair(task, rq):
    if task is new:
        task.vruntime = rq.min_vruntime
    if task wakes from sleep:
        task.vruntime = max(task.vruntime, rq.min_vruntime - wakeup_granularity)
    insert task into rb_tree ordered by vruntime
```

min_vruntime 让每个 runqueue 有自己的坐标系。迁移任务时, 需要把任务的 vruntime 从源 runqueue 坐标转换到目标 runqueue 坐标, 否则任务可能因为跨

CPU 迁移得到不公平优势或惩罚。

EEVDF 思路：从公平份额到最早合格截止

现代公平调度进一步引入虚拟 deadline 思路。直观地说，任务不仅有“已经用了多少 CPU”的账本，还可以有“按权重和 slice 计算出的虚拟截止时间”。调度器倾向选择已经 eligible 且虚拟 deadline 最早的实体，以改善延迟和公平之间的折中。

```
entity fields:
    vruntime
    weight
    slice
    virtual_deadline = vruntime + slice / weight_scaled

pick:
    among eligible entities
    choose earliest virtual_deadline
```

这里不需要把工业实现细节全部搬进原理卷，但要说明动机：单纯最小 `vruntime` 关注长期公平，交互延迟还要通过 `slice`、`wakeup` 和 `eligibility` 调节。调度器演进的方向，是把“公平”和“延迟目标”放进同一个可维护账本。

实时调度和普通调度的边界

实时任务不能和普通 CFS 任务只靠 `nice` 值竞争。实时调度类通常优先于普通调度类，FIFO/RR 实时任务能长时间压制普通任务。`deadline` 调度还要处理 `runtime`、`period`、`deadline` 和准入控制。

调度类	目标	风险
stop/deadline	CPU 停止、deadline 保证	最高优先级路径错误会冻结系统
real-time	固定优先级实时响应	饥饿普通任务，需要限额
FIFO/RR		
fair	权重公平与通用吞吐	不能保证硬实时
idle	没有其他任务时运行	不应影响正常调度

实时任务需要限额和监控，否则一个死循环实时线程可以让系统失去响应。工业内核通常提供 RT throttling，防止实时任务占满全部 CPU 时间。

cgroup 配额造成的周期性延迟

容器或服务组常被配置 CPU quota。假设周期 100ms，配额 20ms。组内任务在本周期累计运行 20ms 后会被 throttle，即使机器还有空闲 CPU，也要等下个周期补充预算。

```
cgroup runtime:
    on task execution:
        group.runtime_remaining -= delta
    if group.runtime_remaining <= 0:
        throttle all runnable tasks in group
    at period timer:
        refill group.runtime_remaining
        unthrottle group
```

这种机制会形成很典型的尾延迟：请求前 20ms 很快，预算耗尽后突然等待到周期刷新。排查容器内延迟时，必须看 `throttled time`、`throttled periods` 和 `quota` 配置，而不是只看宿主 CPU 利用率。

优先级反转是调度器和锁的交叉问题

低优先级任务持锁，高优先级任务等锁，中优先级任务占 CPU，会导致高优先级任务间接被中优先级任务阻塞。调度器单独看 `runnable` 集合时看不出问题，因为真正持有资源的是低优先级任务。

```
priority inheritance:
  H waits on lock owned by L
  boost L effective priority to at least H
  if L waits on another owner X:
    propagate boost to X
  when lock released:
    recompute effective priority
```

实现上要维护 `owner`、`waiters`、`effective priority` 和继承链。释放锁时不能简单恢复原优先级，因为任务可能还持有另一个被高优先级任务等待的锁。这个例子说明同步原语不能和调度器完全隔离。

调度器证据：如何解释一次延迟

一次请求慢，要按时间分段：

```
request latency =
  runnable wait
+ on-cpu execution
+ sleeping on I/O or lock
+ page fault/reclaim
+ scheduler and interrupt overhead
```

常用证据包括 `perf sched`、`/proc/<pid>/sched`、`tracepoint`、`off-CPU profile`、`/proc/schedstat`、`cgroup CPU` 统计、`pidstat -w`。不同证据回答不同问题。

证据	回答的问题	层次
<code>/proc/<pid>/sched</code>	任务运行、等待、迁移等统计	单任务
<code>perf sched</code>	调度事件时间线	系统级
<code>cgroup cpu.stat</code>	是否被 <code>quota throttle</code>	服务组
<code>tracepoint</code>	<code>wakeup</code> 、 <code>switch</code> 、 <code>migrate</code> 事件	内核事件
<code>sched_*</code>		
<code>off-CPU profile</code>	睡眠等待在哪里发生	锁/I/O

调度器优化要先定位是哪一段时间异常。若大部分时间在 `off-CPU` 等锁，改 `CFS` 参数不会解决；若大部分时间 `runnable wait`，才应看 `CPU` 配额、优先级、`runqueue` 和负载均衡。

调度器工程验收

- 能解释 `task` 为什么不在 `CPU`: `runnable`、`sleeping`、`zombie`、`throttled` 要分清。
- 能说明每 `CPU runqueue` 与负载均衡的取舍。
- 能推导唤醒抢占对交互延迟和上下文切换成本的影响。
- 能解释 `vruntime`、`min_vruntime` 和迁移坐标转换的关系。
- 能说明 `cgroup quota` 为什么会制造周期性尾延迟。
- 能用调度事件证据区分 `on-CPU` 慢和 `off-CPU` 等待。

2.3 同步、IPC 与等待状态机

并发、同步、IPC 与信号

原理

多线程程序的核心风险不是“同时执行”这个事实，而是共享状态缺少清晰的不变量。两个线程同时修改一个队列，硬件只能执行 load、store、CAS、fence 等局部操作；它不知道 head、tail、size、closed 之间应该满足什么关系。操作系统和运行库提供 mutex、condition variable、semaphore、futex、pipe、signal 等机制，帮助程序把共享状态、等待和唤醒写成可检查的协议。

互斥锁保护的不是某个变量，而是一组字段之间的不变量。条件变量也不是消息队列，它表达的是“某个谓词暂时不满足，线程可以睡眠，等状态可能改变后再重新检查”。正确等待必须同时包含谓词、锁和退出路径。只写 `wait()` 而不写完整谓词，会遇到虚假唤醒、丢失唤醒和关闭卡死。

一个有界队列足以说明同步原理。队列状态包括 buffer、capacity、head、tail、size、closed。生产者等待 `size < capacity || closed`，消费者等待 `size > 0 || closed`。push 改变 buffer、tail、size；pop 改变 buffer、head、size；close 改变 closed 并唤醒所有等待者。任何一个字段在锁外修改，都会破坏状态机。

IPC 是跨进程同步和数据交换。pipe 把字节流连接到 fd，socket 把通信扩展到本机或网络，shared memory 共享页但不自动提供同步，message queue 提供消息边界，eventfd 把计数型事件暴露成 fd readiness。IPC 的关键不是哪个接口高级，而是数据在哪里、所有权如何转移、背压如何发生、关闭如何传播。

信号是异步控制流。它可以通知进程发生了外部事件，例如终端中断、子进程退出、非法内存访问、定时器到期。信号难学，是因为它打断正常执行路径，而且信号处理函数只能安全调用很少一部分函数。工程上常把信号转成更普通的事件，例如写 pipe、设置原子标志或使用 signalfd，再交给主循环处理。

实践

同步实践先写不变量表。

字段	含义	受谁保护
capacity	最大槽数，固定大于 0	构造后只读
head	下一个可读槽	mutex
tail	下一个可写槽	mutex
size	当前元素数量	mutex
closed	是否拒绝新写入	mutex

然后写等待谓词：

- 生产者等待：`size < capacity || closed`
- 消费者等待：`size > 0 || closed`
- 关闭等待：`running_workers == 0` 或 `queue_empty && all_workers_stopped`

IPC 实践也按同样方式写。pipe 的状态包括读端引用数、写端引用数、内核缓冲区、读写阻塞队列。写端关闭后，读端读空返回 EOF；读端关闭后，写端写入会失败或收到信号；缓冲区满时写阻塞或返回 EAGAIN；缓冲区空时读阻塞或返回 EAGAIN。

信号实践要避免在 handler 里做复杂逻辑。推荐的纸上模型是：handler 只记录最小事实，例如设置原子标志；主循环在安全位置检查标志并执行真正清理。若系统使用 signalfd，则信号也进入 fd/event loop 模型，和其他 I/O 事件统一处理。

算法与实现分析

环形缓冲区

环形缓冲区是设备驱动、pipe、socket buffer 和日志队列的基础结构。它用固定数组保存数据，用 head 指向下一个可读位置，用 tail 指向下一个可写位置。若额外维护 size，判断空满很直接；若不维护 size，则常留一个空槽，用 head == tail 表示空，用 next(tail) == head 表示满。

```
bool push(Ring* r, byte b) {
    if (r->size == r->capacity) return false;
    r->buf[r->tail] = b;
    r->tail = (r->tail + 1) % r->capacity;
    r->size += 1;
    return true;
}

bool pop(Ring* r, byte* out) {
    if (r->size == 0) return false;
    *out = r->buf[r->head];
    r->head = (r->head + 1) % r->capacity;
    r->size -= 1;
    return true;
}
```

push/pop 都是 O(1)。难点不在算法复杂度，而在并发所有权。单生产者单消费者可以用两个原子索引实现无锁队列；多生产者多消费者通常先用 mutex 保护不变量，等有证据证明锁是瓶颈，再考虑更复杂结构。

信号量

信号量表达可用资源数量。P/acquire 操作在计数大于 0 时减一，否则等待；V/release 操作加一并唤醒等待者。它适合连接池许可、队列空槽、I/O in-flight 限额，不适合保护多字段复杂不变量。

```
acquire(sem):
    lock(sem.lock)
    while sem.count == 0:
        wait(sem.cv, sem.lock)
    sem.count -= 1
    unlock(sem.lock)

release(sem):
    lock(sem.lock)
    sem.count += 1
    notify_one(sem.cv)
    unlock(sem.lock)
```

信号量的常见 bug 是许可泄漏：acquire 成功后，错误路径忘记 release。工程上要用作用域对象或 finally 路径保证释放。另一个 bug 是把信号量当成关闭广播；关闭是额外状态，必须单独表达。

条件变量

条件变量的算法核心是“检查谓词、睡眠、醒来后重新检查”。虚假唤醒不是异常，而是接口允许的行为；因此 wait 必须写在 while 循环中。

```
lock(m)
while !predicate():
    wait(cv, m)
// predicate is true while holding m
do_state_transition()
unlock(m)
```

notify 的语义也要讲清：通知不是把数据交给等待者，只是说明谓词可能变真。若生产者先通知再修改状态，消费者醒来仍看不到新状态；若修改状态后忘记通知，

等待者可能永远睡眠；若关闭时只通知一个等待队列，其他等待者可能卡住。

IPC 的背压算法

pipe 和 socket 都需要背压。缓冲区满时，写入者要么阻塞，要么在非阻塞模式返回 `EAGAIN`。背压算法的本质是有界队列：

```
write(pipe, bytes):
    while bytes not empty:
        if pipe.buffer_full():
            if nonblocking: return bytes_written_or_EAGAIN
            sleep_until_not_full_or_closed()
        copy_some_bytes_into_buffer()
        wake_readers()
```

这里的 `copy_some_bytes` 说明短写是正常路径。只要缓冲区剩余空间小于请求长度，写入就可能部分完成。调用者必须把“已完成多少字节”作为状态保存，而不是假设一次 `write` 完成整个消息。

测试

并发测试要覆盖正常路径、等待路径和关闭路径。只测单线程 `push/pop` 没有意义，因为它没有触发同步问题。

- 多生产者多消费者下，元素不丢、不重、不乱破坏队列不变量。
- 队列空时消费者等待，生产者 `push` 后消费者能醒来。
- 队列满时生产者等待，消费者 `pop` 后生产者能醒来。
- `close` 能唤醒所有生产者和消费者，不留下永久睡眠线程。
- `timeout` 或 `cancel` 路径不会破坏 `size/head/tail`。
- `pipe` 读写能处理 `EOF`、`EPIPE`、`EAGAIN` 和短读短写。
- 信号只改变允许改变的最小状态，不在 `handler` 中执行不安全操作。

测试报告必须写出等待谓词和唤醒来源。若测试只说“不会死锁”，但没有说明哪个线程等什么、谁负责唤醒、关闭时怎样退出，这个测试不可维护。

源码级补充：Linux IPC、信号与事件 fd

pipefs 与 pipe_buffer

Linux 管道不是用户态数组，而是内核 `pipe inode` 和一组 `pipe_buffer`。每个 `buffer` 可以引用匿名页、`page cache` 页或 `splice` 传递的页片段。管道容量有默认值和上限，常见默认容量是 65536 字节量级，但会受内核版本和配置影响。

```
pipe_write(file, iov):
    pipe = file.private_data
    lock(pipe.mutex)
    while pipe_full(pipe):
        if nonblocking: return -EAGAIN
        wait(pipe.wr_wait, pipe.mutex)
    copy_or_splice_pages_into_pipe_buffers()
    wake_readers(pipe.rd_wait)
    unlock(pipe.mutex)
```

读源码时看 `fs/pipe.c: pipe_write、pipe_read、pipe_wait、pipe_buffer`。关注点不是函数名，而是关闭语义：写端全关且缓冲为空时读返回 `EOF`；读端全关时写失败并可能触发 `SIGPIPE`；非阻塞模式返回 `EAGAIN`。

System V 与 POSIX IPC

System V IPC 提供 semaphore、message queue、shared memory，典型接口是 `semget/semop`、`msgget/msgsnd/msgrcv`、`shmget/shmat`。POSIX IPC 提供命名 semaphore、POSIX message queue 和共享内存对象，接口如 `sem_open`、`mq_open`、`shm_open`。

机制	特点	适用边界
System V semaphore	内核维护计数和 undo 语义	传统系统和跨进程资源计数
POSIX semaphore	named/unnamed 两类	进程间或线程间许可控制
System V msgqueue	类型化消息，内核队列	需要消息边界但吞吐不极致
POSIX mq	描述符化，可通知	与 fd/event loop 集成更方便
shared memory	共享页，不自带同步	大数据交换，需 futex/锁配合

共享内存常和 futex 一起使用：数据放在共享页，锁字也放在共享页；无竞争时用户态原子操作完成，有竞争时 futex 让内核按这个用户地址建立等待队列。这样数据路径不必每次进入内核。

eventfd、timerfd、signalfd 和 memfd

Linux 把很多事件转成 fd，是为了让 epoll 能统一等待。

接口	内核语义	常见用途
eventfd	64 位计数器，read 消费，write 增加	线程/进程通知、io_uring、KVM kick
timerfd	定时器到期次数可读	event loop 中处理超时
signalfd	把被 mask 的信号作为 fd 读取	避免复杂 async signal handler
memfd	匿名内存文件，可 seal	共享内存、沙箱传递只读 blob

eventfd 的 EFD_SEMAPHORE 模式每次 read 只消耗 1，否则 read 返回当前计数并清零。这个小差异决定它表达的是“累计事件”还是“许可”。memfd sealing 可以禁止继续写、增长或收缩，让发送方把一段内存变成稳定对象再交给接收方。

信号投递路径

信号不是立即调用用户函数。内核先把信号加入 pending 集合；线程返回用户态前检查 pending signal；若未被屏蔽且有 handler，内核在用户栈上构造 signal frame，把返回地址改成 handler；handler 执行后通过 sigreturn 恢复原上下文。

```
send_signal(task, sig, info):
    add_to_pending(task.signal_pending, sig, info)
    if task sleeping interruptibly:
        wake_up(task)

return_to_user(task):
    if has_unblocked_pending_signal(task):
        build_signal_frame_on_user_stack()
        task.regs.ip = handler_address
        task.regs.sp = signal_frame_sp
```

这解释了为什么 handler 可用函数受限：它运行在异步插入的用户控制流里，可能打断 malloc、printf、锁持有路径。signalfd 的工程价值在于把异步信号改造成普通 fd 事件，降低重入风险。

IPC 性能决策树

需求	推荐思路
少量控制事件	eventfd、pipe、socketpair
需要消息边界	Unix datagram socket、POSIX mq、自定义帧协议
大块数据同机传输	shared memory + futex/eventfd 通知
要跨机器扩展	TCP/QUIC/RPC，明确超时、重试和幂等
要把文件数据转发给 socket	sendfile、splice、zero-copy 路径
要统一事件循环	fd 化 接 □: epoll + eventfd/timerfd/signalfd

性能不是只看平均延迟。管道有内核拷贝和容量上限；共享内存吞吐高但同步复杂；socket 语义完整但协议开销高；message queue 保留边界但容量和优先级策略会影响尾延迟。选择 IPC 时要写出数据大小、频率、是否需要消息边界、是否跨主机、背压和关闭语义。

内核锁、死锁与可证明的等待

原理

操作系统进入并发世界以后，错误不再只来自单个函数写错变量，而来自两个执行流同时触碰同一份状态。中断处理程序可能和普通内核线程同时访问设备队列；两个 CPU 可能同时把任务放入 runqueue；一个进程关闭 fd 时，另一个线程可能正在通过同一个 open file description 发起 I/O。锁的目的不是让代码“看起来串行”，而是给共享对象规定一个可证明的临界区：谁能读，谁能写，写入完成前谁必须等待，等待时是否允许睡眠。

从裸机单片机演进过来时，最早的“锁”通常是关中断。它简单有效，因为只有一个 CPU，且并发主要来自中断打断主循环。进入多核后，关本地中断不能阻止另一个 CPU 访问同一对象，于是需要自旋锁；进入可能长时间等待的路径后，自旋浪费 CPU，于是需要 mutex、semaphore 和等待队列。每一种同步原语都绑定一个上下文限制：中断上下文不能睡眠；持有自旋锁时不能调用可能阻塞的函数；持有文件系统锁时不能随意回调用户内存复制路径；调度器自己的锁必须在切换前后保持严格顺序。

核心理念

锁不是性能技巧，而是状态所有权的边界。设计锁时先写对象不变量，再决定哪些操作必须在同一把锁下完成。

内核锁设计要同时回答四个问题：保护哪个对象，持锁期间允许做什么，等待者在哪里排队，失败路径如何释放。很多死锁来自第四个问题被忽略：正常路径释放锁，错误路径提前返回却忘了释放；或者关闭路径拿到 A 锁后等待 B，而另一个路径拿到 B 后等待 A。

实践

下面是常见内核锁原语的实践边界：

原语	适用场景	禁止事项	典型对象
----	------	------	------

关中断	单核短临界区，中断共享变量	多核共享保护，长时间计算	tick 计数、单核设备寄存器影子状态
自旋锁	多核短临界区，不能睡眠的上下文	持锁 I/O、持锁 page fault、持锁等待用户事件	runqueue、描述符环、引用计数
mutex	进程上下文中可能等待的互斥	中断上下文获取，持有后进入不可重入路径	inode 元数据、文件系统目录更新
读写锁	读多写少且临界区短	长读者导致写者饥饿，升级锁顺序混乱	mount 表、路由表快照
RCU	读路径极热，读者不阻塞写者	立即释放旧对象，读区内睡眠	fdtable、路径缓存、配置表
semaphore	计数资源，允许多个持有者	用它表达复杂对象不变量	buffer 数量、并发 I/O 限额

一个合理的锁设计文档至少包含下面字段：

```
Object:
    runqueue[cpu]

Invariant:
    task.state == RUNNABLE implies task is on exactly one runqueue
    rq.nr_running == number of tasks linked from rq.head

Lock:
    rq.lock protects rq.head, rq.nr_running, task.rq_link

Allowed while holding:
    enqueue/dequeue, choose_next, update vruntime

Forbidden while holding:
    page allocation with reclaim, disk I/O, copy_from_user, waiting on mutex
```

等待队列必须把“检查条件”和“入队睡眠”放在同一把锁保护下，否则会出现 lost wakeup。正确模型不是“先看条件，不满足就睡”，而是“在保护条件的锁内，把自己放到等待集合中，然后原子地释放锁并切换状态”。唤醒方也在同一把锁下改变条件并唤醒等待者。

算法与实现分析

自旋锁

最小自旋锁可以用原子交换实现：

```
lock(spinlock *l):
    while atomic_exchange(l->held, true) == true:
        cpu_relax()

unlock(spinlock *l):
    atomic_store_release(l->held, false)
```

这个算法的互斥性来自原子交换：同一时刻只有一个 CPU 能看到旧值为 false。复杂度不是固定常数，因为竞争时等待时间取决于持锁者运行时间和 CPU 调度。实现上必须使用 acquire/release 语义，否则 CPU 或编译器可能把临界区内存访问重排到锁外。真实内核还会加入持有者记录、递归检查、抢占关闭、IRQ 保存和等待统计。

普通 test-and-set 自旋锁在高竞争下会让所有 CPU 反复写同一个 cache line。ticket lock 用取号保证公平：

```
lock(ticket_lock *l):
    my = fetch_add(&l->next, 1)
    while load_acquire(&l->owner) != my:
        cpu_relax()

unlock(ticket_lock *l):
    store_release(&l->owner, l->owner + 1)
```

ticket lock 的优点是 FIFO 公平，缺点是所有等待者仍然轮询 owner cache line。队列锁进一步让每个等待者轮询自己的节点，降低缓存抖动。教学内核可以先实现 test-and-set，再用压力测试观察竞争成本，最后引入 ticket 或 MCS 解释公平性和 cache line 迁移。

mutex 和等待队列

mutex 的关键不是把自旋改成睡眠，而是在失败时把当前线程从 runnable 集合移到等待队列：

```
mutex_lock(m):
    disable_preempt()
    spin_lock(m.wait_lock)
    if m.owner == null:
        m.owner = current
        spin_unlock(m.wait_lock)
        enable_preempt()
        return
    enqueue(m.waiters, current)
    current.state = BLOCKED
    schedule_unlocking(m.wait_lock)
    enable_preempt()
    retry mutex_lock(m)

mutex_unlock(m):
    spin_lock(m.wait_lock)
    if empty(m.waiters):
        m.owner = null
    else:
        next = dequeue(m.waiters)
        m.owner = next
        next.state = RUNNABLE
        enqueue_runqueue(next)
    spin_unlock(m.wait_lock)
```

这里三个不变量：owner 要么为空要么指向持有线程；等待队列里的线程必须是 BLOCKED；被唤醒线程必须重新进入 runnable 集合。若 unlock 先释放 owner 再唤醒，另一个新线程可能插队，造成等待者饥饿。若唤醒时没有持有等待队列锁，等待节点可能和 timeout/cancel 路径并发释放。

死锁条件与锁顺序

经典死锁需要四个条件同时成立：互斥、持有并等待、不可抢占、循环等待。内核无法完全消除前三个条件，因为对象确实需要互斥，很多资源不能强制抢占。因此工程上主要打破循环等待：给锁分层，规定全局顺序。

```
Allowed lock order:
    tasklist_lock
    -> mm.lock
    -> vma.lock
    -> page.lock
```

```

superblock.lock
-> inode.lock
   -> pagecache.lock
   -> buffer.lock
netns.lock
-> socket.lock
   -> skb_queue.lock

```

锁顺序不是文档装饰，而要能被测试和运行时检查。简化 lockdep 可以在每个线程记录当前持有锁栈，每次获取新锁时加入一条“已持有锁 -> 新锁”的有向边；若图出现环，就报告潜在死锁。

```

on_lock_acquire(new_lock):
    for held in current.held_locks:
        graph_add_edge(held.class, new_lock.class)
    if graph_has_cycle():
        report_deadlock_risk(current, held, new_lock)
    push(current.held_locks, new_lock)

```

这个检查的成本取决于锁类数量和边数量，不能在极热路径无节制开启，但作为调试构建非常有价值。更重要的是，它把“某次测试没有卡死”升级成“锁顺序图没有出现已知环”。

RCU 的读写分离

RCU 解决的是读路径极热、写路径较少的场景。读者进入 RCU 临界区时不获取传统互斥锁，只保证自己不会在临界区内被回收对象悬空；写者发布新对象后，必须等所有旧读者经过 quiescent state 才释放旧对象。

```

reader():
    rcu_read_lock()
    p = rcu_dereference(global_ptr)
    use(p)
    rcu_read_unlock()

writer():
    old = global_ptr
    new = clone_and_update(old)
    rcu_assign_pointer(global_ptr, new)
    call_rcu(old, free_after_grace_period)

```

RCU 的不变量是：任何读者看到的对象在读侧临界区结束前仍然有效。它不保证读者看到最新状态，也不适合读区内睡眠的普通实现。把 RCU 当成“无锁万能替代品”会导致非常隐蔽的 use-after-free。

测试

锁测试不能只跑正常路径。最低测试包括：

1. 互斥测试：多个 CPU 同时递增共享计数器，最终值必须等于操作次数。
2. lost wakeup 测试：让唤醒和入睡交错，等待者不能永久睡眠。
3. 中断上下文测试：持有自旋锁时触发同类中断，检查是否需要 irqsave。
4. 死锁检测测试：故意构造 A 后 B 与 B 后 A，lockdep 必须报告环。
5. 饥饿测试：大量短任务竞争 mutex，等待时间不能无限增长。
6. 退出路径测试：加锁后在每个错误分支注入失败，所有锁计数必须归零。

本章合格标准：能说明每把锁保护哪个对象；能判断自旋锁、mutex、RCU 的上下文限制；能写出 lost wakeup 的反例和修复；能用锁顺序图解释死锁风险。

源码级补充：qspinlock、rwsem、seqlock 与 lockdep

raw spinlock、ticket lock 与 queued spinlock

简单 test-and-set 锁在竞争下会让所有 CPU 反复写同一 cache line。ticket lock 提供 FIFO 公平，但所有等待者仍然轮询同一个 owner。queued spinlock 把等待者组织成队列，思想接近 MCS：每个 CPU 主要等待自己的节点，减少 cache line 抖动。

```
mcs_lock(lock, node):
    node.locked = true
    pred = xchg(lock.tail, node)
    if pred != null:
        pred.next = node
        while node.locked:
            cpu_relax()

mcs_unlock(lock, node):
    if node.next == null:
        if cmpxchg(lock.tail, node, null):
            return
    wait until node.next != null
    node.next.locked = false
```

Linux 的 qspinlock 比这段伪代码更紧凑，读 [kernel/locking/qspinlock.c](#) 时重点看三件事：pending bit 如何处理短竞争，MCS queue 如何承载长竞争，锁路径如何配合 preempt 和 paravirt。不要把 queued lock 理解成“更复杂所以一定更快”，它是为高竞争多核场景付出的复杂度。

rwlock、rwsem 和乐观自旋

读写锁允许多个读者并行，一个写者独占。rwlock 通常用于短临界区，不能睡眠；rwsem 是可睡眠读写信号量，适合 VMA、文件系统等较长路径。rwsem 还可能做 optimistic spinning：如果持有者正在 CPU 上运行，等待者短暂自旋，期望持有者很快释放，避免睡眠/唤醒成本。

原语	优点	风险
rwlock	读者并行，路径短	长读者或写频繁时收益差，不能睡眠
rwsem optimistic spin	可睡眠，适合较长临界区 避免短等待睡眠成本	饥饿、公平性、唤醒策略复杂 持有者不运行时浪费 CPU

读源码可看 [kernel/locking/rwsem.c](#)。关注 wait list、owner、reader count、writer handoff。若一个路径持有 rwsem 后等待另一个也需要此 rwsem 的工作完成，就容易形成隐藏死锁。

seqlock：写者优势的读一致性

seqlock 用一个递增序列号表达写入是否发生。读者开始时读取 seq，复制数据，结束时再读 seq；如果 seq 变化或为奇数，重试。写者持锁并把 seq 变奇数，写完后变偶数。

```
read:
```

```

do {
    seq = read_seqbegin(&s)
    local = shared_data
} while (read_seqretry(&s, seq))

write:
    write_seqlock(&s)
    shared_data = new_value
    write_sequnlock(&s)

```

seqlock 适合读多写少、读者可重试、数据可复制的场景，例如时间数据。它不适合读者不能重试、数据包含指针且对象生命周期复杂、写者频繁导致读者长期重试的场景。

per-CPU 与 preempt/migrate 控制

per-CPU 变量让每个 CPU 有自己的副本，减少锁和 cache line bouncing。但访问 per-CPU 数据时，要确保当前执行不会迁移到另一个 CPU，否则先取到 CPU0 的指针，随后迁移到 CPU1 继续使用，会破坏语义。

```

this_cpu_counter_add(delta):
    preempt_disable()
    per_cpu(counter, smp_processor_id()) += delta
    preempt_enable()

```

有些路径用 `migrate_disable` 而不是完全禁止抢占，表示任务可以被抢占但不能迁移 CPU。读调度和网络栈源码时，这类边界经常出现。per-CPU 不是无锁魔法，它把共享变成分片，同时引入汇总、CPU hotplug 和迁移限制。

内存屏障原语

原语	语义
<code>smp_rmb</code>	约束读-读顺序
<code>smp_wmb</code>	约束写-写顺序
<code>smp_mb</code>	完整内存屏障，约束读写重排
<code>smp_store_release</code>	发布数据前的写入对获取方可见
<code>smp_load_acquire</code>	看到发布指针后能看到发布前初始化

屏障必须服务于具体协议。若无法说出“谁发布、谁获取、保护哪组字段”，就不应随便加 `smp_mb`。错误屏障可能既没修 bug，又增加维护负担。

lockdep 报告怎么读

lockdep 报告通常会给出两个锁类、已观察到的获取顺序、当前试图形成的新顺序和调用栈。读报告时先找锁类名，再画边：A 后 B，B 后 A 是否形成环；然后检查是否真实同一实例、同一上下文，还是锁类标注过粗。

```

Observed order:
    inode_lock -> page_lock
New order:
    page_lock -> inode_lock
Result:
    possible circular locking dependency

```

不要把 lockdep 当成“测试失败噪声”。它发现的是潜在等待环，可能在当前负载下未死锁，但在错误时序下会卡死。高质量内核代码要能解释每条锁边的必要性，或调整锁类/顺序消除环。

第二阶段深讲：同步原语要写成状态机

同步章节若只讲 mutex、condition variable、semaphore 的 API，很容易产生错觉：只要调用了锁，就安全。真实系统的同步安全来自对象状态机。每个共享对象都应该明确：有哪些字段，字段之间的不变量是什么，哪些事件改变状态，等待者等的谓词是什么，关闭和取消怎样唤醒所有路径。

有界队列：最小但完整的同步对象

一个有界队列包含 buffer、capacity、head、tail、size、closed、读等待队列和写等待队列。它的核心不变量是：

```
0 <= size <= capacity
head and tail are always in [0, capacity)
size == number of occupied slots
closed implies no new successful push begins
waiters sleep only while their predicate is false
```

生产者等待 `size < capacity || closed`，消费者等待 `size > 0 || closed`。注意关闭状态必须写进谓词，否则关闭后空队列上的消费者可能永远睡眠，满队列上的生产者也可能永远睡眠。

正确 wait 的四步

正确等待不是一个 `wait()` 调用，而是四个动作的组合：持锁检查谓词，把自己加入等待集合，释放锁并睡眠，醒来后重新持锁检查谓词。

```
wait_until(q, predicate):
    lock(q.lock)
    while !predicate():
        enqueue(q.waiters, current)
        current.state = SLEEPING
        schedule_unlocking(q.lock)
        lock(q.lock)
    unlock(q.lock)
```

这段伪代码中的 `schedule_unlocking` 表达一个关键原子性：任务状态变成 sleeping、等待节点进入队列、锁释放、调度走，必须对唤醒方形成一致顺序。若先释放锁再入队，唤醒可能发生在空窗口里，造成 lost wakeup。

notify 不是消息

条件变量和等待队列的 `notify` 或 `wake` 只是告诉等待者“相关状态可能变了”。它不携带数据，也不保证谓词已经为真，更不保证只有一个等待者会醒。醒来后必须重新检查谓词。

```
consumer:
    lock(m)
    while queue.empty() and not queue.closed:
        wait(cv, m)
    if queue.empty() and queue.closed:
        return EOF
    item = queue.pop()
    unlock(m)

producer:
    lock(m)
    queue.push(item)
    notify_one(cv)
    unlock(m)
```

若消费者用 `if` 而不是 `while`，虚假唤醒或多个消费者竞争同一个 item 都可能破坏逻辑。若 producer 在修改状态前 notify，消费者醒来仍可能看到旧状态。若关闭时忘记 `notify all`，等待者可能永久挂起。

信号量和条件变量的边界

信号量表达“许可数量”，条件变量表达“对象状态变化”。它们能互相模拟一部分场景，但语义侧重点不同。

问题	适合的原语	原因
连接池最多 N 个并发连接	semaphore	许可数量就是核心状态
队列不空/不满	condition variable	谓词依赖多个字段
等待某个请求完成一次	completion/future	一次性状态转移
等待多个 fd readiness	epoll/wait queue	等待集合是 fd 对象
读多写少配置表	RCU/seqlock	读路径要极轻

把 semaphore 用来表达复杂队列状态，会让关闭、取消和错误路径变得隐晦；把 condition variable 用来表达简单许可，也会多出不必要的 mutex 和谓词。选择原语前先写状态机。

futex 的状态机

用户态 mutex 常用一个整数表示状态：0 表示 unlocked，1 表示 locked 无已知等待者，2 表示 locked 且可能有等待者。futex 只在竞争路径进入内核。

```
lock state:
  0 unlocked
  1 locked, no known waiter
  2 locked, contended

lock:
  CAS 0 -> 1 succeeds: acquired
  otherwise set/observe 2 and futex_wait(state, 2)

unlock:
  exchange state -> 0
  if old == 2: futex_wake(state, 1)
```

内核 futex wait 的关键动作是检查用户地址仍等于 expected。若用户态状态已经变了，内核不能睡眠，而要返回让用户态重试。这个比较把用户态原子状态和内核等待队列连接起来。

读多写少：seqlock 和 RCU

读多写少对象不一定适合普通 mutex。seqlock 让读者无锁复制数据，若写者更新期间序列号变化，读者重试。它适合时间戳、统计快照这类可重试读取，不适合包含指针且对象可能被释放的复杂结构。

```
read seqlock:
  do:
    seq = read_begin()
    local = copy(shared)
    while read_retry(seq)

write seqlock:
  write_lock()
  update shared
  write_unlock()
```

RCU 则适合指针发布：读者拿到旧对象也没关系，只要旧对象在读侧临界区结束前不释放。写者创建新对象、原子发布指针、等待 grace period 后释放旧对象。

RCU 的难点不是读，而是更新和回收。

关闭、取消和超时

完整同步对象必须处理关闭、取消和超时。否则测试只覆盖正常路径，系统一退出就卡死。

```
pop_with_timeout(q, deadline):
    lock(q.lock)
    while q.empty and not q.closed:
        remaining = deadline - now
        if remaining <= 0:
            unlock(q.lock)
            return TIMEOUT
        timed_wait(q.not_empty, q.lock, remaining)
    if q.empty and q.closed:
        unlock(q.lock)
        return CLOSED
    item = pop(q)
    notify_one(q.not_full)
    unlock(q.lock)
    return item
```

timeout 路径要把等待节点从队列移除，不能留下悬挂指针。取消路径也一样。关闭路径要唤醒所有等待者，让它们重新检查 `closed`。这些路径是同步代码最容易漏测的部分。

状态机测试矩阵

有界队列的测试不应只验证 `push/pop`。应按状态和事件组合设计：

状态	事件	期望
empty open	blocking pop	消费者睡眠
empty open	producer push	消费者被唤醒并取得 item
full open	blocking push	生产者睡眠
full open	consumer pop	生产者被唤醒并写入
empty closed	pop	返回 CLOSED/EOF
full closed	push	返回 CLOSED/EPIPE
waiting	timeout	等待节点移除，返回 TIMEOUT
waiting	cancel	等待节点移除，不破坏队列

这类测试能暴露三种经典 bug：lost wakeup、关闭不唤醒、timeout 后等待节点悬挂。压力测试只能偶然撞到这些问题，状态机测试能稳定覆盖。

同步状态机验收

- 每个共享对象都有字段表和不变量。
- 每个等待都有明确谓词，且谓词包含关闭或取消条件。
- 每个状态改变都说明唤醒哪个等待队列。
- 每个 timeout/cancel 路径都移除等待节点。
- 每个关闭路径都唤醒所有可能等待者。
- 每个测试都能说明覆盖了哪个状态和事件。

2.4 并发证据、实验与验收

第二阶段证据与验收：把卡顿、死锁和饥饿拆开

第二阶段的目标不是背更多调度算法和同步原语，而是能解释一个系统为什么没有推进。用户说“卡住了”，内核可能在等 CPU、等锁、等 I/O、等内存回收、等定时器、等信号，也可能已经退出但没人回收。本节把证据、问题类型和验收方法放在一起。

先分类：没有推进的六种状态

现象	内核解释	主要证据
CPU 忙但请求慢	runnable 竞争、锁自旋、软中断占用	perf top、perf sched、runqueue
CPU 不忙但请求慢	线程 sleeping 等事件	off-CPU profile、等待队列、I/O 统计
周期性尖峰	cgroup throttle、定时批处理、GC/回收	cgroup cpu.stat、tracepoint
永久等待	lost wakeup、死锁、关闭未唤醒	等待图、锁依赖、线程栈
低优先级饿死	高优先级或实时任务长期占用	调度类、优先级、runtime 统计
zombie 增多	父进程未 wait 或 reaper 异常	ps、进程树、wait 路径

分类的价值是避免乱调参数。若线程在 I/O wait，修改时间片无效；若被 cgroup 限流，优化业务代码可能只降低配额内 CPU 时间，仍有周期性等待；若是 lost wakeup，增加 CPU 只会让 bug 更随机。

线程栈是等待原因的第一证据

卡住时，先看线程栈。用户态栈能显示线程停在哪个库调用；内核栈能显示它睡在 futex、poll、read、writeback、page fault、mutex、waitpid 还是其他路径。

```
questions for a blocked thread:
Is it in futex_wait?
Is it in epoll_wait or poll?
Is it in read/write/fsync?
Is it in page fault or reclaim?
Is it waiting for child exit?
Is it runnable but not scheduled?
```

只有 runnable 但没被调度，才优先看调度器。若栈显示在 futex_wait，要找持锁者；若在 epoll_wait，要找 fd readiness 和关闭路径；若在 waitpid，要找子进程状态。

等待图：死锁和优先级反转的共同语言

等待图把“谁等谁”显式化。线程等待锁，锁由另一个线程持有；线程等待 I/O，请求由设备 completion 完成；线程等待子进程，子进程由调度器和 exit 路径推进。死锁是等待图成环，优先级反转是等待图上的 owner 得不到 CPU。

```
wait graph example:
T1 waits lock A, owned by T2
T2 waits lock B, owned by T3
T3 waits lock C, owned by T1
=> cycle, deadlock
```

```
priority inversion:
```



```
High waits lock A, owned by Low
Medium runnable and preempts Low
=> no cycle, but scheduler must boost Low
```

这个区分很重要。死锁需要打破环；优先级反转需要让持锁者运行；lost wakeup 则可能没有 owner，因为等待者睡在一个再也不会被通知的队列上。

调度证据的最小集合

一次调度问题至少收集：

```
minimum scheduling evidence:
number of runnable tasks
context switch rate
per-task runtime and wait time
migrations
cgroup throttled time
interrupt and softirq distribution
```

指标	解释	风险
nr_running	可运行任务数量	高表示 CPU 竞争
voluntary cs	主动阻塞切换	I/O/锁/等待事件多
involuntary cs	被抢占切换	时间片、优先级、竞争
migrations	CPU 迁移次数	可能损失 cache/NUMA 局部性
throttled time	cgroup 配额等待	容器周期性延迟
softirq time	网络/块等延后处理	可能抢占业务线程

这些指标要放到同一条时间线上看。平均值很容易掩盖尾延迟：一个 100ms 周期里前 20ms 很快，后 80ms 被 throttle，平均 CPU 使用并不能解释 p99。

同步证据的最小集合

同步问题至少收集对象状态、等待谓词、持有者、等待者和关闭状态。

```
minimum synchronization evidence:
object id and state fields
lock owner
waiter list
wait predicate
last state transition
close/cancel/timeout flag
```

如果一个队列卡住，要能回答：队列是空还是满，closed 是否为真，谁应该唤醒 not_empty，谁应该唤醒 not_full，等待者的 timeout 是否已经过期。没有这些字段，日志再多也只是噪声。

第二阶段负面测试

注入点	目的	期望
fork 第 N 步失败	验证资源回滚	无泄漏、无半成品可见

exec commit 前失败	验证旧程序保留	返回错误，旧地址空间仍可用
wait 与 exit 并发	验证 zombie 回收	只回收一次
唤醒发生在入队前	验证无 lost wakeup	等待者不永久睡眠
锁顺序反转	验证 deadlock 检测	报告等待环
高优先级等低优先级锁	验证优先级继承	持锁者被提升并释放锁
cgroup budget 耗尽	验证 throttle/unthrottle	周期刷新后恢复运行
关闭有等待者的队列	验证广播唤醒	所有等待者退出

这些测试比“创建几个线程跑一跑”更接近内核质量要求。它们检查的是状态机边界，而不是碰巧跑通的正常路径。

第二阶段完成标准

- 能从 fork 的每个阶段说出可见性、失败点和回滚对象。
- 能解释 exec 的提交点，以及 commit 前后失败语义为什么不同。
- 能区分 runnable wait、on-CPU、off-CPU sleep 和 zombie。
- 能说明每 CPU runqueue、唤醒抢占、负载均衡和 cgroup 配额的关系。
- 能为一个同步对象写出字段、不变量、等待谓词和关闭路径。
- 能画出死锁等待图、优先级反转等待图和 lost wakeup 时序。
- 能用证据判断一个卡顿属于调度问题、锁问题、I/O 问题还是退出回收问题。

到这个标准，进程、调度和同步才不再是 API 名称，而是可证明推进的状态机。

第二阶段源码与实验：把任务、调度、同步落到可验证对象

前面的章节已经把进程生命周期、调度策略和同步状态机讲成了模型。模型还不够。读者必须能把模型落到三类材料上：真实内核源码里的对象和函数、可运行的小实验、出错时能区分原因的观测命令。本节不追求把 Linux 源码逐行翻译，而是给出一条能读通的路线：先找对象，再找状态转移，再找失败路径，最后用实验验证自己的理解。

源码阅读地图：先读结构，再读路径

操作系统源码很大，直接从入口函数一路跳会很快迷路。更可靠的读法是先确定“哪些结构保存状态”，再读“哪些函数改变状态”。以 Linux 为参照，第二阶段最相关的路径如下。

主题	典型源码位置	读的时候要抓住什么
任务结构	include/linux/sched.h	task_struct 的状态、调度实体、父子关系、引用对象
fork 路径	kernel/fork.c	kernel_clone、copy_process、copy_mm、copy_files、copy_thread
exec 路径	fs/exec.c	二进制格式检查、参数复制、新 mm 提交点、凭据转换

exit/wait	kernel/exit.c	do_exit、zombie、reparent、wait 扫描和最终释放
核心调度	kernel/sched/core.c	enqueue/dequeue、wakeup、schedule、context switch 调用边界
公平调度	kernel/sched/fair.c	vruntime、min_vruntime、唤醒放置、负载均衡
实时调度	kernel/sched/rt.c、kernel/sched/deadline.c	固定优先级、预算、deadline 和准入控制
futex	kernel/futex/core.c	用户态原子值和内核等待队列如何连接
mutex/rtmutex	kernel/locking/mutex.c、kernel/locking/rtmutex.c	owner、waiter、优先级继承、慢路径
pipe 等待	fs/pipe.c	条件谓词、等待队列、关闭唤醒、非阻塞语义
RCU	kernel/rcu/tree.c	读侧临界区、宽限期、延迟释放

读源码时不要先追所有宏。先写出每个对象的字段表：哪些字段是身份，哪些字段是状态，哪些字段是链接关系，哪些字段是引用计数，哪些字段只在持锁时可改。然后再读改变这些字段的路径。这样能避免把源码读成函数调用列表。

fork 的源码读法：每一步都问可见性

fork 或 clone 的阅读重点不是“它创建了进程”，而是每一步创建出的对象什么时候对其他 CPU 可见。一个半初始化任务若提前进入 PID 表、父子列表或 runqueue，后续调度、信号、wait 都可能观察到破碎状态。

```
source reading checklist for fork:
1. task object and kernel stack allocated?
2. scheduler fields initialized but not runnable?
3. credentials, files, fs, signal, mm copied or shared?
4. child register frame prepared?
5. pid allocated and published?
6. parent-child relation linked?
7. task finally made runnable?
8. every failure label releases exactly the acquired references?
```

这条路线能解释源码里的错误标签。若 copy_files 成功、copy_mm 失败，错误路径必须释放文件表引用；若 PID 已经发布，回滚就要同时撤销 PID 映射；若任务已经 runnable，很多失败不能再简单 free，而要走退出路径。源码中的阶段边界，就是原理章节里的生命周期边界。

上下文切换：保存的不是“整个进程”

上下文切换常被抽象成“保存 A，恢复 B”。这句话容易误导。内核并不会每次复制整个进程资源；它保存的是继续执行所需的 CPU 上下文，并切换当前 CPU 指向的任务、栈、地址空间和调度状态。地址空间、文件表、凭据这些对象通过引用挂在 task 上，不会在每次切换时复制。

项目	切换时发生什么	为什么重要
内核栈	从 prev 的内核栈切到 next 的内核栈	每个线程有自己的内核执行上下文

通用寄存器	保存调用约定要求保存的部分，并恢复 next 的部分	保证内核函数返回后能继续执行
RIP/返回地址	由切换汇编和栈帧间接恢复	next 像是从上次 <code>schedule</code> 后继续
页表根	进程地址空间不同才需要切换或标记切换	影响 TLB、PCID 和内存隔离
FPU/SIMD 状态	通常惰性或按需保存恢复	AVX 状态很大，不能无脑复制
TLS/段寄存器	用户线程切换时要保持线程局部状态正确	影响 <code>thread_local</code> 和 <code>libc</code> 线程数据
调度统计	记录运行时间、等待时间和切换事件	后续诊断依赖这些统计

在 x86-64 Linux 中，可以沿着 `schedule()`、`context_switch()`、`switch_to` 和架构相关切换汇编去读。读的时候重点看三件事：prev 的栈指针保存在哪里，next 的栈指针从哪里恢复，地址空间切换在哪个条件下发生。明白这三点，才算真正理解“线程切换比进程切换轻”的具体含义。

一个可检查的调度器数据结构

为了把调度算法写得可测，先不要急着实现所有策略。可以先写一个最小模型：任务状态、runqueue、等待原因和调度统计必须显式存在。

```
#include <cstdint>
#include <deque>
#include <limits>
#include <stdexcept>
#include <vector>

using TaskId = std::size_t;
using ProcessId = std::uint32_t;

constexpr TaskId SCHED_IDLE_TASK = std::numeric_limits<TaskId>::max();
constexpr ProcessId SCHED_INVALID_PID = 0;

enum class TaskState {
    Runnable,
    Running,
    Sleeping,
    Zombie,
};

enum class WaitReason {
    None,
    Mutex,
    Io,
    Timer,
    ChildExit,
    PageFault,
};

struct Task {
    TaskId tid = SCHED_IDLE_TASK;
    ProcessId pid = SCHED_INVALID_PID;
    int priority = 0;
    TaskState state = TaskState::Runnable;
    WaitReason wait_reason = WaitReason::None;
    std::uint64_t vruntime = 0;
    std::uint64_t runnable_since = 0;
```

```

    std::uint64_t total_runtime = 0;
    std::uint64_t total_wait = 0;
};

struct RunQueue {
    std::deque<TaskId> rr;
    std::uint64_t clock = 0;
};

```

这段结构虽然小，却已经能写出硬断言：只有 `Runnable` 能入队；调度选中的任务必须从 `Runnable` 变成 `Running`；进入 `Sleeping` 时必须记录 `wait_reason`；进入 `Zombie` 后不能再回到 `runqueue`；每次 `tick` 要更新当前任务的 `runtime`，而不是误把 `sleeping` 时间算成运行时间。

```

void enqueue(RunQueue& rq, std::vector<Task>& tasks, TaskId tid) {
    auto& task = tasks.at(tid);
    if (task.state != TaskState::Runnable) {
        throw std::logic_error("only runnable task may enter runqueue");
    }
    task.runnable_since = rq.clock;
    rq.rr.push_back(tid);
}

TaskId pick_next(RunQueue& rq, std::vector<Task>& tasks) {
    while (!rq.rr.empty()) {
        const TaskId tid = rq.rr.front();
        rq.rr.pop_front();
        auto& task = tasks.at(tid);
        if (task.state == TaskState::Runnable) {
            task.total_wait += rq.clock - task.runnable_since;
            task.state = TaskState::Running;
            return tid;
        }
    }
    return SCHED_IDLE_TASK;
}

```

很多教学代码的调度器 bug 来自“队列里只有 `tid`，没有状态断言”。任务睡眠后忘记出队、退出后仍被选中、唤醒时重复入队，都会在压力测试下变成随机错误。把状态检查写进数据结构入口，能让错误尽早暴露。

futex 实验：用户态快路径为什么还需要内核

`futex` 的实验目标不是背 `futex(2)` 参数，而是观察“无竞争不进内核，有竞争才睡眠”。可以写一个用户态 `mutex`：先用 `CAS` 从 0 改到 1；失败后把状态改成 2，再调用 `futex wait`；解锁时把状态置 0，若旧状态是 2，则 `futex wake`。

```

observe futex:
    strace -f -e futex ./futex_mutex_demo uncontended
    strace -f -e futex ./futex_mutex_demo contended
    perf stat -e context-switches,cpu-migrations ./futex_mutex_demo contended

```

无竞争版本中，绝大多数 `lock/unlock` 不应产生 `futex` 系统调用；竞争版本中，应该能看到 `wait/wake`。若每次加锁都进内核，说明 `fast path` 失败；若竞争版本 CPU 占满但 `futex` 调用很少，可能在忙等；若 `wait` 后没人 `wake`，要检查 `unlock` 是否根据旧状态唤醒。

lost wakeup 的可复现实验

`lost wakeup` 最难的地方是它看起来像偶发卡住。要让它稳定复现，可以故意写一个错误版本：等待者先检查谓词，释放锁，然后再把自己加入等待队列。唤醒者

在这个窗口里改变条件并 notify，等待者随后入队睡眠，就会错过唯一一次唤醒。

```
wrong wait sequence:
lock(m)
if (!ready) {
    unlock(m)           // window begins
    enqueue(waiters, self)
    sleep()
}

waker:
lock(m)
ready = true
notify_one()
unlock(m)
```

正确实验要用 barrier 固定时序：让等待线程停在“释放锁之后、入队之前”，再让唤醒线程完成 notify，最后放行等待线程。期望结果是错误实现永久睡眠，正确实现不会睡眠或会立即重新检查谓词。这个测试比随机跑一万次更有价值，因为它直接覆盖了危险窗口。

condition variable 的三个负面案例

条件变量的常见错误可以拆成三个小实验，每个实验都应该有明确期望。

错误	构造方法	期望现象
if 代替 while	两个消费者等待一个 item，notify all 后同时醒来	一个消费者拿到 item，另一个必须重新睡眠；错误实现会读空队列
修改状态前 notify	producer 先 notify，再 push item	consumer 醒来看到旧状态，可能继续睡或错误返回
关闭不广播	多个 producer/consumer 睡在不同队列，close 只唤醒一边	另一边永久等待

这三个案例对应三条规则：等待必须循环检查谓词；状态改变和 notify 的顺序必须让等待者能看到新状态；关闭是一种状态变化，必须唤醒所有可能等待这个状态的线程。

same-pool future deadlock

线程池和 future 是现代 C++ 程序里很常见的同步组合。一个典型死锁是：线程池只有 N 个 worker，N 个任务都在等待同一个线程池中尚未运行的子任务。所有 worker 都被等待占住，子任务无人执行。

```
bad pattern:
worker task A:
    future = pool.submit(B)
    return future.get()    // blocks a worker

if all workers do this:
    no worker remains to run B
```

修复方向有三种。第一，把等待改成 continuation：B 完成后触发 A 的后续逻辑，不占住 worker。第二，允许 worker 在等待时帮助执行队列中的其他任务，但要小心重入和优先级。第三，把阻塞等待放到专门的 blocking pool，计算 pool 不做无限期等待。这个例子说明同步问题不只在内核；用户态运行时也必须有调度意识。

调度观测命令：每个命令回答一个问题

观测命令不是越多越好。关键是每个命令要回答一个具体问题。长命令不要塞进表格；实验报告里应把命令、问题和结论分开写。

```
cat /proc/<pid>/sched
```

它回答单个任务运行了多久、等待了多久、迁移了几次。若 `runnable wait` 高，说明任务已经可运行但长期拿不到 CPU。

```
perf stat -e context-switches,cpu-migrations,page-faults ./workload
```

它回答切换、迁移和缺页是否异常。上下文切换暴涨常见于线程过多、锁竞争或频繁阻塞；迁移过多可能破坏 `cache` 和 `NUMA` 局部性；缺页多时要进一步区分 `minor fault`、`major fault` 和回收。

```
perf sched record ./workload
perf sched latency
```

这组命令回答“任务被唤醒后等了多久才真正运行”。若唤醒延迟集中在某些线程或某些 CPU 上，要继续检查 `runqueue`、`CPU affinity`、优先级和 `cgroup quota`。

```
pidstat -w -p <pid> 1
```

它区分自愿上下文切换和非自愿上下文切换。自愿切换多，通常说明线程主动阻塞在 I/O、锁或等待事件；非自愿切换多，通常说明被抢占或 CPU 竞争强。

```
cat /sys/fs/cgroup/<group>/cpu.stat
```

它回答服务组是否被 CPU 配额限流。若 `throttled time` 和 `throttled periods` 增长，`p99` 延迟很可能来自配额周期，而不是业务函数突然变慢。

```
strace -f -e futex,clone,execve,wait4 ./workload
```

它回答线程、进程和 `futex` 系统调用行为。若大量线程停在 `futex wait`，就要寻找锁 `owner`、等待谓词和唤醒路径；若 `wait4` 长期不返回，要检查子进程是否退出以及父子关系是否正确。

```
perf top
perf record ./workload
```

它们回答 CPU 真正在执行哪里。若热点在自旋、内存拷贝、哈希查找或内核软中断，优化方向完全不同。

做报告时不要只贴命令输出。要写清楚：实验负载是什么，预期瓶颈是什么，命令观察到的现象是否支持预期，若不支持下一步查什么。例如 `p99` 延迟高，同时 `cpu.stat` 显示大量 `throttled time`，说明要先看 `CPU quota`；若 `perf sched latency` 显示唤醒后等待长，但 `cgroup` 没限流，要看 `runqueue`、优先级和 `CPU affinity`。

从源码到实验的闭环

第二阶段的最低闭环可以按下面顺序完成。

1. 读 `copy_process`，画出 `fork` 的阶段图和失败回滚表。
2. 写一个 `fork/exec/wait` 小程序，用 `strace -f` 验证父子返回值、`exec` 成功路径和 `wait` 回收。
3. 读 `schedule` 到架构切换路径，说明一次线程切换保存哪些状态。
4. 写一个 Round Robin 或公平调度模型，断言 `sleeping/zombie` 任务不能入队。
5. 写一个错误 `condition variable` 示例，用 `barrier` 稳定制造 `lost wakeup`。

6. 写 一 个 `futex mutex` 或 阅 读 `pthread mutex` 的 `futex` 行 为， 用 `strace -e futex` 区分快路径和慢路径。
7. 用 `perf sched` 或 `/proc/<pid>/sched` 分析一次人为制造的 `runnable wait`。

完成这些实验后，读者不再只是知道概念名称，而是能把一个现象定位到对象、状态、源码路径和观测证据。到这一步，任务、调度与同步这一章才真正具备工程教材的质量。

第 3 章 特权边界、程序装载与内存管理

3.1 系统调用、权限与边界复制

系统调用、权限与内核边界

原理

系统调用是用户态请求内核代为操作受保护资源的入口。用户程序不能直接修改页表、调度队列、磁盘控制器或其他进程的状态；它只能把请求和参数交给内核。CPU 通过特权级、异常入口和约定寄存器完成控制权转移，内核再检查参数、权限和对象状态。

系统调用不是普通函数调用。普通函数调用仍在同一特权级、同一地址空间和同一信任边界里执行；系统调用会从用户态进入内核态，切换到受控入口，使用内核栈或内核上下文，检查用户指针，并在返回前恢复用户态。这个边界使得内核可以拒绝非法请求，也使系统调用有固定开销。

特权到底保护了什么

特权级保护的不是一个抽象名分，而是一组具体硬件状态。用户态不能改页表根，不能关中断，不能改中断向量，不能写任意控制寄存器，不能访问只映射给内核的 MMIO，不能把自己的代码标成内核入口。CPU 在执行敏感指令、地址翻译和异常返回时都会检查当前模式。

受保护对象	为什么危险	合法路径
页表根/地址空间控制	可映射任意物理页，读取内核或其他进程	内核内存管理代码
中断使能和向量表	可阻止调度或劫持异常入口	内核入口和中断子系统
设备 MMIO 寄存器	可重置设备、窃取数据、破坏存储	驱动通过 fd/ioctl/sysfs 等受控接口
特权返回状态	可伪造内核态返回或跳转地址	低级 entry/return 代码检查
I/O 权限和端口	可直接访问平台设备	内核 I/O 权限管理或驱动

x86-64 用 Ring 0 和 Ring 3 表达常见用户/内核边界。Ring 3 运行普通用户程序，Ring 0 运行内核；CPU 在执行敏感指令、访问 supervisor-only 页、装载控制寄存器、返回低特权级时检查 CPL 和相关权限位。

层级	角色	系统调用入口
Ring 3	用户程序	执行 <code>syscall</code> 请求进入内核
Ring 0	内核	由 IA32_LSTAR MSR 指向的入口接管
VMX root	hypervisor	可选择拦截 guest 的敏感事件

特权边界的验收问题是：用户程序是否能绕过内核直接改变受保护状态？若答案是能，系统调用实现得再漂亮也没有隔离。

为什么进入内核要换栈

用户栈由用户程序控制。它可能没有映射，可能只读，可能指向恶意构造的数据，也可能在系统调用前被故意放到临界边界。内核不能在用户栈上保存自己的返回地

址、锁状态和局部变量，否则用户就能破坏内核控制流。因此跨特权级入口必须切到可信内核栈或受控异常栈。

```
on syscall from user:
    hardware saves minimal return state
    entry code switches from user rsp to current.kernel_stack
    entry code saves registers into pt_regs
    C syscall code runs on kernel stack
```

每个线程通常有自己的内核栈。线程在用户态运行时使用用户栈；进入内核后使用内核栈；若阻塞在系统调用中，内核栈保存它在内核里的等待位置。调度器切换线程时，既要能恢复用户态 trap frame，也要能恢复睡眠中的内核调用链。

函数调用、库调用和系统调用的三层区别

很多初学者把 printf、write、系统调用混在一起。实际上它们处在三层边界上。普通函数调用只改变当前进程内部的 RIP 和栈；库调用仍在用户态，只是封装了更复杂逻辑；系统调用才跨越 CPU 特权边界进入内核。

调用类型	发生在哪里	边界含义
普通函数调用	同一进程、同一特权级	调用者和被调者互相信任同一地址空间
libc/API 调用	用户态库代码	可做缓冲、格式化、兼容处理，但不能直接改内核对象
系统调用	用户态陷入内核态	CPU 切换特权，内核检查权限和对象状态

printf("x") 通常先把格式化结果写到用户态 stdio buffer；buffer 满、遇到换行或显式 flush 时，libc 可能调用 write；write wrapper 把系统调用号和参数放进寄存器，执行 syscall。内核并不知道 printf 的格式化细节，它只看见 fd、用户 buffer 和长度。

```
printf("hello\n")
-> libc formats into FILE buffer
-> libc write wrapper
-> CPU syscall instruction
-> kernel sys_write
-> fd table, file object, device/file path
```

这层区分能解释许多现象：程序崩溃时 stdio buffer 可能没写出；write 返回成功不等于数据落盘；系统调用开销来自特权切换、参数检查和内核路径，不是 C 函数调用本身。

系统调用入口的硬件动作逐步展开

以 x86-64 的 syscall 为例，用户态执行指令后，CPU 不会查 C 函数表。它读取预先由内核配置的 MSR，切换到内核入口地址，保存返回地址和标志的一部分，改变特权相关状态。随后汇编入口完成 swapgs、切换内核栈、构造 pt_regs。

```
user mode before syscall:
    rax = syscall number
    rdi, rsi, rdx, r10, r8, r9 = args
    rip = address after syscall
    rsp = user stack

hardware/syscall entry:
    save return rip into rcx
    save flags into r11
    load kernel entry rip from MSR
```

```
enter privileged execution path

low-level kernel entry:
  swappgs to kernel per-cpu base
  load current thread kernel stack
  save user registers into pt_regs
  call C syscall dispatcher
```

不同架构寄存器名和指令不同，但结构类似：约定参数位置，触发陷入，保存返回状态，切换到可信上下文，分发表项，返回前处理信号/抢占/审计。把这条路径记清，后面看 entry 汇编就不会把它当成神秘代码。

返回用户态比进入内核更危险

进入内核时，内核获得控制权；返回用户态时，内核要把控制权交还给不可信程序。返回前必须保证：目标 RIP 是用户地址，返回 CPL 是 Ring 3，用户栈指针不会让内核继续使用，挂起信号和抢占已处理，调试/审计状态一致，寄存器中不能泄露内核敏感数据。

```
return_to_user(regs):
  if need_resched: schedule()
  if signal_pending: deliver_signal(regs)
  if tracing_or_audit: syscall_exit_work(regs)
  sanitize_user_return_state(regs)
  restore_user_registers(regs)
  sysretq or iretq
```

许多安全漏洞发生在返回路径：错误使用快速返回指令、没有正确验证用户返回地址、泄露内核寄存器内容、返回前忘记处理单步/信号状态。系统调用不是“进入内核执行函数再回来”这么简单，入口和出口都是安全边界。

参数复制是系统调用安全性的核心。用户态传入的指针不能被内核直接相信，因为它可能无效、不可读、不可写、跨页、在检查后被另一个线程改掉，或指向内核不应访问的区域。内核必须使用 `copy_from_user/copy_to_user` 一类机制，把用户内存当作不可信输入处理。

用户指针为什么不能直接解引用

用户指针看起来只是一个地址，但它属于用户地址空间的契约，不属于内核可信内存。直接解引用会产生四类问题。第一，地址可能无效，内核访问会 `fault`。第二，地址可能指向用户不可读/不可写区域。第三，地址所在页可能跨页，前半可读后半不可读。第四，另一个线程可能在检查之后修改数据，造成 `TOCTOU`。

```
bad:
  // directly trusts user pointer
  header = *(struct Header*)user_ptr
  use(header.length)

better:
  header = copy_struct_from_user(user_ptr)
  validate(header.length)
  data = copy_bytes_from_user(user_ptr + sizeof(header), header.length)
```

`copy` 不是低效保守，而是把不可信输入转成内核拥有的稳定快照。对于大 I/O，内核可以 `pin` 用户页或使用零拷贝，但那会引入页生命周期、DMA 映射和 `completion` 责任，不能简单理解成“直接用用户指针”。

EFAULT 是如何产生的

当内核复制用户数据时，用户页可能不存在或权限不允许。若普通内核代码访问坏地址直接 `fault`，通常是内核 bug；但 `copy_from_user` 的 `fault` 是预期错误路径。内核通过异常表或等价机制告诉 `page fault handler`：如果 `fault` 发生在这段用户复制指令上，不要 `panic`，而是跳到 `fixup` 路径返回失败。

```
copy loop instruction at address A:
    load byte from user pointer
    store byte to kernel buffer

exception table:
    faulting instruction A -> fixup address F

page fault handler:
    if fault_pc found in exception table:
        regs.rip = fixup_address
        return
    else:
        handle normal kernel/user fault
```

这解释了为什么用户指针复制需要专用 API。它不仅做范围检查，还和低级异常处理配合，把硬件 fault 转换成系统调用错误码。教学 OS 也应该明确区分“内核自己访问坏地址”和“内核按用户请求复制时遇到坏用户页”。

权限检查不是一个 if

一次系统调用经常要穿过多层权限。以 `openat` 为例，路径解析要检查每级目录搜索权限；`mount namespace` 决定看到哪棵树；符号链接可能改变目标；LSM 可在路径或 `inode` 层拒绝；文件 `mode`、`ACL`、`capability` 决定读写执行；打开 `flags` 决定是否创建、截断、追加；资源限制决定还能否分配 `fd`。

```
openat(dirfd, path, flags):
    resolve dirfd in current fd table
    walk path under mount namespace
    check execute/search permission on directories
    handle symlink policy and flags
    get stable inode/dentry reference
    check DAC/ACL/capability
    call LSM hooks
    create file object with open flags
    allocate fd in current process fd table
```

这也是为什么错误码能提供信息。`ENOENT`、`ENOTDIR`、`EACCES`、`EPERM`、`ELOOP`、`EMFILE` 分别表示不同层次失败。把所有失败都写成“权限不够”会掩盖状态机。

root、capability 和 namespace 的演进

早期 Unix 常把 `uid 0` 作为全能 root。现代系统把 root 权限拆成 `capability`，例如 `CAP_NET_ADMIN`、`CAP_SYS_ADMIN`、`CAP_DAC_OVERRIDE`。`namespace` 又让“这个 `capability` 对哪个对象集合有效”成为问题。容器内 root 不应自动拥有宿主全部权限。

机制	解决的问题	新复杂性
uid/gid/mode capability	简单主体和文件权限 把 root 权限拆小	粒度粗，root 过大 哪个路径检查哪个 capability 要准确
namespace	隔离进程看到的资源视图	capability 必须相对 user namespace 解释
LSM seccomp	策略化强制访问控制 收缩系统调用面	hook 覆盖面和策略复杂度 只看 syscall/参数，难懂对象语义

权限系统的本质是最小授权。内核应让调用者只获得完成任务所需能力；任何跳过对象所属 `namespace` 的 `capability` 检查，都可能把容器内权限放大成宿主权

限。

权限不只有文件 mode 位。一个请求是否允许，可能取决于用户 ID、组 ID、capability、namespace、cgroup、seccomp、LSM、安全标签、mount 选项、fd 打开模式、对象引用和当前进程状态。OS 学习中的常见错误，是把“我是这个进程”误以为“我拥有所有资源”。

错误码也是契约。EACCESS 表示权限不足，ENOENT 表示对象不存在，EBADF 表示 fd 无效，EFAULT 表示用户指针不可访问，EINTR 表示系统调用被信号打断，EAGAIN 表示非阻塞对象暂时不能推进。高质量 OS 程序必须按错误码分支，而不是把所有失败都写成 fatal。

实践

分析一个系统调用时，按下面顺序写报告：

1. 用户态传入什么：整数、fd、地址、长度、flags、路径字符串。
2. 内核查找什么对象：进程表、fd 表、路径、inode、socket、VMA。
3. 检查哪些权限：对象存在、打开模式、读写执行、用户指针、资源限制。
4. 改变哪些状态：offset、buffer、引用计数、队列、页表、信号状态。
5. 返回什么：字节数、对象句柄、0/-1、errno、部分完成数量。
6. 哪些错误可重试，哪些错误必须失败，哪些错误表示调用者 bug。

以 `read(fd, buf, len)` 为例。内核先用 fd 查进程 fd 表，确认对象可读；再检查用户 buffer 是否可写；然后根据对象类型走普通文件、pipe、socket 或设备路径。普通文件可能从 page cache 复制，pipe 可能等待缓冲区，socket 可能返回当前已有字节。返回值为正时表示读取字节数，返回 0 可能是 EOF，返回 -1 时 errno 才解释失败原因。

以 `execve(path, argv, envp)` 为例。它不是创建新进程，而是在当前进程中替换用户态程序映像。PID 通常保持不变，地址空间被替换，用户栈重新构造，信号处理和 fd 状态按规则继承或关闭。若把 exec 理解成“启动新进程”，就无法解释 shell 中 fork+exec 的组合。

算法与实现分析

系统调用分发

系统调用入口通常把系统调用号和参数放在约定寄存器中。内核入口保存用户上下文，检查系统调用号范围，从系统调用表中找到处理函数，再把参数交给对应实现。返回时把结果或错误码放回约定位置。

```
syscall_entry(trap_frame):
    nr = trap_frame.reg[SYSCALL_NR]
    if nr >= syscall_table_size:
        return -ENOSYS
    current.user_context = trap_frame
    result = syscall_table[nr](trap_frame.args)
    trap_frame.return_value = result
    return_to_user(trap_frame)
```

这段算法的关键不是函数指针表，而是入口边界。内核必须确认当前线程确实来自用户态，不能让用户随便伪造内核上下文；必须处理信号、抢占、审计、seccomp 和 tracing；必须保证错误路径也能恢复到一致状态。

用户指针复制

用户指针复制要处理跨页、部分可访问和竞态。简单版本可以逐字节或逐页检查；高性能版本会利用硬件异常处理快速路径。教学模型先写清语义：复制失败时，系统调用返回 EFAULT，内核对象不能处在半提交状态。

```
copy_from_user(dst, user_ptr, len):
    copied = 0
    while copied < len:
        page = translate_user_page(user_ptr + copied, READ)
        if page == fault:
            return -EFAULT
        n = min(bytes_until_page_end(user_ptr + copied), len - copied)
        memcpy(dst + copied, kernel_map(page), n)
        copied += n
    return copied
```

真实内核还要处理 TOCTOU：检查之后用户线程可能修改内存。因此安全策略是把用户数据复制到内核缓冲区后再解析，不在解析过程中反复信任用户地址。路径字符串、数组参数和结构体参数都应遵守这个原则。

权限检查的顺序

权限检查要先定位对象，再检查主体是否有权操作对象，最后检查操作是否符合对象当前状态。以 `write(fd)` 为例：`fd` 必须存在；open file description 必须允许写；文件系统不能只读；用户 buffer 必须可读；长度不能超过限制；对象当前不能处于不允许写入的状态。

```
sys_write(fd, user_buf, len):
    file = fd_table_lookup(current, fd)
    if file == null: return -EBADF
    if !file.flags.can_write: return -EBADF
    if len > MAX_RW_COUNT: len = MAX_RW_COUNT
    kbuf = copy_from_user(user_buf, len)
    if kbuf.error: return -EFAULT
    return file.ops.write(file, kbuf, len)
```

不同错误码表达不同层次。`fd` 不存在和 `fd` 不可写都可能返回 `EBADF`；路径权限不足可能是 `EACCES`；文件系统只读可能是 `EROFS`；用户指针错误是 `EFAULT`。错误码越准确，调用者越能做正确恢复。

fork、exec、wait 的组合

Unix 风格进程创建不是一个巨大 `spawn`，而是 `fork`、`exec`、`wait` 的组合。`fork` 复制当前进程抽象；`exec` 替换当前进程映像；`wait` 回收子进程退出状态。这个设计让 `shell` 能在 `fork` 后、`exec` 前修改子进程 `fd`，例如重定向 `stdin/stdout`。

```
pid = fork()
if pid == 0:
    dup2(output_fd, STDOUT_FILENO)
    execve(program, argv, envp)
    _exit(127)
else:
    status = waitpid(pid)
```

算法不变量是：父进程和子进程在 `fork` 后分离推进；`exec` 成功后不会返回原用户程序；子进程退出后先变成 `zombie`，直到父进程 `wait` 取走退出状态。若没有 `zombie` 状态，父进程可能永远无法知道子进程怎样退出；若没有 `reaped`，进程表项会泄漏。

系统调用重启与信号

阻塞系统调用可能被信号打断。内核要决定返回 `EINTR`，还是在信号处理结束后自动重启系统调用。这个规则影响用户程序正确性：某些调用可安全重启，某些调用已经部分完成，不能假装从未发生。

```
blocking_read(fd, buf, len):
    while no_data_available(fd):
```

```

ret = sleep_interruptible(fd.waitq)
if ret == INTERRUPTED:
    if bytes_copied > 0:
        return bytes_copied
    if syscall_restartable(current):
        return -ERESTARTSYS
    return -EINTR
return copy_available_data()

```

不变量是：已经对用户可见的部分完成不能被自动撤销。若 `read` 已经复制了 10 个字节，再收到信号，返回 10 比返回 `EINTR` 更能表达真实状态。调用者必须循环处理短读短写。

TOCTOU 与路径权限

time-of-check to time-of-use 是系统调用常见漏洞。比如先检查路径 `/tmp/a` 权限，再打开这个路径；中间攻击者把路径替换成符号链接，最终操作对象已经不是被检查对象。正确做法是把权限检查绑定到已经解析并持有引用的对象。

```

unsafe_open(path):
    if may_access_path(path):
        return open_path(path)

safe_open(path):
    inode = resolve_and_get_inode(path)
    if inode == null:
        return -ENOENT
    if !may_access_inode(current, inode):
        put_inode(inode)
        return -EACCES
    return make_file_from_inode(inode)

```

路径名是查找输入，不是稳定对象。`fd`、`inode` 引用和 `file` 引用才是内核能持有的对象身份。安全系统调用设计要尽量让检查和使用落在同一个对象引用上。

测试

系统调用测试要验证边界，而不只是验证 happy path。

- 非法 `fd` 返回 `EBADF`，不应修改其他状态。
- 只读 `fd` 写入失败，错误应表达权限问题。
- 用户 `buffer` 不可访问时返回指针错误，不应让内核崩溃。
- 短读短写被调用者正确循环处理。
- `EINTR` 和 `EAGAIN` 被区分处理。
- `fork` 后父子进程共享 open file description 的 offset，独立 open 则 offset 分离。
- `exec` 后用户地址空间替换，但按规则保留或关闭 `fd`。
- `seccomp`、资源限制或 `capability` 缺失时，请求被拒绝且有可解释错误。

测试结论要写清“内核没有做什么”。例如权限失败时没有写入数据，非法指针没有部分污染内核对象，系统调用被信号打断后调用者能判断是否需要重试。负面保证是 OS 边界的核心。

源码级补充：entry_SYSCALL_64、capability、LSM 与 vDSO

x86-64 syscall 入口

x86-64 快速系统调用通常使用 `syscall/sysret`，入口地址和相关段选择子由 MSR 配置。进入内核后，低级入口要处理 `swaps`、保存用户栈、切到内核栈、保存寄存器、检查 `tracing/seccomp`，再进入 C 层分发。

```
entry_SYSCALL_64:
    swaps
    save user rsp into per-cpu area
    load kernel rsp
    push pt_regs
    call do_syscall_64
    check signals, reschedule, tracing
    restore regs
    sysretq or iretq
```

读 Linux 时 看 `arch/x86/entry/entry_64.S`、`arch/x86/entry/common.c` 和 `arch/x86/entry/syscalls/syscall_64.tbl`。系统调用号表把 ABI 固定下来；`SYSCALL_DEFINE3(write,...)` 这类宏把 C 实现接入表项。不要把系统调用入口理解成普通函数指针调用，它先是特权级和寄存器协议。

access_ok 与 copy 用户

`access_ok` 只做范围和用户地址边界的快速判断，不保证后续复制一定成功。真正复制仍可能 `fault`，所以 `copy_from_user` 必须有异常修复表或等价机制。真实内核会为用户访问指令建立 `fixup`：访问失败时跳到错误处理地址，而不是让内核普通 `page fault` 变成 `panic`。

```
copy_from_user(dst, src, len):
    if !access_ok(src, len):
        return bytes_not_copied
    instrument_usercopy_before()
    n = raw_copy_from_user(dst, src, len)
    instrument_usercopy_after()
    return n
```

这里返回值语义也很重要：很多内核 `helper` 返回“未复制字节数”，上层再转换成 `EFAULT` 或短完成。写教学代码时可以返回错误码，但必须明确是否允许部分复制。

capable、ns_capable 和 user namespace

`capability` 判断不再只是“`uid 0`”。`capable(CAP_NET_ADMIN)` 检查当前主体是否有网络管理能力；`ns_capable(user_ns, cap)` 把能力放进某个 `user namespace` 里解释。容器依赖 `user namespace` 把容器内 `root` 映射成宿主上非特权身份。

```
if (!ns_capable(net->user_ns, CAP_NET_ADMIN))
    return -EPERM;
```

安全边界的常见错误是检查了全局 `capability`，却忘了对象所属 `namespace`；或者在 `user namespace` 中给了过宽能力，导致容器内操作影响宿主对象。读源码时要跟着对象走：这个 `network namespace`、`mount namespace`、`inode` 或 `cgroup` 属于哪个 `user namespace`。

LSM hook 和 seccomp

LSM 在 VFS、进程、socket、BPF 等路径插入 `security_*` hook。SELinux、AppArmor、BPF LSM 都可在这些点拒绝操作。`seccomp` 则在系统调用入口附近按 BPF filter 检查系统调用号和参数，返回 `allow`、`errno`、`trap`、`trace`、`kill`、`user notify` 等动作。

```
sys_openat:
    file = path_openat(...)
    ret = security_file_open(file)
```

```

if ret:
    fput(file)
    return ret
install_fd(file)

```

安全检查要放在对象已解析且引用稳定之后，否则路径 TOCTOU 仍可能存在。seccomp 适合收缩系统调用面，但它看不到所有内核对象语义；LSM 能看对象语义，但策略复杂。二者互补，不是替代关系。

vDSO 为什么能加速时间调用

有些“系统调用”只是读取内核维护的只读数据，例如当前时间。频繁进入内核会有上下文切换开销。vDSO 把一小段内核提供的用户态代码和只读数据映射到进程地址空间，使 `clock_gettime` 等调用能在用户态完成快路径。

```

clock_gettime():
    if vDSO available and clock mode stable:
        read seqlock-protected time data in user mapping
        return computed time
    else:
        syscall clock_gettime

```

vDSO 依赖 seqlock 一类协议：内核更新时改变序列号，用户态读取若发现并发更新就重试。它展示了一个重要设计：不是所有内核服务都必须每次陷入内核，稳定只读数据可通过受控共享映射给用户态。

系统调用表、ABI 和兼容层

系统调用 ABI 一旦发布，就很难改变。系统调用号、参数寄存器、返回值约定、错误码范围和结构体布局都属于用户态契约。Linux x86-64 的系统调用表在 `arch/x86/entry/syscalls/syscall_64.tbl` 一类文件中维护；32 位兼容进程还可能走 compat syscall，把 32 位结构体转换为内核内部格式。

```

syscall ABI:
    rax = syscall number
    rdi, rsi, rdx, r10, r8, r9 = arguments
    rax = return value or negative errno

```

这解释了为什么内核常新增系统调用而不是随意改变旧系统调用语义。用户态程序、libc、容器 seccomp profile、审计工具和 trace 工具都依赖 ABI 稳定。教学 OS 也应把 syscall 编号集中管理，并写兼容性测试。

审计、ptrace 和 restart block

系统调用入口不只分发表项。它还可能经过 audit、ptrace、seccomp、ftrace 和 signal 检查。调试器通过 ptrace 可在 syscall enter/exit 停住进程，修改寄存器或观察参数；安全审计记录主体、对象和结果；信号可能让阻塞系统调用返回 EINTR 或进入 restart。

```

do_syscall_64(regs):
    nr = regs.orig_ax
    if ptrace_or_seccomp_active:
        nr = syscall_trace_enter(regs, nr)
    ret = dispatch_syscall(nr, regs)
    regs.ax = ret
    syscall_exit_work(regs)

```

restart block 用于保存重启系统调用所需状态。比如带 timeout 的阻塞调用被信号打断后，剩余 timeout 需要重新计算，而不能简单从头开始等待。系统调用重启是用户可见语义和内核内部状态的交叉点。

资源限制：rlimit 与 errno

权限检查之外，系统调用还要检查资源限制。进程可打开 fd 数、文件大小、地

址空间、栈、锁定内存、进程数都可能受 rlimit 或 cgroup 限制。错误码要表达限制来源。

限制	触发路径	常见错误
open files	open/dup/socket/accept	EMFILE/ENFILE
file size	write/truncate	EFBIG 或信号
address space	mmap/brk	ENOMEM
locked memory	mlock	ENOMEM/EPERM
process count	fork/clone	EAGAIN

这类错误通常不是权限不足，而是资源边界。高质量程序应把 EMFILE、ENOMEM、EAGAIN 分开处理：释放资源、限流、等待或明确失败。

用户内存数组和二次复制

execve、sendmsg、ioctl 这类系统调用会传入指针数组或嵌套结构。内核不能只复制外层结构后继续信任内层用户指针。正确做法是逐层验证长度上限，把需要长期使用的数据复制到内核内存或 pin 住用户页。

```
copy_iovec(user_iov, count):
    if count > UIO_MAXIOV:
        return -EINVAL
    k_iov = copy_array_from_user(user_iov, count)
    for each entry in k_iov:
        if entry.len overflows total:
            return -EINVAL
        if !access_ok(entry.base, entry.len):
            return -EFAULT
    return k_iov
```

二次复制和整数溢出是系统调用漏洞高发区。长度相加要检查溢出，数组个数要有上限，结构体 padding 不能泄露内核栈数据，输出结构体应先清零再填字段。

系统调用、缺页和文件 I/O 实现深讲

本节把系统调用、虚拟内存、fd、VFS 和 page cache 连成一条实现路径。用户看到的是 open/read/write/mmap/fork/exec，内核看到的是 fd 表、file、inode、VMA、PTE、folio、bio、request 和设备 completion。深入理解 OS，必须能把这些对象按生命周期串起来。

系统调用入口的最小状态

CPU 进入系统调用入口后，内核必须拿到系统调用号、参数寄存器、用户返回地址、用户栈、状态寄存器和当前线程。入口代码先建立可信执行环境，再调用 C 层分发。

```
syscall entry state:
    user RIP and RFLAGS
    user stack pointer
    syscall number
    argument registers
    current task pointer
    kernel stack pointer
    page table context
```

入口路径不能直接相信用户栈。用户栈可能不可访问、被另一个线程修改，甚至故意指向内核地址。参数寄存器只是整数，指针参数要在具体系统调用里通过 copy_from_user 或 iov 机制处理。

fd 表、file 和 inode

文件描述符不是文件本身。fd 是进程 fd 表中的整数索引；索引指向 open file description, 也就是内核的 `struct file;file` 再指向 inode 或 socket 等具体对象。多个 fd 可以指向同一个 file, 共享文件 offset; 多个 file 可以指向同一个 inode, 各自有不同 offset 和 flags。

```
process files_struct:
  fd 0 -> file A -> inode terminal
  fd 1 -> file B -> inode output.txt
  fd 2 -> file B -> inode output.txt

dup(1):
  new fd points to same file B
  file offset is shared

open same path again:
  new file C points to same inode
  file offset is independent
```

这解释了 shell 重定向、dup2、fork 后 fd 共享、close-on-exec 的行为。若父子进程 fork 后共享同一个 file, 它们写同一 fd 时共享 offset; 若各自重新 open, 同一 inode 也有不同 offset。

open 的路径解析

`open("/a/b/c")` 不是一次查表。内核从当前 root 或 cwd 出发, 逐级解析 dentry, 处理挂载点、符号链接、权限、缓存命中和可能的文件系统 I/O。路径解析必须防止 race 和越权。

```
open path:
  choose starting point: root or cwd
  split path components
  for each component:
    lookup dentry in dcache
    if miss, ask filesystem lookup
    follow mount point if needed
    handle symlink subject to flags and limits
    check execute/search permission on directory
  check final file permissions and flags
  allocate struct file
  install into fd table
```

路径解析有大量安全边界。攻击者可能在检查后替换路径组件; 符号链接可能指向意外位置; 目录权限和文件权限不同; `O_NOFOLLOW`、`O_CLOEXEC`、`openat`、`openat2` 都是在修复具体 race 或权限边界。

read 的三条路径：缓存命中、缓存未命中、直接 I/O

普通文件 read 首先查 page cache。若命中, 内核把缓存页复制到用户 buffer; 若未命中, 文件系统和块层发起 I/O, 设备完成后再复制; 若使用 `O_DIRECT`, 则尽量绕过 page cache, 把用户页或内核 bounce buffer 直接交给块层。

路径	数据流	代价
page cache hit	cache page 到用户 buffer	copy 成本, 低延迟
page cache miss	storage 到 page cache, 再 copy	I/O 等待, 后续可复用
direct I/O	用户页和块设备之间 DMA 或接近 DMA	对齐、pin 页、生命周期复杂

直接 I/O 不是“一定更快”。它减少 page cache 污染和复制, 但增加对齐、页

pin、设备队列和应用缓存管理成本。数据库常用它，是因为数据库自己管理缓存和写入顺序；普通应用通常受益于 page cache。

page cache 的 xarray 思想

文件的 page cache 需要按文件 offset 快速找到页。Linux 使用 address space 和 xarray 一类结构管理文件页。概念上可以理解为：

```
file address_space:
    key = page index within file
    value = folio containing file data

read page:
    index = file_offset / PAGE_SIZE
    folio = xarray_lookup(mapping, index)
    if folio exists and uptodate:
        copy from folio
    else:
        allocate folio and start read I/O
```

folio/page 的状态位很重要：uptodate 表示内容有效，dirty 表示需要写回，writeback 表示正在写，locked 表示某个路径正在操作。缺少这些状态，多个线程会重复读同一页或同时写回同一页。

write 的脏页路径

普通 buffered write 通常先把用户数据复制到 page cache，把页标记 dirty，然后返回。后台 writeback 或 fsync 再把 dirty 页写到存储设备。

```
buffered write:
    lookup or create page cache folio
    copy_from_user into folio
    mark folio dirty
    update file size if extending
    return bytes copied

later writeback:
    pick dirty folios
    filesystem maps file blocks
    submit bios to block layer
    on completion clear writeback
```

这解释了为什么 write 可能很快而 fsync 很慢。前者只把数据推进到内核内存和文件系统状态，后者要等设备日志顺序满足持久化要求。

mmap 读文件：不经过 read 也用 page cache

mmap 文件后，用户第一次访问对应地址会触发缺页。内核通过 VMA 找到文件和 offset，再从 page cache 找或读对应页，安装 PTE。之后用户态 load/store 直接访问映射页。

```
file mmap fault:
    CPU raises page fault at virtual address
    kernel finds VMA
    offset = vma.file_offset + (addr - vma.start)
    folio = page_cache_get_or_read(file.mapping, offset)
    install PTE mapping folio
    return to faulting instruction
```

所以 read 和 mmap 不是两套完全分离的缓存。它们常共享 page cache。一个进程 read 过的文件页，另一个进程 mmap 访问可能命中；反过来也成立。区别在于数据进入用户态的方式：read 复制，mmap 建 PTE。

COW fault 的实现分支

MAP_PRIVATE 文件映射或 **fork** 后匿名页共享，都可能使用 COW。写入时，硬件发现 PTE 不可写，触发保护异常。内核检查 VMA 允许写且 PTE 是 COW，然后复制页。

```
COW write fault:
  locate VMA and PTE
  verify write is allowed by VMA
  if PTE is COW:
    old = pte.page
    new = allocate_page()
    copy_page(new, old)
    install writable PTE to new page
    decrement old refcount
    flush TLB for address
  else:
    signal permission fault
```

COW 的难点在并发。两个线程可能同时 **fault** 同一页；**fork**、**unmap**、**reclaim** 也可能并发修改页表。内核必须用页表锁、页引用计数和重试逻辑保证不会复制错误页或泄漏引用。

缺页错误如何变成信号或 EFAULT

同一个坏地址，在不同上下文下结果不同。用户态直接访问坏地址，内核通常发送 **SIGSEGV**。系统调用 **copy_from_user** 过程中访问坏地址，系统调用应返回 **EFAULT**。内核自己访问坏内核地址，则是内核 bug，可能 **oops** 或 **panic**。

上下文	fault 结果	原因
用户指令 load/store	signal	用户程序违反地址空间规则
内核 copy user	-EFAULT	系统调用参数错误，可返回给调用者
内核普通指针	oops/panic	内核自身错误或硬件故障
缺页可修复	返回重试指令	懒分配、COW、文件页读入

这就是 exception table 的价值：内核某些 copy 用户路径可以声明“若这里 **fault**，跳到修复位置返回错误”，而不是让内核崩溃。

epoll 的 readiness 不是 completion

文件和 socket 的 **readiness** 只表示“现在尝试某类操作可能不阻塞”，不是保证一定读到固定字节，也不是 I/O completion。多线程或边缘触发下，**readiness** 状态可能在应用处理前就变化。

```
epoll wait:
  if socket receive queue not empty:
    report readable
  user thread wakes
  another thread may consume data first
  read may still return EAGAIN on nonblocking fd
```

所以非阻塞 I/O 必须在循环中处理 **EAGAIN**。**epoll** 降低“等待很多 fd”的成本，但不取消对象状态变化和并发竞争。**readiness** API 和 **io_uring** completion API 的语义不同，不能混用理解。

io_uring 的提交和完成环

io_uring 用共享 ring 减少系统调用次数。用户态写 submission queue entry，内核消费 SQE 并执行请求，完成后写 completion queue entry。关键仍是所有权、顺序和生命周期。

```
io_uring lifecycle:
  userspace fills SQE
  userspace advances SQ tail with store-release
  kernel observes SQ tail with load-acquire
  kernel validates fd, buffer, opcode
  request is issued to file/socket/block layer
  completion CQE is written
  userspace reads CQE and recycles resources
```

若 `buffer` 在 `completion` 前释放，异步路径可能使用悬空内存；若 `fd` 在请求执行前关闭，内核必须通过引用计数保证对象要么仍有效，要么请求明确失败；若应用不消费 CQE，完成队列会背压。

fd 生命周期和引用计数

并发 `close` 和 I/O 是系统调用实现的经典难点。用户关闭 `fd` 后，`fd` 表项删除，新的 `open` 可能复用同一整数。但已经拿到 `struct file` 引用的内核 I/O 可以继续完成。

```
sys_read(fd):
  file = fget(fd)           // increments file refcount if fd valid
  if file == null:
    return -EBADF
  ret = vfs_read(file)
  fput(file)                // may release after last ref
  return ret

sys_close(fd):
  file = remove fd table entry
  fput(file)
```

这解释了为什么 `fd` 整数不能长期当对象身份。`close` 后同一个数字可能马上指向另一个文件。内核内部必须使用对象引用，用户态高层库也要小心 `fd reuse` 带来的取消和多线程 `race`。

文件系统日志和 page cache 的关系

`page cache` 保存文件数据，文件系统日志通常保护元数据或特定模式下的数据顺序。`dirty page` 写回和日志提交是相关但不同的状态机。`fsync` 必须把两者推进到一致点。

```
fsync(file):
  write dirty file data pages
  wait data writeback completion
  commit filesystem metadata transaction
  issue flush/FUA as needed
  return only after required durability point
```

不同文件系统和挂载选项会改变数据与元数据顺序。例如 `ordered` 模式要求相关数据块在提交元数据前写出；`writeback` 模式可能只保证元数据一致；`journal data` 模式连数据也进日志。应用不能凭感觉推断崩溃语义，必须理解目标文件系统契约。

把一次请求完整编号

分析复杂 I/O 时，给每一层对象编号能避免混乱。

编号	对象	状态问题
1	用户 <code>fd</code> 整数	是否仍指向预期 <code>file</code>
2	<code>struct file</code>	引用计数、 <code>offset</code> 、 <code>flags</code>
3	<code>inode/address space</code>	文件大小、 <code>page cache</code> 、权限

4	folio/page	uptodate、dirty、locked、writeback
5	bio/request	块映射、队列、超时
6	DMA buffer/descriptor	map/unmap、所有权、completion
7	device command	status、错误码、reset 影响

任何一层状态没写清楚，都会产生 bug。例如 fd 已 close 但 file 引用还在，page dirty 但 writeback 未完成，request 已超时但设备稍后仍 completion，reset 后旧 DMA 仍可能写内存。

实现级测试矩阵

- dup 后两个 fd 共享 offset；重新 open 的 fd 不共享 offset。
- O_CLOEXEC fd 在 exec 后关闭，普通 fd 按规则保留。
- 坏用户指针在 read/write 中返回 EFAULT，不导致内核崩溃。
- MAP_PRIVATE 写入后文件内容不变，MAP_SHARED 按规则变 dirty。
- fork 后 COW 写入不影响父子另一方。
- close 与并发 read/write 不造成 use-after-free，fd reuse 不混淆对象。
- page cache 命中路径不发块 I/O，冷缓存路径能观察到块 I/O。
- fsync 后再模拟崩溃，文件数据和目录项语义符合预期。
- 非阻塞 fd 在未 ready 时返回 EAGAIN，epoll 事件后仍能处理竞态。
- io_uring 请求在 completion 前不能释放 buffer 或丢失对象引用。

第三阶段深讲：特权边界、装载和内存提交点

系统调用、exec 和缺页处理都在改变“谁能访问什么”。这类代码最怕半提交：权限已经放开但对象还没准备好，地址空间已经切换但旧程序还要返回错误，页表已经失效但物理页提前释放。学习这一章时，应把每条路径拆成准备、验证、提交、回滚四个阶段，而不是只记住 API 名字。

边界代码的共同结构

边界代码有三个共同问题。第一，输入来自不可信主体：系统调用参数来自用户寄存器和用户内存，ELF 文件可能被攻击者构造，page fault 地址可能来自错误或恶意访问。第二，执行路径会改变全局可见状态：fd 表、地址空间、页表、文件页、凭据和调度状态。第三，失败必须可解释：有些失败返回错误码，有些发送信号，有些说明内核自身已经损坏。

```
boundary operation:
  snapshot untrusted input
  validate object identity and permissions
  allocate temporary resources
  build new state off to the side
  commit with a short ordered sequence
  publish visibility only after invariants hold
  roll back temporary resources on every pre-commit failure
```

这个结构适用于 openat、execve、mmap、mprotect、COW fault 和文件写回。源码阅读时，只要找到“提交点”在哪里，前后语义就会清楚很多：提交点之前失败要回到旧状态，提交点之后失败只能按新状态处理或终止任务。

系统调用返回值也是 ABI

系统调用的结果不只是一个整数。ABI 规定返回值寄存器、错误码编码、哪些寄存器被破坏、哪些状态在返回前必须处理。libc wrapper 再把内核返回值转换成 `-1` 和 `errno`。如果教学 OS 忽略这些细节，用户程序就只能靠猜。

返回类型	例子	调用者应该怎样理解
非负数量	<code>read/write</code> 返回字节数	可能短读/短写，不等于请求全部完成
新句柄	<code>open/socket/dup</code> 返回 <code>fd</code>	<code>fd</code> 是进程表索引，不是对象永久身份
<code>0</code> 表示成功	<code>close/mprotect/munmap</code>	只说明接口语义完成，不说明确持久化
<code>-errno</code>	内核内部常见约定	wrapper 转成 <code>errno</code> 或直接传递
信号	用户访存错误、非法指令	错误发生在用户执行流，不一定有 <code>syscall</code> 返回

`close` 的错误尤其容易被误解。`fd` 表项可能已经释放并可被复用，即使底层写回错误在 `close` 期间才报告。用户态不能在 `close` 返回错误后再重试同一个 `fd`，因为这个整数可能已经指向别的文件。正确设计要把持久化错误尽量通过 `fsync` 这类明确边界暴露。

restartable syscall 和 EINTR

阻塞系统调用可能在等待期间收到信号。内核有时返回 `EINTR`，有时把系统调用改写成可重启状态，让信号处理返回后重新执行。重启不是简单“再调一次函数”，因为部分 I/O、超时参数和用户可见副作用必须精确处理。

```
blocking read:
  copy no bytes yet
  sleep waiting for data
  signal arrives
  if handler installed:
    either return -EINTR
    or arrange restart after signal frame

partial read:
  copied some bytes
  signal arrives
  return copied bytes, not -EINTR
```

这个规则解释了为什么系统调用实现要记录“是否已经产生用户可见进展”。已经复制给用户的字节、已经消耗的 `socket` 数据、已经创建的 `fd`，不能因为信号到来就假装从未发生。测试时应同时覆盖“阻塞前被打断”和“部分完成后被打断”。

copy_from_user 的分段提交

大 `read/write` 不应一次性申请巨大内核缓冲。常见实现会分段处理用户 `iovec` 或页片段。分段处理带来短完成语义：前几段成功，后面遇到坏页或信号，返回值可能是已经完成的字节数。

```
write_iter(file, iov):
  done = 0
  for each segment in iov:
    while segment has bytes:
      chunk = copy_chunk_from_user(segment)
      if chunk fault:
        return done > 0 ? done : -EFAULT
      ret = file_write_chunk(file, chunk)
```



```

if ret < 0:
    return done > 0 ? done : ret
done += ret
return done

```

这里的提交粒度是 chunk。若 chunk 已写入 pipe 或 socket queue，就不能在后续 fault 时撤销。文件系统写入还要区分 page cache dirty、journal commit 和设备持久化。一本教材若只说“复制用户 buffer 再写文件”，读者会漏掉短写、EFAULT 和持久化边界。

exec 的三道权限门

`execve` 至少经过三道权限门。第一道是路径和文件权限：调用者是否能走到该文件，文件是否可执行，挂载是否 `noexec`。第二道是二进制格式和装载权限：ELF 段权限是否合理，解释器是否允许执行，栈是否可执行。第三道是凭据转换：`setuid`、`file capability`、`LSM`、`no_new_privs` 和 `ptrace` 规则共同决定新程序的身份。

```

exec permission gates:
  path permission and executable inode
  binary format and memory layout policy
  credential transition and security hooks

```

这三道门的错误码和失败时机不同。路径不存在是 `ENOENT`；不是可执行格式可能是 `ENOEXEC`；参数过大是 `E2BIG`；权限不足是 `EACCES` 或 `EPERM`；解释器不存在也会让整个 `exec` 失败，但旧程序必须还能拿到错误。把这些都归为“启动失败”，无法写出 shell、容器运行时或服务管理器。

ELF 段权限不是 section 权限

运行时装载看 program header，链接和调试看 section header。很多 ELF 被 `strip` 后没有完整 section 信息，仍然能运行。教学装载器如果读取 `.text/.data/.bss` 名字来建立映射，就是把链接器视角误当成内核视角。

视角	使用对象	典型消费者
运行时装载	<code>PT_LOAD/PT_INTERP/PT_PHDR</code>	内核 <code>exec</code> 、动态链接器
链接和重定位	section、symbol、relocation	链接器、objdump、调试器
调试展开	DWARF、CFI、eh_frame	gdb、异常展开、profiler

段映射还要处理文件尾页。若 `p_filesz` 结束在页中间，而 `p_memsz` 更大，同一页中从文件末尾到页末的部分必须清零；后续整页 BSS 也要清零或懒分配零页。这个细节若错，未初始化全局变量会读到旧文件内容或未清理内存。

地址空间切换的提交顺序

`exec` 提交时，最危险的是先激活新页表再访问旧地址空间数据，或先释放旧 mm 再需要从旧用户栈取参数。安全做法是：提交前把 `path`、`argv`、`envp`、解释器路径和 `auxv` 所需数据复制到内核或新栈页；新 mm 完整可用后再短序列切换。

```

safe exec commit:
  prepare new_mm, new_stack, new_regs
  copy all old user data before old_mm is dropped
  stop sibling threads if multithreaded
  install credentials that belong to new image
  install current.mm = new_mm
  switch hardware page table
  make old_mm unreachable from current
  drop old_mm references
  return to new user entry

```

顺序里有两个可见性点：`current.mm` 对内核其他路径可见，硬件页表根对 CPU 访存可见。多核系统还要考虑同一地址空间是否仍在其他 CPU 活跃。若旧 mm 的页表页在远端 CPU TLB 失效前被释放，后果和普通 `munmap` 的 shutdown 错误一样严重。

`mprotect` 的权限收缩和扩大

`mprotect` 改权限时，扩大权限和收缩权限的风险不同。扩大权限要检查 VMA 的 `may` 权限和文件打开模式；收缩权限要立刻让硬件旧权限失效。比如把 RW 改成 R，如果不 flush TLB，CPU 可能继续使用缓存的 `writable` 翻译。

```
mprotect(range, new_prot):
    split vmas at range boundaries
    for each vma in range:
        if new_prot exceeds vma.may_prot:
            rollback splits if needed
            return -EACCESS
    update vma.prot
    update present PTE permission bits
    flush TLB for affected range
```

VMA 切分是另一个回滚点。若先切分出三个 VMA，后续某段权限检查失败，需要合并回原状态或保证中间状态不会被观察到。真实内核会通过锁保护 mm 结构，让其他线程不能在半修改区间中处理 fault。

`munmap` 和延迟释放

`munmap` 删除映射后，用户不能再访问这段虚拟地址。但对应物理页不一定立刻释放：其他进程可能映射同一文件页，I/O 可能持有页引用，RCU 或 TLB gather 可能延迟释放页表页。语义上的不可访问和资源上的最终释放是两件事。

```
munmap(range):
    remove or split VMAs
    clear PTEs in range
    collect old PTE pages and mapped pages into batch
    flush TLB for range on all active CPUs
    drop page references after stale translations are gone
    free page table pages when no walker can see them
```

这能解释为什么内存统计不是瞬时下降，也解释为什么页表释放要和 TLB shutdown、页表遍历锁、RCU 等机制配合。教学 OS 可简化，但必须保留顺序原则：先撤销可达性，再失效旧翻译，最后释放底层资源。

page fault 的锁顺序

缺页路径会碰到 mm 锁、VMA 锁、页表锁、page cache 锁、folio 锁、文件系统锁和 I/O 等待。锁顺序错了会出现非常隐蔽的死锁。一个基本原则是：先稳定地址空间和 VMA，再稳定页表项，再稳定物理页或文件页；等待 I/O 时尽量不持有会阻塞其他 fault 的大锁。

```
file page fault:
    take mmap read lock or VMA lock
    find and validate VMA
    locate page cache folio
    if folio locked or needs I/O:
        drop/reduce locks that block unrelated faults
        wait for folio
        retry validation
    take page table lock
    install PTE if still valid
    drop locks
```

这里的 `retry` 很重要。等待 I/O 期间，另一个线程可能 `munmap` 或 `mprotect`

了同一区间。醒来后不能直接安装 PTE，必须重新验证 VMA 和权限。许多内存管理 bug 都来自“睡眠前检查过，醒来后不再检查”。

源码阅读路线

这一阶段读 Linux 源码可以按路径而不是文件顺序推进。

路径	关键文件或函数	读法
系统调用入口	arch/x86/entry/、kernel/seccomp.c	参数、seccomp、返回工作
用户复制	arch/x86/lib/usercopy*、lib/iov_iter.c	fault fixup、短拷贝、iov
exec/ELF	fs/exec.c、fs/binfmt_elf.c	准备新镜像和提交点
VMA 管理	mm/mmap.c、include/linux/mm_types.h	maple tree、切分、合并
缺页	mm/memory.c、mm/filemap.c	anon/file/COW fault 分支
页表失效	mm/tlb.c、架构 TLB 代码	batch、shutdown、延迟释放
回收/OOM	mm/vmscan.c、mm/oom_kill.c	reclaim 顺序和 victim 选择

每条路径都按同一张表做笔记：输入对象、持有锁、可睡眠点、提交点、回滚对象、用户可见结果。这样能把几万行源码压成可理解的状态机。

边界反例集

高质量测试要故意打破边界。

反例	构造方法	必须观察到
用户指针半页坏	iovec 前半有效后半 unmapped	短完成或 EFAULT，内核不崩
exec 坏解释器	ELF 的 PT_INTERP 指向不存在路径	exec 失败，旧程序继续
ELF 溢出段 mprotect 后旧权限	p_vaddr + p_memsz 整数溢出 RW 改 R 后立即写	拒绝装载 必须 fault，不能被 TLB 旧权限放行
fault 等 I/O 时 munmap close 与 read 并发	线程 A fault 等页，线程 B unmap 一个线程 read 已 fget，另一个 close 并复用 fd	A 醒来后重新验证并失败 read 用旧 file 引用，fd 数字可复用
memcg OOM	cgroup 内触发限制	只在 cgroup 内选择 victim

这些反例比正常路径更能检验实现质量。正常路径只证明代码能跑，反例才能证明边界没有漏。

3.2 exec、ELF 与用户地址空间建立

exec、ELF 装载与用户栈

原理

`fork` 复制了一个进程的执行环境，而 `exec` 把当前进程的用户态程序替换成另一个程序。这个替换不是“创建新进程”，因为 `pid`、父子关系、部分资源限制和很多内核对象仍然属于同一个进程；它也不是普通文件读取，因为内核必须验证可执行文件格式、建立新的地址空间、设置入口点、构造用户栈、处理解释器、重置部分信号语义并关闭带 `close-on-exec` 标志的 `fd`。

ELF 装载器是系统调用边界、文件系统、虚拟内存和权限模型的交汇点。用户给内核一个路径和参数数组，内核必须在不信任用户内存的前提下读取这些字符串；路径解析得到 `inode` 后，还要检查执行权限、文件类型、挂载选项和安全策略；读取 ELF 头后，装载器根据 `program header` 而不是 `section header` 建立映射。`section` 是链接和调试视角，`program segment` 才是运行时装载视角。

核心思想

`exec` 的核心契约是原子替换用户态映像：成功后旧程序不能继续执行；失败时旧程序必须仍然可运行，除非失败发生在明确不可恢复的提交点之后。

真实系统还要处理动态链接。若 ELF 带 `PT_INTERP`，内核并不直接跳到主程序入口，而是同时映射解释器，把控制权交给动态链接器。动态链接器再加载共享库、重定位符号，最后跳到用户程序的入口。内核只负责把 `aux vector`、入口、`program header` 信息和随机数等必要元数据放到用户栈。

实践

一个教学内核中的 `execve(path, argv, envp)` 可以拆成九个阶段：

- 1. 从用户内存复制 `path`、`argv`、`envp`，设置长度上限。
- 2. 路径解析并打开可执行 `inode`。
- 3. 检查文件类型、执行权限、`noexec` 挂载选项和安全策略。
- 4. 读取并验证 ELF header。
- 5. 扫描 `program header`，计算新地址空间布局。
- 6. 创建临时地址空间，映射 `PT_LOAD` 段和用户栈。
- 7. 若存在 `PT_INTERP`，装载解释器并设置实际入口。
- 8. 构造用户栈：`argc`、`argv` 指针、`envp` 指针、`auxv`、字符串和对齐填充。
- 9. 提交：切换进程 `mm`、寄存器入口和栈指针，关闭 `CLOEXEC` `fd`。

这里最容易讲错的是失败回滚。前七步应尽量在临时对象中完成：临时 `mm`、临时 `VMA`、临时页、临时 `fd` 引用。只有全部验证成功后，才进入提交阶段并销毁旧地址空间。否则一个坏 ELF 就可能把调用者进程破坏到无法返回错误码。

对象	成功后状态	失败回滚
旧 mm	被新 mm 替换并释放引用	保持可运行
新 mm	成为 current 的地址空间	释放所有 VMA 和页
文件引用	可执行文件可关闭或保留映射引用	put inode/file 引用
fd table	关闭 <code>FD_CLOEXEC</code> 条目	不应改变
信号处理	捕获处理器重置，忽略语义按规则保留	不应改变

寄存器	RIP 指向入口, RSP 指向新用 户栈	不应改变
-----	--------------------------	------

算法与实现分析

ELF header 验证

ELF 验证要防止把任意文件当代码执行。最小检查包括 magic、class、endianness、machine、type、program header 范围和整数溢出。

```
validate_elf(file):
    eh = read_at(file, 0, sizeof(Elf64_Ehdr))
    require eh.magic == 0x7f 'E' 'L' 'F'
    require eh.class == ELFCLASS64
    require eh.machine == EM_X86_64
    require eh.phentsize == sizeof(Elf64_Phdr)
    require eh.phnum <= MAX_PHDRS
    require checked_mul(eh.phnum, eh.phentsize, &ph_size)
    require checked_add(eh.phoff, ph_size, &ph_end)
    require ph_end <= file.size or file supports sparse read policy
```

复杂度是 $O(\text{phnum})$ 。真正关键的是边界检查： $p_offset + p_filesz$ 、 $p_vaddr + p_memsz$ 、页对齐和权限组合都可能溢出。装载器不能相信编译器或链接器一定生成合法文件，因为攻击者可以手工构造 ELF。

段映射

PT_LOAD 段描述文件区间到虚拟地址区间的映射。文件大小 p_filesz 是需要从文件读出的内容，内存大小 p_memsz 包括 BSS 零填充。

```
map_load_segment(mm, file, ph):
    require ph.memsz >= ph.filesz
    require same_page_offset(ph.vaddr, ph.offset)
    start = page_down(ph.vaddr)
    end   = page_up(ph.vaddr + ph.memsz)
    file_start = page_down(ph.offset)
    prot = prot_from_flags(ph.flags)
    map_file(mm, start, end, file, file_start, prot)
    zero_range(mm, ph.vaddr + ph.filesz, ph.vaddr + ph.memsz)
```

若采用 demand paging, `map_file` 不必立即读入所有页面，只需建立 VMA，等 page fault 时按文件偏移填充页面。这样 `exec` 延迟低，内存占用也低。若采用 eager loading，实现简单但启动大程序会有明显 I/O 峰值。

用户栈构造

用户栈是 ABI 契约的一部分。教学系统可以简化 auxv，但不能把 argv 字符串和 argv 指针混为一谈。

```
build_user_stack(argv, envp, auxv):
    sp = USER_STACK_TOP
    for s in reverse(strings(argv, envp)):
        sp -= len(s) + 1
        copy_to_new_user(sp, s)
        remember_pointer(s, sp)
    sp = align_down(sp, 16)
    push_null()
    push_auxv(auxv)
    push_null()
    push_pointers(envp)
    push_null()
    push_pointers(argv)
```

```
push(argc)
return sp
```

栈构造要设置总大小上限，防止用户传入巨量参数耗尽内核内存。所有用户字符串必须先复制到内核或临时页中，因为在 `exec` 期间旧地址空间即将被替换；若边复制边销毁旧 `mm`，就可能读到无效地址。

提交点

提交阶段要尽量短，并且顺序清晰：

```
commit_exec(new_image):
    old_mm = current.mm
    current.mm = new_image.mm
    current.regs.ip = new_image.entry
    current.regs.sp = new_image.stack_pointer
    reset_caught_signal_handlers(current)
    close_cloexec_fds(current.files)
    activate_mm(new_image.mm)
    drop_mm(old_mm)
    return_to_user()
```

提交点之后不应再执行可能失败且需要返回旧程序的操作。比如关闭 `CLOEXEC` `fd` 一般可以在提交逻辑中完成，因为关闭失败不应阻止 `exec` 成功；但读取解释器、分配栈页和权限检查必须放在提交点之前。

测试

1. 正常 ELF：静态程序能打印 `argc/argv/envp`，入口地址和栈对齐正确。
2. 坏 magic：返回 `ENOEXEC`，旧程序继续运行。
3. 无执行权限：返回 `EACCES`，`fd table` 不变化。
4. 段溢出：构造 `p_vaddr + p_memsz` 溢出的 ELF，装载器必须拒绝。
5. BSS 清零：全局未初始化数组初值必须为零。
6. `FD_CLOEXEC`：带标志 `fd` 在新程序中不可见，普通 `fd` 仍可用。
7. 大 `argv`：超过限制返回 `E2BIG`，不破坏旧地址空间。
8. 动态链接入口：带 `PT_INTERP` 时入口应进入解释器而不是主程序 `e_entry`。

本章合格标准：能区分 `fork` 和 `exec`；能解释 ELF program header 的运行时意义；能画出用户栈布局；能指出 `exec` 的失败回滚边界和提交点。

源码级补充：load_elf_binary、auxv 与 binfmt

ELF 结构字段

读 ELF 装载源码时要区分三类头：`Elf64_Ehdr` 描述整个文件，`Elf64_Phdr` 描述运行时段，`Elf64_Shdr` 描述链接/调试节。内核运行装载主要看 program header；section header 即使被 stripped，也不影响程序运行。

字段或类型	含义	装载用途
<code>PT_LOAD</code>	可映射段	建立 VMA，设置 R/W/X 权限

PT_INTERP	动态解释器路径	装载 <code>ld-linux.so</code> 并把入口交给它
PT_PHDR	program header 自身位置	传给用户态动态链接器
PT_GNU_STACK	栈权限提示	决定用户栈是否可执行
e_entry	ELF 入口虚拟地址	静态程序入口或解释器最终跳转参考

Linux 源码入口是 `fs/binfmt_elf.c` 的 `load_elf_binary`。读时抓住阶段：读取 header，读取 program header，检查 interpreter，建立新 mm，映射 load segments，设置 brk，构造栈和 auxv，设置寄存器。

aux vector 布局

auxv 是内核放到用户栈上的键值数组，给动态链接器和 libc 传递信息。常见项包括 `AT_PHDR`、`AT_PHENT`、`AT_PHNUM`、`AT_ENTRY`、`AT_BASE`、`AT_PAGESZ`、`AT_RANDOM`、`AT_EXECFN`、`AT_NULL`。

```
user stack top:
strings for argv/envp
random bytes
auxv: AT_PHDR, value
      AT_PHENT, value
      AT_PHNUM, value
      AT_ENTRY, value
      AT_RANDOM, pointer
      AT_NULL, 0
envp pointers, NULL
argv pointers, NULL
argc
```

`AT_RANDOM` 为 stack canary 和 ASLR 提供随机种子；`AT_BASE` 指向动态链接器基址；`AT_PHDR` 让动态链接器找到主程序段表。若 auxv 构造错误，程序可能在 main 前就崩溃。

解释器脚本和 binfmt

`#!/bin/sh` 脚本不是 ELF。内核识别 shebang 后，把解释器路径和脚本路径重写成新的 argv，再递归执行解释器。Linux 用 binfmt 框架支持多种二进制格式：ELF、脚本、misc 注册格式等。

```
execve("script.sh", ["script.sh", "a"]):
  read first line "#!/bin/sh"
  new argv = ["/bin/sh", "script.sh", "a"]
  execve("/bin/sh", new argv)
```

这里要限制递归深度，否则脚本解释器互相指向会无限递归。binfmt_misc 允许用户注册自定义格式，例如通过魔数把某类文件交给解释器。安全检查必须覆盖最终解释器和原脚本，不能只检查第一层路径。

vfork + exec 为什么高效但危险

vfork 让子进程在 exec 或 exit 前与父进程共享地址空间，父进程通常被挂起。它避免了复制页表和 COW 设置，适合立刻 exec 的路径；但子进程若在 exec 前修改内存或调用复杂函数，会破坏父进程状态。

```
child = vfork()
if child == 0:
    // only async-signal-safe, minimal setup
    dup2(fd, STDOUT_FILENO)
    execve(path, argv, envp)
    _exit(127)
```

现代系统常用 `posix_spawn` 在库和内核之间选择更安全高效的实现。理解 `vfork` 的意义，不是鼓励随便使用，而是理解 `exec` 前后地址空间提交点的重要性。

setuid、凭据变化与 no_new_privs

`exec` 不只是换代码。执行 `setuid/setgid` 文件时，进程凭据可能变化；`capability` 也会根据文件能力、`bounding set`、`ambient set` 和 `no_new_privs` 重新计算。动态链接器和环境变量处理必须考虑提权执行，避免用户用 `LD_PRELOAD` 一类环境影响高权限程序。

```
exec_security_transition(file):
    read inode mode, owner, file capabilities
    if no_new_privs:
        suppress privilege gain
        compute new effective/permitted/inheritable caps
        clear dangerous personality and env effects
        install new credentials at commit
```

这解释了为什么 `exec` 路径会调用 LSM hook 和凭据提交逻辑。安全检查不能只看 ELF 格式合法，还要看执行后主体身份是否改变、哪些 fd 继承、`dumpable` 标志如何更新、`ptrace` 是否允许继续。

ASLR、PIE 与栈保护输入

`exec` 建立新地址空间时，会选择随机化的 `mmap base`、`stack base`、`brk` 和 `VDSO` 位置。PIE 主程序也可随机基址加载。内核还向 `auxv` 提供 `AT_RANDOM`，供 `libc` 初始化 `stack canary`。

机制	exec 阶段输入
ASLR	随机选择 <code>mmap</code> 、 <code>stack</code> 、 <code>brk</code> 、 <code>VDSO</code> 、PIE 基址
<code>stack canary</code>	<code>AT_RANDOM</code> 提供随机字节
<code>NX stack</code>	<code>PT_GNU_STACK</code> 和默认策略决定栈可执行性
<code>RELRO/动态链接</code>	<code>program header</code> 和 <code>dynamic section</code> 给链接器信息

这些保护不是 `exec` 独立完成的，但 `exec` 负责把初始地址布局 and `auxv` 种子交给用户态运行时。若地址布局固定或随机数可预测，后续保护会明显削弱。

exec 与多线程

多线程进程调用 `exec` 时，成功后只留下调用 `exec` 的线程，其他线程被清理。因为新程序映像不可能继承旧程序的线程栈、锁状态和用户态同步对象。内核必须停止同一线程组中的其他线程，并让共享资源进入一致状态。

```
de_thread_for_exec(current):
    signal other threads to exit
    wait until thread group is single-threaded
    take over thread group leader identity if needed
    proceed to install new mm
```

这也是为什么 `exec` 提交点必须非常谨慎：旧线程可能持有用户态锁，但 `exec` 后这些锁没有意义；内核对象如 `fd table`、`credentials`、`signal struct` 还要按规则继承或重置。

binfmt_misc 与解释器安全边界

`binfmt` 框架允许注册自定义格式，把某些 `magic` 或扩展名交给解释器。它方便运行脚本、模拟器格式或语言运行时，但也扩大了 `exec` 策略面。解释器路径、参数重写、递归深度、权限继承和 `mount noexec` 都必须纳入检查。

```
registered format:
  magic = "\xCA\xFE"
  interpreter = "/usr/bin/custom-runner"
exec file:
  kernel rewrites argv:
    custom-runner original-file original-args...
```

安全模型必须说明权限检查作用于原文件、解释器，还是二者都检查。只检查脚本可执行而不检查解释器，或绕过 noexec 挂载，都会形成边界漏洞。

3.3 虚拟内存、缺页与页缓存

虚拟内存与地址空间

原理

虚拟内存把每个进程看到的地址空间和物理内存分开。进程使用虚拟页，页表把虚拟页映射到物理页，并附带 readable、writable、executable、present 等权限。一次访问是否合法，不只取决于地址是否存在，还取决于访问类型和权限位。

隔离来自默认不可见。一个进程不能随便读写另一个进程的页；没有映射的地址会触发缺页；只读页不能写；不可执行页不能取指。操作系统通过这些规则同时提供保护、共享和按需分配。

虚拟地址空间不仅是页表。内核还维护 VMA 或等价区域结构，用来描述一段虚拟地址的来源和权限：匿名内存、栈、堆、共享库、文件映射、设备映射。页表项是某一页当前是否能翻译，VMA 是这一段地址理论上是否合法、缺页时应该怎样处理。没有页表项不一定是错误，可能只是尚未按需分配。

一次访问的大致路径是：CPU 发出虚拟地址；TLB 命中则直接得到物理地址和权限；TLB 未命中则硬件或内核进行页表遍历；页表项不存在或权限不符时触发异常；内核根据 VMA 判断是合法缺页、权限错误还是非法访问；合法缺页可能分配物理页、从文件读页、建立 COW 私有页或映射零页。

copy-on-write 是虚拟内存最有代表性的协议。fork 时，父子进程可以先共享物理页，并把页表标记为只读和 COW。任何一方写入时触发保护异常，内核复制物理页，更新写入方页表，再继续执行。这样 fork 不需要立刻复制整个地址空间，但必须正确维护引用计数和权限。

实践

实验的 PageTable 支持按页映射。读访问要求 present 和 readable；写访问要求 writable；执行访问要求 executable。地址翻译保留页内 offset，帮助读者理解“页号转换，页内偏移不变”。

输入	计算
page size	4096
virtual address	0x401234
virtual page number	0x401
offset	0x234
physical page number	查页表得到
physical address	physical_page * 4096 + offset

权限矩阵要单独写：

访问	present	readable	writable	结果
load	yes	yes	no	成功
store	yes	yes	no	权限错误
fetch	yes	yes	no	NX 错误
load	no	-	-	缺页或非法

观察真实系统时，要区分虚拟大小、RSS、page cache、minor fault 和 major fault。虚拟地址空间很大，不代表物理内存已经占用；RSS 增加，可能是匿名页，也可能是文件页；minor fault 可能只是建立映射，major fault 更可能等待存储设备。

算法与实现分析

物理页分配：free list、bitmap 和 buddy

最简单的物理页分配器是 free list。每个空闲页框放进链表，分配时弹出一个，释放时放回去。它的分配和释放都是 $O(1)$ ，适合固定大小页框；缺点是难以快速找到连续页，也难以检测重复释放和碎片分布。

```

page_alloc():
    if free_list.empty(): return null
    page = free_list.pop()
    page.refcount = 1
    return page

page_free(page):
    assert(page.refcount == 0)
    free_list.push(page)

```

bitmap 分配器用一个 bit 表示一个页框是否空闲。它节省元数据，容易扫描连续区域，但朴素扫描是 $O(n)$ 。可以通过分层 bitmap 或缓存上次搜索位置降低平均成本。

buddy allocator 用 2 的幂大小管理连续页块。分配 8 页时，寻找 order=3 的块；如果只有更大的块，就不断拆分；释放时，如果相邻 buddy 也空闲，就合并成更大块。buddy 的分配和释放大约是 $O(\log N)$ ，能较好处理连续页需求，但仍会产生内部碎片。

虚拟地址查找：VMA 和页表

页表回答“这一页当前映射到哪里”，VMA 回答“这一段地址理论上是否合法”。缺页时，内核先查 VMA：若地址不属于任何 VMA，就是非法访问；若属于可按需分配的匿名 VMA，就分配零页；若属于文件映射 VMA，就从 page cache 找页或发起 I/O。

VMA 可以用有序区间结构保存。查找地址属于哪个区间，朴素线性扫描是 $O(n)$ ；平衡树查找是 $O(\log n)$ ；现代内核还会使用更适合区间查询和缓存局部性的结构。教学模型可以先用有序数组，因为它能把语义讲清楚。

```

handle_page_fault(addr, access):
    vma = find_vma(addr)
    if vma == null: signal(SIGSEGV)
    if !vma.allows(access): signal(SIGSEGV)
    page = materialize_page(vma, addr)
    install_pte(addr, page, vma.permissions)

```

COW 的引用计数

COW 的不变量是：共享页必须只读，物理页有引用计数；写入共享页时，写入方得到新副本。若 refcount 为 1，可以直接把页改成可写；若 refcount 大于 1，必须复制。

```

handle_write_fault(vaddr):
    pte = lookup_pte(vaddr)
    page = pte.page
    if !pte.cow: signal(SIGSEGV)
    if page.refcount == 1:
        pte.writable = true
        pte.cow = false
    else:
        new_page = alloc_page()
        copy_page(new_page, page)
        page.refcount -= 1
        install_pte(vaddr, new_page, writable=true, cow=false)

```

这段算法最容易错在引用计数和权限更新顺序。若忘记把共享页设为只读，写入不会触发 fault，父子进程会互相污染；若复制后忘记减少旧页引用计数，会泄漏；若 TLB 中仍缓存旧权限，必须刷新或失效对应条目。

页面置换

当物理内存不足时，内核要回收页面。理想算法 OPT 总是淘汰未来最晚再访问的页面，但它需要知道未来，不能实现。LRU 淘汰最近最久未使用的页，更接近实际局部性，但精确 LRU 维护成本高。Clock 算法用访问位近似 LRU：扫描环形列表，访问位为 1 就清零并跳过，访问位为 0 就淘汰。

```

clock_evict():
    while true:
        page = clock_hand.page
        if page.referenced:
            page.referenced = false
            advance(clock_hand)
        else if page.dirty:
            schedule_writeback(page)
            advance(clock_hand)
        else:
            remove_and_return(page)

```

置换策略必须和文件系统、匿名页、swap、dirty writeback 结合。干净文件页可以直接丢弃，未来再从文件读；脏文件页要先写回；匿名页若要回收，需要 swap 或压缩；被锁定或正在 I/O 的页不能随便回收。

多级页表

64 位地址空间太大，不能为每个虚拟页都预留一个线性页表。多级页表把虚拟页号拆成多段索引，只为实际使用的区域分配中间页表页。以四级页表为例，虚拟地址被拆成 L4、L3、L2、L1 和页内 offset。

```

walk_page_table(root, vaddr, create):
    table = root
    for level in [L4, L3, L2]:
        index = index_for(level, vaddr)
        if table[index] not present:
            if not create:
                return null
            table[index] = allocate_page_table()
            table = table[index].next_table
    return &table[index_for(L1, vaddr)]

```

查找复杂度按层数是常数，但每层都可能带来内存访问。TLB 的价值就在于缓存最近翻译，避免每次访问都走多级页表。页表设计还影响内核内存占用：稀疏地址空间只分配用到的页表页，密集映射则消耗更多元数据。

TLB shutdown

改变页表后，CPU TLB 中可能仍缓存旧翻译。单核系统可以本地失效；多核系统中，同一地址空间可能正在其他 CPU 运行，需要发送 shutdown IPI 让它们失效对应条目。

```
unmap_page(mm, vaddr):
    pte = walk_page_table(mm.root, vaddr, false)
    old = clear_pte(pte)
    cpus = mm.active_cpus
    for cpu in cpus except current_cpu:
        send_tlb_shutdown(cpu, mm, vaddr)
    local_invlpg(vaddr)
    wait_for_shutdown_ack(cpus)
    put_page(old.page)
```

顺序很重要。若在其他 CPU 失效 TLB 前就释放物理页，该页可能被重新分配给别的对象，而旧 CPU 仍通过过期 TLB 写入它，形成严重内存破坏。TLB shutdown 是虚拟内存和多核同步的交叉点。

overcommit 与 OOM

虚拟内存允许系统承诺比物理内存更多的虚拟地址。例如 `mmap` 或 `malloc` 成功不代表物理页已经分配，首次写入才可能触发分配。overcommit 提高灵活性，但内存真正不足时必须选择失败策略。

```
handle_anon_fault(vma, addr):
    page = alloc_page()
    if page == null:
        victim = oom_select_process()
        if victim == null:
            return SIGBUS_OR_ENOMEM
        kill(victim)
        retry_allocation
    zero_page(page)
    install_pte(addr, page, WRITE | USER)
```

OOM killer 不是随意杀进程。它通常综合内存占用、权限、oom score、是否关键服务、cgroup 限制和可回收性。容器场景下，cgroup 内 OOM 应优先限制在该 cgroup 内，而不是扩大到整个宿主。

测试

测试要覆盖成功翻译、写只读页失败、访问未映射页失败、unmap 后访问失败、页内 offset 保留。错误结果必须携带可读诊断，不能只返回一个空值。

- `fork` 后父子共享只读 COW 页，任一方写入后得到私有副本。
- `mmap` 文件页首次访问触发填页，第二次访问走热路径。
- 栈增长必须受上限约束，不能无限扩张。
- `mprotect` 改变权限后，旧访问策略失效。
- `munmap` 后再次访问必须失败。
- 用户态指针传入系统调用时，内核必须检查可读/可写范围。

权限测试尤其重要。没有权限测试的虚拟内存模型，会把地址转换误写成普通哈希表查找，丢掉操作系统最重要的保护语义。

源码级补充：PTE 标志、THP、swap、mlock 与 PSI

页表项标志

x86-64 PTE 典型标志包括 `present`、`writable`、`user`、`accessed`、`dirty`、`global`、`NX` 等。操作系统把 VMA 权限、文件打开模式、COW 状态和硬件标志合成 PTE。注意 COW 不是单纯硬件位，通常由软件在只读 PTE 和附加元数据中表达。

标志	语义
<code>present</code>	PTE 可用于地址翻译，否则触发缺页
<code>writable</code>	<code>store</code> 是否允许
<code>user</code>	ring 3 是否可访问
<code>accessed</code>	CPU 访问后置位，用于回收近似 LRU
<code>dirty</code>	CPU 写入后置位，用于写回和 COW 判断
<code>NX</code>	禁止取指执行

权限收缩必须刷新 TLB。例如把页从可写改成只读，如果旧 TLB 仍保留可写权限，写入可能绕过新的 COW 或 `mprotect` 规则。

`mprotect`、`munmap` 和 VMA 切分

`mprotect` 改变区间权限，`munmap` 删除映射。二者都可能把一个 VMA 切成三段：前半保持原权限，中间修改或删除，后半保持原权限。

```
before:
  [0x1000, 0x9000) RW
mprotect [0x3000, 0x5000) R
after:
  [0x1000, 0x3000) RW
  [0x3000, 0x5000) R
  [0x5000, 0x9000) RW
```

VMA 切分后，要更新页表权限、失效 TLB、处理锁定页和文件映射引用。教学实现若只改一个 `flags` 字段，会在部分区间修改时出错。

Transparent Huge Page

普通页常为 4KiB，大页可为 2MiB 或更大。THP 尝试把连续匿名页合并成 PMD 级大页，减少 TLB miss。代价是分配连续内存更难，拆分和 COW 成本更高，延迟可能出现尖峰。

```
khugepaged:
  scan VMAs marked hugepage-friendly
  find 512 contiguous 4KiB pages
  allocate 2MiB huge page
  copy or remap contents
  replace PTE table with PMD huge mapping
```

THP 是性能优化，不应改变用户语义。发生 `mprotect`、`munmap`、COW 或内存压力时，大页可能被拆回普通页。性能报告要说明 THP 是否开启，否则 TLB 和内存延迟数据难以比较。

swap cache 和 swappiness

匿名页回收需要把内容写到 swap。swap slot 是匿名页在交换区的位置；swap cache 避免同一 swap 页被重复读写。swappiness 影响内核在文件页和匿名页之间的回收倾向。

```
swap_out(page):
  slot = allocate_swap_slot()
  write_page_to_swap(slot, page)
  replace_pte_with_swap_entry(vaddr, slot)
  free_physical_page(page)
```

```
swap_in(fault):
    slot = fault.swap_entry
    page = swap_cache_lookup_or_read(slot)
    install_pte(fault.addr, page)
```

swap 让系统在内存压力下继续运行，但延迟可能变得非常高。服务端系统常用 PSI、cgroup memory.high 和限流来避免进入严重 swap 抖动。

mlock 和不可回收页

mlock 把页锁在内存中，常用于实时路径、密码材料或避免 page fault 的场景。锁定页不能被普通回收换出，因此必须受权限和 rlimit 限制。

```
mlock(addr, len):
    check RLIMIT_MEMLOCK
    for each VMA page in range:
        fault page in if needed
        mark unevictable
```

滥用 mlock 会减少可回收内存，影响整机。教学 OS 要把“不可回收”作为页面状态加入回收测试，证明回收器不会错误丢弃 locked page。

mmap、page cache 与文件映射

原理

read 和 write 把文件内容显式复制到用户缓冲区，而 mmap 把文件区间映射到进程地址空间，让普通 load/store 触发文件页面的调入和回写。它看起来像内存功能，实际是虚拟内存、文件系统和 page cache 的共同契约：VMA 记录虚拟地址区间；page cache 记录文件偏移到物理页的缓存；缺页处理把两者连接起来；写回系统决定脏页何时落盘。

为什么需要 mmap？第一，它减少系统调用和复制，适合随机访问大文件。第二，它让多个进程共享同一份文件页，天然支持共享库和共享内存。第三，它把文件 I/O 纳入页表权限和缺页机制，统一了“按需加载”。但它也带来更复杂的一致性：一个进程通过 write 修改文件，另一个进程通过 mmap 读同一区间，看到的应是同一份 page cache 语义，而不是两个互相独立的缓存。

核心思想

mmap 不是把文件一次性读进内存，而是建立“虚拟地址 -> 文件偏移 -> page cache 页面”的延迟绑定。

MAP_SHARED 和 MAP_PRIVATE 是理解文件映射的分水岭。共享映射的写入会把 page cache 页面标脏，之后由 msync、fsync 或后台写回落盘；私有映射初始可共享文件页，但第一次写入触发 COW，产生匿名页，文件本身不被修改。共享库代码段通常是只读文件映射；进程堆和栈通常是匿名映射；数据库会谨慎选择 mmap 或 direct I/O，因为错误的写回假设会破坏延迟和持久化边界。

实践

一个教学 mmap 需要以下对象：

对象	保存什么	不变量
VMA	虚拟区间、权限、flags、file、offset	VMA 不重叠，权限不超过文件打开模式
page table entry	虚拟页到物理页、权限、dirty/accessed	PTE 权限不能超过 VMA 权限

page cache	inode、文件页索引、物理页、	同一 inode+index 对应一个缓存页
page	dirty、refcount	
anon page	私有 COW 后的页面	不再回写到文件
writeback item	脏页、目标块、提交状态	写回完成前页面不能被当成干净页回收

文件映射缺页路径如下：

```
handle_page_fault(addr, access):
    vma = find_vma(current.mm, addr)
    if vma == null or access not allowed by vma:
        send_sigsegv()
    page_index = vma.file_offset_pages + ((addr - vma.start) / PAGE_SIZE)
    page = page_cache_get_or_load(vma.file, page_index)
    if access == WRITE and vma.private:
        page = copy_page(page)
        mark_anon(page)
    install_pte(addr, page, pte_prot(vma, access))
```

`mmap` 实现中最重要的错误路径是长度、偏移和权限检查。映射长度为零、地址未对齐、偏移未按页对齐、写共享映射但 `fd` 没有写权限、文件系统不支持映射、区间越过用户地址空间，都必须明确返回错误。不要把所有失败都压成一个 `EINVAL`，否则测试和调试会失去边界信息。

算法与实现分析

VMA 查找

最简单的 VMA 管理可以用按起始地址排序的数组或链表，查找复杂度为 $O(n)$ 。这对教学模型足够，但进程有大量映射时会拖慢 page fault。真实内核会使用树结构，Linux 近年使用 `maple tree` 管理 VMA，以降低查找、插入和遍历成本。

```
find_vma(mm, addr):
    for vma in mm.vmas_by_start:
        if vma.start <= addr and addr < vma.end:
            return vma
        if addr < vma.start:
            return null
    return null
```

插入 VMA 时必须检查相邻区间是否可合并。若权限、`flags`、`file` 和连续 `offset` 一致，就可以合并，减少 VMA 数量。若频繁切分和合并出错，会导致 `munmap` 后仍然能访问、或合法区间被错误 `SIGSEGV`。

page cache 索引

page cache 的键通常是 `(inode, page_index)`。这个键保证 `read/write` 与 `mmap` 共享同一页：

```
page_cache_get_or_load(file, index):
    key = (file.inode, index)
    lock(page_cache.lock)
    page = radix_tree_lookup(page_cache, key)
    if page != null:
        page.refcount++
        unlock(page_cache.lock)
        return page
    page = allocate_page_locked()
    radix_tree_insert(page_cache, key, page)
    unlock(page_cache.lock)
    read_file_page(file, index, page)
```

```
unlock_page(page)
return page
```

这里先插入 locked page，再执行 I/O，是为了阻止多个线程同时读取同一文件页。后来的缺页者发现页面存在但 locked，就等待页面解锁。否则并发 page fault 会造成重复 I/O，甚至最后一个完成者覆盖前一个完成者的状态。

shared 与 private 写故障

写故障要区分三种情况：VMA 不允许写，私有映射需要 COW，共享映射需要让 PTE 可写并跟踪脏页。

```
handle_write_fault(vma, addr, old_page):
    if not vma.writable:
        send_sigsegv()
    if vma.private:
        new_page = allocate_page()
        copy(new_page, old_page)
        install_pte(addr, new_page, WRITE | USER)
        put_page(old_page)
        return
    mark_page_dirty(old_page)
    install_pte(addr, old_page, WRITE | USER | SHARED)
```

COW 的复杂度是一次页面复制 $O(\text{PAGE_SIZE})$ 。它用写时成本换取 fork/exec 和私有映射的低启动成本。共享映射的复杂性则在持久化：PTE dirty、page dirty、文件系统 journal dirty 和块设备队列不是同一件事。用户调用 msync(MS_SYNC) 期望脏页写回并等待完成，但这仍不必然等价于包含目录项的业务提交，后者需要文件系统和上层协议共同保证。

回收与写回

内存压力下，干净文件页可以直接回收，因为它们能从文件重新读入；脏文件页必须先写回；匿名页如果没有 swap，就不能随意丢弃。

```
reclaim_page(page):
    if page.refcount != 0 or page.locked:
        return BUSY
    if page.file_backed and not page.dirty:
        remove_from_page_cache(page)
        free(page)
        return OK
    if page.file_backed and page.dirty:
        enqueue_writeback(page)
        return RETRY_LATER
    if page.anon and swap_available:
        write_to_swap(page)
        free(page)
        return OK
    return BUSY
```

这个算法说明 page cache 不只是性能缓存，也是内存回收策略的一部分。错误地回收 dirty page 会丢数据；错误地保留所有文件页会让匿名内存被挤爆。

测试

1. 基本映射：映射文件第一页，读取字节应等于文件内容。
2. 懒加载：mmap 后不访问不应触发磁盘读，第一次访问才缺页。
3. 权限：只读 fd 不能建立可写 MAP_SHARED。
4. 私有写：MAP_PRIVATE 写入后文件内容不变。

5. 共享写：MAP_SHARED 写入后 read 同区间能看到 page cache 更新。
6. munmap：解除映射后访问同地址触发 SIGSEGV 或等价错误。
7. 截断：文件被 truncate 后访问越界映射要触发 SIGBUS 或教学系统定义的文件映射错误。
8. 回写：脏页经过 msync 后，模拟块设备能观察到写请求。

本章合格标准：能画出 VMA、PTE、page cache、inode 的关系；能解释 private/shared 映射的写故障差异；能说明 msync、fsync 和业务提交不是同一个边界。

源码级补充：do_mmap、XArray、writeback、0_DIRECT 与 DAX

do_mmap 到 mmap_region

Linux 的 mmap 路径会从系统调用参数进入 do_mmap，经过权限、地址选择、长度和 flags 校验后，调用 mmap_region 建立或合并 VMA。文件映射还会调用 file->f_op->mmap，让具体文件系统或设备设置 vm_ops。

```
sys_mmap
-> ksys_mmap_pgoff
-> vm_mmap_pgoff
-> do_mmap
-> get_unmapped_area
-> mmap_region
-> file->f_op->mmap
```

读源码时看 mm/mmap.c、mm/filemap.c。重点关注 vm_flags：VM_READ/WRITE/EXEC/SHARED/MAYWRITE/LOCKED/HUGEPAGE 等标志共同决定 fault、mprotect、mlock、回收和写回行为。

address_space 与 XArray

page cache 不挂在 fd 上，而挂在文件的 address_space 上。一个 inode 的 i_mapping 用 XArray 以页索引为 key 保存缓存页或 folio。这样不同进程、不同 fd、read/write/mmap 都能找到同一份文件页。

```
filemap_fault(vmf):
mapping = vmf.vma.file.f_mapping
index = offset_to_page_index(vmf.address)
folio = xa_load(mapping.i_pages, index)
if folio missing:
    folio = page_cache_sync_readahead(...)
    read_folio_from_filesystem(...)
install_pte(vmf.address, folio)
```

XArray 替代老的 radix tree，支持稀疏索引、并发查找和特殊 entry。读源码时不必先精通 XArray 实现，但要知道 page cache 的 key 是文件偏移，不是虚拟地址。

readahead 与访问模式

顺序读时，内核会预读后续页；随机读时，过度预读会浪费 I/O 和内存。应用可用 madvise 给出提示：MADV_SEQUENTIAL、MADV_RANDOM、MADV_DONTNEED、MADV_HUGEPAGE。

```
on_page_cache_miss(file, index):
if sequential_pattern_detected(file):
    submit_readahead(index + 1, window)
```

```
else:
    read_single_page(index)
```

预读失败案例：数据库随机读大文件，错误地被当成顺序流，page cache 被无用页污染；或者媒体顺序读取未触发预读，设备队列深度上不去。性能报告要注明访问模式和缓存状态。

writeback 与错误范围

脏页写回由 flusher 线程、内存压力、fsync/msync 等触发。Linux 用 writeback_control 描述写回目标，用 address_space 记录写回错误。一个 write 成功后发生的异步写回错误，可能在之后的 fsync 才报告。

```
fsync(file):
    filemap_fdatawrite(mapping)
    filemap_fdatawait(mapping)
    err = file_check_and_advance_wb_err(file)
    if err:
        return err
    return filesystem_fsync_metadata(file)
```

错误范围跟踪很重要。否则一个进程造成的写回错误可能被另一个不相关进程消费，或者永远没人知道。可靠程序必须检查 fsync 返回值。

O_DIRECT 和 DAX

O_DIRECT 尝试绕过 page cache，把用户页直接用于块 I/O。它通常要求地址、长度和文件偏移对齐设备或文件系统要求。绕过 page cache 不等于绕过一致性：若同一文件同时被 buffered I/O 和 direct I/O 访问，内核必须同步或失效相关缓存页。

DAX 面向持久内存，允许文件直接映射到持久介质地址，跳过 page cache。这样 load/store 可直接访问持久介质，但持久化顺序、cache flush 和失败原子性更需要应用和文件系统谨慎处理。

模式	优点	风险
buffered I/O	page cache、预读、合并、易用	双重缓存、fsync 边界易误解
O_DIRECT	减少缓存污染，数据库可控	对齐、短 I/O、混用一致性复杂
DAX	跳过 page cache，低延迟持久内存	cache flush、持久顺序、硬件依赖强

截断、SIGBUS 与映射失效

文件映射后，文件可能被另一个进程 truncate。若映射区间超过新的文件大小，访问超出部分通常触发 SIGBUS，因为虚拟地址属于 VMA，但背后的文件页不再存在。这和访问完全未映射地址的 SIGSEGV 不同。

```
file_truncate(inode, new_size):
    invalidate page cache pages beyond new_size
    unmap mappings beyond new_size or mark invalid
    wake waiters

fault_mapped_file(addr):
    if page_index beyond inode.size:
        signal(SIGBUS)
```

这个边界常被忽略。mmap 程序不能假设映射后文件大小永远不变，尤其是共享文件、日志文件和被管理进程替换的文件。

msync、fsync 和 fdatasync

msync 面向映射区间，把 dirty mapped pages 写回；**fsync** 面向文件，等待文件数据和必要元数据持久；**fdatasync** 通常只保证读取文件数据所需的元数据。它们重叠但不等价。

接口	语义重点
msync	映射页写回，可按地址范围控制
fsync	文件数据和必要元数据持久化
fdatasync	文件数据和读取必需元数据，不一定同步所有时间戳
syncfs	某挂载文件系统的脏数据同步

数据库和存储服务必须明确使用哪个边界。只调用 **msync** 某段映射，不一定同步目录项或业务 manifest；只调用 **fsync** 文件，不一定说明应用的多文件事务已提交。

userfaultfd 和用户态缺页处理

userfaultfd 允许用户态接管某些缺页事件，用于虚拟机迁移、按需加载和用户态分页。内核把 fault 事件交给用户态管理进程，后者填充页面并唤醒 faulting 线程。

```

fault on registered range:
  kernel queues userfault event
  faulting thread sleeps
  pager reads event
  pager copies page data into address
  pager wakes faulting thread

```

这个机制说明 page fault 不只属于内核内部，也可以成为用户态系统软件的接口。但它要求严格权限和资源控制，否则恶意 pager 可以让线程永久睡眠或制造内存压力。

系统调用与内存深讲：一次请求如何安全穿过边界

系统调用和内存管理是 OS 隔离能力的核心。系统调用控制“用户能请求什么”，页表控制“用户能访问什么”。二者常常一起出现：用户把指针传给系统调用，内核必须检查这段用户内存；缺页可能在系统调用复制参数时发生；**exec** 会替换地址空间并重建用户栈。

为什么用户指针不能直接解引用

用户传入的地址可能不存在、只读、跨页、被另一个线程并发修改，甚至在检查后被 **munmap**。内核若直接解引用，轻则崩溃，重则形成安全漏洞。因此内核要用专门的 copy 接口，并准备处理 copy 期间的 page fault。

```

sys_write(fd, user_buf, len):
  file = lookup_fd(fd)
  if file == null:
    return -EBADF
  if not file.can_write:
    return -EBADF
  kbuf = allocate_temp(min(len, MAX_RW_COUNT))
  ret = copy_from_user(kbuf, user_buf, len)
  if ret < 0:
    return -EFAULT
  return vfs_write(file, kbuf, len)

```

注意顺序：先查 fd 和写权限，再复制用户数据；复制失败不能留下半提交文件状态。对很大的写入，真实系统不会一次复制全部，而是分段处理；每段成功后返回值可能是短写，调用者必须循环。

page fault 的三种结论

同样是 page fault，内核结论可能完全不同。

结论	条件	处理
合法缺页	地址属于 VMA，权限允许，只是 PTE 不存在	分配页、读文件页或建立 COW 页
权限错误	地址属于 VMA，但访问类型不允许	发送 SIGSEGV 或返回 EFAULT
非法地址	不属于任何 VMA	发送 SIGSEGV 或返回 EFAULT

区别取决于上下文。用户态普通 load/store 错误通常给进程发信号；内核在 copy_from_user 中遇到用户地址错误，通常让系统调用返回 EFAULT。内核访问自己错误地址则是内核 bug，不能当普通用户错误处理。

fork、COW 与 exec 的组合

shell 启动程序常见流程是 fork 后在子进程中设置 fd，再 exec。若 fork 立刻复制整个地址空间，启动大进程会很慢。COW 让父子先共享只读页，只有写入时才复制。exec 成功后，子进程旧地址空间被替换，很多 COW 页根本不用复制。

```
parent shell:
    pid = fork()

child after fork:
    dup2(file, STDOUT)
    execve(program)

kernel optimization:
    fork shares pages as read-only COW
    exec discards old user mappings
    only pages written before exec are copied
```

这说明机制之间是组合出来的。fork 的语义简单，COW 让它便宜，exec 让它变成启动新程序，fd 继承让 shell 能做重定向。很多 OS 设计的美感来自小机制组合，而不是一个巨大的 spawn 黑盒。

内存回收为什么会影响系统调用延迟

系统调用可能看起来只是写 socket 或打开文件，却在中途分配内存。若内存不足，分配器可能触发回收；回收可能写回脏页；写回可能等待块设备；块设备完成再唤醒当前线程。于是一个普通请求的尾延迟可能来自内存和 I/O，而不是业务代码。

```
sys_send()
-> allocate socket buffer
-> memory pressure
-> reclaim pages
-> dirty page writeback
-> block I/O wait
-> scheduler wakeup
-> return to send path
```

这也是为什么真实性能分析要看 off-CPU 时间、page fault、writeback 和 block I/O。只看 CPU profiler 会漏掉大部分等待。

系统调用与内存练习

1. 为什么 `copy_from_user` 失败应该返回 `EFAULT`，而不是让内核 panic？
2. `fork` 后父子进程共享 COW 页，父进程写入一页后，页表和物理页引用计数怎样变化？
3. 为什么 `exec` 失败前应尽量保留旧地址空间可运行？
4. `mmap(MAP_PRIVATE)` 写入文件映射页后，文件内容为什么不变？
5. 为什么内存压力会让看似无关的系统调用变慢？

系统调用错误矩阵

高质量系统调用实现要能区分不同失败原因。把所有失败都返回一个通用错误，会让调用者无法恢复。

错误	典型原因	调用者策略
<code>EBADF</code>	fd 不存在或模式不允许	程序 bug 或句柄生命周期错误
<code>EFAULT</code>	用户指针不可访问	程序 bug，不能重试同一指针
<code>EINTR</code>	阻塞调用被信号打断	根据已完成字节数决定是否重试
<code>EAGAIN</code> <code>ENOMEM</code> <code>EIO</code>	非阻塞对象暂时不能推进 内存不足或限制触发 设备或底层 I/O 错误	等 readiness 后重试 降级、释放资源或失败返回 上报、切换副本、进入恢复流程
<code>EACCES</code>	权限不足	请求授权或拒绝操作

错误码本身就是 API 的一部分。比如 `EAGAIN` 表示状态可能稍后改变，`EFAULT` 表示调用者传了坏地址，二者的恢复策略完全不同。

页表权限和文件权限的区别

初学者容易混淆“我能打开文件”和“我能访问内存”。文件权限是文件系统对象上的访问控制；页表权限是 CPU 执行每次内存访问时检查的硬件权限。二者可以关联，但不是同一个东西。

```
open file read-only:
  file.can_write = false

mmap file private writable:
  VMA may allow write
  first write creates anonymous COW page
  file content is not modified

mmap file shared writable:
  fd must allow write
  dirty page belongs to file mapping
  msync/fsync affects persistence
```

这就是为什么 `MAP_PRIVATE` 可以让只读文件映射在进程内“可写”：写入的是私有匿名副本，不是文件本体。反过来，`MAP_SHARED` 可写映射必须受文件打开模式和文件系统权限限制。

本章小结

- 系统调用跨越信任边界，必须检查 fd、权限、用户指针和对象状态。
- page fault 不是单一错误；它可能是合法懒加载、权限错误或非法地址。

- fork、COW、exec、fd 继承共同组成高效进程启动模型。
- 内存压力可能把普通系统调用拖入回收和 I/O 等待。
- 错误码要表达恢复策略，而不是只表达“失败了”。

系统调用入口的生命周期

系统调用入口不是简单跳到一个 C 函数。CPU 从用户态进入内核后，内核要切换到受信任内核栈，保存用户寄存器，建立当前线程的系统调用上下文，验证系统调用号和参数，再调用具体实现。返回前还要处理信号、抢占、审计、ptrace/seccomp 等边界。

```
syscall_entry:
    switch_to_kernel_stack()
    save_user_registers()
    nr = user_registers.syscall_number
    if seccomp_denies(nr):
        return_error_or_kill()
    ret = syscall_table[nr](arg0, arg1, ...)
    if signal_pending(current):
        prepare_signal_frame_or_restart()
    if need_reschedule:
        schedule()
    restore_user_registers(ret)
    return_to_user()
```

这条路径解释了几个现象。系统调用可能被信号打断，也可能在返回用户态前被抢占；同一个系统调用在 tracing 或 seccomp 下成本不同；错误返回不是异常抛出，而是 ABI 规定的返回值和错误码。系统调用接口必须稳定，因为它是用户程序和内核之间的长期二进制契约。

用户指针的 TOCTOU 问题

即使内核先检查用户地址范围，再复制数据，也不能假设这段内存检查后不变。另一个线程可能修改 buffer、调用 munmap，或改变内存内容。正确模型是：检查只能证明“此刻看起来可访问”，真正复制必须能处理 fault；若需要稳定数据，内核要复制到内核缓冲区后再验证和使用。

```
bad_path(user_header):
    if validate_user_header(user_header):
        // user thread may change fields here
        use_header_fields_directly(user_header)

better_path(user_header):
    hdr = copy_from_user(user_header, sizeof(Header))
    if !validate_header(hdr):
        return -EINVAL
    use_private_copy(hdr)
```

不是所有用户数据都必须一次复制完。大 I/O 通常分段复制或使用 pinned pages、iov iterator、zero-copy 机制。但无论哪种方式，所有权和生命周期都要写清楚：谁能修改这段内存，设备或内核何时完成访问，失败时如何解除 pin 或引用。

VMA、PTE 和 page cache 的三层关系

很多虚拟内存错误来自混淆三层对象。VMA 是虚拟地址区间的策略；PTE 是某个虚拟页当前的硬件映射；page cache 是文件内容在内存中的缓存页。文件 mmap 首次访问时，VMA 说明这段地址来自哪个文件和偏移；page cache 提供或创建对应页；PTE 把该页映射进进程地址空间。

对象	回答的问题	典型变化
VMA	这段地址是否合法，权限和来源是什么	<code>mmap/munmap/mprotect</code> 改变
PTE	这一页现在是否可由硬件访问	缺页、换出、COW、 <code>unmap</code> 改变
page cache page	文件某偏移的内容是否在内存	<code>read</code> 、 <code>mmap fault</code> 、 <code>writeback</code> 、 <code>reclaim</code> 改变

因此，没有 PTE 不一定是错误；可能是合法 VMA 的懒加载。没有 VMA 通常才是非法地址。page cache 中有页，也不代表某进程已经有 PTE；多个进程可以通过不同 PTE 映射同一个 page cache 页。

缺页处理的完整分支

缺页处理要先分类，再处理。分类至少包括：地址是否在用户范围，是否属于 VMA，访问权限是否允许，PTE 是否存在但权限不符，是否 COW，是否文件页，是否栈增长，是否内核 copy 用户数据时发生。

```
handle_fault(mm, addr, access, context):
    if !is_user_range(addr):
        return fault_result(context, BAD_ADDRESS)
    vma = find_vma(mm, addr)
    if vma == null:
        return fault_result(context, NO_VMA)
    if is_stack_growth(vma, addr):
        expand_stack_or_fail(vma, addr)
    if !vma.allows(access):
        return fault_result(context, PERMISSION)
    pte = walk_pte(mm, addr, create=true)
    if pte.present and access.write and pte.cow:
        return handle_cow_fault(pte)
    if !pte.present:
        return materialize_page(vma, addr, access)
    return fault_result(context, PERMISSION)
```

`fault_result` 取决于上下文。用户态访问失败通常发信号；系统调用复制用户内存失败通常返回 `EFAULT`；内核访问内核坏地址通常 `panic`。把这些路径混成一个错误码，会掩盖边界。

页面回收的实际顺序

内存回收不是随机找页释放。回收器通常先尝试容易回收的页：干净文件页可以直接丢；脏文件页要写回；匿名页需要 swap 或压缩；被 `mlock`、DMA pin、正在 I/O 或引用计数异常的页不能回收。

```
reclaim_scan():
    for page in inactive_list:
        if page.pinned or page.under_writeback:
            skip(page)
        else if page.clean and page.file_backed:
            remove_from_page_cache(page)
            free(page)
        else if page.dirty and page.file_backed:
            start_writeback(page)
        else if page.anonymous and swap_available:
            write_to_swap(page)
        else:
            keep_or_escalate_pressure()
```

这个顺序解释了为什么内存压力会传导到存储 I/O。系统缺内存时，回收器可能把脏页写回设备；设备慢时，回收也慢；回收慢又会让分配路径阻塞，最终表现为系统调用尾延迟升高。

TLB、PCID 和 shutdown 成本

TLB 缓存虚拟地址翻译。切换地址空间时，若没有 x86-64 PCID 这样的标签，CPU 需要清空大量 TLB 项；有 PCID 时，可以保留不同地址空间的翻译，但页表修改仍要失效相关条目。多核下，其他 CPU 可能正在用同一地址空间运行，必须通过 IPI 通知它们失效。

操作	必要性	成本来源
local invalidate	当前 CPU 旧翻译不能继续用	pipeline/TLB 扰动
remote shutdown	其他 CPU 旧翻译不能继续用	IPI、等待 ack、跨核同步
batch unmap	合并多个失效请求	延迟释放与复杂生命周期
PCID	减少切换地址空间的全局 flush	标识管理、复用和一致性规则

这也是为什么高频 `mmap/munmap/mprotect` 可能拖慢多线程程序。它们不是只改一张表，还可能引发跨 CPU 协议。优化思路通常是批量映射、复用区域、减少权限翻转和避免在热路径频繁改变页表。

分配器层次：页、slab 和用户堆

内核内存分配有层次。页分配器管理物理页框；slab/slub 等对象分配器管理固定大小内核对象；用户态 `malloc` 在进程地址空间内管理堆，并通过 `brk/mmap` 向内核申请更大区域。

分配层	管理对象	典型问题
page allocator	物理页和连续页块	外部碎片、NUMA、本地性
slab allocator	inode、task、dentry 等固定对象	对象复用、构造析构、缓存热度
vm allocator	内核虚拟连续区域	页表开销、TLB、地址空间碎片
user malloc	用户堆小对象	arena、碎片、RSS 与虚拟大小差异

理解层次能避免错误诊断。用户进程 RSS 增长不一定是内核泄漏；slab 增长可能来自 dentry/inode cache；虚拟地址空间变大不一定占用物理页；页分配失败可能是连续块碎片而不是总内存为零。

exec 的失败原子性

`execve` 成功后旧程序消失；失败时通常应让调用进程还能继续运行并看到错误码。这要求内核在替换地址空间时谨慎排序。若过早销毁旧地址空间，然后发现 ELF 无效或无法分配栈，就可能让进程处于不可恢复半状态。

```
execve(path, argv, envp):
    file = open_executable(path)
    image = validate_elf(file)
    new_mm = build_new_address_space(image, argv, envp)
    if any_step_failed:
        destroy_partial_new_mm()
    return -errno
```



```
old_mm = current.mm
current.mm = new_mm
switch_page_table(new_mm)
destroy_old_mm(old_mm)
start_at_entry(image.entry)
```

真实实现还要处理解释器脚本、动态链接器、setuid、安全标签、fd 的 `CLOEXEC`、信号处理器重置和多线程 `exec` 语义。核心原则仍然是：新镜像完全准备好之前，不要破坏失败后必须保留的旧执行环境。

内存与系统调用测试矩阵

测试	触发方式	期望
坏用户读指针	<code>write(fd, bad, len)</code>	返回 <code>EFAULT</code> ，内核不崩溃
坏用户写指针	<code>read(fd, bad, len)</code>	返回 <code>EFAULT</code> 或短读语义清楚
COW 写入	fork 后父子分别写同页	内容分离，引用计数正确
权限缺页	写只读映射或执行 NX 页	信号或错误，不能绕过权限
munmap 竞态	复制期间另线程 unmap	返回错误或短完成，不 <code>use-after-free</code>
TLB 失效	mprotect 后旧权限访问	旧权限不能继续生效
内存压力	限制内存后触发分配	有 OOM/ENOMEM 证据和恢复路径

这些测试共同证明：边界错误不会变成内核错误，权限变化能真正影响硬件访问，资源不足有可解释失败，部分完成语义被调用者看见。

3.4 内核分配器与 C++20 内存模型

内核内存分配器

原理

虚拟内存章节讲页表和地址空间，本章看内核自己怎样管理内存。内核需要分配页表页、进程结构、文件对象、socket buffer、目录项缓存、驱动 DMA buffer、日志记录和临时缓冲区。这些对象大小不同、生命周期不同、对齐要求不同，有些可以睡眠等待内存，有些在中断上下文不能睡眠。

内核分配器通常分层。最底层管理物理页框，例如 `buddy allocator`；上层把页切成固定大小对象，例如 `slab/slub`；再上层可能有专用缓存，例如 `task struct cache`、`inode cache`、`dentry cache`。分层的原因是单一分配器无法同时高效处理连续页、大量小对象、构造析构、缓存局部性和调试需求。

实践

实践先写对象分类表：

对象	分配来源	约束
页表页	buddy order-0 页	页对齐，通常不可移动
DMA buffer	连续物理页或 IOMMU 映射	设备可访问，对齐和边界限制

task 结构	slab cache	高频分配释放，需要初始化
socket buffer	slab + page fragment	网络吞吐和生命周期复杂
日志缓冲	kmalloc 或专用池	失败路径也要可用

然后为每类对象写：是否可睡眠、是否可回收、是否可移动、是否需要清零、是否可能从中断上下文分配。

算法与实现分析

buddy allocator

buddy allocator 把空闲内存按 order 管理，order=k 表示 2^k 个连续页。分配时找最小足够 order；没有就拆更大的块。释放时尝试和 buddy 合并。

```
alloc(order):
    for k in order..MAX_ORDER:
        if free[k] not empty:
            block = free[k].pop()
            while k > order:
                k -= 1
                buddy = split(block, k)
                free[k].push(buddy)
            return block
    return null

free(block, order):
    while order < MAX_ORDER:
        buddy = find_buddy(block, order)
        if buddy not free: break
        remove buddy from free[order]
        block = merge(block, buddy)
        order += 1
    free[order].push(block)
```

buddy 的优势是能管理连续页，释放时能合并。问题是内部碎片：请求 9 页必须分配 16 页。长期运行后，高 order 连续块也可能稀缺。

slab cache

slab 分配器为固定大小对象建立缓存。一个 slab 通常由一页或多页组成，切成多个对象槽。分配就是从 freelist 取一个槽；释放就是放回 freelist。对象构造可以在 slab 创建时完成，减少重复初始化成本。

```
slab_alloc(cache):
    if cache.partial not empty:
        obj = pop_free_object(cache.partial.first)
        return obj
    slab = allocate_new_slab(cache)
    cache.partial.push(slab)
    return pop_free_object(slab)
```

slab 的分配释放接近 $O(1)$ 。它适合大量同类型对象，例如 inode、dentry、task。实现要处理 per-CPU cache，以减少多核锁竞争；还要处理调试模式下的红区、poison、double free 检测。

kmalloc 和大小类别

通用 kmalloc 通常把请求大小映射到一组 size class，例如 32、64、128、256 字节。每个 size class 背后是一个 slab cache。这样可以用固定对象缓存服务变长请求。

```
kmalloc(size):
```

```
class = size_to_class(size)
return slab_alloc(kmalloc_cache[class])
```

size class 的代价是内部碎片。请求 129 字节可能分到 256 字节槽。分配器设计要在碎片、速度和缓存局部性之间折中。

GFP 标志和上下文

内核分配 API 通常带标志，说明是否允许睡眠、是否允许 I/O 回收、是否要求 DMA 区域等。中断上下文不能睡眠，所以只能使用不会阻塞的分配；普通进程上下文可以等待回收。

```
if in_interrupt():
    page = alloc_page(GFP_ATOMIC)
else:
    page = alloc_page(GFP_KERNEL)
```

这解释了为什么内核代码不能随便分配内存。分配失败不是理论问题，尤其在中断、持锁、内存压力和回收路径中。高质量代码必须定义失败策略：返回错误、丢包、使用预留池、降级还是 panic。

per-CPU 缓存

多核系统中，所有小对象分配都争同一把 slab 锁会很慢。per-CPU cache 让每个 CPU 维护少量本地空闲对象，常见分配释放不需要全局锁。

```
kmem_cache_alloc(cache):
    cpu_cache = cache->percpu[current_cpu()]
    if cpu_cache->freelist not empty:
        return cpu_cache->freelist.pop()
    refill_from_global(cache, cpu_cache)
    return cpu_cache->freelist.pop()

kmem_cache_free(cache, obj):
    cpu_cache = cache->percpu[current_cpu()]
    cpu_cache->freelist.push(obj)
    if cpu_cache->count > HIGH_WATERMARK:
        drain_to_global(cache, cpu_cache)
```

per-CPU 缓存降低锁竞争，但会增加内存滞留：每个 CPU 都可能囤积一批对象。高质量实现要有高低水位、CPU 下线 drain、内存压力回收和统计对账。

内存调试与防御

分配器是许多内核漏洞的放大器。越界写、use-after-free、double free 和未初始化读取都可能破坏任意内核对象。调试构建应加入红区、poison、隔离队列和调用栈记录。

```
debug_free(obj):
    if obj->magic != LIVE_MAGIC:
        panic("double free or corrupted object")
    fill(obj->memory, POISON_FREE)
    obj->magic = FREE_MAGIC
    quarantine_push(obj)

debug_alloc(cache):
    obj = real_alloc(cache)
    fill(obj->memory, POISON_ALLOC)
    obj->magic = LIVE_MAGIC
    return obj
```

隔离队列会延迟复用刚释放对象，使 use-after-free 更容易在测试中暴露。代价是内存占用和性能下降，所以通常只在调试或安全强化配置中开启。

测试

- buddy 分配释放后，空闲页总数守恒。
- 分裂和合并后，不存在重叠空闲块。
- slab 分配对象满足对齐，释放后能复用。
- double free、越界写和 use-after-free 在调试模式能被检测。
- GFP_ATOMIC 路径不能睡眠，失败时有明确降级策略。
- 内存压力下，分配器统计能解释碎片和失败原因。

源码级补充：buddy、SLUB、GFP、vmalloc 与 kmemleak

buddy 的 free_area 和迁移类型

Linux buddy 不只是按 order 分链表，还按迁移类型分组，例如 movable、unmovable、reclaimable、CMA 等。这样做是为了降低长期碎片：可移动页尽量放在一起，未来需要高 order 连续页时更容易通过 compaction 得到。

```
struct free_area {
    list_head free_list[MIGRATE_TYPES];
    unsigned long nr_free;
};
```

读 mm/page_alloc.c 时重点看 __rmqueue、__free_pages_ok、watermark、zone 和 migratetype。一次 order-9 分配失败，不一定表示系统没有 512 页空闲，可能是没有连续的 512 页。

SLAB、SLUB、SLOB 的取舍

SLAB 强调对象缓存和复杂队列；SLUB 简化元数据，成为常见默认；SLOB 面向极小系统，结构简单但伸缩性弱。现代主线常见是 SLUB。读 mm/slub.c 时看 kmem_cache、per-CPU freelist、partial slab、debug red zone 和 poison。

```
kmem_cache_alloc(cache):
    if current_cpu.freelist:
        return pop(current_cpu.freelist)
    if cache.node.partial:
        slab = take_partial(cache.node)
        load_cpu_freelist(slab)
        return pop(current_cpu.freelist)
    slab = new_slab_from_buddy()
    return first_object(slab)
```

SLUB 的快路径很短，但 debug 选项会显著增加检查成本。教材中讲“分配是 O(1)”时，要同时说明这是平均快路径；慢路径可能进入 buddy、回收、compaction 或 OOM。

kmalloc size class 与内部碎片

kmalloc(130) 可能进入 192 或 256 字节 size class，取决于架构和配置。向上取整带来内部碎片，但换来固定大小缓存的速度和对齐。对象若频繁分配，最好建立专用 kmem_cache，可以提供构造函数、调试名和更好统计。

GFP_NOIO、GFP_NOFS 和回收递归

GFP_KERNEL 允许睡眠和回收；GFP_ATOMIC 用于中断或持自旋锁路径；GFP_NOIO 禁止通过 I/O 回收；GFP_NOFS 禁止进入文件系统回收路径。后两者用于避免递归死锁。

```
filesystem_write_path():
    lock(inode)
```

```
buf = kmalloc(size, GFP_NOFS)
// avoid reclaim entering filesystem and taking inode locks again
```

若文件系统持有 inode 锁时用 GFP_KERNEL 分配，内存压力下回收可能进入同一文件系统并等待 inode 锁，形成递归死锁。GFP flag 是锁和回收的边界，不是性能装饰。

vmalloc 与物理不连续

kmalloc 通常返回物理连续内存，适合 DMA 或小对象；vmalloc 返回虚拟连续但物理页可不连续的内存，适合大缓冲或模块映射。代价是需要建立页表，TLB 局部性更差，不能直接给不支持 scatter-gather 的设备 DMA。

```
vmalloc(size):
    area = allocate_virtual_area(size)
    pages = alloc_individual_pages(npages)
    map_pages(area, pages)
    return area.addr
```

kmemleak、KASAN 和 KFENCE

内存泄漏和 use-after-free 很难靠肉眼找。kmemleak 通过扫描可达指针寻找未释放对象；KASAN 使用 shadow memory 检测越界和释放后访问；KFENCE 用低频保护页检测堆错误，开销比 KASAN 低但覆盖率也低。

工具	能发现	代价
kmemleak	长期未释放对象	扫描和误报，需要调试配置
KASAN	越界、use-after-free	内存和性能开销高
KFENCE	采样式堆越界和 UAF	开销低，非确定覆盖

内存碎片、compaction 和 CMA

内存碎片分为内部碎片和外部碎片。内部碎片来自 size class 向上取整；外部碎片来自空闲页分散，无法满足高 order 连续页。compaction 通过迁移可移动页，把空闲页聚合成连续块。CMA 预留可迁移区域，给需要连续物理内存的设备使用。

```
compact_zone(zone):
    free_scanner = end_of_zone
    migrate_scanner = start_of_zone
    while scanners_not_met:
        page = find_movable_page(migrate_scanner)
        free = find_free_page(free_scanner)
        migrate(page, free)
```

并非所有页都能迁移。页表页、驱动 pin 住的 DMA 页、mlock 页和某些内核对象不可移动。高 order 分配失败时，诊断要看总空闲页、最大连续块、不可移动页比例和 compaction 失败原因。

内存控制组和分配归属

cgroup v2 可以限制一组进程的内存。分配器需要把许多用户可归因的内存记到 memcg，例如 page cache、匿名页、部分 slab 对象。这样容器内泄漏不会无限吃掉宿主内存。

```
charge_memcg(page, current.memcg):
    if memcg.current + page.size > memcg.max:
        reclaim_memcg(memcg)
        if still over limit:
            trigger_memcg_oom(memcg)
    memcg.current += page.size
```

memcg OOM 和全局 OOM 要区分。前者应限制在 cgroup 内，后者说明整机资源不足。测试容器内存限制时，要观察 `memory.current`、`memory.events` 和 OOM victim。

对象生命周期对账

内核内存质量不只看分配成功，还要能证明对象生命周期闭合。每类对象应有 alloc/free 计数、当前活跃数、高水位和失败次数。对复杂对象，还应记录 owner 和引用来源。

```
object_accounting:
    alloc_count++
    active_count++
    high_watermark = max(high_watermark, active_count)

free:
    active_count--
    free_count++
```

测试结束时 active count 不为零，就是泄漏线索；free count 大于 alloc count，说明 double free 或计数损坏；高水位异常说明负载下对象滞留。生产系统不一定记录所有栈，但调试构建应支持按对象类型打开详细追踪。

第三阶段 C++20 模型：从 VMA 到 PTE 的可测内存管理器

虚拟内存不能只靠文字理解。读者至少要实现一个小模型，能把 VMA、PTE、物理页、COW 和缺页处理连起来。这个模型不追求模拟所有硬件细节，而是追求状态可检查：非法地址不能访问，权限收缩必须生效，COW 写入必须复制，引用计数必须闭合。

模型边界

教学模型保留四类对象。

对象	责任	不做什么
<code>PhysicalMemory</code>	分配页框、维护引用计数、复制页	不模拟 NUMA、cache、DMA
<code>AddressSpace</code>	管理 VMA 和页表	不模拟多级页表的每级页表页
<code>PageTableEntry</code>	保存页框、权限、COW 标志	不模拟 accessed/dirty 的硬件置位细节
<code>FaultHandler</code>	按 VMA/PTE/访问类型处理缺页	不模拟真实信号栈和架构异常帧

先省略一部分硬件细节，是为了让核心不变量清晰。等这些不变量稳定后，再加入 accessed/dirty、swap、file mapping、TLB shutdown 和 memcg，才不会把系统写成一团不可测代码。

类型和常量

先用强类型别名和命名常量写清地址单位。所有有语义的数字都要命名。

```
#include <algorithm>
#include <cstdint>
#include <deque>
#include <limits>
#include <stdexcept>
#include <unordered_map>
#include <utility>
#include <vector>
```



```

using VirtualAddress = std::uint64_t;
using PageNumber = std::uint64_t;
using FrameNumber = std::uint64_t;

constexpr std::uint64_t VM_PAGE_SIZE = 4096;
constexpr VirtualAddress VM_USER_TOP = 0x0000800000000000ULL;
constexpr FrameNumber VM_INVALID_FRAME =
    std::numeric_limits<FrameNumber>::max();

enum class AccessKind {
    Read,
    Write,
    Execute,
};

```

地址计算要集中放在小函数里，避免到处手写除法和取模。

```

constexpr PageNumber page_number(VirtualAddress address) {
    return address / VM_PAGE_SIZE;
}

constexpr std::uint64_t page_offset(VirtualAddress address) {
    return address % VM_PAGE_SIZE;
}

constexpr VirtualAddress page_down(VirtualAddress address) {
    return address - page_offset(address);
}

constexpr VirtualAddress page_up(VirtualAddress address) {
    const auto offset = page_offset(address);
    if (offset == 0) {
        return address;
    }
    return address + (VM_PAGE_SIZE - offset);
}

```

权限和 VMA

VMA 是区间策略，不是当前硬件映射。它说明地址是否属于进程、理论上允许什么访问、缺页时应该怎样补页。

```

struct VmPermissions {
    bool readable = false;
    bool writable = false;
    bool executable = false;
};

bool allows(const VmPermissions& permissions, AccessKind access) {
    switch (access) {
        case AccessKind::Read:
            return permissions.readable;
        case AccessKind::Write:
            return permissions.writable;
        case AccessKind::Execute:
            return permissions.executable;
    }
    throw std::logic_error("unknown access kind");
}

enum class VmaKind {

```

```

    Anonymous,
    FilePrivate,
    FileShared,
};

struct Vma {
    VirtualAddress start = 0;
    VirtualAddress end = 0;
    VmPermissions permissions;
    VmaKind kind = VmaKind::Anonymous;
    std::uint64_t file_offset = 0;
};

```

VMA 插入时必须检查页对齐、非空区间、用户地址范围和不重叠。教学模型可用有序 vector；查找是线性扫描，语义最直观。

```

class VmaTable {
public:
    void insert(Vma vma) {
        this->validate_new_vma(vma);
        auto position = std::ranges::lower_bound(
            this->vmass_, vma.start, {}, &Vma::start);
        this->vmass_.insert(position, std::move(vma));
    }

    [[nodiscard]] const Vma* find(VirtualAddress address) const {
        for (const auto& vma : this->vmass_) {
            if (address < vma.start) {
                return nullptr;
            }
            if (vma.start <= address && address < vma.end) {
                return &vma;
            }
        }
        return nullptr;
    }

private:
    void validate_new_vma(const Vma& vma) const {
        if (vma.start >= vma.end) {
            throw std::invalid_argument("empty VMA");
        }
        if (page_offset(vma.start) != 0 || page_offset(vma.end) != 0) {
            throw std::invalid_argument("VMA must be page aligned");
        }
        if (vma.end > VM_USER_TOP) {
            throw std::invalid_argument("VMA outside user range");
        }
        for (const auto& existing : this->vmass_) {
            const bool disjoint = vma.end <= existing.start || existing.end <= vma.start;
            if (!disjoint) {
                throw std::invalid_argument("overlapping VMA");
            }
        }
    }

    std::vector<Vma> vmass_;
};

```

这段代码故意没有实现自动合并。教学阶段先让每次插入的边界完全显式；等测试覆盖充分后，再把相邻 VMA 合并作为优化加入。

物理页和引用计数

物理内存模型用 frame number 标识页框，页内容用固定大小数组表示。为了节省书中篇幅，可以把内容简化为 byte vector；真实实现中应避免在热路径频繁分配。

```
struct PhysicalPage {
    std::vector<std::uint8_t> bytes = std::vector<std::uint8_t>(VM_PAGE_SIZE);
    std::uint32_t refcount = 0;
};

class PhysicalMemory {
public:
    FrameNumber allocate_zeroed() {
        FrameNumber frame = VM_INVALID_FRAME;
        if (!this->free_list_.empty()) {
            frame = this->free_list_.front();
            this->free_list_.pop_front();
            auto& page = this->pages_.at(frame);
            std::ranges::fill(page.bytes, 0);
            page.refcount = 1;
            return frame;
        }

        frame = static_cast<FrameNumber>(this->pages_.size());
        PhysicalPage page;
        page.refcount = 1;
        this->pages_.push_back(std::move(page));
        return frame;
    }

    FrameNumber clone(FrameNumber source) {
        const FrameNumber target = this->allocate_zeroed();
        this->pages_.at(target).bytes = this->pages_.at(source).bytes;
        return target;
    }

    void retain(FrameNumber frame) {
        this->pages_.at(frame).refcount += 1;
    }

    void release(FrameNumber frame) {
        auto& page = this->pages_.at(frame);
        if (page.refcount == 0) {
            throw std::logic_error("double release of physical page");
        }
        page.refcount -= 1;
        if (page.refcount == 0) {
            this->free_list_.push_back(frame);
        }
    }

    [[nodiscard]] std::uint32_t refcount(FrameNumber frame) const {
        return this->pages_.at(frame).refcount;
    }

private:
    std::vector<PhysicalPage> pages_;
};
```

```
std::deque<FrameNumber> free_list_;
};
```

引用计数是 COW 的核心证据。每个测试结束时，物理页活跃数、引用计数和页表映射数要能对上，否则模型可能泄漏或重复释放。

PTE 和地址空间

PTE 保存当前硬件可见映射。VMA 允许写不等于 PTE 当前可写；COW 页的 VMA 允许写，但 PTE 要暂时只读，让第一次写触发 fault。

```
struct PageTableEntry {
    FrameNumber frame = VM_INVALID_FRAME;
    VmPermissions permissions;
    bool present = false;
    bool cow = false;
};

class AddressSpace {
public:
    void map_vma(Vma vma) {
        this->vmass_.insert(vma);
    }

    [[nodiscard]] const Vma* find_vma(VirtualAddress address) const {
        return this->vmass_.find(address);
    }

    [[nodiscard]] PageTableEntry* find_pte(VirtualAddress address) {
        const auto iterator = this->page_table_.find(page_number(address));
        if (iterator == this->page_table_.end()) {
            return nullptr;
        }
        return &iterator->second;
    }

    void install_pte(VirtualAddress address, PageTableEntry pte) {
        this->page_table_[page_number(address)] = pte;
    }

private:
    VmaTable vmass_;
    std::unordered_map<PageNumber, PageTableEntry> page_table_;
};
```

这个页表是哈希表，不是多级页表。它足以表达“某个虚拟页当前是否 present、权限是什么、是否 COW”。后续实践卷再把它替换成四级页表或模拟 TLB。

缺页处理

缺页处理先查 VMA，再查权限，再决定是新分配、COW 还是权限错误。返回结果要区分用户可见错误。

```
enum class FaultResult {
    Resolved,
    SegmentationFault,
    ProtectionFault,
};

class FaultHandler {
public:
    explicit FaultHandler(PhysicalMemory& memory) : memory_(memory) {}
```

```

FaultResult handle(AddressSpace& space, VirtualAddress address, AccessKind
access) {
    const Vma* vma = space.find_vma(address);
    if (vma == nullptr) {
        return FaultResult::SegmentationFault;
    }
    if (!allows(vma->permissions, access)) {
        return FaultResult::ProtectionFault;
    }

    PageTableEntry* pte = space.find_pte(address);
    if (pte != nullptr && pte->present) {
        return this->handle_present_pte(*pte, access);
    }

    PageTableEntry new_pte;
    new_pte.frame = this->memory_.allocate_zeroed();
    new_pte.present = true;
    new_pte.permissions = vma->permissions;
    space.install_pte(address, new_pte);
    return FaultResult::Resolved;
}

private:
FaultResult handle_present_pte(PageTableEntry& pte, AccessKind access) {
    if (allows(pte.permissions, access)) {
        return FaultResult::Resolved;
    }
    if (access == AccessKind::Write && pte.cow) {
        return this->resolve_cow(pte);
    }
    return FaultResult::ProtectionFault;
}

FaultResult resolve_cow(PageTableEntry& pte) {
    if (this->memory_.refcount(pte.frame) == 1) {
        pte.permissions.writable = true;
        pte.cow = false;
        return FaultResult::Resolved;
    }

    const FrameNumber old_frame = pte.frame;
    const FrameNumber new_frame = this->memory_.clone(old_frame);
    this->memory_.release(old_frame);
    pte.frame = new_frame;
    pte.permissions.writable = true;
    pte.cow = false;
    return FaultResult::Resolved;
}

PhysicalMemory& memory_;
};

```

这段实现有一个清晰的不变量：COW 处理前，PTE 不可写；处理后，当前地址空间获得可写私有页；旧 frame 引用计数减少。若 refcount 为 1，不复制，只把权限改回可写。

fork 的 COW 准备

fork 时复制页表，但共享物理页，把双方 PTE 改为只读 COW。VMA 不需要复制物理页，只要复制区间策略。

```

fork address space:
  copy VMA table
  for each present writable anonymous PTE:
    clear writable in parent PTE
    set cow in parent PTE
    create child PTE pointing to same frame
    clear writable in child PTE
    set cow in child PTE
    retain physical frame

```

教学代码实现 fork 时要特别注意父进程 PTE 也要改成只读。如果只改子进程，父进程写入不会 fault，会直接污染共享页。真实系统还要 flush 父进程对应 TLB，否则父 CPU 可能继续使用旧 writable 权限。

mprotect 的模型

mprotect 至少要改 VMA 权限和已有 PTE 权限。教学模型可以先拒绝跨 VMA 的复杂修改，只允许整段 VMA 修改；实践卷再实现切分。

```

mprotect simplified:
  find exact VMA [start, end)
  verify new permissions do not exceed may permissions
  update VMA permissions
  for each PTE in range:
    pte.permissions = intersect(pte.permissions, new_permissions)
    if new permissions remove write:
      record need_tlb_flush

```

注意“扩大权限”不能只改 PTE，还要检查原始 may 权限；“缩小权限”不能只改 VMA，还要改已经 present 的 PTE。否则已有映射会继续按旧权限访问。

测试用例

这个模型至少要有以下测试。

测试	步骤	期望
未映射地址	不建 VMA，访问地址	返回段错误结果
懒分配	建匿名 VMA，首次读	分配零页并 resolved
写只读 VMA	VMA 只读，执行写 fault	返回权限错误结果
COW refcount>1	父子共享页，子写	子获得新 frame，旧 frame refcount 减一
COW refcount=1	单独持有 COW 页后写	不复制，只恢复 writable
mprotect 收缩	RW 改 R 后写	写失败，旧权限不能生效
double release	对同一 frame release 两次	抛出逻辑错误或测试失败

测试要断言内部状态，而不是只断言返回值。例如 COW 测试必须检查父子 frame 是否不同、内容是否正确、旧 frame 引用计数是否符合预期。只有这样才能发现“看似成功但泄漏引用”的错误。

从教学模型到真实内核

这个模型和真实内核差距很大，但差距是有方向的。

- 把哈希页表替换为多级页表，可学习页表页分配和页表遍历。
- 给 PTE 加 accessed/dirty，可学习回收和写回。
- 给 VMA 加文件对象和 offset，可学习 mmap 与 page cache。

- 给地址空间加 active CPU 集合，可学习 TLB shutdown。
- 给物理页加 memcg、zone、order 和 migratetype，可学习 buddy、NUMA、compaction。
- 给 fault handler 加可睡眠点和 retry，可学习真实缺页路径的并发。

不要一开始就实现所有功能。先让最小模型通过严格测试，再逐步加入复杂字段。操作系统工程的质量来自可维护的不变量，而不是一次性写出大量不可验证的机制。

第 4 章 设备、I/O 与文件系统

4.1 驱动、DMA 与设备队列

设备驱动、中断下半部与 DMA

原理

设备驱动把硬件寄存器、中断、DMA 和内核对象连接起来。应用看到的是 fd、socket、块设备或帧缓冲；驱动看到的是 MMIO 寄存器、描述符环、设备状态位、中断线、DMA 地址和错误码。驱动的难点在于硬件和 CPU 并发运行：CPU 写描述符，设备异步读取；设备写内存，CPU 稍后读取；中断可能在任意时刻到来。

最简单设备可以用 programmed I/O：CPU 直接读写设备寄存器或数据端口。PIO 易懂，但吞吐低，CPU 被数据搬运占住。DMA 让设备直接读写内存，CPU 只准备描述符和缓冲区。DMA 提高吞吐，也引入缓存一致性、地址映射、内存屏障和 buffer 生命周期问题。

驱动通常分成上半部和下半部。中断上半部快速确认设备、读取状态、屏蔽或清除中断，把复杂工作推迟到底半部、工作队列、软中断或普通内核线程。这样可以减少中断关闭时间，避免在硬中断上下文执行可能睡眠的操作。

实践

先画一个网卡或磁盘的描述符环：

```
descriptor ring:
  head: device consumes descriptors
  tail: driver produces descriptors
  desc[i]:
    dma_address
    length
    flags
    status
```

驱动发送路径通常是：从内核分配 buffer，映射成设备可见 DMA 地址，填描述符，执行内存屏障，更新 tail，通知设备。完成路径通常是：设备 DMA 完成并写 status，触发中断，驱动回收描述符，解除 DMA 映射，释放 buffer，唤醒等待者或上报 completion。

算法与实现分析

MMIO 寄存器访问

MMIO 把设备寄存器映射到物理地址。访问 MMIO 不能当成普通内存优化：编译器不能随便重排，CPU 也要遵守设备要求的顺序。驱动通常使用专用 readl/writel 一类接口。

```
device_start(dev):
  writel(dev->mmio + CTRL, CTRL_RESET)
  wait_until(readl(dev->mmio + STATUS) & READY)
  writel(dev->mmio + IRQ_MASK, RX_IRQ | TX_IRQ)
```

轮询 wait 要有超时。设备可能坏掉，寄存器可能永远不变。驱动若在内核里无限等待，会拖死系统。

描述符环

描述符环是设备驱动的核心队列。生产者和消费者可以是 CPU 和设备，也可以反过来。环满时不能继续提交，环空时不能继续回收。

```
submit_tx(desc_ring, packet):
    next = (tail + 1) % ring_size
    if next == head:
        return -EAGAIN
    desc[tail].addr = dma_map(packet.data)
    desc[tail].len = packet.len
    desc[tail].flags = OWNED_BY_DEVICE
    memory_barrier()
    tail = next
    writel(DOORBELL, tail)
```

这里的屏障非常关键。CPU 必须先把描述符内容写完，再更新 tail 或 doorbell 通知设备；否则设备可能看到半初始化描述符。

DMA 和缓存一致性

若 CPU cache 和设备 DMA 不自动一致，驱动必须在设备读内存前清理 cache，在设备写内存后失效 cache。即使硬件支持 IOMMU，也要建立设备可访问地址映射。

```
prepare_dma_to_device(buf):
    cache_clean(buf)
    dma_addr = iommu_map(buf.phys)
    return dma_addr

finish_dma_from_device(buf):
    cache_invalidate(buf)
    parse_received_data(buf)
```

DMA buffer 的生命周期必须覆盖设备访问全过程。提交后不能释放或复用 buffer，直到 completion 表明设备不再访问它。这个规则和异步 I/O 的 buffer 所有权完全一致。

中断合并和轮询

高吞吐设备如果每个包都中断一次，CPU 会被中断淹没。中断合并让设备积累多个完成再中断；NAPI 一类机制在高负载下从中断转为轮询，批量处理包，降低中断成本。代价是单个包延迟可能增加。

```
rx_interrupt():
    disable_rx_irq()
    schedule_poll()

poll(budget):
    processed = 0
    while processed < budget and rx_desc_ready():
        handle_packet()
        processed += 1
    if no_more_packets():
        enable_rx_irq()
```

这里的 budget 是背压工具。一次 poll 不能无限处理，否则其他任务得不到 CPU。

测试

- MMIO 等待必须有超时，设备无响应时返回错误。
- 描述符环满时返回背压，不覆盖未完成描述符。
- 提交描述符前执行必要内存屏障。
- DMA buffer 在 completion 前不能释放或复用。

- 中断处理上半部不执行可能睡眠操作。
- 批量 poll 遵守 budget，不能垄断 CPU。
- 设备 reset 后，驱动能清理 in-flight 请求并通知上层失败。

源码级补充：Linux 设备模型、PCI、NVMe、virtio 与 IOMMU

设备模型：bus、driver、device、class

Linux 设备模型把发现、匹配、生命周期和 sysfs 统一起来。总线负责枚举和匹配，driver 提供 probe/remove，device 表示具体硬件或虚拟设备，class 给用户空间暴露类别视图，kobject/kset 负责引用计数和 sysfs 层次。

```
struct bus_type      -> match(device, driver)
struct device_driver -> probe(device), remove(device)
struct device        -> parent, bus, driver, kobject
struct class         -> /sys/class view
```

probe 成功后驱动必须拥有明确资源集合：MMIO 映射、IRQ、DMA mask、队列、字符/块/网络注册对象.remove 路径按相反顺序撤销，并拒绝新请求、等待 in-flight 请求完成。

PCI 驱动骨架

```
static const struct pci_device_id ids[] = {
    { PCI_DEVICE(VENDOR, DEVICE) },
    { 0 }
};

probe(pdev, id):
    pci_enable_device(pdev)
    pci_request_regions(pdev)
    mmio = pci_iomap(pdev, bar)
    dma_set_mask_and_coherent(pdev, DMA_BIT_MASK(64))
    request_irq(pdev->irq, handler, IRQF_SHARED, name, dev)
    init_queues(dev)

remove(pdev):
    stop_queues(dev)
    free_irq(pdev->irq, dev)
    pci_iounmap(pdev, mmio)
    pci_release_regions(pdev)
    pci_disable_device(pdev)
```

错误路径要能从任一步失败回滚。例如 request_irq 失败时，要释放 MMIO 和 PCI region；队列初始化失败时，要释放 IRQ。真实源码中这类 goto err_* 标签比正常路径更能体现驱动质量。

字符、块、网络三类设备

类别	核心对象	典型回调
字符设备	cdev、file_operations	open/read/write/ioctl/ poll
块设备	gendisk、request_queue、 blk-mq	submit_bio、queue_rq、 completion
网络设备	net_device、net_device_ ops、NAPI	ndo_start_xmit、poll、 set_rx_mode

同一个硬件可能暴露多个接口。比如 NVMe 是块设备，网卡是网络设备，串口是字符设备。VFS 统一 fd 入口，但驱动对象和 completion 路径完全不同。

streaming DMA 与 coherent DMA

`dma_alloc_coherent` 返回 CPU 和设备一致可见的内存，适合描述符环；`dma_map_single/sg` 把已有 buffer 临时映射给设备，适合数据包或块 I/O。streaming DMA 要在方向、同步和 unmap 上严格配对。

```
tx_submit(buf):
    dma = dma_map_single(dev, buf.data, buf.len, DMA_TO_DEVICE)
    if dma_mapping_error(dev, dma): return -EIO
    fill_desc(dma, buf.len)
    dma_wmb()
    ring_doorbell()

tx_complete(desc):
    dma_unmap_single(dev, desc.dma, desc.len, DMA_TO_DEVICE)
    free_buf(desc.buf)
```

IOMMU 为设备建立地址空间，限制设备只能 DMA 到授权页。没有 IOMMU 或映射错误时，设备 bug 可能覆盖任意内存；有 IOMMU 时，错误 DMA 会变成 IOMMU fault，至少能阻止破坏扩大。

NAPI、NVMe 与 virtio 阅读入口

网络驱动可读 `drivers/net/ethernet/intel/e1000e/` 或 `virtio-net`；NVMe 可读 `drivers/nvme/host/`；virtio 核心可读 `drivers/virtio/`。阅读时按同一模板：

1. probe 如何发现设备、分配队列、注册上层对象。
2. submit 路径如何填描述符、DMA map、通知设备。
3. completion 路径如何从中断/NAPI/poll 回收资源。
4. reset/remove 路径如何停止新请求并清理 in-flight。
5. 错误统计在哪里暴露给 `ethtool`、`sysfs`、`dmesg` 或 `block stats`。

ACPI platform 设备与 PCIe 自描述设备

并非所有设备都能像 PCIe 一样完整自描述。x86-64 PC/服务器上，可枚举高速设备通常来自 PCIe 配置空间；板载控制器、固件虚拟设备、电源和热管理资源常由 ACPI 表描述。platform driver 的匹配不依赖 vendor/device id，而依赖 ACPI id 或平台总线注册信息。

```
ACPI resource sketch:
HID/UID identify the platform device
MMIO resource describes register window
IRQ resource describes interrupt routing
_CRS reports current resource settings
_PS0/_PS3 may describe power-state transitions
```

probe 阶段要把描述转换成内核资源：`platform_get_resource` 取得 MMIO 区间，`devm_ioremap_resource` 建立映射，`platform_get_irq` 取得中断，电源管理回调处理设备 suspend/resume。教学系统可以把 ACPI 资源简化成静态设备表，但仍要保留“描述硬件”和“驱动绑定”两个阶段。

托管资源和错误回滚

Linux 驱动大量使用 devm 托管资源。资源绑定到 device 生命周期，remove 或 probe 失败时自动释放。托管资源不是为了偷懒，而是为了减少 probe 多阶段失败路径中的泄漏。

```

probe(dev):
    state = devm_kzalloc(dev, sizeof(*state))
    mmio = devm_ioremap_resource(dev, res)
    irq = platform_get_irq(dev, 0)
    ret = devm_request_threaded_irq(dev, irq, top, thread, flags, name, state)
    if ret:
        return ret
    ret = register_upper_object(state)
    if ret:
        return ret
    return 0

```

但 devm 不能替代所有生命周期管理。已经暴露给上层的字符设备、网络设备或块设备必须先停止新请求、撤销注册、等待 in-flight 完成，再让托管资源释放。否则用户态仍可能通过旧 fd 进入已经释放的 MMIO 或队列。

ioctl、sysfs 与用户态控制面

驱动给用户态暴露控制面时，常见三种方式：ioctl、sysfs 属性和 netlink。ioctl 适合和 fd 绑定的设备私有命令；sysfs 适合稳定、简单、可文本化的设备属性；netlink 适合网络和复杂事件通知。控制面设计必须有版本、长度、权限和兼容性边界。

```

ioctl(file, cmd, user_arg):
    switch cmd:
        case GET_STATUS:
            status = snapshot_device_status()
            return copy_to_user(user_arg, status)
        case RESET_DEVICE:
            if !capable(CAP_SYS_ADMIN):
                return -EPERM
            return reset_device_safely()
        default:
            return -ENOTTY

```

ioctl 的常见漏洞来自信任用户结构体长度、未检查 padding、把内核指针泄露给用户、或在 reset 中没有和 I/O 路径同步。控制命令不是旁路，它和 read/write/interrupt 一样会并发访问设备状态。

runtime PM、suspend 与 resume

现代设备需要省电。runtime power management 允许设备空闲时关闭时钟或电源域；系统 suspend/resume 要在整机睡眠和唤醒时保存恢复设备状态。驱动必须区分“设备逻辑对象仍存在”和“硬件暂时不可访问”。

```

runtime_suspend(dev):
    stop_new_io(dev)
    wait_inflight(dev)
    save_registers(dev)
    disable_irq(dev)
    disable_clock(dev)

runtime_resume(dev):
    enable_clock(dev)
    restore_registers(dev)
    enable_irq(dev)
    restart_queues(dev)

```

如果 resume 后忘记恢复 DMA mask、队列基址或中断 mask，设备可能表现为偶发超时。电源管理 bug 常常只在压力、热插拔、suspend 循环或低功耗平台上出现，因此测试必须覆盖这些状态转换。

驱动并发不变量

驱动至少要维护下面不变量：

- 提交给设备的描述符在 completion 前不能释放。
- 设备可 DMA 的 buffer 必须有有效映射，方向必须匹配。
- remove/reset 开始后，不再接受新请求。
- 中断处理看到 dying 状态时只确认和丢弃，不再提交新工作。
- 用户 fd 持有期间，驱动私有对象引用不能降到零。
- 上半部只做不可睡眠操作，复杂恢复进入线程化 IRQ 或 workqueue。

把这些不变量写进测试，才能发现“正常收发能跑”之外的问题。驱动不是能读写设备就合格，而是要在错误、中断风暴、热拔、reset 和并发 close 下仍然收敛。

4.2 块层、I/O 调度与完成语义

块设备层、I/O 调度与提交顺序

原理

文件系统最终要把逻辑块读写交给块设备。若每个上层请求都直接变成一次磁盘操作，系统会遭遇两个问题：机械磁盘上寻道开销巨大，随机请求顺序会让吞吐崩溃；即使在 SSD/NVMe 上，请求大小、队列深度、合并策略和 flush 顺序也直接影响延迟和持久化语义。因此操作系统在文件系统和驱动之间放置块层，负责把 bio 合并、拆分、排序、限流、派发和完成。

块层的目标不是“越重排越好”。对普通读写，重排可以提高吞吐；对日志提交、数据库事务和文件系统 barrier，错误重排会破坏崩溃一致性。I/O 调度器必须同时理解性能和顺序约束：哪些请求可合并，哪些可越过，哪些必须在 flush 后才能报告完成。

核心思想

块层把“文件系统想读写哪些块”转化为“设备能高效且按约束完成的请求序列”。它是性能优化层，也是持久化契约层。

从设备演进看，机械磁盘偏好按扇区顺序扫描；SATA SSD 降低了寻道成本但仍受擦写块和队列深度影响；NVMe 提供多队列并行，单一全局电梯队列反而会成为瓶颈。教材不能只讲一个 SCAN 算法就结束，而要说明为什么算法随硬件变化。

实践

块层常用对象如下：

对象	含义	关键字段
bio	上层的一段块 I/O 意图	sector、len、page list、op、flags、endio
request	设备队列中的可派发请求	合并后的 bio 链、方向、优先级、deadline
request queue	块设备的软件队列	pending、inflight、depth、scheduler
hardware queue	多队列设备的硬件提交队列	doorbell、completion、CPU 亲和

flush request 缓存刷新或持久化屏障

preflush、FUA、barrier
dependency

一次写入的路径可以写成：

```
filesystem_writeback(page):
    bio = build_bio(page.block, page.data, WRITE)
    submit_bio(bio)

submit_bio(bio):
    q = bio.device.queue
    if can_merge(q.last_request, bio):
        merge(q.last_request, bio)
    else:
        req = make_request(bio)
        scheduler_insert(q, req)
        dispatch_if_possible(q)
```

这里的关键是 bio 的生命周期。bio 完成并不等于调用者数据结构一定可释放，除非 endio 回调已经把 page 状态、错误码和等待者唤醒都处理完。对写请求而言，“设备报告完成”也不必然等于掉电后数据存在，除非请求带有 FUA 或之前执行了正确 flush。

算法与实现分析

合并与拆分

合并减少请求数量，提高吞吐。前向合并是新 bio 紧接已有请求后方；后向合并是新 bio 紧接已有请求前方。

```
try_merge(req, bio):
    if req.op != bio.op or req.device != bio.device:
        return false
    if req.end_sector == bio.start_sector:
        append(req, bio)
        return true
    if bio.end_sector == req.start_sector:
        prepend(req, bio)
        return true
    return false
```

合并复杂度取决于查找结构。只看队尾是 $O(1)$ ，但合并机会少；用按 sector 排序的树查找相邻请求是 $O(\log n)$ ，合并率更高。拆分发生在请求跨越设备最大段数、最大字节数、DMA 边界或 zone 限制时。教学模型可以只实现连续扇区合并，但要保留“设备约束可能要求拆分”的接口。

SCAN 和 deadline

SCAN 电梯算法按当前磁头方向服务请求，到尽头后反向。它适合解释机械磁盘为什么怕随机寻道。

```
pick_scan(queue, head, direction):
    candidates = requests_in_direction(queue, head, direction)
    if empty(candidates):
        direction = reverse(direction)
        candidates = requests_in_direction(queue, head, direction)
    return nearest_by_sector(candidates, head, direction)
```

SCAN 的吞吐较好，但远处请求可能等待很久。deadline 调度器给读写请求设置截止时间，选择时优先处理过期请求，否则继续按 sector 顺序提升吞吐。

```
pick_deadline(queue):
    if has_expired(queue.read_deadline):
        return pop_oldest(queue.read_fifo)
    if has_expired(queue.write_deadline):
        return pop_oldest(queue.write_fifo)
    return pop_by_sector_order(queue)
```

deadline 的本质是用最大等待时间约束吞吐优化。它不保证实时性，但能避免单纯电梯算法在混合负载下让读请求被大量写请求拖垮。

CFQ/BFQ 与公平性

桌面和多租户环境还需要进程间公平。按进程或控制组建立 I/O 队列，可以避免一个大顺序写任务压制交互读。BFQ 这类预算公平队列给每个实体分配服务预算，用“服务了多少字节”和“等待了多久”共同决定下一个队列。

```
pick_fair(queue_set):
    entity = min_virtual_finish_time(queue_set.active)
    req = entity.pop_next_request()
    entity.service += req.bytes
    entity.vfinish = entity.vstart + entity.service / entity.weight
    if entity.has_more():
        reinsert(entity)
    return req
```

公平调度会牺牲部分全局顺序性，尤其在机械磁盘上可能增加寻道。工程选择取决于目标：批处理吞吐、桌面响应、数据库尾延迟还是云主机隔离。

flush、FUA 与 barrier

现代存储有缓存。普通写完成可能只表示数据到达设备缓存，不表示掉电持久。flush 请求要求设备把缓存写入介质；FUA 要求当前写绕过或强制落到持久介质；barrier 规定顺序，防止前后的写被重排穿越。

```
submit_ordered_write(data_req):
    if data_req.requires_preflush:
        submit(FLUSH)
        wait_flush_done()
    submit(data_req with FUA if requested)
    wait_data_done()
    if data_req.requires_postflush:
        submit(FLUSH)
        wait_flush_done()
    complete_to_filesystem()
```

这段逻辑解释了为什么 `fsync` 慢：它不只是把 page cache 交给设备，还可能要求等待队列中相关写完成，并发出 flush 来建立掉电后的顺序证据。

多队列 NVMe

NVMe 设备支持多个 submission/completion queue。块层会把请求映射到 CPU 本地硬件队列，减少全局锁竞争。

```
submit_nvme(req):
    hctx = map_cpu_to_hw_queue(current.cpu)
    tag = allocate_tag(hctx)
    cmd = build_nvme_command(req, tag)
    sq = hctx.submission_queue
    sq.entries[sq.tail] = cmd
    sq.tail = next(sq.tail)
    ring_doorbell(hctx)
```

多队列提高并行度，但也削弱了全局排序。需要顺序语义的请求必须带明确依

赖，不能假设“提交顺序就是完成顺序”。

测试

- 1. 合并测试：连续写 bio 应合并成较少 request，非连续不得错误合并。
- 2. 拆分测试：超过最大请求大小时必须拆分，完成回调仍只向上层报告一次整体结果或清晰的部分错误。
- 3. SCAN 测试：给定 head 和方向，选择顺序可手算。
- 4. deadline 测试：过期读请求应越过大量未过期写请求。
- 5. flush 顺序测试：日志数据、flush、commit block 的完成顺序不能被重排破坏。
- 6. 多队列测试：同一队列 tag 唯一，completion 能找到原请求。
- 7. 错误注入：设备返回 I/O error 时，page、bio、request、等待者状态都能收敛。

本章合格标准：能说明 bio、request、queue 的区别；能分析 SCAN、deadline、公平调度的取舍；能解释 flush/FUA/barrier 为什么属于正确性而不仅是性能。

源码级补充：blk-mq、IO 统计、优先级与 io_uring

blk-mq 对象关系

blk-mq 把软件提交队列映射到硬件队列，减少单锁瓶颈。核心对象包括 request_queue、blk_mq_tag_set、hardware context、software context 和 tag。tag 是请求在硬件队列中的身份，用 completion 找回 request。

```
submit_bio
-> blk_mq_submit_bio
-> get request tag
-> map current CPU to hctx
-> driver queue_rq
-> device completion interrupt
-> blk_mq_complete_request
-> bio end_io
```

读源码看 block/blk-mq.c、block/bio.c 和具体驱动的 queue_rq。关注 request 从 free tag 到 in-flight，再到 completion 释放 tag 的生命周期。

IO 统计从哪里来

/proc/diskstats、/proc/[pid]/io、iostat 看到的数字来自块层和任务 I/O 统计。它们能说明请求数、扇区数、队列时间、服务时间，但不能直接告诉你应用层哪个业务请求慢。需要把应用 trace、系统调用和块层事件串起来。

指标	解释注意点
read/write IOPS	请求次数，受合并影响，不等于应用调用次数
sectors	数据量，可和吞吐换算
await/latency	队列等待加服务时间，受队列深度影响

util	设备忙碌时间比例，多队列设备上不等同饱和度
------	-----------------------

错误诊断不要只看 util。NVMe util 高不一定坏，机械盘 util 高更可能意味着排队；await 上升要结合队列深度、flush、写回和 cgroup 限流。

I/O 优先级和 cgroup

ionice、I/O priority class 和 blk-cgroup 可影响不同任务的 I/O 服务顺序。云主机和容器环境常通过 cgroup v2 的 io controller 控制权重或限速。公平性会改变尾延迟：低权重任务可能明显变慢，高权重任务不应被后台写回完全淹没。

io_uring 与块 I/O

io_uring 用共享提交队列 SQ 和完成队列 CQ 减少系统调用往返，支持固定 buffer、固定文件、poll、timeout、链式请求和异步取消。它不是“绕过内核”，而是把提交和完成批量化，让内核在更少入口中处理更多请求。

```
userspace:
    sqe = get_sqe()
    sqe.op = READV
    sqe.fd = fixed_file
    sqe.buf = fixed_buffer
    submit_sq_tail()
    io_uring_enter()

kernel:
    consume SQEs
    issue async work or direct submit
    post CQEs with result
```

io_uring 的正确性问题和普通异步 I/O 一样：buffer 在 completion 前不能释放，取消可能和完成竞态，部分完成必须按 CQE 结果处理。fixed buffer/file 提高性能，但也把生命周期责任推给应用。

writeback、dirty throttling 与块层的关系

应用 write 把页标脏后，脏页并不会无限积累。内核用 dirty ratio、dirty bytes、writeback 线程和 per-bdi 状态控制写回。当脏页过多时，写入者可能被迫参与平衡，表现为一次普通 write 变慢。

```
balance_dirty_pages(task, mapping):
    dirty = global_dirty_pages()
    if dirty < dirty_background:
        return
    wake_writeback_threads()
    if dirty > dirty_limit:
        sleep_or_writeback_until_below_limit(task)
```

这个机制把存储压力反馈给应用。没有 throttling，快 CPU 可以瞬间制造大量脏页，把内存挤爆；有 throttling，局部写入延迟会上升，但系统保持可控。诊断写慢时要区分：是用户态复制慢，page cache 分配慢，dirty throttling，块层排队，还是设备 flush。

zone 设备和顺序写约束

某些设备把空间划成 zone，并要求一个 zone 内按顺序写入。文件系统和块层必须知道 zone write pointer，不能随意把随机写重排进去。这个例子说明块层不只是性能优化，还承载设备语义。

```
submit_zone_write(req):
    zone = lookup_zone(req.sector)
    if req.sector != zone.write_pointer:
        return -EIO
```

```
dispatch(req)
zone.write_pointer += req.length
```

对普通块设备，逻辑块可随机覆盖；对 zone 设备，文件系统可能要采用 log-structured 布局或显式 reset zone。硬件约束改变，上层算法就必须改变。

请求取消、超时与错误完成

块请求提交后可能超时、被设备 reset、被上层取消或完成失败。取消不是“把请求从队列里删除”这么简单，因为请求可能已经在设备硬件队列中，甚至设备正在 DMA。

```
timeout_request(req):
    mark req timed_out
    ask driver eh_timed_out(req)
    if driver says reset:
        freeze_queue()
        reset_device()
        complete_lost_requests(EIO)
        unfreeze_queue()
    else if request completed meanwhile:
        ignore timeout
```

这里存在竞态：timeout 线程准备处理请求时，中断也可能完成请求。request 状态机必须保证 completion 只发生一次，tag 只释放一次，bio endio 只调用一次。块层测试要专门构造 timeout 和 completion 交错。

从 fio 数字到 OS 解释

I/O benchmark 不能只报 MB/s 和 IOPS。高质量报告要写明 direct/buffered、sync/async、iodepth、block size、read/write mix、文件系统、是否预热、是否清缓存、是否包含 fsync。

参数	含义	影响
bs	每次 I/O 大小	合并、吞吐和延迟
iodepth	in-flight 请求数量	设备并行度和排队
direct	是否绕过 page cache	缓存干扰和对齐要求
fsync	是否等待持久化	flush 延迟和事务成本
rw	顺序/随机、读/写混合	调度器和介质特性

如果不说明这些参数，“磁盘很快”或“磁盘很慢”没有工程意义。OS 学习里的性能测试必须可复现，也必须能解释观测值背后的路径。

4.3 VFS、文件对象与路径解析

文件、设备与 I/O

原理

文件描述符是用户态访问内核对象的句柄。它可以指向普通文件、管道、socket、设备或匿名内核对象。进程拿到的是一个小整数，但内核维护的是对象、offset、权限、引用计数和关闭状态。

文件 I/O 的第一层对象关系是：路径字符串经过路径解析找到目录项和 inode；`open` 产生 open file description；进程 fd 表保存一个小整数到 open file description 的引用。`dup` 产生新的 fd，但通常仍指向同一个 open file description，因此共享 offset 和状态。再次 `open` 同一路径通常产生新的 open file description，因此 offset 分离。

普通文件读写经常经过 page cache。读命中 page cache 时不碰设备；读未命中时可能提交块设备请求并等待完成。写通常先把用户字节复制进 page cache，把页标记为 dirty，再由后台 writeback 或 `fsync` 推进到设备。`write` 返回不等于数据安全落盘，这一点必须反复强调。

设备可以看作有特殊行为的文件对象。终端、磁盘、网卡、`timerfd`、`eventfd`、`epoll`、`pipe`、`socket` 都可以放进 fd/event 模型，但它们的 read/write 语义不同：有的返回字节流，有的返回结构化事件，有的可能阻塞，有的可能短读短写，有的只表示 readiness。

阻塞、非阻塞和异步是三种控制流模型。阻塞 I/O 让当前线程睡眠直到有结果；非阻塞 I/O 在不能推进时返回 `EAGAIN`，调用者等待 readiness 后重试；异步 I/O 提交请求，completion 稍后返回结果。三者没有绝对高低，关键是请求身份、buffer 生命周期、短读短写、错误传播和背压。

背压是 I/O 系统的安全阀。无界队列能让短 benchmark 看起来很快，因为生产者从不等待；一旦设备变慢，内存会持续增长，尾延迟扩大，最终进程可能被杀。可靠系统要同时限制 item 数、字节数和 in-flight 请求数，并在关闭时 drain 或 cancel。

实践

纸上实践一：画 fd 引用图。

```
process fd table:
fd 3 -> open file description A -> inode X
fd 4 -> open file description A -> inode X    # dup
fd 5 -> open file description B -> inode X    # open same path again
```

这张图能解释 offset 行为：fd 3 和 fd 4 共享 offset，fd 5 有独立 offset。并行文件读取如果使用 fd 3 和 fd 4，就可能因为共享 offset 得到不可预期的分片；使用 `pread` 或独立 open file description 更清楚。

纸上实践二：写出短写循环。任何 `write` 都可能只写入一部分字节，尤其是 pipe、socket、非阻塞 fd 或被信号打断时。正确代码必须记录已完成字节数，继续写剩余部分，遇到 `EINTR` 重试，遇到 `EAGAIN` 进入等待，而不是把短写当成完整成功。

纸上实践三：区分 readiness 和 message。`epoll` 告诉你 fd 可能可读或可写，不告诉你协议帧已经完整。网络程序必须有用户态 receive buffer、解析状态机和背压策略。把 fd readiness 当成完整请求，是事件驱动程序常见错误。

```
PreparedBuffer
-> Submitted
-> InFlight
-> Completed(bytes, error)
-> ConsumedByProtocol
-> BufferReleased
```

异步 I/O 中，buffer 在 completion 前不能被上游复用；completion 失败不能被提交路径吞掉；cancel 后也可能收到迟到 completion，所以请求身份必须稳定。

算法与实现分析

fd 分配表

进程的 fd table 可以先看作数组：下标是 fd，元素是指向 open file description 的引用。open 要找最小空闲 fd；close 清空槽位并减少引用计数；dup 找新槽位并增加引用计数。

```
alloc_fd(file):
    for i in 0..max_fd:
        if table[i] == null:
            table[i] = file
            file.refcount += 1
            return i
    return -EMFILE

close(fd):
    file = table[fd]
    if file == null: return -EBADF
    table[fd] = null
    file.refcount -= 1
    if file.refcount == 0:
        file.ops.release(file)
```

线性扫描是 $O(n)$ ，但简单。真实系统会用 bitmap 或 hint 记录下一个可能空闲位置，把常见分配降到接近 $O(1)$ 。无论怎么优化，不变量都是：fd 只是进程局部索引，open file description 才保存 offset、flags 和对象引用。

路径解析

路径解析把字符串变成目录项和 inode。绝对路径从根目录开始，相对路径从当前工作目录开始；每个路径分量都要查目录；遇到符号链接可能要跳转；权限和 mount namespace 会改变可见结果。

```
resolve(path):
    node = path.starts_with("/") ? root : current.cwd
    for component in split(path, "/"):
        if component == "." continue
        if component == "..": node = node.parent
        else:
            node = lookup_child(node, component)
            if node is symlink:
                node = resolve_symlink(node)
    return node
```

路径解析的复杂度大致和路径分量数相关，每个目录查找又取决于文件系统目录结构。缓存 dentry 可以避免每次都从磁盘读目录。安全实现还要限制符号链接递归深度，避免循环。

page cache 读写

普通文件读路径先查 page cache。若缓存命中，从内核页复制到用户 buffer；若未命中，分配 page cache 页，提交块 I/O，等待完成，再复制。写路径通常复制用户数据到 page cache，标记 dirty，并在之后由 writeback 或 fsync 推进设备写。

```
buffered_read(file, offset, len):
    while len > 0:
        page_index = offset / PAGE_SIZE
        page = page_cache_lookup(file, page_index)
        if page == null:
            page = read_page_from_storage(file, page_index)
        n = copy_page_to_user(page, offset % PAGE_SIZE, len)
        offset += n
        len -= n
```

这个算法解释了冷读和热读差异。热读可能完全不碰设备；冷读可能等待存储。benchmark 若不说明缓存状态，就无法比较。

设备队列和完成

块设备、网卡和异步 I/O 都可以抽象成提交队列和完成队列。提交时生成 request，放入设备或驱动队列；完成时中断或轮询拿到结果，唤醒等待任务或投递 completion。

```
submit(req):
    req.id = next_id()
    req.state = IN_FLIGHT
    device_queue.push(req)
    kick_device()

complete(req_id, status, bytes):
    req = lookup(req_id)
    req.status = status
    req.bytes = bytes
    req.state = COMPLETED
    wake(req.waiter)
```

请求身份非常重要。异步完成可能乱序返回；取消后的请求也可能迟到完成；buffer 所有权必须跟着请求身份走。没有 request id，就无法把 completion 正确对应到提交者。

fd、file 和 inode 的并发引用

fd 是进程 fd 表下标，不是对象身份。内核执行 read(fd) 时，先把 fd 转成 struct file 引用；只要引用拿到，即使另一个线程随后 close(fd)，当前 I/O 也能在 file 对象上完成。close 删除的是 fd 表槽位，并减少 file 引用；它不能把正在执行的 I/O 用到的 file 直接释放。

```
sys_read(fd, buf, len):
    file = fget(fd)
    if file == null:
        return -EBADF
    ret = vfs_read(file, buf, len)
    fput(file)
    return ret

sys_close(fd):
    file = files_table_remove(fd)
    if file == null:
        return -EBADF
    fput(file)
    return 0
```

这个规则解释 fd reuse 风险：线程 A 保存整数 fd=5，线程 B close(5)，线程 C open 得到新的 fd=5。A 再用整数 5 时，访问的可能已经是另一个对象。内核靠 file 引用保护内部生命周期，用户态也要避免把裸 fd 长期跨线程传递而没有所有权约定。

open flags 的作用域

flag 不是都绑定在同一层。fd flag 绑定 fd 表项；file status flag 绑定 open file description。dup 后两个 fd 指向同一个 open file description，所以共享 file status flag 和 offset；但 close-on-exec 是 fd flag，每个 fd 可以不同。

标志	作用范围	边界
----	------	----

O_CLOEXEC	fd 表项	exec 提交时关闭该 fd，不影响其他 fd 的标志
O_NONBLOCK	open file description	dup 后 共享，影响 read/write/accept/connect 等等待行为
O_APPEND	open file description	每次写前在内核内原子移动到文件尾
O_DIRECT	file I/O 路径	对齐、页 pin、缓存一致性和短 I/O 更复杂
O_SYNC/O_DSYNC	写入持久化语义	返回前要推进数据或元数据到更强边界

O_APPEND 不是用户态先 lseek 到末尾再 write。后者在并发下会竞争：两个线程都看到同一个旧文件尾，然后互相覆盖。append 语义要求内核在持有文件系统相关锁的状态下决定写入位置。

短读短写不是异常

普通文件顺序读常尽量返回请求长度，但很多 fd 都允许短读短写。pipe 可能只剩一部分数据；socket 是字节流，接收缓冲里有什么就先给什么；非阻塞 fd 无法继续推进时返回 EAGAIN；信号可能让系统调用返回 EINTR；存储错误可能让已经完成的部分和错误同时出现。

```
write_all(fd, buf, len):
    done = 0
    while done < len:
        n = write(fd, buf + done, len - done)
        if n > 0:
            done += n
            continue
        if errno == EINTR:
            continue
        if errno == EAGAIN:
            wait_writable(fd)
            continue
        return error
    return done
```

读路径也需要协议层状态机。对 TCP，recv 返回 20 字节不表示一个业务消息完整；对 UDP，一次 recvmsg 对应一个 datagram，buffer 太小会截断；对终端，行规程可能改变返回时机。OS 提供 fd 语义，应用协议必须自己定义消息边界。

buffered I/O 和 direct I/O 的不同风险

buffered I/O 经过 page cache，适合普通应用和共享缓存；direct I/O 尽量绕过 page cache，适合数据库一类自己管理缓存和写入顺序的程序。direct I/O 不是“更高级的 read/write”，它把对齐、页 pin、设备队列、缓存一致性和错误处理暴露得更多。

路径	优点	代价
buffered read	命中 page cache 很快，可共享缓存	多一次复制，缓存污染可能影响其他任务
buffered write	快速标脏，后台批量写回	write 返回不代表持久化
direct read/write	减少 page cache 干扰，适合自管缓存	对齐、pin 页、短 I/O 和并发一致性复杂

mmap	load/store 直接访问映射	缺页、SIGBUS、dirty/writeback 语义更隐蔽
------	-------------------	---------------------------------

direct I/O 和 buffered I/O 混用尤其危险。一个路径绕过 page cache，另一个路径读写 page cache，若文件系统没有正确失效或同步缓存，应用可能看到旧数据。真实系统会在文件系统层处理这类一致性，但应用设计仍应尽量避免无计划混用。

epoll item 绑定的是 file

epoll_ctl 把一个 fd 对应的 file 和 interest mask 加进 epoll 实例。之后整数 fd 关闭并复用，不代表旧 epoll item 自动变成新文件的监听项。事件返回里带着用户当初登记的 fd 数字，但内核里的等待关系绑定的是旧 file。

```
epoll_ctl(epfd, ADD, fd):
    file = fget(fd)
    epitem.file = file
    epitem.fd_number_for_user = fd
    attach_poll_callback(file, epitem)

event delivery:
    file state changes
    callback enqueues epitem
    epoll_wait returns stored fd_number and event mask
```

事件循环通常要给每个连接对象加 generation 或指针身份，而不是只根据 fd 整数找状态。close、dup、fork、exec、线程并发都可能让 fd 数字的直觉失效。

io_uring 的提交、完成和取消

io_uring 把“提交请求”和“收完成结果”拆成两个环。SQE 中的 user_data 是应用识别 completion 的关键；CQE 中的 res 是最终字节数或负 errno。提交成功只表示内核接收了请求，不表示 I/O 完成。

```
submit:
    fill SQE with opcode, fd, buffer, length, user_data
    publish SQ tail
    enter kernel or rely on SQPOLL

complete:
    read CQE
    id = cqe.user_data
    result = cqe.res
    release or reuse buffer only now
```

取消请求要按竞态理解：取消到达时，请求可能还在队列里，可能已经派发给设备，可能刚刚完成。应用必须能接受“取消成功”“取消失败因为已经完成”“先收到完成后收到取消结果”等情况。只要 completion 可能到来，buffer 和请求对象就不能提前复用。

源码阅读入口

fd 表和引用看 fs/file.c、include/linux/fdtable.h；open 和路径解析看 fs/open.c、fs/namei.c；read/write 主干看 fs/read_write.c；page cache 看 mm/filemap.c；epoll 看 fs/eventpoll.c；io_uring 看 io_uring/。阅读时跟住一个 fd：它从整数变成 file，如何增加引用，如何进入具体 file operations，错误码在哪里返回，最后一个引用在哪里释放。

测试

测试覆盖 open/write/read、dup 后共享内容、close 一个 fd 后另一个 fd 仍有效、关闭所有引用后访问失败、未知 fd 返回诊断。还要测试 fd 分配单调或复用策略是否有明确规则。

- dup 后 offset 共享，独立 open 后 offset 分离。
- pread 不改变 current offset。
- pipe 空读阻塞或非阻塞返回 EAGAIN，写端关闭后读端得到 EOF。
- 读端关闭后写 pipe 失败，不应静默丢数据。
- socket 或 pipe 短写后，调用者继续写剩余字节。
- readiness 事件只触发解析尝试，不被当成完整消息。
- 异步请求完成前，buffer 所有权不能释放。
- 队列达到字节上限时，生产者受到背压。

第一版必须先保证句柄生命周期可解释。持久化和崩溃恢复放到下一章单独讨论。

4.4 文件系统持久化与故障测试

文件系统设计：目录、inode、缓存与一致性

原理

文件系统把块设备上的字节数组组织成命名对象。用户看到路径、目录、普通文件、链接和权限；内核维护 superblock、inode、目录项、数据块、空闲空间、页缓存和日志。文件系统不是“把文件内容写到磁盘”这么简单，它要解决命名、分配、缓存、权限、并发和崩溃一致性。

最小文件系统可以从四类对象开始。superblock 描述整个文件系统，例如块大小、inode 表位置、空闲块位图位置。inode 描述一个文件的元数据和数据块地址。directory 是从名字到 inode 号的映射。free space manager 记录哪些块和 inode 可用。

实践

画出一个小文件系统布局：

```
block 0: boot or reserved
block 1: superblock
block 2: inode bitmap
block 3: block bitmap
block 4..15: inode table
block 16..end: data blocks
```

再画路径解析：`/home/a.txt` 先从根 inode 读取目录数据，查找 `home`，得到 home inode，再读取 home 目录，查找 `a.txt`，得到文件 inode，最后根据 inode 的 block 指针读取数据。

算法与实现分析

inode 和块映射

inode 需要把文件 offset 映射到磁盘块。小文件可以用 direct block 指针；大文件需要 indirect、double indirect 或 extent。extent 用起始块和长度描述连续范围，适合大连续文件。


```

block_for_offset(inode, offset):
    logical = offset / BLOCK_SIZE
    if logical < NDIRECT:
        return inode.direct[logical]
    logical -= NDIRECT
    indirect_block = read_block(inode.indirect)
    return indirect_block[logical]

```

direct/indirect 结构简单，但大文件查找可能多一次或多次 I/O。extent 查找需要在 extent 列表或树中搜索，复杂度可能是 $O(\log n)$ ，但连续文件元数据更少。

目录查找

最简单目录是线性表：每个目录项保存 name 和 inode number。查找是 $O(n)$ 。目录小的时候足够；目录很大时，需要 hash 或 B-tree。

```

dir_lookup(dir_inode, name):
    for each entry in read_directory(dir_inode):
        if entry.name == name:
            return entry.inode_number
    return -ENOENT

```

目录缓存 dentry 可以把常见路径查找变快。缓存必须处理失效：rename、unlink、mount namespace、权限变化都可能影响路径结果。

空闲空间管理

bitmap 是最直观的空闲块管理。每个 bit 表示一个块是否空闲。分配时扫描 0 bit，置 1；释放时清 0。连续分配可以扫描连续 0 bit。

```

alloc_block():
    for i in 0..num_blocks:
        if bitmap[i] == 0:
            bitmap[i] = 1
            mark_bitmap_dirty()
            return i
    return -ENOSPC

```

bitmap 简单但扫描可能慢。实际文件系统会按块组、预分配、extent 和局部性策略减少碎片。分配策略要尽量让同一文件的数据块连续，让同一目录下文件局部性更好。

VFS 层

VFS 给不同文件系统提供统一接口。上层调用 open/read/write，VFS 根据 inode 或 file operations 分发到 ext、tmpfs、procfs、设备文件等具体实现。这个抽象让“一切皆文件”可行，但每种对象语义仍不同。

```

vfs_read(file, user_buf, len):
    if !file.can_read: return -EBADF
    return file.ops.read(file, user_buf, len)

```

VFS 的价值是统一生命周期和权限边界：fd 引用 file，file 引用 inode 或设备对象，inode 引用具体文件系统对象。关闭 fd 不等于 inode 消失，unlink 路径名也不等于已打开文件立刻不可用。

创建文件的回滚路径

文件系统修改通常跨多个对象：分配 inode、分配目录项、更新父目录、可能分配数据块、更新 bitmap。任意一步失败都要回滚，否则会留下孤儿 inode 或错误 bitmap。

```

create_file(parent, name):

```

```

inode = alloc_inode()
if inode.error:
    return inode.error
ret = init_inode(inode)
if ret: goto out_inode
ret = dir_add_entry(parent, name, inode.number)
if ret: goto out_inode
mark_inode_dirty(inode)
mark_inode_dirty(parent)
return inode

out_inode:
    free_inode(inode)
    return ret

```

这只是内存中状态的回滚。若某些修改已经写到磁盘，崩溃恢复还需要日志或 fsck。文件系统代码的难点是同时维护运行时一致性和崩溃后可恢复性。

unlink 与打开文件

Unix 语义中，**unlink** 删除目录项，不一定立即删除文件内容。若文件仍被 fd 打开，inode 引用计数仍大于零，数据块要等最后一个引用释放后才能回收。

```

unlink(parent, name):
    inode = dir_lookup(parent, name)
    remove_dir_entry(parent, name)
    inode.link_count -= 1
    if inode.link_count == 0 and inode.open_count == 0:
        free_inode_and_blocks(inode)

close(file):
    inode = file.inode
    inode.open_count -= 1
    if inode.link_count == 0 and inode.open_count == 0:
        free_inode_and_blocks(inode)

```

这个协议解释了为什么删除正在写的日志文件后，磁盘空间可能不立刻释放；也解释了为什么路径名和已打开 fd 是两个不同层次的对象身份。

fsck 的基本思想

没有日志或日志也损坏时，检查工具需要从磁盘结构重建一致性。最小 fsck 会扫描 inode 表、目录树和 bitmap，比较“被引用块集合”和“bitmap 标记集合”。

```

fsck():
    referenced_blocks = empty_set()
    reachable_inodes = walk_from_root_directory()
    for inode in reachable_inodes:
        for block in inode.blocks:
            if block already in referenced_blocks:
                report_duplicate_block(block)
            add(referenced_blocks, block)
    compare_with_block_bitmap(referenced_blocks)
    repair_or_report()

```

fsck 的复杂度通常与文件系统大小相关，可能很慢。日志文件系统的目标之一就是要把崩溃恢复从全盘扫描缩小到重放最近事务。

测试

- 创建文件会分配 inode 和目录项，失败时不能泄漏块。
- unlink 已打开文件后，fd 仍可读写，最后引用释放后再回收 inode。

- rename 后路径解析结果原子切换，不能出现两个半状态。
- 大目录查找有明确复杂度和缓存策略。
- 空闲块 bitmap 崩溃后能通过日志或检查工具恢复一致。
- page cache 中脏页写回后，inode size 和数据块状态一致。

源码级补充：VFS、ext4、jbd2、xattr、quota、FUSE 与 fsck

VFS 操作表

VFS 的核心不是一个对象，而是一组对象和操作表：`super_block` 描述挂载实例，`inode` 描述文件身份，`dentry` 缓存名字关系，`file` 描述一次打开。操作表包括 `file_operations`、`inode_operations`、`super_operations`。

操作表	典型职责
<code>file_operations</code>	<code>read/write/mmap/fsync/poll/ioctl</code> ，面向打开文件
<code>inode_operations</code>	<code>lookup/create/unlink/rename/getattr/setattr</code> ，面向名字和元数据
<code>super_operations</code>	挂载级同步、inode 分配释放、统计信息

读 `fs/namei.c` 和 `fs/open.c` 时，要把路径名解析、权限检查、`dentry` 缓存和最终 `file` 创建分开看。路径字符串只是输入，`dentry/inode/file` 才是稳定对象。

ext4 extent 与 mballoc

ext4 用 extent 描述连续块范围。小文件 extent 可直接放在 inode 内；大文件使用 extent tree。空间分配由 mballoc 按块组、局部性、预分配和碎片情况选择块。

```
logical file block
-> ext4 extent tree lookup
-> physical block range
-> build bio for page cache writeback
```

读 `fs/ext4/extents.c`、`fs/ext4/mballoc.c`。关注 `ext4_map_blocks` 如何把逻辑块映射到物理块，分配失败如何返回 `ENOSPC`，以及预分配如何减少碎片但增加回滚复杂度。

jbd2 事务、commit 与 checkpoint

jbd2 把元数据修改组织成事务。handle 表示一个修改上下文，transaction 收集多个 handle，commit 线程把描述符、元数据和 commit record 写入 journal，checkpoint 再把已提交修改传播回主文件系统位置。

```
ext4_journal_start()
  modify metadata buffers
  ext4_handle_dirty_metadata()
  ext4_journal_stop()

jbd2 commit thread:
  write descriptor blocks
  write metadata blocks
  write commit block with checksum
  checkpoint later
```

读 `fs/jbd2/commit.c`。重点不是记函数名，而是理解 commit record 是恢复判断边界：commit 完整则 redo，commit 缺失或 checksum 坏则忽略该事务。

xattr、ACL、quota 与 FUSE

扩展属性让文件挂载额外元数据，例如 security.selinux、system.posix_acl_access、用户自定义 xattr。安全标签不只是字符串；LSM 在访问时读取标签并按策略决策。备份或复制若丢失 xattr，恢复后的系统可能权限完全不同。

quota 给用户、组或项目限制块和 inode 使用。它需要在分配路径同步更新计数，失败时返回 EDQUOT。FUSE 则把文件系统实现放到用户态，内核 VFS 通过 fuse_req 把 lookup/read/write 等请求交给用户态 daemon。

机制	优点	风险
xattr/ACL quota	表达安全标签和细粒度权限 防止单用户耗尽文件系统	复制备份时容易遗漏 计数一致性、恢复和性能开销
FUSE	用户态开发文件系统，安全 边界更清晰	上下文切换、daemon 死亡、 缓存一致性

fsck 五步检查

以 e2fsck 思路看，恢复工具会检查 superblock、inode、目录、连接关系和 bitmap。每一步都在验证“被引用集合”和“元数据记录集合”是否一致。

- 1. superblock 和块组描述符是否合理。
- 2. inode 表中每个 inode 的类型、大小、块指针是否合理。
- 3. 目录项是否指向合法 inode，目录结构是否可达。
- 4. link count 与目录引用数是否一致。
- 5. block/inode bitmap 与实际引用集合是否一致。

fsck 能修一些结构错误，但不能恢复应用层语义。例如两个业务文件必须同版本，这件事只能由应用 manifest 或数据库日志表达。

mount、namespace 与传播

挂载不是简单把一个磁盘目录接到路径上。内核维护 mount 对象和 mount namespace，不同进程可以看到不同路径树。容器常用独立 mount namespace、bind mount、只读挂载和 pivot_root/chroot 构造自己的文件系统视图。

```
mount namespace:
/
  /proc      procfs
  /sys       sysfs read-only
  /app       bind mount application root
  /data      writable volume
```

mount propagation 决定一个 namespace 中的挂载变化是否传播到其他 namespace。若容器中的挂载意外传播回宿主，可能形成安全漏洞；若宿主挂载没有传播到需要的 namespace，服务可能看不到设备或数据卷。文件系统学习必须把“磁盘布局”和“命名空间视图”分开。

特殊文件系统：tmpfs、procfs、sysfs、overlayfs

并非所有文件系统都对应磁盘。tmpfs 把文件内容放在内存和 swap 中；procfs 暴露进程和内核状态；sysfs 暴露设备模型和 kobject；overlayfs 把 lower 和 upper 层合成一个视图。VFS 的统一接口让这些对象都能被 open/read/write，但语义完全不同。

文件系统	主要用途	语义注意点
------	------	-------

tmpfs	临时文件、共享内存	受内存和 swap 限制, 断电不保留
procfs	进程和内核状态	内容动态生成, 不是普通磁盘文件
sysfs	设备和驱动属性	一文件一属性, 写入可能触发设备操作
overlayfs	容器镜像层叠	copy-up、whiteout、rename 语义复杂

这些文件系统说明“文件”是内核对象接口, 不等于“磁盘上的一段字节”。读 `/proc/meminfo` 是内核现场生成文本; 写 `/sys/.../online` 可能触发 CPU 或设备上线下线; overlay 中修改 lower 文件会先 copy-up 到 upper。

dcache、负 dentry 与路径缓存

路径解析很频繁, 不能每次都读目录块。dcache 缓存 dentry, 包括存在的名字和不存在的名字。负 dentry 表示某目录下某名字不存在, 可以加速反复失败查找。

```
lookup(parent, name):
    dentry = dcache_lookup(parent, name)
    if dentry exists and valid:
        return dentry.inode or ENOENT for negative
    inode = filesystem_lookup(parent.inode, name)
    dcache_insert(parent, name, inode or negative)
    return inode
```

缓存失效是难点。rename、unlink、mount、远端文件系统变化和权限变化都可能影响缓存结果。VFS 需要引用计数、序列号、RCU path walk 和慢路径重试, 才能在高并发路径查找中兼顾性能和一致性。

rename、hard link 与 link count

hard link 让多个目录项指向同一个 inode。link count 记录目录项引用数, open file 引用则是另一套运行时引用。rename 可以把一个目录项原子移到另一个位置, 甚至替换已有目标。

```
rename(old_parent, old_name, new_parent, new_name):
    lock directories in stable order
    old_inode = lookup(old_parent, old_name)
    target = lookup(new_parent, new_name)
    journal intent
    remove old dir entry
    install new dir entry
    adjust link counts if replacing
    commit
```

rename 的复杂性来自跨目录锁顺序、目标存在时的替换、目录不能移动到自己的子树、崩溃恢复和 dcache 失效。教学文件系统可以先限制为同目录普通文件 rename, 再逐步加入跨目录和替换语义。

应用层一致性不能外包给文件系统

文件系统保证自己的元数据一致, 不保证应用多个文件之间的业务关系。比如配置更新涉及 `config.json` 和 `index.json` 两个文件, 文件系统无法知道它们必须同版本。应用要用 manifest、版本号、checksum 和原子切换表达业务提交。

```
publish_version(v):
    write_all("objects/v/data", data)
    fsync("objects/v/data")
    write_all("manifests/v", checksum_and_metadata)
    fsync("manifests/v")
    rename("manifests/v", "CURRENT")
```

```
fsync("manifests")
```

这也是数据库使用 WAL 的原因。文件系统日志保护文件系统结构；数据库日志保护数据库事务；应用 manifest 保护应用版本。层次不同，日志语义不同。

I/O 与文件系统深讲：从 fd 到磁盘块

文件系统和 I/O 是许多教材最难讲清的部分，因为它们跨越对象模型、缓存、设备、并发和持久化。用户看到的是 read、write、open、close，内核看到的是 fd 表、file、inode、page cache、bio、request、DMA 和 completion。

一次 buffered read 的完整路径

```
read(fd, user_buf, len)
-> fd table lookup
-> file object permission check
-> inode and page cache lookup
-> if cache miss: submit block I/O
-> sleep waiting for completion
-> driver interrupt completes request
-> wake reader
-> copy page cache bytes to user buffer
-> update file offset
-> return bytes read
```

这个路径里有三个不同的“成功”。块设备成功读取页面，不代表用户 buffer 已经复制；用户复制成功，不代表文件 offset 更新一定已经对其他共享 fd 可见；read 返回字节数，也不代表下一次读不会因为 EOF、信号或错误返回不同结果。OS 教材必须不断区分这些边界。

一次 buffered write 的完整路径

write 更容易误解。普通 buffered write 通常只是把数据复制到 page cache，并把页标脏。数据什么时候写到设备，由后台 writeback、内存压力、fsync 或文件系统策略决定。

```
write(fd, user_buf, len)
-> fd table lookup
-> check write permission
-> copy user bytes into page cache
-> mark page dirty
-> update inode size if needed
-> return bytes written

later:
writeback dirty page
-> build bio
-> block scheduler
-> driver DMA
-> completion
-> mark page clean
```

因此 write 返回和持久化不是一回事。可靠程序需要 fsync 文件，必要时还要 fsync 父目录，并用 manifest 表达业务提交。

fd、file、inode 的引用关系

对象	保存内容	生命周期
fd slot	进程局部整数到 file 的引用	open/dup/close 改变

file object	offset、flags、ops、inode 引用	引用计数为 0 时释放
inode	文件元数据、数据块索引、权限	link count 和 open count 共同决定回收
dentry	名字到 inode 的缓存映射	rename/unlink/mount 会影响有效性

`dup` 后两个 `fd` 指向同一个 `file object`，所以共享 `offset`；独立 `open` 同一路径得到不同 `file object`，所以 `offset` 分离；`unlink` 删除目录项，但已打开 `file` 仍引用 `inode`，所以文件内容可能继续存在到最后一个 `fd` 关闭。

块层为什么存在

文件系统关心逻辑块，设备关心队列、扇区、对齐、最大请求、`flush` 和 `completion`。块层在中间做合并、拆分、排序和顺序约束。

```
file system:
  write inode block 100
  write data blocks 200..240
  flush journal commit

block layer:
  merge adjacent writes
  split too-large requests
  enforce flush/FUA order
  dispatch to hardware queue
```

没有块层，文件系统要直接理解每种设备队列；没有顺序约束，文件系统日志可能被设备重排破坏；没有合并，吞吐会被小请求拖垮。

文件系统练习

1. 画出 `dup`、`open`、`fork` 后 `fd`、`file`、`inode` 的引用图。
2. 为什么 `unlink` 一个打开文件后，磁盘空间可能不立刻释放？
3. 为什么 `write` 成功后立刻断电，重启不一定看到新数据？
4. 为什么 `direct I/O` 和 `mmap` 同时访问同一文件会有一致性风险？
5. 设计一个测试，证明 `page cache` 热读没有访问块设备。

持久化故障矩阵：从 `write` 到 `fsync`

可靠文件输出要把每个阶段的崩溃结果写清楚。

阶段	崩溃后可能看到	正确恢复态度
<code>write</code> 返回后	数据只在 <code>page cache</code> ，磁盘可能无变化	不能宣布业务提交
<code>fsync(file)</code> 后	文件内容和部分元数据持久	若路径是新建文件，还要考虑目录
<code>rename(tmp, final)</code> 后	名字可能切换，目录项未必持久	需要 <code>fsync(parent dir)</code>
<code>manifest</code> 写入前	数据文件可能存在但未被承认	读取者忽略孤儿文件
<code>manifest fsync</code> 后	版本记录可校验	读取者按 <code>manifest</code> 校验并采用

这个矩阵解释了为什么可靠系统常用临时文件、`rename`、目录 `fsync` 和 `manifest`。不是每个程序都需要这么重，但如果程序声称“崩溃后不产生半成品”，就必须定义这些阶段。

文件系统常见误区

- `write` 成功不等于数据落盘。
- `close` 不是可靠提交协议，错误可能来自之前的 `writeback`。
- `rename` 原子可见不等于目录项已经持久。
- 删除路径名不等于已打开文件对象立即消失。
- `page cache` 不是“可有可无的缓存”，它参与 `mmap`、回收和写回。
- `direct I/O` 绕过 `page cache` 不等于绕过所有一致性问题。

本章小结

- `fd` 是进程局部句柄，`file object` 保存 `offset` 和 `flags`，`inode` 保存文件身份。
- `buffered I/O` 通过 `page cache`，冷/热缓存性能差异巨大。
- 块层负责合并、拆分、排序和 `flush` 顺序。
- 文件系统必须同时维护运行时一致性和崩溃后可恢复性。
- 可靠写出需要明确业务提交点，通常不能只依赖 `write`。

VFS：为什么需要统一文件对象模型

不同文件系统的磁盘布局差异很大，设备文件、管道、`socket` 和普通文件也不是同一种存储对象。用户态却希望都能通过 `open/read/write/close` 操作。VFS 的作用是提供统一对象模型：路径解析得到 `dentry` 和 `inode`；打开后得到 `file object`；具体操作通过 `file operations` 或 `inode operations` 分发给底层实现。

VFS 对象	语义	例子
superblock	一个挂载文件系统实例	ext4、tmpfs、procfs 的挂载点
inode	文件身份和元数据	普通文件、目录、设备节点
dentry	名字到 inode 的缓存关系	<code>/usr/bin/sh</code> 的路径组件
file	一次打开实例	<code>offset</code> 、 <code>flags</code> 、权限和 <code>ops</code>
mount	命名空间中的挂载关系	<code>bind mount</code> 、 <code>overlay</code> 、 <code>chroot</code> 边界

VFS 解释了“一切皆文件”的边界。普通文件的 `read` 走 `page cache`；管道的 `read` 走内存环形缓冲区；`socket` 的 `read` 走网络接收队列；设备节点的 `read` 进入驱动。统一 API 不等于统一实现，调用者仍要根据对象类型处理短读、阻塞、错误和 `readiness`。

路径解析：名字查找不是字符串拼接

路径解析要逐级查找目录项，处理 `...`、符号链接、挂载点、权限、并发 `rename` 和命名空间边界。它不能简单把字符串 `split` 后查哈希表，因为每一级都可能跨文件系统或触发权限检查。

```
path_lookup(start, path):
    current = start
    for component in split(path):
        if component == ".":
            continue
        if component == "..":
```

```

    current = parent_respecting_mount_namespace(current)
    continue
    dentry = lookup_dentry(current, component)
    if dentry.is_symlink:
        path = expand_symlink(dentry, remaining_path)
        restart_with_symlink_limit()
    if crosses_mount(dentry):
        current = mounted_root(dentry)
    else:
        current = dentry.inode
    return current

```

路径查找的失败也要分类。组件不存在返回 `ENOENT`；中间组件不是目录返回 `ENOTDIR`；权限不足返回 `EACCES`；符号链接层数太多返回 `ELOOP`；路径过长返回 `ENAMETOOLONG`。这些错误让调用者知道是创建父目录、修正路径还是请求权限。

目录、rename 和原子可见性

目录是名字到 inode 的映射。它本身也是文件，只是内容由文件系统解释。rename 的关键价值是原子可见：观察者要么看到旧名字，要么看到新名字，不应看到半个名字。这个性质让临时文件发布成为可能。

```

atomic_replace(path, data):
    tmp = create(path + ".tmp")
    write_all(tmp, data)
    fsync(tmp)
    rename(tmp, path)
    fsync(parent_directory(path))

```

这里的原子可见和持久化仍然不同。rename 完成后，其他进程查路径应看到新文件；但断电后目录项是否仍在，要看父目录是否持久化。很多可靠更新协议必须同时关心“运行时读者看到什么”和“崩溃后恢复看到什么”。

page cache 一致性：read、write、mmap

普通 buffered I/O 和文件 mmap 通常共享 page cache。进程 write 修改 page cache 页后，另一个进程通过 mmap 读同一页，应该看到一致内容；反过来，mmap 写共享映射后，read 也应看到页缓存中的新数据。这个共享减少了重复缓存，但增加了一致性规则。

访问方式	数据所在位置	一致性关注点
buffered read	page cache 到用户 buffer	热页不访问设备，冷页触发 I/O
buffered write	用户 buffer 到 page cache	返回不等于落盘，页变 dirty
MAP_SHARED mmap	PTE 映射 page cache 页	写入变 dirty，回写时机独立
MAP_PRIVATE mmap	COW 匿名页	写入不改变文件内容
direct I/O	用户页到设备 DMA	与 page cache 的失效/同步边界

direct I/O 试图绕过 page cache，常用于数据库。它减少双重缓存，但要求处理对齐、短 I/O、设备错误和与 buffered/mmap 混用的一致性。若同一文件同时被 direct write 和 buffered read 访问，内核必须失效或同步相关 page cache 页，否则读者可能看到旧数据。

inode 元数据和数据块分配

文件系统不只保存文件内容，还要保存大小、权限、时间戳、link count、extent 或 block map。写入一个新文件块可能同时改变多个结构：分配位图、inode size、

extent tree、目录项和 journal。

```
append_block(file, data):
    block = allocate_free_block()
    update_allocation_bitmap(block)
    write_data(block, data)
    update_inode_extent(file.inode, block)
    update_inode_size(file.inode)
    mark_metadata_dirty(file.inode)
```

崩溃一致性的难点就在这里。如果位图显示块已分配，但 inode 没有引用它，出现空间泄漏；如果 inode 引用块，但位图仍显示空闲，未来可能把同一块分给另一个文件；如果目录项指向未初始化 inode，路径查找会读到坏对象。日志和写序约束用于避免这些不一致状态在重启后被当成合法事实。

日志文件系统的三种常见模式

日志文件系统把元数据或数据更新先写入 journal，再提交。恢复时根据 commit record redo 完整事务，忽略不完整事务。不同模式在性能和数据保证之间取舍。

模式	写入内容	崩溃后保证
metadata journal	journal 只记录元数据	文件系统结构一致，文件新数据可能旧或空洞
ordered mode	数据先写，再提交元数据 journal	新元数据不会指向未写入垃圾数据
data journal	数据和元数据都写 journal	语义更强，写放大更高

日志不是万能。设备可能重排写入，所以 commit 前后需要 flush/FUA 等顺序保证；journal 空间有限，所以要 checkpoint；checksum 用于发现 torn write；事务太大影响延迟，太小影响吞吐。文件系统设计的核心是把这些成本变成可解释的提交协议。

块调度算法：从磁头到 NVMe

传统磁盘有寻道成本，因此电梯算法会按扇区顺序合并和排序请求，减少磁头来回移动。SSD 和 NVMe 没有机械磁头，但仍有队列深度、并行通道、写放大、flush 成本和公平性问题。块调度从“减少寻道”演进到“控制延迟、合并请求、按 cgroup 分配带宽”。

策略	优点	问题
FCFS	简单，保持提交顺序	随机 I/O 性能差，尾延迟不可控
SSTF SCAN/elevator	优先近距离请求，减少寻道 有方向地扫过磁盘	远处请求可能饥饿 参数依赖介质，交互延迟 仍需控制
deadline BFQ/权重公平	给请求设置过期时间 按进程或 cgroup 分配带宽	吞吐和延迟需要权衡 实现复杂，需识别同步/异步流
mq 调度	多硬件队列并行提交	CPU 亲和性、锁和合并难度增加

现代块层还要处理 flush 和 FUA。普通写请求完成，可能只是设备缓存接收；flush 要求设备把缓存内容推到持久介质；FUA 要求特定写绕过或强制落到稳定介质。文件系统的 fsync 正确性依赖这些语义。

驱动生命周期：probe、open、submit、remove

设备驱动不是只有 read/write。驱动要在设备发现时 probe，分配队列和 DMA 资源；打开设备时建立引用；提交请求时保证 buffer 生命周期；中断或 polling 完成请求；设备移除时拒绝新请求并等待旧请求完成。

```
driver_probe(dev):
    map_mmio(dev)
    allocate_queues(dev)
    register_interrupt(dev)
    register_device_node(dev)

driver_remove(dev):
    mark_dying(dev)
    wake_all_waiters(dev)
    stop_queues(dev)
    wait_inflight_requests(dev)
    unregister_interrupt(dev)
    unmap_mmio(dev)
```

热插拔和错误恢复把生命周期变复杂。用户线程可能正在阻塞读设备时设备被拔出；中断可能在 remove 期间到达；DMA 可能尚未完成。驱动必须有 dying 状态、引用计数、in-flight 计数和唤醒所有等待者的关闭路径。

I/O 错误传播

写回错误可能晚于 write 出现。用户调用 write 时数据进入 page cache，后台 writeback 才发现设备 EIO。内核必须把错误记录在 inode 或 file 的错误状态里，让后续 fsync、close 或再次写入时能报告。

```
writeback_complete(page, status):
    if status == EIO:
        mapping.error = EIO
        mark_page_error(page)
    end_writeback(page)

fsync(file):
    writeback_dirty_pages(file.mapping)
    err = file.mapping.consume_error_since(file.open_time)
    if err:
        return -err
    flush_device()
    return 0
```

这也是为什么可靠程序不能忽略 fsync 和 close 错误。错误可能代表先前看似成功的 buffered write 实际没有持久化。高质量 API 文档应明确错误是在当前调用中发生，还是报告此前异步写回失败。

文件系统测试：不仅测功能，还测崩溃

文件系统测试要覆盖运行时一致性、权限、并发和崩溃恢复。只测试“创建文件后能读回”远远不够。

测试类型	方法	期望
路径错误	构造 ENOENT/ENOTDIR/ELOOP/EACCES	错误码准确，目标不被破坏
引用生命周期	open 后 unlink、dup、fork、close	inode/file 引用计数正确
page cache	buffered read/write/mmap 混用	内容一致，dirty 状态正确
direct I/O	非对齐和混用 buffered I/O	返回明确错误或同步缓存

写回错误	注入设备 EIO	fsync/close 可观察到错误
崩溃恢复	在事务各阶段断电重放	fsck/journal 恢复到合法状态
并发 rename	多线程查找和替换路径	观察到旧或新，不能看到坏中间态

崩溃测试通常需要 fault injection 或模拟块设备：在第 N 次写、flush 或 journal commit 后强制崩溃，然后重挂载检查不变量。没有这类测试，文件系统只能证明正常路径可用，不能证明持久化语义可信。

第 5 章 网络入口与持久化语义

5.1 socket、队列与协议入口

socket、网络入口与内核边界

原理

网络系统会在第三册展开，本章只讲操作系统必须提供的网络入口：socket 为什么是 fd，连接和监听如何变成内核对象，阻塞和非阻塞 I/O 如何与调度器协作，poll/epoll 为什么不是“循环 read”的语法糖。把 socket 放在文件描述符模型里，是 Unix 设计的强大统一：进程用 fd 表引用网络端点，权限、关闭、继承、阻塞、异步通知和事件等待都能复用已有 OS 机制。

socket 与普通文件的差异在于它不是稳定字节数组，而是协议状态机和队列集合。TCP socket 至少包含发送缓冲、接收缓冲、连接状态、重传计时器、拥塞控制状态和等待队列。UDP socket 没有连接字节流，但有端口绑定、报文边界和接收队列。应用看到的是 send/recv，内核维护的是协议栈状态、网卡队列和计时器。

核心思想

socket 是 fd 表中的一个内核对象，但它的可读可写由协议状态、缓冲区、水位线和错误队列共同决定。

网络入口必须先把 OS 契约讲清楚，再进入协议细节。阻塞 recv 在没有数据时会让线程睡在 socket 等待队列；非阻塞 recv 返回 EAGAIN；poll 把“是否可读可写”注册为等待条件；epoll 用就绪队列降低大量 fd 的重复扫描成本。这些都是调度、等待队列和 fd 生命周期的延伸。

实践

最小 socket 子系统可以包含：

对象	作用	不变量
socket fd	进程引用网络端点的句柄	fd 引用有效时 socket refcount 大于零
socket object bind table	保存协议族、类型、状态、ops 本地地址端口到 socket 的映射	状态决定允许的系统调用 同一四元组不能被非法重复绑定
listen queue send buffer	半连接和已完成连接队列 用户写入但未发送或未确认的数据	backlog 限制必须生效 字节数受 SO_SNDBUF 限制
receive buffer	已到达但未被用户读取的数据	可读事件与缓冲水位一致

典型 TCP 服务端路径：

```
s = socket(AF_INET, SOCK_STREAM)
bind(s, 0.0.0.0:8080)
listen(s, backlog=128)
while true:
    c = accept(s)
    handle_client(c)
```

内核对应状态是：`socket` 创建未绑定对象；`bind` 加入端口表；`listen` 把对象转成监听状态并初始化队列；三次握手完成后，新连接进入 `accept queue`；`accept` 取出一个已完成连接，创建新的 `fd` 指向新的 `connected socket`。监听 `socket` 和连接 `socket` 不是同一个对象。

算法与实现分析

`bind` 冲突检查

端口绑定要考虑地址、端口、协议、`reuse` 标志和 `namespace`。教学模型可以先实现严格唯一：

```
bind(sock, local_addr):
    lock(bind_table.lock)
    if bind_table.contains(sock.netns, sock.proto, local_addr):
        unlock(bind_table.lock)
        return EADDRINUSE
    sock.local = local_addr
    sock.state = BOUND
    bind_table.insert(sock)
    unlock(bind_table.lock)
    return OK
```

真实系统的 `SO_REUSEADDR` 和 `SO_REUSEPORT` 会放宽规则，但不能简化成“允许重复”。复用规则还要区分监听 `socket`、已连接四元组、`TIME_WAIT` 和负载均衡策略。先掌握严格唯一模型，再扩展复用规则，能避免把安全边界讲错。

`listen` 与 `accept` 队列

监听 `socket` 有两个不同队列：握手未完成的半连接队列，以及握手完成等待用户 `accept` 的队列。简化模型可以只保留完成队列：

```
tcp_connection_established(listener, child):
    lock(listener.lock)
    if listener.acceptq.len == listener.backlog:
        drop_or_reset(child)
    else:
        enqueue(listener.acceptq, child)
        wake_waiters(listener.read_waitq)
    unlock(listener.lock)

accept(listener):
    lock(listener.lock)
    while empty(listener.acceptq):
        if listener.nonblock:
            unlock(listener.lock)
            return EAGAIN
        sleep_on(listener.read_waitq, listener.lock)
    child = dequeue(listener.acceptq)
    fd = install_fd(child)
    unlock(listener.lock)
    return fd
```

不变量是：`accept queue` 中的 `child socket` 已经是 `connected` 状态；队列非空时监听 `socket` 对 `poll` 表现为可读；关闭监听 `socket` 必须唤醒等待 `accept` 的线程并让它们返回错误。

`send/recv` 与背压

发送路径不是把用户数据直接交给网卡。数据先进入 `socket send buffer`，再由协议栈分段、排队、重传和确认。

```
send(sock, user_buf, len):
```

```

copied = 0
lock(sock.lock)
while copied < len:
    space = sock.sndbuf_limit - sock.sndbuf_used
    if space == 0:
        if sock.nonblock:
            break_or_EAGAIN(copied)
        sleep_on(sock.write_waitq, sock.lock)
        continue
    n = min(space, len - copied)
    copy_from_user_into_skb(sock, user_buf + copied, n)
    copied += n
    tcp_push_pending(sock)
unlock(sock.lock)
return copied

```

这段伪代码解释短写：阻塞 socket 通常尽量等待空间，非阻塞 socket 在缓冲区满时可能只写入部分数据或返回 `EAGAIN`。接收路径类似：接收缓冲为空时阻塞或返回 `EAGAIN`；对 TCP 返回字节流，对 UDP 每次读取保留报文边界语义。

poll 与 epoll

朴素 poll 每次扫描 fd 数组，复杂度 $O(n)$ ：

```

poll(fds, timeout):
    ready = []
    for fd in fds:
        mask = fd.file.ops.poll(fd.file)
        if mask != 0:
            ready.append((fd, mask))
    if ready not empty:
        return ready
    register_waiters(fds, current)
    schedule_timeout(timeout)
    retry scan

```

epoll 把关注集合保存在内核中。当 socket 状态变化时，回调把对应 epitem 放入 ready list，用户调用 `epoll_wait` 主要消费 ready list。这样大量空闲连接不会每次都被全量扫描。

```

socket_state_change(sock):
    for epitem in sock.poll_subscribers:
        if sock.poll_mask & epitem.interest:
            enqueue_once(epitem.epoll.ready_list, epitem)
            wake_waiters(epitem.epoll.waitq)

```

边沿触发和水平触发的区别也来自 ready 条件。水平触发下，只要缓冲区仍有数据，下一次等待还会报告；边沿触发只在状态从不可读变可读时报告，用户必须一直读到 `EAGAIN`，否则可能再也收不到事件。

TCP 状态边界

操作系统教材不必在本册完整推导 TCP，但要能解释 socket 系统调用面对的状态：

状态	用户可见行为	内核重点
LISTEN	可 accept, 不能普通 recv 数据	accept queue、backlog、SYN 处理
SYN-SENT	connect 进行中	超时、错误返回、非阻塞完成事件

ESTABLISHED	可 send/recv	缓冲、重传、拥塞控制、窗口
FIN-WAIT/CLOSE-WAIT	等待关闭	EOF、shutdown、剩余数据读取
TIME-WAIT	fd 通常已关但四元组保留	防旧包污染新连接

测试

1. fd 生命周期：dup socket 后关闭一个 fd，连接对象不能提前释放。
2. bind 冲突：同一地址端口重复绑定应失败，释放后可再次绑定。
3. listen backlog：超过 backlog 的完成连接必须被拒绝或延迟，不能无限增长。
4. accept 阻塞：无连接时阻塞，有连接到达后唤醒；非阻塞返回 `EAGAIN`。
5. send 背压：发送缓冲满时阻塞或短写，空间释放后唤醒写等待者。
6. recv EOF：对端关闭后，已缓冲数据读完再返回 EOF。
7. poll 水平触发：缓冲仍有数据时重复报告可读。
8. epoll 边沿触发：未读尽数据时不会凭空产生新边沿事件。

本章合格标准：能说明 socket 为什么是 fd；能画出 bind/listen/accept/connect/send/recv 的对象变化；能解释阻塞、非阻塞、poll、epoll 的等待语义。

源码级补充：sk_buff、协议栈入口、NAPI 与 Netfilter

sk_buff 是网络栈的“页”和“bio”

Linux 网络栈中，sk_buff 是数据包在内核中的主要载体。它不仅保存字节，还保存协议头指针、校验和状态、路由信息、引用计数、分片信息和共享数据区。理解 skb，才能把驱动、协议栈和 socket 接起来。

```
sk_buff:
    head, data, tail, end
    len, data_len
    mac_header, network_header, transport_header
    dev, sk
    checksum state
    shared_info: fragments, gso, refcount
```

skb_clone 共享数据但复制元数据，skb_copy 复制数据，kfree_skb 释放引用。性能问题常来自不必要 copy；正确性问题常来自共享 skb 被误写、引用计数错误或线性区和 fragment 区理解错误。

接收路径：从 NAPI 到 socket 队列

高吞吐网络接收通常走 NAPI。中断只负责调度 poll，poll 在 budget 内批量取包，把 skb 送入协议栈。

```
nic_interrupt:
    disable_rx_interrupt()
    napi_schedule()

napi_poll(budget):
    while work < budget and rx_desc_ready:
        skb = build_skb_from_rx_desc()
```

```

napi_gro_receive(skb)
if no_more_work:
    napi_complete()
    enable_rx_interrupt()

ip_rcv -> tcp_v4_rcv -> tcp_v4_do_rcv
-> append to socket receive queue
-> wake waiters

```

GRO 可以把多个小包合并成较大 skb，降低协议栈成本；但它也改变了观测粒度。抓包、trace 和应用 recv 的边界不一定一一对应。测试网络栈时要区分 wire packet、skb、TCP segment 和应用读出的字节。

发送路径：tcp_sendmsg 到驱动队列

发送路径从用户 buffer 复制或引用到 skb，进入 TCP 发送队列，按拥塞窗口、接收窗口和 MSS 分段，最后到 qdisc 和驱动。

```

sendmsg
-> inet_sendmsg
-> tcp_sendmsg
-> copy into skb or attach pages
-> tcp_push_pending_frames
-> ip_queue_xmit
-> qdisc enqueue/dequeue
-> dev_queue_xmit
-> driver ndo_start_xmit

```

这个路径解释了 send 成功的边界：数据进入本机 TCP 发送状态，不等于对端应用收到，更不等于业务提交。若连接稍后 reset，应用必须根据协议状态判断请求是否需要重试。

Netfilter hook 点

Netfilter 在 IP 路径上提供 hook：PRE_ROUTING、LOCAL_IN、FORWARD、LOCAL_OUT、POST_ROUTING。防火墙、NAT、容器端口映射和连接跟踪都依赖这些点。

hook	典型用途
NF_INET_PRE_ROUTING	入站后、路由前，DNAT、早期过滤
NF_INET_LOCAL_IN	目的地是本机，进入 socket 前过滤
NF_INET_FORWARD	转发路径过滤
NF_INET_LOCAL_OUT	本机发出包，路由后处理
NF_INET_POST_ROUTING	出站前，SNAT、最后修改

Netfilter 让网络语义不仅由 socket 代码决定，还受规则表、namespace、连接跟踪和 NAT 影响。诊断“包为什么没到应用”时，要检查驱动、IP、路由、Netfilter、socket 绑定和应用读取，而不是只看一个层次。

socket option 与内核策略

socket option 是应用调整内核协议策略的入口。例如 SO_RCVBUF 和 SO_SNDBUF 控制缓冲区，TCP_NODELAY 影响 Nagle，SO_REUSEPORT 允许多个监听 socket 分担连接，SO_KEEPALIVE 打开保活探测。

选项	影响	风险
----	----	----

TCP_NODELAY	小包立即发送	更多包，可能降低吞吐
SO_REUSEPORT	多 socket 绑定同端口	负载分配和安全检查更复杂
SO_RCVBUF	接收缓冲上限	太小丢吞吐，太大耗内存
SO_LINGER	close 时处理未发送数据	可能阻塞或丢弃

选项不是调优魔法。每个选项都改变状态机和资源边界，必须通过负载测试和故障测试验证。

connect 的阻塞、非阻塞和错误取回

connect 不是简单地“连上或失败”。阻塞 TCP connect 会发起握手并睡眠等待结果；非阻塞 connect 通常返回 EINPROGRESS，之后用户用 poll/epoll 等待可写，再用 getsockopt(SO_ERROR) 取真正结果。可写事件只表示连接状态有变化，不表示一定成功。

```
nonblocking_connect(sock, addr):
    ret = connect(sock, addr)
    if ret == 0:
        connected
    else if errno == EINPROGRESS:
        wait EPOLLOUT
        err = getsockopt(sock, SOL_SOCKET, SO_ERROR)
        if err == 0:
            connected
        else:
            connect failed with err
    else:
        immediate failure
```

程序只看到 EPOLLOUT 就开始发送，是常见错误。连接拒绝、超时、路由不可达都可能通过 socket 错误状态返回。内核把异步错误挂在 socket 上，应用必须取走并处理。

recv 返回值的含义

TCP 是字节流，recv 的返回值表达当前 socket 状态，不表达业务协议状态。

结果	内核含义	应用动作
n > 0	从接收队列取出 n 字节	放入协议解析缓冲，继续解析消息边界
0	对端有序关闭，且本端已读完缓冲数据	处理 EOF，不能再期待新字节
-EAGAIN	非阻塞 socket 暂无数据	等待下一次可读事件
-EINTR	被信号打断	根据业务决定重试或退出
-ECONNRESET	连接被复位	丢弃未完成请求，按协议决定是否重试

EOF 不等于错误，reset 也不等于普通 EOF。若对端发送 FIN，已经在接收缓冲里的数据仍可读；读完后才返回 0。若收到 RST，未完成的字节流不再可信，很多协议必须把当前请求标成结果未知。

TCP 不是消息队列

应用写入两次，接收端可能一次读到合并后的字节；应用写入一次，接收端可能分多次读到。TCP 保证有序字节流，不保留 `write` 调用边界。业务协议必须显式 framing。

```
length_prefixed_protocol:
  read bytes into buffer
  while buffer has at least 4 bytes:
    len = parse_u32(buffer[0:4])
    if buffer has less than 4 + len:
      break
    message = buffer[4:4+len]
    dispatch(message)
    remove consumed bytes
```

这个状态机必须处理半包、粘包、超长长度、恶意长度、连接关闭和超时。把一次 `recv` 当成一条完整消息，是 TCP 服务端最常见的教学级错误。

Nagle、delayed ACK 和小包延迟

`TCP_NODELAY` 关闭 Nagle 后，小写入更快发出，但包数增加；Nagle 打开时，小包可能等待未确认数据；接收端 delayed ACK 又可能延迟确认。两者组合会让“写了几个字节为什么几十毫秒后才到”变成真实问题。

机制	目的	可能副作用
Nagle	合并小包，减少网络开销	请求/响应小消息延迟上升
Delayed ACK	减少纯 ACK 包	与 Nagle 组合造成等待包数和 CPU 开销上升
<code>TCP_NODELAY</code>	降低小消息等待	忘记 <code>uncork</code> 会拖延发送
<code>TCP_CORK</code>	手动聚合发送	

调优不能只背“低延迟就开 `TCP_NODELAY`”。如果协议能批量发送，保留聚合可能更好；如果是交互 RPC，小消息阻塞会直接进入尾延迟。

超时、重试和幂等不由 socket 自动解决

socket 超时只能告诉应用某个等待没有在期限内完成，不能告诉远端是否处理了请求。发送超时、接收超时、连接 `reset`、进程崩溃都可能让请求处在“可能提交、可能未提交”的灰区。应用协议要用请求 ID、幂等键、事务日志或去重表解决语义。

```
rpc_with_idempotency:
  id = allocate_request_id()
  send(id, operation, payload)
  if timeout:
    reconnect_or_retry(id)
  server:
    if id already committed:
      return old result
    execute operation
    record result by id
    return result
```

OS 负责提供字节流、错误码、关闭语义和等待机制；“这笔业务是否执行过”是应用协议层的责任。网络章节若不讲这个边界，读者会把 TCP 可靠传输误解成业务可靠提交。

5.2 网络路径与背压

网络与持久化深讲：流、事件和提交

网络和持久化看起来相距很远，一个处理远端通信，一个处理本地存储。但它们共享几个 OS 问题：缓冲区所有权、阻塞与唤醒、错误传播、部分完成、重试与幂等。网络 send 可能短写，磁盘 write 可能只进 page cache；TCP 连接可能 reset，块设备写入可能返回 EIO；应用都必须写出可恢复状态机。

TCP 字节流为什么没有消息边界

TCP 提供可靠有序字节流，不提供应用消息边界。一次 send 的 100 字节，接收方可能分两次 recv 到 40 和 60 字节，也可能和下一次 send 合并读到。应用协议必须自己定义长度、分隔符或帧格式。

```
read_frame(sock):
    while buffer.len < HEADER_SIZE:
        recv_more(sock, buffer)
    len = parse_length(buffer.header)
    while buffer.len < HEADER_SIZE + len:
        recv_more(sock, buffer)
    return extract_frame(buffer)
```

这解释了为什么 readiness 不是消息。epoll 告诉你 socket 可能可读，不告诉你完整帧已经到达。若把一次可读事件当成完整请求，协议解析会在高负载或网络分片时失败。

网络背压

发送缓冲区、接收缓冲区、协议队列和用户态队列都必须有限。若应用生产速度超过网络发送速度，队列会增长。正确系统要让生产者感受到背压，而不是无限缓存。

```
send_with_backpressure(sock, data):
    while data not empty:
        n = send(sock, data)
        if n > 0:
            data = data[n:]
        else if errno == EAGAIN:
            wait_for_writable(sock)
        else:
            fail_connection(errno)
```

背压也适用于持久化。后台写盘跟不上，dirty page 不能无限增长；内核会限制脏页比例，让写入者参与 writeback 或阻塞。这样牺牲局部延迟，换取系统整体不被内存耗尽。

提交协议和幂等恢复

网络服务通常要处理“请求已经执行但响应丢失”的情况。持久化系统要处理“数据已经写一部分但 commit 记录没写”的情况。二者都需要幂等标识。

```
request:
    client_id
    request_id
    operation
    payload

server:
    if request_id already committed:
        return previous_result
    prepare operation
    commit durable record
    return result
```

幂等不是“重复执行也没事”这么简单，而是系统能识别重复请求，并返回同一个已提交结果。没有 request id，网络重试可能造成重复扣款、重复写文件或重复发

布版本。

断电恢复和网络重试的共同思维

两者都要求你列出中间状态。

领域	中间状态	恢复规则
网络请求	received、processing、committed、replied	committed 后重复请求返回旧结果
文件输出	temp、synced、renamed、manifest	manifest 后才被读取者信任
日志事务	intent、data written、commit	commit 存在则 redo, 不存在则忽略

学习 OS 时要养成这种状态枚举习惯。它比背诵某个 API 更接近工程正确性。

网络与持久化练习

1. 为什么 TCP 服务端不能假设一次 `recv` 得到一个完整请求？
2. 设计一个带 request id 的幂等接口，说明响应丢失后客户端如何重试。
3. 比较 socket send buffer 背压和 dirty page 背压的共同点。
4. 写出 manifest 提交前后崩溃的恢复规则。
5. 为什么 commit record 需要 checksum 和 `fsync`？

网络错误和文件错误的相似性

网络错误常被误解成“连接断了”，文件错误常被误解成“磁盘坏了”。实际上二者都需要精细分类。

领域	错误	处理
socket	<code>EAGAIN</code>	等可写/可读事件后继续
socket	connection reset	丢弃连接状态，决定请求是否可重试
socket	short write	保存剩余 buffer，下次继续发送
文件	short write	循环写或上报部分完成
文件	<code>fsync</code> failure	业务提交失败，不能发布 manifest
日志	checksum mismatch	停止恢复到最后完整记录

共同原则是：先判断是否部分完成，再判断是否可重试，再判断重试是否幂等。没有这个顺序，网络服务和持久化系统都会在故障时产生重复、丢失或半提交。

案例：上传文件服务

上传文件服务同时涉及网络 and 持久化。服务端从 socket 接收字节，写入临时文件，计算 checksum，`fsync`，`rename`，更新 manifest，再返回成功。任何一步失败都要有恢复规则。

```
upload(request_id, socket):
    tmp = create_tmp(request_id)
    while not end_of_body:
        bytes = recv(socket)
        if bytes.error: abort_tmp(tmp)
        write_all(tmp, bytes)
        update_checksum(bytes)
```

```
fsync(tmp)
rename(tmp, final_name(request_id))
fsync(parent_dir)
commit_manifest(request_id, final_name, checksum)
send_success(socket)
```

若成功响应丢失，客户端可能重试同一个 request id。服务端必须识别 manifest 中已有提交，返回同一结果，而不是再创建第二个文件。这就是网络幂等和持久化提交的交叉点。

本章小结

- TCP 是字节流，应用协议必须自己定义消息边界。
- 网络和存储都可能部分完成，调用者必须保存状态继续推进。
- 背压是系统稳定性的机制，不是性能失败。
- 幂等 request id 能把网络重试和持久化提交连接起来。
- 日志、manifest 和 checksum 让崩溃恢复有可判定事实。

网络栈分层：从网卡帧到 socket

网卡收到的是链路层帧，用户程序读到的是 socket 字节或数据报。中间要经过驱动、软中断或 polling、链路层解析、IP 层路由和分片处理、传输层状态机、socket 缓冲区和唤醒机制。

```
NIC DMA frame
-> driver RX ring
-> network poll budget
-> Ethernet header parse
-> IP validation and routing
-> TCP/UDP demux by ports
-> socket receive queue
-> wake epoll/read waiter
```

每层都有丢弃理由。网卡 ring 满会丢包；校验和错会丢包；IP TTL 为 0 会丢包；没有匹配 socket 会发 RST 或 ICMP；socket receive buffer 满会产生背压或丢弃。网络栈不是“把数据送给进程”，而是一系列有限队列和状态机。

UDP 和 TCP 的不同契约

UDP 保留报文边界，但不保证可靠、有序、去重和拥塞控制；TCP 保证可靠有序字节流，但不保留消息边界。应用选择协议时，实际上是在选择哪些语义由内核提供，哪些语义由应用自己实现。

协议	提供	不提供
UDP	数据报边界、端口复用、校验和	重传、排序、拥塞控制、连接状态
TCP	可靠有序字节流、流量控制、拥塞控制	应用消息边界、请求幂等、业务提交
QUIC 类协议	用户态连接、流、多路复用、加密集成	仍需应用定义业务语义和提交规则

UDP 适合应用能容忍丢包或自己实现重传的场景，例如实时音视频、查询协议或自定义传输。TCP 适合需要可靠字节流的场景，但应用仍要处理半关闭、reset、短读短写和重复请求。

TCP 状态机和关闭语义

TCP 连接有状态：LISTEN、SYN-SENT、ESTABLISHED、FIN-WAIT、CLOSE-WAIT、TIME-WAIT 等。关闭不是单个动作，而是双向字节流各自结束。收到 FIN 表示对方不再发送数据，但本端仍可发送；reset 表示连接异常终止，未读或未确认数据可能丢失。

事件	socket 读写表现	应用含义
peer FIN	<code>recv</code> 返回 0	对方正常关闭发送方向
local shutdown write	本端发送 FIN	告诉对方本端不再发送
RST	<code>ECONNRESET</code> 等错误	连接状态丢弃，业务是否完成未知
TIME_WAIT	连接四元组暂时保留	吸收迟到报文，避免旧连接污染新连接

服务端常见 bug 是把 `recv` 返回 0 当成错误，或把 `send` 成功当成对方业务已经处理。TCP 只能说明字节进入本机协议栈或被对端确认到传输层，不能说明远端应用已经提交业务。

流量控制和拥塞控制

流量控制保护接收方：接收窗口告诉发送方还能接收多少字节。拥塞控制保护网络：发送方根据丢包、延迟或 ACK 估计网络可承受速率。二者都能让 `send` 变慢或返回 `EAGAIN`，但原因不同。

```
tcp_send_loop:
while app_has_data:
    if send_buffer_full:
        wait_app_backpressure()
    if receiver_window_zero:
        wait_window_update()
    if congestion_window_limited:
        wait_ack_or_timer()
    transmit_allowed_segments()
```

这解释了为什么“网络慢”不能只看带宽。可能是接收方应用不读导致窗口关闭；可能是中间网络拥塞导致拥塞窗口缩小；可能是本机 socket buffer 太小；可能是 Nagle、delayed ACK 或小包策略影响延迟。诊断要看队列、窗口、重传、RTT 和应用读写速率。

epoll/readiness：事件不是工作完成

readiness API 告诉应用“某 fd 可能不会阻塞”，而不是“完整请求已经准备好”。边缘触发模式下，应用必须一直读到 `EAGAIN`，否则可能错过后续事件；水平触发模式下，只要条件仍成立，事件会重复出现。

```
on_readable(fd):
while true:
    n = recv(fd, buf)
    if n > 0:
        parser.feed(buf[0:n])
        while parser.has_frame():
            handle_frame(parser.next_frame())
    else if n == 0:
        close_connection(fd)
        break
    else if errno == EAGAIN:
        break
    else:
        fail_connection(fd, errno)
```

break

事件循环必须把连接状态保存在对象里：已读但未解析的字节、待发送但未写完的响应、超时时间、是否半关闭、request id 和持久化提交状态。没有这些状态，短读短写一出现，程序就会丢帧或重复发送。

accept 队列、listen backlog 和过载

服务器调用 `listen(backlog)` 后，内核维护连接建立相关队列。过载时，SYN 队列、accept 队列、应用线程池、socket buffer 或业务队列都可能成为瓶颈。只调大一个 backlog 不一定解决问题。

队列	堵塞表现	对策
SYN 队列	新连接握手失败或重传	SYN cookie、限流、扩容
accept 队列	连接已建立但应用未 accept	更快 accept、多进程、backlog 调整
worker 队列	已接收请求等待处理	背压、负载均衡、降级
send buffer	响应写不出去	非阻塞写、限速慢客户端
持久化队列	fsync 或日志提交积压	批量提交、限流、隔离磁盘

一个健壮服务会在每层设置上限，并在上限触发时明确拒绝或降级。无限 accept、无限读 body、无限缓存响应，都会把远端慢客户端或恶意客户端变成本机内存问题。

持久化日志：redo、undo 和幂等

崩溃恢复常见两类思路。redo 日志记录“提交后应当产生的结果”，恢复时重做已提交事务；undo 日志记录“修改前的旧值”，未提交事务崩溃后回滚。实际数据库可能使用更复杂的 WAL，但核心仍是先写可恢复证据，再改变主数据。

日志类型	写入顺序	恢复规则
redo	日志 intent/data 先持久，commit 后主数据可稍后写	有 commit 则 redo，无 commit 忽略
undo	旧值日志先持久，再覆盖主数据	无 commit 则 undo，有 commit 保留
manifest	内容文件先完整持久，再发布版本记录	只采用 manifest 中可校验版本

幂等 request id 与日志结合后，服务才能处理响应丢失。commit record 中记录 request id 和结果摘要；客户端重试时，服务查到已提交记录，直接返回同一结果。没有这层记录，重试只能变成“再执行一次”，这对扣款、发货、发布版本等操作通常是错误的。

批量提交和延迟权衡

每个请求都单独 `fsync`，语义清楚但吞吐受 flush 延迟限制。批量提交把多个请求放进同一次 journal commit 或 group commit，摊薄 flush 成本，但会增加单个请求等待批次的延迟。

```
group_commit_loop:
    wait_until(batch_not_empty or timeout)
    batch = collect_pending_requests(max_batch)
    append_records(batch)
    fsync(log)
    for req in batch:
        mark_committed(req)
        wake_waiter(req)
```


批量提交的正确性条件是：只有 `fsync` 成功后才能向这些请求返回 `committed`；`fsync` 失败时，整个批次都必须失败或进入恢复状态；批次中的每条记录要有长度、checksum 和 request id，恢复时能跳过不完整尾部。

网络服务的端到端状态机

把网络 and 持久化合起来，一个请求至少经过这些状态：RECEIVING、PARSED、VALIDATED、EXECUTING、LOGGED、COMMITTED、REPLYING、DONE。错误和重试必须能从每个状态解释。

状态	持有资源	失败处理
RECEIVING	socket buffer、解析缓冲区	连接断开则丢弃未完整请求
VALIDATED	请求对象、权限结果	拒绝时不改变持久状态
EXECUTING	内存中业务变更或临时文件	失败则回滚临时资源
LOGGED	WAL/manifest 候选记录	未 commit 崩溃后忽略或 undo
COMMITTED	持久 commit record	重试返回同一结果
REPLYING	待发送响应 buffer	发送失败不撤销已提交业务

最后一行尤其重要。业务已经 committed 后，响应发送失败不能回滚业务，因为客户端可能稍后重试并期望得到同一结果。应用协议必须把“业务提交”和“响应送达”分开。

网络与持久化测试矩阵

测试	注入故障	期望
TCP 分片 短写	每次只发送 1 字节 send 只接受部分响应	parser 仍能组帧 状态机保存剩余字节
连接 reset	commit 前/后分别 reset	前者不提交，后者可 幂等查询
重复 request id	响应丢失后重试	返回同一 committed 结果
fsync 失败	日志提交阶段返回 EIO	请求失败，不能发布 commit
崩溃恢复	commit 前/后断电	前者忽略，后者 redo/可查询
背压	慢客户端和慢磁盘同时出现	队列有上限，系统不 OOM

这些测试能证明服务不是只在理想网络和理想磁盘下正确，而是在短读短写、重复、断线、断电和过载下仍然有清晰语义。

5.3 持久化、日志与恢复

持久化、崩溃一致性与恢复

原理

持久化问题的核心是：系统可能在任意两步之间崩溃。程序调用 `write`，内核可能只把数据放进 page cache；设备可能只完成了一部分请求；目录项和文件内容可

能以不同顺序持久化；应用日志可能写了意图但没有应用数据；manifest 可能发布了一个并不完整的输出。崩溃一致性讨论的不是“正常运行时看起来对不对”，而是“重启后能否解释磁盘状态”。

write、fsync、rename 和 manifest 分别处在不同层次。write 把字节交给内核；fsync 推进文件内容和部分元数据的持久化；rename 可以提供目录项替换的原子可见性，但目录本身是否持久也需要考虑；manifest 是业务层承认某批输出有效的提交点。只有把这些层次分开，才能写可靠输出。

文件系统常用日志或 copy-on-write 技术维护自己的元数据一致性，但这不自动等于应用数据一致。文件系统保证的是它自己的结构能恢复，不保证你的两个文件之间满足业务事务。应用如果需要“数据文件和索引文件同时生效”，就必须设计应用层提交协议。

最小可靠写出协议通常是：写临时文件，写完后 fsync 临时文件，校验大小和 checksum，rename 到最终路径，fsync 父目录，最后写或更新 manifest。这个协议不是唯一答案，但它展示了 prepared 和 committed 的区别。

日志恢复要区分 redo 和 undo。redo 日志记录“应该完成什么”，崩溃后可以重放；undo 日志记录“如何撤销未完成修改”。write-ahead logging 的关键规则是：描述修改的日志必须先于实际修改持久化，否则崩溃后无法判断磁盘上的半成品是否应该保留。

实践

纸上实践先写提交状态机：

```
NoOutput
-> TempCreated
-> DataWritten
-> DataSynced
-> RenamedVisible
-> DirectorySynced
-> ManifestCommitted
```

每个状态都要写出崩溃后恢复规则：

崩溃点	恢复策略
TempCreated	删除临时文件或忽略
DataWritten	校验失败则删除，不能发布
DataSynced	仍未提交，等待 rename 或清理
RenamedVisible	若 manifest 未提交，按业务规则决定是否采用
DirectorySynced	文件名持久，但业务仍以 manifest 为准
ManifestCommitted	读取者可以信任并校验文件

实践中要避免把“文件存在”当成业务成功。目录中有最终文件名，只说明某次 rename 可能发生过；manifest 提交并通过 checksum，才说明业务层承认它属于某个版本或批次。

再写一份日志协议：

```
append log record: transaction T intends to write block B with checksum C
fsync log
write data block
fsync data
append commit record for T
fsync log
```

恢复时扫描日志：有 commit 的事务必须 redo；没有 commit 的事务按规则忽略或 undo；checksum 不匹配的记录停止或进入人工恢复。这里的重点不是实现数

据库，而是理解日志先行、提交记录和幂等恢复。

算法与实现分析

原子 rename 发布

很多应用用临时文件加 rename 发布结果。算法分为写入、同步、替换、同步目录四步：

```
write_atomic(path, bytes):
    tmp = path + ".tmp." + unique_id()
    fd = open(tmp, O_CREAT | O_EXCL | O_WRONLY)
    write_all(fd, bytes)
    fsync(fd)
    close(fd)
    rename(tmp, path)
    dirfd = open(parent_dir(path))
    fsync(dirfd)
    close(dirfd)
```

这个算法保证读者要么看到旧文件，要么看到新文件，不应看到半个最终文件。但它仍然只是单文件发布。若业务需要多个文件同时生效，还需要 manifest 或事务日志。

manifest 提交

manifest 把多个输出文件绑定成一个版本。数据文件可以先各自写好并 fsync；manifest 最后写入，列出版本号、文件名、大小和 checksum。读取者只读取 manifest 指向的文件。

```
commit_version(version, files):
    for file in files:
        write_atomic(file.tmp_path, file.bytes)
        verify_checksum(file.tmp_path)
        rename(file.tmp_path, file.final_path)
    manifest = build_manifest(version, files)
    write_atomic("MANIFEST.new", manifest)
    rename("MANIFEST.new", "MANIFEST")
```

manifest 的优点是恢复简单：重启后读取 MANIFEST，把它当作权威事实；不在 manifest 里的临时文件或孤儿文件可以清理。缺点是提交粒度较粗，manifest 本身也需要可靠写出。

redo 日志

redo 日志适合“提交后必须能重放”的场景。日志记录足够的信息，使恢复程序可以重新执行已经提交但未完全应用的修改。日志记录必须先持久化，数据页才能写出。

```
transaction_write(T, block, data):
    log_append(INTENT, T, block, checksum(data))
    fsync(log)
    write(block, data)
    log_append(COMMIT, T)
    fsync(log)
```

恢复算法扫描日志：看到 INTENT 但没有 COMMIT 的事务不应用；看到 COMMIT 的事务检查数据是否已经应用，未应用则 redo。redo 操作必须幂等，即重复执行不会改变最终正确结果。

检查点

日志无限增长不可接受，所以系统要做 checkpoint。checkpoint 把当前稳定状态写成快照，并记录“恢复只需从这个日志位置之后开始”。算法难点是 checkpoint

期间仍可能有新事务进入，因此要记录 epoch 或 LSN。

```
checkpoint():
    begin_lsn = log.current_lsn()
    flush_dirty_pages_up_to(begin_lsn)
    write_checkpoint_record(begin_lsn)
    fsync(log)
    truncate_log_before(begin_lsn)
```

checkpoint 的正确性依赖顺序：只有当某个 LSN 之前的脏数据都已经安全写出，才能截断之前的日志。否则崩溃后会丢失 redo 所需信息。

测试

持久化测试必须做故障注入。只在正常路径写文件再读取，不足以证明崩溃一致性。

- 在每个提交状态后模拟崩溃，恢复程序能给出确定结果。
- 临时文件残留不会被读取者当成有效输出。
- manifest 指向缺失文件时，读取者拒绝该版本。
- checksum 错误时，恢复程序能定位坏文件并拒绝提交。
- 重复执行恢复程序是幂等的，不会一次恢复成功、二次恢复破坏状态。
- write 成功但 fsync 失败时，业务提交必须失败。
- rename 后但 manifest 前崩溃，恢复策略必须明确。

测试报告不要只写“断电恢复成功”。必须列出崩溃点、磁盘上允许出现的文件集合、恢复后权威事实、被清理对象和未覆盖边界。可靠性来自状态枚举，不来自运气。

源码级补充：WAL、COW 文件系统、RAID、LVM 与 media failure

崩溃模型分层

崩溃一致性至少有三类故障：transaction crash，系统在两步之间断电；system crash，内核或整机停止但介质仍可靠；media failure，介质本身丢失或返回坏数据。日志能处理前两类的一部分，但不能让坏盘自动恢复数据，后者需要冗余、校验和副本。

故障	例子	需要的机制
transaction crash	commit 前断电	WAL/journal/manifest 状态机
system crash	内核 panic、掉电	fsync、flush、replay、fsck
media failure	坏块、整盘损坏	RAID、复制、checksum、备份

LSN 与 checkpoint

WAL 系统通常给每条日志记录 LSN。数据页记录 pageLSN，表示该页包含到哪个日志位置的修改。恢复时若 pageLSN 小于 committed LSN，就 redo；checkpoint 记录某个 LSN 之前的脏页已经安全传播。

```
redo_recovery():
    start = last_checkpoint_lsn
    for record in log from start:
        if record.committed and page.page_lsn < record.lsn:
```

```
    apply(record)
    page.page_lsn = record.lsn
```

LSN 的价值是让恢复可判定，而不是靠文件时间戳或猜测。数据库、文件系统 journal 和 manifest 版本号都在解决同一问题：给恢复程序一个可比较的事实。

COW 文件系统

btrfs、ZFS 等 COW 文件系统不在原地覆盖元数据，而是写新块，再用新的根指针发布版本。snapshot、clone、send/receive 都建立在“旧版本块仍被引用，新版本写到新位置”的基础上。

```
cow_update(tree, key, value):
    new_leaf = copy_and_modify(old_leaf)
    new_parent = copy_and_point_to(new_leaf)
    ...
    write_all_new_blocks()
    atomically_update_super_root(new_root)
```

COW 避免部分原地覆盖造成的结构破坏，但带来写放大、碎片和空间回收复杂度。snapshot 多时，一个旧块可能被许多版本引用，不能简单释放。

RAID 与 write hole

RAID 0 条带化提高吞吐但无冗余；RAID 1 镜像；RAID 5/6 用奇偶校验节省空间；RAID 10 结合镜像和条带。RAID 5 写入数据和 parity 时若断电，可能出现 write hole：数据和校验不一致。

级别	优点	风险
RAID0	高吞吐、高容量利用	任一盘坏全丢
RAID1	简单冗余，读可并行	容量减半
RAID5/6	容量效率高，有校验	小写放大、write hole、重建压力
RAID10	性能和可靠性较好	成本高

write hole 可用日志、非易失缓存、bitmap 或 copy-on-write parity 缓解。这里再次说明：持久化正确性来自顺序证据和冗余，不来自“写了两次应该差不多”。

LVM 和 device mapper

Linux device mapper 把上层块设备映射到底层设备区域，LVM 的 linear、snapshot、thin provisioning 都建立在此之上。dm-snapshot 用 COW 保存旧块，dm-thin 动态分配块，带来空间耗尽时的错误传播问题。

```
bio to /dev/mapper/vg-lv
-> dm target maps logical sector
-> lower device sector(s)
-> submit cloned bio
-> complete original bio after all lower bios complete
```

读 `drivers/md/dm.c` 和具体 target。关注 bio clone、错误合并、flush 传播和 snapshot 元数据一致性。

第 6 章 安全、错误恢复与可观测性

6.1 隔离边界与能力收敛

安全、隔离与最小权限

原理

操作系统的隔离不是一个单独功能，而是贯穿 CPU、内存、文件、进程、设备和网络的约束集合。没有隔离，一个进程可以读写任意内存、伪造设备请求、修改别人的文件、消耗全部 CPU 和内存。安全机制的目标不是让程序“不会出错”，而是让错误和恶意行为被限制在授权边界内。

最基础隔离来自 CPU 特权级和页表权限。用户态不能执行特权指令，不能访问内核页，不能修改页表。系统调用是受控入口，内核在入口处检查参数和权限。文件权限、用户 ID、组 ID、capability、namespace、cgroup、seccomp 和 LSM 则在更高层限制对象访问和资源消耗。

实践

安全分析先写主体、对象、操作、权限：

主体	对象	操作	检查
进程 进程	虚拟页 文件	read/write/exec open/read/write	页表权限、VMA mode、owner、 capability、mount
进程	系统调用	exec/fork/ioctl	seccomp、LSM、资源 限制
容器	CPU/内存	消耗资源	cgroup quota/limit

然后写威胁模型：攻击者控制什么输入，目标对象是什么，期望阻止什么。没有威胁模型的“安全”讨论会变成口号。

算法与实现分析

访问控制检查

访问控制算法接收主体、对象和操作，返回允许或拒绝。Unix mode 位是最小模型：如果主体是 owner，看 owner bits；否则如果主体属于 group，看 group bits；否则看 other bits。

```
may_access(subject, inode, op):
    if subject.uid == inode.uid:
        bits = inode.mode.owner
    else if subject.groups contains inode.gid:
        bits = inode.mode.group
    else:
        bits = inode.mode.other
    return bits.allow(op)
```

capability 是对 root 权限的拆分。与其让 uid 0 拥有全部能力，不如把网络配置、挂载、修改时间、加载模块等能力拆开。这样服务可以只拿最小需要的能力。

namespace 和 cgroup

namespace 改变“看见什么”，cgroup 限制“能用多少”。PID namespace 让进程看到自己的进程树；mount namespace 改变文件系统视图；network namespace 隔离网卡、路由和端口。cgroup 限制 CPU、内存、I/O 和进程数量。

```
container_start():
    create_namespaces(pid, mount, net, ipc, uts)
    configure_cgroup(cpu_quota, memory_limit, pids_max)
    drop_capabilities()
    apply_seccomp_filter()
    exec_init_process()
```

容器不是虚拟机。它共享宿主内核，所以内核漏洞仍可能突破容器边界。容器安全依赖 namespace、cgroup、capability、seccomp、LSM 和镜像文件系统共同工作。

seccomp 过滤

seccomp 允许进程限制自己未来能调用哪些系统调用。典型服务启动后，只保留 read、write、epoll、futex、clock 等必要调用，拒绝 mount、ptrace、模块加载等危险入口。

```
seccomp_check(syscall_nr, args):
    action = bpf_program.evaluate(syscall_nr, args)
    if action == ALLOW: continue_syscall()
    if action == ERRNO: return -EPERM
    if action == KILL: terminate_process()
```

seccomp 的价值是缩小攻击面。即使攻击者控制了进程内存，也很难调用被过滤的内核入口。但过滤规则写错也会让程序在正常路径崩溃，所以必须用测试覆盖。

测试

- 用户态访问内核地址必须失败。
- 只读映射写入必须触发权限错误。
- 文件权限拒绝时，不创建、不截断、不写入目标文件。
- drop capability 后，相关特权操作必须失败。
- namespace 中不可见的对象不能通过旧路径访问。
- cgroup 内存限制触发时，系统有明确 OOM 或失败策略。
- seccomp 规则允许正常路径，拒绝非必要危险系统调用。

源码级补充：DAC、MAC、capability、namespace、Landlock 与硬化

访问控制模型

DAC 由对象拥有者决定访问，典型是 UNIX mode、uid、gid；MAC 由系统策略决定，典型是 SELinux 和 AppArmor；RBAC 按角色授权；ABAC 按属性决策。Linux 实际路径通常叠加 DAC、capability、LSM、mount flags 和 namespace。读 fs/namei.c、security/security.c、security/selinux/ 时，要追踪同一个对象从路径解析到权限 hook 的全过程。

```
open(path, O_WRONLY):
    resolve path to inode and mount
    check mount is writable
    check DAC mode or CAP_DAC_OVERRIDE
    call security_inode_permission()
    check file type and open flags
    create struct file
```

安全检查必须绑定实际对象。只检查字符串路径，再使用路径，是 TOCTOU 风险；解析后持有 inode、dentry 或 file 引用，再检查权限，才有稳定对象。权限失败时还要保证没有副作用：不能先 truncate 再发现权限不足。

capability 集合

进程有 permitted、effective、inheritable、bounding、ambient 等 capability 集合。bounding set 给进程树设置上界；ambient set 影响 exec 后保留的能力。容器安全经常要求清空不必要 capability，尤其避免 CAP_SYS_ADMIN、CAP_SYS_PTRACE、CAP_NET_RAW。

```
drop_privileges():
    clear_ambient_capabilities()
    drop_bounding_set_except(required_caps)
    set_no_new_privs()
    install_seccomp_filter()
```

能力检查要放在对象所属 user namespace 下解释。ns_capable(user_ns, CAP_NET_ADMIN) 和全局 capable 不是同一个边界。容器内 root 若只在容器 user namespace 有能力，不应自动获得宿主网络或宿主挂载的管理权。

namespace 与 cgroup v2

namespace	隔离内容
PID	进程编号和父子视图
Mount	挂载树和路径视图
Network	网络设备、路由、端口空间
IPC	System V/POSIX IPC 对象
UTS	hostname/domainname
User	uid/gid 映射和 namespace 内 capability
CGroup/Time	cgroup 视图、部分时间视图

cgroup v2 用 cgroup.controllers 声明可用 controller，用 cgroup.subtree_control 向子树下发 controller，用 memory、cpu、io、pids 等文件表达限制。资源限制也是安全边界：没有 pids 限制，fork bomb 会扩大到宿主；没有 memory 限制，单服务可触发全局回收和 OOM；没有 io 限制，后台写入会拖慢其他租户。

Landlock、seccomp user notification 和 BPF LSM

Landlock 允许非特权进程给自己加文件访问规则，是基于 LSM 的沙箱。seccomp user notification 可把某些系统调用交给监督进程决策。BPF LSM 允许加载受验证的 BPF 程序到安全 hook。这些机制的共同边界是：它们可以进一步限制自己或被管理进程，但不能凭空授予原本没有的权限。

```
sandbox_process:
    create Landlock ruleset
    add allowed read-only paths
    restrict_self()
    install seccomp allowlist
    exec workload
```

这类沙箱必须有负面测试：禁止路径打开失败、禁止 syscall 被拒绝、规则安装后不能再获得新权限。只验证正常路径能运行，不足以证明沙箱有效。

TPM、measured boot 和 sealed storage

TPM 可记录启动链测量值，remote attestation 让远端验证机器启动了预期固件、bootloader、内核和策略；sealed storage 让密钥只在特定测量状态下解封。它解决的是“这台机器是不是以预期状态启动”，不是运行时所有访问控制。内核被测量启动后，仍然要依赖权限、LSM、seccomp 和更新机制维护运行时边界。

ASLR、stack canary、CFI 与 KASLR

ASLR 随机化栈、堆、mmap 和 PIE 基址；stack canary 检测栈溢出覆盖返回地址；CFI 限制间接控制流；KASLR 随机化内核基址。它们是缓解措施，不是权限模型替代品。

机制	防什么	不能防什么
ASLR/KASLR	降低地址可预测性	信息泄漏后效果下降
stack canary	简单栈返回地址覆盖	任意写、逻辑漏洞
CFI	非法间接跳转	合法路径上的权限绕过
W $\bar{X}$ /NX	数据页执行	JIT 配置错误、ROP

安全测试要同时证明正常路径可用、禁止路径失败、失败有日志、敏感数据不泄露。只打开硬化选项但没有负面测试，不能说明边界有效。

open 权限检查不能只看路径字符串

路径是输入，inode 和 file 才是稳定对象。安全代码若先检查字符串路径，再在另一步打开文件，中间目录项可能被替换，形成 TOCTOU。更可靠的做法是让内核路径解析在同一次操作里完成权限、mount、LSM 和 open flag 检查，或者使用目录 fd、`openat2` 约束和已经打开的 fd 作为稳定能力。

```
unsafe pattern:
    if path starts with "/safe":
        open(path)

safer pattern:
    dirfd = open("/safe", O_PATH | O_DIRECTORY)
    openat2(dirfd, relative_path,
            RESOLVE_BENEATH | RESOLVE_NO_SYMLINKS)
```

这里的关键不是某个 API 名字，而是对象绑定：检查和使用必须落在同一个解析结果上。符号链接、bind mount、rename、并发 unlink、mount namespace 都会改变“同一个字符串”指向的对象。

exec 时 capability 和 no_new_privs 改变边界

权限不是进程启动后固定不变。exec 会重新计算凭据、环境、dumpable 状态、secureexec、文件 capability 和 setuid 影响。若进程设置 `no_new_privs`，后续 exec 不能获得比当前更多的权限；seccomp 过滤器通常要求先设置这个标志，防止低权限进程安装过滤器后再通过 exec 提权。

```
exec credential sketch:
    read current cred
    inspect executable mode and file capabilities
    if no_new_privs:
        suppress privilege gain
        compute new permitted/effective/ambient sets
        apply secureexec rules if privilege boundary changed
        commit new cred at exec commit point
```

ambient capability 容易被误用。它能跨 exec 传递给非特权可执行文件，所以启动服务前应清理不需要的 ambient set，并收紧 bounding set。否则子进程可能在没预期的路径上保留网络、ptrace 或管理能力。

seccomp 规则要按参数和返回动作设计

只按 syscall 号 allow/deny 往往太粗。例如 `ioctl`、`prctl`、`clone`、`socket` 的危险性高度依赖参数。高质量 seccomp profile 至少要区分常用参数组合，并为拒绝路径选择明确动作：返回 `errno`、kill 进程、记录日志或交给 user notification。

```
seccomp policy sketch:
allow read/write/close/futex/clock_gettime
allow mmap only without PROT_EXEC|PROT_WRITE together
allow socket only AF_UNIX if service does not need network
reject mount/ptrace/bpf/module loading
return EPERM for expected probes
kill process for impossible dangerous calls
```

规则还要随库和运行时变化测试。动态链接器、线程库、DNS、日志库都可能调用你没想到的系统调用。部署前应在真实启动路径、热路径、错误路径和 reload 路径下跑 profile，而不是只跑主函数。

Landlock 的边界：只能收紧，不能授权

Landlock 适合让普通进程给自己加文件系统沙箱。它的 ruleset 声明要处理哪些访问类型，再把路径 fd 加入 allow list，最后 `restrict_self`。一旦限制生效，进程不能用 Landlock 获得原来没有的权限，只能在原权限基础上进一步收紧。

```
landlock flow:
ruleset = create_ruleset(handled_access_fs)
dirfd = open(allowed_dir, O_PATH | O_DIRECTORY)
add_path_beneath_rule(ruleset, dirfd, allowed_access)
prctl(PR_SET_NO_NEW_PRIVS, 1)
landlock_restrict_self(ruleset)
exec or run workload
```

测试要覆盖允许目录、禁止目录、符号链接、rename、临时文件和继承。若只测试“程序还能启动”，无法说明沙箱真的挡住了写入、删除、执行或路径逃逸。

BPF LSM 和传统 LSM 的关系

LSM hook 把安全检查插入到 inode、file、task、socket、bpf 等对象操作中。传统 LSM 如 SELinux、AppArmor 使用预先加载的策略；BPF LSM 允许受验证的 BPF 程序挂到安全 hook 上，适合做可观测、渐进式或环境相关的策略。

```
security_inode_permission(inode, mask):
for each registered LSM:
    ret = lsm->inode_permission(inode, mask)
    if ret != 0:
        return ret
return 0
```

BPF LSM 仍受 verifier、权限和 hook 上下文限制。程序不能任意睡眠，不能随便访问内核内存，返回值语义也要遵守 hook 约定。它是内核安全框架的一部分，不是绕过 DAC/capability 的后门。

容器逃逸常见入口

容器共享宿主内核，所以边界经常坏在“多给了一个能力或设备”。常见风险包括挂载宿主敏感路径、保留 `CAP_SYS_ADMIN`、开放容器引擎 socket、允许特权设备、未限制 `/proc` 和 `/sys`、seccomp profile 过宽、cgroup pids/memory 未限。

风险	为什么危险	收紧方向
<code>CAP_SYS_ADMIN</code>	覆盖大量管理操作	默认删除，只按具体需要添加
宿主容器引擎 socket	等价于控制宿主容器引擎	不挂载，改用受限代理
特权设备节点	可能直接访问内核或硬件接口	devices cgroup 和 最小设备集

宽松 <code>/proc</code>	泄露进程、内核和 namespace 信息	hidepid、只读挂载、裁剪视图
无 pids/memory 限制	fork bomb 或内存压力拖垮宿主	cgroup v2 限额

安全隔离要按“最小必要对象”配置，而不是按“程序跑起来需要什么就全开”。每放开一个 `capability`、设备、挂载或 `syscall`，都要写清楚它对应的业务需求和负面测试。

6.2 panic、错误路径与恢复策略

panic、错误路径与恢复策略

原理

操作系统必须区分普通错误、进程错误、内核 bug 和硬件不可恢复错误。普通错误应返回 `errno`，例如文件不存在、权限不足、非阻塞 I/O 暂时不可用；进程错误应向进程发送信号，例如非法地址访问；内核 bug 可能触发 `WARN`、`oops` 或 `panic`；硬件错误可能要求隔离设备、下线页面或重启系统。若把所有错误都 `panic`，系统不可用；若把所有错误都吞掉，数据会被悄悄破坏。

`panic` 的含义是内核认为继续运行比停止更危险。典型原因包括核心不变量破坏、调度器状态自相矛盾、文件系统元数据检测到不可恢复损坏、内核栈溢出、无法处理的机器检查。`panic` 不是调试打印的高级版本，而是系统级 `fail-stop` 策略：停止服务，尽量保留证据，避免进一步破坏状态。

核心思想

高质量 OS 实现不只写正常路径，还要把每个失败点的责任写清楚：返回错误、终止进程、重试、降级、隔离，还是 `panic`。

错误路径的工程难点在资源回收。系统调用可能已经分配页、获取锁、增加引用、插入表项、发起 I/O，然后第七步失败。正确实现必须让失败路径按照相反顺序撤销已完成步骤，并且不能撤销还没完成的步骤。C 语言内核常用 `goto out_...` 标签表达这种线性回滚；C++ 内核或教学模拟器可以用 `RAII` 表达资源所有权，但必须注意中断上下文、`panic` 路径和 `no-throw` 约束。

实践

错误分类可以写成下表：

类别	处理方式	示例
可预期用户错误	返回 <code>errno</code>	<code>ENOENT</code> 、 <code>EACCES</code> 、 <code>EAGAIN</code>
用户进程违规	信号或终止该进程	<code>page fault</code> 权限错误、非法指令
资源暂时不足	返回错误、等待或回收	<code>ENOMEM</code> 、队列满、端口耗尽
设备局部故障	重置设备、隔离队列、上报 I/O error	磁盘超时、网卡 <code>reset</code>
内核不变量破坏	<code>WARN</code> / <code>oops</code> / <code>panic</code>	双重释放、 <code>runqueue</code> 计数错误
持久化一致性风险	<code>remount read-only</code> 、 <code>replay journal</code> 、 <code>panic</code>	元数据校验失败

一个系统调用实现应在设计阶段列出失败点：


```

open(path, flags):
    copied_path = copy_path_from_user(path)           may fail EFAULT/ENAMETOOLONG
    parent = resolve_parent(copied_path)              may fail ENOENT/EACCES
    inode = lookup_or_create(parent, name, flags)      may fail ENOENT/EEXIST/EROFS
    file = alloc_file(inode, flags)                   may fail ENOMEM
    fd = alloc_fd(current.files)                       may fail EMFILE
    install_fd(fd, file)                               commit point
    return fd

```

提交点之前失败，要释放已拿到的引用；提交点之后失败，通常不能再返回“没有打开”，而要让 fd 表保持一致或执行明确关闭。把提交点写出来，可以避免半初始化 fd 留在表里。

算法与实现分析

errno 返回与用户可见契约

errno 不是随便选的数字。调用者会根据错误码决定重试、降级、提示用户还是终止。

```

copy_from_user_checked(dst, user_ptr, len):
    if not user_range_valid(user_ptr, len, READ):
        return EFAULT
    if len > MAX_COPY:
        return EINVAL
    fault = copy_may_fault(dst, user_ptr, len)
    if fault:
        return EFAULT
    return OK

```

把非法用户地址返回 EFAULT，而不是 panic，是因为这是用户进程可触发的预期错误。内核必须把“不可信输入”当正常情况处理。只有当内核自己的页表、锁状态或对象引用被破坏时，才进入更严重路径。

goto 回滚模板

C 风格内核常见模式：

```

sys_create(path):
    name = null
    inode = null
    file = null

    name = copy_name(path)
    if name == null:
        ret = ENOMEM
        goto out

    inode = create_inode(name)
    if is_error(inode):
        ret = error(inode)
        goto out_name

    file = alloc_file(inode)
    if file == null:
        ret = ENOMEM
        goto out_inode

    ret = install_fd(file)
    if ret < 0:
        goto out_file

```



```

return ret

out_file:
    put_file(file)
out_inode:
    put_inode(inode)
out_name:
    free(name)
out:
    return ret

```

这个结构的优点是回滚顺序与获取顺序相反，且每个标签只释放已经成功获取的资源。缺点是标签命名和跳转边界容易腐烂，需要测试每个失败注入点。C++ RAII 可以减少样板，但内核中不是所有路径都适合异常；析构函数也不能在持有自旋锁或 panic 路径中做复杂阻塞操作。

WARN、BUG、oops 与 panic

可以把严重程度分成四级：

机制	含义	处理
WARN	发现异常但可能继续	打印栈、计数、继续或降级
BUG/oops	当前内核路径不可继续	杀死当前进程或停止当前 CPU，视上下文而定
panic	整个系统不可安全继续	停机、重启或进入 crash dump
watchdog	系统长时间无响应	触发诊断、中断栈采样或 panic

教学系统可以实现 `assert_kernel_invariant`：调试构建 panic，发布构建记录错误并尽量隔离。但对文件系统元数据损坏这类风险，继续写入可能扩大破坏，remount read-only 或 panic 是合理选择。

看门狗与故障转储

watchdog 检测“没有前进”。硬锁死可能表现为中断不再发生；软锁死可能表现为某 CPU 长时间不调度；RCU stall 表示读侧临界区过久导致回收无法完成。

```

watchdog_tick(cpu):
    now = time()
    if now - cpu.last_schedule_time > SOFTLOCKUP_LIMIT:
        dump_cpu_stack(cpu)
        warn("soft lockup")
    if now - cpu.last_interrupt_time > HARDLOCKUP_LIMIT:
        panic("hard lockup")

```

panic 后的目标不是恢复服务，而是保留证据：panic 原因、CPU 寄存器、栈、当前任务、锁持有、最近日志、脏页状态和设备错误。crash dump 的价值在于让下一次启动后能分析，而不是在 panic 路径中做复杂修复。

重试与幂等

不是所有失败都应该立即返回。临时内存不足可以触发回收后重试；设备超时可以 reset 后重试；网络发送可以等待窗口。但重试必须有边界，并且操作要么幂等，要么有明确的提交记录。

```

retry_io(req):
    while req.retries < MAX_RETRIES:
        submit(req)
        status = wait_completion(req, timeout)
        if status == OK:
            return OK

```

```

if not is_transient(status):
    return status
reset_path_if_needed(req.device)
req.retries++
return EIO

```

对写请求，若设备状态不明，简单重试可能造成重复写。对普通块覆盖写通常可接受；对“追加一条记录”这类上层操作，必须有序列号、校验或日志来识别重复提交。

测试

1. `errno` 矩阵：非法指针、权限不足、资源耗尽返回不同错误码。
2. 失败注入：在系统调用每个分配点失败，引用计数和锁计数最终归零。
3. 提交点测试：提交前失败无用户可见副作用，提交后状态自治。
4. `WARN` 测试：非致命不变量异常记录证据但不破坏后续测试。
5. `panic` 测试：核心不变量破坏后停止调度，输出原因和栈。
6. `watchdog` 测试：构造关中死循环或不调度循环，能触发诊断。
7. 设备重试：临时错误可重试，永久错误返回 `EIO` 且唤醒等待者。
8. `crash dump`：`panic` 后保存当前任务、寄存器、锁和最近日志。

本章合格标准：能区分 `errno`、信号、`oops`、`panic`；能为系统调用写出失败回滚路径；能解释提交点、幂等重试和故障证据的关系。

源码级补充：`oops`、`KASAN`、`KCSAN`、`KFENCE`、`fault injection` 与 `kdump`

`BUG`、`WARN`、`oops` 和 `panic` 的代码路径

Linux 中 `WARN_ON` 打印栈但通常继续；`BUG_ON` 表示当前路径不可继续；`oops` 是内核异常报告，可能杀死当前进程或导致 `panic`；`panic` 停止系统或按配置重启。读 `kernel/panic.c` 和架构异常处理代码时，看报告如何收集寄存器、栈、`taint`、`loaded modules` 和当前任务。

```

bad_kernel_access:
    fault in kernel mode
    -> do_error_trap or page fault path
    -> oops_begin()
    -> show registers, stack, code bytes
    -> decide die, panic, or kill current task

```

错误严重程度取决于上下文。用户进程传坏指针应返回 `EFAULT`；内核在持有自旋锁时 `page fault`，可能说明严重 `bug`；文件系统检测到元数据不一致继续写入可能扩大损坏，因此 `remount read-only` 或 `panic` 都是合理策略。

`KASAN`、`KCSAN` 和 `KFENCE`

工具	发现问题	取舍
KASAN	越界、 <code>use-after-free</code> 、部分栈/全局错误	高开销，高覆盖
KCSAN	数据竞争，非原子并发访问	采样式，可能需多跑

这些工具不会替代锁设计。它们帮助你发现违反不变量的具体证据，例如哪个对象释放后被哪个路径继续访问，哪个字段被两个 CPU 无同步写入。测试报告应保留工具输出中的对象地址、分配栈、释放栈和访问栈。

fault injection

Linux 提供多种故障注入，例如 `fail_page_alloc`、`failslab`、`fail_make_request`。目标是在可控位置让分配、slab、块 I/O 失败，验证错误路径。

```
test plan:
  fail next kmalloc in sys_open
  expect no fd installed, inode ref dropped

  fail block request number N during fsync
  expect fsync returns EIO and writeback error recorded
```

没有故障注入，错误回滚代码很可能从未执行过。高质量 OS 代码要让每个 `goto out_*` 标签都能被测试触发。故障注入测试必须检查“没有发生什么”：没有泄漏引用、没有遗留锁、没有半初始化对象进入全局表。

kdump 与 /proc/vmcore

kdump 预留一段 crashkernel 内存。panic 后通过 kexec 启动捕获内核，导出 `/proc/vmcore`，再用 crash 工具结合 vmlinux 符号分析。panic 路径越少依赖损坏系统，dump 成功率越高。

```
panic
-> stop other CPUs
-> save registers and notes
-> kexec crash kernel
-> expose /proc/vmcore
-> analyze task, locks, stacks, memory
```

生产环境中，panic 策略要结合服务目标：有的系统宁可快速重启，有的系统必须保留 dump，有的存储系统检测到元数据损坏要停止写入避免扩大破坏。

错误恢复的状态机写法

恢复不是“再试一次”。每个可恢复对象都应有明确状态：正常、降级、恢复中、不可用、已隔离。状态转换要有进入条件、退出条件和超时策略。

```
device state:
  ONLINE
    on timeout -> RECOVERING
  RECOVERING
    stop queues
    reset hardware
    replay configuration
    complete lost requests
    if success -> ONLINE
    if fail count exceeded -> FAILED
  FAILED
    reject new requests with EIO
    keep diagnostic evidence
```

没有状态机，恢复路径容易和正常 I/O 并发踩踏：reset 时仍提交新请求，失败请求没有唤醒等待者，或恢复成功后旧 completion 又回来释放同一资源。

错误预算和降级

并非所有错误都要立即 panic 或无限重试。系统可以设置错误预算：短时间少量校验失败触发重试，连续失败触发隔离，核心元数据错误触发只读或 panic。错误预算让系统避免因为瞬时故障过度反应，也避免因为反复失败无限拖延。

现象	初始处理	超过预算后
块 I/O 超时	重试、reset 队列	下线设备或返回 <code>EIO</code>
内存分配失败	回收、压缩、等待	OOM kill 或返回 <code>ENOMEM</code>
网络重传增多	降低发送速率	熔断连接或限流
文件系统校验失败	停止相关写入	remount read-only 或 panic

错误预算必须可观测：重试次数、最后错误、进入降级时间、当前状态和影响对象都要暴露。否则系统处在降级状态时，用户只会看到“偶尔很慢”。

cleanup attribute 与 RAII 的边界

用户态 C 可以用 `__attribute__((cleanup))` 模拟作用域清理，C++ 可以用 RAII。内核 C 常用 `goto out`，是因为错误路径需要精确控制上下文、锁和释放顺序。三种方法没有绝对高低，关键是所有权必须清楚。

```
// conceptual RAII style
FileRef file = open_checked(path)
PageRef page = allocate_page()
LockGuard guard(inode.lock)
commit_update(file, page)
```

RAII 适合普通进程上下文中的资源组合；但析构函数不应隐藏可能睡眠的复杂 I/O，也不应在 panic 路径中试图获取新锁。教学 OS 可以使用 C++20 RAII 管理引用和锁，但要为中断上下文、noexcept 和显式提交点写清规则。

panic 报告模板

panic 输出要稳定、短而有证据。最低字段如下：

```
panic report:
  reason
  cpu id and current task
  instruction pointer and stack pointer
  trap vector or syscall number
  held locks
  taint or loaded modules
  last N kernel log records
  affected device or inode if any
  reboot/dump policy
```

报告不是越长越好。panic 时系统可能已经损坏，输出过多可能再次阻塞。核心原则是保留能定位第一现场的事实，而不是在崩溃路径中做复杂分析。

6.3 可观测性和调试证据

可观测性、调试与性能边界

原理

操作系统问题很少只靠读代码解决。线程为什么不运行，可能是调度等待、锁等待、I/O 阻塞、page fault、信号、cgroup 限流或下游服务；内存为什么上涨，可能是真实泄漏、allocator 缓存、page cache、mmap、线程栈或内核 buffer；写文件为什么慢，可能是 page cache 脏页、设备队列、fsync、目录同步或日志提交。没有观测证据，结论只是猜测。

可观测性要把 OS 状态映射到证据。进程和线程状态可以从 `/proc`、`ps`、`top`、调度 `trace` 观察；系统调用可以用 `strace` 观察；CPU 热点可以用采样 profiler；`page fault`、`context switch`、I/O 等待和网络状态可以用系统计数器和 tracing 工具；应用层必须补充业务 `trace`，记录请求身份、队列等待、状态机阶段和错误码。

调试时要区分 on-CPU 和 off-CPU。on-CPU 说明线程正在执行指令；off-CPU 说明线程没有运行，可能在等待。CPU profiler 只能解释运行时消耗在哪里，解释不了大部分时间都在睡眠的请求。尾延迟问题常常要从 off-CPU 等待开始看。

内存调试要区分 `virtual size`、`resident set`、`anonymous memory`、`file-backed page`、`page cache`、`allocator arena` 和 `kernel memory`。`malloc` 后不触页，可能只增加虚拟地址；首次写入才分配物理页；释放内存后 RSS 不马上下降，也可能是 `allocator` 保留 `arena`。把所有 RSS 增长都叫泄漏，是不合格诊断。

I/O 调试要区分 `buffered I/O`、`direct I/O`、`mmap`、`page cache hit/miss`、`dirty writeback`、`fsync` 和设备吞吐。一次写很快，可能只是进入 `page cache`；一次读很快，可能只是热缓存；一次 `fsync` 很慢，可能是在偿还之前累计的脏页。实验必须控制冷缓存和热缓存，否则数字不可比较。

实践

本章的实践模板是“现象、假设、证据、反证、结论”。

字段	请求延迟升高示例
现象	p95 从 20ms 升到 500ms
假设	worker 等锁或等 I/O
证据	trace 中 <code>queue_wait</code> 低， <code>blocked_io</code> 高，线程睡在 <code>writeback</code>
反证	on-CPU profiler 没有热函数占用 500ms
结论	延迟主要来自 I/O 完成等待，不是 CPU 算法变慢

程序卡死时，先列线程状态：哪个线程持有锁，哪个线程等待条件变量，哪个线程在 `join`，关闭标志是否设置，是否有线程仍可能唤醒等待者。若所有消费者都睡在 `not_empty`，而关闭路径只通知 `not_full`，就能定位到等待谓词和通知对象不匹配。

实践中要保留原始证据：命令、时间、输入规模、机器信息、内核版本、容器限制、采样频率、日志片段和无法观察的字段。没有这些，报告不可复现。

算法与实现分析

事件日志

最简单的可观测性算法是事件日志。每个关键状态转换追加一条结构化记录：时间、线程、请求 ID、事件类型、对象 ID、旧状态、新状态和错误码。日志不能只写自然语言，否则无法聚合和验证。

```
record_event(req_id, object, old_state, new_state, reason):
    entry.ts = monotonic_time()
    entry.cpu = current_cpu()
    entry.thread = current_thread()
    entry.req_id = req_id
    entry.object = object
    entry.old_state = old_state
    entry.new_state = new_state
    entry.reason = reason
    ring_append(trace_buffer, entry)
```

事件日志本身也要有背压。无限日志会把故障变成磁盘或内存问题。常见做法是固定大小环形缓冲区、按级别采样、按请求 ID 打开详细追踪，或者把关键错误同步写入可靠日志。

延迟分解

请求延迟不能只记录总耗时。一个可行的分解至少包括 queue wait、runnable wait、on CPU、blocked on lock、blocked on I/O、downstream wait 和 cleanup。实现上可以在状态转换处打点，然后按请求 ID 聚合。

```
transition(req, new_state):
    now = clock()
    req.time_in_state[req.state] += now - req.last_ts
    req.state = new_state
    req.last_ts = now
```

这个算法的复杂度是每次状态转换 $O(1)$ ，但要求状态枚举稳定。若所有等待都记成 blocked，后续无法判断是锁、I/O 还是 page fault。

资源对账

资源泄漏调试需要对账算法。每次 open 增加 fd 计数，每次 close 减少；每次分配对象记录 owner，每次释放删除记录；系统退出或测试结束时检查剩余对象。

```
track_alloc(id, type, owner):
    live[id] = {type, owner, stack}

track_free(id):
    live.erase(id)

assert_no_leak():
    for each object in live:
        report(object)
```

真实系统不能给所有对象都保存完整 stack，因为成本太高；可以采样、按类型开启、只在测试构建开启。但思想不变：资源必须能被计数，生命周期必须能闭合。

off-CPU 分析

很多性能问题不在 CPU 火焰图里，因为线程大部分时间不在 CPU 上。off-CPU 分析记录线程从运行态切到睡眠态的原因，以及被谁唤醒。

```
on_schedule_out(task, reason, wait_object):
    task.offcpu_start = clock()
    task.wait_reason = reason
    task.wait_object = wait_object

on_wakeup(task, waker):
    delta = clock() - task.offcpu_start
    offcpu_histogram[task.wait_reason].add(delta)
    record_wakeup_edge(waker, task, task.wait_object)
```

这个数据能区分等锁、等 I/O、等页、等网络、等定时器和等子进程。若 p99 延迟主要来自 off-CPU 的 writeback 等待，优化业务 CPU 代码不会解决问题。

最小可复现报告

高质量调试报告要让别人能复现判断。最小格式如下：

```
Problem:
    现象、影响范围、首次出现时间

Environment:
    内核版本、硬件/容器限制、文件系统、负载规模
```


Timeline:

关键事件按时间排序

Evidence:

系统调用、trace、线程状态、资源计数、日志、指标

Rejected hypotheses:

已经排除的解释和证据

Conclusion:

当前最能解释现象的机制

Boundary:

仍未验证的假设和下一步实验

报告的价值不在于一次猜中，而在于缩小不确定性。写出反证能防止团队反复讨论已经排除的方向。

测试

可观测性也需要测试。一个系统若只能在成功时输出日志，失败时没有状态，就不可维护。

- 每个长时间等待点都能输出等待原因或 trace 事件。
- 每个请求都有稳定身份，能串起进入队列、开始执行、阻塞、完成和失败。
- 错误码保留原始原因，不被统一改写成 unknown error。
- 资源计数可对账：打开 fd 数、活跃线程数、队列长度、in-flight I/O、分配对象数。
- 性能测试区分冷启动、热缓存、预热、输入规模和测量次数。
- 故障注入能触发日志和恢复路径，而不是只测试成功路径。

调试报告的验收标准是别人能复现你的判断。若报告只写“怀疑是系统问题”，没有线程状态、系统调用、等待点、资源计数或反证，它不是 OS 分析。

源码级补充：printk、ftrace、perf、eBPF、strace、/proc、/sys 与 PSI

printk 和 dmesg

printk 有 loglevel、rate limiting、console 输出和 ring buffer。panic 前后的日志可能是唯一证据，但过量 printk 会扰动时序，甚至拖慢系统。驱动错误路径应记录设备、请求 id、状态寄存器、errno，而不是只写“failed”。

ftrace 和 tracepoint

ftrace 可以追踪函数入口、函数图和静态 tracepoint。tracepoint 的价值是稳定字段：调度切换、irq、block、net、syscalls 都有可解析事件。

```
trace-cmd record -e sched:sched_switch -e block:block_rq_issue
trace-cmd report
```

阅读 [kernel/trace/](#) 时，看 trace event 如何定义字段、如何写入 per-CPU ring buffer、如何被用户态读取。tracepoint 是把证据落成可采集事件。

perf 与 PMU

perf stat 看计数，perf record/report 看采样调用栈，perf top 看实时热点。PMU 事件能观察 cycles、instructions、cache misses、branch misses、TLB misses。IPC 低可能来自 cache miss、分支错误、前端供给不足或后端等待，不能只说“CPU 慢”。

```
perf stat -e cycles,instructions,cache-misses,branches,branch-misses ./workload
perf record -g ./workload
perf report
```

eBPF 和 bpftrace

eBPF 程序可挂到 kprobe、uprobe、tracepoint、profile、LSM、cgroup、网络路径。BPF map 保存状态，helper 提供受控内核访问。bpftrace 适合快速一行观测。

```
bpftrace -e 'tracepoint:syscalls:sys_enter_write { @[comm] = count(); }'
bpftrace -e 'kprobe:vfs_read { @[kstack] = count(); }'
```

eBPF 的边界是 verifier、安全权限和观测开销。生产系统应限制采样频率和输出量，避免观测本身制造故障。

strace、ltrace 和 gdb

strace 基于 ptrace 观察系统调用 enter/exit，能回答“程序在请求内核做什么”；ltrace 观察动态库调用，能回答“用户态库在做什么”；gdb/kgdb 用符号和断点观察状态。strace 看不到内核内部等待原因，perf/ftrace/eBPF 可以补上。

/proc、/sys、sysctl 和 systemd journal

/proc/meminfo、/proc/interrupts、/proc/slabinfo、/proc/[pid]/status、/proc/[pid]/stack 是运行状态窗口；/sys 暴露 kobject、设备、cgroup、block queue 参数；/proc/sys 是 sysctl 控制面；journald 给系统日志结构化字段。

PSI: Pressure Stall Information

PSI 记录 CPU、memory、IO stall 时间，回答“任务因为资源压力停顿多久”。它比单纯利用率更接近用户感受到的延迟。内存 PSI 高说明任务因回收或内存不足停顿；I/O PSI 高说明任务等待 I/O；CPU PSI 高说明 runnable 但等不到 CPU。

```
cat /proc/pressure/cpu
cat /proc/pressure/memory
cat /proc/pressure/io
```

PSI 适合做过载保护信号。服务可以在 memory/io pressure 高时限流，避免把队列堆到超时。

工具选择决策表

可观测性工具很多，选择时先问问题类型。

问题	优先工具	证据
程序卡在哪个系统调用	<code>strace -tt -T</code>	syscall、耗时、errno
CPU 时间花在哪里	<code>perf record -g</code>	on-CPU 调用栈
线程为什么不运行	off-CPU trace、sched trace	睡眠原因、唤醒者
磁盘请求慢在哪里	block trace、iostat、perf sched	队列和 completion
内存压力是否存在	PSI、/proc/meminfo、vmstat	stall、reclaim、swap
内核路径偶发错误	tracepoint、kprobe、bpftrace	事件字段和栈
容器被限制	cgroup v2 文件、journal	throttled、OOM、pids limit

不要一上来抓所有数据。先用低成本指标定位层次，再用高精度 trace 验证假设。观测也会扰动系统，尤其是高频 kprobe、过量 printk 和全量系统调用 trace。

一次延迟诊断的完整演示

假设 HTTP 服务 p99 从 80ms 升到 2s。诊断不应直接改代码，而应建立证据链。

```
step 1: application trace
  parse: 1ms, business: 5ms, persist: 1900ms

step 2: strace
  fsync(fd) = 0 <1.87s>

step 3: /proc/pressure/io
  some avg10 increased

step 4: block trace
  many flush requests wait behind writeback

step 5: conclusion
  tail latency dominated by storage flush and dirty writeback
```

反证也要写：CPU profiler 没有 2s on-CPU 热点；网络 recv/send 没有长时间阻塞；锁等待直方图没有对应峰值。这样结论才不是“看起来像磁盘”。

指标设计：counter、gauge、histogram、event

系统指标至少分四类。

类型	含义	OS 示例
counter	单调增加事件数	syscall 次数、错误次数、重试次数
gauge	当前状态值	队列长度、活跃线程、脏页数量
histogram	分布	请求延迟、锁等待、I/O 完成时间
event	结构化状态转换	任务睡眠、唤醒、请求提交、panic

错误率只用 counter 不够，还要有错误码和对象维度；延迟只报平均值不够，要有 p50/p95/p99 和最大值；队列只报当前值不够，最好有高水位和丢弃次数。

低开销 instrumentation 原则

观测点应贴近状态转换，而不是在循环里无节制打印。推荐做法：

- 热路径用 per-CPU counter，避免全局锁。
- 高频事件采样或按条件开启。
- 错误路径记录结构化字段和稳定 id。
- 日志消息带 request id、task id、object id。
- trace buffer 有固定容量和丢弃计数。
- 观测代码本身不能持有业务关键锁太久。

可观测性也是代码，必须有性能预算。一个 tracing patch 若把锁竞争扩大，就可能把被观察系统变成另一个系统。

验收：可复现诊断包

本卷要求最终报告能够打包复现。最小诊断包包括：

```
diagnostic bundle:
  exact command and workload
```

```
kernel version and hardware/container limits
application logs with request ids
relevant /proc and /sys snapshots
trace or perf data
benchmark raw results
analysis notes with rejected hypotheses
```

报告不是把命令输出堆在一起，而是把证据组织成因果链：现象是什么，在哪层出现，哪些解释被排除，当前结论的边界在哪里。能做到这一点，才算真正掌握 OS 调试。

6.4 安全与观测深讲

安全与观测深讲：没有证据就没有边界

安全和可观测性必须一起讲。权限检查失败时，要能证明没有改变目标对象；进程被 seccomp 拒绝时，要能看到拒绝的系统调用；cgroup 限制触发时，要能解释是内存、CPU 还是进程数限制；panic 发生时，要保留足够证据。没有观测能力，安全边界只能靠想象。

威胁模型写法

安全分析应先写威胁模型，而不是先列工具名。

```
Threat model:
  attacker controls:
    HTTP request body and headers
    uploaded file content
  attacker does not control:
    service binary
    kernel code
    host root credentials
  assets:
    user database
    service token
    host filesystem
  required boundaries:
    process cannot read host secrets
    process cannot mount or ptrace
    process cannot consume unlimited memory
```

这样才能判断 namespace、cgroup、capability、seccomp 和文件权限各自解决哪一部分问题。否则“用了容器”不等于安全，“用了 seccomp”也不等于安全。

最小权限的阶段化

服务启动阶段和运行阶段需要的权限不同。启动可能需要读取配置、绑定端口、打开日志；运行时只需要处理已有 fd 和少量系统调用。权限应随阶段下降。

```
startup:
  read config
  bind port 80
  open log file

drop:
  chroot or mount namespace
  drop capabilities
  set no_new_privs
  install seccomp allowlist
  enter cgroup limits

serve:
```

```
accept, read, write, epoll, futex, clock
```

权限下降要可测试。测试不应只证明正常请求成功，还要证明 `mount`、`ptrace`、打开宿主敏感路径、超额 `fork` 和超额内存分配都会失败。

错误路径的证据

错误路径也要写日志和计数器，但不能泄露敏感信息。比如权限拒绝应记录主体、对象类型、操作和拒绝原因，但不应把密钥内容写入日志。`panic` 记录寄存器、栈、锁和任务状态，但不能依赖复杂文件系统路径。

事件	必要证据	避免
权限拒绝	uid、capability、对象类型、操作	打印敏感文件内容
OOM	cgroup、进程、内存统计、victim	只写 killed without reason
seccomp 拒绝	syscall number、action、pid	静默杀进程无诊断
panic	原因、CPU、寄存器、栈、锁	panic 路径做复杂阻塞 I/O

从现象到证据链

一个好的 OS 调试结论要形成证据链。

```
Symptom:
p99 latency increased from 50ms to 2s

Evidence chain:
application trace: requests stuck in write phase
syscall trace: fsync takes 1.8s
kernel metrics: dirty pages high
block trace: queue depth high, flush waits long
conclusion: tail latency dominated by storage flush
```

这种报告比“怀疑磁盘慢”有用，因为它说明了每一层的证据和排除方向。若没有 `syscall trace`，也可能是应用锁；若没有 `block trace`，也可能是文件系统 `journal`；证据链让下一步实验明确。

安全与观测练习

1. 写一个网络服务的威胁模型，列出攻击者能控制和不能控制的对象。
2. 设计 `seccomp allowlist`，并说明正常路径需要哪些系统调用。
3. 构造权限拒绝测试，证明失败时目标文件没有被创建或截断。
4. 写一份 OOM 诊断报告，包含 `cgroup`、进程和内存统计。
5. 给一次 `panic` 设计最小 `crash dump` 字段。

安全边界的测试矩阵

安全机制必须能被测试。下面是一组最小矩阵：

边界	测试	期望
页表隔离	用户态读内核地址	page fault 或进程终止
文件权限	无权限 <code>open/write</code>	返回拒绝且目标不变
capability	drop 后执行特权操作	返回 <code>EPERM</code>
namespace	访问不可见路径或进程	不可见或返回不存在

cgroup	超过内存或进程限制	OOM/失败限制在 cgroup 内
seccomp	调用禁止 syscall	按策略返回 errno 或杀进程
观测	触发拒绝和 OOM	日志/计数器能解释原因

测试安全时要检查“没有发生什么”。权限拒绝后文件不能被截断；seccomp 拒绝后进程不能完成危险操作；cgroup OOM 不能扩大成宿主全局 OOM。负面保证是安全测试的核心。

可观测性常见误区

- CPU 低不代表系统没压力，可能都在 off-CPU 等 I/O 或锁。
- RSS 增长不一定是泄漏，可能是 page cache、arena 或 mmap。
- p99 变差不一定是平均路径慢，可能是少数 fsync、reclaim 或锁等待。
- 日志多不等于可观测，结构化字段和请求身份更重要。
- 没有反证的结论只是猜测。

本章小结

- 安全需要威胁模型，不能只列机制名。
- 最小权限要随阶段下降，并用测试证明危险操作失败。
- 错误路径必须留下足够证据，但不能泄露敏感数据。
- 调试报告要形成现象、证据、反证、结论和边界的链条。
- 安全和可观测性互相支撑：看不见的边界很难被信任。

访问控制：主体、客体、操作和上下文

权限判断可以抽象成四元组：主体是谁，客体是什么，操作是什么，上下文是什么。主体可能是 uid、gid、capability、进程安全标签或容器身份；客体可能是 inode、socket、进程、设备或命名空间对象；操作是 read、write、execute、mount、ptrace 等；上下文包括路径、挂载选项、是否 setuid、是否处于用户命名空间等。

```
may_access(subject, object, operation, context):
    if object.type == inode:
        if dac_denies(subject.uid, object.mode, operation):
            if !has_capability(subject, override_cap(operation)):
                return DENY
    if lsm_policy_denies(subject.label, object.label, operation):
        return DENY
    if mount_flags_deny(context.mount, operation):
        return DENY
    return ALLOW
```

真实系统通常叠加多层策略：传统 UNIX DAC、capability、ACL、LSM、namespace 边界和挂载选项。安全 bug 常来自只检查其中一层，或在路径解析和最终对象使用之间发生 TOCTOU。正确做法是尽量在持有对象引用后检查权限，并让检查结果绑定到实际对象而不是字符串路径。

capability：把 root 权限拆小

传统 root 权限过大。capability 把特权拆成多个位，例如绑定低端口、修改网络配置、加载模块、改变文件所有者、绕过 DAC 等。服务启动时可能短暂需要某些 capability，运行时应丢弃。

能力类型	能做什么	风险
<code>CAP_NET_BIND_SERVICE</code>	绑定低端口	可在启动后丢弃
<code>CAP_SYS_ADMIN</code>	大量系统管理操作	过大，接近半个 root
<code>CAP_SYS_PTRACE</code>	调试或观察其他进程	可读敏感内存
<code>CAP_DAC_OVERRIDE</code>	绕过文件权限检查	可读写本不该访问的文件
<code>CAP_NET_ADMIN</code>	改网络接口和路由	可影响宿主网络

最小权限不是“非 root 运行”一句话。一个非 root 进程若仍有危险 capability，仍可能扩大影响面。反过来，一个 root 启动的进程若完成初始化后清空 capability、设置 `no_new_privs` 并进入受限 namespace，风险会显著降低。

namespace：隔离名字，不隔离内核

namespace 改变进程看到的名字空间：mount namespace 改变路径树，pid namespace 改变进程编号视图，net namespace 改变网络设备和路由视图，uts namespace 改变 hostname，ipc namespace 改变 System V/POSIX IPC 视图，user namespace 改变 uid/gid 映射。

namespace	隔离内容	仍然共享
mount	文件系统挂载视图	同一内核 VFS 和底层设备风险
pid	进程编号和父子视图	调度器、内核内存、系统调用实现
net	网卡、路由、端口空间	协议栈代码和内核漏洞面
user	uid/gid 映射和部分能力	内核仍必须正确限制能力转换
ipc	IPC 对象名字空间	内核 IPC 实现

容器不是虚拟机，因为容器共享宿主内核。namespace 隔离的是名字和视图，不是内核代码本身。若内核某系统调用存在提权漏洞，容器内进程可能利用同一个内核漏洞逃逸。因此容器安全通常要叠加 seccomp、capability、只读挂载、cgroup 和及时更新内核。

cgroup：资源边界也是安全边界

没有资源限制，攻击者可以通过 fork bomb、内存耗尽、磁盘写满或 CPU 忙等影响整机。cgroup 给进程集合设置资源边界，例如 CPU 权重/配额、内存上限、进程数、I/O 权重和设备访问。

资源	没有限制的风险	期望证据
CPU	忙等拖慢其他服务	throttled time、quota hit 计数
memory	OOM 扩散到宿主	cgroup OOM 事件、victim、统计
pids	fork bomb 填满进程表	pids.current/max、fork 失败
I/O	大量写回拖慢共享磁盘	io latency、队列深度、权重
devices	访问宿主敏感设备	设备拒绝日志和 errno

资源限制要和观测一起配置。只设置 `memory.max` 但不记录 OOM 事件，故障时只能看到进程消失；只设置 CPU quota 但不看 throttled time，容易误判成应用锁或网络慢。

seccomp：收缩系统调用面

seccomp 允许进程限制自己可调用的系统调用。最小化系统调用面能减少内核攻击入口。网络服务正常处理请求可能只需要 `read/write/recv/send/epoll/futex/clock/mmap/brk/close` 等有限集合，不应该在运行阶段调用 `mount`、`ptrace`、`init_module`。

```
seccomp_policy:
  allow read, write, close
  allow epoll_wait, epoll_ctl
  allow recvfrom, sendto, accept4
  allow futex, clock_gettime
  allow mmap, munmap, brk
  deny mount, ptrace, bpf, init_module
```

allowlist 要从实际 trace 推导，而不是凭感觉写。测试时先在日志模式收集正常路径系统调用，再切到拒绝模式；同时要测试危险调用确实失败。seccomp 失败动作也要明确：返回 `errno`、kill 线程、kill 进程或记录审计。

安全日志：可审计但不泄密

安全事件日志需要足够结构化，便于按主体、对象、操作和原因查询。日志字段应包含 request id、pid、uid、容器/cgroup、操作、对象标识、策略名称和结果。敏感数据如 token、密码、密钥、完整请求体不应写入普通日志。

```
security_event:
  request_id=...
  pid=...
  uid=...
  cgroup=...
  operation=open_write
  object=/data/report
  policy=readonly_mount
  result=denied
  errno=EROFS
```

日志也要有完整性和留存策略。攻击者若能删除或篡改日志，事后审计会失效；日志若无限增长，又会变成磁盘耗尽风险。高安全系统通常把安全事件发送到独立收集端，并给本地日志设置限额和轮转。

性能观测：USE 与 RED 方法

系统性能观测可以用两套简单框架。USE 关注资源：Utilization、Saturation、Errors，即资源使用率、排队饱和度和错误。RED 关注服务请求：Rate、Errors、Duration，即请求速率、错误率和耗时分布。

框架	适用对象	指标例子
USE	CPU、内存、磁盘、网卡、锁、队列	CPU busy、runqueue、EIO、drop、reclaim
RED	HTTP/RPC/数据库请求	QPS、5xx、p50/p99、超时

这两套方法要结合。用户看到的是 RED：请求慢或失败；工程师定位要落到 USE：哪个资源饱和或错误。比如 p99 上升可能来自 CPU runnable wait、锁等待、page fault、dirty writeback、网络重传或 cgroup throttling。

tracepoint、采样和事件日志

观测手段有不同成本。计数器成本低，但细节少；事件 trace 细节多，但可能扰动系统；采样 profiler 能看到热点，但对短暂事件和 off-CPU 等待不敏感；分布式 trace 能申请求路径，但需要上下文传播。

手段	适合回答	局限
计数器	是否发生、发生多少次	难以解释单个请求
直方图	延迟分布和尾部	需要合理 bucket
事件 trace	谁在何时做了什么	数据量大，需过滤
CPU sampling	on-CPU 热点在哪里	看不到 sleeping 时间
off-CPU trace	线程等待在哪里	栈采集和符号成本

一个成熟诊断通常先看低成本指标定位方向，再打开更细粒度 trace 验证假设。直接全量抓所有事件，往往会制造过多噪声和额外开销。

panic、oops 和可恢复错误

内核错误并非都要 panic。用户传坏指针应返回 EFAULT；设备 I/O 失败应向上返回 EIO；某个驱动检测到可重置设备错误，可以隔离设备并恢复。panic 适用于内核不变量被破坏且继续运行可能造成更大损坏的情况，例如内核栈破坏、关键锁结构损坏、无法恢复的内存一致性错误。

错误类型	处理	证据
用户错误	返回 errno 或发信号	syscall、pid、地址、errno
设备错误 资源不足	重试、reset、上报 EIO ENOMEM/OOM/限流	设备状态、请求 id、超时 cgroup、分配大小、victim
内核 bug	oops 或 panic	寄存器、栈、锁、CPU、taint

panic 路径要少依赖。此时文件系统、调度器或锁可能已经不可信，所以 crash dump 和 console 输出应尽量使用简单、预留或早期初始化的路径。复杂日志系统在 panic 中可能再次阻塞或破坏证据。

故障注入：证明错误路径真的可用

错误路径如果不测试，基本等于没实现。故障注入让内核或应用在可控点返回错误：分配失败、copy 用户失败、磁盘第 N 次写失败、网络短写、时钟跳变、锁超时、设备 completion 丢失。

```
fault_injection_plan:
  fail allocation at callsite X once
  force copy_from_user to fail after 128 bytes
  return EIO on block write number N
  drop network response after commit
  delay wakeup by 500ms
```

每个注入点都要有预期：资源是否释放，引用计数是否回到原值，目标对象是否保持不变，错误是否传给调用者，日志是否足够诊断。故障注入把“理论上能处理错误”变成可验证事实。

安全与观测的最终验收

一个边界合格的系统必须同时满足三件事：正常路径能完成；越权路径明确失败且没有副作用；失败路径留下足够证据，能说明主体、客体、操作、原因和

影响范围。

第 7 章 实践方法与验收

7.1 实践路线与验收标准

实践方法与验收

原理

操作系统教材的实践目标不是写一个玩具内核名字，而是把关键状态机写清楚。调度器保护的是任务状态；页表保护的是地址和权限；文件描述符表保护的是对象生命周期。每个模块都必须能输出诊断，让错误路径可复查。

验收的顺序是：状态机正确，权限边界正确，生命周期正确，等待和关闭正确，持久化边界正确，然后再讨论性能和源码阅读。没有这些基础，读 Linux 源码也只会变成函数名堆砌。

本册暂时不展开实践卷工程，但理论卷也必须有实践方法。每章都要能落成一份小报告：问题是什么，抽象对象是什么，不变量是什么，正常路径如何推进，错误路径如何退出，哪些证据证明结论，哪些真实内核行为没有覆盖。

源码阅读要服务现象。不要从内核源码第一个文件开始顺序读，也不要吧函数名搬进笔记。正确方式是从一个现象入口进入：为什么线程睡眠、为什么缺页、为什么 fd 仍有效、为什么 `write` 可能短写、为什么 `fsync` 慢、为什么信号打断系统调用。读源码时只追踪足以解释现象的对象和状态。

验收不是追求“模型覆盖真实 OS 全部细节”。教学模型必须小，但小模型不能破坏核心契约。调度模型可以没有 CFS/EEVDF 细节，但不能调度 blocked 线程；页表模型可以没有多级页表，但不能忽略权限；fd 模型可以没有完整 VFS，但不能把 dup 后引用关系写错；持久化模型可以不实现真实文件系统，但不能把 write 成功当成业务提交。

实践

理论卷推荐使用下面的报告结构：

```
Title:
    要解释的 OS 现象或机制

State:
    涉及的内核对象、用户态对象、硬件对象

Invariant:
    必须始终成立的关系

Operation:
    正常路径的状态转移

Failure:
    错误路径、取消路径、关闭路径、崩溃点

Evidence:
    命令、trace、日志、计数器、测试断言

Boundary:
    当前模型没有覆盖的真实内核细节
```

每章至少完成一个纸上状态机。调度章写 runqueue；同步章写有界队列；锁章写锁顺序图；系统调用章写参数检查；exec 章写提交点；内存章写页表翻译和缺页；mmap 章写 VMA 到 page cache 的路径；块层章写 bio 到 request 的转换；I/O 章

写 fd 引用图；socket 章写 accept/send/recv 等待队列；panic 章写失败回滚；持久化章写提交状态机；观测章写一次性能或卡死诊断报告。

实践时要避免两个极端。第一个极端是只画概念图，不写错误路径；第二个极端是直接读真实内核大段源码，却没有抽象状态。合格学习路径是先有小状态机，再用真实系统现象校验它的边界。

逐章项目

本册理论卷可以配一条纸上项目线，不需要先写完整内核：

1. 写一个裸机超级循环，列出每个任务的周期和最坏执行时间。
2. 把一个阻塞 I/O 函数改成状态机，证明它每次 step 都会返回。
3. 加入中断事件队列，证明 ISR 只做短路径工作。
4. 写启动状态机：BSS、data、早期 console、页表、调度器、init 任务。
5. 加入定时器和时间片，写出上下文保存字段。
6. 实现 FCFS、RR、优先级和 MLFQ 的调度表模拟。
7. 写 lost wakeup 测试，证明等待队列和谓词由同一把锁保护。
8. 为有界队列写等待谓词、关闭路径和背压策略。
9. 写锁顺序图、lost wakeup 反例和 lockdep 检查思路。
10. 写系统调用分发、用户指针复制和错误码矩阵。
11. 写 exec 装载流程：ELF 验证、段映射、用户栈、CLOEXEC、提交点。
12. 手算页表翻译、COW 写故障和 Clock 页面置换。
13. 画 mmap 缺页路径，区分 shared/private、dirty page 和 msync。
14. 实现 buddy/slab 的纸上分配释放轨迹，检查碎片和守恒。
15. 画驱动描述符环、DMA 映射、completion 和 reset 清理路径。
16. 设计块层 bio 合并、deadline 调度、flush/FUA 顺序测试。
17. 画 fd 引用图、page cache 读写路径和异步 completion。
18. 设计 inode、目录项、bitmap 和 VFS 的最小文件系统模型。
19. 画 socket bind/listen/accept/send/recv 的状态机和等待队列。
20. 设计临时文件、fsync、rename、manifest 的可靠提交协议。
21. 写 capability、namespace、cgroup、seccomp 的最小权限报告。
22. 写 errno、信号、WARN、panic 的错误分类和回滚路径报告。
23. 为一次故障写现象、假设、证据、反证、边界报告。

评分矩阵

能力	合格	高质量
状态机	能列出状态和转换	能列出错误、取消、关闭、崩溃边界
算法	能写伪代码	能分析复杂度、参数和不变量

实现	能说明数据结构	能说明并发、权限和生命周期风险
测试	覆盖正常路径	覆盖边界路径、反例和故障注入
观测	有日志或命令输出	有可复现证据、反证和未验证边界

测试

最低验收包括：round-robin 调度、blocked/terminated 过滤、锁顺序检查、页表权限、unmap、exec 失败回滚、fd dup、fd close 和未知句柄诊断。扩展验收包括：优先级 aging、copy-on-write、mmap shared/private、块层 flush 顺序、socket 非阻塞等待、短写、崩溃恢复和设备阻塞。

- 调度：runnable、running、blocked、zombie、reaped 的状态转换可解释。
- 同步：等待谓词完整，关闭路径能唤醒所有等待者。
- 锁：锁保护对象明确，锁顺序无环，持锁上下文不违反睡眠规则。
- 内存：地址翻译保留 offset，权限错误和合法缺页能区分。
- syscall：用户指针、fd、权限和错误码都有检查。
- exec：ELF 验证、段映射、用户栈和提交点可解释。
- mmap：VMA、PTE、page cache 与 dirty/writeback 状态一致。
- I/O：fd、open file description、inode、page cache 和设备请求不混淆。
- 驱动：描述符环、DMA buffer 和中断下半部生命周期正确。
- 块层：bio 合并、request 派发、flush/FUA/barrier 顺序可验证。
- 文件系统：路径解析、inode、目录项和空闲空间管理可解释。
- socket：阻塞、非阻塞、poll、epoll 与等待队列关系明确。
- 安全：最小权限、namespace、cgroup、seccomp 有明确边界。
- 错误恢复：errno、信号、WARN、panic 和回滚路径不混用。
- 持久化：write、fsync、rename、manifest 的提交边界清楚。
- 观测：每个结论有证据和反证，不把猜测写成事实。
- 源码阅读：能说明入口、对象、状态变化、返回给用户态的现象。

测试失败时要先问是哪条不变量断了：任务状态、地址权限、对象引用、等待谓词、提交边界还是恢复规则。把失败归到不变量，比只看报错信息更接近真实 OS 调试。

7.2 练习、案例和错误分析

综合案例与练习卷

一本操作系统书要真正有用，必须让读者能把章节串起来。本节给出几个贯穿案例。它们不是实践卷工程，但可以作为理论卷的验收题：读者应能写出状态机、关键对象、不变量、失败路径和测试证据。

案例一：shell 执行 cmd > out.txt

这个案例串起进程、fd、路径、exec、调度和文件写入。

```
shell:
fd = open("out.txt", O_CREAT | O_TRUNC | O_WRONLY)
pid = fork()
if pid == 0:
    dup2(fd, STDOUT_FILENO)
    close(fd)
    execve("cmd", argv, envp)
close(fd)
waitpid(pid)
```

要求回答：

1. fork 后父子进程的地址空间、fd 表和 open file description 如何关联？
2. dup2 后 stdout 指向哪个 file object？
3. exec 成功后哪些 fd 保留，哪些会因为 CLOEXEC 关闭？
4. cmd 写 stdout 时，数据经过哪些内核对象？
5. waitpid 为什么需要 zombie 状态？

案例二：HTTP 服务端处理一个请求

这个案例串起 socket、epoll、线程池、内存分配、文件读取和 send。

```
event loop:
events = epoll_wait()
for event in events:
    if event.fd == listen_fd:
        accept connections
    else:
        read request bytes
        parse frame
        dispatch worker

worker:
    open file
    read through page cache
    send response with short-write loop
```

要求回答：

1. listen socket 和 accepted socket 是不是同一个对象？
2. 为什么一次 recv 不等于一个完整 HTTP 请求？
3. 文件热缓存和冷缓存下，read 路径差异在哪里？
4. send 返回 EAGAIN 时，用户态状态机如何保存剩余数据？
5. 服务端关闭连接时，哪些等待队列要被唤醒？

案例三：数据库提交一条记录

这个案例串起 page cache、日志、fsync、rename、块层 flush 和崩溃恢复。

```
transaction:
    append WAL intent
    fsync WAL
    update data page
    append commit record
```

```
fsync WAL
later checkpoint data pages
```

要求回答：

1. 为什么 intent 记录要先于数据页持久化？
2. commit record 写入前崩溃，恢复应如何处理？
3. commit record 写入后数据页未写完，恢复为什么可以 redo？
4. checksum 如何帮助识别 torn write？
5. checkpoint 为什么不能截断尚未安全覆盖的数据所需日志？

案例四：系统变慢的诊断

现象：请求 p99 从 80ms 升到 3s，CPU 使用率不高。

要求写出诊断路径：

1. collect thread states
2. split on-CPU and off-CPU time
3. inspect syscall latency
4. inspect page fault and reclaim
5. inspect block I/O queue and flush
6. inspect cgroup throttling
7. form evidence chain

合格报告必须说明排除了什么。比如 CPU profiler 没有热点，只能说明 CPU 不是主因，不能自动证明磁盘是主因；还需要 syscall trace、block trace 或 dirty page 证据。

期末式综合题

1. 解释从定时器中断到任务抢占的完整路径。要求包含 trap frame、need_reschedule、preempt_count、runqueue 和上下文切换。
2. 解释从用户态 `write(fd, buf, len)` 到 page cache dirty page 的完整路径。要求包含 fd、file、inode、用户指针复制、短写和错误码。
3. 解释从 mmap 文件页第一次读取到 page fault 完成的完整路径。要求包含 VMA、PTE、page cache、块 I/O 和 TLB。
4. 解释一个设备 DMA 写入内存后，CPU 如何安全读取数据。要求包含 DMA map、cache invalidate、completion 和 buffer 生命周期。
5. 解释容器隔离为什么不是虚拟机。要求包含 namespace、cgroup、capability、seccomp、共享内核和逃逸风险。
6. 设计一个崩溃一致的多文件发布协议。要求包含临时文件、fsync、rename、父目录 fsync、manifest、checksum 和恢复规则。

综合题评分重点不是背 API，而是能不能把对象、状态、权限、等待、提交和证据串成闭环。只写正常路径不给满分；必须写错误路径和边界。

参考答案框架

综合题不要求背 Linux 函数名，而要求结构完整。推荐按下面框架答题：

Objects:

涉及哪些用户态、内核、硬件对象

Normal path:

正常路径逐步推进，每步改变什么状态

Permission:

哪些地方做权限或边界检查

Waiting:

哪些地方可能阻塞，等待队列在哪里

Failure:

每个失败点如何返回、回滚或提交部分结果

Observation:

用什么日志、trace、计数器或测试证明结论

这个框架能防止答案只写“调用 A 然后调用 B”。操作系统题目的本质是对象和状态，不是函数名接龙。

样例解析：从 write 到持久化

一个完整答案应类似下面这样展开：

1. 用户态调用 `write(fd, buf, len)`，fd 是进程局部整数。
2. 内核在 fd table 找到 file object，检查是否允许写。
3. 内核用 `copy_from_user` 分段复制用户 buffer，失败返回 `EFAULT`。
4. 普通文件写入 page cache，相关页变 dirty，inode size 可能更新。
5. `write` 返回写入字节数；这不是持久化边界。
6. 后台 `writeback` 或 `fsync` 构造 bio，经块层和驱动提交到设备。
7. 设备 completion 后，页可从 `writeback` 变 `clean`。
8. 若要业务提交，还需要目录、rename、manifest 或日志协议。

错误路径也必须写：fd 无效返回 `EBADF`；用户指针坏返回 `EFAULT`；空间不足可能返回 `ENOSPC`；设备错误可能在 `fsync` 或 `close` 时报出。只写正常路径不合格。

样例解析：线程卡死

先不要猜锁。按顺序排查：

1. 线程是 `RUNNABLE` 还是 `SLEEPING`？
2. 若 `RUNNABLE`，看 `runqueue`、优先级、CPU 配额和抢占。
3. 若 `SLEEPING`，看 wait channel：锁、I/O、page fault、futex、timer 还是 `waitpid`。
4. 若等锁，找持锁者和锁顺序图。
5. 若等 I/O，看块层队列、设备完成和 dirty `writeback`。
6. 若等 futex/条件变量，看等待谓词和关闭路径是否唤醒。
7. 最后用 `trace` 或日志证明等待原因，而不是只看现象。

全书复习清单

- 能从硬件解释特权级、页表、中断、DMA 和启动顺序。

- 能从状态机解释进程、线程、等待、唤醒和退出。
- 能从边界解释系统调用、用户指针、错误码和权限。
- 能从缓存解释 page cache、mmap、writeback 和 fsync。
- 能从队列解释块层、设备驱动、socket 和 epoll。
- 能从恢复规则解释日志、manifest、checksum 和幂等。
- 能从证据链解释性能、安全、panic 和资源泄漏。

案例五：从按键中断到终端程序读到字符

这个案例把硬件、中断、驱动、TTY、等待队列和用户态读串起来。一个键盘或串口设备产生输入事件，设备控制器触发中断；驱动读取设备寄存器，把扫描码或字节放入内核缓冲；TTY 层处理行规程、回显和特殊字符；正在 `read` 的 shell 被唤醒，复制数据到用户 buffer。

```
keyboard interrupt:
  save trap frame
  driver reads scancode
  translate scancode to key event
  tty line discipline updates buffer
  wake read_waitq

shell read:
  if tty buffer empty:
    sleep(read_waitq)
  copy chars to user buffer
  return byte count
```

要求回答：

1. 中断处理函数为什么不能等待用户程序消费字符？
2. TTY 缓冲区满时，驱动和上层分别有哪些选择？
3. `Ctrl-C` 为什么不是普通字符那么简单？
4. shell 阻塞在 `read` 时，调度器看到的线程状态是什么？
5. 如果用户传入坏 buffer，错误应该在哪里被发现？

案例六：fork 后多线程程序的陷阱

多线程程序调用 `fork` 时，子进程通常只复制调用 `fork` 的那个线程。其他线程不在子进程继续运行，但它们持有的锁状态可能被复制进来。如果子进程在 `exec` 前调用复杂库函数，就可能等待一个永远没人释放的锁。

```
thread A:
  lock(malloc_lock)
  ... interrupted here ...

thread B:
  pid = fork()

child:
  // only thread B exists, but malloc_lock may look locked
  printf("hello") // may call malloc and deadlock
  execve(...)
```

要求回答：

1. fork 复制了哪些进程状态，哪些线程不会继续存在？
2. 为什么 fork 后 exec 前只能调用 async-signal-safe 的少量函数？
3. pthread_atfork 试图解决什么问题？
4. COW 如何降低 fork 成本，但不能解决锁状态复制问题？
5. shell 的 fork/exec 模型为什么在单线程 shell 中更简单？

案例七：内存压力导致网络服务超时

现象：服务端 CPU 不高，网络连接没有明显丢包，但请求大量超时。诊断发现 p99 卡在 send 或 write 附近。可能路径是：请求处理需要分配 buffer；内存压力触发回收；回收要写回脏页；块设备 flush 慢；线程 sleeping；socket 发送迟滞得不到调度。

```
request handler:
  allocate response buffer
  -> reclaim
  -> dirty page writeback
  -> block device wait
  -> scheduler sleeps current
  -> client timeout
```

要求回答：

1. 为什么 CPU 低不代表服务健康？
2. 哪些指标能区分锁等待、I/O 等待和 cgroup throttling？
3. dirty page 背压和 socket 背压有什么相似处？
4. 如果 OOM killer 触发，应该在日志中看到哪些字段？
5. 如何设计限流，让服务在内存压力下早失败而不是拖到超时？

案例八：容器内服务被限制后无法写日志

一个容器服务启动正常，但运行时写日志返回只读文件系统错误。排查发现根文件系统以只读方式挂载，只有 /var/log/app 应该挂可写卷，但实际没有挂载。这个问题串起 mount namespace、VFS、路径解析、权限和观测。

```
open("/var/log/app/current.log", O_CREAT|O_APPEND|O_WRONLY)
-> path lookup in container mount namespace
-> parent directory resolves to readonly mount
-> VFS denies write
-> return -EROFS
```

要求回答：

1. 为什么宿主上路径存在，不代表容器 namespace 中可见？
2. EACCES、EROFS、ENOENT 分别说明什么？
3. 这个失败应记录哪些安全/运维字段？
4. capability 能不能绕过只读挂载？为什么不应依赖它？
5. 测试镜像时如何证明只读根文件系统和可写日志卷都按预期工作？

算法推导题：三种页面置换策略

给定页面访问序列：

```
7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2
```


物理页框数量为 3。分别手算 FIFO、LRU 和 Clock 的缺页次数，并回答：

1. 为什么 FIFO 可能出现 Belady anomaly?
2. 精确 LRU 的维护成本在哪里?
3. Clock 的 referenced bit 何时清零，何时给第二次机会?
4. 脏页和干净页在回收成本上有什么差异?
5. 文件页和匿名页在回收路径上有什么差异?

算法推导题：调度延迟和响应时间

给定三个任务：

任务	到达时间	CPU 总需求	行为
A	0	24 tick	CPU 密集
B	2	6 tick	每运行 1 tick 后 I/O 等待 4 tick
C	4	3 tick	短任务

分别分析 FCFS、RR（时间片 2）、MLFQ（两级队列）下的完成时间、平均等待时间和 B 的最大响应延迟。要求说明：

1. 为什么 FCFS 会产生 convoy effect?
2. 时间片变小如何影响响应和上下文切换成本?
3. MLFQ 为什么倾向于保留 B 的高优先级?
4. 若 A 持有 B 需要的锁，单纯提高 B 优先级是否足够?
5. 如何用 trace 证明响应延迟来自调度而不是 I/O?

算法推导题：日志恢复

给定如下日志片段，其中每条记录都有 LSN、事务号、类型和 checksum。假设最后一条记录 checksum 损坏：

```
LSN 10: TX1 INTENT write page P old=a new=b checksum ok
LSN 11: TX1 COMMIT checksum ok
LSN 12: TX2 INTENT write page Q old=x new=y checksum ok
LSN 13: TX2 COMMIT checksum bad
```

回答：

1. redo 日志恢复时 TX1 和 TX2 分别如何处理?
2. 如果是 undo 日志，记录内容和恢复规则会有什么不同?
3. 为什么 checksum 损坏后不能继续信任后续日志尾部?
4. checkpoint 能缩短哪些扫描，不能省略哪些安全检查?
5. manifest 发布协议如何用类似思想只采用完整版本?

设计题：最小教学内核的验收标准

虽然本册暂不实现实践卷，但读者应能写出一个最小教学内核的验收标准。该内核不必追求性能，但必须证明机制边界清楚。

1. 启动：能从 bootloader 入口建立栈、清 BSS、输出早期日志、解析内存地图。

2. 中断：能注册时钟中断和一个简单设备中断，保存 trap frame 并返回。
3. 调度：至少支持两个内核线程抢占运行，阻塞线程不会被选中。
4. 同步：mutex 或 sleep queue 不丢唤醒，关闭路径唤醒所有等待者。
5. 虚存：能建立用户页表，用户态不能访问内核页，缺页错误可诊断。
6. 系统调用：能验证用户指针，返回区分 EFAULT/EBADF/EINVAL 的错误。
7. 文件抽象：至少有 fd 表和一个内存文件对象，支持 read/write/close 生命周期。
8. 观测：panic 输出 CPU、RIP、RSP、当前线程、最近系统调用和锁状态。

验收不是看功能数量，而是看每个功能是否有不变量、错误路径和测试证据。一个小而严谨的内核，比一个功能多但边界不清的内核更适合学习。

答题评分细则

为了把“会背概念”和“理解系统”区分开，综合题按五类给分：

维度	满分特征
对象	明确列出用户对象、内核对象、硬件对象和引用关系
状态	写出正常状态转换、等待状态、关闭状态和错误状态
边界	说明权限、用户指针、生命周期和持久化边界
算法	给出关键算法或伪代码，并说明复杂度或取舍
证据	给出测试、trace、日志、计数器或崩溃恢复验证办法

扣分规则也很明确：只写正常路径扣状态分；只写 API 名字扣对象分；不写错误码扣边界分；不写测试证据扣证据分；把 write 返回当作持久化提交、把 TCP 当作消息协议、把容器当作虚拟机，属于概念性错误。

案例九：内核模块加载与 device mapper 创建

这个案例串起权限、ELF/模块格式、符号解析、内核内存分配、设备模型和块层。加载 device mapper target 时，用户态通常通过 modprobe 或内核自动加载模块，随后通过 dm ioctl 创建映射设备。

```
modprobe dm-snapshot
-> finit_module()
-> check CAP_SYS_MODULE and LSM
-> copy module image from fd
-> verify ELF sections and relocations
-> resolve exported symbols
-> allocate module memory
-> apply relocations
-> run module_init

dmsetup create vg-lv
-> ioctl(/dev/mapper/control)
-> parse target table
-> create mapped device
-> register gendisk/request path
```

要求回答：

1. 为什么加载模块需要比普通 open 更强的 capability?
2. 模块初始化失败时，已分配的 text/data、sysfs 节点和设备注册如何回滚?
3. device mapper 如何把上层 bio 映射到底层设备?
4. flush/FUA 请求穿过 dm target 时为什么不能丢失顺序语义?
5. 观测时应看哪些证据：dmesg、/sys/module、/dev/mapper、/proc/devices 还是 block trace?

案例十：容器从 clone 到 exec 的完整路径

容器启动不是一个单独系统调用。它组合 clone/unshare、namespace、cgroup、mount、capability、seccomp、pivot root/chroot 和 exec。

```
runtime:
  clone with CLONE_NEWUSER | CLONE_NEWPID | CLONE_NEWNS | CLONE_NEWNET
  write uid_map/gid_map
  configure cgroup v2 limits
  setup mount namespace and readonly rootfs
  drop bounding/ambient capabilities
  set no_new_privs
  install seccomp filter
  execve(container_init)
```

要求回答：

1. user namespace 为什么要先建立 uid/gid 映射?
2. pid namespace 中的 1 号进程退出时，容器内其他进程会怎样?
3. mount namespace 的只读根和可写 volume 如何同时存在?
4. seccomp 应在 exec 前还是后安装？各有什么取舍？
5. 如何证明容器不能 ptrace 宿主进程、不能 mount 宿主路径、不能突破内存限制？

案例十一：TCP 服务端从 socket 到 epoll

这个案例把 fd、socket、网络栈、等待队列和 epoll ready list 串起来。

```
server:
  listen_fd = socket()
  bind(listen_fd, addr)
  listen(listen_fd, backlog)
  epoll_ctl(ep, ADD, listen_fd, EPOLLIN)

event:
  packet arrives
  -> NIC/NAPI
  -> IP/TCP
  -> established socket receive queue
  -> sk_data_ready
  -> ep_poll_callback
  -> epoll_wait returns fd
```

要求回答：

1. listen socket 和 accepted socket 在内核对象上有什么区别？
2. SYN queue、accept queue、receive queue 分别堵塞时表现如何？

3. EPOLLET 下为什么必须读到 `EAGAIN`?
4. 慢客户端导致 send buffer 堵塞时, 服务端状态机保存什么?
5. 如何用 ss、`/proc/net/tcp`、perf/ftrace 或 eBPF 证明瓶颈在网络栈还是应用队列?

案例十二：文件系统从 mount 到 fsck

挂载文件系统时, 内核读取 superblock, 建立 VFS super_block、root dentry 和 inode 缓存。崩溃后, journal replay 或 fsck 负责恢复结构一致性。

```
mount /dev/sdb1 /mnt
-> open block device
-> read superblock
-> validate block size and feature flags
-> load journal if present
-> create VFS super_block
-> instantiate root dentry

after crash:
-> journal replay committed transactions
-> fsck scans inode/block bitmap if needed
```

要求回答:

1. mount 时为什么要检查 feature flags?
2. journal replay 和 fsck 分别解决什么问题?
3. ext4 ordered 模式为什么要求数据先于相关元数据 commit?
4. orphan inode list 的作用是什么?
5. fsck 如何发现 bitmap 标记空闲但 inode 引用该块的矛盾?

案例十三：从按键到字符设备到 read

这个案例扩展前面的终端输入路径, 要求读者把字符设备对象写清楚。

```
keyboard IRQ
-> input driver reads scancode
-> input subsystem emits key event
-> tty line discipline buffers char
-> wake read waitqueue
-> read(/dev/tty, buf, len)
-> copy_to_user
```

要求回答:

1. 字符设备如何通过 major/minor 找到 file_operations?
2. IRQ handler、input core、TTY line discipline 分别保存什么状态?
3. canonical mode 下为什么要等换行才返回?
4. Ctrl-C 如何变成 SIGINT, 而不是普通字符?
5. 用户 buffer 坏时, 已经从 TTY buffer 取出的字符如何处理?

案例十四：内存压力下的页面回收与 OOM 决策

现象: 服务进程不断分配内存, 系统延迟升高, 最后某个进程被杀。

```
allocation slow path
-> direct reclaim
-> shrink_lruvec
-> writeback dirty file pages
-> shrink_slab
-> compaction
-> still below watermark
-> OOM select victim
-> kill and reap memory
```

要求回答：

1. active/inactive LRU 如何帮助选择回收页？
2. 干净文件页、脏文件页、匿名页、mlock 页分别如何处理？
3. cgroup OOM 和全局 OOM 的影响范围有何不同？
4. OOM killer 为什么不能随便杀内核线程或关键服务？
5. 如何用 `/proc/meminfo`、PSI、tracepoint 和 OOM 日志形成证据链？

案例十五：系统调用被信号中断后的重启路径

阻塞系统调用遇到信号时，内核可能返回 `EINTR`，也可能在 handler 返回后重启。这个案例考察系统调用边界、等待队列、信号投递和用户可见语义。

```
read(fd, buf, len)
-> no data, sleep interruptible
-> signal arrives
-> wake task
-> prepare signal frame
-> handler runs in user mode
-> sigreturn
-> restart syscall or return EINTR
```

要求回答：

1. 哪些等待可以被信号中断，哪些不应被普通信号中断？
2. 已经完成部分读写时，为什么不能简单重启？
3. `SA_RESTART` 改变了什么，不改变什么？
4. 内核内部的 `ERESTARTSYS` 一类返回值为什么不直接暴露给用户？
5. `strace` 中如何观察 restarted syscall？

7.3 C++20 参考实现阅读

C++20 参考实现：把原理落实成可运行数据结构

本节把前文反复出现的数据结构和算法放进一份可编译的 C++20 参考实现。它不是要替代真实 Linux 内核，也不是实践卷的完整 OS，而是服务原理卷：每个机制都用尽量小的模型保留关键不变量，让读者能看到“为什么必须这样维护状态”。浅层教材常把操作系统讲成一组名词：进程、线程、页表、文件、网络。真正的理解要再往下走一层：这些名词在内核里都要变成可更新的数据结构，每一次更新都要满足状态不变量，每一个不变量破坏后都会出现可复现的错误。

核心思想

操作系统原理的落点不是“知道某个机制存在”，而是能回答四个问题：为什么需要这个机制，内核用什么数据结构保存它，哪些状态转换必须原子完成，测试怎样证明错误路径也被覆盖。本节的参考实现按这个标准组织。

这份代码覆盖十四组对象：

- CFS 风格 runqueue：用 `std::set` 表达红黑树按 `vruntime` 选择最亏任务。
- 等待队列：展示 `sleeping/runnable` 状态转换和 `wakeup` 的所有权。
- `futex` 字：展示用户态值比较与内核等待队列之间如何避免 `lost wakeup`。
- `wait-for graph`：用非递归拓扑剥离检测死锁环。
- 伙伴分配器：展示按阶拆分、按 `buddy` 地址合并的物理页框分配。
- 页表、TLB 和 COW：展示 `fork` 后父子页表如何收紧权限，写入时如何复制页。
- LRU page cache：展示冷热页、dirty 页、回收和写回之间的关系。
- `extent map`：展示文件逻辑块到物理块的区间映射。
- `deadline block scheduler`：展示寻道优化不能饿死过期请求。
- IOMMU：展示设备 IOVA 到页的映射、方向权限和 `unmap` 后 `IOTLB` 失效。
- NVMe completion queue：展示 `phase tag` 如何区分环形队列的新旧 completion。
- `fd/file/inode`：展示 `fd` 整数、`open file description`、`inode` 的三层关系。
- `journaling block device`：展示先写日志、崩溃后重放、再清空日志的恢复语义。
- `TCP connection state`：展示 `SYN`、`ESTABLISHED`、重传定时器、`FIN` 和 `TIME-WAIT` 的最小状态机。

为什么参考实现不能只写 `happy path`

操作系统最常见的错误不在正常路径，而在“刚好被打断”的地方：任务刚要睡眠时事件已经发生，`fork` 刚收紧 PTE 但 TLB 还保留 `writable` 翻译，块设备写了日志但还没写 `home block` 就掉电，TCP 发出数据后 `ACK` 丢失，`page cache` 回收 `dirty` 页时漏掉写回。为了把这些边界说清楚，参考实现每个模块都配了测试。测试不是为了展示一串绿灯，而是把不变量写成可执行断言。

模块	关键不变量	测试覆盖的反例
runqueue	runnable 任务只能在一个可运行集合中，running 任务不能同时留在树里	阻塞任务先出调度路径，再进入 <code>wait queue</code>
futex	用户态值仍等于期望值时才允许睡眠	值已变化时 <code>wait</code> 立即失败，不能丢 <code>wakeup</code>
deadlock	<code>wait-for graph</code> 有环才是死锁	链式等待不报错，加入回边后报错
buddy	每个空闲页框只属于一个阶的一个块	释放相邻页后必须合并回高阶块
COW	<code>fork</code> 后父子可写页都先变只读，写入方独占复制	子写不影响父，父再写不影响子

LRU	clean 回收不写盘, dirty 回收必须写回	淘汰 clean 页不增加写回计数, 淘汰 dirty 页增加
journal	已提交日志崩溃后必须可重放, 未落 home block 前不能假装完成	crash point 后 home block 为空, recover 后内容出现
TCP	只要还有未确认数据就要有重传定时器	RTT 到期触发重传, ACK 后取消定时器

调度：公平不是轮流，而是维护欠账

最简单的调度器可以把任务放进 FIFO 队列：谁先来谁先跑。这个模型在单片机前后台循环中还能凑合，一旦任务权重不同、任务会睡眠、I/O 密集任务需要低延迟唤醒，FIFO 就不够了。CFS 的核心不是“每个任务跑同样时间”，而是“按权重维护虚拟运行时间”。权重高的任务同样运行 1ms，`vruntime` 增长更慢，因此它在红黑树里更容易保持靠左，得到更多 CPU。

参考实现中 `CfsRunQueue` 用 `std::set` 模拟红黑树。排序键是 `vruntime`，若相等再用 `task id` 打破平局。这个设计体现两个内核事实。第一，`pick next` 只需要取最左实体，因此是有序集合的最小元素。第二，任务运行后 `vruntime` 会变化，必须先从树里取出、更新账本、再按新 `key` 插回去；如果直接修改树中元素的 `key`，平衡树顺序就被破坏。

```
weighted_delta = delta_exec * NICE_0_WEIGHT / task.weight
task.vruntime += weighted_delta
runqueue.insert(task)
```

复杂度来自操作需求：插入、删除、取最小都是 $O(\log n)$ 或更好。链表可以做到入队 $O(1)$ ，但每次寻找最小 `vruntime` 都要扫描；堆取最小很快，却不适合任意任务睡眠时从中间删除。调度器的算法选择不是抽象偏好，而是睡眠、唤醒、迁移、调权这些操作共同逼出来的。

睡眠唤醒：lost wakeup 是状态顺序错误

等待队列看起来像一个普通队列：任务睡眠时入队，事件发生时唤醒。但真正的问题是条件检查和入队必须连在一起。若线程先检查“条件不满足”，准备睡眠；就在它还没入队时，另一个 CPU 改变条件并执行 `wake`；随后第一个线程才入队睡下去，就再也没人叫醒它。这就是 `lost wakeup`。

参考实现把 `WaitQueue` 和 `FutexWord` 分开，是为了说明两层语义：普通等待队列只保存睡眠任务，`futex` 还要比较用户态字的当前值。真实 `futex` 的快路径完全在用户态完成，只有比较失败或需要阻塞时才进入内核；慢路径进入内核后，必须重新验证用户态值仍等于期望值，才能把任务挂到等待队列。

```
if futex_word != expected:
    return EAGAIN
enqueue current into futex wait queue
schedule()
```

这里的 `EAGAIN` 不是小细节，而是避免 `lost wakeup` 的关键返回值。它告诉用户态锁：值已经变了，你不应该睡眠，应当重新尝试获取锁。参考测试先构造一个值不相等的 `futex wait`，要求任务仍保持 `running`；再构造值相等路径，要求任务进入 `sleeping`，随后 `wake_one` 把它恢复为 `runnable`。

死锁检测：不是“卡住”就叫死锁

死锁的严格定义是等待关系中存在环。任务 A 等 B，B 等 C，这只是阻塞链；只有 C 又等 A，才形成任何一个任务都无法推进的闭环。内核中很多锁不会默认做全局死锁检测，因为维护全局 `wait-for graph` 成本高，且锁路径极热。但教学实现可以把这个图显式建出来，让“死锁”从感觉变成算法。

参考实现的 `DeadlockDetector` 维护边 `waiter -> owner`，检测时不用递归

DFS，而用 Kahn 拓扑剥离：先找入度为 0 的节点，把它们和出边不断删掉；如果最后还有节点没被剥掉，剩下部分必然在环里。这个算法复杂度是 $O(V + E)$ ，也适合内核风格的显式队列实现。

```
ready = all nodes with indegree 0
while ready not empty:
    node = pop_front(ready)
    for successor in graph[node]:
        indegree[successor]--
        if indegree[successor] == 0:
            push_back(successor)
cycle exists if visited != node_count
```

这个实现还展示了清理边的重要性。任务退出、锁释放、超时取消后，等待边必须删除；否则检测器会看到已经不存在的等待关系，误报死锁。测试中先建立 $1 \rightarrow 2 \rightarrow 3$ ，不应报死锁；加入 $3 \rightarrow 1$ 后必须报；清理任务 2 后环断开。

伙伴分配：为什么物理页框不是随便找一页

页分配器若只维护一个空闲页链表，分配单页很简单，但分配连续页会越来越困难。DMA、巨大页、页表页、某些设备缓冲区都可能需要连续物理页。伙伴系统把内存按 2^k 页为一块管理：大块可以拆成两个同阶 buddy，小块释放后若 buddy 也空闲就合并回大块。

参考实现的 `BuddyAllocator` 保留三个核心状态：每个 order 的 free list、已分配块的起始页、已分配块的 order。分配时先把请求页数向上取整到 2^k ，再找不小于该 order 的最小空闲块，逐级拆分。释放时用 $\text{start_page} \wedge (1 \ll \text{order})$ 找 buddy；只要 buddy 在同阶 free list 中，就移除 buddy 并合并到更高阶。

```
buddy = start_page XOR (1 << order)
if buddy is free at same order:
    remove buddy
    start_page = min(start_page, buddy)
    order++
else:
    insert current block into free_list[order]
```

这个 XOR 公式依赖块按 2^k 对齐。它说明了为什么伙伴分配器要用阶管理，而不是任意长度区间：只有对齐块才有唯一 buddy。测试分配第 0 页和第 1 页，再依次释放，最后必须合并回最高阶的一个完整空闲块。若释放路径忘记合并，短期测试也许能过，长时间运行后连续页会耗尽。

页表、TLB 与 COW：权限收紧必须让 CPU 看见

COW 的表面目标是 fork 快：父子先共享物理页，等写入时再复制。它真正困难的地方是权限传播。fork 时，父进程原来 writable 的 PTE 必须被改成 read-only，子进程也安装 read-only PTE，并标记 COW。若只改子页表，父进程写共享页不会 fault，会直接破坏子进程视图。

还有一个硬件层问题：CPU 可能已经把旧的 writable PTE 缓存在 TLB 里。页表改成只读后，如果不失效对应 TLB 项，CPU 仍可能用旧翻译完成写入。参考实现的 `fork_cow` 在收紧父 PTE 后调用 `parent_tlb.invalidate_page`，这正是把软件页表变化同步给硬件翻译缓存的动作。

```
parent.pte.writable = false
parent.pte.cow = true
child.pte = parent.pte
invalidate parent TLB entry
```

写 fault 时还要分两种情况。若物理页只剩当前地址空间引用，内核可以直接把 PTE 改回 writable；若仍被父子共享，就必须分配新页并复制内容。参考实现用 `shared_ptr::unique()` 表达引用计数判断，虽然真实内核用 `struct page` 的

refcount，但不变量相同：共享页不能原地写。

LRU page cache：缓存不是越多越好

page cache 把文件内容缓存在内存中，读文件先查缓存，写文件通常先改缓存页并标 dirty，再由回写路径落盘。缓存的核心矛盾是内存有限：保留热页能加速访问，但冷页必须被回收；回收 clean 页只需丢弃，回收 dirty 页必须先写回，否则数据丢失。

参考实现的 `LruPageCache` 用 `std::list` 维护从热到冷的页，用哈希表从 page id 找到 list iterator。访问命中时把节点 splice 到表头，复杂度 $O(1)$ ；未命中且容量满时淘汰表尾。dirty 位决定淘汰是否增加 writeback 计数。

```
on access(page):
    if cached:
        move page to LRU head
    else:
        if full:
            evict tail, write back if dirty
        insert new page at head
```

真实内核的回收比这复杂得多：有 active/inactive list，有匿名页和文件页，有 cgroup、NUMA、dirty throttle、writeback congestion，还有 direct reclaim 和 kswapd。但最小模型已经展示了关键语义：冷热判断决定谁被牺牲，dirty 决定牺牲前是否需要 I/O。测试刻意先淘汰 clean 页，再淘汰 dirty 页，证明两条路径不同。

extent 与块调度：文件系统和设备之间还有一层翻译

文件系统不一定为每个逻辑块保存一个物理块号。连续区域可以用 extent 表示：逻辑起点、物理起点、长度。这样一个大文件的连续数据只需少量记录。参考实现的 `ExtentMap` 拒绝重叠 extent，并能把逻辑块 10 翻译成对应物理块 202。这说明文件读写不是直接把 offset 扔给磁盘，而是先经过文件系统布局翻译。

```
logical block 8..11 -> physical block 200..203
lookup(10) = 200 + (10 - 8) = 202
```

翻译完成后，请求进入块层。块调度器面对两个目标：减少寻道或队列跳转成本，同时不能让旧请求饿死。参考实现的 `DeadlineBlockScheduler` 给读写请求不同 deadline：读请求 deadline 短，写请求 deadline 长。dispatch 时若有过期请求，先发过期请求；否则选择离当前磁头位置更近的请求。SSD 上“寻道”概念弱化，但队列深度、合并、优先级、超时仍然存在。

策略	解决的问题	可能的代价
按 sector 接近度调度	降低机械盘寻道或设备队列跳转成本	远处请求可能等待太久
deadline	给请求最长等待时间	可能牺牲局部最优顺序
读优先	改善交互延迟	写回堆积时需要限流
请求合并	减少设备命令数量	合并等待可能增加单个请求延迟

IOMMU 与 NVMe：设备也有地址翻译和队列协议

没有 IOMMU 时，设备 DMA 可以直接访问物理地址，一旦驱动或设备出错，可能覆盖任意内存。IOMMU 给设备建立 IOVA 到物理页的映射，并附带方向权限：ToDevice 表示设备读内存，FromDevice 表示设备写内存。参考实现的 `IommuDomain` 在 unmap 时同时移除 IOTLB，强调设备侧翻译缓存也要失效。

NVMe completion queue 展示另一个硬件协议细节：环形队列槽位会被反复使用，主机怎么知道这个槽里的 completion 是新一轮还是上一轮留下的旧值？答案

是 phase tag。队列绕回时，设备 phase 翻转；主机期望 phase 也在读完一圈后翻转。参考实现初始化时把所有槽 phase 设成非期望值，避免空队列被误读成已有 completion。

```
if slot.phase != expected_phase:
    queue is empty from host view
else:
    consume completion
    advance head
    if head wrapped:
        expected_phase = !expected_phase
```

这个例子说明，驱动并不是“读写寄存器”这么浅。驱动要维护共享内存队列、缓存一致性、DMA 映射生命周期、中断或轮询、错误状态和设备协议。任何一个 bit 的初始值错了，都可能让主机读到不存在的完成事件。

fd、open file 与 inode：一个整数背后三层对象

用户态看到的 fd 是整数，但内核不是把文件内容直接挂在整数上。fd 表项指向 open file description，open file description 保存当前 offset、flags 等打开实例状态，再指向 inode 或 vnode，inode 表示文件身份和持久数据。这样 dup 后两个 fd 共享 offset，而两次独立 open 通常有不同 offset。

参考实现的 `FileDescriptorTable::dup` 返回另一个 fd，但两个 fd 指向同一个 `OpenFile`。测试先写入 `hello`，dup 后检查两个 fd 的 offset 相同；再把一个 fd seek 到开头，从另一个 fd 读，也能观察到共享 offset 的效果。这比“fd 是文件编号”的说法更接近真实语义。

page cache 和 fsync 又把文件语义往下推进一层。写入先改 `Folio` 并标 dirty，此时稳定存储仍为空；只有 `fsync` 才把 dirty folio 拷到 storage。真实系统里还要处理元数据日志、磁盘缓存 flush、写屏障和错误上报，但最小模型已经把“write 返回”和“数据持久”区分开。

日志恢复：崩溃一致性不是靠祈祷

文件系统更新常常不是单块写。例如创建文件可能要改目录项、inode 位图、inode 本身和数据块映射。如果系统在中间掉电，磁盘可能处于“目录项指向一个还没初始化的 inode”这类不一致状态。日志的基本思想是先把意图写到 journal，提交后再把更新应用到 home location；崩溃恢复时，已提交日志可以重放，未提交日志丢弃。

参考实现的 `JournaledBlockDevice` 有一个显式 crash point: `AfterJournalAppend`。事务提交时先把记录追加到 journal，若此时崩溃，home block 还没变化；恢复时 `recover` 重放 journal，把 block 7 更新为 `metadata-v2`，再清空 journal。这展示了 write-ahead logging 的核心承诺：只要日志中有已提交意图，恢复后 home location 必须达到同样结果。

```
commit:
    append records to journal
    if crash: stop here
    replay journal to home blocks
    clear journal

recover:
    replay committed journal records
    clear journal
```

真实文件系统还会区分 descriptor block、commit block、checkpoint、revoke record、barrier 和 checksum。教学模型没有展开这些格式，是为了突出最根本的顺序：先有可恢复的意图，再改最终位置。若顺序反过来，崩溃后就无法判断哪些更新完整、哪些更新只写了一半。

TCP 状态机：网络不是 send/recv 两个函数

socket API 把网络隐藏成 send 和 recv，但内核协议栈内部是状态机、计时器、重传队列和拥塞控制共同工作。参考实现的 TcpConnection 只保留最小路径：主动打开进入 SYN-SENT，收到 SYN-ACK 后进入 ESTABLISHED；发送数据后记录 unacked bytes 并启动 RT0；RT0 到期触发重传；ACK 到达后取消定时器；close 进入 FIN-WAIT-1，收到 FIN ACK 后进入 TIME-WAIT。

CLOSED -> SYN-SENT -> ESTABLISHED -> FIN-WAIT-1 -> TIME-WAIT

这个模型刻意保留 RT0，而不是只写状态跳转。因为网络系统的本质是不可靠事件流：包可能丢、ACK 可能丢、定时器可能先于 ACK 触发。没有重传计时器，TCP 就无法从丢包中恢复；没有 unacked 账本，内核就不知道重传什么；没有 TIME-WAIT，旧连接的迟到报文可能污染新连接。

复杂度与工程取舍汇总

下面这张表把参考实现里的主要数据结构和复杂度放在一起。读操作系统源码时，先找这张表对应的结构，再看锁和生命周期，会比直接陷入函数细节更稳。

机制	数据结构	主要复杂度	取舍
CFS runqueue	有序树	插入/删除 $O(\log n)$ ，最小 $O(1)$ 或 $O(\log n)$	公平排序强，常数成本高于 FIFO
futex wait	哈希桶 + wait queue	wait/wake 近似 $O(1)$ 加队列成本	快路径在用户态，慢路径复杂
deadlock graph	map + set	检测 $O(V + E)$	适合诊断，不适合每次锁操作都跑
buddy allocator	分阶 free list	拆分/合并 $O(\max_order)$	连续页友好，内部碎片来自 2^k 取整
TLB cache	哈希表	lookup 近似 $O(1)$	快但必须维护失效协议
LRU cache	list + hash	命中移动 $O(1)$	简洁，但真实冷热判断更复杂
extent map	有序区间表	lookup $O(e)$ 教学版；真实实现常用树	连续文件紧凑，碎片多时记录增加
deadline scheduler	pending vector 教学版	dispatch $O(n)$	易读，真实内核会用更合适的队列结构
journal	顺序记录	commit/replay $O(records)$	恢复简单，写放大增加
TCP state	有限状态机 + timer	tick $O(1)$ 教学版	状态清晰，真实协议栈还要拥塞和乱序队列

从教学代码到真实内核源码

这份实现有意保持单文件、单线程、确定性测试，因此没有加入真实内核必须面对的 SMP 锁、RCU、内存屏障、中断上下文、NUMA、设备错误恢复和安全检查。读者不应把它当成 Linux 的缩小版，而应把它当成源码阅读前的骨架图。带着骨架图去看真实源码时，可以按下面路径对照：

机制	Linux 阅读入口	对照问题
----	------------	------

任务与调度	kernel/sched/fair.c、 include/linux/sched.h	vruntime 何时更新，实体 何时入树出树
等待队列/futex	kernel/futex/、include/ linux/wait.h	条件检查和入队如何避免 lost wakeup
页表/COW	mm/memory.c、kernel/fork. c	fork 如 何 write-protect, fault 如何复制页
伙伴分配	mm/page_alloc.c	order、zone、迁移类型如 何影响分配
page cache	mm/filemap.c、mm/vmscan.c	folio 如何进入缓存，回收 如何处理 dirty
块层	block/	请求如何合并、排序、超时、 完成
日志文件系统	fs/ext4/、fs/jbd2/	commit block 和 checkpoint 如 何 保 证 恢复
网络 TCP	net/ipv4/tcp*.c	状态机、RT0、拥塞窗口和 重传队列如何协作
NVMe/IOMMU	drivers/nvme/host/、 drivers/iommu/	队 列 phase、DMA map/unmap、IOTLB 失 效在哪里发生

编译与运行

源码放在本原理卷目录：

```
source/latex/listings/os_reference_implementation.cpp
```

可用下面命令单独编译运行：

```
g++ -std=c++20 -Wall -Wextra -Wpedantic \
    source/latex/listings/os_reference_implementation.cpp \
    -o /tmp/os_reference_implementation
/tmp/os_reference_implementation
```

期望输出：

```
os_reference_implementation: all tests passed
```

读完本节后，最低验收不是背出这些类名，而是能独立解释：为什么 futex 要比较用户态值后才睡眠；为什么 COW fork 要同时收紧父子 PTE 并失效 TLB；为什么 dirty page 被回收前必须写回；为什么 journal 要先记录意图再改 home block；为什么 TCP 的正确性离不开定时器和未确认队列。

完整源码

```
#include <algorithm>
#include <array>
#include <cstdlib>
#include <deque>
#include <iostream>
#include <list>
#include <map>
#include <memory>
#include <optional>
#include <set>
#include <span>
```



```

#include <stdexcept>
#include <string>
#include <string_view>
#include <unordered_map>
#include <utility>
#include <vector>

namespace osbook {

constexpr std::size_t OS_PAGE_SIZE = 4096;
constexpr std::size_t OS_NVME_QUEUE_SIZE = 8;
constexpr std::size_t OS_MAX_FILE_DESCRIPTOR = 64;
constexpr std::uint64_t OS_NICE_0_WEIGHT = 1024;
constexpr std::uint64_t OS_DEFAULT_TASK_WEIGHT = 1024;
constexpr std::uint64_t OS_HIGH_TASK_WEIGHT = 2048;
constexpr std::uint64_t OS_LOW_TASK_WEIGHT = 512;
constexpr std::size_t OS_BUDDY_MAX_ORDER = 5;
constexpr std::uint64_t OS_READ_DEADLINE_TICKS = 4;
constexpr std::uint64_t OS_WRITE_DEADLINE_TICKS = 12;
constexpr std::uint64_t OS_TCP_INITIAL_RTO_TICKS = 3;
constexpr int OS_INVALID_FD = -1;

[[noreturn]] void fail(std::string_view message) {
    throw std::runtime_error(std::string(message));
}

void require(bool condition, std::string_view message) {
    if (!condition) {
        fail(message);
    }
}

enum class TaskState {
    Runnable,
    Running,
    Sleeping,
    Zombie,
};

enum class AccessMode {
    Read,
    Write,
    Execute,
};

enum class DmaDirection {
    ToDevice,
    FromDevice,
    Bidirectional,
};

enum class CrashPoint {
    None,
    AfterJournalAppend,
};

enum class TcpState {
    Closed,
    SynSent,
    Established,
    FinWait1,
    TimeWait,
};

struct Task {
    int id = 0;
    std::string name;
    TaskState state = TaskState::Runnable;
    std::uint64_t weight = OS_DEFAULT_TASK_WEIGHT;
    std::uint64_t vruntime = 0;
    std::uint64_t executed = 0;
    std::string wait_reason;

    Task(int task_id, std::string task_name, std::uint64_t task_weight)
        : id(task_id), name(std::move(task_name)), weight(task_weight) {}
};

using TaskPtr = std::shared_ptr<Task>;

struct TaskByVirtualRuntime {
    bool operator()(const TaskPtr& left, const TaskPtr& right) const {
        if (left->vruntime != right->vruntime) {

```

```

        return left->vruntime < right->vruntime;
    }
    return left->id < right->id;
}
};

class CfsRunQueue {
public:
    bool empty() const {
        return this->tree_.empty();
    }

    std::size_t size() const {
        return this->tree_.size();
    }

    void enqueue(const TaskPtr& task) {
        require(task != nullptr, "cannot enqueue null task");
        require(task->state == TaskState::Runnable, "only runnable tasks can enter runqueue");
        this->tree_.insert(task);
    }

    void dequeue(const TaskPtr& task) {
        require(task != nullptr, "cannot dequeue null task");
        const auto erased = this->tree_.erase(task);
        require(erased == 1, "task must be present in runqueue exactly once");
    }

    TaskPtr pick_next() {
        if (this->tree_.empty()) {
            return nullptr;
        }
        const auto iterator = this->tree_.begin();
        TaskPtr task = *iterator;
        this->tree_.erase(iterator);
        task->state = TaskState::Running;
        return task;
    }

    void account_and_requeue(const TaskPtr& task, std::uint64_t delta_exec) {
        require(task != nullptr, "cannot account null task");
        require(task->state == TaskState::Running, "only running task can be accounted");
        const std::uint64_t weighted_delta = delta_exec * OS_NICE_0_WEIGHT / task->weight;
        task->executed += delta_exec;
        task->vruntime += weighted_delta;
        task->state = TaskState::Runnable;
        this->enqueue(task);
    }

    bool contains_task(int task_id) const {
        for (const TaskPtr& task : this->tree_) {
            if (task->id == task_id) {
                return true;
            }
        }
        return false;
    }

private:
    std::set<TaskPtr, TaskByVirtualRuntime> tree_;
};

class WaitQueue {
public:
    void sleep(const TaskPtr& task, std::string_view reason) {
        require(task != nullptr, "cannot sleep null task");
        require(task->state != TaskState::Zombie, "zombie task cannot sleep");
        task->state = TaskState::Sleeping;
        task->wait_reason = std::string(reason);
        this->waiters_.push_back(task);
    }

    std::optional<TaskPtr> wake_one() {
        if (this->waiters_.empty()) {
            return std::nullopt;
        }
        TaskPtr task = this->waiters_.front();
        this->waiters_.pop_front();
        task->state = TaskState::Runnable;
        task->wait_reason.clear();
        return task;
    }
};

```

```

std::vector<TaskPtr> wake_all() {
    std::vector<TaskPtr> ready;
    while (!this->waiters_.empty()) {
        TaskPtr task = this->waiters_.front();
        this->waiters_.pop_front();
        task->state = TaskState::Runnable;
        task->wait_reason.clear();
        ready.push_back(task);
    }
    return ready;
}

std::size_t size() const {
    return this->waiters_.size();
}

private:
    std::deque<TaskPtr> waiters_;
};

class Scheduler {
public:
    TaskPtr create_task(std::string name, std::uint64_t weight) {
        auto task = std::make_shared<Task>(this->next_task_id++, std::move(name), weight);
        this->runqueue_.enqueue(task);
        return task;
    }

    TaskPtr schedule_next() {
        return this->runqueue_.pick_next();
    }

    void tick_and_requeue(const TaskPtr& task, std::uint64_t delta_exec) {
        this->runqueue_.account_and_requeue(task, delta_exec);
    }

    void block_current(const TaskPtr& task, WaitQueue& queue, std::string_view reason) {
        require(task != nullptr, "cannot block null task");
        require(task->state == TaskState::Running, "only running task blocks through this path");
        queue.sleep(task, reason);
    }

    void wake_to_runqueue(const TaskPtr& task) {
        require(task != nullptr, "cannot wake null task");
        require(task->state == TaskState::Runnable, "woken task must be runnable");
        this->runqueue_.enqueue(task);
    }

    std::size_t runnable_count() const {
        return this->runqueue_.size();
    }

private:
    int next_task_id_ = 1;
    CfsRunQueue runqueue_;
};

struct PhysicalPage {
    int id = 0;
    std::array<std::uint8_t, OS_PAGE_SIZE> bytes{};

    explicit PhysicalPage(int page_id) : id(page_id) {}
};

using PagePtr = std::shared_ptr<PhysicalPage>;

class PageAllocator {
public:
    PagePtr allocate_zeroed() {
        auto page = std::make_shared<PhysicalPage>(this->next_page_id++);
        page->bytes.fill(0);
        return page;
    }

private:
    int next_page_id_ = 1;
};

struct PageTableEntry {
    PagePtr page;
    bool present = false;
};

```

```

    bool writable = false;
    bool executable = false;
    bool cow = false;
};

struct TlbKey {
    int asid = 0;
    std::uint64_t vpn = 0;

    bool operator==(const TlbKey& other) const {
        return this->asid == other.asid && this->vpn == other.vpn;
    }
};

struct TlbKeyHash {
    std::size_t operator()(const TlbKey& key) const {
        const std::size_t asid_hash = std::hash<int>{}(key.asid);
        const std::size_t vpn_hash = std::hash<std::uint64_t>{}(key.vpn);
        return asid_hash ^ (vpn_hash << 1U);
    }
};

class Tlb {
public:
    void insert(int asid, std::uint64_t vpn, const PageTableEntry& entry) {
        this->entries_[TlbKey{asid, vpn}] = entry;
    }

    std::optional<PageTableEntry> lookup(int asid, std::uint64_t vpn) const {
        const auto iterator = this->entries_.find(TlbKey{asid, vpn});
        if (iterator == this->entries_.end()) {
            return std::nullopt;
        }
        return iterator->second;
    }

    void invalidate_page(int asid, std::uint64_t vpn) {
        this->entries_.erase(TlbKey{asid, vpn});
    }

    void invalidate_asid(int asid) {
        for (auto iterator = this->entries_.begin(); iterator != this->entries_.end(); ) {
            if (iterator->first.asid == asid) {
                iterator = this->entries_.erase(iterator);
            } else {
                ++iterator;
            }
        }
    }

private:
    std::unordered_map<TlbKey, PageTableEntry, TlbKeyHash> entries_;
};

class AddressSpace {
public:
    AddressSpace(int asid, PageAllocator& allocator) : asid_(asid), allocator_(allocator) {}

    int asid() const {
        return this->asid_;
    }

    void map_anonymous(std::uint64_t vpn, bool writable, bool executable) {
        PageTableEntry entry;
        entry.page = this->allocator_.allocate_zeroed();
        entry.present = true;
        entry.writable = writable;
        entry.executable = executable;
        this->entries_[vpn] = entry;
    }

    std::uint8_t read_byte(std::uint64_t virtual_address, Tlb& tlb) {
        const auto [vpn, offset] = this->split_address(virtual_address);
        PageTableEntry entry = this->resolve(vpn, AccessMode::Read, tlb);
        return entry.page->bytes[offset];
    }

    void write_byte(std::uint64_t virtual_address, std::uint8_t value, Tlb& tlb) {
        const auto [vpn, offset] = this->split_address(virtual_address);
        PageTableEntry& entry = this->entry_for_write(vpn, tlb);
        entry.page->bytes[offset] = value;
        tlb.insert(this->asid_, vpn, entry);
    }
};

```

```

}

AddressSpace fork_cow(int child_asid, Tlb& parent_tlb) {
    AddressSpace child(child_asid, this->allocator_);
    for (auto& [vpn, entry] : this->entries_) {
        PageTableEntry child_entry = entry;
        if (entry.present && entry.writable) {
            entry.writable = false;
            entry.cow = true;
            child_entry.writable = false;
            child_entry.cow = true;
            parent_tlb.invalidate_page(this->asid_, vpn);
        }
        child.entries_[vpn] = child_entry;
    }
    return child;
}

std::size_t mapped_pages() const {
    return this->entries_.size();
}

private:
std::pair<std::uint64_t, std::size_t> split_address(std::uint64_t virtual_address) const {
    return {virtual_address / OS_PAGE_SIZE, static_cast<std::size_t>(virtual_address % OS_PAGE_SIZE)};
}

PageTableEntry resolve(std::uint64_t vpn, AccessMode mode, Tlb& tlb) {
    if (const std::optional<PageTableEntry> cached = tlb.lookup(this->asid_, vpn); cached.has_value()) {
        this->check_access(cached.value(), mode);
        return cached.value();
    }
    const auto iterator = this->entries_.find(vpn);
    require(iterator != this->entries_.end() && iterator->second.present, "page fault: no present PTE");
    this->check_access(iterator->second, mode);
    tlb.insert(this->asid_, vpn, iterator->second);
    return iterator->second;
}

PageTableEntry& entry_for_write(std::uint64_t vpn, Tlb& tlb) {
    const auto iterator = this->entries_.find(vpn);
    require(iterator != this->entries_.end() && iterator->second.present, "write fault: no present PTE");
    PageTableEntry& entry = iterator->second;
    if (entry.writable) {
        return entry;
    }
    require(entry.cow, "write fault: permission denied");
    if (!entry.page.unique()) {
        PagePtr copied = this->allocator_.allocate_zeroed();
        copied->bytes = entry.page->bytes;
        entry.page = copied;
    }
    entry.writable = true;
    entry.cow = false;
    tlb.invalidate_page(this->asid_, vpn);
    return entry;
}

void check_access(const PageTableEntry& entry, AccessMode mode) const {
    require(entry.present, "page fault: not present");
    if (mode == AccessMode::Write) {
        require(entry.writable, "page fault: write denied");
    }
    if (mode == AccessMode::Execute) {
        require(entry.executable, "page fault: execute denied");
    }
}

int asid_ = 0;
PageAllocator& allocator_;
std::map<std::uint64_t, PageTableEntry> entries_;
};

struct DmaMapping {
    PagePtr page;
    std::size_t length = 0;
    DmaDirection direction = DmaDirection::Bidirectional;
};

class IommuDomain {
public:
    std::uint64_t map(PagePtr page, std::size_t length, DmaDirection direction) {

```

```

    require(page != nullptr, "cannot map null DMA page");
    require(length > 0 && length <= OS_PAGE_SIZE, "invalid DMA mapping length");
    const std::uint64_t iova = this->next_iova_;
    this->next_iova_ += OS_PAGE_SIZE;
    this->mappings_[iova] = DmaMapping{std::move(page), length, direction};
    this->iotlb_[iova] = this->mappings_.at(iova);
    return iova;
}

void unmap(std::uint64_t iova) {
    const auto erased = this->mappings_.erase(iova);
    require(erased == 1, "DMA unmap must remove an existing mapping");
    this->iotlb_.erase(iova);
}

std::uint8_t device_read(std::uint64_t iova, std::size_t offset) const {
    const DmaMapping& mapping = this->mapping_for(iova, DmaDirection::ToDevice);
    require(offset < mapping.length, "DMA read out of range");
    return mapping.page->bytes[offset];
}

void device_write(std::uint64_t iova, std::size_t offset, std::uint8_t value) {
    DmaMapping& mapping = this->mutable_mapping_for(iova, DmaDirection::FromDevice);
    require(offset < mapping.length, "DMA write out of range");
    mapping.page->bytes[offset] = value;
    this->iotlb_[iova] = mapping;
}

std::size_t mapping_count() const {
    return this->mappings_.size();
}

private:
bool direction_allows(DmaDirection actual, DmaDirection required) const {
    return actual == DmaDirection::Bidirectional || actual == required;
}

const DmaMapping& mapping_for(std::uint64_t iova, DmaDirection required) const {
    const auto iterator = this->iotlb_.find(iova);
    require(iterator != this->iotlb_.end(), "IOMMU fault: no IOTLB entry");
    require(this->direction_allows(iterator->second.direction, required), "IOMMU fault: wrong DMA direction");
    return iterator->second;
}

DmaMapping& mutable_mapping_for(std::uint64_t iova, DmaDirection required) {
    const auto iterator = this->mappings_.find(iova);
    require(iterator != this->mappings_.end(), "IOMMU fault: no mapping");
    require(this->direction_allows(iterator->second.direction, required), "IOMMU fault: wrong DMA direction");
    return iterator->second;
}

std::uint64_t next_iova_ = OS_PAGE_SIZE;
std::unordered_map<std::uint64_t, DmaMapping> mappings_;
std::unordered_map<std::uint64_t, DmaMapping> iotlb_;
};

struct NvmeCompletion {
    std::uint16_t command_id = 0;
    std::uint16_t status = 0;
    bool phase = true;
};

class NvmeCompletionQueue {
public:
    explicit NvmeCompletionQueue(std::size_t queue_size) : entries_(queue_size) {
        require(queue_size >= 2, "NVMe completion queue must have at least two slots");
        for (NvmeCompletion& completion : this->entries_) {
            completion.phase = !this->expected_phase_;
        }
    }

    void device_push(std::uint16_t command_id, std::uint16_t status) {
        NvmeCompletion& slot = this->entries_.at(this->device_tail_);
        slot.command_id = command_id;
        slot.status = status;
        slot.phase = this->device_phase_;
        this->device_tail_ = (this->device_tail_ + 1) % this->entries_.size();
        if (this->device_tail_ == 0) {
            this->device_phase_ = !this->device_phase_;
        }
    }
};

```



```

std::optional<NvmeCompletion> host_poll() {
    const NvmeCompletion& slot = this->entries_.at(this->host_head_);
    if (slot.phase != this->expected_phase_) {
        return std::nullopt;
    }
    NvmeCompletion completion = slot;
    this->host_head_ = (this->host_head_ + 1) % this->entries_.size();
    if (this->host_head_ == 0) {
        this->expected_phase_ = !this->expected_phase_;
    }
    return completion;
}

std::size_t host_head() const {
    return this->host_head_;
}

private:
    std::vector<NvmeCompletion> entries_;
    std::size_t device_tail_ = 0;
    std::size_t host_head_ = 0;
    bool device_phase_ = true;
    bool expected_phase_ = true;
};

struct Folio {
    std::vector<std::uint8_t> bytes;
    bool uptodate = false;
    bool dirty = false;
    bool writeback = false;

    Folio() : bytes(OS_PAGE_SIZE, 0) {}
};

using FolioPtr = std::shared_ptr<Folio>;

class Inode {
public:
    explicit Inode(std::string name) : name_(std::move(name)) {}

    const std::string& name() const {
        return this->name_;
    }

    std::size_t size() const {
        return this->size_;
    }

    std::vector<std::uint8_t> read_at(std::size_t offset, std::size_t length) {
        std::vector<std::uint8_t> result;
        result.reserve(length);
        std::size_t remaining = length;
        std::size_t cursor = offset;
        while (remaining > 0 && cursor < this->size_) {
            const std::size_t page_index = cursor / OS_PAGE_SIZE;
            const std::size_t page_offset = cursor % OS_PAGE_SIZE;
            const std::size_t chunk = std::min({remaining, OS_PAGE_SIZE - page_offset, this->size_ - cursor});
            FolioPtr folio = this->get_or_load_folio(page_index);
            result.insert(result.end(), folio->bytes.begin() + static_cast<std::ptrdiff_t>(page_offset),
                folio->bytes.begin() + static_cast<std::ptrdiff_t>(page_offset + chunk));
            remaining -= chunk;
            cursor += chunk;
        }
        return result;
    }

    std::size_t write_at(std::size_t offset, std::span<const std::uint8_t> data) {
        std::size_t written = 0;
        while (written < data.size()) {
            const std::size_t cursor = offset + written;
            const std::size_t page_index = cursor / OS_PAGE_SIZE;
            const std::size_t page_offset = cursor % OS_PAGE_SIZE;
            const std::size_t chunk = std::min(data.size() - written, OS_PAGE_SIZE - page_offset);
            FolioPtr folio = this->get_or_load_folio(page_index);
            std::copy_n(data.begin() + static_cast<std::ptrdiff_t>(written), static_cast<std::ptrdiff_t>(chunk),
                folio->bytes.begin() + static_cast<std::ptrdiff_t>(page_offset));
            folio->uptodate = true;
            folio->dirty = true;
            written += chunk;
        }
        this->size_ = std::max(this->size_, offset + data.size());
        return written;
    }
};

```

```

}

void fsync() {
    this->storage_.resize(this->size_, 0);
    for (auto& [page_index, folio] : this->cache_) {
        if (!folio->dirty) {
            continue;
        }
        folio->writeback = true;
        const std::size_t file_offset = page_index * OS_PAGE_SIZE;
        if (file_offset < this->size_) {
            const std::size_t chunk = std::min(OS_PAGE_SIZE, this->size_ - file_offset);
            std::copy_n(folio->bytes.begin(), static_cast<std::ptrdiff_t>(chunk),
                this->storage_.begin() + static_cast<std::ptrdiff_t>(file_offset));
        }
        folio->dirty = false;
        folio->writeback = false;
    }
}

std::vector<std::uint8_t> stable_storage() const {
    return this->storage_;
}

private:
FolioPtr get_or_load_folio(std::size_t page_index) {
    const auto cached = this->cache_.find(page_index);
    if (cached != this->cache_.end()) {
        return cached->second;
    }
    auto folio = std::make_shared<Folio>();
    const std::size_t file_offset = page_index * OS_PAGE_SIZE;
    if (file_offset < this->storage_.size()) {
        const std::size_t chunk = std::min(OS_PAGE_SIZE, this->storage_.size() - file_offset);
        std::copy_n(this->storage_.begin() + static_cast<std::ptrdiff_t>(file_offset),
            static_cast<std::ptrdiff_t>(chunk), folio->bytes.begin());
    }
    folio->uptodate = true;
    this->cache_[page_index] = folio;
    return folio;
}

std::string name_;
std::size_t size_ = 0;
std::vector<std::uint8_t> storage_;
std::map<std::size_t, FolioPtr> cache_;
};

class OpenFile {
public:
    explicit OpenFile(std::shared_ptr<Inode> inode) : inode_(std::move(inode)) {
        require(this->inode_ != nullptr, "open file requires inode");
    }

    std::vector<std::uint8_t> read(std::size_t length) {
        std::vector<std::uint8_t> data = this->inode_->read_at(this->offset_, length);
        this->offset_ += data.size();
        return data;
    }

    std::size_t write(std::span<const std::uint8_t> data) {
        const std::size_t written = this->inode_->write_at(this->offset_, data);
        this->offset_ += written;
        return written;
    }

    void seek(std::size_t offset) {
        this->offset_ = offset;
    }

    std::size_t offset() const {
        return this->offset_;
    }

    std::shared_ptr<Inode> inode() const {
        return this->inode_;
    }

private:
    std::shared_ptr<Inode> inode_;
    std::size_t offset_ = 0;
};

```

```

class FileDescriptorTable {
public:
    int install(std::shared_ptr<OpenFile> file) {
        require(file != nullptr, "cannot install null file");
        for (int fd = 0; fd < static_cast<int>(OS_MAX_FILE_DESCRIPTOR); ++fd) {
            if (!this->files_.contains(fd)) {
                this->files_[fd] = std::move(file);
                return fd;
            }
        }
        return OS_INVALID_FD;
    }

    int dup(int old_fd) {
        std::shared_ptr<OpenFile> file = this->get(old_fd);
        return this->install(std::move(file));
    }

    std::shared_ptr<OpenFile> get(int fd) const {
        const auto iterator = this->files_.find(fd);
        require(iterator != this->files_.end(), "bad file descriptor");
        return iterator->second;
    }

    void close(int fd) {
        const auto erased = this->files_.erase(fd);
        require(erased == 1, "close requires valid fd");
    }

    std::size_t size() const {
        return this->files_.size();
    }

private:
    std::map<int, std::shared_ptr<OpenFile>> files_;
};

class FutexWord {
public:
    explicit FutexWord(int initial_value) : value_(initial_value) {}

    int load() const {
        return this->value_;
    }

    void store(int value) {
        this->value_ = value;
    }

    bool wait_if_equal(const TaskPtr& task, int expected_value) {
        require(task != nullptr, "futex cannot wait a null task");
        if (this->value_ != expected_value) {
            return false;
        }
        this->waiters_.sleep(task, "futex");
        return true;
    }

    std::optional<TaskPtr> wake_one() {
        return this->waiters_.wake_one();
    }

    std::size_t waiter_count() const {
        return this->waiters_.size();
    }

private:
    int value_ = 0;
    WaitQueue waiters_;
};

class DeadlockDetector {
public:
    void add_wait_edge(int waiter_task_id, int owner_task_id) {
        require(waiter_task_id != owner_task_id, "self wait is an immediate deadlock");
        this->wait_for_[waiter_task_id].insert(owner_task_id);
        this->wait_for_.try_emplace(owner_task_id);
    }

    void clear_task(int task_id) {
        this->wait_for_.erase(task_id);
    }

```

```

    for (auto& [node, successors] : this->wait_for_) {
        static_cast<void>(node);
        successors.erase(task_id);
    }
}

bool has_cycle() const {
    std::map<int, std::size_t> indegree;
    for (const auto& [node, successors] : this->wait_for_) {
        indegree.try_emplace(node, 0);
        for (const int successor : successors) {
            ++indegree[successor];
        }
    }

    std::deque<int> ready;
    for (const auto& [node, degree] : indegree) {
        if (degree == 0) {
            ready.push_back(node);
        }
    }

    std::size_t visited = 0;
    while (!ready.empty()) {
        const int node = ready.front();
        ready.pop_front();
        ++visited;
        const auto iterator = this->wait_for_.find(node);
        if (iterator == this->wait_for_.end()) {
            continue;
        }
        for (const int successor : iterator->second) {
            auto degree = indegree.find(successor);
            require(degree != indegree.end(), "wait-for graph indegree must contain every node");
            --degree->second;
            if (degree->second == 0) {
                ready.push_back(successor);
            }
        }
    }

    return visited != indegree.size();
}

std::size_t edge_count() const {
    std::size_t count = 0;
    for (const auto& [node, successors] : this->wait_for_) {
        static_cast<void>(node);
        count += successors.size();
    }
    return count;
}

private:
    std::map<int, std::set<int>> wait_for_;
};

class BuddyAllocator {
public:
    explicit BuddyAllocator(std::size_t max_order) : max_order_(max_order), free_lists_(max_order + 1) {
        require(max_order <= OS_BUDDY_MAX_ORDER, "test buddy allocator max order too large");
        this->free_lists_.at(max_order).insert(0);
    }

    std::size_t allocate_pages(std::size_t page_count) {
        const std::size_t target_order = this->order_for_pages(page_count);
        std::size_t order = target_order;
        while (order <= this->max_order_ && this->free_lists_.at(order).empty()) {
            ++order;
        }
        require(order <= this->max_order_, "buddy allocator out of memory");

        auto block = this->free_lists_.at(order).begin();
        std::size_t start_page = *block;
        this->free_lists_.at(order).erase(block);

        while (order > target_order) {
            --order;
            const std::size_t buddy = start_page + (std::size_t{1} << order);
            this->free_lists_.at(order).insert(buddy);
        }
    }
};

```

```

    this->allocated_orders_[start_page] = target_order;
    return start_page;
}

void free_pages(std::size_t start_page) {
    auto allocation = this->allocated_orders_.find(start_page);
    require(allocation != this->allocated_orders_.end(), "free must reference an allocated buddy block");
    std::size_t order = allocation->second;
    this->allocated_orders_.erase(allocation);

    while (order < this->max_order_) {
        const std::size_t buddy = start_page ^ (std::size_t{1} << order);
        auto& free_list = this->free_lists_.at(order);
        const auto buddy_iterator = free_list.find(buddy);
        if (buddy_iterator == free_list.end()) {
            break;
        }
        free_list.erase(buddy_iterator);
        start_page = std::min(start_page, buddy);
        ++order;
    }
    this->free_lists_.at(order).insert(start_page);
}

std::size_t free_blocks_at_order(std::size_t order) const {
    require(order <= this->max_order_, "free block order out of range");
    return this->free_lists_.at(order).size();
}

std::size_t allocated_block_count() const {
    return this->allocated_orders_.size();
}

private:
    std::size_t order_for_pages(std::size_t page_count) const {
        require(page_count > 0, "allocation must request at least one page");
        std::size_t order = 0;
        std::size_t capacity = 1;
        while (capacity < page_count) {
            capacity <<= 1U;
            ++order;
        }
        require(order <= this->max_order_, "allocation request exceeds allocator capacity");
        return order;
    }

    std::size_t max_order_ = 0;
    std::vector<std::set<std::size_t>> free_lists_;
    std::map<std::size_t, std::size_t> allocated_orders_;
};

struct CacheEntry {
    int page_id = 0;
    bool dirty = false;
};

class LruPageCache {
public:
    explicit LruPageCache(std::size_t capacity) : capacity_(capacity) {
        require(capacity > 0, "LRU cache capacity must be positive");
    }

    void access(int page_id, bool write) {
        const auto cached = this->index_.find(page_id);
        if (cached != this->index_.end()) {
            cached->second->dirty = cached->second->dirty || write;
            this->entries_.splice(this->entries_.begin(), this->entries_, cached->second);
            cached->second = this->entries_.begin();
            return;
        }

        if (this->entries_.size() == this->capacity_) {
            this->evict_one();
        }
        this->entries_.push_front(CacheEntry{page_id, write});
        this->index_[page_id] = this->entries_.begin();
    }

    bool contains(int page_id) const {
        return this->index_.contains(page_id);
    }
}

```

```

void flush_all() {
    for (CacheEntry& entry : this->entries_) {
        if (!entry.dirty) {
            continue;
        }
        ++this->writeback_count_;
        entry.dirty = false;
    }
}

std::size_t size() const {
    return this->entries_.size();
}

std::size_t writeback_count() const {
    return this->writeback_count_;
}

private:
void evict_one() {
    require(!this->entries_.empty(), "cannot evict from empty page cache");
    auto victim = this->entries_.end();
    --victim;
    if (victim->dirty) {
        ++this->writeback_count_;
    }
    this->index_.erase(victim->page_id);
    this->entries_.erase(victim);
}

std::size_t capacity_ = 0;
std::list<CacheEntry> entries_;
std::unordered_map<int, std::list<CacheEntry>::iterator> index_;
std::size_t writeback_count_ = 0;
};

struct Extent {
    std::uint64_t logical_start = 0;
    std::uint64_t physical_start = 0;
    std::uint64_t length = 0;
};

class ExtentMap {
public:
    void add_extent(std::uint64_t logical_start, std::uint64_t physical_start, std::uint64_t length) {
        require(length > 0, "extent length must be positive");
        const std::uint64_t logical_end = logical_start + length;
        for (const auto& [existing_start, extent] : this->extents_) {
            static_cast<void>(existing_start);
            const std::uint64_t existing_end = extent.logical_start + extent.length;
            const bool disjoint = logical_end <= existing_end || existing_end <= logical_start;
            require(disjoint, "extent must not overlap an existing extent");
        }
        this->extents_[logical_start] = Extent{logical_start, physical_start, length};
    }

    std::optional<std::uint64_t> lookup(std::uint64_t logical_block) const {
        for (const auto& [start, extent] : this->extents_) {
            static_cast<void>(start);
            if (logical_block < extent.logical_start) {
                break;
            }
            if (logical_block < extent.logical_start + extent.length) {
                return extent.physical_start + (logical_block - extent.logical_start);
            }
        }
        return std::nullopt;
    }

    std::size_t extent_count() const {
        return this->extents_.size();
    }

private:
    std::map<std::uint64_t, Extent> extents_;
};

struct BlockRequest {
    int id = 0;
    std::uint64_t sector = 0;
    bool write = false;
    std::uint64_t deadline = 0;
};

```



```

};

class DeadlineBlockScheduler {
public:
    int submit(std::uint64_t sector, bool write, std::uint64_t now) {
        const int request_id = this->next_request_id++;
        const std::uint64_t delay = write ? OS_WRITE_DEADLINE_TICKS : OS_READ_DEADLINE_TICKS;
        this->pending_.push_back(BlockRequest{request_id, sector, write, now + delay});
        return request_id;
    }

    std::optional<BlockRequest> dispatch(std::uint64_t now) {
        if (this->pending_.empty()) {
            return std::nullopt;
        }
        const auto request = this->choose_request(now);
        BlockRequest selected = *request;
        this->pending_.erase(request);
        this->head_sector_ = selected.sector;
        return selected;
    }

    std::size_t pending_count() const {
        return this->pending_.size();
    }

private:
    std::vector<BlockRequest>::iterator choose_request(std::uint64_t now) {
        auto best = this->pending_.begin();
        for (auto iterator = this->pending_.begin(); iterator != this->pending_.end(); ++iterator) {
            if (this->is_better(*iterator, *best, now)) {
                best = iterator;
            }
        }
        return best;
    }

    bool is_better(const BlockRequest& candidate, const BlockRequest& current, std::uint64_t now) const {
        const bool candidate_expired = candidate.deadline <= now;
        const bool current_expired = current.deadline <= now;
        if (candidate_expired != current_expired) {
            return candidate_expired;
        }
        if (candidate_expired && candidate.deadline != current.deadline) {
            return candidate.deadline < current.deadline;
        }
        const std::uint64_t candidate_seek = this->seek_distance(candidate.sector);
        const std::uint64_t current_seek = this->seek_distance(current.sector);
        if (candidate_seek != current_seek) {
            return candidate_seek < current_seek;
        }
        return candidate.id < current.id;
    }

    std::uint64_t seek_distance(std::uint64_t sector) const {
        if (sector >= this->head_sector_) {
            return sector - this->head_sector_;
        }
        return this->head_sector_ - sector;
    }

    int next_request_id_ = 1;
    std::uint64_t head_sector_ = 0;
    std::vector<BlockRequest> pending_;
};

struct JournalRecord {
    int transaction_id = 0;
    int block_id = 0;
    std::string payload;
};

class JournaledBlockDevice {
public:
    void begin_transaction() {
        require(!this->transaction_active_, "nested transactions are not supported in this model");
        this->transaction_active_ = true;
        this->active_transaction_id_ = this->next_transaction_id++;
        this->pending_.clear();
    }

    void write_block(int block_id, std::string payload) {

```

```

    require(this->transaction_active_, "write_block requires an active transaction");
    this->pending_.push_back(JournalRecord{this->active_transaction_id_, block_id, std::move(payload)});
}

void commit(CrashPoint crash_point) {
    require(this->transaction_active_, "commit requires an active transaction");
    for (const JournalRecord& record : this->pending_) {
        this->journal_.push_back(record);
    }
    this->transaction_active_ = false;
    this->pending_.clear();
    if (crash_point == CrashPoint::AfterJournalAppend) {
        return;
    }
    this->replay_journal();
}

void recover() {
    this->transaction_active_ = false;
    this->pending_.clear();
    this->replay_journal();
}

std::string read_block(int block_id) const {
    const auto iterator = this->home_.find(block_id);
    if (iterator == this->home_.end()) {
        return {};
    }
    return iterator->second;
}

std::size_t journal_record_count() const {
    return this->journal_.size();
}

private:
    void replay_journal() {
        for (const JournalRecord& record : this->journal_) {
            this->home_[record.block_id] = record.payload;
        }
        this->journal_.clear();
    }

    int next_transaction_id_ = 1;
    int active_transaction_id_ = 0;
    bool transaction_active_ = false;
    std::vector<JournalRecord> pending_;
    std::vector<JournalRecord> journal_;
    std::map<int, std::string> home_;
};

class TcpConnection {
public:
    void active_open(std::uint64_t now) {
        require(this->state_ == TcpState::Closed, "active_open requires CLOSED");
        this->state_ = TcpState::SynSent;
        this->unacked_segments_ = 1;
        this->retransmission_deadline_ = now + OS_TCP_INITIAL_RTO_TICKS;
    }

    void receive_syn_ack() {
        require(this->state_ == TcpState::SynSent, "SYN-ACK is only valid in SYN-SENT");
        this->state_ = TcpState::Established;
        this->unacked_segments_ = 0;
        this->retransmission_deadline_.reset();
    }

    void send_data(std::size_t byte_count, std::uint64_t now) {
        require(this->state_ == TcpState::Established, "send_data requires ESTABLISHED");
        require(byte_count > 0, "send_data requires a non-empty payload");
        this->unacked_bytes_ += byte_count;
        if (!this->retransmission_deadline_.has_value()) {
            this->retransmission_deadline_ = now + OS_TCP_INITIAL_RTO_TICKS;
        }
    }

    bool tick(std::uint64_t now) {
        if (!this->retransmission_deadline_.has_value() || now < this->retransmission_deadline_.value()) {
            return false;
        }
        ++this->retransmission_count_;
        this->retransmission_deadline_ = now + OS_TCP_INITIAL_RTO_TICKS;
    }
};

```

```

    return true;
}

void acknowledge_all() {
    this->unacked_bytes_ = 0;
    this->unacked_segments_ = 0;
    this->retransmission_deadline_.reset();
}

void close(std::uint64_t now) {
    require(this->state_ == TcpState::Established, "close requires ESTABLISHED");
    this->state_ = TcpState::FinWait1;
    this->unacked_segments_ = 1;
    this->retransmission_deadline_ = now + OS_TCP_INITIAL_RTO_TICKS;
}

void receive_fin_ack() {
    require(this->state_ == TcpState::FinWait1, "FIN ACK requires FIN-WAIT-1");
    this->state_ = TcpState::TimeWait;
    this->acknowledge_all();
}

TcpState state() const {
    return this->state_;
}

std::size_t retransmission_count() const {
    return this->retransmission_count_;
}

std::size_t unacked_bytes() const {
    return this->unacked_bytes_;
}

private:
    TcpState state_ = TcpState::Closed;
    std::size_t unacked_bytes_ = 0;
    std::size_t unacked_segments_ = 0;
    std::size_t retransmission_count_ = 0;
    std::optional<std::uint64_t> retransmission_deadline_;
};

std::vector<std::uint8_t> bytes(std::string_view text) {
    return {text.begin(), text.end()};
}

std::string text_of(const std::vector<std::uint8_t>& data) {
    return {data.begin(), data.end()};
}

template <typename Function>
void require_throws(Function action, std::string_view message) {
    bool thrown = false;
    try {
        action();
    } catch (const std::runtime_error&) {
        thrown = true;
    }
    require(thrown, message);
}

void test_cfs_scheduler_and_wait_queue() {
    Scheduler scheduler;
    TaskPtr low = scheduler.create_task("low", OS_LOW_TASK_WEIGHT);
    TaskPtr normal = scheduler.create_task("normal", OS_DEFAULT_TASK_WEIGHT);
    TaskPtr high = scheduler.create_task("high", OS_HIGH_TASK_WEIGHT);

    TaskPtr first = scheduler.schedule_next();
    require(first != nullptr, "scheduler must pick a task");
    scheduler.tick_and_requeue(first, 8);

    TaskPtr second = scheduler.schedule_next();
    require(second != nullptr, "scheduler must pick second task");
    scheduler.tick_and_requeue(second, 8);

    TaskPtr third = scheduler.schedule_next();
    require(third != nullptr, "scheduler must pick third task");
    WaitQueue io_wait;
    scheduler.block_current(third, io_wait, "disk");
    require(io_wait.size() == 1, "blocked task must enter wait queue");

    std::optional<TaskPtr> woken = io_wait.wake_one();
}

```

```

    require(woken.has_value(), "wake_one must wake task");
    scheduler.wake_to_runqueue(woken.value());
    require(scheduler.runnable_count() == 3, "all three tasks should be runnable again");

    require(low->executed + normal->executed + high->executed == 16, "two accounted time slices expected");
}

void test_cow_page_table_and_tlb() {
    PageAllocator allocator;
    Tlb tlb;
    AddressSpace parent(1, allocator);
    constexpr std::uint64_t TEST_VA = 0x1000;
    constexpr std::uint8_t PARENT_INITIAL_VALUE = 42;
    constexpr std::uint8_t CHILD_COW_VALUE = 7;
    constexpr std::uint8_t PARENT_COW_VALUE = 9;
    parent.map_anonymous(TEST_VA / OS_PAGE_SIZE, true, false);
    parent.write_byte(TEST_VA, PARENT_INITIAL_VALUE, tlb);
    require(parent.read_byte(TEST_VA, tlb) == PARENT_INITIAL_VALUE, "parent write must be readable");

    AddressSpace child = parent.fork_cow(2, tlb);
    require(parent.mapped_pages() == 1 && child.mapped_pages() == 1, "fork should preserve mappings");
    child.write_byte(TEST_VA, CHILD_COW_VALUE, tlb);
    require(child.read_byte(TEST_VA, tlb) == CHILD_COW_VALUE, "child COW write must change child");
    require(parent.read_byte(TEST_VA, tlb) == PARENT_INITIAL_VALUE, "child COW write must not change parent");

    parent.write_byte(TEST_VA, PARENT_COW_VALUE, tlb);
    require(parent.read_byte(TEST_VA, tlb) == PARENT_COW_VALUE, "parent COW write must succeed");
    require(child.read_byte(TEST_VA, tlb) == CHILD_COW_VALUE, "parent COW write must not change child");
}

void test_iommu_and_nvme_completion_queue() {
    PageAllocator allocator;
    PagePtr page = allocator.allocate_zeroed();
    constexpr std::uint8_t DMA_INITIAL_VALUE = 11;
    constexpr std::uint8_t DMA_DEVICE_VALUE = 22;
    constexpr std::uint16_t NVME_STATUS_SUCCESS = 0;
    page->bytes.at(0) = DMA_INITIAL_VALUE;
    IommuDomain domain;
    const std::uint64_t iova = domain.map(page, OS_PAGE_SIZE, DmaDirection::Bidirectional);
    require(domain.device_read(iova, 0) == DMA_INITIAL_VALUE, "device must read mapped page");
    domain.device_write(iova, 1, DMA_DEVICE_VALUE);
    require(page->bytes.at(1) == DMA_DEVICE_VALUE, "device write must update mapped page");
    domain.unmap(iova);
    require(domain.mapping_count() == 0, "unmap must remove DMA mapping");

    NvmeCompletionQueue queue(OS_NVME_QUEUE_SIZE);
    require(!queue.host_poll().has_value(), "empty completion queue must not report a completion");
    constexpr std::uint16_t FIRST_COMMAND_ID = 10;
    constexpr std::uint16_t SECOND_COMMAND_ID = 11;
    queue.device_push(FIRST_COMMAND_ID, NVME_STATUS_SUCCESS);
    queue.device_push(SECOND_COMMAND_ID, NVME_STATUS_SUCCESS);
    std::optional<NvmeCompletion> first = queue.host_poll();
    std::optional<NvmeCompletion> second = queue.host_poll();
    require(first.has_value() && first->command_id == FIRST_COMMAND_ID, "first completion id mismatch");
    require(second.has_value() && second->command_id == SECOND_COMMAND_ID, "second completion id mismatch");
}

void test_fd_table_page_cache_and_fsync() {
    auto inode = std::make_shared<Inode>("notes.txt");
    auto file = std::make_shared<OpenFile>(inode);
    FileDescriptorTable table;
    const int fd = table.install(file);
    require(fd != OS_INVALID_FD, "fd install must succeed");

    const std::vector<std::uint8_t> hello = bytes("hello");
    require(table.get(fd)->write(hello) == hello.size(), "write must copy all bytes");

    const int dup_fd = table.dup(fd);
    require(dup_fd != OS_INVALID_FD, "dup must return a valid fd");
    require(table.get(dup_fd)->offset() == table.get(fd)->offset(), "dup must share open-file offset");

    table.get(fd)->seek(0);
    const std::vector<std::uint8_t> read_back = table.get(dup_fd)->read(hello.size());
    require(text_of(read_back) == "hello", "read must observe page cache contents");

    require(inode->stable_storage().empty(), "buffered write should not be durable before fsync");
    inode->fsync();
    require(text_of(inode->stable_storage()) == "hello", "fsync must copy dirty cache to stable storage");

    table.close(fd);
    require(table.size() == 1, "close one fd should leave duplicated fd alive");
    table.close(dup_fd);
}

```

```

    require(table.size() == 0, "all fds closed");
}

void test_futex_deadlock_and_buddy_allocator() {
    auto waiter = std::make_shared<Task>(1, "waiter", OS_DEFAULT_TASK_WEIGHT);
    waiter->state = TaskState::Running;
    FutexWord futex(0);
    require(!futex.wait_if_equal(waiter, 1), "futex must not sleep when user value changed first");
    require(waiter->state == TaskState::Running, "failed futex compare must leave task runnable/running");
    require(futex.wait_if_equal(waiter, 0), "matching futex value must place task on wait queue");
    require(waiter->state == TaskState::Sleeping && futex.waiter_count() == 1, "futex wait must sleep atomically");
    futex.store(1);
    std::optional<TaskPtr> woken = futex.wake_one();
    require(woken.has_value() && woken.value()->state == TaskState::Runnable, "futex wake must return one task");

    DeadlockDetector detector;
    detector.add_wait_edge(1, 2);
    detector.add_wait_edge(2, 3);
    require(!detector.has_cycle(), "chain wait graph must not be a deadlock");
    detector.add_wait_edge(3, 1);
    require(detector.has_cycle(), "cycle in wait-for graph must be reported");
    detector.clear_task(2);
    require(!detector.has_cycle(), "removing a task from the cycle must break the deadlock");
    require(detector.edge_count() == 1, "clear_task must remove incoming and outgoing edges");

    BuddyAllocator allocator(OS_BUDDY_MAX_ORDER);
    const std::size_t first = allocator.allocate_pages(1);
    const std::size_t second = allocator.allocate_pages(1);
    require(first == 0 && second == 1, "buddy allocator should split the first pages deterministically");
    require(allocator.allocated_block_count() == 2, "two blocks should be allocated");
    allocator.free_pages(first);
    allocator.free_pages(second);
    require(allocator.allocated_block_count() == 0, "all buddy allocations should be released");
    require(allocator.free_blocks_at_order(OS_BUDDY_MAX_ORDER) == 1, "buddy free should coalesce to one block");
}

void test_lru_extent_block_scheduler_and_journal() {
    LruPageCache cache(2);
    cache.access(1, true);
    cache.access(2, false);
    cache.access(1, false);
    cache.access(3, false);
    require(cache.contains(1) && cache.contains(3), "LRU cache must retain recently touched pages");
    require(!cache.contains(2), "LRU cache must evict the cold page");
    require(cache.writeback_count() == 0, "clean eviction should not write back");
    cache.access(4, true);
    cache.access(5, false);
    require(cache.writeback_count() == 1, "dirty eviction must trigger writeback");
    cache.flush_all();
    require(cache.writeback_count() == 2, "flush_all should write remaining dirty entries and skip clean ones");

    ExtentMap extents;
    extents.add_extent(0, 100, 8);
    extents.add_extent(8, 200, 4);
    const std::optional<std::uint64_t> physical = extents.lookup(10);
    require(physical.has_value() && physical.value() == 202, "extent lookup must translate inside the extent");
    require(!extents.lookup(20).has_value(), "extent lookup outside all extents should miss");
    require_throws([&extents]() { extents.add_extent(4, 300, 2); }, "overlapping extents must be rejected");

    DeadlineBlockScheduler scheduler;
    const int far_read = scheduler.submit(90, false, 0);
    const int near_write = scheduler.submit(10, true, 0);
    const int expired_read = scheduler.submit(70, false, 0);
    static_cast<void>(far_read);
    static_cast<void>(near_write);
    std::optional<BlockRequest> first = scheduler.dispatch(OS_READ_DEADLINE_TICKS);
    require(first.has_value() && first->id == expired_read, "expired read should dispatch before seek optimization");
    std::optional<BlockRequest> second = scheduler.dispatch(OS_READ_DEADLINE_TICKS + 1);
    require(second.has_value() && second->sector == 90, "remaining closest request to current head should dispatch next");
    require(scheduler.pending_count() == 1, "one block request should remain queued");

    JournaledBlockDevice device;
    device.begin_transaction();
    device.write_block(7, "metadata-v2");
    device.commit(CrashPoint::AfterJournalAppend);
    require(device.read_block(7).empty(), "home block should not change before recovery after a crash");
    require(device.journal_record_count() == 1, "journal must retain committed intent after crash");
    device.recover();
    require(device.read_block(7) == "metadata-v2", "recovery must replay committed journal records");
    require(device.journal_record_count() == 0, "journal should be empty after replay");
}

```

```

void test_tcp_retransmission_state_machine() {
    TcpConnection connection;
    connection.active_open(100);
    require(connection.state() == TcpState::SynSent, "active open should send SYN and enter SYN-SENT");
    require(!connection.tick(102), "RTO should not fire before deadline");
    require(connection.tick(103), "RTO should fire at the retransmission deadline");
    require(connection.retransmission_count() == 1, "SYN retransmission should be counted");
    connection.receive_syn_ack();
    require(connection.state() == TcpState::Established, "SYN-ACK should establish the connection");
    connection.send_data(128, 200);
    require(connection.unacked_bytes() == 128, "send_data must track unacknowledged bytes");
    require(connection.tick(203), "data RTO should trigger retransmission");
    connection.acknowledge_all();
    require(!connection.tick(206), "ACK should cancel retransmission timer");
    connection.close(300);
    require(connection.state() == TcpState::FinWait1, "close should enter FIN-WAIT-1");
    connection.receive_fin_ack();
    require(connection.state() == TcpState::TimeWait, "FIN ACK should enter TIME-WAIT");
}

void run_all_tests() {
    test_cfs_scheduler_and_wait_queue();
    test_cow_page_table_and_tlb();
    test_iommu_and_nvme_completion_queue();
    test_fd_table_page_cache_and_fsync();
    test_futex_deadlock_and_buddy_allocator();
    test_lru_extent_block_scheduler_and_journal();
    test_tcp_retransmission_state_machine();
}

} // namespace osbook

int main() {
    try {
        osbook::run_all_tests();
        std::cout << "os_reference_implementation: all tests passed\n";
        return 0;
    } catch (const std::exception& error) {
        std::cerr << "test failure: " << error.what() << '\n';
        return 1;
    }
}

```


能力清单

- 能从晶体管开关、逻辑门、寄存器和时钟解释状态机为什么能成为 CPU 的基础。
- 能手跑 X64S 教学子集的 `mov/add/jmp/syscall` 程序，说明 `RIP`、`RSP`、寄存器、`RFLAGS`、`CPL` 如何变化。
- 能从轮询设备推导出中断，从死循环推导出定时器抢占，从同权代码推导出特权级和系统调用。
- 能画出一条 x86-64 `load/store` 指令经过 `RIP`、寄存器、`ALU`、`MMU`、`TLB`、`cache` 和异常入口的路径。
- 能区分 `ISA`、微架构和 `ABI`，并说明 OS 哪些代码依赖 `ISA`，哪些性能现象来自微架构。
- 能解释总线、`MMIO`、`DMA`、`IOMMU`、中断控制器和设备描述符环怎样共同完成一次 `I/O`。
- 能说明特权级限制了哪些指令和状态，为什么系统调用必须是受控陷入而不是普通函数调用。
- 能把一次系统调用拆成用户态入口、内核对象、权限检查、状态更新和返回值。
- 能解释线程为何没有推进：等 `CPU`、等锁、等页面、等 `I/O`、等信号还是等子进程。
- 能解释从复位向量到 `kernel_main` 的启动阶段和每个阶段禁止事项。
- 能说明中断上半部、下半部、定时器 `tick`、抢占点和上下文保存。
- 能判断自旋锁、`mutex`、`semaphore`、`RCU` 的上下文限制，并用锁顺序图解释死锁。
- 能解释 `exec` 的 `ELF` 验证、段映射、用户栈构造、动态解释器和提交点。
- 能手算一次虚拟地址翻译，并区分合法缺页、权限错误和非法地址。
- 能把一次 x86-64 `#PF` 拆成 `CR2`、`error code`、`VMA` 查找、`PTE` 判定、`COW/page cache` 修复、`TLB` 失效和返回重试。
- 能画出 `VMA`、`PTE`、`page cache`、`inode` 的关系，并区分 `MAP_SHARED` 与 `MAP_PRIVATE`。
- 能比较 `buddy`、`slab`、`kmalloc size class` 的用途、复杂度和碎片成本。
- 能画出驱动描述符环、`DMA` 映射、`completion` 和 `buffer` 生命周期。
- 能说明 `bio`、`request`、`request queue`、`flush`、`FUA` 和 `barrier` 如何共同影响 `I/O` 顺序。
- 能画出 `fd`、`open file description`、`inode`、`page cache` 和设备请求之间的对象关系。
- 能解释 `inode`、目录项、空闲块 `bitmap`、`VFS` 和 `page cache` 如何协作。
- 能解释 `socket` 作为 `fd` 的生命周期，说明 `bind`、`listen`、`accept`、`connect`、`send`、`recv` 的等待语义。

- 能分析 `capability`、`namespace`、`cgroup`、`seccomp` 如何共同形成最小权限边界。
- 能区分 `errno`、信号、`WARN`、`oops`、`panic`，并为系统调用写出失败回滚路径。
- 能为可靠写出设计 `prepared`、`syncd`、`renamed`、`manifest committed` 的提交状态机。
- 能写出包含现象、假设、证据、反证和未验证边界的 OS 调试报告。

Linux 源码阅读索引

本册不是 Linux 内核源码注释，但每个重要机制都应能落到真实源码。下面的路径以主线 Linux 内核常见目录为准；不同版本函数名会变化，阅读时以当前源码树为准。目标不是背路径，而是知道“这个概念在真实系统里由哪些结构和函数承载”。

主题	关键路径	重点对象或函数
x86 启动	arch/x86/boot/, arch/x86/kernel/head_64.S	实模式入口、长模式入口、早期页表
内核初始化	init/main.c, arch/x86/kernel/setup.c	start_kernel、setup_arch、initcall
系统调用入口	arch/x86/entry/entry_64.S, arch/x86/entry/syscalls/syscall_64.tbl	entry_SYSCALL_64、do_syscall_64
x86 page fault	arch/x86/mm/fault.c, arch/x86/mm/tlb.c	CR2、error code、用户/内核 fault、TLB flush 和 shutdown
调度器	kernel/sched/core.c, kernel/sched/fair.c, kernel/sched/rt.c	__schedule、enqueue_task_fair、pick_next_task
进程创建退出	kernel/fork.c, kernel/exit.c	copy_process、do_exit、wait
futex	kernel/futex/	futex_wait、futex_wake、PI futex
锁与 lockdep	kernel/locking/, kernel/locking/lockdep.c	qspinlock、mutex slow path、lock class graph
虚拟内存	mm/memory.c, mm/mmap.c, mm/rmap.c	handle_mm_fault、do_mmap、COW、VMA/PTE 生命周期
页面回收	mm/vmscan.c, mm/swap.c, mm/oom_kill.c	shrink_lruvec、kswapd、OOM killer
内核分配器	mm/page_alloc.c, mm/slub.c, mm/vmalloc.c	buddy、SLUB、kmem_cache_alloc
ELF 装载	fs/binfmt_elf.c, fs/exec.c	load_elf_binary、create_elf_tables
VFS 与 open	fs/open.c, fs/namei.c, fs/read_write.c	path_openat、vfs_read、vfs_write
page cache	mm/filemap.c, mm/readahead.c, fs/fs-writeback.c	filemap_fault、readahead、writeback
ext4 与 jbd2	fs/ext4/, fs/jbd2/	extent、mballoc、journal commit、recovery
块层	block/blk-mq.c, block/bio.c, block/mq-deadline.c	blk_mq_submit_bio、bio、request、tag
设备模型	drivers/base/, include/linux/device.h	device、driver、bus_type、kobject
PCI/NVMe/virtio	drivers/pci/, drivers/nvme/host/, drivers/virtio/	probe/remove、queue、interrupt、DMA

网络栈	net/core/, net/ipv4/, include/linux/skbuff.h	sk_buff、tcp_v4_rcv、tcp_ sendmsg
epoll	fs/eventpoll.c	ep_poll_callback、ready list、wait queue
namespace/cgroup	kernel/nsproxy.c, kernel/ cgroup/	namespace clone、cgroup v2 controller
LSM/seccomp	security/, kernel/seccomp. c	security_* hooks、BPF filter、seccomp actions
tracing/perf/eBPF	kernel/trace/, kernel/ events/, kernel/bpf/	tracepoint、perf event、 BPF program/map
panic/kdump	kernel/panic.c, kernel/ kexec_core.c	panic、oops、kdump、 crashkernel

源码阅读最低标准：先找核心结构体，再找对象创建路径、状态转换路径、失败回滚路径和观测点。只搜索一个函数名，通常看不到系统设计。